

Statistical Methods for Life Sciences

Eva Freyhult, Olga Dethlefsen, Payam Emami

Table of contents

Preface	1
I Introduction	3
1 Why do we care about probability?	7
1.1 Do studies give different results?	7
2 Describing an uncertain observation	9
2.1 Population, sample and random variable	9
2.2 Probability: axioms and common rules	10
2.3 Conditional probability	10
2.4 Independence between observations	11
Exercises	12
3 Describing outcomes with probability models	15
3.1 Probability models and probability distributions	15
3.2 A normal model for individual heights	15
3.3 Continuous outcomes: probability is area	16
3.4 Standardisation: how far from the mean?	19
3.5 Percentiles: finding a cutoff	20
3.6 Discrete outcomes: counting students	21
3.7 From one student to a count	21
3.8 Probability mass function: the probability of each count	23
3.8.1 Cumulative distribution function: adding probabilities	25
3.9 Continuous vs. discrete	26
3.10 Other common distributions in life sciences	26
3.11 Simulation: generating outcomes from a probability model	27
3.12 Approximating probabilities by simulation	27
3.13 Key message	28
4 Repeating the sampling	29
4.1 From individual observations to study results	29
4.2 Imagine repeating the study	29
4.3 Simulating repeated studies	30
4.4 Why do the sample means look normal?	31
4.5 What if we have only one study?	31

4.5.1	Bootstrap: resampling the observed data	32
5	Describing precision	35
5.1	How variable is a sample mean?	35
5.2	Standard error of the sample mean	35
5.3	How does sample size affect precision?	36
5.4	Estimating the standard error from data	36
5.4.1	Estimating SE using the bootstrap	37
5.5	From the normal to the t-distribution	38
5.6	More measurements or more individuals?	39
5.7	Using SE to assess an observed result	40
5.8	An individual value is not a sample mean	41
5.9	Return to the two research groups	42
6	Exercises	45
	Exercises	45
	Appendix: discrete random variables	55
	Expected value and variance	58
	Linear transformations and combinations	59
	Simulate distributions	59
	Parametric discrete distributions	62
	Uniform	62
	Bernoulli	62
	Binomial	62
	Poisson	63
	Negative binomial	64
	Geometric	64
	Hypergeometric distribution	64
	Summary of discrete distributions	65
	Appendix: cotinuous random variables	67
	Parametric continuous distributions	71
	Normal distribution	71
	Sum of two normal random variables	74
	Central limit theorem, CLT	75
	Chi-square distribution	77
	F-distribution	78
	t-distribution	80
	Distributions in R	81
II	Statistical Inference	83
	Learning outcomes	85
7	Introduction to hypothesis tests	87
7.1	The null and alternative hypothesis	87
7.2	To perform a hypothesis test	88
7.3	The p-value	88
7.4	Significance level and error types	88

8 Hypothesis testing using resampling	91
8.1 Resampling from a known population under H_0	91
8.2 Bootstrap resampling	94
8.2.1 Permutation	97
9 Parametric tests	101
9.1 Mean comparison	101
9.1.1 One sample	101
9.1.2 Two samples	103
9.2 Proportions	105
9.2.1 One sample	105
9.2.2 Two samples, proportions	107
9.2.3 Paired samples	107
9.3 Variance	108
10 Point and interval estimates	111
10.1 Point estimate	111
10.2 Bias and precision	111
10.3 Interval estimates	112
10.3.1 Bootstrap interval	112
10.3.2 Confidence interval	114
10.3.3 Confidence interval of proportions	114
10.3.4 Confidence interval of the mean	117
11 Rank-based tests	119
12 Rank-based tests	121
12.1 Two independent samples	121
12.2 Paired samples	122
13 Multiple testing	123
13.1 Error types	123
13.2 Multiple testing	123
13.3 Bonferroni correction	125
13.4 Benjamini-Hochbergs FDR procedure	125
13.5 'Adjusted' p-values	125
14 Analysis of variance (ANOVA)	127
14.1 Sum of squares	127
14.2 Mean squares and F statistic	129
14.3 Assumptions	130
14.4 Post hoc test	130
14.5 Non-parametric alternative	131
15 Exercises: Statistical inference	133
15.1 Hypothesis tests, resampling	133
15.2 Hypothesis tests, parametric	135
15.3 Point and interval estimates	141
15.4 Rank-based tests	145
15.5 Multiple testing	146
15.6 Analysis of variance (ANOVA)	147

III Linear regression	155
Learning outcomes	157
16 Introduction to linear models	159
16.1 Why linear models?	159
16.2 Statistical vs. deterministic relationship	160
16.3 What linear models are and are not	160
16.4 Terminology	163
16.5 Simple linear regression	163
16.6 Least squares	165
16.7 The linear model describes an expected value	167
16.8 Intercept and Slope	168
16.9 From simple regression to more general linear models	168
16.10 Hypothesis testing	170
16.11 Overall model test: the F-test	171
16.12 Vector-matrix notations	173
16.13 Confidence intervals and prediction intervals	176
16.14 Assessing model fit	178
R^2 : summary of the fitted model	178
R^2 and correlation coefficient	181
$R^2(adj)$	181
16.15 The assumptions of a linear model	181
16.15.1 Checking assumptions	182
16.16 Influential observations	185
17 Generalized linear models	187
17.1 Why do we need GLM?	187
17.2 Logistic regression	189
17.2.1 Hypothesis testing	193
17.2.2 Deviance	194
17.2.3 Odds ratios	195
17.2.4 Other covariates	196
17.3 Poisson regression	197
Rate ratios	200
18 Common cases	201
18.1 Example: simple linear regression	202
18.2 Example: multiple regression	204
18.3 Example: categorical variable	207
18.4 Example: categorical & numerical variables	209
18.5 Example: interactions	212
18.6 Example: logistic regression with categorical variable	214
Exercises (introduction to linear models)	217
Exercises (model diagnostics)	223
Answers to selected exercises	223
Exercises (regression coefficients)	233

IV Mixed models	241
Environment	243
Packages	243
19 Simple linear regression	245
20 Mixed models	257
20.1 Why “Mixed”?	258
20.1.1 Fixed Effects	258
20.1.2 Random effect	259
20.2 Ingredients	260
20.2.1 Grouping Structure of the Data	260
20.2.2 Design for fixed and random effects	261
20.3 Examples (Mixed models in R)	262
20.3.1 Predicting the random intercept	267
20.3.2 Random slope	269
20.3.3 Nested and crossed designs	272
20.3.4 Understand what you are doing!	284
20.4 Model selection and comparisons using LRT	299
20.5 Model diagnostic	304
20.6 P-value calculation and marginal means	306
20.6.1 ANOVA type I, II, and III	310
20.7 Power calculation	312
21 Mixed models (mathematical details)	315
22 Ordinary Least Squares regression	317
22.1 Mathematical Formulation	317
22.2 Correlated residuals	318
23 Generalized Least Squares (GLS)	321
23.1 Example	321
23.2 Structure of V	329
23.2.1 Design matrix	334
23.3 Structure of V (cont.)	336
24 Estimation of parameters	339
24.1 Bias of MLE	342
24.2 Restricted Maximum Likelihood (REML)	344
24.3 Other methods of fitting	347
25 Best Linear Unbiased Estimators (BLUE)	349
26 Random Effects	353
27 Final mixed model form	359
28 p-value for mixed models (Satterthwaite approximation)	361
28.1 How rare is this value?!	361
28.2 Standard normal distribution	363
28.3 Putting it all together (T)	366

28.4 t-statistics for OLS	369
28.4.1 degrees of freedom of GLS	372
29 Summary	377
30 Exercises: Mixed Effects Models	379
30.1 Random intercepts and random slopes	379
30.2 Nested design	394
30.3 Lab, technician and sample	399
30.4 Model comparisons	402
30.5 Computing p-values for mixed models	404
30.6 Logistic mixed model	406
V Survival Analysis	411
31 Introduction	413
31.1 Why survival analysis?	413
31.2 Survival and censoring times	414
31.3 Independent censoring	416
31.4 Left and interval censoring	416
31.5 Kaplan-Meier estimator	418
31.5.1 Tooth filling example	418
31.5.2 Interpretations	422
31.6 Comparisons of hazards	425
31.7 Mantel-Cox (log-rank test)	426
32 Regression with survival response	429
32.1 Cox proportional hazards model	429
32.2 Example with brain cancer data	430
32.2.1 Coefficient interpretation	431
32.2.2 Global statistical significance	432
32.2.3 Concordance	433
32.2.4 Predictions	433
32.3 Time-dependent covariates	435
32.4 Multiple events: competing risks	437
33 Exercises: Survival analysis	439
33.1 Kaplan-Meier curves and log-rank test	439
33.2 Cox regression	443
33.3 Adjusted survival curves from a Cox model	447
33.4 Time-varying predictors	448
33.5 Competing risks	450
33.5.1 Comparing CIFs	450
33.5.2 Competing Risks Regression	453
VI Principal Component Analysis	457
Learning outcomes	459
34 Background	461

35 Two-dimensional data	467
36 Multiple dimensional data	471
37 Notes for PCA	473
37.1 Missing	473
37.2 Scaling	473
38 Output from PCA	475
38.1 Principal components	475
38.2 Biplot (Scores + Loadings)	476
38.3 Scree plot	477
Exercises: Principal component analysis	479
Exercise 1: PCA of all samples	480
Exercise 2: Looking beyond PC1 and PC2	484
Exercise 3: Biology, experiment, or both?	488
Exercise 4: Which probes contribute to a component?	492
Exercise 5: A simple biplot	496
 VII Clustering	 501
Aims	503
Learning outcomes	503
 39 Clustering: art of finding groups	 505
39.1 Introduction	505
39.2 Common usage	506
39.3 Partition methods vs. hierarchical	506
39.4 Distance and similarity measures	509
39.4.1 Distances for interval variables	511
39.4.2 Distances for binary data	513
39.5 Partition methods: k-means & PAM	514
39.5.1 k-means	514
39.5.2 PAM, partition around medoids	515
39.6 HCL, hierarchical clustering	517
39.6.1 Linkage & Agglomerative clustering	520
39.6.2 Getting clusters	524
39.7 How many clusters?	525
39.7.1 Elbow method	525
39.7.2 Silhouette method	527
39.7.3 Gap	528
39.7.4 pvclust	529
39.8 Two-way clustering	530
39.9 Additional comments & resources	532
 References	 533
 Exercises	 535
Answers to exercises	536

VIII From Linear Models to Machine Learning	543
40 From inference to prediction	545
40.1 What is supervised learning?	545
40.2 Data splitting	546
train, validation & test sets	546
cross validation	547
Leave-one-out cross-validation	548
Nested cross-validation	548
40.3 Evaluating regression	548
Model fit	549
Prediction error	550
Which metric should we use?	551
40.4 Evaluating classification	551
40.5 Evaluating classification models	551
Accuracy and misclassification rate	552
Confusion matrix	552
Sensitivity	553
Specificity	553
Precision	553
Classification thresholds	554
ROC curve and ROC AUC	554
Which metric should we use?	556
41 Regularization	557
41.1 Regularized regression	557
41.2 Bias-variance trade-off	559
41.3 Ridge, Lasso and Elastic Nets	560
41.4 Logistic Lasso	564
Exercises (Lasso)	565
Load data & reformat data	565
split data and scale	567
fit Lasso	568
train Lasso model	569
create final model	570
predict BMI	570
References	573

Preface

This repository contains teaching and learning materials prepared and used during the “Statistical Methods for Life Sciences” course, organized by NBIS, National Bioinformatics Infrastructure Sweden. The course is open for PhD students, postdoctoral researcher and other employees within Swedish universities. The materials are geared towards life scientists wanting to be able to understand and use basic statistical and machine learning methods. More about the 2026 course: <https://uppsala.instructure.com/courses/119661>.

Part I

Introduction

Statistical Methods for Life Sciences

This session introduces probability as a framework for understanding variation in experimental data. We cover random variables, events and basic probability rules, continuous and discrete probability models, and commonly used distributions. We then use simulation and resampling to explore probabilities and sampling distributions, before introducing the standard error, precision, and the connection between the normal and t-distributions as a foundation for statistical inference.

Chapter 1

Why do we care about probability?

Statistical Methods for Life Sciences

1.1 Do studies give different results?

Two research groups independently sample individuals from a group of PhD students taking “Statistical Methods for Life Science” course, and measure their heights.

They ended up with sample means of 172 and 175 cm respectively.

Could this difference arise simply because they sampled different individuals?

Let’s discuss:

- What information would help you answer?
- Would a difference of 3 cm be surprising if individual heights varied very little? What if they varied a lot?
- Would your answer change if each group studied 5 individuals rather than 100?
- If we repeated this study many times, would we always get the same sample mean?
- What exactly is the population we want to learn about?
- How were the students selected? Could selection systematically favour taller or shorter people?
- Were heights measured in the same way in both groups?
- How could we describe which sample means are common and which are unusual?

! Key message

Different samples from the same population can give different results. Probability models help us describe how much sampling variation to expect and which results would be unusual under our assumptions.

When discussing the question above, some of the following terms may come to mind:

- Population, sample, sample size
- Biological variability, variance, standard deviation (SD)
- Sampling variability, statistic
- Random variable, event, sampling distribution
- Standard error (SE) and precision
- Selection bias, measurement error
- Probability model and assumptions

Let's explore how these concepts connect and how probability helps us understand variation in our results.

Chapter 2

Describing an uncertain observation

Statistical Methods for Life Sciences

2.1 Population, sample and random variable

Suppose we want to study the heights of PhD students at a particular university.

- The **population** is all PhD students at that university.
- A **sample** is the group we select and measure, e.g. PhD students enrolled in a specific course.
- The **sample size**, n , is the number selected.

A **random variable** describes a numerical outcome that we do not know before making a random selection or carrying out an experiment.

- For example, let X be the height of a randomly selected student.
- Depending on whom we select, we might observe 168, 175 or 182 cm.
- Before selection, we use X to represent the uncertain height.
- After measurement, we have an **observed value**, such as $x = 175$ cm.

The variation comes from who happens to be selected — not from their height changing when we select them.

The **sample space** is the set of all possible outcomes or values the random variable can take. In our example, the random variable can take any value between 0 cm and infinity: $S \in [0, \infty)$

An **event** is a condition we are interested in, e.g. a selected student being taller than 180 cm: $X > 180$.

2.2 Probability: axioms and common rules

Axioms of probability

Probabilities lie between 0 and 1

$$0 \leq P(E) \leq 1 \text{ for any event } E \subseteq S$$

where the S is the **sample space**, a set of all possible outcomes.

Something must happen: the probability that an outcome in the sample space occurs is 1.

$$P(S) = 1$$

For mutually exclusive events, the probability that either occurs equals the sum of their individual probabilities.

$$P(E \cup F) = P(E) + P(F) \text{ if } E \text{ and } F \text{ are disjoint events}$$

Common rules

The **complement** of being taller than 180 cm (E) is being 180 cm or shorter (E'):

$$P(E') = 1 - P(E)$$

$$P(X \leq 180) = 1 - P(X > 180)$$

The probability of **impossible event** is zero

$$P(\emptyset) = 0$$

An event cannot be more likely than an event that contains it (**probability of a subset**). If $E \subseteq F$, then

$$P(F) \geq P(E)$$

The probability that at least one of two events occurs is the sum of their probabilities, minus the probability that both occur (**addition rule**). Let $E, F \subseteq S$ be any two events, then

$$P(E \cup F) = P(E) + P(F) - P(E \cap F)$$

2.3 Conditional probability

Does knowing that a student uses an Android phone changes the probability they are taller than 180 cm?

Let's imagine a hypothetical population of 100 students, in which:

- 8 are taller than 180 cm.
- 40 use Android phones; 5 of these are taller than 180 cm.

$$P(X > 180) = \frac{8}{100} = 0.08$$

$$P(X > 180 \mid \text{Android}) = \frac{5}{40} = 0.125$$

The vertical bar means “**given that**”.

Conditional probability restricts attention to a specified group, e.g. Android users. Here, knowing that the student uses Android phone increases the probability of a student being above 180 cm tall from 8% to 12.5%. This is, however, not an evidence that phone choice causes differences in height (i.e., it is an association; no causation is implied).

In general:

$$P(A \mid B) = \frac{P(A \cap B)}{P(B)}, \quad P(B) > 0.$$

where, $A \cap B$ means that both conditions are satisfied.

2.4 Independence between observations

Observations are **independent** if knowing the value of one does not change the probability distribution of the other.

For independent events A and B :

$$P(B \mid A) = P(B),$$

or equivalently,

$$P(A \cap B) = P(A)P(B).$$

Suppose two students are sampled independently. If 20% of the population is taller than 180 cm, knowing that the first student is taller than 180 cm does not change the probability that the second student is taller than 180 cm.

$$P(B \mid A) = P(B) = 0.20.$$

Here, A and B refer to the first and second student, respectively, being taller than 180 cm.

The probability that both exceed 180 cm is therefore:

$$P(A \cap B) = P(A)P(B) = 0.20 \times 0.20 = 0.04$$

! Key message

Different samples can produce different results. Probability gives us a language for describing these uncertain outcomes and calculating how likely they are.

To use it correctly, we must specify the population, how we sample, what we measure, and the events of interest.

Exercises

Exercise 2.1 (What is random?). We randomly select 10 adults from a defined population and record their BMI. The first person's BMI is 24.5.

Let's model as BMI as a continuous, positive measurement.

Identify:

- The sample size.
- The sample space for one BMI measurement.
- The random variable.
- Its observed value.
- An example event involving BMI.

💡 Solution

- **Sample size:** $n = 10$.
- **Sample space:** $S = [0, \infty)$ under our positive, continuous model.
- **Random variable:** X = the BMI of the first randomly selected adult.
- **Observed value:** $x = 24.5$.
- **Example event:** the first adult's BMI exceeds 25, $E = \{X > 25\}$.

Exercise 2.2 (What can we calculate — and what is missing?). Suppose 50% of adults in our hypothetical population have a BMI above 25.

For an adult randomly selected from this population:

- What is the probability that their BMI is 25 or less?
- Can we calculate the probability that their BMI exceeds 30 from this information alone?

💡 Solution

By the **complement rule**:

$$P(X \leq 25) = 1 - P(X > 25) = 1 - 0.50 = 0.50.$$

We cannot calculate $P(X > 30)$ exactly from this information. Because $\{X > 30\} \subseteq \{X > 25\}$, we know only that:

$$0 \leq P(X > 30) \leq 0.50.$$

We need more information about how BMI values are distributed above 25.
This motivates our next topic: **probability distributions and models**.

Chapter 3

Describing outcomes with probability models

Statistical Methods for Life Sciences

In the previous section's BMI exercise, knowing that 50% of adults have a BMI above 25 was not enough to calculate the probability that a randomly selected adult has a BMI above 30.

To calculate this probability, we need more information about how BMI values are distributed above 25. A probability model is one way to describe this distribution.

3.1 Probability models and probability distributions

A **probability model** is a mathematical description of a random process. It specifies possible outcomes, their probabilities and the assumptions about how observations arise.

A **random variable** X assigns a numerical value to each outcome. Its **probability distribution** describes how probability is distributed across those values.

A continuous distribution is described by a **probability density function**, $f(x)$.

While discrete distribution is described by a **probability mass function**, $p(x)$.

3.2 A normal model for individual heights

For instance, our model might assume that we independently select students from a population where heights are normally distributed with mean 170 cm

and SD 7 cm. Let's see how to calculate the probability that a student is taller than 180cm.

Given our model, the probability distribution of the height (X) of one randomly selected student is:

$$X \sim N(\mu, \sigma^2), \quad \mu = 170, \quad \sigma = 7.$$

where **mean** μ and **variance** σ^2 are distribution parameters.

- **Expected value:** $E[X] = \mu$, the long-run average.
- **Variance:** $\text{Var}(X) = E[(X - \mu)^2]$, the expected squared distance from the mean.
- **Standard deviation (SD):** $\sigma = \sqrt{\text{Var}(X)}$ the square root of variance, expressed in the original units

So here:

$$E[X] = 170 \text{ cm},$$

$$\text{Var}(X) = E[(X - \mu)^2] = \sigma^2 = 7^2 = 49 \text{ cm}^2,$$

$$\text{SD}(X) = \sqrt{\text{Var}(X)} = 7 \text{ cm}.$$

And the **probability density function**, $f(x)$ describing distribution has a form:

$$f(x) = \frac{1}{\sigma\sqrt{2\pi}} \exp\left[-\frac{(x - \mu)^2}{2\sigma^2}\right].$$

For a continuous distribution, the density function $f(x)$ gives the **height of the density curve** at a particular value x . A higher density means that values around x are relatively more common than values around points with lower density.

Note that normality is a modelling assumption, not a universal property of biological measurements.

3.3 Continuous outcomes: probability is area

The density itself is not a probability. For continuous variables, probabilities are represented by **areas under the density curve**.

The probability that X falls between a and b is the area under the density curve between these values:

$$P(a < X \leq b) = \int_a^b f(x) dx.$$

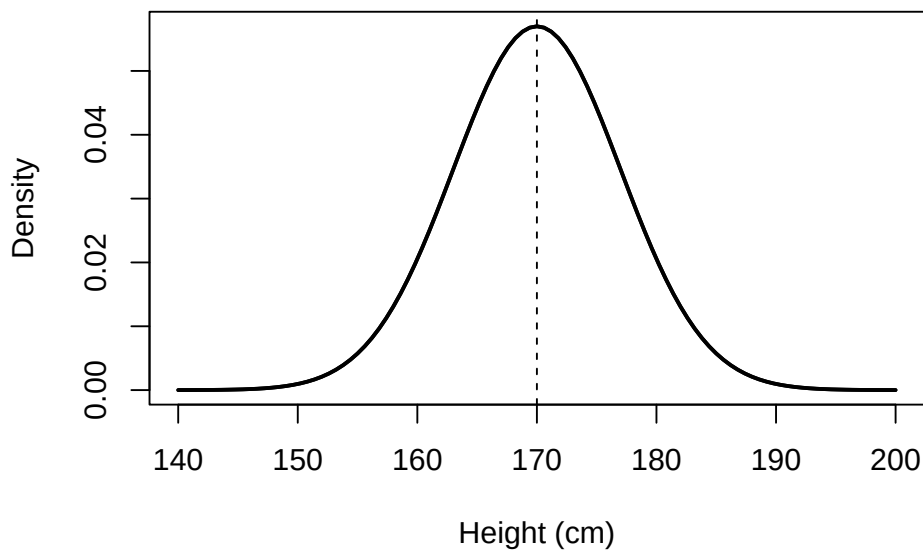


Figure 3.1: Probability density function of the height of a student.

The integral adds up the density across the interval.

The total area under the density curve is 1:

$$\int_{-\infty}^{\infty} f(x) dx = 1.$$

For example, for our height model, the probability density function (PDF) is:

$$f(x) = \frac{1}{7\sqrt{2\pi}} \exp \left[-\frac{(x-170)^2}{98} \right].$$

The probability $P(X > 180)$ is the area under the normal density curve to the right of 180, shown below:

In theory, we calculate this area by integration:

$$P(X > 180) = \int_{180}^{\infty} f(x) dx.$$

The **cumulative distribution function (CDF)**, $F(x)$, gives the accumulated area to the left of a cutoff:

$$F(x) = P(X \leq x) = \int_{-\infty}^x f(t) dt.$$

```
x <- seq(145, 195, length.out = 500)
plot(
```

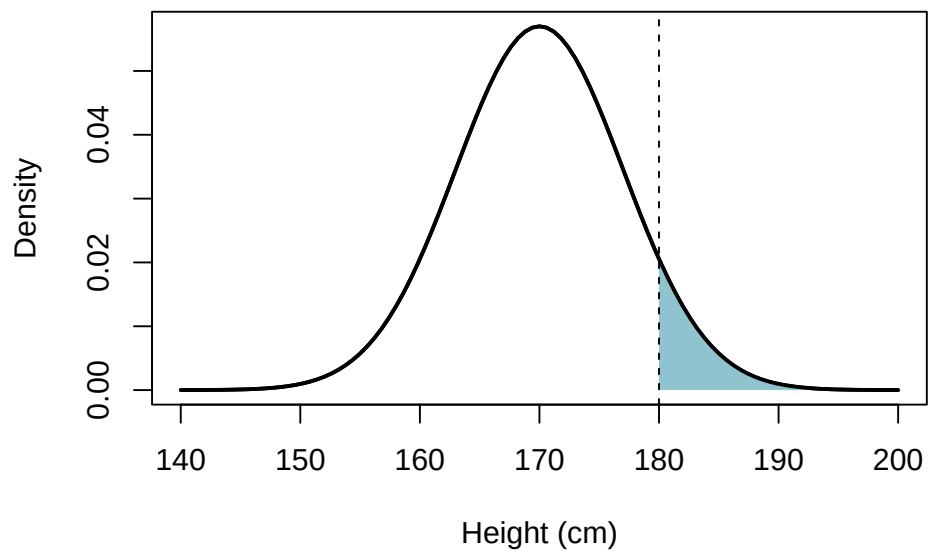


Figure 3.2: Probability density function of the height of a student. The shaded area represents the probability of height above 180 cm.

```
x,
pnorm(x, mean = 170, sd = 7),
type = "l",
lwd = 2,
xlab = "Height (cm)",
ylab = "Cumulative probability",
main = ""
)
```

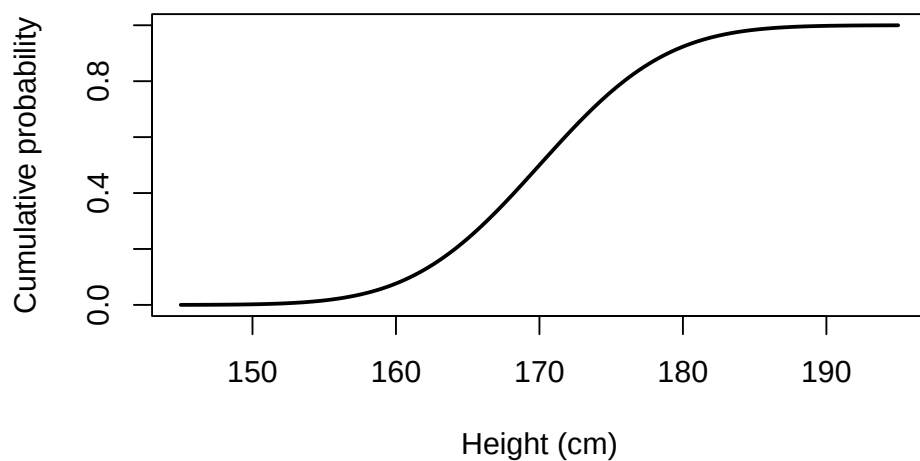


Figure 3.3: Cumulative distribution function for the height of one randomly selected student.

Since total area under the density curve is 1, the probability a student being above 180 cm tall is the complement of the probability at or below 180 cm:

$$P(X > 180) = 1 - F(180).$$

The normal CDF cannot be calculated using a simple closed-form expression. Historically, probabilities were obtained from tabulated values of the **standard normal distribution** - a normal distribution with mean 0 and SD 1. Today, we can calculate them numerically.

E.g., in R `pnorm()` evaluates the normal CDF directly, and for our model with mean 170 cm and SD 7 cm:

```
pnorm(180, mean = 170, sd = 7)
```

```
[1] 0.9234363
```

This gives $F(180) \approx 0.9234$. Therefore:

$$P(X > 180) = 1 - F(180) \approx 1 - 0.9234 = 0.0766.$$

So, under this model, about **7.7% of students** are taller than 180 cm.

In R, we can also obtain the upper-tail probability directly by setting `lower.tail = FALSE` and arrive at the same results:

```
pnorm(180, mean = 170, sd = 7, lower.tail = FALSE)
```

```
[1] 0.07656373
```

3.4 Standardisation: how far from the mean?

A **z-score** expresses distance from the mean in SD units:

$$Z = \frac{X - \mu}{\sigma}.$$

- Subtracting the mean centres the distribution at zero.
- Dividing by the SD rescales it to have a standard deviation of one.

For a normally distributed variable, standardisation gives a new variable Z :

$$Z \sim N(0, 1),$$

that follows the **standard normal distribution**.

Given our height example, we can locate where a measurement e.g. 180 cm lies relative to the mean:

$$z = \frac{180 - 170}{7} \approx 1.43.$$

This height is about **1.43 SD above the mean**.

Standardisation changes the scale, but not the corresponding probability. Being above 180 cm is equivalent to having a z-score above $10/7 \approx 1.43$.

Writing Φ for the standard normal cumulative distribution function:

$$\begin{aligned} P(X > 180) &= P\left(Z > \frac{180 - 170}{7}\right) \\ &= 1 - \Phi\left(\frac{10}{7}\right) \\ &\approx 0.0766. \end{aligned}$$

We get the same probability in R using the standard normal distribution, which is the default for `pnorm()`:

```
z <- (180 - 170) / 7
pnorm(z, lower.tail = FALSE)
```

```
[1] 0.07656373
```

3.5 Percentiles: finding a cutoff

So far, we have started with a height and calculated a probability. Now we reverse the question:

Below which height do 95% of students fall?

The **95th percentile**, $x_{0.95}$, satisfies:

$$P(X \leq x_{0.95}) = 0.95.$$

We first find this cutoff on the standard normal scale.

The symbol Φ denotes the **standard normal cumulative distribution function**:

$$\Phi(z) = P(Z \leq z).$$

It gives the probability below a specified z-score. Its inverse, Φ^{-1} , goes in the opposite direction: it gives the z-score for a specified cumulative probability.

For a probability of 0.95:

$$z_{0.95} = \Phi^{-1}(0.95) \approx 1.645.$$

This value comes from the standard normal distribution: approximately 95% of its area lies below 1.645. We obtain it using software or a standard normal table.

In R:

```
#|code-fold: show
qnorm(0.95)
```

```
[1] 1.644854
```

To convert this z-score back to centimeters, we multiply by the SD and add the mean:

$$\begin{aligned} x_{0.95} &= \mu + \sigma z_{0.95} \\ &\approx 170 + 7 \times 1.645 \\ &\approx 181.5 \text{ cm.} \end{aligned}$$

We can also obtain the height cutoff directly in R:

```
qnorm(0.95, mean = 170, sd = 7)
```

```
[1] 181.514
```

Under our model, 95% of students are at or below approximately 181.5 cm, and 5% are above it.

A percentile converts a probability into a cutoff. In R:

- `pnorm()` goes from a value to a cumulative probability;
- `qnorm()` goes from a cumulative probability to a value.

3.6 Discrete outcomes: counting students

Not all outcomes are continuous. In life sciences, we may instead be interested in counts, such as the number of bacterial colonies on a plate or the number of sequencing reads assigned to a gene. The same probability concepts apply, but discrete outcomes are described using probabilities at specific values rather than areas under a continuous curve.

Let's imagine we are interested in how many students in a sample are taller than 180 cm. We sample 20 PhD students from a course at UU and record how many of them are taller than 180 cm.

Let K denote this count. Its possible values are:

$$K \in \{0, 1, 2, \dots, 20\}.$$

Unlike height, this is a **discrete random variable**: it can take only specific, whole-number values, from 0 to 20.

3.7 From one student to a count

For one student, the outcome “taller than 180 cm or not” can be coded as 1 or 0, respectively.

This follows a **Bernoulli distribution**, which describes a single random outcome with two possible values:

- 1: success
- 0: failure

If the probability of success is p , then the probability of failure is $1 - p$:

$$P(Y = y) = \begin{cases} p, & y = 1, \\ 1 - p, & y = 0. \end{cases}$$

The expected value and variance are:

$$E[Y] = p, \quad \text{Var}(Y) = p(1 - p).$$

In our example, for one randomly selected student:

- $Y = 1$ if the student is taller than 180 cm,
- $Y = 0$ otherwise.

Since X denotes the student's height, under our height model:

$$p = P(X > 180) \approx 0.0766.$$

Thus:

$$Y \sim \text{Bernoulli}(0.0766).$$

So the Bernoulli distribution describes whether **one student** is taller than 180 cm or not.

A **binomial distribution** describes the number of successes in n independent Bernoulli trials, where each trial has the same probability of success, p .

If Y_i denotes the Bernoulli outcome for student i , then the total number of successes is:

$$K = \sum_{i=1}^n Y_i.$$

The binomial distribution is written as:

$$K \sim \text{Bin}(n, p).$$

The expected value and variance are:

$$E[K] = np,$$

$$\text{Var}(K) = np(1 - p).$$

In our example, we sample 20 students and define a success as being taller than 180 cm. Since

$$p = P(X > 180) \approx 0.0766,$$

the number of students taller than 180 cm follows:

$$K \sim \text{Bin}(20, 0.0766).$$

The expected count is therefore:

$$E[K] = 20 \times 0.0766 \approx 1.53.$$

This means that across many repeated samples of 20 students, the average number taller than 180 cm would be about **1.5 students**. Each individual observed count, however, must be a whole number.

3.8 Probability mass function: the probability of each count

For a discrete random variable, the **probability mass function (PMF)** gives the probability of each possible value:

$$p_K(k) = P(K = k).$$

Unlike a continuous density, where probability is represented by an area, the value of a PMF is itself a probability.

For a binomial random variable,

$$K \sim \text{Bin}(n, p),$$

the probability mass function is:

$$P(K = k) = \binom{n}{k} p^k (1 - p)^{n-k}, \quad k = 0, 1, \dots, n.$$

Here, $\binom{n}{k}$ counts the number of ways in which exactly k successes can occur among n trials.

In our example,

$$K \sim \text{Bin}(20, 0.0766),$$

where K is the number of students taller than 180 cm among 20 independently selected students.

The PMF therefore gives the probability of observing each possible count, from 0 to 20 students.

We can calculate and plot these probabilities in R using `dbinom()`:

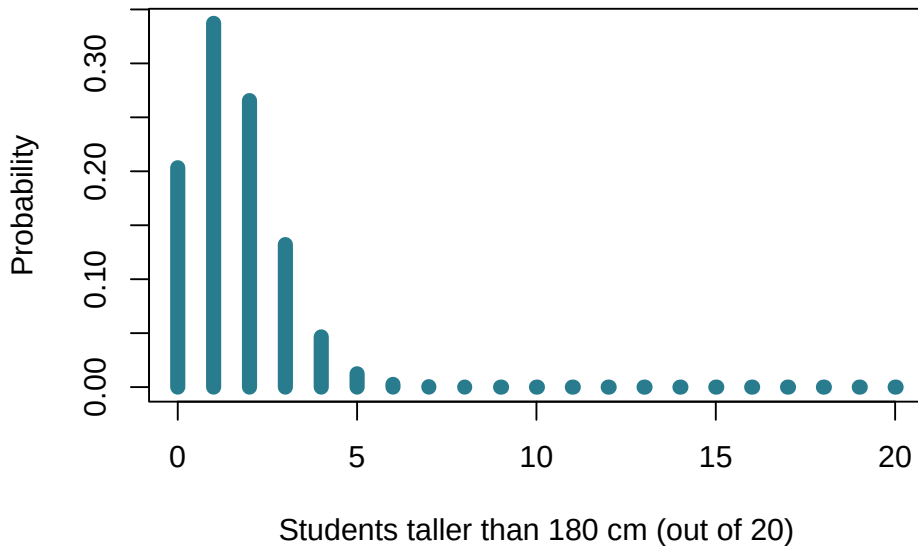


Figure 3.4: Probability of each possible number of students taller than 180 cm among 20 selected students.

The height of each bar is the probability of observing that particular count. For example, the bar at $k = 2$ represents:

$$P(K = 2).$$

The probabilities across all possible counts sum to 1:

$$\sum_{k=0}^{20} P(K = k) = 1.$$

As an example, let's calculate the probability that exactly five students out of 20 are taller than 180cm. To do that, we evaluate the PMF at $k = 5$:

$$P(K = 5) = \binom{20}{5} p^5 (1-p)^{15}.$$

Or in R:

```
dbinom(5, size = 20, prob = p_tall)
```

```
[1] 0.01234989
```

Note that for a continuous variable, a PDF gives a density and probabilities are obtained from areas. For a discrete variable, a PMF gives probabilities directly at individual values.

3.8.1 Cumulative distribution function: adding probabilities

The **cumulative distribution function** (CDF) gives the probability of a value at or below a cutoff (same meaning as for continuous distribution).

For our count variable (K):

$$F_K(k) = P(K \leq k) = \sum_{j=0}^k P(K = j).$$

For example, the probability of four or fewer students being taller than 180 cm is:

$$F_K(4) = P(K = 0) + P(K = 1) + \dots + P(K = 4).$$

For continuous variables, we integrate the density. For discrete variables, we add the probability masses.

In R, `pbinom()` evaluates the binomial CDF:

```
pbinom(4, size = 20, prob = p_tall)
```

```
[1] 0.9846016
```

What is the probability that five or more students are of 20 will be taller than 180cm?

As before, we can use the complement rule and this time write:

$$P(K \geq 5) = 1 - P(K \leq 4) = 1 - F_K(4).$$

In R:

```
1 - pbinom(
  4, size = 20, prob = p_tall,
  lower.tail = TRUE
)
```

```
[1] 0.01539839
```

```
#or
```

```
pbinom(
  4, size = 20, prob = p_tall,
  lower.tail = FALSE
)
```

[1] 0.01539839

3.9 Continuous vs. discrete

Concept	Continuous example: height	Discrete example: count
Random variable	X : one student's height	K : number taller than 180 cm among 20 students
Model	Normal	Binomial
Distribution function	PDF: $f(x)$ gives density	PMF: $p_K(k) = P(K = k)$ gives probability
Probability calculation	Integrate density over an interval	Sum probabilities of individual values
CDF	$F_X(x) = P(X \leq x)$	$F_K(k) = P(K \leq k)$
R: density or mass	<code>dnorm()</code> gives density	<code>dbinom()</code> gives probability
R: cumulative probability	<code>pnorm()</code>	<code>pbinom()</code>
Upper tail	$P(X > 180) = 1 - F_X(180)$	$P(K \geq 5) = 1 - F_K(4)$

The CDF has the same meaning in both cases: probability at or below a cutoff. The difference is how we accumulate probability: integration for a continuous density, summation for discrete masses.

3.10 Other common distributions in life sciences

Beyond normal and binomial distributions, several other models are useful:

- **Poisson:** counts within a fixed time, area or volume, such as colony counts under suitable conditions. The mean and variance are equal.
- **Negative binomial:** counts with more variability than Poisson allows. Commonly used for RNA-seq counts across biological replicates.
- **Log-normal:** positive measurements whose logarithms are normally distributed, such as some biological concentrations.
- **Exponential:** waiting times between events occurring independently at a constant rate.
- **Hypergeometric:** counts when sampling without replacement from a finite set, such as overlap between a selected gene list and a pathway.

The **t**, **chi-squared** and **F** distributions also appear in statistical inference, often as distributions of test statistics.

3.11 Simulation: generating outcomes from a probability model

Once we have specified a probability model, we can use it to **simulate possible outcomes**.

For example, under our height model:

$$X \sim N(170, 7^2),$$

we can simulate the height of one randomly selected student:

```
rnorm(1, mean = 170, sd = 7)
```

```
[1] 173.6441
```

Each simulation gives one possible outcome under the model.

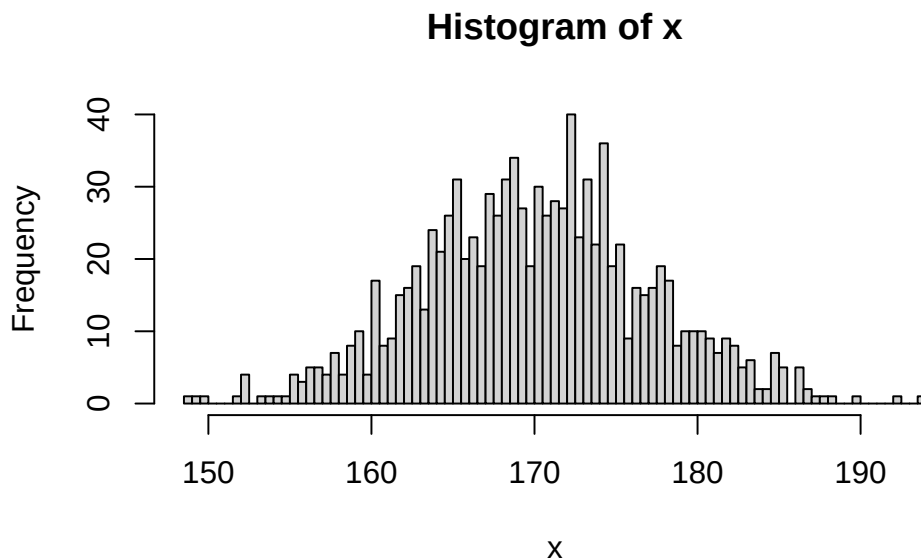
We can also simulate many students:

```
x <- rnorm(1000, mean = 170, sd = 7)
head(x)
```

```
[1] 162.4422 170.9747 169.4068 165.3335 152.3874 164.8540
```

The individual simulated values vary, but with many simulations their distribution resembles the probability distribution specified by the model.

```
hist(x, n = 100)
```



3.12 Approximating probabilities by simulation

Simulation can also be used to approximate probabilities.

For example, we previously calculated:

$$P(X > 180) \approx 0.077.$$

Using simulated heights, we can estimate the same probability by calculating the proportion that exceed 180 cm:

```
mean(x > 180)
```

```
[1] 0.083
```

With a large number of simulations, this value should be close to the probability calculated from the normal distribution.

The law of large numbers states that if the same experiment is performed many times the average of the result will be close to the expected value.

3.13 Key message

! Key message

A probability model lets us move from describing possible outcomes to calculating how likely they are, under stated assumptions.

For continuous measurements, probabilities are areas under a density curve; for discrete counts, they are sums of probability masses.

In both cases, the CDF gives the probability at or below a cutoff, while a percentile answers the reverse question: which cutoff corresponds to a given probability?

Simulation generates possible outcomes under an assumed probability model.

By repeating the simulation many times, we can explore the variation and probabilities implied by that model.

Chapter 4

Repeating the sampling

Statistical Methods for Life Sciences

4.1 From individual observations to study results

So far, we have used probability models to describe the height of one randomly selected student.

In a study, however, we usually observe several individuals and calculate a summary, such as the sample mean.

If we repeated the study with different students, would we obtain the same sample mean?

A **parameter** describes a population. For example, the population mean height is denoted by μ .

A **statistic** is calculated from a sample. For example, the sample mean is

$$\bar{X} = \frac{1}{n} \sum_{i=1}^n X_i.$$

Before collecting the data, $\bar{X}$ is a random variable because its value depends on which individuals are selected.

After collecting the data, we obtain one observed value, denoted by $\bar{x}$.

4.2 Imagine repeating the study

Consider our hypothetical population of student heights, with

$$\mu = 170 \text{ cm} \quad \text{and} \quad \sigma = 7 \text{ cm}.$$

Imagine repeating the following study many times:

1. Randomly select five students.
2. Measure their heights.
3. Calculate their mean height.
4. Record the mean and select a new sample.

Each sample is likely to contain different students, so the sample means will not all be identical.

Provided that the observations are sampled independently from the same population, the expected value of the sample mean is the population mean:

$$E[\bar{X}] = \mu = 170 \text{ cm}.$$

The distribution of the sample means across repeated studies is called the **sampling distribution of the sample mean**.

i Note

The population distribution describes variation among individuals.
The sampling distribution describes variation among statistics calculated from different samples.

4.3 Simulating repeated studies

We can explore the sampling distribution by simulating many independent studies from our normal population model.

Before looking at the results, predict:

- Will sample means vary as much as individual heights?
- What will happen if each study includes 100 students rather than five?

All three distributions are centred near the population mean of 170 cm.

However:

- Individual heights show substantial variation.
- Means based on five students vary less than individual heights.
- Means based on 100 students vary much less.

Thus, **sample means vary less than individual observations, and means based on larger samples vary less than means based on smaller samples.**

This principle is not limited to means. Other statistics, such as a sample proportion or a difference between two means, also have sampling distributions.

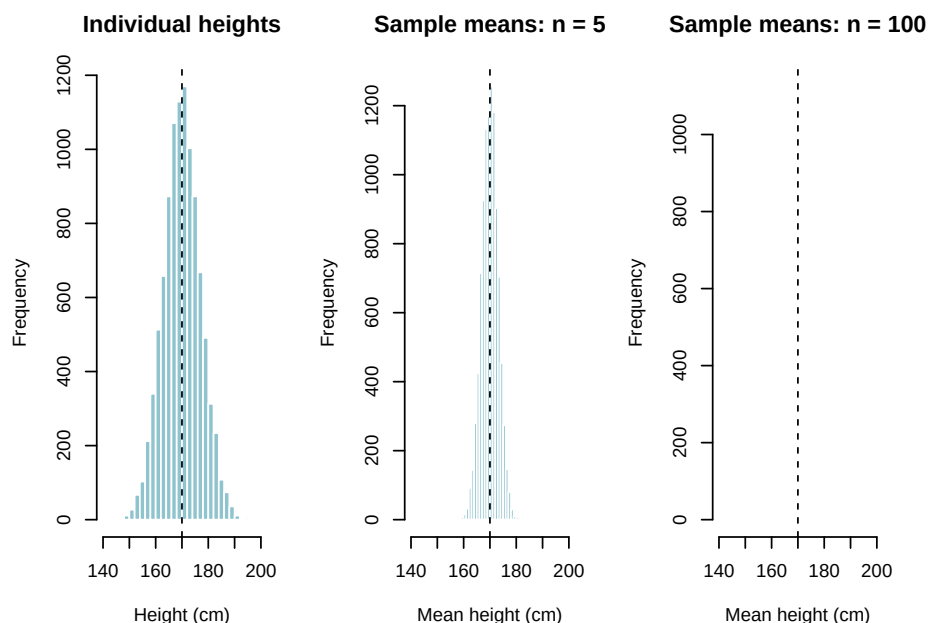


Figure 4.1: Individual heights and sample means are shown on the same horizontal scale. Larger samples produce less variable sample means.

4.4 Why do the sample means look normal?

In this example, individual heights follow a normal distribution. The mean of independent observations from a normal population is therefore also normally distributed.

The **central limit theorem** extends this result beyond normal populations:

For many population distributions, the sampling distribution of the sample mean becomes approximately normal as the sample size increases.

This does not mean that the individual observations become normally distributed. It is the **distribution of the sample mean across repeated samples** that becomes approximately normal.

How large the sample must be for a useful approximation depends on the shape of the population distribution and the presence of extreme values.

4.5 What if we have only one study?

The previous simulation assumes that we know the population model and can generate many new samples from it.

In practice, we usually have only one observed sample.

For example, if we measure 20 students, we have one sample containing 20 observations—not 20 independent samples.

Can we use this sample to approximate how the sample mean might vary across repeated studies?

4.5.1 Bootstrap: resampling the observed data

The **bootstrap** treats the observed sample as an approximation of the population and repeatedly resamples from it.

Imagine writing the 20 observed heights on cards:

1. Draw one card at random, record its value and return it.
2. Repeat until 20 values have been drawn.
3. Calculate their mean.
4. Repeat the procedure many times.

The cards are returned after every draw, so sampling is performed **with replacement**. An observed height can therefore appear more than once in a bootstrap sample, while another may not appear at all.

Sampling without replacement would not work: selecting all 20 cards would reproduce the original sample and therefore always give the same mean.

Bootstrap distribution of the mean

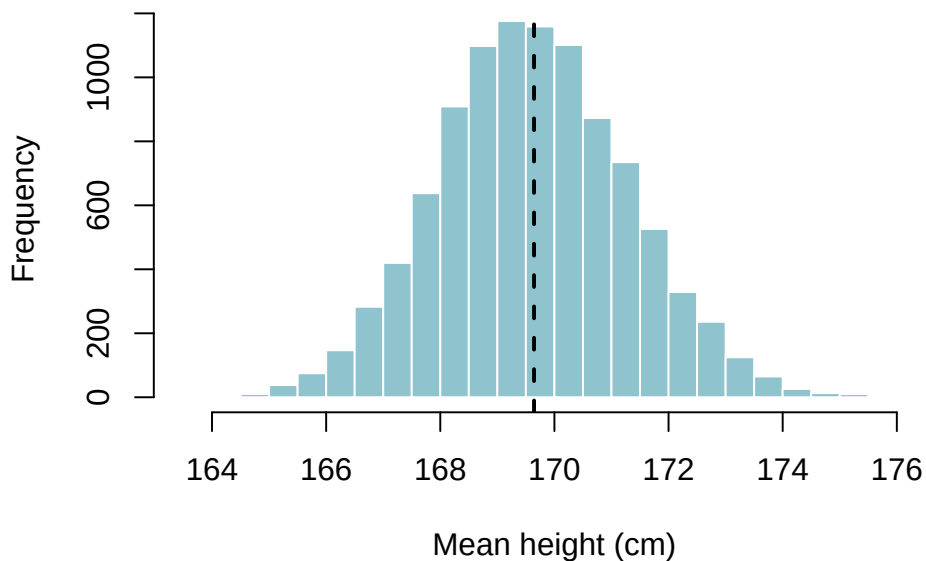


Figure 4.2: Bootstrap distribution of the mean based on one illustrative sample of 20 students.

The bootstrap means are centred near the **observed sample mean**, not necessarily the true population mean.

The spread of the bootstrap distribution approximates how much the sample mean would vary across repeated samples.

Bootstrapping does not create new biological information. Its results still depend on whether the original sample adequately represents the population.

This simple bootstrap also assumes independent observations. It cannot correct selection bias or compensate for a poorly designed study.

! Key message

A statistic, such as a sample mean, varies from sample to sample.

The distribution of its possible values across repeated samples is called its **sampling distribution**.

Sample means vary less than individual observations, and means based on larger samples generally vary less.

Next, we need a way to quantify the spread of the sampling distribution. This leads to the **standard error**.

Chapter 5

Describing precision

Statistical Methods for Life Sciences

5.1 How variable is a sample mean?

In the previous section, we saw that sample means vary across repeated samples.

The **standard error (SE)** quantifies this sampling variation.

It is important to distinguish between the standard deviation and the standard error:

- **Standard deviation (SD):** describes variation among individual observations.
- **Standard error (SE):** describes variation among estimates calculated from repeated samples of the same size.

For the sample mean, the SE is the standard deviation of its sampling distribution.

A smaller SE means that repeated samples tend to produce more similar estimates. We therefore say that the estimate is **more precise**.

i SD and SE answer different questions

SD: How different are individuals from one another?

SE: How much would our estimate vary if we repeated the study?

5.2 Standard error of the sample mean

For n independent observations from a population with standard deviation σ , the standard error of the sample mean is

$$\text{SE}(\bar{X}) = \frac{\sigma}{\sqrt{n}}.$$

The formula shows that the sampling variability of the mean depends on:

- the variability among individuals, represented by σ ;
- the number of independent observations, represented by n .

Greater variation among individuals produces a larger SE. A larger sample size produces a smaller SE.

For our height model, the population SD is 7 cm:

$$\begin{aligned} n = 5 : \quad \text{SE}(\bar{X}) &= \frac{7}{\sqrt{5}} \approx 3.13 \text{ cm}, \\ n = 100 : \quad \text{SE}(\bar{X}) &= \frac{7}{\sqrt{100}} = 0.70 \text{ cm}. \end{aligned}$$

This explains the sampling distributions from the previous section: means based on 100 students were much more tightly concentrated around 170 cm than means based on five students.

Increasing the sample size improves the **precision of the mean**. It does not reduce the biological variation among individual students.

5.3 How does sample size affect precision?

Because

$$\text{SE}(\bar{X}) = \frac{\sigma}{\sqrt{n}},$$

the SE decreases with the square root of the sample size.

This means that doubling the sample size does not halve the SE.

To halve the SE, we must multiply the sample size by four:

$$\frac{\sigma}{\sqrt{4n}} = \frac{1}{2} \frac{\sigma}{\sqrt{n}}.$$

Increasing the sample size therefore improves precision, but with diminishing returns.

5.4 Estimating the standard error from data

The formula

$$\text{SE}(\bar{X}) = \frac{\sigma}{\sqrt{n}}$$

uses the **population standard deviation**, σ .

In practice, σ is usually unknown. We estimate it from the observed sample using the **sample standard deviation**, s :

$$s = \sqrt{\frac{1}{n-1} \sum_{i=1}^n (x_i - \bar{x})^2}.$$

The sample SD describes the variation among the observed individuals and provides an estimate of the population SD, σ .

We can therefore estimate the standard error of the sample mean by

$$\widehat{\text{SE}}(\bar{X}) = \frac{s}{\sqrt{n}}.$$

...

For example, suppose we measure the heights of 20 students:

```
set.seed(2027)

heights <- rnorm(
  20,
  mean = 170,
  sd = 7
)

# calculate sample SD
s <- sd(heights)
s
```

```
[1] 7.635459
```

```
# estimate the SE of the sample mean
n <- length(heights)
SE <- s / sqrt(n)
SE
```

```
[1] 1.70734
```

This estimated SE describes how much the sample mean would be expected to vary across repeated samples of 20 independent students.

5.4.1 Estimating SE using the bootstrap

If we generated bootstrap means in the previous section, we can also estimate the SE by calculating their standard deviation:

```
bootstrap_means <- replicate(
  10000,
  mean(
    sample(
```

```

    heights,
    size = length(heights),
    replace = TRUE
  )
)
sd(bootstrap_means)

```

```
[1] 1.671085
```

The formula-based and bootstrap estimates both describe the estimated sampling variability of the mean. Their values will usually be similar, although not necessarily identical.

5.5 From the normal to the t-distribution

If the population SD, σ , is known, standardising the sample mean gives

$$Z = \frac{\bar{X} - \mu}{\sigma/\sqrt{n}} \sim N(0, 1).$$

If σ is unknown, which is the usual real-data situation, we replace it with the sample SD, s :

$$T = \frac{\bar{X} - \mu}{s/\sqrt{n}}.$$

Now the denominator is itself estimated from the sample, which introduces extra uncertainty.

Therefore, under a normal population model,

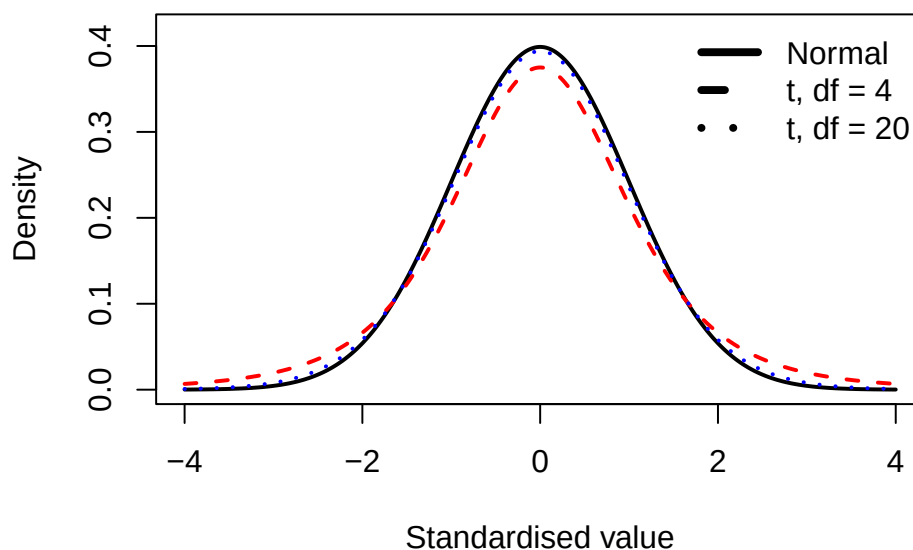
$$T \sim t_{n-1}.$$

The **t-distribution** is similar to the standard normal distribution but has heavier tails. As the sample size increases, it approaches $N(0, 1)$.

Because s is estimated from the sample, there is **extra uncertainty**.

The **t-distribution** accounts for this uncertainty:

- it looks similar to the normal distribution;
- it has heavier tails;
- with larger samples, it approaches the standard normal distribution.



5.6 More measurements or more individuals?

Suppose that we want to estimate the mean height in the population.

We could collect 100 measurements in two different ways:

- measure five students 20 times each;
- measure 100 different students once each.

Would these designs give equally precise estimates of the population mean?

Solution

No. Although both designs produce 100 measurements, they do not contain the same amount of independent information about variation between students.

In

$$\widehat{\text{SE}}(\bar{X}) = \frac{s}{\sqrt{n}},$$

n is the number of **independent students**, not simply the total number of measurements.

If the between-student SD is approximately 7 cm:

$$\text{Five students: } \text{SE}(\bar{X}) \approx \frac{7}{\sqrt{5}} = 3.13 \text{ cm},$$

$$\text{One hundred students: } \text{SE}(\bar{X}) \approx \frac{7}{\sqrt{100}} = 0.70 \text{ cm}.$$

Repeated measurements can reduce measurement error and give a more precise measurement of each student.

However, repeatedly measuring the same five students does not give us 100 independent students. Those five students could still happen to be unusually tall or unusually short.

Therefore, measuring 100 different students generally gives a more precise estimate of the **population mean**, assuming that each height is measured reasonably accurately.

In biological studies, this is why technical replicates cannot replace independent biological replicates.

5.7 Using SE to assess an observed result

Previously, we used a probability distribution to ask whether an individual height was unusual.

We can now ask a different question:

Is an observed **sample mean** unusual relative to its expected sampling variability?

Assume that student heights follow our normal population model:

$$X \sim N(170, 7^2).$$

For a sample of 100 independent students, the sample mean follows

$$\bar{X} \sim N\left(170, \frac{7^2}{100}\right).$$

The sampling distribution therefore has:

$$E[\bar{X}] = 170$$

and

$$\text{SE}(\bar{X}) = \frac{7}{\sqrt{100}} = 0.70 \text{ cm.}$$

Suppose that we observe a sample mean of 172 cm.

Its distance from the assumed population mean, measured in standard error units, is

$$z = \frac{\bar{x} - \mu_0}{\sigma/\sqrt{n}} = \frac{172 - 170}{0.70} \approx 2.86.$$

Thus, the observed sample mean is approximately 2.86 SE above the assumed population mean.

Under the assumed model,

$$P(\bar{X} \geq 172) = 1 - \Phi\left(\frac{172 - 170}{0.70}\right) \approx 0.0021.$$

In R:

```
pnorm(  
  172,  
  mean = 170,  
  sd = 7 / sqrt(100),  
  lower.tail = FALSE  
)
```

```
[1] 0.002137367
```

If the population mean is 170 cm and the model assumptions hold, the probability of obtaining a sample mean of 172 cm or higher from 100 independent students is approximately 0.2%.

Such a sample mean would therefore be unusual under the assumed model.

 Warning

This is the probability of obtaining a result at least this large **under the assumed model**.

It is not the probability that the model or the assumed population mean is correct.

5.8 An individual value is not a sample mean

An individual height of 172 cm is not unusual under the same population model.

For an individual student, we use the individual SD of 7 cm:

$$P(X \geq 172) = 1 - \Phi\left(\frac{172 - 170}{7}\right) \approx 0.3875.$$

```
pnorm(  
  172,  
  mean = 170,  
  sd = 7,  
  lower.tail = FALSE  
)
```

```
[1] 0.3875485
```

Approximately 38.8% of individual students are expected to be at least 172 cm tall under this model.

The cutoff of 172 cm is the same in both calculations, but the relevant distributions are different:

- Individual heights have an SD of 7 cm.

- Means of 100 heights have an SE of 0.70 cm.

A value can therefore be common for an individual observation but unusual for a sample mean.

5.9 Return to the two research groups

In our opening example, two research groups obtained mean heights of 172 cm and 175 cm.

Their observed difference was therefore

$$\bar{x}_2 - \bar{x}_1 = 175 - 172 = 3 \text{ cm.}$$

We now know that **different random samples can produce different sample means, even when they come from the same population.**

So an observed difference of 3 cm may simply reflect **sampling variation.**

The important question is:

Is a difference of 3 cm larger than we would expect from sampling variation alone?

To answer this, we would consider the statistic

$$\bar{X}_2 - \bar{X}_1$$

and its **sampling distribution**: the distribution of differences we would obtain if we repeatedly drew two independent samples from the same population under the same conditions.

How much these differences vary depends on, among other things:

- the variation among individuals;
- the number of independent individuals in each sample;
- the sampling design and model assumptions.

We can then compare our **observed difference** with the variation expected under a specified model.

This is us from **probability** to **statistical inference.**

In the next session, we will use **confidence intervals and hypothesis tests** to quantify uncertainty and assess evidence about population parameters.

! Key message

Different samples produce different results.

The sampling distribution describes how a statistic varies across repeated samples, and the standard error quantifies this variation.

Larger independent samples generally give more precise estimates and smaller standard errors.

Statistical inference uses the observed data and a probability model to assess what the data tell us about the population.

Chapter 6

Exercises

Statistical Methods for Life Sciences

Exercises

Exercise 6.1 (What is random?). We randomly select 10 adults from a defined population and record their BMI. The first person's BMI is 24.5.

Let's model BMI as a continuous, positive measurement.

Identify:

- The sample size.
- The sample space for one BMI measurement.
- The random variable.
- Its observed value.
- An example event involving BMI.

Solution

- **Sample size:** $n = 10$.
- **Sample space:** $S = [0, \infty)$ under our positive, continuous model.
- **Random variable:** X = the BMI of the first randomly selected adult.
- **Observed value:** $x = 24.5$.
- **Example event:** the first adult's BMI exceeds 25, $E = \{X > 25\}$.

Exercise 6.2 (What can we calculate — and what is missing?). Suppose 50% of adults in our hypothetical population have a BMI above 25.

For an adult randomly selected from this population:

- a) What is the probability that their BMI is 25 or less?

- b) Can we calculate the probability that their BMI exceeds 30 from this information alone?

 Solution

a)

By the **complement rule**:

$$P(X \leq 25) = 1 - P(X > 25) = 1 - 0.50 = 0.50.$$

b)

We cannot calculate $P(X > 30)$ exactly from this information. Because $\{X > 30\} \subseteq \{X > 25\}$, we know only that:

$$0 \leq P(X > 30) \leq 0.50.$$

Exercise 6.3 (A probability model for BMI). In the previous exercise, knowing only that 50% of adults have a BMI above 25 was not enough to calculate the probability of a BMI above 30.

Suppose we now use the following probability model for BMI in our hypothetical population:

$$X \sim N(25, 4^2).$$

For one randomly selected adult, calculate in R:

- What is the probability that their BMI exceeds 30?
- What is the probability that their BMI is between 20 and 30?
- How many standard deviations above the mean is a BMI of 30?
- Below which BMI value do 95% of adults fall according to this model?

 Solution

a)

The model tells us that

$$\mu = 25, \quad \sigma = 4.$$

For a BMI above 30:

$$P(X > 30) = 1 - F(30).$$

In R:

```
pnorm(30, mean = 25, sd = 4, lower.tail = FALSE)
```

```
[1] 0.1056498
```

b)

For a BMI between 20 and 30:

$$P(20 < X < 30) = F(30) - F(20).$$


```
pnorm(30, mean = 25, sd = 4) -  
  pnorm(20, mean = 25, sd = 4)
```

```
[1] 0.7887005
```

c)

To locate a BMI of 30 relative to the mean, we standardise:

$$z = \frac{30 - 25}{4} = 1.25.$$

A BMI of 30 is therefore 1.25 standard deviations above the mean.

We can also calculate the upper-tail probability using the standard normal distribution:

```
z <- (30 - 25) / 4
```

```
pnorm(z, lower.tail = FALSE)
```

```
[1] 0.1056498
```

d)

For the 95th percentile, we want the value $x_{0.95}$ such that

$$P(X \leq x_{0.95}) = 0.95.$$

On the standard normal scale,

$$z_{0.95} = \Phi^{-1}(0.95) \approx 1.645.$$

In R:

```
qnorm(0.95)
```

```
[1] 1.644854
```

We then transform back to the BMI scale:

$$x_{0.95} = \mu + \sigma z_{0.95} = 25 + 4(1.645) \approx 31.6.$$

So approximately 95% of adults have a BMI below 31.6 under this model.

We can also calculate the cutoff directly in R:

```
qnorm(0.95, mean = 25, sd = 4)
```

```
[1] 31.57941
```

Exercise 6.4 (A discrete probability model). Suppose that 15% of bacterial isolates from a particular population are resistant to an antibiotic.

We randomly and independently select 30 isolates and let K be the number that are resistant.

- What distribution is appropriate for K ? Specify its parameters.
- What values can K take?
- What is the expected number of resistant isolates?
- What is the probability that exactly 4 isolates are resistant?
- What is the probability that 4 or more isolates are resistant?
- Simulate 10,000 values of K in R. Use the simulated values to approxi-

mate the probability that 4 or more isolates are resistant. Compare your estimate with the exact probability calculated above.

💡 Solution

a)

For each isolate, resistance is a binary outcome:

$$Y = \begin{cases} 1, & \text{resistant,} \\ 0, & \text{not resistant.} \end{cases}$$

with

$$P(Y = 1) = p = 0.15.$$

Thus,

$$Y \sim \text{Bernoulli}(0.15).$$

The number of resistant isolates among 30 independent isolates is

$$K = \sum_{i=1}^{30} Y_i,$$

so

$$K \sim \text{Bin}(30, 0.15).$$

b)

The possible values are

$$K \in \{0, 1, 2, \dots, 30\}.$$

c)

The expected number of resistant isolates is

$$E[K] = np = 30 \times 0.15 = 4.5.$$

d)

For exactly four resistant isolates,

$$P(K = 4) = \binom{30}{4} (0.15)^4 (0.85)^{26}.$$

In R:

```
dbinom(4, size = 30, prob = 0.15)
```

```
[1] 0.2028089
```

e)

For four or more resistant isolates,

$$P(K \geq 4) = 1 - P(K \leq 3).$$

In R:

```
1 - pbinom(3, size = 30, prob = 0.15, lower.tail = TRUE)
```

```
[1] 0.6783401
```

```
# or
```

```
pbinom(3, size = 30, prob = 0.15, lower.tail = FALSE)
```

```
[1] 0.6783401
```

f)

We can simulate repeated samples from the binomial model:

```
set.seed(2026)
```

```
K_sim <- rbinom(  
  10000,  
  size = 30,  
  prob = 0.15  
)
```

The simulated probability of observing four or more resistant isolates is:

```
mean(K_sim >= 4)
```

```
[1] 0.6756
```

This should be close to the exact probability obtained above:

```
pbinom(  
  3,  
  size = 30,  
  prob = 0.15,  
  lower.tail = FALSE  
)
```

```
[1] 0.6783401
```

We can simulate repeated samples from the binomial model:

```
set.seed(2026)
```

```
K_sim <- rbinom(  
  10000,  
  size = 30,  
  prob = 0.15  
)
```

The simulated probability of observing four or more resistant isolates is:

```
mean(K_sim >= 4)
```

```
[1] 0.6756
```

This should be close to the exact probability:

```
pbinom(
  3,
  size = 30,
  prob = 0.15,
  lower.tail = FALSE
)
```

```
[1] 0.6783401
```

The simulated value will not be exactly the same because it is based on a finite number of random simulations. With many simulations, the estimate tends to get closer to the model probability.

Exercise 6.5 (How precise is the estimated mean?). A random sample of 36 adults has

$$\bar{x} = 26.2$$

and sample standard deviation

$$s = 3.6.$$

Suppose we want to compare the observed mean BMI with a value of 25.

- Estimate the standard error of the sample mean.
- How far is the observed sample mean from 25, measured in estimated standard error units?
- Should we compare this standardised statistic with a standard normal distribution or a t-distribution? Why?
- How many degrees of freedom does the t-distribution have?
- If we instead sampled 100 adults and obtained approximately the same sample SD, what would happen to the standard error?

Solution

a)

Because the population SD, σ , is unknown, we estimate the standard error using the sample SD:

$$\widehat{\text{SE}}(\bar{X}) = \frac{s}{\sqrt{n}}.$$

For $n = 36$ and $s = 3.6$,

$$\widehat{\text{SE}}(\bar{X}) = \frac{3.6}{\sqrt{36}} = \frac{3.6}{6} = 0.6.$$

b)

The observed mean is

$$26.2 - 25 = 1.2$$

BMI units above 25.

In estimated standard error units,

$$T = \frac{\bar{x} - 25}{s/\sqrt{n}} = \frac{26.2 - 25}{0.6} = 2.$$

So the observed mean is **2 estimated standard errors above 25**.

c)

Because σ is unknown and has been replaced by the sample SD, s , the appropriate reference distribution is a **t-distribution**.

d) For a one-sample mean,

$$df = n - 1 = 35.$$

Therefore,

$$T \sim t_{35}$$

under the corresponding normal population model.

e)

If $n = 100$ and $s \approx 3.6$,

$$\widehat{SE}(\bar{X}) = \frac{3.6}{\sqrt{100}} = 0.36.$$

The larger sample therefore gives a **smaller standard error** and a more precise estimate of the population mean.

Exercise 6.6 (Estimating uncertainty with the bootstrap). Suppose we observe BMI in a sample of 20 adults:

```
bmi <- c(
  21.8, 22.4, 23.1, 23.7, 24.0,
  24.3, 24.8, 25.0, 25.2, 25.5,
  25.9, 26.1, 26.4, 26.8, 27.1,
  27.5, 28.0, 28.6, 29.2, 30.1
)
```

- Calculate the observed sample mean.
- Create one bootstrap sample by sampling 20 observations from **bmi with replacement**. Calculate its mean.
- Repeat this procedure 5,000 times and store the bootstrap means.
- Plot the distribution of the bootstrap means. Around what value is the bootstrap distribution centred?
- Use the bootstrap distribution to estimate the probability that a bootstrap sample mean is greater than 26.
- Use the bootstrap distribution to estimate the standard error of the sample mean.
- Compare the bootstrap estimate of SE with the usual estimate: $\frac{s}{\sqrt{n}}$

 Solution

a) The observed sample mean is:

```
mean(bmi)
```

```
[1] 25.775
```

b) A single bootstrap sample is obtained by sampling from the observed values with replacement:

```
set.seed(2026)
```

```
bmi_boot <- sample(  
  bmi,  
  size = length(bmi),  
  replace = TRUE  
)
```

```
mean(bmi_boot)
```

```
[1] 25.66
```

The bootstrap sample contains the same number of observations as the original sample, but some observations may appear more than once and others may not appear at all.

c) We repeat the resampling procedure many times:

```
set.seed(2026)
```

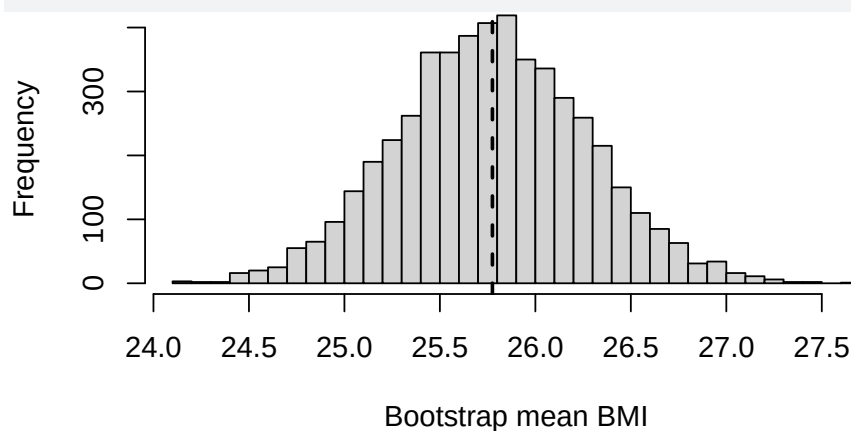
```
bootstrap_means <- replicate(  
  5000,  
  mean(  
    sample(  
      bmi,  
      size = length(bmi),  
      replace = TRUE  
    )  
  )  
)
```

Each value in `bootstrap_means` is the mean of one bootstrap sample.

d) We can inspect the empirical distribution of the bootstrap means:

```
hist(
  bootstrap_means,
  breaks = 30,
  main = "",
  xlab = "Bootstrap mean BMI"
)

abline(
  v = mean(bmi),
  lty = 2,
  lwd = 2
)
```



The bootstrap distribution should be centred close to the **observed sample mean**.

- e) We can estimate the probability that a bootstrap sample mean exceeds 26 by calculating the proportion of bootstrap means above 26:

```
mean(bootstrap_means > 26)
```

```
[1] 0.3222
```

This estimates

$$P(\bar{X}^* > 26),$$

where $\bar{X}^*$ denotes a bootstrap sample mean.

- f) The bootstrap estimate of the standard error is the standard deviation of the bootstrap means:

```
sd(bootstrap_means)
```

```
[1] 0.4961505
```

Thus,

$$\widehat{SE}_{\text{bootstrap}} = SD(\bar{X}^*).$$

- g) The usual estimated standard error of the sample mean is

$$\widehat{\text{SE}}(\bar{X}) = \frac{s}{\sqrt{n}}.$$

In R:

```
sd(bmi) / sqrt(length(bmi))
```

```
[1] 0.5015699
```

The two estimates should be similar.

The bootstrap uses repeated resampling of the observed data to construct an **empirical distribution of the statistic**. This distribution can be used to study its variability and, when appropriate, estimate probabilities under the resampling scheme.

Appendix: discrete random variables

A categorical random variable has nominal or ordinal outcomes such as {red, blue, green} or {tiny, small, average, large, huge}.

A discrete random variable usually represents a count and has a countable number of possible values, such as {1, 2, 3, 4, 5, 6}; {0, 2, 4, 6, 8} or all integers.

A discrete or categorical random variable can be described by its **probability mass function (PMF)**.

The probability that the random variable X takes the value x is denoted $P(X = x) = p(x)$. Note that:

1. $0 \leq p(x) \leq 1$, a probability is always between 0 and 1.
2. $\sum_x p(x) = 1$, the probabilities over all possible outcomes sum to 1.

Example 6.1 (The number of dots on a die). When rolling a die there are six possible outcomes: 1, 2, 3, 4, 5 and 6, each of which has the same probability, if the die is fair. The outcome of one die roll can be described by a random variable X . The probability of a particular outcome x is denoted $P(X = x)$ or $p(x)$.

The probability mass function of a fair six-sided die can be summarized in a table;

```
tibble(x = 1:6, `p(x)` = round(rep(1/6, 6), 3)) |> gt()
##kable(matrix(c(as.character(1:6), format(rep(1/6,6),
  ↪ digits=3)),ncol=6, byrow=TRUE, dimnames=list(c('x', 'p(x)'),
  ↪ c())) %>% kable_styling()
```

or in a bar plot;

```
data.frame(x=1:6, p=1/6) %>% ggplot(aes(x=x, y=p)) +
  ↪ geom_bar(stat="identity") + theme_bw() + ylim(c(0,.25)) +
  ↪ scale_x_continuous(breaks = 1:6) + ylab("p(x)")
```

Table 6.1: Probability mass function of a fair six-sided dice.

x	p(x)
1	0.17
2	0.17
3	0.17
4	0.17
5	0.17
6	0.17

Table 6.2: Probability mass function of a nucleotide site.

x	p(x)
A	0.4
C	0.2
T	0.1
G	0.3

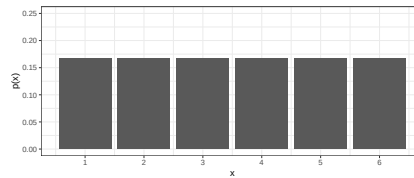


Figure 6.1: Probability mass function of a fair six-sided dice.

Example 6.2 (Nucleotide at a given site). The nucleotide at a given genomic site can be one of the four nucleotides: {A, C, T, G}. Unlike the sides on a die, the four nucleotides are usually not equally likely. The nucleotide at the site can be described by a random variable, X . The probability mass function can be summarized in a table or a bar plot.

```
p <- c(A=0.4, C=0.2, T=0.1, G=0.3)
tibble(x=names(p), "p(x)"=p) |> gt()
#kable(matrix(c(names(p), p), ncol=4, byrow=TRUE,
  ↪ dimnames=list(c('x', 'p(x)'), c())) %>% kable_styling()
```

```
data.frame(p) %>% rownames_to_column("x") %>% ggplot(aes(x=x,
  ↪ y=p)) + geom_bar(stat="identity") + theme_bw()
```

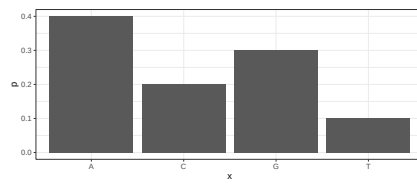


Figure 6.2: Probability mass function of a nucleotide site.

Example 6.3 (CFU). The number of bacterial colony-forming units (CFU) on a plate is a random variable that can be described by a PMF such as the one shown in Figure 6.3.

```
x=0:50
ggplot(data.frame(x=x, fx=dpois(x, lambda=25)), aes(x,fx)) +
  geom_bar(stat="identity") + theme_bw() + ylab("p(x)")
```

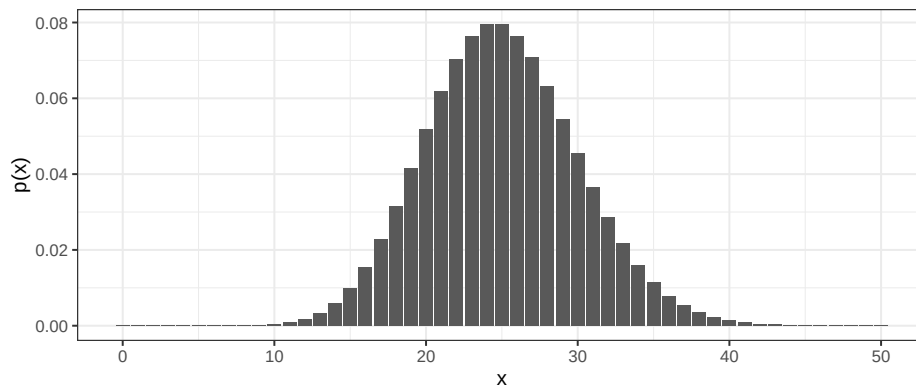


Figure 6.3: Probability mass distribution of the number of bacterial colonies on an agar plate.

The probability mass function can be used to compute the probability of an event. For example:

- Getting an even number when rolling a fair six-sided die: sum up the probability for 2, 4, and 6:

$$p(\text{even}) = p(2) + p(4) + p(6) = 0.5.$$

- Getting fewer than 15 CFUs on an agar plate: sum up the probabilities:

$$P(X < 15) = \sum_{k=0}^{14} p(k) = 0.01.$$

The **cumulative distribution function (CDF)** is defined for discrete numeric random variables as

$$F(x) = P(X \leq x).$$

In words, $F(x)$ is the probability that the random variable X takes a value less than or equal to x . For the CFU example, the probability of getting fewer than 15 CFUs on an agar plate can be written as $F(14) = P(X \leq 14) = P(X < 15)$.

Expected value and variance

Two common properties of a probability distribution, such as a probability mass function, are the **expected value** and the **variance**.

The **expected value** is the average outcome of a random variable over many trials and is denoted $E[X]$ or μ and can be computed by summing up all possible outcome values weighted by their probability:

$$E[X] = \mu = \sum_{x \in S} xp(x),$$

where S is the set of all possible outcomes.

For a **uniform distribution**, where every outcome has the same probability, the expected value can be computed as the sum of all outcome values divided by the total number of outcome values.

For a finite population in which every object is equally likely to be selected, the expected value can also be computed as the population mean, i.e., by summing over outcome values for all objects in the population;

$$E[X] = \mu = \frac{1}{N} \sum_{i=1}^N x_i,$$

where N is the number of objects in the population.

The **variance** is a measure of spread defined as the expected value of the squared difference between the random variable and its expected value:

$$\text{var}(X) = \sigma^2 = E[(X - E[X])^2] = \sum_{x \in S} (x - \mu)^2 p(x).$$

The variance can also be computed by summing over the outcome value for every object in the population:

$$\text{var}(X) = \sigma^2 = \frac{1}{N} \sum_{i=1}^N (x_i - \mu)^2,$$

where N is the number of objects in the population.

The **standard deviation** is the square root of the **variance**. The standard deviation of a random variable is usually denoted σ . The standard deviation is always positive and on the same scale as the outcome values.

Linear transformations and combinations

The expected value and variance of combined or transformed random variables can often be computed using the following rules:

$$E(aX) = aE(X)$$

$$E(X + Y) = E(X) + E(Y)$$

$$E[aX + bY] = aE[X] + bE[Y],$$

where a and b are constants.

$$\text{var}(aX) = a^2\text{var}(X)$$

For independent random variables X and Y :

$$\text{var}(aX + bY) = a^2\text{var}(X) + b^2\text{var}(Y).$$

Simulate distributions

Once a random variable's probability distribution is known, probabilities, such as $P(X = x)$, $P(X < x)$ and $P(X \geq x)$, and properties, such as expected value and variance, of the random variable can be computed. If the distribution is not known, simulation can be used to approximate these values.

Example 6.4 (Simulate coin toss). In a single coin toss, the probability of heads is 0.5. In 20 coin tosses, what is the probability of at least 15 heads?

The outcome of a single coin toss is a random variable, X , with two possible outcomes $\{H, T\}$. We know that $P(X = H) = 0.5$. The random variable of interest is the number of heads in 20 coin tosses, Y . The probability that we need to compute is $P(Y \geq 15)$.

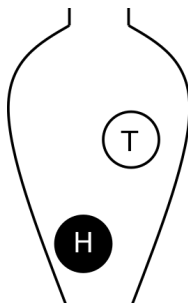


Figure 6.4: A coin toss. Urn model with one black ball (heads) and one white ball (tails).

A single coin toss can be modelled by an urn with two balls. When a ball is drawn randomly from the urn, the probability to get the black ball (heads) is $P(X = H) = 0.5$.

In R we can simulate random draws from an urn model using the function `sample`.

```
set.seed(260602)
## A single coin toss
sample(c("H", "T"), size=1)
```

```
[1] "T"
```

```
## Another coin toss
sample(c("H", "T"), size=1)
```

```
[1] "T"
```

Every time you run `sample`, a new coin toss is simulated.

If we want to simulate tossing 20 coins (or one coin 20 times), we can use the same urn model, provided that the ball is replaced after each draw.

The argument `size` tells the function how many balls we want to draw from the urn. To draw 20 balls from the urn, set `size=20`. Remember to replace the ball after each draw.

```
## 20 independent coin tosses
(coins <- sample(c("H", "T"), size=20, replace=TRUE))
```

```
[1] "T" "T" "T" "T" "H" "H" "T" "T" "H" "T" "T" "H" "T" "T" "H" "T" "T" "T" "H"
[20] "T"
```

How many heads did we get in the 20 random draws?

```
## How many heads?
sum(coins == "H")
```

```
[1] 6
```

We can repeat this experiment (tossing 20 coins and counting the number of heads) several times to estimate the distribution of number of heads in 20 coin tosses.

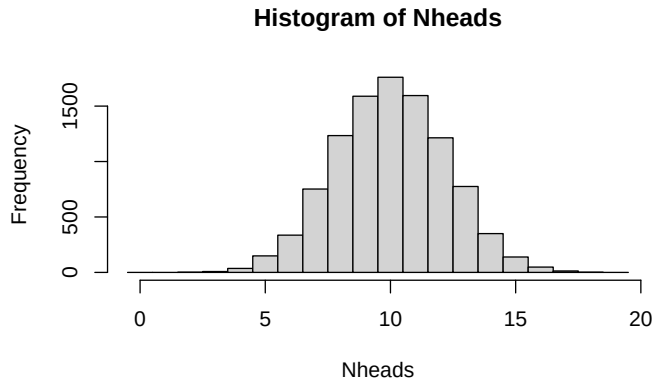
To do the same thing several times, we use the function `replicate`.

To simulate tossing 20 coins and counting the number of heads 10000 times, do the following:

```
Nheads <- replicate(10000, {
  coins <- sample(c("H", "T"), size=20, replace=TRUE)
  sum(coins == "H")
})
```

Plot the distribution of the number of heads in a histogram.

```
hist(Nheads, breaks=(0:20)-0.5)
```



Now, let's get back to the question: when tossing 20 coins, what is the probability of at least 15 heads?

$$P(Y \geq 15)$$

Count how many times out of our 10000 experiments, the number of heads is 15 or greater.

```
sum(Nheads >= 15)
```

```
[1] 201
```

From this, we conclude that

$$P(Y \geq 15) = 201/10000 = 0.0201$$

Note that this is an approximation of the probability. The more times we repeat the experiment, the better our approximation will be. In fact, the number of heads in a series of independent coin tosses follows a binomial distribution, which is a discrete parametric distribution. We will get back to this later in the chapter.

Resampling can also be used to compute other properties of a random variable, such as its expected value.

The **law of large numbers** states that if the same experiment is performed many times, the average result will be close to the expected value.

The coin flip is a common example in statistics, but many situations with two outcomes can be modeled using similar models. If we consider the outcome of interest success (like heads in the coin example) and the alternative outcome failure, we can model, for example:

- Drug effect: A patient can respond to drug treatment (success) or not (failure)
- Side effect: After a treatment a patient might experience a side effect (success) or not (failure)
- Treatment or placebo: Randomly assign a study participant to the treatment or placebo group

- Antibiotic resistance: A bacterium is either resistant to an antibiotic or not

The probability of success and failure can be equal, like in the coin example, but they do not have to be. If the probability of a patient responding to a treatment is $p = 0.80$, then the probability of the patient not responding is $1 - p = 0.20$. This can be modeled using an urn model with 4 black balls (cured) and 1 white ball (not cured).

Parametric discrete distributions

A discrete parametric distribution is a probability distribution described by a set of parameters. In many situations, data can be assumed to follow a parametric distribution.

Uniform

In a uniform distribution, every possible outcome has the same probability. With n different outcomes, the probability for each outcome is $1/n$.

Bernoulli

A Bernoulli trial is a random experiment with two outcomes: success and failure. The probability of success, $P(\text{success}) = p$, is constant. The probability of failure is $P(\text{failure}) = 1 - p$.

When coding, it is convenient to code success as 1 and failure as 0.

The outcome of a Bernoulli trial is a discrete random variable, X .

$$P(X = x) = p(x) = \begin{cases} p & \text{if } x = 1 \text{ success} \\ 1 - p & \text{if } x = 0 \text{ failure} \end{cases}$$

Using the definitions of expected value and variance it can be shown that:

$$\begin{aligned} E[X] &= p \\ \text{var}(X) &= p(1 - p) \end{aligned}$$

A Bernoulli trial can be simulated, as seen in previous section, using the function `sample`.

Binomial

A Bernoulli trial describes a single success/failure experiment. A binomial random variable counts the number of successes in n such trials.

The number of successes in a series of Bernoulli trials is a random variable, X , that follows the **binomial distribution**, if:

- The number of trials, n , is fixed.
- Each trial is independent.

- The probability of success, p , is the same for each trial.

$$X = \sum_{i=1}^n Z_i,$$

where all Z_i describe the outcome of independent and identical Bernoulli trials with probability p for *success* ($P(Z_i = 1) = p$).

The probability mass function of X is called the binomial distribution. In short, we use the notation:

$$X \sim \text{Bin}(n, p)$$

The probability mass function is

$$P(X = k) = \binom{n}{k} p^k (1 - p)^{n-k}$$

It can be shown that

$$\begin{aligned} E[X] &= np \\ \text{var}(X) &= np(1 - p) \end{aligned}$$

A binomial random variable is the number of successes when sampling n objects *with* replacement from an urn with objects of two types, of which the interesting type (success) has probability p .

The probability mass function, $P(X = k)$ can be computed using the R function `dbinom` and the cumulative distribution function $P(X \leq k)$ can be computed using `pnbinom`.

Examples of binomial random variables;

- The number of patients responding to a treatment out of n patients in a study, if the probability of a patient responding to treatment is p .
- The number of patients experiencing a side effect out of n patients in a study, if the probability of a side effect is p .
- the number of mutations in a gene of length n , if the mutations are independent and identically distributed and the probability of a mutation at every single position is p .

Poisson

The Poisson distribution is a discrete probability distribution that expresses the probability of a given number of events occurring in a fixed interval of time or space, given that these events occur with a known constant mean rate and independently of the time since the last event. It is commonly used for rare events.

A rare disease has a very low probability for a single individual. The number of individuals in a large population that catch the disease in a certain time period can be modelled using the Poisson distribution.

The probability mass function has a single parameter, λ , the constant mean rate, and can be described as:

$$P(X = k) = \frac{\lambda^k}{k!} e^{-\lambda}.$$

The parameter λ is both the expected value and the variance of the distribution:

In the case where the Poisson distribution is used to approximate the binomial distribution, the expected value $\lambda = n\pi$, where n is the number of objects sampled from the population and π is the probability of a single object being of the interesting type.

The variance is equal to the expected value:

$$\text{var}(X) = E[X] = \lambda = n\pi$$

The Poisson distribution can approximate the binomial distribution if n is large ($n > 20$) and π is small ($\pi < 0.05$ and $n\pi < 10$).

Examples of Poisson random variables;

- A rare disease has a very low probability for a single individual. The number of individuals in a large population that catch the disease in a certain time period is a Poisson random variable.
- Number of reads aligned to a gene region

Negative binomial

A negative binomial distribution describes the number of failures that occur before a specified number of successes, r , has occurred in a sequence of independent and identically distributed Bernoulli trials. r is also called the dispersion parameter.

In R: `dnbinom`, `pnbinom`, `qnbino`

Geometric

The geometric distribution is a special case of the negative binomial distribution in which $r = 1$.

In R: `dgeom`, `pgeom`, `qgeom`

Hypergeometric distribution

The hypergeometric distribution occurs when sampling n objects *without* replacement from an urn with N objects of two types, of which there are K objects of the interesting type. The probability of selecting an object of the interesting type is $p = K/N$.

The probability mass function

$$P(X = k) = \frac{\binom{K}{k} \binom{N-K}{n-k}}{\binom{N}{n}}$$

can be computed in R using **dhyp**er and the cumulative distribution function $P(X \leq k)$ can be computed using **phyper**.

Examples of hypergeometric random variables:

- In a student group of 25 individuals, 10 are R beginners. If 5 individuals are randomly chosen to belong to group A, the number of R beginners in group A is a hypergeometric random variable.
- A drug company is producing 1000 pills per day, 5% have an amount of active substance below an acceptable threshold. In a random sample of 10 pills, how many contain too little active substance? The number of pills with too little active substance is a hypergeometric random variable.
- You investigate the differential expression of 10000 genes, 150 of these genes belong to the super interesting pathway XXX. You run a black-box algorithm that reports 80 differentially expressed (DE) genes. If the black-box algorithm chooses DE genes at random, the number of DE genes that belong to pathway XXX is a hypergeometric random variable.

Summary of discrete distributions

- **Bernoulli**: one success/failure trial
- **Binomial**: number of successes in n independent Bernoulli trials
- **Poisson**: number of events in a fixed interval
- **Hypergeometric**: number of successes in sampling without replacement
- **Geometric**: number of failures before the first success
- **Negative binomial**: number of failures before the r -th success
- **Uniform**: all outcomes equally likely

The binomial, hypergeometric, negative binomial and Poisson distributions have similarities. For large N (large population), the hypergeometric and binomial distributions are very similar. The Poisson distribution has equal mean and variance, whereas the binomial has a variance smaller than the mean, and the negative binomial has a variance greater than the mean.

```
N=100
n=20
x <- 0:n
probs <- c(0.1, 0.3, 0.5)
df <- rbind(data.frame(prob=probs) |> group_by(prob) |>
  ↪ reframe(data.frame(x=x) |> mutate(px=dbinom(x, n, prob))),
  ↪ .groups="drop") |> mutate(distribution="Binomial"),
data.frame(prob=probs) |> group_by(prob) |>
  ↪ reframe(data.frame(x=x) |> mutate(px=dpois(x, n*prob))),
  ↪ .groups="drop") |> mutate(distribution="Poisson"),
data.frame(prob=probs) |> group_by(prob) |>
  ↪ reframe(data.frame(x=x) |> mutate(px=dnbinom(x, n*(1-prob),
  ↪ 1-prob))), .groups="drop") |> mutate(distribution="Negative
  ↪ binomial"),
```

```
data.frame(prob=probs) |> group_by(prob) |>
  ↪ reframe(data.frame(x=x) |> mutate(px=dhyper(x, round(N*prob),
  ↪ round(N*(1-prob)), n)), .groups="drop") |>
  ↪ mutate(distribution="Hypergeometric"))

df|>ggplot(aes(x=x, y=px, color=distribution,
  ↪ shape=factor(prob))) + geom_point(size=5) + geom_line() +
  ↪ theme_bw() + scale_color_colorblind() + ylab("p(x)") +
  ↪ scale_shape_discrete("")
```

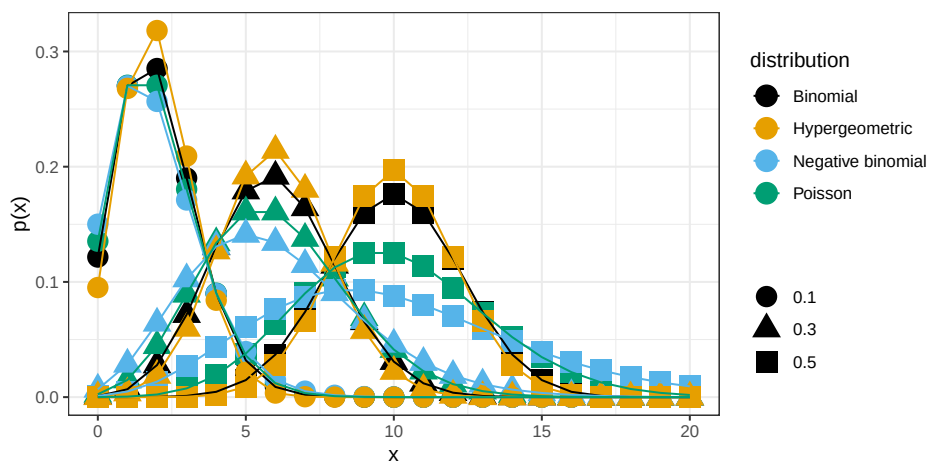


Figure 6.5: Probability mass functions for the binomial distribution ($n=20$, $p=0.1$, 0.3 or 0.5), hypergeometric distribution ($N=100$, $n=20$, $p=0.1$, 0.3 or 0.5), negative binomial distribution ($n=20$, $r=n*p$, $p=0.1$, 0.3 or 0.5) and Poisson distribution ($n=20$, $p=0.1$, 0.3 or 0.5).

In R, probability mass functions, $P(X = x)$, for the binomial, hypergeometric, negative binomial and Poisson distributions can be computed using the functions `dbinom`, `dhyper`, `dnbinom` and `dpois`, respectively.

Cumulative distribution functions, $P(X \leq x)$ can be computed using `pbinom`, `phyper`, `pnbinom` and `ppois`.

Also, functions for computing an x such that $P(X \leq x) = q$, where q is a probability of interest, are available using `qbinom`, `qhyper`, `qnbinom` and `qpois`.

Appendix: cotinuous random variables

A continuous random variable is not limited to discrete values, but can take any value within one or several intervals on the real line.

Examples of continuous random variables include weight, height, speed, and fluorescence intensity.

A continuous random variable can be described by its **probability density function**, PDF. For a continuous random variable, the probability of a single value is zero, i.e., $P(X = x) = 0$. Probabilities are instead represented by the area under the probability density function.

For discrete random variables, probabilities are assigned to individual values through a probability mass function (PMF). For continuous random variables, probabilities are assigned to intervals through a probability density function (PDF).

```
## ounce <- 0.0283495231
## pound <- 0.45359237
## ggplot(babies, aes(x=wt*ounce)) + theme_bw() + xlab("x") +
  ↪ ylab("f(x)") + geom_density() #+ ggtitle("Weight (kg) of a
  ↪ newborn baby"))
##plot(ggplot(fat, aes(x=weight*pound)) + theme_bw() + xlab("x")
  ↪ + ylab("f(x)") + geom_density() + ggtitle("Weight (kg) of a
  ↪ man"))
##Simulate weights
baby <- data.frame(smoke=sample(1000, x=0:2,
  ↪ prob=c(0.45,0.45,0.1), replace=TRUE)) %>% mutate(m=c(3.4,
  ↪ 3.0, 2.5)[smoke+1], wt=rnorm(n(), mean=m, sd=0.5))
ggplot(baby, aes(x=wt)) + theme_bw() + xlab("x") + ylab("f(x)") +
  ↪ geom_density()
```

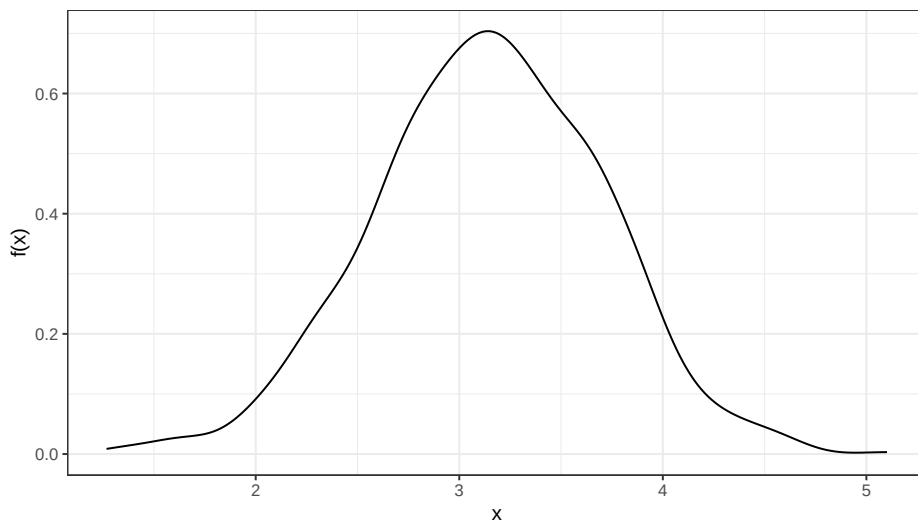


Figure 6.6: Probability density function of the weight of a newborn baby.

```
x <- seq(-4,4,.01)
ggplot(data.frame(x=x, fx=dnorm(x)), aes(x,fx)) + geom_line() +
  theme_bw() + ylab("f(x)")
```

The probability density function, $f(x)$, is defined such that the total area under the curve is 1.

$$\int_{-\infty}^{\infty} f(x)dx = 1$$

```
#ggplot(baby, aes(x=wt)) + theme_bw() + xlab("x") + ylab("f(x)")
  + geom_density()
#plot(ggplot(data.frame(w=wt), aes(x=w)) + theme_bw() + xlab("x")
  + ylab("f(x)") + geom_density())

df.wt <- density(baby$wt)
df.wt <- data.frame(x=df.wt$x, y=df.wt$y)
plot(ggplot(baby, aes(x=wt)) + theme_bw() + xlab("x") +
  ylab("f(x)") + geom_density() + geom_area(data=df.wt %>%
  filter(x<3.75, x>2.33), aes(x=x, y=y)) +
  scale_x_continuous(breaks=c(2,2.33,3,3.75,4,5),
  labels=c('2','a','3','b','4','5')) + geom_hline(yintercept=0)
  + theme(panel.grid.minor = element_blank(),
  panel.grid.major.x = element_line(color = c("grey92", NA,
  "grey92", NA, "grey92", "grey92"))))
#plot(ggplot(data.frame(w=wt), aes(x=w)) + theme_bw() + xlab("x")
  + ylab("f(x)") + geom_density() + geom_area(data=df.wt %>%
  filter(x<3.75, x>2.33), aes(x=x, y=y)) +
  scale_x_continuous(breaks=c(2,2.33,3,3.75,4,5),
  labels=c('2','a','3','b','4','5')) + geom_hline(yintercept=0)
  + theme(panel.grid=element_blank()))
```

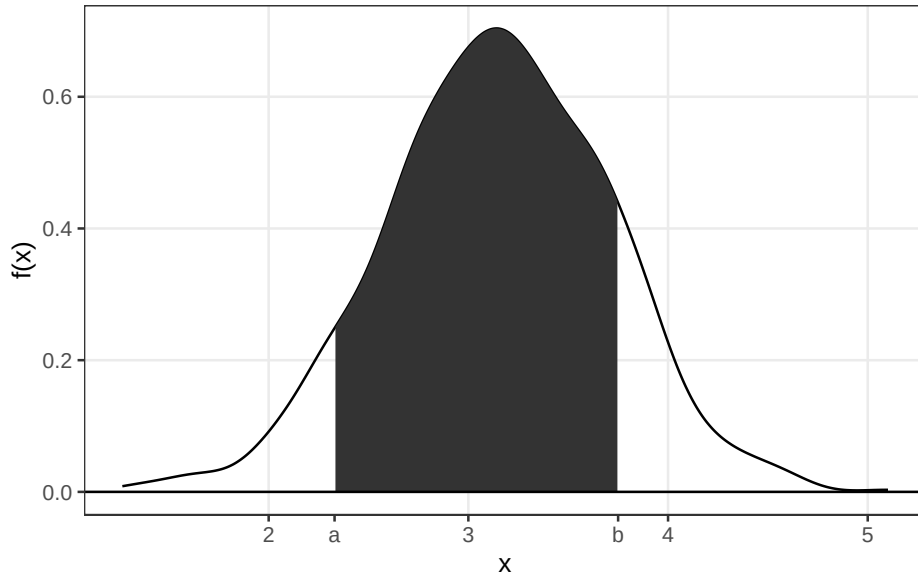


Figure 6.7: The probability $P(a \leq X \leq b)$ can be computed by computing the area under the probability density function between a and b .

```
w<-seq(1.5,5.5,.01)
df.nwt <- data.frame(w=w, f=dnorm(w, 3.5, 0.5))
#ggplot(df.nwt, aes(x=w, y=f)) + geom_line() + theme_bw() +
  ↪ xlab("Weight (kg)") + ylab("f(x)")

ggplot(df.nwt, aes(x=w, y=f)) + geom_line() + theme_bw() +
  ↪ xlab("Weight (kg)") + ylab("f(x)") + geom_area(data=df.nwt
  ↪ %>% filter(w<3.75, w>2.33)) +
  ↪ scale_x_continuous(breaks=c(2,2.33,3,3.75,4,5),
  ↪ labels=c('2','a','3','b','4','5')) + geom_hline(yintercept=0)
  ↪ + theme(panel.grid.minor = element_blank(),
  ↪ panel.grid.major.x = element_line(color = c("grey92", NA,
  ↪ "grey92", NA, "grey92", "grey92")))
```

Probabilities are computed by integrating the probability density function over the interval of interest. The area under the curve from a to b is the probability that the random variable X takes a value between a and b :

$$P(a \leq X \leq b) = \int_a^b f(x)dx$$

The **cumulative distribution function**, CDF, sometimes called just the distribution function, $F(x)$, is defined as:

$$F(x) = P(X \leq x) = \int_{-\infty}^x f(t)dt$$

The cumulative distribution function is increasing from 0 to 1.

```
plot(ggplot(df.nwt, aes(x=w, y=f)) + geom_line() + theme_bw() +
  ↪ xlab("x") + ylab("f(x)") + geom_area(data=df.nwt %>%
  ↪ filter(w<4.0)) + annotate("label",label=sprintf("P(X<4.0) =
  ↪ F(4.0) = %.2f", pnorm(4,3.5,0.5)), x=2.7, y=0.4, hjust=0))
df.nwt$F <- pnorm(df.nwt$w, 3.5, 0.5)
plot(ggplot(df.nwt, aes(x=w, y=F)) + geom_line() + xlab("x") +
  ↪ ylab("F(x)") + theme_bw() + geom_point(data=data.frame(w=4,
  ↪ F=pnorm(4,3.5,.5))) +
  ↪ annotate("label",label=sprintf("F(4.0)=%.2f",
  ↪ pnorm(4,3.5,.5)), x=4, y=.84, hjust=-0.2))##+
  ↪ ggtitle("Cumulative distribution function")
```

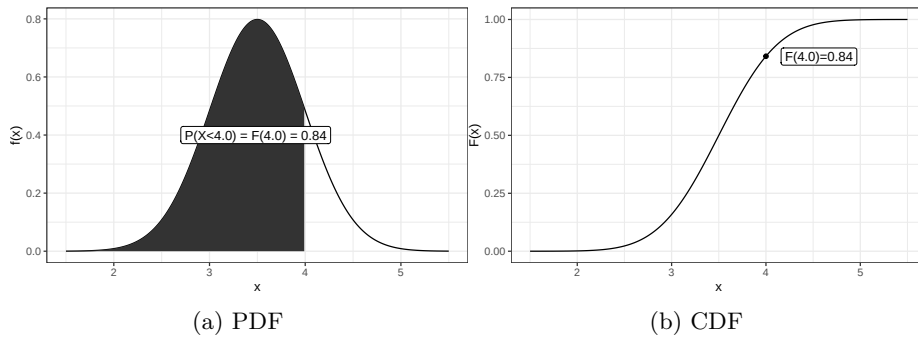


Figure 6.8: Probability density function (PDF) and cumulative distribution function (CDF).

$$P(X \leq x) = F(x)$$

As we know that the total probability (over all x) is 1, we can conclude that

$$P(X > x) = 1 - F(x)$$

and thus

$$P(a \leq X \leq b) = F(b) - F(a)$$

Note that $P(a \leq X \leq b) = P(a < X < b) = P(a < X \leq b) = P(a \leq X < b)$, as the probability of a single value is zero.

Parametric continuous distributions

Two important properties of a distribution are its expected value, μ , which describes the distribution's location, and its variance, σ^2 , which describes the spread.

The expected value, or population mean, is defined as;

$$E[X] = \mu = \int_{-\infty}^{\infty} xf(x)dx$$

We will learn more about the expected value and how to estimate a population mean from a sample later in the course.

The population variance is defined as the expected value of the squared distance from the population mean;

$$\sigma^2 = E[(X - \mu)^2] = \int_{-\infty}^{\infty} (x - \mu)^2 f(x)dx$$

These definitions are similar to the definitions of expected value and variance for discrete random variables, but with summation replaced by integration.

The square root of the variance is the standard deviation, σ .

Normal distribution

The normal distribution (sometimes referred to as the Gaussian distribution) is a common bell-shaped probability distribution. Many continuous random variables can be described by the normal distribution or be approximated by the normal distribution. The normal distribution is especially important because many statistical methods are based on it directly, and many other distributions can be approximated by it.

The normal probability density function

$$f(x) = \frac{1}{\sqrt{2\pi}\sigma} e^{-\frac{1}{2}\left(\frac{x-\mu}{\sigma}\right)^2}$$

describes the distribution of a normal random variable, X , with expected value μ and standard deviation σ , e and π are two common mathematical constants, $e \approx 2.71828$ and $\pi \approx 3.14159$.

In short, we write $X \sim N(\mu, \sigma^2)$ to denote that X is normally distributed with expected value μ and variance σ^2 (and standard deviation σ).

```
#ggplot(pop.FN, aes(x=Bodyweight)) + geom_histogram(binwidth=1,
  ↪ aes(y=stat(density)), color="white") + theme_bw() +
  ↪ geom_line(data=den.FN, aes(x=x, y=nfx), color="red")
x <- seq(-3.5, 3.5, .1)
dN <- data.frame(x=x, fx=dnorm(x))
plot(ggplot(dN, aes(x=x, y=fx)) + geom_line() +
  ↪ scale_x_continuous(breaks=-3:3,
  ↪ labels=c(expression(mu-3*sigma),expression(mu-2*sigma),
  ↪ expression(mu-1*sigma), expression(mu), expression(mu+sigma),
  ↪ expression(mu + 2*sigma), expression(mu + 3*sigma))) +
  ↪ xlab("") + ylab("f(x)") + theme_bw())
```

The normal distribution is symmetric around μ , and its density $f(x) \rightarrow 0$ as $x \rightarrow \infty$ and as $x \rightarrow -\infty$.

As $f(x)$ is known, the cumulative distribution function $F(x) = \int_{-\infty}^x f(x)dx$ can be computed.

```
x <- seq(-3.5, 3.5, .1)
dN <- data.frame(x=x, fx=dnorm(x))
dN <- dN |> mutate(Fx = pnorm(x))
ggplot(dN, aes(x=x, y=fx)) + geom_line() +
  ↪ scale_x_continuous(breaks=-3:3,
  ↪ labels=c(expression(mu-3*sigma),expression(mu-2*sigma),
  ↪ expression(mu-1*sigma), expression(mu), expression(mu+sigma),
  ↪ expression(mu + 2*sigma), expression(mu + 3*sigma))) +
  ↪ xlab("") + ylab("f(x)") + theme_bw() + ggtitle("Probability
  ↪ density function")
ggplot(dN, aes(x=x, y=Fx)) + geom_line() +
  ↪ scale_x_continuous(breaks=-3:3,
  ↪ labels=c(expression(mu-3*sigma),expression(mu-2*sigma),
  ↪ expression(mu-1*sigma), expression(mu), expression(mu+sigma),
  ↪ expression(mu + 2*sigma), expression(mu + 3*sigma))) +
  ↪ xlab("") + ylab("F(x)") + theme_bw() + ggtitle("Cumulative
  ↪ distribution function")
```

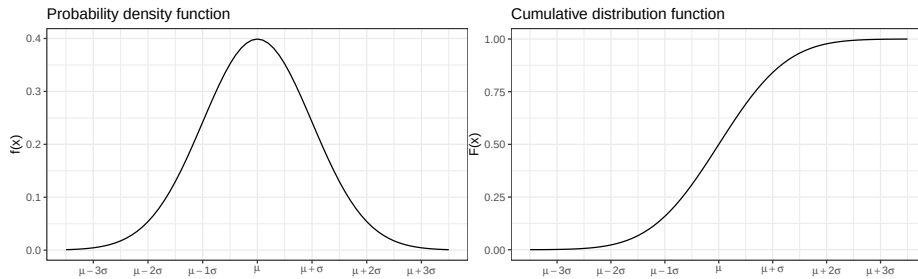


Figure 6.9: Normal probability density function and cumulative distribution functions.

Using transformation rules we can define Z , a random variable that is standard normally distributed (mean 0 and standard deviation 1).

$$Z = \frac{X - \mu}{\sigma}, Z \sim N(0, 1)$$

Standardization transforms a normal random variable X to a standard normal random variable Z , which makes probability calculations easy using tables and software. The cumulative standard normal distribution is denoted $F(z)$ and can be computed in R using the function `pnorm(z)`.

```
w<-seq(-3.5,3.5,.01)
df.nwt <- data.frame(w=w, f=dnorm(w))
plot(ggplot(df.nwt, aes(x=w, y=f)) + geom_line() + theme_bw() +
  ↪ xlab("x") + ylab("f(x)") + geom_area(data=df.nwt %>%
  ↪ filter(w<0.7)) + annotate("label",label=sprintf("P(Z \\leq z)
  ↪ = F(z)", x=-1.0, y=0.1, hjust=0) +
  ↪ scale_x_continuous(breaks=c(-2,0,+.7,2),
  ↪ labels=c("-2","0","z","2")) + theme(panel.grid.minor =
  ↪ element_blank(), panel.grid.major.x = element_line(color =
  ↪ c("grey92", "grey92", NA, "grey92"))))
```

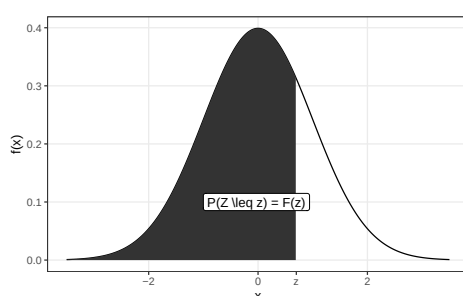


Figure 6.11: The shaded area under the curve is the tabulated value $P(Z \leq z) = F(z)$.

```
z <- sapply(seq(-3.4, -3.49, -0.01), function(x) seq(x,0,0.1))
z <- -z[nrow(z):1,]
z <- sapply(seq(0, 0.09, 0.01), function(x) seq(x,3.49,0.1))
zscore.df <- apply(pnorm(z), 2, function(x) sprintf("%.4f",x))
row.names(zscore.df) <- sprintf("%.1f", z[,1])
colnames(zscore.df) <- seq(0,0.09,0.01)
kable(zscore.df) %>% column_spec(1, bold=TRUE) |> kable_styling()
```

Table 6.3: Cumulative distribution function for the standard normal distribution. The table gives $F(z) = P(Z \leq z)$ for standard normal Z .

	0	0.01	0.02	0.03	0.04	0.05	0.06	0.07	0.08	0.09
0.0	0.5000	0.5040	0.5080	0.5120	0.5160	0.5199	0.5239	0.5279	0.5319	0.5359
0.1	0.5398	0.5438	0.5478	0.5517	0.5557	0.5596	0.5636	0.5675	0.5714	0.5753
0.2	0.5793	0.5832	0.5871	0.5910	0.5948	0.5987	0.6026	0.6064	0.6103	0.6141
0.3	0.6179	0.6217	0.6255	0.6293	0.6331	0.6368	0.6406	0.6443	0.6480	0.6517
0.4	0.6554	0.6591	0.6628	0.6664	0.6700	0.6736	0.6772	0.6808	0.6844	0.6879
0.5	0.6915	0.6950	0.6985	0.7019	0.7054	0.7088	0.7123	0.7157	0.7190	0.7224
0.6	0.7257	0.7291	0.7324	0.7357	0.7389	0.7422	0.7454	0.7486	0.7517	0.7549
0.7	0.7580	0.7611	0.7642	0.7673	0.7704	0.7734	0.7764	0.7794	0.7823	0.7852
0.8	0.7881	0.7910	0.7939	0.7967	0.7995	0.8023	0.8051	0.8078	0.8106	0.8133
0.9	0.8159	0.8186	0.8212	0.8238	0.8264	0.8289	0.8315	0.8340	0.8365	0.8389
1.0	0.8413	0.8438	0.8461	0.8485	0.8508	0.8531	0.8554	0.8577	0.8599	0.8621

1.1	0.8643	0.8665	0.8686	0.8708	0.8729	0.8749	0.8770	0.8790	0.8810	0.8830
1.2	0.8849	0.8869	0.8888	0.8907	0.8925	0.8944	0.8962	0.8980	0.8997	0.9015
1.3	0.9032	0.9049	0.9066	0.9082	0.9099	0.9115	0.9131	0.9147	0.9162	0.9177
1.4	0.9192	0.9207	0.9222	0.9236	0.9251	0.9265	0.9279	0.9292	0.9306	0.9319
1.5	0.9332	0.9345	0.9357	0.9370	0.9382	0.9394	0.9406	0.9418	0.9429	0.9441
1.6	0.9452	0.9463	0.9474	0.9484	0.9495	0.9505	0.9515	0.9525	0.9535	0.9545
1.7	0.9554	0.9564	0.9573	0.9582	0.9591	0.9599	0.9608	0.9616	0.9625	0.9633
1.8	0.9641	0.9649	0.9656	0.9664	0.9671	0.9678	0.9686	0.9693	0.9699	0.9706
1.9	0.9713	0.9719	0.9726	0.9732	0.9738	0.9744	0.9750	0.9756	0.9761	0.9767
2.0	0.9772	0.9778	0.9783	0.9788	0.9793	0.9798	0.9803	0.9808	0.9812	0.9817
2.1	0.9821	0.9826	0.9830	0.9834	0.9838	0.9842	0.9846	0.9850	0.9854	0.9857
2.2	0.9861	0.9864	0.9868	0.9871	0.9875	0.9878	0.9881	0.9884	0.9887	0.9890
2.3	0.9893	0.9896	0.9898	0.9901	0.9904	0.9906	0.9909	0.9911	0.9913	0.9916
2.4	0.9918	0.9920	0.9922	0.9925	0.9927	0.9929	0.9931	0.9932	0.9934	0.9936
2.5	0.9938	0.9940	0.9941	0.9943	0.9945	0.9946	0.9948	0.9949	0.9951	0.9952
2.6	0.9953	0.9955	0.9956	0.9957	0.9959	0.9960	0.9961	0.9962	0.9963	0.9964
2.7	0.9965	0.9966	0.9967	0.9968	0.9969	0.9970	0.9971	0.9972	0.9973	0.9974
2.8	0.9974	0.9975	0.9976	0.9977	0.9977	0.9978	0.9979	0.9979	0.9980	0.9981
2.9	0.9981	0.9982	0.9982	0.9983	0.9984	0.9984	0.9985	0.9985	0.9986	0.9986
3.0	0.9987	0.9987	0.9987	0.9988	0.9988	0.9989	0.9989	0.9989	0.9990	0.9990
3.1	0.9990	0.9991	0.9991	0.9991	0.9992	0.9992	0.9992	0.9992	0.9993	0.9993
3.2	0.9993	0.9993	0.9994	0.9994	0.9994	0.9994	0.9994	0.9995	0.9995	0.9995
3.3	0.9995	0.9995	0.9995	0.9996	0.9996	0.9996	0.9996	0.9996	0.9996	0.9997
3.4	0.9997	0.9997	0.9997	0.9997	0.9997	0.9997	0.9997	0.9997	0.9997	0.9998

Some values of particular interest:

$$F(1.64) = 0.95$$

$$F(1.96) = 0.975$$

Because the normal distribution is symmetric,

$$F(-z) = 1 - F(z).$$

$$F(-1.64) = 0.05$$

$$F(-1.96) = 0.025$$

$$P(-1.96 < Z < 1.96) = 0.95$$

Sum of two normal random variables

If $X \sim N(\mu_1, \sigma_1^2)$ and $Y \sim N(\mu_2, \sigma_2^2)$ are two independent normal random variables, then their sum is also a random variable:

$$X + Y \sim N(\mu_1 + \mu_2, \sigma_1^2 + \sigma_2^2)$$

and

$$X - Y \sim N(\mu_1 - \mu_2, \sigma_1^2 + \sigma_2^2)$$

The standard deviation of the sum or difference of two independent normal random variables is thus $\sqrt{\sigma_1^2 + \sigma_2^2}$.

This can be extended to the case with n independent and identically distributed random variables X_i ($i = 1 \dots n$). If all X_i are normally distributed with mean μ and standard deviation σ ,

$$X_i \sim N(\mu, \sigma^2)$$

then

$$\sum_{i=1}^n X_i \sim N(n\mu, n\sigma^2)$$

and therefore

$$\bar{X} = \frac{1}{n} \sum_{i=1}^n X_i \sim N\left(\mu, \frac{\sigma^2}{n}\right).$$

Central limit theorem, CLT

The **central limit theorem** states that the sum or mean of n independent and identically distributed random variables is approximately normally distributed, if n is large enough. The mean and variance of the sum are $n\mu$ and $n\sigma^2$, respectively.

As a result of the central limit theorem, sample means and sample proportions are often approximately normally distributed, when the sample is large enough. A rule of thumb is that the sample size $n > 30$ is large enough, but this of course depends on the distribution of the individual random variables.

Example 6.5 (A skewed distribution). A left skewed distribution has a heavier left tail than right tail. An example might be age at death of natural causes, as few individuals die of natural causes at young ages.

```
miceage <- 2*(2-rgamma(10000, 3, 9))
data.frame(age=miceage) |> ggplot(aes(x=age)) + geom_density() +
  ↪ theme_bw()
```

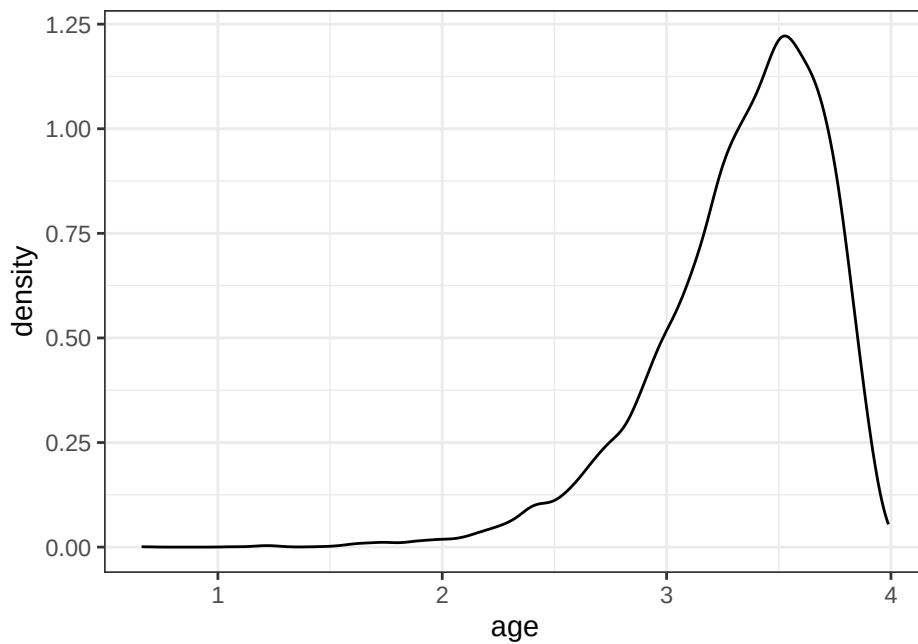


Figure 6.12: A left skewed distribution. Can for example show the distribution of age of a mouse who died of natural causes.

Randomly sample 3, 5, 10, 15, 20, 30 values and compute the mean value, m . Repeat many times to get the distribution of mean values.

```
rs <- data.frame(n = c(3,5,10,15,20,30)) |> group_by(n) |>
  ↪ reframe(m=replicate(10000, mean(sample(miceage, n))),
  ↪ .groups="drop")
rs |> ggplot(aes(x=m, fill=factor(n))) + geom_histogram(bins=50)
  ↪ + theme_bw() + facet_wrap(~factor(n))+
  ↪ theme(legend.position="none")
```

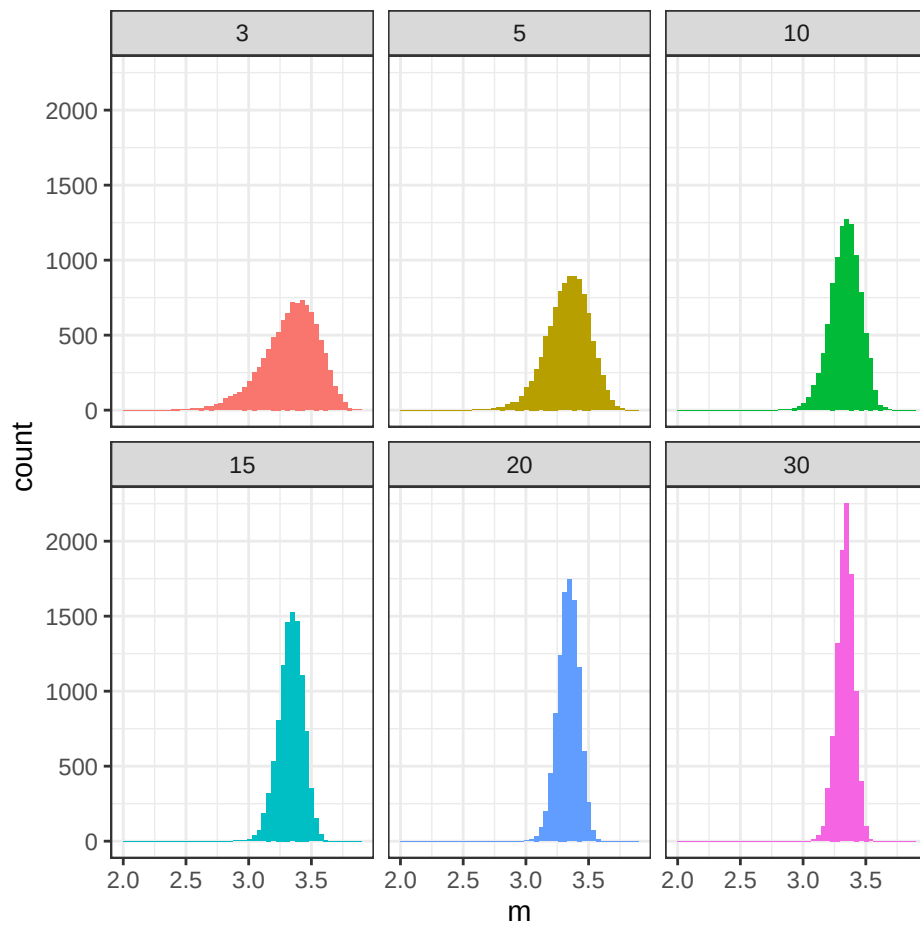


Figure 6.13: Distribution of sample means, where the means are computed based on random samples of sizes 3, 5, 10, 15, 20 and 30, respectively.

Note, mean is just the sum divided by the number of samples n .

As the sample size increases, the distribution of the sample mean becomes more symmetric and closer to normal.

In addition to the normal distribution, three other continuous distributions are especially important in statistical inference: the χ^2 , F, and t distributions

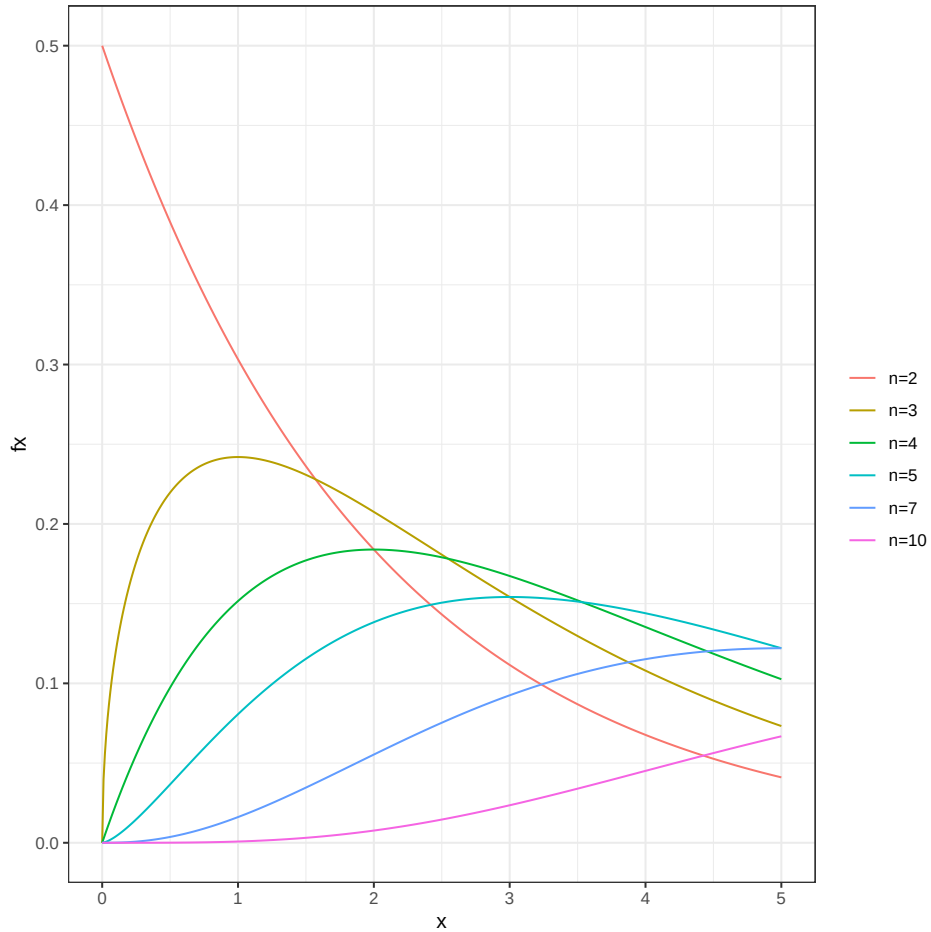
Chi-square distribution

The random variable $Y = \sum_{i=1}^n X_i^2$ is χ^2 distributed with n degrees of freedom, if X_i are independent identically distributed random variables $X_i \in N(0, 1)$.

In short $Y \in \chi^2(n)$.

```
x <- seq(-0.5, .01)
dX <- data.frame(x=x, n=rep(c(2, 3, 4, 5, 7, 10), each=length(x)))
  ↪ %>%
```

```
mutate(fx=dchisq(x, df=n))
#dT <- data.frame(x=x, fx=dt(x, df=n-1))
ggplot(dX, aes(x=x, y=fx, color=factor(paste0("n=",n),
↪ levels=paste0("n=",sort(unique(n)))))) + geom_line() +
↪ theme_bw() + scale_color_discrete("") + ylim(c(0,0.5))
```

Figure 6.14: The χ^2 -distribution.

Example 6.6. The sample variance $S^2 = \frac{1}{n-1} \sum_{i=1}^n (X_i - \bar{X})^2$ is such that $\frac{(n-1)S^2}{\sigma^2}$ is χ^2 distributed with $n - 1$ degrees of freedom.

F-distribution

If $U \sim \chi^2(n_1)$ and $V \sim \chi^2(n_2)$ are independent, then the ratio

$$\frac{U/n_1}{V/n_2} \sim F(n_1, n_2)$$

is F-distributed with n_1 and n_2 degrees of freedom.


```

x <- seq(-0,5, .01)
dX <- data.frame(x=x, n1=rep(c(2,4,10), each=length(x)),
  ↪ n2=rep(c(2,4,10), each=length(x)*3)) %>%
  mutate(fx=df(x, df1=n1, df2=n2))
ggplot(dX, aes(x=x, y=fx, color=factor(sprintf("n1=%i, n2=%i",n1,
  ↪ n2), levels=sprintf("n1=%i, n2=%i",rep(sort(unique(n1)),
  ↪ each=3), sort(unique(n2)))))) + geom_line() + theme_bw() +
  ↪ scale_color_discrete("") + ylim(c(0,0.8))

```

Warning: Removed 47 rows containing missing values or values outside the scale range (`geom\_line()`).

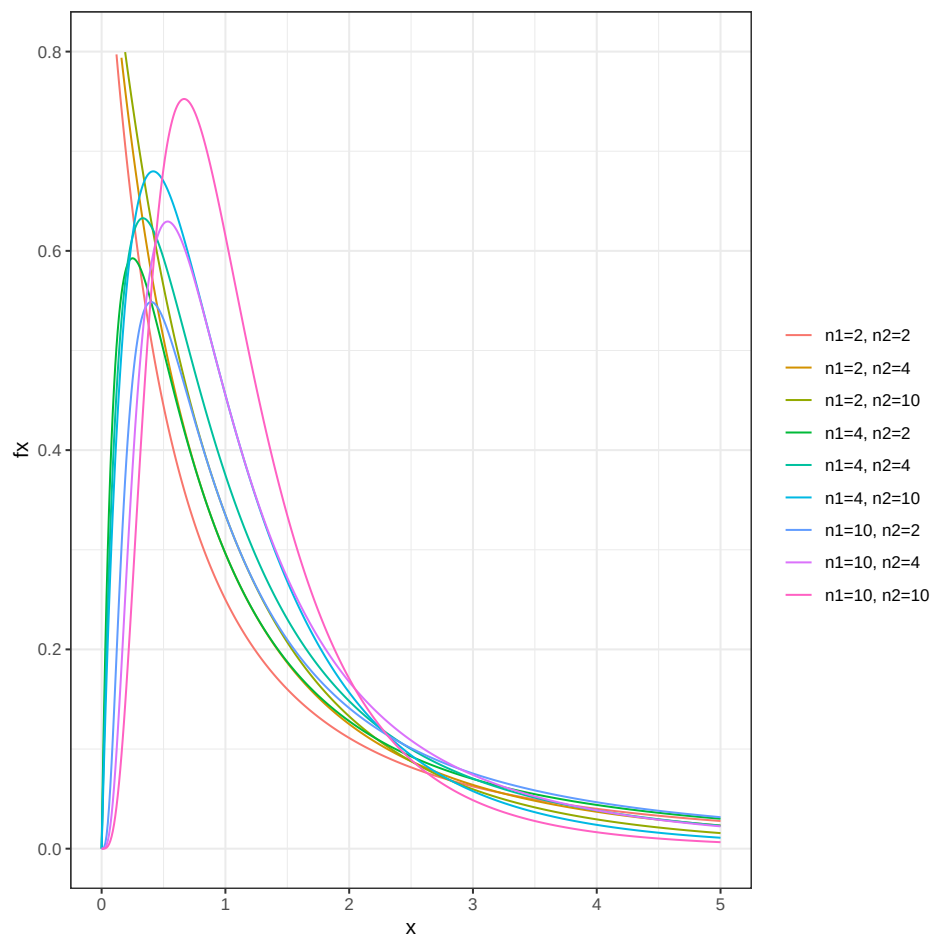


Figure 6.15: The F-distribution

Example 6.7. The ratio of two sample variances is F-distributed, if the two samples are independent and normally distributed.

t-distribution

If $Z \sim N(0, 1)$ and $U \sim \chi^2(v)$ are independent, then the random variable

$$T = \frac{Z}{\sqrt{U/v}} \sim t(v)$$

is t-distributed with v degrees of freedom.

A common example is based on the sample mean, $\bar{X}$, and sample variance, S^2 , of a sample of size n from a normal distribution. The standardized sample mean

$$T = \frac{\bar{X} - \mu}{S/\sqrt{n}} \sim t(n-1)$$

is t-distributed with $n - 1$ degrees of freedom.

```
x <- seq(-3.5, 3.5, .01)
dT <- data.frame(x=x, n=rep(c(2, 3, 5, 7, 10, 15, 20, 30),
  ↳ each=length(x))) %>%
  mutate(fx=dt(x, df=n-1))
#dT <- data.frame(x=x, fx=dt(x, df=n-1))
ggplot(dT, aes(x=x, y=fx, color=factor(paste0("n=", n),
  ↳ levels=paste0("n=", sort(unique(n)))))) + geom_line() +
  ↳ theme_bw() + scale_color_discrete("")
```

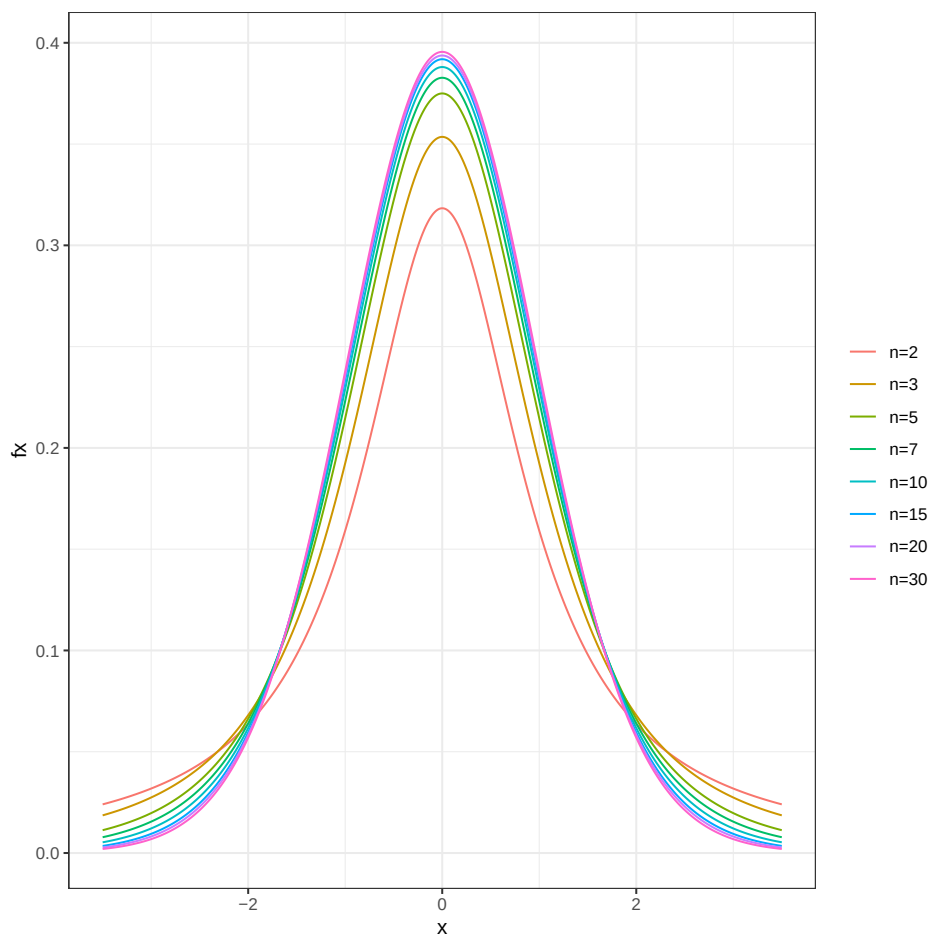


Figure 6.16: The t-distribution.

Distributions in R

Probability density functions for the normal, t , χ^2 and F distributions can in R be computed using functions `dnorm`, `dt`, `dchisq`, and `df`, respectively.

Cumulative distribution functions can be computed using `pnorm`, `pt`, `pchisq` and `pf`.

Also, functions for computing an x such that $P(X < x) = q$, where q is a probability of interest are available using `qnorm`, `qt`, `qchisq` and `qf`.

As with distributions, R uses the general naming scheme `r<dist>`, `p<dist>`, `q<dist>` and `d<dist>` for random number generation, cumulative distribution function, quantile function and probability density function, respectively.

Part II

Statistical Inference

Learning outcomes

- to define null and alternative hypothesis
- to perform a hypothesis test using resampling
- to perform a t-test
- to understand and define sampling distribution and standard error
- to compute standard error of mean and proportions
- to compute confidence interval of mean and proportions using the normal approximation
- to compute confidence interval of mean using the t-distribution

Chapter 7

Introduction to hypothesis tests

Statistical inference is the process of drawing conclusions about properties of a population based on observations from a random sample.

A **hypothesis test** is a type of statistical inference used to evaluate if a hypothesis about a population is supported by the observations of a random sample (i.e by the data available).

Typically, the hypotheses that are tested are assumptions about properties of the population, such as proportion, mean, mean difference, or variance. For example, a hypothesis test can be used to assess if the population mean is greater than 0.

In summary, a hypothesis test involves defining a null and alternative hypothesis, selecting an appropriate test statistic and computing the observed value of this statistic. Finally, the probability of the observed value, or something more extreme, if the null hypothesis is true is computed. Based on this probability and a predefined significance level, the null hypothesis is either rejected or not rejected.

7.1 The null and alternative hypothesis

There are two hypotheses involved in a hypothesis test, the **null hypothesis** H_0 and the **alternative hypothesis** H_1 .

The null hypothesis is in general neutral; “no change”, “no difference between the groups”, “no association”. In general we want to show that H_0 is false.

The alternative hypothesis expresses what the researcher is interested in. Depending on the research question, the alternative hypothesis can be two-sided or one-sided. A two-sided alternative hypothesis is non-directional, such as “the treatment has an effect”, “there is a difference between the groups”, “there is an association”, while a one-sided alternative hypothesis is directional, such as “the treatment has a positive effect”.

7.2 To perform a hypothesis test

1. Define H_0 and H_1
2. Select an appropriate **significance level**, α
3. Select an appropriate test statistic, T , and compute the observed value, t_{obs}
4. Assume that H_0 is true and compute the **sampling distribution** of T under H_0 .
5. Compare the observed value, t_{obs} , with the computed sampling distribution under H_0 (the **null distribution**) and compute a **p-value**.
6. Based on the p-value, either reject or fail to reject H_0 .

The **sampling distribution** is the distribution of a sample statistic, such as a sample mean or sample proportion. The sampling distribution can be described theoretically or approximated by repeatedly drawing samples from a population.

A **null distribution** is the sampling distribution of a test statistic when the null hypothesis is true.

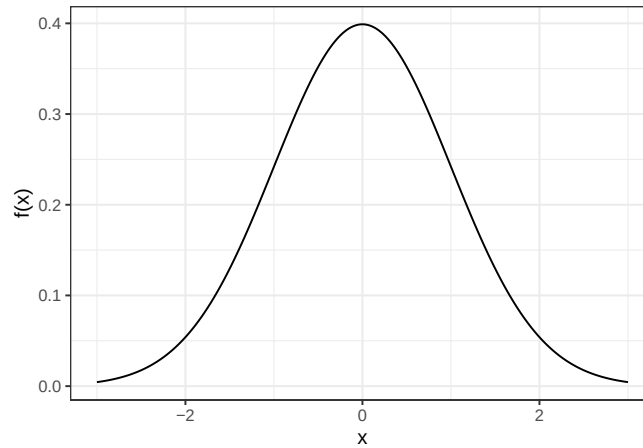


Figure 7.1: A null distribution

7.3 The p-value

The **p-value** is the probability of observing a value of the test statistic at least as extreme as the observed value, if H_0 is true.

The p-value is **not** the probability that H_0 is true.

7.4 Significance level and error types

A hypothesis test is used to draw inferences about a population based on a random sample. The conclusion drawn from a hypothesis test may of course be wrong. There are two types of errors:

Type I error: rejecting H_0 when it is true. This is also called a false positive or a false alarm. Examples: “The test says that you are covid-19 positive, when

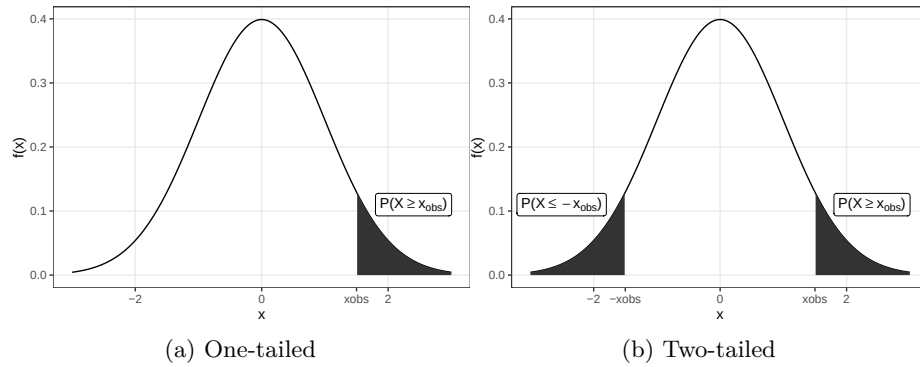


Figure 7.2: The p-value is the probability to observe x_{obs} or something more extreme, if the null hypothesis is true. The p-value is illustrated for a one-tailed test (left) and for a two-tailed test (right).

Table 7.1: Error types.

Decision	H is true	H is false
Fail to reject H_0	Correct decision	Type II error (miss)
Reject H_0	Type I error (false alarm)	Correct decision

you actually are not”, “*The test says that the drug has a positive effect on patient symptoms, but it actually has not*”.

Type II error: failing to reject H_0 when it is false. This is also called a false negative or a miss. Examples: “*The test says that you are covid-19 negative, when you actually have covid-19*”, “*The test says that the drug has no effect on patient symptoms, when it actually has*”.

The probabilities of type I and type II errors are denoted α and β , respectively.

$$\alpha = P(\text{type I error}) = P(\text{false alarm}) = P(\text{Reject } H_0 \mid H_0 \text{ is true})$$

$$\beta = P(\text{type II error}) = P(\text{miss}) = P(\text{Fail to reject } H_0 \mid H_1 \text{ is true})$$

The significance level, α , is the probability of a type I error, i.e., the probability of rejecting H_0 when it is true. In other words, it is the risk of false alarm, i.e., to say “*I have a hit*”, “*I found a difference*”, when the null hypothesis (“*there is no difference*”) is true.

The risk of false alarm is controlled by setting the significance level to a desired value. We usually want to keep the risk of false alarm (type I error) low, but making α smaller increases the risk of missing a true effect (type II error), unless the sample size is increased.

The significance level should be set before the hypothesis test is performed. Common values to use are $\alpha = 0.05$ or 0.01 .

If the p-value is above the significance level, $p > \alpha$, H_0 is not rejected.

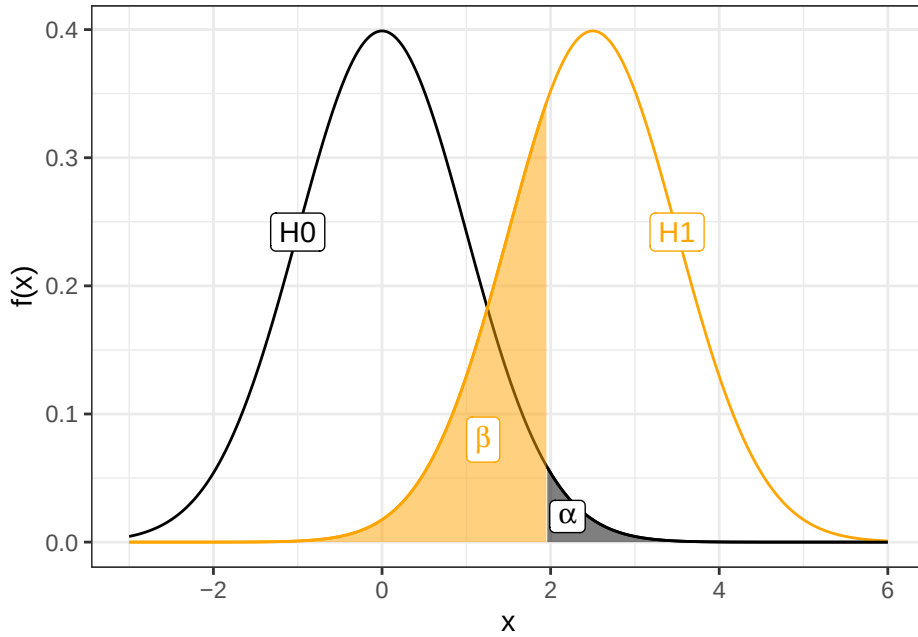


Figure 7.3: The probability density functions under H_0 and H_1 , respectively. The probability of type I error (α) and type II error (β) are indicated.

If the p-value is below the significance level, $p \leq \alpha$, H_0 is rejected.

Another property of a statistical test is the **statistical power**, defined as

$$\text{power} = 1 - \beta = P(\text{Reject } H_0 \mid H_1 \text{ is true}).$$

High power means that the test is likely to detect a true effect if it exists. The power of a test can be increased by increasing the sample size or by increasing the significance level, α .

Chapter 8

Hypothesis testing using resampling

The null distribution, that is, the sampling distribution of a test statistic under the null hypothesis, is sometimes known exactly or can be derived under a set of assumptions. When the null distribution is not known, it can often be estimated using resampling.

In this chapter, we consider three resampling-based approaches to hypothesis testing: simulation from a known null model, bootstrap, and permutation.

8.1 Resampling from a known population under H_0

If it is possible to simulate sampling under the null hypothesis, resampling can be used to estimate the null distribution, just like we did in the previous chapter when estimating probabilities and probability distributions using resampling.

This is done by setting up a model under the null hypothesis (e.g. using an urn model) and drawing random samples from this model repeatedly.

Example 8.1 (Proportions, pollen allergy). Let's assume we know that the proportion of people with pollen allergy in Sweden is 0.3. We suspect that the number of people with pollen allergy has increased in Uppsala in the last couple of years and want to investigate this.

Observe 100 people from Uppsala, 42 of these were allergic to pollen. Is there a reason to believe that the proportion of people with pollen allergy in Uppsala $\pi > 0.3$?

Null and alternative hypotheses

H_0 : The proportion of people with pollen allergy in Uppsala is the same as in Sweden as a whole.

H_1 : The proportion of people with pollen allergy in Uppsala is greater than in Sweden as a whole.

or expressed differently:

$$H_0 : \pi = \pi_0$$

$$H_1 : \pi > \pi_0$$

where π is the unknown proportion of pollen allergy in the Uppsala population and $\pi_0 = 0.3$ is the proportion of pollen allergy in Sweden.

Significance level

Here we let $\alpha = 0.05$.

Test statistic

Here we are interested in the proportion of people with pollen allergy in Uppsala. An appropriate test statistic could be the number of people with pollen allergy in a sample of size $n = 100$, X . As an alternative we can use the proportion of people with pollen allergy in a sample of size n ,

$$P = \frac{X}{n}$$

Let's use P as our test statistic and compute the observed value, p_{obs} . In our sample of 100 people from Uppsala, the proportion of people with pollen allergy is $p_{obs} = 42/100 = 0.42$.

Null distribution

The sampling distribution of P under H_0 (i.e. when the null hypothesis is true) is what we call the null distribution.

H_0 states that $\pi = 0.3$. We can model this using an urn model as follows;

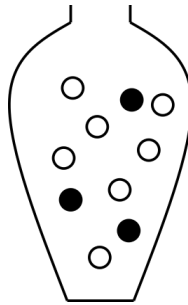


Figure 8.1: An urn model of the null hypothesis $\pi = 0.3$. The black balls represent allergic and the white balls non-allergic.

Using this model, we can simulate taking a sample of size 100 many times.

```
## Urn
urn <- rep(c(0, 1), c(7, 3))
## Sample 100 times with replacement
x <- sample(urn, 100, replace=TRUE)
## Compute proportion of samples that are allergic (1)
mean(x)

[1] 0.33

## Set the seed to get the same result if we redo the analysis
set.seed(13)
## Repeat drawing sample of size 100 and computing proportion
  ↪ allergic 100000 times
p <- replicate(100000, mean(sample(rep(c(0, 1), c(7, 3)), 100,
  ↪ replace=TRUE)))
```

Plot the distribution

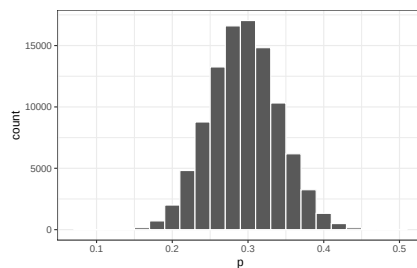


Figure 8.2: Estimated null distribution of the sample proportion under the null hypothesis.

Compute p-value

Compare the observed value, $p_{obs} = 0.42$ to the null distribution.

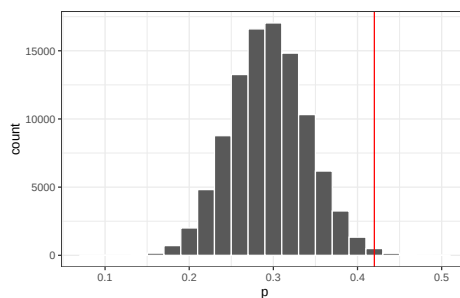


Figure 8.3: Estimated null distribution. The observed value is marked by a red vertical line.

The p-value is the probability of getting the observed value or higher, if the null hypothesis is true.

Use the null distribution to calculate the p-value, $P(P \geq 0.42 \mid H_0)$.

```
## How many times
sum(p >= 0.42)
```

```
[1] 703
```

```
## p-value
pval <- mean(p >= 0.42)
```

$$p = P(P \geq 0.42 \mid H_0) = 0.00703$$

Reject or fail to reject H_0 ?

As the p-value is $< \alpha = 0.05$ the null hypothesis is rejected. This means that we can conclude that there is reason to believe that the proportion of people with pollen allergy in Uppsala is greater than 0.3.

8.2 Bootstrap resampling

Bootstrap is a resampling technique that resamples with replacement from the available data (random sample) to construct new simulated samples. These bootstrap samples can be used to estimate the sampling distribution of a statistic. The estimated distribution is commonly used to estimate properties such as standard error and construct interval estimates, but can also be used to perform hypothesis testing.

We will get back to constructing bootstrap interval estimates in Section 10.3.1. Here we will focus on hypothesis testing using bootstrap.

To use the bootstrap for hypothesis testing, we first transform the observed sample so that it satisfies the null hypothesis, and then resample with replacement from this modified sample.

For a one sample test of mean, the hypotheses might be;

$$H_0 : \mu = \mu_0$$

$$H_1 : \mu > \mu_0$$

Let our observed sample of size n be $x_1, x_2, \dots, x_n$. For simplicity, we use the sample mean, $\bar{X}$, as the test statistic, whose observed value is $m_{obs} = \bar{x}$.

To construct a sampling distribution when H_0 is true we would need a modified sample with mean μ_0 , which can be constructed by computing $x'_i = x_i - m_{obs} + \mu_0$. A null distribution can then be estimated using bootstrapping as follows;

1. Take a random sample of size n with replacement from $x'_1, x'_2, \dots, x'_n$. Note that the same value might be selected more than once as we sample with replacement. Denote our bootstrap resample x^* .
2. Compute the mean value of the bootstrap resample x^* and denote the mean m^* .

Repeating 1 and 2 many times, for example $B = 1000$, yields a bootstrap estimate of the null distribution of sample means.

The p-value can be estimated as the number of times $m^* \geq m_{obs}$ divided by B

$$p = \frac{\sum_{k=1}^B I(m_k^* \geq m_{obs})}{B},$$

where $I(\cdot)$ is the indicator function.

Example 8.2 (Mean value, bootstrap). Men with a waist circumference greater than 94 cm have been shown to have an increased risk of cardiovascular disease. Based on the following waist circumferences of 12 diabetic men, is there reason to believe that the mean waist circumference of diabetic men is greater than 94 cm?

97, 97, 89, 84, 124, 107, 99, 122, 102, 114, 109, 84

```
x <- c(97, 97, 89, 84, 124, 107, 99, 122, 102, 114, 109, 84)
```

Null and alternative hypotheses

$$H_0 : \mu = 94$$

$$H_1 : \mu > 94$$

Significance level

Here we let $\alpha = 0.05$.

Test statistic

Here we will use the sample mean, $\bar{X}$, as test statistic.

The observed value of the test statistic is the mean of the observed sample values, $m_{obs} = \bar{x}$.

$$m_{obs} = 102.3$$

```
mobs <- mean(x)
```

Null distribution

Assume that H_0 is true and compute the **sampling distribution** of the sample mean $\bar{X}$. We will estimate the null distribution using bootstrap resampling.

First, modify the observed sample so that its mean is 94 cm, in accordance with the null hypothesis. This is done by subtracting the observed sample mean, m_{obs} , and adding 94 to the observations.

```
xnull <- x - mobs + 94
```

Resample with replacement to get a bootstrap sample of size 12 (same as the original sample) and compute the mean.

```
mean(sample(xnull, replace=TRUE))
```

```
[1] 91.33
```

Repeat 1000 times.

```
mnull <- replicate(1000, mean(sample(xnull, replace=TRUE)))
```

Plot the null distribution in a histogram and compare with the observed value m_{obs} .

```
ggplot(data.frame(m=mnull), aes(x=m)) +  
  geom_histogram(bins=25, color="white") +  
  theme_bw() +  
  geom_vline(xintercept=mobs, color="red")
```

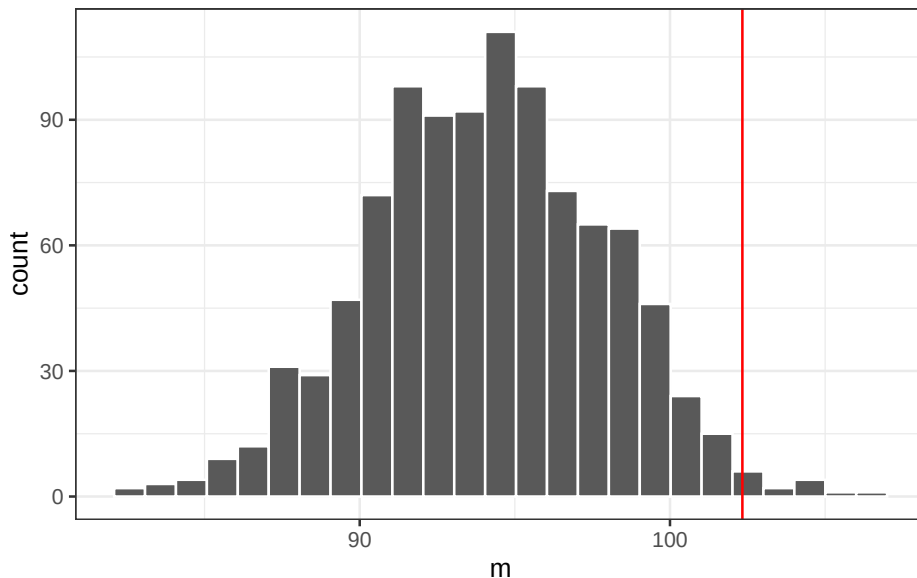


Figure 8.4: Null distribution of mean waist circumferences in cm. Observed mean is marked with a red vertical line.

8.2.0.1 Compute p-value

The p-value is the probability of seeing the observed value or something more extreme.

```
p <- mean(mnull >= mobs)
```

The computed p-value is 0.013. As the computed p-value $< \alpha = 0.05$ the null hypothesis is rejected and we conclude that there is reason to believe that diabetic men have a mean waist circumference greater than 94 cm.

8.2.1 Permutation

Another resampling technique is permutation. A permutation test answers the question whether an observed effect could be explained simply by the random assignment of experimental units to groups under the null hypothesis. Permutation tests are commonly used in clinical settings where a treatment group is compared to a control group.

A permutation test involves comparing two (or more) groups, usually by computing the difference of a property of the two groups, e.g. a mean difference, where the null hypothesis is that there is no difference between the groups with respect to the variable of interest.

In a permutation test, the null distribution is computed by repeatedly permuting (rearranging) the observations so that they are randomly assigned to the two groups. This can be done either by keeping the group labels fixed and randomly reallocating the observed values between the groups, or equivalently by keeping the observed values fixed and randomly permuting the group labels. After each permutation, the test statistic (e.g. mean difference) is recomputed. Repeating this many times gives the null distribution of the test statistic under the null hypothesis.

The null distribution is a distribution of test statistics that could arise purely because of the random assignment to groups, assuming the null hypothesis is true. If the observed effect is much more extreme than what could be explained just by the randomization, we can conclude that there is a difference between the groups (the treatment has an effect).

Example 8.3 (Does a high-fat diet lead to increased body weight?). The effect of high-fat diet on the body weight of mice is studied in an experiment. The study setup is as follows:

1. 24 female mice are ordered from a lab.
2. Randomly, 12 of the 24 mice are assigned to receive high-fat diet, the remaining 12 are controls (ordinary diet).
3. Body weight is measured after one week.

The observed values, mouse weights in grams, are summarized below;

high-fat	25	30	23	18	31	24	39	26	36	29	23	32
ordinary	27	25	22	23	25	37	24	26	21	26	30	24

Null and alternative hypotheses

$$H_0 : \mu_{HF} = \mu_N \iff \mu_{HF} - \mu_N = 0$$

$$H_1 : \mu_{HF} > \mu_N \iff \mu_{HF} - \mu_N > 0$$

where μ_{HF} is the (unknown) mean body weight of the high-fat mouse population and μ_N is the mean body-weight of the control mouse population.

Studied population: Female mice that can be ordered from a lab.

Test statistic

Here we are interested in the mean difference between high-fat and control mice and an appropriate test statistic can be the mean difference, $D = \bar{X}_{HF} - \bar{X}_N$, where

- $\bar{X}_{HF}$ is a random variable describing the mean weight of 12 (randomly selected) mice on high-fat diet. $E[\bar{X}_{HF}] = E[X_{HF}] = \mu_{HF}$
- $\bar{X}_N$ is a random variable describing the mean weight of 12 (randomly selected) mice on ordinary diet. $E[\bar{X}_N] = E[X_N] = \mu_N$

The mean difference $D = \bar{X}_{HF} - \bar{X}_N$ is also a random variable.

Observed values;

```
## 12 HF mice
xHF <- c(25, 30, 23, 18, 31, 24, 39, 26, 36, 29, 23, 32)
## 12 control mice
xN <- c(27, 25, 22, 23, 25, 37, 24, 26, 21, 26, 30, 24)

## Compute mean body weights of the two samples
mHF <- mean(xHF)
mN <- mean(xN)
## Compute mean difference
dobs <- mHF - mN
```

Mean weight of sample control mice (ordinary diet): $\bar{x}_N = 25.83$

Mean weight of sample mice on high-fat diet: $\bar{x}_{HF} = 28.00$

Difference in sample mean weights: $d_{obs} = \bar{x}_{HF} - \bar{x}_N = 2.1667$

Null distribution

If high-fat diet has no effect, i.e. if H_0 was true, the result would be as if all mice were given the same diet. What can we expect if all mice are fed with the same type of food?

This can be done using permutation.

The 24 mice were initially from the same population, depending on how the mice are randomly assigned to the high-fat and normal group, the mean weights would differ, even if the two groups were treated the same.

Assume H_0 is true, i.e. assume all mice are exchangeable and

1. Randomly reassign 12 of the 24 mice to 'high-fat' and the remaining 12 to 'control'.
2. Compute difference in mean weights

If we repeat steps 1-2 many times, we get the null distribution of difference in mean weights.

```
## All 24 body weights in a vector
x <- c(xHF, xN)
## Mean difference
dobs <- mean(x[1:12]) - mean(x[13:24])
## Permute once
y <- sample(x)
## Compute mean difference
mean(y[1:12]) - mean(y[13:24])
```

```
[1] -3.5
```

```
## Repeat the above many times
dnull.perm <- replicate(n = 100000, {
  y <- sample(x)
  ## Mean difference
  mean(y[1:12]) - mean(y[13:24])
})
```

Plot the null distribution and the observed difference.

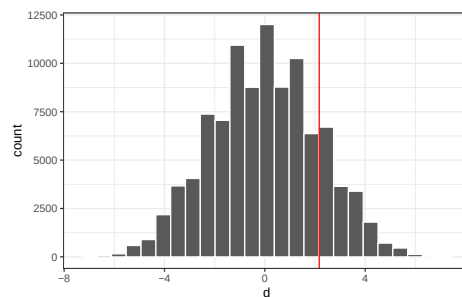


Figure 8.5: Null distribution of the mean difference D .

Compute p-value

What is the probability to get an at least as extreme mean difference as our observed value, d_{obs} , if H_0 was true?

```
## Compute the p-value
pval <- mean(dnull.perm >= dobs)
```

$$P(D \geq d_{obs} \mid H_0) = 0.169$$

Reject or fail to reject H_0 ?

As $0.169 > 0.05$, we fail to reject the null hypothesis. The data do not provide evidence that the high-fat diet increases body weight in mice.

Chapter 9

Parametric tests

In the previous chapter, we used resampling techniques to estimate sampling distributions for hypothesis testing. If the null distribution is already known, or can be derived based on a set of assumptions, resampling is not necessary.

Parametric tests are hypothesis tests in which the null distribution of the test statistic is derived from a probability model described by a small number of parameters, such as the mean and variance of a normal distribution.

We can follow the same steps as before to perform a hypothesis test:

1. Define H_0 and H_1
2. Select an appropriate significance level, α
3. Select an appropriate test statistic, T , and compute the observed value, t_{obs}
4. Assume that H_0 is true and derive the null distribution of the test statistic based on appropriate assumptions.
5. Compare the observed value, t_{obs} , with the null distribution and compute a p-value. The p-value is the probability of observing a value of the test statistic at least as extreme as the observed value, if H_0 is true.
6. Based on the p-value, either reject H_0 or fail to reject H_0 .

In this section we will present a few common situations in which the null distribution can be described parametrically.

9.1 Mean comparison

9.1.1 One sample

A one-sample test for a mean compares the mean of one sample to a prespecified value.

For example, we might know that the weight of a mouse on normal diet is normally distributed with mean 24.0 g and standard deviation 3 g and want to compare the weight of a sample of 12 mice on high-fat diet to the known mean value for mice on normal diet.

The hypotheses:

$$H_0 : \mu = \mu_0$$

$$H_1 : \mu \neq \mu_0$$

The alternative hypothesis, H_1 , above is for the two-sided hypothesis test. Other options are the one-sided H_1 ; $H_1 : \mu > \mu_0$ or $H_1 : \mu < \mu_0$.

If

$$X \sim N(\mu, \sigma^2)$$

(this could for example be the weight of a mouse on high-fat diet) then the sample mean

$$\bar{X} \sim N\left(\mu, \frac{\sigma^2}{n}\right).$$

If σ is known under the null hypothesis, then the test statistic

$$Z = \frac{\bar{X} - \mu_0}{\frac{\sigma}{\sqrt{n}}}$$

is normally distributed, $\sim N(0, 1)$.

In many situations the the population standard deviation is not known, but can instead be estimated from the sample. For large sample sizes, σ can be replaced by the sample standard deviation s and the test statistic will still be normally distributed according to the **central limit theorem**.

For small n and unknown σ , if the observations are drawn from a normal distribution, we can use the **one-sample t-test**, that uses the test statistic

$$T = \frac{\bar{X} - \mu_0}{\frac{s}{\sqrt{n}}},$$

which is t-distributed with $df = n - 1$ degrees of freedom.

Once we have an appropriate test statistic, T with known distribution, we can compute the observed value, t_{obs} and use the null distribution to compute the p-value, $P(|T| \geq |t_{obs}|)$.

In R, the functions **pnorm** and **pt** are useful for computing these probabilities. The **one-sample t-test** can also be performed using the function **t.test**, like this;

```
##The observed 12 mouse weights;
x <- c(25, 30, 23, 18, 31, 24, 39, 26, 36, 29, 23, 32)
## Under the null hypothesis mu=24
##Perform the t-test to investigate if the population mean
  differs from 24.
t.test(x, mu=24)
```


One Sample t-test

```

data:  x
t = 2.3153, df = 11, p-value = 0.04092
alternative hypothesis: true mean is not equal to 24
95 percent confidence interval:
 24.19742 31.80258
sample estimates:
mean of x
      28

```

9.1.2 Two samples

A two-sample test for a difference in means is used to determine if two population means are equal.

Two independent samples are collected (one from each population) and the means are compared. This test can for example be used to determine if a treatment group is different from a control group, in terms of the mean of a property of interest.

The null hypothesis is

$$H_0 : \mu_2 = \mu_1.$$

The alternative hypothesis can either be two-sided

$$H_1 : \mu_2 \neq \mu_1$$

or one-sided

$$H_1 : \mu_2 > \mu_1 \text{ or } H_1 : \mu_2 < \mu_1.$$

Assume observations from both populations are normally distributed;

$$\begin{aligned} X_1 &\sim N(\mu_1, \sigma_1^2) \\ X_2 &\sim N(\mu_2, \sigma_2^2) \end{aligned}$$

Then it follows that the sample means will also be normally distributed;

$$\begin{aligned} \bar{X}_1 &\sim N(\mu_1, \sigma_1^2/n_1) \\ \bar{X}_2 &\sim N(\mu_2, \sigma_2^2/n_2) \end{aligned}$$

The mean difference $D = \bar{X}_2 - \bar{X}_1$ is thus also normally distributed:

$$D = \bar{X}_2 - \bar{X}_1 \sim N\left(\mu_2 - \mu_1, \frac{\sigma_2^2}{n_2} + \frac{\sigma_1^2}{n_1}\right)$$

If H_0 is true:

$$D = \bar{X}_2 - \bar{X}_1 \sim N\left(0, \frac{\sigma_2^2}{n_2} + \frac{\sigma_1^2}{n_1}\right)$$

The test statistic:

$$Z = \frac{\bar{X}_2 - \bar{X}_1}{\sqrt{\frac{\sigma_2^2}{n_2} + \frac{\sigma_1^2}{n_1}}}$$

is standard normal, i.e. $Z \sim N(0, 1)$.

However, note that the test statistic requires the standard deviations σ_1 and σ_2 to be known.

What if the population standard deviations are not known?

If the sample sizes are large, we may replace the population standard deviations by the sample standard deviations and, according to the **central limit theorem**, assume that the test statistic is still normally distributed. The test statistic then becomes

$$Z = \frac{\bar{X}_2 - \bar{X}_1}{\sqrt{\frac{s_2^2}{n_2} + \frac{s_1^2}{n_1}}} \sim N(0, 1)$$

and proceed as before.

For small sample sizes, if the populations are approximately normal, t-based methods are used.

$$t = \frac{\bar{X}_2 - \bar{X}_1}{\sqrt{\frac{s_2^2}{n_2} + \frac{s_1^2}{n_1}}}$$

If it can be assumed that both X_1 and X_2 are normally distributed and have equal variances, Student's t-test can be used. For equal variances the pooled sample variance can be computed;

$$s_p^2 = \frac{(n_1 - 1)s_1^2 + (n_2 - 1)s_2^2}{n_1 + n_2 - 2}$$

and the test statistic

$$t = \frac{\bar{X}_2 - \bar{X}_1}{\sqrt{s_p^2\left(\frac{1}{n_1} + \frac{1}{n_2}\right)}}$$

is t-distributed with $n_1 + n_2 - 2$ degrees of freedom.

If equal variances cannot be assumed, Welch's t-test can be used. This test does not assume equal variances, and is the default in R's `t.test`. The test statistic is the same as above, but the degrees of freedom are approximated using the Welch-Satterthwaite formula.

The t-test is implemented in R, e.g. in the function `t.test` in the R-package `stats`, both Student's t-test for equal variances and Welch's t-test for unequal variances.

9.2 Proportions

9.2.1 One sample

Example 9.1 (Pollen allergy). Let's get back to the pollen example!

Assume that the proportion of pollen allergy in Sweden is known to be 0.3. Observe 100 people from Uppsala, 42 of these were allergic to pollen. Is there a reason to believe that the proportion who are allergic to pollen in Uppsala $\pi > 0.3$?

1. Null and alternative hypotheses

$$H_0 : \pi = \pi_0$$

$$H_1 : \pi > \pi_0,$$

where $\pi_0 = 0.30$ in this example. Other possible alternative hypotheses are $H_1 : \pi < \pi_0$ or $H_1 : \pi \neq \pi_0$, but in this particular example we are only interested in the alternative that $\pi > \pi_0$.

2. Significance level

Set the significance level to $\alpha = 0.05$.

3. Test statistic

Here, we will use X , the number of allergic persons in a random sample of size $n = 100$. The observed value is $x_{obs} = 42$.

4. Null distribution

X is binomially distributed, so there is no need to use resampling here, we can use the binomial distribution to answer the question.

5. p-value

The p-value is the probability of observing a value at least as extreme as the observed value, if H_0 is true. In this case, the p-value is the probability of x_{obs} or something higher,

$$P(X \geq 42) = 1 - P(X \leq 41) = [1 - \text{pbinom}(41, 100, 0.3)] = 0.007174$$

6. Can H_0 be rejected?

As $p < 0.05$ H_0 is rejected and we conclude that there is reason to believe that the proportion of people with pollen allergy in Uppsala is higher than 0.3.

This p-value can also be computed using the exact binomial test;

```
binom.test(42, 100, 0.3, alternative="greater")
```

Exact binomial test

```
data: 42 and 100
number of successes = 42, number of trials = 100, p-value = 0.007174
alternative hypothesis: true probability of success is greater than 0.3
95 percent confidence interval:
 0.3364797 1.0000000
sample estimates:
probability of success
          0.42
```

An alternative approach is to use the **central limit theorem** and use the normal approximation.

As a result of the central limit theorem, the distribution of number or proportion of allergic individuals in a sample of size n is approximately normal. At least if the sample is large enough. A common rule of thumb is that both $n\pi_0$ and $n(1 - \pi_0)$ should be at least 5.

Here, the sample size is 100 and $n\pi_0 = 100 \cdot 0.3 = 30$ and $n(1 - \pi_0) = 100 \cdot 0.7 = 70$, which are both greater than 5, so the normal approximation can be used.

This approximate test for a proportion is implemented in the R function `prop.test`,

1-sample proportions test with continuity correction

```
data: 42 out of 100, null probability 0.3
X-squared = 6.2976, df = 1, p-value = 0.006045
alternative hypothesis: true p is greater than 0.3
95 percent confidence interval:
 0.3372368 1.0000000
sample estimates:
      p
0.42
```

but it can of course also be calculated using only the normal distribution table.

The normal distribution has two parameters, mean and standard deviation.

From the binomial distribution we know that $E[X] = n\pi$ and $\text{var}(X) = n\pi(1 - \pi)$.

The standard error of X is

$$SE = \sqrt{n\pi(1 - \pi)}$$

If H_0 is true $\pi = \pi_0$ and

$$X \sim N(n\pi_0, n\pi_0(1 - \pi_0))$$

With this null distribution and our observed value $x_{obs} = 42$, the p-value can be computed.

$$p = P(X \geq 42)$$

As we are now approximating a discrete distribution using a continuous distribution we need to use a trick called *continuity correction*, which simply means that we let each integer be represented by a region ± 0.5 from its value. Hence,

$$\begin{aligned} p &= P(X \geq 42) = [\text{continuity correction}] = \\ &P(X \geq 41.5) = P\left(Z \geq \frac{41.5 - n\pi_0}{\sqrt{n\pi_0(1 - \pi_0)}}\right) = \\ &P\left(Z \geq \frac{41.5 - 30}{\sqrt{100 * 0.3 * (1 - 0.3)}}\right) = P(Z \geq 2.51) = 1 - P(Z \leq 2.51) = [Z\text{-table}] = 0.0060 \end{aligned}$$

As $0.00605 < 0.05$ we reject H_0 and conclude that there is reason to believe that the proportion of people allergic to pollen in Uppsala is greater than 0.3.

9.2.2 Two samples, proportions

$$H_0 : \pi_1 - \pi_2 = 0$$

$$H_1 : \pi_1 - \pi_2 \neq 0$$

Alternatively, a one sided alternative hypothesis can be used;

$$H_1 : \pi_1 - \pi_2 > 0 \text{ or } H_1 : \pi_1 - \pi_2 < 0.$$

If the sample sizes are large the normal approximation can be used, an appropriate test statistic is

$$Z = \frac{P_1 - P_2}{\sqrt{P(1 - P) \left(\frac{1}{n_1} + \frac{1}{n_2}\right)}},$$

where P is the proportion in the merged sample of size $n_1 + n_2$. $Z \in N(0, 1)$ and p-value can be computed using the standard normal distribution.

The two-sample test for proportions is implemented in the function `prop.test`.

9.2.3 Paired samples

Sometimes two observations are naturally linked, rather than coming from two independent groups. This happens, for example, in before-and-after studies where the same subjects are measured twice, or in matched study designs where each individual in one group is paired with a similar individual in the other group.

Examples of paired data include:

- measurements before and after treatment in the same patient

- observations on matched case-control pairs

In a paired design, the analysis should take the pairing into account. Treating paired observations as independent would ignore important information and usually reduce the power of the test.

The null hypothesis in a paired analysis is usually that the mean difference within pairs is zero.

Let the paired observations be (X_{i1}, X_{i2}) for $i = 1, 2, \dots, n$, and define the within-pair differences

$$D_i = X_{i2} - X_{i1}.$$

This can now be handled as a one sample test based on the difference and the hypotheses can then be written as

$$H_0 : \mu_D = 0$$

$$H_1 : \mu_D \neq 0,$$

where μ_D is the population mean of the within-pair differences.

If a one-sided question is of interest, the alternative hypothesis could instead be

$$H_1 : \mu_D > 0 \quad \text{or} \quad H_1 : \mu_D < 0.$$

The paired t-test is simply a one-sample t-test applied to the differences. If the differences are approximately normally distributed, then the test statistic

$$T = \frac{\bar{D}}{s_D/\sqrt{n}}$$

is t-distributed with $n - 1$ degrees of freedom, where $\bar{D}$ is the sample mean difference and s_D is the sample standard deviation of the differences.

9.3 Variance

The test of equal variances in two groups is based on the null hypothesis

$$H_0 : \sigma_1^2 = \sigma_2^2$$

If the two samples both come from populations with normal distributions, the sample variances (also random variables) are

$$S_1^2 = \frac{1}{n_1 - 1} \sum_{i=1}^{n_1} (X_{1i} - \bar{X}_1)^2 \quad S_2^2 = \frac{1}{n_2 - 1} \sum_{i=1}^{n_2} (X_{2i} - \bar{X}_2)^2$$

It can be shown that $\frac{(n_1-1)S_1^2}{\sigma_1^2} \sim \chi^2(n_1-1)$ and $\frac{(n_2-1)S_2^2}{\sigma_2^2} \sim \chi^2(n_2-1)$.

Hence, the test statistic for comparing the variances of two populations

$$F = \frac{S_1^2}{S_2^2}$$

is F -distributed with $n_1 - 1$ and $n_2 - 1$ degrees of freedom.

In R a test of equal variances can be performed using the function `var.test`.

Chapter 10

Point and interval estimates

10.1 Point estimate

Unknown population parameters can be estimated from random samples drawn from the population of interest. A value computed from the sample can be used as a **point estimate**, that is, a single-number estimate of the population parameter.

An **estimator** is a random variable computed from the sample, whereas an **estimate** is its observed value for a particular sample.

Example 10.1 (Pollen). If we are interested in the proportion of the Uppsala population that is allergic to pollen, we can investigate this by studying a random sample, e.g. by randomly selecting 100 people from Uppsala. It is important that the sample is actually random; ideally, every individual in the population should have the same probability of being selected.

In our sample, we observed that 42 of the 100 people have a pollen allergy. Hence, the observed sample proportion is $p = 0.42$.

Based on this random sample, our point estimate of the Uppsala population proportion π is $\pi \approx p = 0.42$. There is uncertainty in this estimate: if the sampling were repeated, a different random sample of 100 people would likely be selected, and the resulting point estimate would be slightly different.

Example 10.2 (Weight). The mean weight of a mouse population, μ , is unknown. By taking a sample of size n and computing the mean weight of the mice in the sample, $\bar{x}$, we obtain a point estimate of the population mean.

10.2 Bias and precision

The sample proportion and sample mean are unbiased estimators of the corresponding population parameters. Although these estimators are unbiased, they still vary from sample to sample. In general, the variability decreases as the sample size increases.

An estimator is **unbiased** if its expected value equals the population parameter it is intended to estimate. This means that if the sampling was repeated many times and the estimator was computed each time, the average value of the estimator would equal the true population parameter.

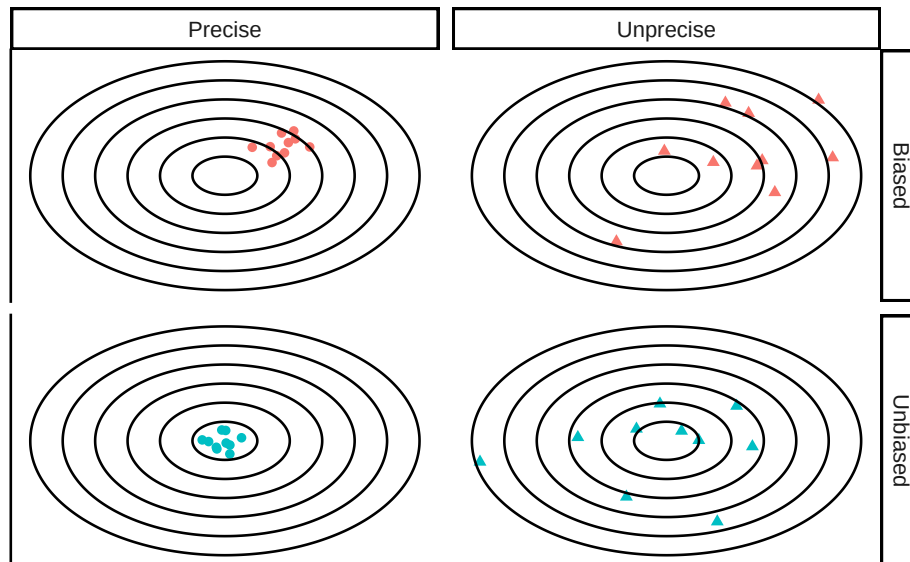


Figure 10.1: Bias and precision.

Even an unbiased estimator is not perfect; it will have a certain amount of uncertainty.

10.3 Interval estimates

A point estimate gives a single best guess of the population parameter, whereas an interval estimate gives a range of plausible values and reflects the uncertainty due to sampling.

To show this uncertainty, an **interval estimate** for a population parameter can be computed instead of reporting only a point estimate. An interval estimate gives a range of plausible values for the population parameter, based on the observed sample. Interval estimation, just like point estimation, is a type of statistical inference.

The width of an interval estimate depends on the variability of the estimator, which is described by its sampling distribution. As seen before, there are different ways to estimate the sampling distribution.

10.3.1 Bootstrap interval

If the sampling distribution of the statistic of interest is unknown, a bootstrap confidence interval can be computed instead.

Bootstrapping is performed by resampling with replacement from the available random sample. The resulting bootstrap distribution can then be used to calculate, for example, a 95% bootstrap interval.

This can be done as follows:

1. Put the entire sample in an urn!

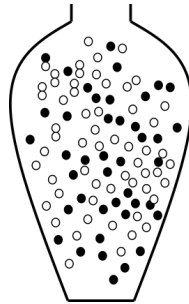


Figure 10.2: An urn model with 42 allergy (black) and 58 non-allergy (white). The black balls represent allergic and the white balls non-allergic.

In R:

```
## the sample consists of 58 non-allergic (0) and 42 allergic
  ↪ (1).
x <- rep(0:1, c(58, 42))
```

2. Sample from the urn with replacement to compute the bootstrap distribution.

```
pboot <- replicate(1000, mean(sample(x, replace=TRUE)))

ggplot(data.frame(x=pboot), aes(x=x)) +
  ↪ geom_histogram(color="white", binwidth=0.02) + theme_bw() +
  ↪ xlab("p")
```

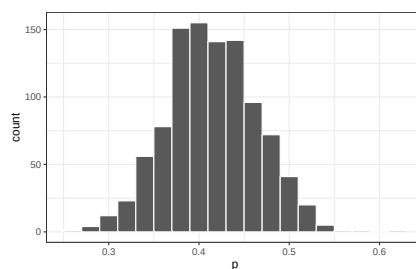


Figure 10.3: Bootstrap distribution.

3. Compute the 95% bootstrap interval. This can be done using the percentile method, by taking the 2.5th and 97.5th percentiles of the bootstrap distribution.

```
ciboot <- quantile(pboot, c(0.025, 0.975))
```

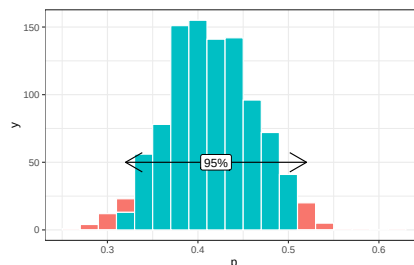


Figure 10.4: Bootstrap distribution and the 95% bootstrap confidence interval.

The 95% bootstrap confidence interval for π is $[0.32, 0.52]$.

10.3.2 Confidence interval

A confidence interval is a type of interval estimate associated with a confidence level.

Definition 10.1. A confidence interval for θ with confidence level $1 - \alpha$ is a method for constructing intervals such that, in repeated sampling, a proportion $1 - \alpha$ of the intervals will contain the true population parameter θ .

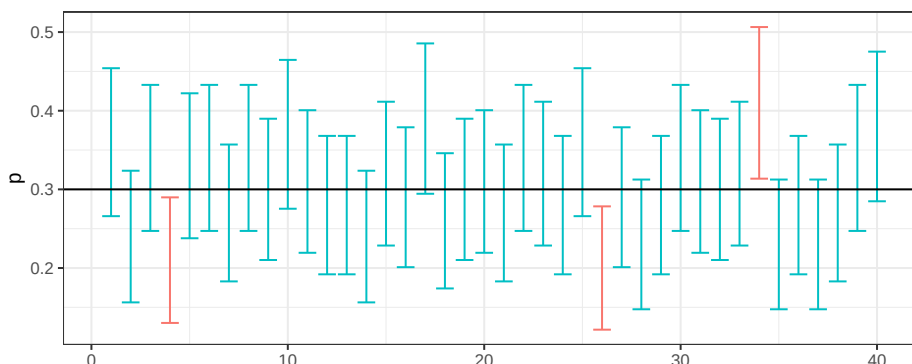


Figure 10.5: A 95% confidence interval procedure gives intervals that cover the true value about 95% of the time.

If the sampling distribution of the statistic of interest is known, the confidence interval can be computed directly.

10.3.3 Confidence interval of proportions

The bootstrap is very useful if the sampling distribution of the statistic is unknown, but in our proportion example we already have an approximate sampling distribution.

From the central limit theorem, the sample proportion random variable P is approximately normally distributed for sufficiently large n :

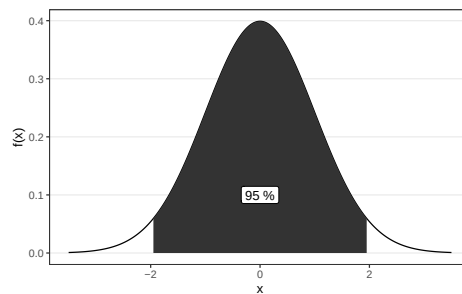


Figure 10.6: The sampling distribution of θ . The probability that an observation of θ falls in the marked region is 0.95.

$$P \sim N\left(\pi, \frac{\pi(1-\pi)}{n}\right)$$

Its standard error is

$$SE = \sqrt{\frac{\pi(1-\pi)}{n}}.$$

It follows that

$$Z = \frac{P - \pi}{SE} \sim N(0, 1).$$

Based on what we know about the standard normal distribution, we can compute an interval around the population proportion π such that the probability that the sample proportion P falls within this interval is $1 - \alpha$:

$$P\left(-z_{\alpha/2} < \frac{P - \pi}{SE} < z_{\alpha/2}\right) = 1 - \alpha.$$

For a 95% confidence interval, $z_{0.025} = 1.96$ (from a table of the standard normal distribution). Other confidence levels of interest include 90% ($z_{0.05} = 1.64$) and 99% ($z_{0.005} = 2.58$). In R, the function `qnorm` is used to compute $z_{\alpha/2}$:

```
## 95% confidence interval, alpha=0.05
qnorm(1-0.05/2)
```

```
[1] 1.959964
```

```
## 90% confidence interval, alpha=0.10
qnorm(1-0.10/2)
```

```
[1] 1.644854
```

```
## 99% confidence interval, alpha=0.01
qnorm(1-0.01/2)
```

[1] 2.575829

The inequality can be rewritten as

$$-z_{\alpha/2} < \frac{P - \pi}{SE} < z_{\alpha/2} \iff P - z_{\alpha/2}SE \leq \pi \leq P + z_{\alpha/2}SE.$$

Hence,

$$P(P - z_{\alpha/2}SE < \pi < P + z_{\alpha/2}SE) = 1 - \alpha.$$

To get the observed confidence interval, replace the random variable P with the observed sample proportion p and replace the unknown standard error by its estimate.

The confidence interval can be expressed in different ways:

$$p - z_{\alpha/2}SE < \pi < p + z_{\alpha/2}SE$$

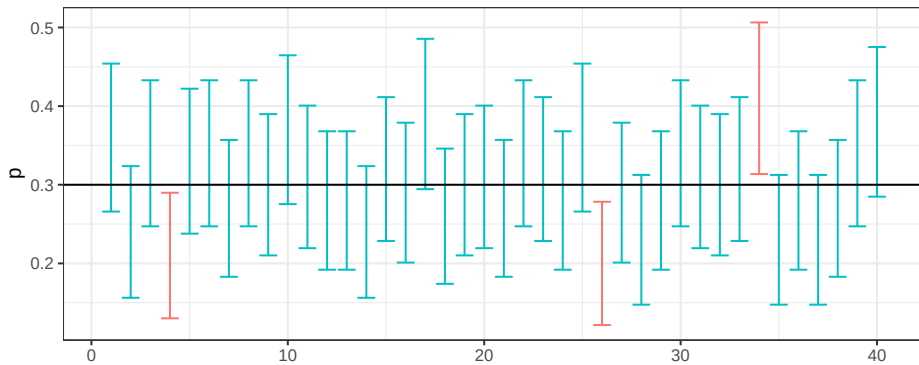
$$\pi = p \pm z_{\alpha/2}SE$$

$$(p - z_{\alpha/2}SE, p + z_{\alpha/2}SE)$$

The approximate 95% confidence interval is therefore

$$\pi = p \pm 1.96\sqrt{\frac{p(1-p)}{n}}.$$

In repeated sampling, approximately 95% of 95% confidence intervals constructed in this way will contain the true population proportion.



Back to our example of the proportion of people allergic to pollen in Uppsala. Here $p = 0.42$ and

$$SE \approx \sqrt{\frac{p(1-p)}{n}} = 0.0493559.$$

Hence, the approximate 95% confidence interval is

$$\pi = 0.42 \pm 1.96 \cdot 0.05 = 0.42 \pm 0.092,$$

or equivalently

$$(0.42 - 0.092, 0.42 + 0.092) = (0.32, 0.52).$$

10.3.4 Confidence interval of the mean

The confidence interval for a mean can be derived similarly.

If a sample of size n consists of independent observations from a normal distribution,

$$X_i \sim N(\mu, \sigma^2),$$

then the sample mean is normally distributed:

$$\bar{X} \sim N\left(\mu, \frac{\sigma^2}{n}\right).$$

If σ is unknown, the statistic

$$T = \frac{\bar{X} - \mu}{s/\sqrt{n}}$$

is t-distributed with $n - 1$ degrees of freedom, in short $T \sim t(n - 1)$.

It follows that

$$P\left(-t_{\alpha/2, n-1} < \frac{\bar{X} - \mu}{S/\sqrt{n}} < t_{\alpha/2, n-1}\right) = 1 - \alpha$$

which can be rewritten as

$$P\left(\bar{X} - t_{\alpha/2, n-1} \frac{S}{\sqrt{n}} < \mu < \bar{X} + t_{\alpha/2, n-1} \frac{S}{\sqrt{n}}\right) = 1 - \alpha.$$

Replacing the random quantities by their observed values gives the $1 - \alpha$ confidence interval for μ :

$$\bar{x} \pm t_{\alpha/2, n-1} \frac{s}{\sqrt{n}}.$$

The t critical values for different confidence levels and degrees of freedom are tabulated and can be computed in R using the function `qt`.

For a 95% confidence interval, i.e. $\alpha = 0.05$, and $n = 5$, the critical value is 2.7764451 and can be computed using

```
n=5  
alpha = 0.05  
## t value  
qt(1-alpha/2, df=n-1)
```

```
[1] 2.776445
```


Chapter 11

Rank-based tests

Chapter 12

Rank-based tests

Hypothesis tests that are based on knowledge of the probability distributions (e.g. normal or binomial) that the data follow are known as **parametric tests**. When data do not meet the parametric test assumptions, we can use **non-parametric** tests, also called distribution free tests, that replace the data with their ranks.

Rank-based tests are often useful when the data are skewed, contain outliers, or are measured on an ordinal scale.

Instead of analyzing the observed values directly, rank-based tests are based on the ranks of the observations. Large values receive high ranks and small values receive low ranks.

Rank-based tests that are commonly used as alternatives to t-tests are:

- one-sample or paired t-test → Wilcoxon signed-rank test
- two-sample t-test → Mann–Whitney test / Wilcoxon rank-sum test

12.1 Two independent samples

For comparing two independent groups when a two-sample t-test is not appropriate, a rank-based alternative is the **Mann–Whitney test**, also known as the **Wilcoxon rank-sum test**.

Suppose we have two independent samples from two populations and want to investigate whether the groups differ in location.

The hypotheses may be written as

$$H_0 : \text{the two groups come from the same distribution}$$
$$H_1 : \text{the two groups differ in location}$$

In practice, the test is often interpreted as a test of whether one group has a higher median than the other, but this interpretation is only valid if the distribution shapes in the two groups are similar.

The test is performed by pooling all observations from both groups, ranking them from smallest to largest, and comparing the sums of ranks between the groups. A significant Mann-Whitney U test means that scores in one group generally rank higher than scores in the other.

In R, the Mann-Whitney / Wilcoxon rank-sum test can be performed using the function `wilcox.test` with two independent samples.

```
x1 <- c(18, 21, 19, 25, 27, 23, 22)
x2 <- c(14, 15, 17, 16, 19, 18, 15)
wilcox.test(x1, x2)
```

Wilcoxon rank sum exact test

data: x1 and x2

W = 47, p-value = 0.003

alternative hypothesis: true location shift is not equal to 0

12.2 Paired samples

For paired data, a rank-based alternative to the paired t-test is the **Wilcoxon signed-rank test**. This test can also be used for the one sample setting, where one sample is compared to a known location parameter.

As for the paired t-test, the analysis is based on the within-pair differences. However, instead of using the observed differences directly, the Wilcoxon signed-rank test uses the ranks of the absolute differences together with their signs.

The hypotheses are commonly expressed as

H_0 : the distribution of within-pair differences is centered at 0

H_1 : the distribution of within-pair differences is not centered at 0

In R, the Wilcoxon signed-rank test can be performed using `wilcox.test` with `paired=TRUE`.

```
before <- c(132, 128, 136, 141, 130, 125, 138, 129)
after <- c(127, 125, 130, 136, 128, 121, 134, 126)
wilcox.test(after, before, paired=TRUE)
```

Wilcoxon signed rank exact test

data: after and before

V = 0, p-value = 0.008

alternative hypothesis: true location shift is not equal to 0

Chapter 13

Multiple testing

Multiple testing is especially important in life-science applications such as genomics, transcriptomics, proteomics, and metabolomics, where thousands of hypotheses may be tested simultaneously.

13.1 Error types

Remember from Section 7.4 that the probability of type I and II errors are denoted α and β , respectively;

$$\alpha = P(\text{type I error}) = P(\text{false alarm}) = P(\text{Reject } H_0 \mid H_0 \text{ is true})$$

$$\beta = P(\text{type II error}) = P(\text{miss}) = P(\text{Fail to reject } H_0 \mid H_1 \text{ is true})$$

and the statistical power

$$\text{power} = 1 - \beta = P(\text{Reject } H_0 \mid H_1 \text{ is true}).$$

13.2 Multiple testing

If a single test is performed, we know that

- $P(\text{One type I error}) = \alpha$

Table 13.1: A hypothesis test either rejects H_0 or fails to reject H_0 . The decision may be correct or incorrect depending on whether H_0 is true or false. TN = true negative, TP = true positive, FN = false negative, FP = false positive.

Decision	H is true	H is false
Fail to reject H_0	TN	Type II error, miss, FN
Reject H_0	Type I error, false alarm, FP	TP

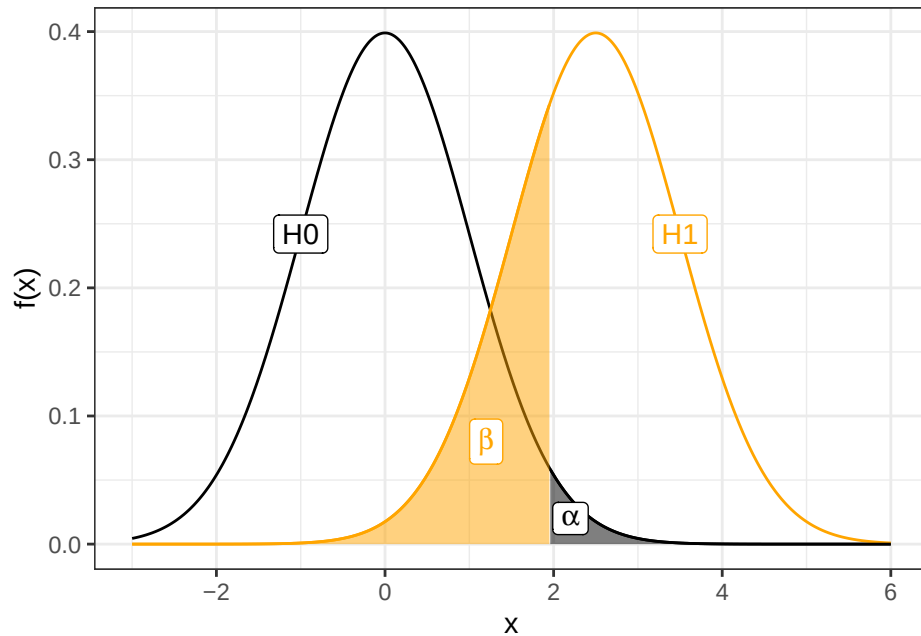
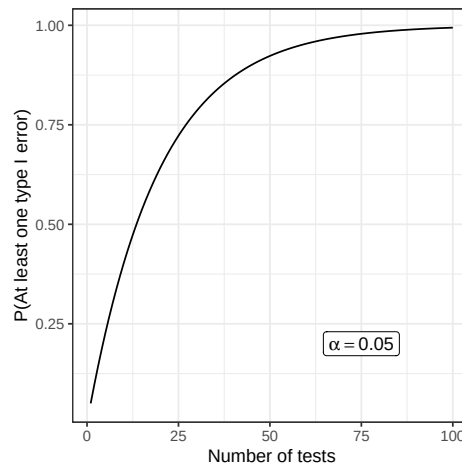


Figure 13.1: The probability density functions under H_0 and H_1 , respectively. The probability of type I error (α) and type II error (β) are indicated.

- $P(\text{No type I error}) = 1 - \alpha$

If m independent tests are performed (e.g. investigate many genes or proteins), the risk of at false alarm (type I error) increases;

- $P(\text{No type I errors in } m \text{ tests}) = (1 - \alpha)^m$
- $P(\text{At least one type I error in } m \text{ tests}) = 1 - (1 - \alpha)^m$



Two common principles for dealing with multiple testing are control of family-wise error rate or false discovery rate.

- FWER: family-wise error rate, control the probability of one or more false

Decision	H is true	H is false
Fail to reject H	TN	FN
Reject H	FP	TP

positive $P(N_{FP} > 0)$, e.g. Bonferroni, Holm

- FDR: false discovery rate, control the expected value of the proportion of false positives among hits (rejected hypotheses), $E[N_{FP}/(N_{FP} + N_{TP})]$ (note: for 0 positives the ratio is defined to be 0), e.g. Benjamini-Hochberg, Storey

13.3 Bonferroni correction

To achieve a family-wise error rate of $FWER \leq \gamma$ when performing m tests, declare significance and reject the null hypothesis for any test with $p \leq \gamma/m$.

A disadvantage of the Bonferroni correction is that it can be overly conservative, especially when many tests are performed.

13.4 Benjamini-Hochbergs FDR procedure

The false discovery rate is the expected proportion of false positives among ‘hits’, i.e. $\frac{FP}{TP+FP}$.

The Benjamini-Hochberg procedure controls the FDR level, γ , under the assumption of independence (and also under certain forms of positive dependence) when performing m tests, as follows:

1. Sort the p-values $p_1 \leq p_2 \leq \dots \leq p_m$.
2. Find the maximum j such that $p_j \leq \gamma \frac{j}{m}$.
3. Declare significance for all tests $1, 2, \dots, j$.

13.5 ‘Adjusted’ p-values

Sometimes an adjusted significance threshold is not reported, but instead ‘adjusted’ p-values are reported.

- Using Bonferroni’s method the ‘adjusted’ p-values are:

$$\tilde{p}_i = \min(mp_i, 1).$$

A feature’s Bonferroni-adjusted p-value represents the smallest family-wise error rate at which that feature would be deemed significant.

- Benjamini-Hochberg’s ‘adjusted’ p-values are often referred to as q -values and can be computed after sorting the p-values as follows:

$$q_i = \min\left(\frac{m}{i}p_i, 1\right)$$

A feature's q -value can be interpreted as the lowest FDR at which the corresponding null hypothesis will be rejected, i.e. the feature will be deemed significant.

In practice, adjusted p-values are usually computed using software such as the R function `p.adjust`.

Example 13.1 (10000 independent tests).

p-value	adj p (Bonferroni)	q-value (B-H)
1.7e-08	0.0002	0.0002
5.8e-08	0.0006	0.0003
3.4e-07	0.0034	0.0011
9.1e-07	0.0091	0.0020
1e-06	0.0100	0.0020
2.4e-06	0.0240	0.0040
2.3e-05	0.2300	0.0329
3.6e-05	0.3600	0.0450
0.00022	1.0000	0.2300
0.00023	1.0000	0.2300
0.00073	1.0000	0.6636
0.0032	1.0000	1.0000
0.0045	1.0000	1.0000
0.0087	1.0000	1.0000
0.0089	1.0000	1.0000
0.012	1.0000	1.0000
0.014	1.0000	1.0000
0.045	1.0000	1.0000
0.08	1.0000	1.0000
0.23	1.0000	1.0000

Chapter 14

Analysis of variance (ANOVA)

In previous sections, we have seen how to compare two groups using t-tests. However, sometimes we want to compare more than two groups. One way to do this is to perform multiple pairwise t-tests, but this approach has some limitations, such as inflating the Type I error rate. An omnibus test, such as analysis of variance (ANOVA), can be used to test for differences among multiple groups in a single test.

With one numeric response variable of interest, such as blood pressure or biomarker concentration, and one categorical variable with k groups, such as treatment group, we can use **one-way ANOVA** to test if the group means are equal.

Hypotheses:

$$H_0 : \mu_1 = \mu_2 = \cdots = \mu_k$$

$$H_1 : \text{not all group means are equal}$$

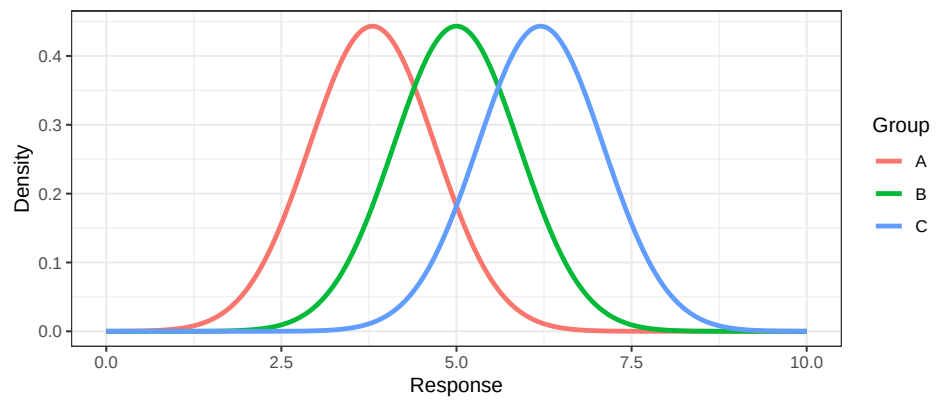
The ANOVA test statistic is based on the ratio of the variation between groups to the variation within groups. If the between-group variation is large relative to the within-group variation, we have evidence against the null hypothesis.

$$F = \frac{\text{variation between groups}}{\text{variation within groups}}$$

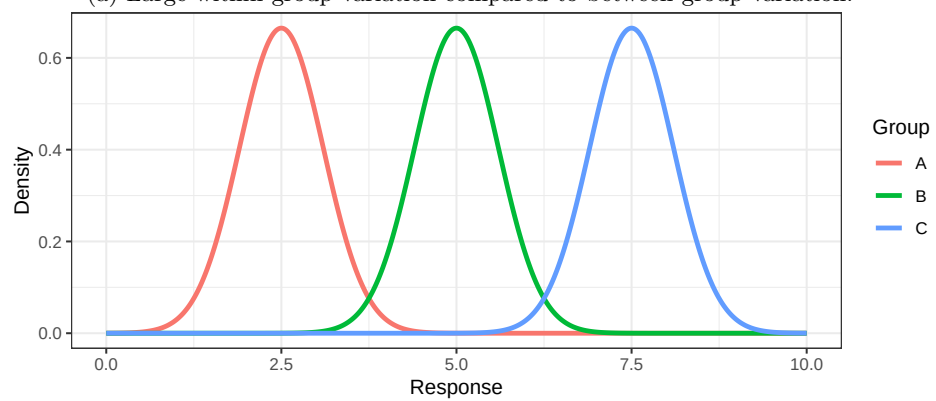
14.1 Sum of squares

The sum of squares (SS) is a measure of variation, the cumulative squared deviation of each observation from the mean.

In ANOVA, we partition the total variation (SS_{total}) in the response variable into two components: the variation between groups ($SS_{between}$) and the variation within groups (SS_{within}).



(a) Large within-group variation compared to between-group variation.



(b) Small within-group variation compared to between-group variation.

Figure 14.1: ANOVA compares the separation between group means to the variation within groups.

$$SS_{total} = SS_{between} + SS_{within},$$

where $SS_{between}$ measures the variation of the group means from the overall mean, and SS_{within} measures the variation of the observations within each group from their respective group means.

$$SS_{between} = \sum_{i=1}^k n_i (\bar{Y}_i - \bar{Y})^2$$

$$SS_{within} = \sum_{i=1}^k \sum_{j=1}^{n_i} (Y_{ij} - \bar{Y}_i)^2$$

The total sum of squares (SS_{total}) is the sum of the squared deviations of each observation from the overall mean:

$$SS_{total} = \sum_{i=1}^k \sum_{j=1}^{n_i} (Y_{ij} - \bar{Y})^2$$

Here, k is the number of groups, N is the total number of observations, n_i is the number of observations in group i , $\bar{Y}_i$ is the mean of group i , and $\bar{Y}$ is the overall mean of all observations.

14.2 Mean squares and F statistic

The mean squares are computed by dividing the sum of squares by their respective degrees of freedom:

$$MS_{total} = \frac{SS_{total}}{N - 1}$$

$$MS_{between} = \frac{SS_{between}}{k - 1}$$

$$MS_{within} = \frac{SS_{within}}{N - k}$$

and the ANOVA test statistic, the F statistic, is computed as the ratio of the mean square between groups to the mean square within groups:

$$F = \frac{MS_{between}}{MS_{within}}$$

Under H_0 , the F statistic is F distributed with $k - 1$ and $N - k$ degrees of freedom.

Using the F distribution, we can compute a p-value for the ANOVA test. If the p-value is less than the significance level α , we reject the null hypothesis and

conclude that there is evidence that at least one group mean differs from the others.

Note: When there are only two groups, one-way ANOVA is equivalent to the two-sample t-test (in the equal-variance case).

14.3 Assumptions

The one-way ANOVA has three main assumptions:

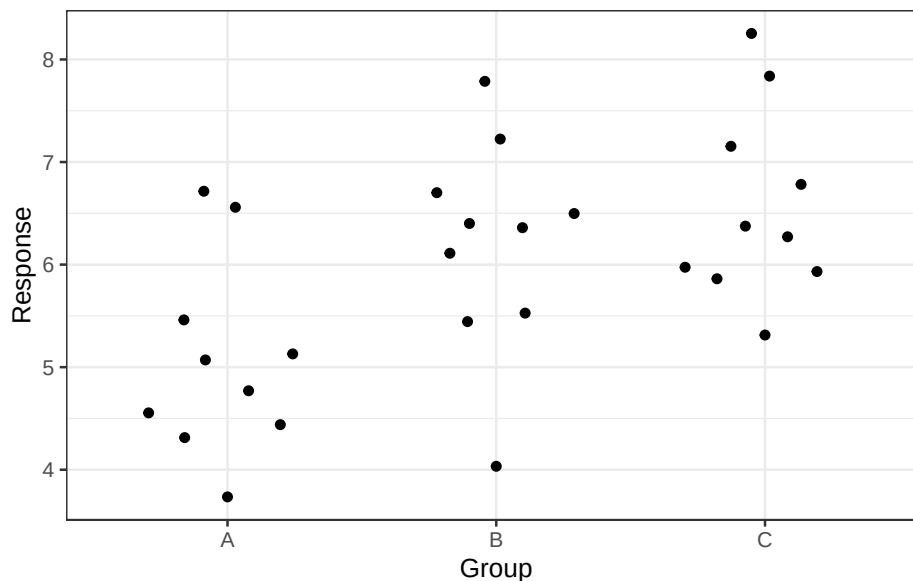
- independent observations
- approximately normally distributed observations within groups
- equal variances across groups

The ANOVA test is fairly robust to violations of the normality assumption, especially when the sample sizes are similar and large. When the assumptions are violated, alternative methods such as the non-parametric Kruskal-Wallis test can be used.

14.4 Post hoc test

The ANOVA test is an omnibus test, meaning that it tests for any difference among the group means. A significant ANOVA result indicates that at least one group mean is different from the others, but it does not specify which groups differ. To identify the specific group differences, we can perform post hoc pairwise comparisons between the groups. As several pairwise comparisons are made, the risk of Type I error increases. Therefore, it is important to adjust for multiple comparisons. A common post hoc test is **Tukey's Honest Significant Difference (HSD)** test, which compares all pairs of group means and controls the family-wise error rate.

In R, a one-way ANOVA can be fitted using `aov`, and post hoc comparisons can be performed using `TukeyHSD`:



```

              Df Sum Sq Mean Sq F value    Pr(>F)
group           2  12.24   6.122     6.435 0.00518 **
Residuals      27  25.68   0.951

```

```
---
```

```
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
```

```

  Tukey multiple comparisons of means
    95% family-wise confidence level

```

```
Fit: aov(formula = response ~ group, data = df)
```

```

$group
      diff      lwr      upr    p adj
B-A 1.133963 0.05254072 2.215452 0.0384525
C-A 1.5008155 0.41935988 2.582271 0.0052283
C-B 0.3668192 -0.71463643 1.448275 0.6812485

```

If the ANOVA test is not significant, post hoc pairwise comparisons are usually not performed.

14.5 Non-parametric alternative

If the assumptions of one-way ANOVA are not fulfilled, a rank-based alternative is the **Kruskal–Wallis** test. This test compares the groups using the ranks of the observations rather than the raw values.

The null hypothesis is that the mean ranks of the groups are the same. A significant Kruskal–Wallis test indicates that at least one group differs from the others, but like ANOVA, it does not show which groups differ.

In R, the Kruskal–Wallis test can be performed using the function `kruskal.test`.

```
Kruskal-Wallis rank sum test
```

data: response by group

Kruskal-Wallis chi-squared = 8.1858, df = 2, p-value = 0.01669

Chapter 15

Exercises: Statistical inference

Exercises marked * are extra and can be skipped if the time is short.

15.1 Hypothesis tests, resampling

Exercise 15.1 (Diet). A diet study aims to study how the hemoglobin (Hb) levels in blood are affected by an iron-rich diet consisting of tofu, soybeans, broccoli, lentils and peas. To perform the study the dietician has recruited 40 male participants, who are randomly assigned to the iron-rich diet or control group (no change in participants diet), 20 participants in each group.

The observed Hb levels (in g/L);

```
ctrl <- c(197, 186, 157, 170, 193, 188, 175, 186, 177, 191, 168,  
  ↪ 193, 191, 189, 188, 192, 179, 186, 197, 203)  
iron <- c(187, 218, 196, 210, 206, 178, 181, 193, 172, 202, 169,  
  ↪ 221, 183, 222, 185, 174, 192, 192, 162, 211)
```

Perform a hypothesis test to investigate if the Hb level is affected (increased or decreased) by the iron-rich diet.

i Solution

Define H_0 and H_1

$H_0 : \mu_{iron} = \mu_{ctrl}$ No difference in mean Hb level between control group and iron-rich diet group.

$H_1 : \mu_{iron} \neq \mu_{ctrl}$

Will use the significance level, $\alpha = 0.05$.

Select test statistic $D = \bar{X}_{iron} - \bar{X}_{ctrl}$, where $\bar{X}_{ctrl}$ is the mean Hb level in a control group of 20 people and $\bar{X}_{iron}$ is the mean Hb level in a diet group of 20 people.

The observed value; d_{obs}

```

miron <- mean(iron)
mctrl <- mean(ctrl)
(dobs <- miron - mctrl)

```

```
[1] 7.4
```

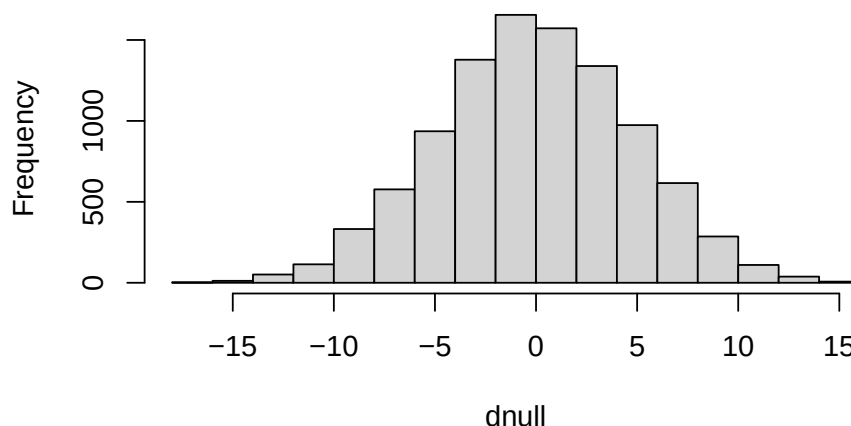
Compute null distribution using permutation.

```

## Under null all observations are equivalent
allobs <- c(iron, ctrl)
dnull <- replicate(10000, {
  ## Permute the 40 observations and assign first 20 to the
  ↪ iron group
  x <- sample(allobs)
  d <- mean(x[1:20]) - mean(x[21:40])
})
hist(dnull)

```

Histogram of dnull



Compute p-value;

```
(p <- mean(abs(dnull) >= abs(dobs)))
```

```
[1] 0.1268
```

As the estimated p-value is 0.1268, $p > \alpha$, we fail to reject the null hypothesis. The data do not provide evidence against the null hypothesis, i.e. there is no reason to believe that the iron-rich diet affects the mean Hb level.

Exercise 15.2 (Pollen*). You want to investigate if the proportion of Swedish students allergic to pollen is greater than 0.3, the proportion in Sweden as a whole. To do this, you observe 20 people in a student group at BMC in Uppsala, 9 of them are allergic to pollen.

Is there reason to believe that the proportion of Swedish students allergic to pollen is greater than 0.3? Perform a hypothesis test to answer the question.

Can you identify any problems with this study setup?

i Solution

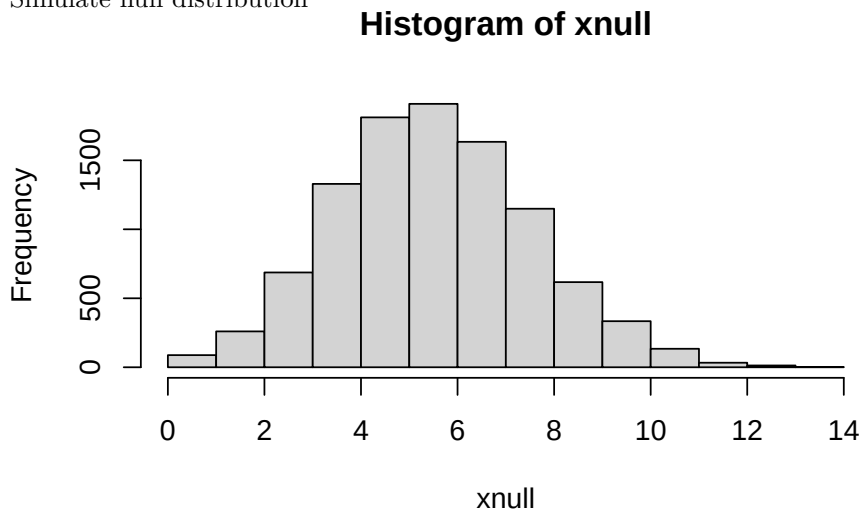
$$H_0 : \pi = 0.3 \quad H_1 : \pi > 0.3$$

Set the significance level to $\alpha = 0.05$.

The test statistic is X , the number of allergic people in a sample of size 20.

$$x_{obs} = 9$$

Simulate null distribution



Compute p-value, i.e. if null is true what is the probability to observe x_{obs} or higher?

```
xobs <- 9
(p <- mean(xnull >= xobs))
```

```
[1] 0.1133
```

As $p > \alpha$ we fail to reject the null hypothesis. The data do not provide evidence against the null hypothesis, i.e. there is no reason to believe that the students are more allergic than the general Swedish population.

Problems with the study: Discuss in your group! Is the sample random? Is it reasonable to select 20 students at BMC to answer a question about all students in Sweden?

15.2 Hypothesis tests, parametric

Exercise 15.3. The hemoglobin value (Hb) in women is on average 140 g/L. You observe the following Hb values in a set of five male blood donors: 154, 140, 147, 162, 172. Assume that Hb is normally distributed. Is there a reason to believe that the mean Hb value in men differs from that in women?

i Solution

Let X denote the Hb value in g/L for male blood donors.

$$H_0 : \mu = 140$$

$$H_1 : \mu \neq 140$$

Use the significance level $\alpha = 0.05$.

Use the test statistic $T = \frac{\bar{X} - \mu_0}{SE}$, where μ_0 is the population mean under H_0 , i.e. 140, and the standard error of the sample mean is estimated by $SE \approx \frac{s}{\sqrt{n}}$.

Compute $t_{obs} = \frac{\bar{x} - 140}{s/\sqrt{n}}$.

```
x <- c(154, 140, 147, 162, 172)
## sample mean
m <- mean(x)
## estimated standard error of the sample mean
SE <- sd(x)/sqrt(5)
## Observed value of the test statistic
(tobs <- (m - 140)/SE)
```

```
[1] 2.676865
```

Compute the p-value. For a two-sided test $P(|T| > |t_{obs}|) = P(T > |t_{obs}|) + P(T < -|t_{obs}|)$.

```
## P(T > tobs)
pt(tobs, df=4, lower.tail=FALSE)
```

```
[1] 0.02770443
```

```
## p = P(t > tobs) + P(t < -tobs) = 2 * P(t > tobs)
(p <- 2*pt(tobs, df=4, lower.tail=FALSE))
```

```
[1] 0.05540887
```

As $p > \alpha$, we fail to reject the null hypothesis. The data do not provide evidence that the Hb value in men differs from that of women (140 g/L). Note that the same can be achieved using the function `t.test`;

One Sample t-test

```
data: x
t = 2.6769, df = 4, p-value = 0.05541
alternative hypothesis: true mean is not equal to 140
95 percent confidence interval:
 139.442 170.558
sample estimates:
mean of x
 155
```

Exercise 15.4. The hemoglobin value (Hb) in men is on average 188 g/L. The Hb values in Exercise 15.3 were actually measured after the men had donated blood. Is there a reason to believe that the mean Hb level for men after blood donation is less than 188 g/L?

i Solution

$H_0 : \mu = 188$ $H_1 : \mu < 188$

Use the significance level $\alpha = 0.05$.

Use the test statistic $T = \frac{\bar{X} - \mu_0}{SE}$, where $\mu_0 = 188$ and the standard error is estimated by $SE \approx \frac{s}{\sqrt{n}}$.

Compute $t_{obs} = \frac{\bar{x} - 188}{s/\sqrt{n}}$

```
## Observed value of test statistic
(tobs <- (m-188)/SE)
```

```
[1] -5.889103
```

Compute the p-value, $P(T < t_{obs})$

```
## p=P(T<tobs)
(p <- pt(tobs, df=4, lower.tail=TRUE))
```

```
[1] 0.002078429
```

As $p < \alpha$ the null hypothesis is rejected and we can conclude that the data provide evidence that the mean Hb value after blood donation is less than 188 g/L.

Note that the same can be achieved using the function `t.test`;

```
t.test(x, mu=188, alternative="less")
```

One Sample t-test

```
data: x
t = -5.8891, df = 4, p-value = 0.002078
alternative hypothesis: true mean is less than 188
95 percent confidence interval:
 -Inf 166.946
sample estimates:
mean of x
 155
```

Exercise 15.5. By observing the Hb values in 5 male blood donors: 154, 140, 147, 162, 172 g/L, and 5 female blood donors: 123, 140, 137, 132, 127 g/L, is there a reason to believe that the Hb level is higher in men than in women?

i Solution

Let X_m denote the Hb value in men and X_w the Hb value in women.

$$H_0 : \mu_m = \mu_w$$

$$H_1 : \mu_m > \mu_w$$

Use the significance level $\alpha = 0.05$.

Use the test statistic $T = \frac{\bar{X}_m - \bar{X}_w}{SE}$. If we assume equal variances we can use Student's t-test and compute SE using the pooled standard deviation. An alternative is to use Welch's t-test (unequal variances t-test) and compute

$SE = \sqrt{\frac{s_m^2}{n_m} + \frac{s_w^2}{n_w}}$. The test statistic is t-distributed and the degrees of freedom are approximated using the Welch-Satterthwaite's equation, as implemented in `t.test`.

```
x <- c(154, 140, 147, 162, 172)
y <- c(123, 140, 137, 132, 127)
## Perform Welch's t-test with unequal variances
tt <- t.test(x, y, alternative="greater")
tt
```

Welch Two Sample t-test

```
data: x and y
t = 3.6171, df = 6.2637, p-value = 0.00517
alternative hypothesis: true difference in means is greater than 0
95 percent confidence interval:
 10.8297      Inf
sample estimates:
mean of x mean of y
 155.0      131.8
```

As the p-value $p = 0.0051699$ is less than α , we reject the null hypothesis and conclude that the data provide evidence that the mean Hb is higher in male blood donors than in female.

Exercise 15.6. Based on statistics from blodcentralen we learn that male Hb values (before donation) are normally distributed with mean 188 g/L and standard deviation 16 g/L. Using this new information and the following observed Hb values in five male donors after donation: 154, 140, 147, 162, 172 g/L, is there reason to believe that the mean Hb value for men after blood donation is lower than 188 g/L?

i Solution

$H_0 : \mu = 188$ $H_1 : \mu < 188$

Use $\alpha = 0.05$.

Under H_0 , the Hb value for a single male donor is normally distributed with mean 188 g/L and standard deviation 16 g/L.

$X \sim N(188, 16^2)$

Since the sample size is 5, the sample mean is normally distributed with $\bar{X} \sim N\left(188, \frac{16^2}{5}\right)$, so the standard error of mean is $SE = \frac{16}{\sqrt{5}}$.

Use the test statistic

$Z = \frac{\bar{X} - 188}{16/\sqrt{5}} \sim N(0, 1)$

Compute z_{obs}

```
x <- c(154, 140, 147, 162, 172)
m <- mean(x)
SE <- 16/sqrt(5)
zobs <- (m - 188)/SE
```

Compute the p-value:

$$p = P(\bar{X} \leq x_{obs}) = P(Z \leq z_{obs}).$$

```
(p <- pnorm(zobs))
```

```
[1] 1.995119e-06
```

As $p < \alpha$, we reject H_0 and conclude that the data provide evidence that the mean Hb value for men after blood donation is lower than 188 g/L.

Exercise 15.7. In order to study the effect of high-fat diet, 12 mice are fed normal diet (control group) and 12 mice are fed high-fat diet. After a couple of weeks, the mouse weights in grams are recorded:

High-fat mice (g): 25, 30, 23, 18, 31, 24, 39, 26, 36, 29, 23, 32

Normal diet mice (g): 27, 25, 22, 23, 25, 37, 24, 26, 21, 26, 30, 24

Does high-fat diet increase body weight in mice?

- Assume equal variances.
- Do not assume equal variances.

i Solution

a)

```
## Student's t-test with pooled variances
t.test(xHF, xN, var.equal=TRUE, alternative="greater")
```

Two Sample t-test

data: xHF and xN

t = 1.0234, df = 22, p-value = 0.1586

alternative hypothesis: true difference in means is greater than 0

95 percent confidence interval:

-1.468785 Inf

sample estimates:

mean of x mean of y

28.00000 25.83333

b)

```
## Unequal variances with Welch approximation to the degrees
of freedom (the default)
```

```
t.test(xHF, xN, var.equal=FALSE, alternative="greater")
```

Welch Two Sample t-test

data: xHF and xN

t = 1.0234, df = 19.818, p-value = 0.1592

alternative hypothesis: true difference in means is greater than 0

95 percent confidence interval:

```

-1.486449      Inf
sample estimates:
mean of x mean of y
28.00000  25.83333

```

Exercise 15.8 (Paired Hb values). Hemoglobin values (g/L) are measured in 8 blood donors before and after blood donation:

```

before <- c(154, 149, 162, 158, 171, 165, 160, 156)
after  <- c(146, 141, 153, 150, 163, 156, 152, 149)

```

Is there evidence that blood donation lowers the mean Hb level? a) State the null and alternative hypotheses. b) Explain why a paired test should be used rather than a two-sample test. c) Perform a paired t-test in R. d) What do you conclude?

i Solution

a) Let $D_i = \text{Hb}_{\text{after},i} - \text{Hb}_{\text{before},i}$ be the within-pair differences. $H_0 : \mu_D = 0$ $H_1 : \mu_D < 0$

b) The observations are paired because each donor is measured twice. The analysis should therefore be based on the within-donor differences.

c)
Paired t-test

```

data:  after and before
t = -35.859, df = 7, p-value = 1.703e-09
alternative hypothesis: true mean difference is less than 0
95 percent confidence interval:
 -Inf -7.695723
sample estimates:
mean difference
 -8.125

```

d) If the p-value is below 0.05, we reject H_0 and conclude that the data provide evidence that blood donation lowers mean Hb.

Exercise 15.9 (Paired t-test). Hemoglobin values (Hb, g/L) are measured in 8 blood donors before and after blood donation:

```

before <- c(154, 149, 162, 158, 171, 165, 160, 156)
after  <- c(146, 141, 153, 150, 163, 156, 152, 149)

```

Is there evidence that blood donation lowers the mean Hb level?

- State the null and alternative hypotheses.
- Explain why a paired test should be used rather than a two-sample test.
- Perform a paired t-test in R.

d) What do you conclude?

i Solution

a) Let $D_i = \text{after}_i - \text{before}_i$ denote the within-donor differences.

$$H_0 : \mu_D = 0$$

$$H_1 : \mu_D < 0$$

b) The observations are paired because each donor is measured twice. The analysis should therefore be based on the within-donor differences.

c) Perform a paired t-test:

```
t.test(after, before, paired=TRUE, alternative="less")
```

Paired t-test

data: after and before

t = -35.859, df = 7, p-value = 1.703e-09

alternative hypothesis: true mean difference is less than 0

95 percent confidence interval:

-Inf -7.695723

sample estimates:

mean difference

-8.125

d) If the p-value is below 0.05, we reject H_0 and conclude that the data provide evidence that blood donation lowers mean Hb.

15.3 Point and interval estimates

Exercise 15.10. You measure the Hb value in 10 50-year old men and get the following observations; 145, 165, 134, 167, 158, 176, 156, 189, 143, 123 g/L.

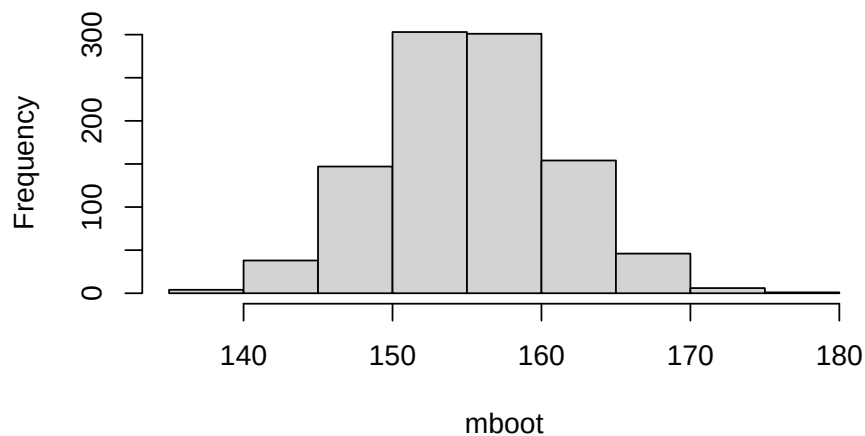
- Compute a 95% bootstrap interval for the mean Hb value.
- Compute the sample mean Hb value
- Compute the sample variance
- Compute the sample standard deviation
- Assume that Hb is normally distributed and compute a point estimate and a 95% confidence interval for the mean Hb value.

i Solution

a)

```
obs <- c(145, 165, 134, 167, 158, 176, 156, 189, 143, 123)
mboot <- replicate(1000, {
  x <- sample(obs, size=10, replace=TRUE)
  mean(x)
})
hist(mboot)
```

Histogram of mboot



```
## The 95% bootstrap interval is:
quantile(mboot, c(0.025, 0.975))
```

```
2.5% 97.5%
143.700 166.805
b) Sample mean
```

```
(m <- mean(obs))
```

```
[1] 155.6
c) Sample variance
```

```
(v <- var(obs))
```

```
[1] 395.1556
d) Sample standard deviation
```

```
(s <- sd(obs))
```

```
[1] 19.87852
```

e) The point estimate is the sample mean, $m = 155.6$.
The sample size is small ($n = 10$) and the population standard deviation unknown, hence we use the t-statistic;

$$T = \frac{\bar{X} - \mu}{\frac{s}{\sqrt{n}}}$$

and compute the 95% confidence interval as

$$m \pm t_{\alpha/2, n-1} \frac{s}{\sqrt{n}}$$

```
n <- length(obs)
tcrit <- qt(0.975, df=9)
##95% confidence interval
c(m - tcrit*s/sqrt(n), m + tcrit*s/sqrt(n))

[1] 141.3798 169.8202
```

Exercise 15.11. A scale has a normally distributed error with mean 0 and standard deviation 2.3 g. You measure an object 10 times and observe the mean weight 43 g.

- Compute a 95% confidence interval of the object's mean weight
- Compute a 90% confidence interval of the object's mean weight

i Hint

Because the measurement error standard deviation is known, a normal-based confidence interval can be used.

i Solution

The measured weight is a random variable $X \sim N(\mu, \sigma^2)$. You know that $\sigma = 2.3$, μ is the weight of the object.

- Compute a 95% confidence interval of the sample mean weight

```
## 95% confidence interval
m <- 43
sigma <- 2.3
n <- 10
z <- qnorm(0.975)
c(m - z*sigma/sqrt(10), m + z*sigma/sqrt(10))

[1] 41.57447 44.42553
```

- Compute a 90% confidence interval of the sample mean weight

```
z <- qnorm(0.95)
c(m - z*sigma/sqrt(10), m + z*sigma/sqrt(10))

[1] 41.80366 44.19634
```

Exercise 15.12 (CI, proportions*). The 95% confidence interval for a proportion can be computed using the formula $p \pm zSE$, where p the sample proportion and the standard error $SE = \sqrt{\frac{p(1-p)}{n}}$. $z = 1.96$ for a 95% confidence interval.

We study the proportion of pollen allergic people in Uppsala and in a random sample of size 100 observe 42 pollen allergic people.

- Calculate a 95% confidence interval for π
- How can we get a narrower confidence interval?

- c) We computed a 95% interval, what if we want a 90% confidence interval?
 d) or a 99% confidence interval?

i Solution

- a)
 [1] 0.3232643 0.5167357
 b) A narrower confidence interval can be obtained by increasing the sample size. It can also be made narrower by lowering the confidence level, for example from 95% to 90%.
 c) Change the z number,
 The approximate confidence interval for a proportion is

$$p \pm zSE$$

For a 90% confidence interval use $z=1.64$

```
p <- 0.42
n <- 100
SE <- sqrt(p*(1-p)/n)
z <- qnorm(0.95)
c(p - z*SE, p + z*SE)
```

```
[1] 0.3388168 0.5011832
```

- d) or a 99% confidence interval?

```
z <- qnorm(0.995)
c(p - z*SE, p + z*SE)
```

```
[1] 0.2928678 0.5471322
```

Exercise 15.13 (Smokers*). You observe 150 students at BMC of which 25 are smokers. Compute a 95% confidence interval for the proportion of smokers among BMC students.

i Solution

Point estimate of the proportion of smokers; $p = 25/150 = 1/6$.
 The approximate confidence interval for a proportion is
 $p \pm zSE$.

```
p <- 25/150
n <- 150
z <- qnorm(0.975)
SE <- sqrt(p*(1-p)/n)
## 95% CI
c(p - z*SE, p + z*SE)
```

```
[1] 0.1070269 0.2263065
```

15.4 Rank-based tests

Exercise 15.14 (Mann-Whitney / Wilcoxon rank-sum test). The following pain scores were recorded for two independent treatment groups A and B. The scores are ordinal, where higher values indicate more pain.

```
A <- c(2, 3, 1, 4, 2, 3, 2)
B <- c(5, 4, 6, 3, 5, 4, 6)
```

- Why might a rank-based test be more appropriate than a two-sample t-test here?
- State the null and alternative hypotheses.
- Perform an appropriate rank-based test.
- What is your conclusion?

i Solution

- The response is ordinal, so a rank-based test is more appropriate than a t-test based on numerical means.
-

H_0 : the two groups come from the same distribution

H_1 : the two groups differ in location

- Perform the test in R:

```
wilcox.test(A, B)
```

```
Wilcoxon rank sum exact test
```

```
data: A and B
```

```
W = 3, p-value = 0.005828
```

```
alternative hypothesis: true location shift is not equal to 0
```

- If the p-value is below 0.05, we reject H_0 and conclude that the data provide evidence that the two treatment groups differ.

Exercise 15.15 (Wilcoxon signed-rank test). The following symptom scores were recorded in 8 patients before and after treatment:

```
before <- c(8, 7, 6, 9, 8, 7, 9, 6)
after <- c(6, 5, 5, 7, 7, 6, 8, 5)
```

You want to investigate if the treatment had an effect.

- Is this a paired design?
- What could be an appropriate test?
- Perform the test in R!

d) What is your conclusion?

i Solution

- a) Each patient is measured twice, so the observations are paired.
- b) The sample size is small and the symptom score is ordinal/discrete, so a rank-based paired test is reasonable.
- c) Perform the Wilcoxon signed rank test in R:

```
wilcox.test(after, before, paired=TRUE, alternative="less")
```

Wilcoxon signed rank exact test

data: after and before

V = 0, p-value = 0.003906

alternative hypothesis: true location shift is less than 0

- d) If the p-value is below 0.05, we reject H_0 and conclude that the data provide evidence that symptom scores are lower after treatment.

15.5 Multiple testing

Exercise 15.16 (Many tests). A study tests 100 independent null hypotheses using significance level $\alpha = 0.05$.

- a) What is the probability of making at least one type I error if all 100 null hypotheses are true?
- b) Compute the Bonferroni-adjusted significance threshold for controlling the family-wise error rate at 0.05.

i Solution

a)

$$P(\text{at least one type I error}) = 1 - (1 - \alpha)^{100} = 1 - (0.95)^{100}$$

```
1 - (1 - 0.05)^100
```

```
[1] 0.9940795
```

b)

$$\frac{0.05}{100} = 0.0005$$

Exercise 15.17 (Adjusted p-values). Eight hypotheses were tested, giving the following p-values:

```
p <- c(0.0003, 0.001, 0.004, 0.012, 0.021, 0.049, 0.11, 0.24)
```

- a) Compute Bonferroni-adjusted p-values.

- b) Compute Benjamini-Hochberg adjusted p-values.
- c) Which hypotheses are significant at FWER 0.05 using Bonferroni?
- d) Which hypotheses are significant at FDR 0.05 using Benjamini-Hochberg?

i Solution

a) and b)

```
p.adjust(p, method="bonferroni")
```

```
[1] 0.0024 0.0080 0.0320 0.0960 0.1680 0.3920 0.8800 1.0000
```

```
p.adjust(p, method="BH")
```

```
[1] 0.00240000 0.00400000 0.01066667 0.02400000 0.03360000 0.06533333 0.12571429
```

```
[8] 0.24000000
```

- c) Bonferroni significance can be assessed either using the adjusted p-values or the Bonferroni threshold $0.05/8$.
- d) Benjamini-Hochberg significance can be assessed using the BH-adjusted p-values and comparing them to 0.05.

15.6 Analysis of variance (ANOVA)

Exercise 15.18 (Why ANOVA?). You want to compare the mean Hb level in three groups: a control group, a vegetarian diet group, and an iron-rich diet group.

- a) Why is a two-sample t-test not sufficient here?
- b) State the null and alternative hypotheses for a one-way ANOVA.
- c) What does a significant ANOVA result tell you?

i Solution

- a) A two-sample t-test compares only two groups at a time. Since there are three groups here, repeated pairwise t-tests would be needed, which increases the risk of Type I error.
- b) The hypotheses are

$$H_0 : \mu_1 = \mu_2 = \mu_3$$

$$H_1 : \text{not all group means are equal}$$

- c) A significant ANOVA result means that there is evidence that at least one group mean differs from the others. However, it does not tell us which groups differ.

Exercise 15.19 (Compute the F statistic). A one-way ANOVA is performed

for $k = 3$ groups with total sample size $N = 18$. The following sums of squares are obtained:

- $SS_{between} = 48$
 - $SS_{within} = 72$
- a) Compute the degrees of freedom for the between-group and within-group variation.
 - b) Compute $MS_{between}$ and MS_{within} .
 - c) Compute the F statistic.

i Solution

- a) The degrees of freedom are

$$df_{between} = k - 1 = 3 - 1 = 2$$

$$df_{within} = N - k = 18 - 3 = 15$$

- b) The mean squares are

$$MS_{between} = \frac{SS_{between}}{df_{between}} = \frac{48}{2} = 24$$

$$MS_{within} = \frac{SS_{within}}{df_{within}} = \frac{72}{15} = 4.8$$

- c) The F statistic is

$$F = \frac{MS_{between}}{MS_{within}} = \frac{24}{4.8} = 5$$

Exercise 15.20 (Interpret an ANOVA table). A one-way ANOVA gives the following table:

Source	Df	Sum Sq	Mean Sq	F value	Pr(>F)
group	2	180.4	90.2	5.63	0.008
Residual	27	432.3	16.0		

- a) What is the null hypothesis?
- b) Is there evidence of a difference among the groups at the 5% significance level?
- c) Which row corresponds to the within-group variation?
- d) Does this result tell which groups differ?

i Solution

a) The null hypothesis is

$$H_0 : \mu_1 = \mu_2 = \mu_3$$

b) Yes. Since the p-value is 0.008 and $0.008 < 0.05$, we reject H_0 .

c) The residual row corresponds to the within-group variation.

d) No. The ANOVA test is an omnibus test. It tells us that at least one group mean differs, but not which groups differ.

Exercise 15.21 (One-way ANOVA in R). The following Hb values (g/L) were observed in three groups of participants:

```
ctrl <- c(145, 152, 149, 143, 150)
diet1 <- c(154, 158, 160, 151, 156)
diet2 <- c(166, 162, 170, 168, 164)
##Combine in a data frame
df <- data.frame(
  Hb = c(ctrl, diet1, diet2),
  group = factor(rep(c("control", "diet1", "diet2"), each=5))
)
```

Perform an ANOVA test to investigate whether the mean Hb levels differ among the three groups.

- State the null and alternative hypotheses.
- Plot the data to visualize the group locations and spread.
- Perform a one-way ANOVA in R.
- Interpret the result.

i Solution

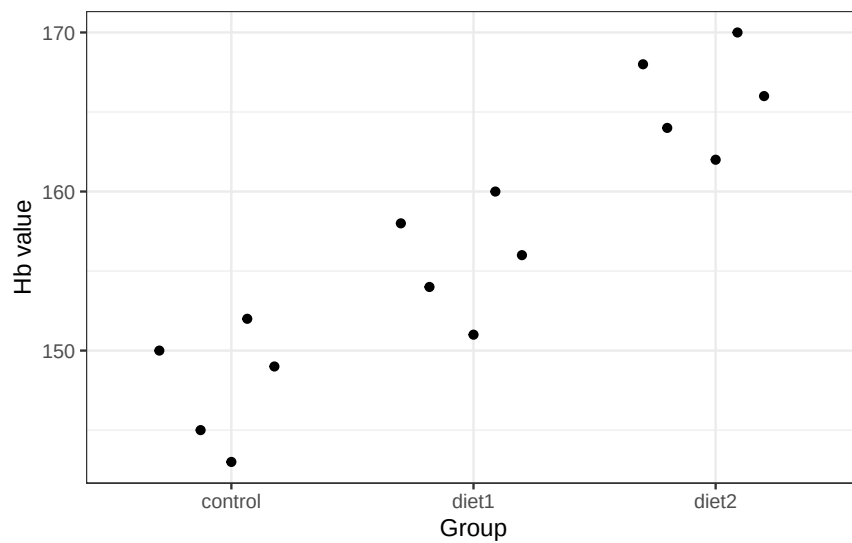
a) The hypotheses are

$$H_0 : \mu_{\text{control}} = \mu_{\text{diet1}} = \mu_{\text{diet2}}$$

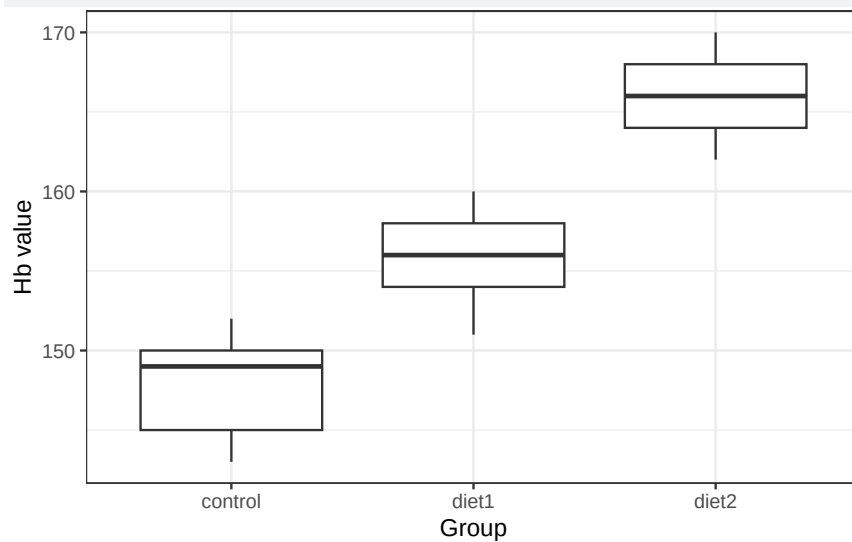
$$H_1 : \text{not all group means are equal}$$

b) Plot the data:

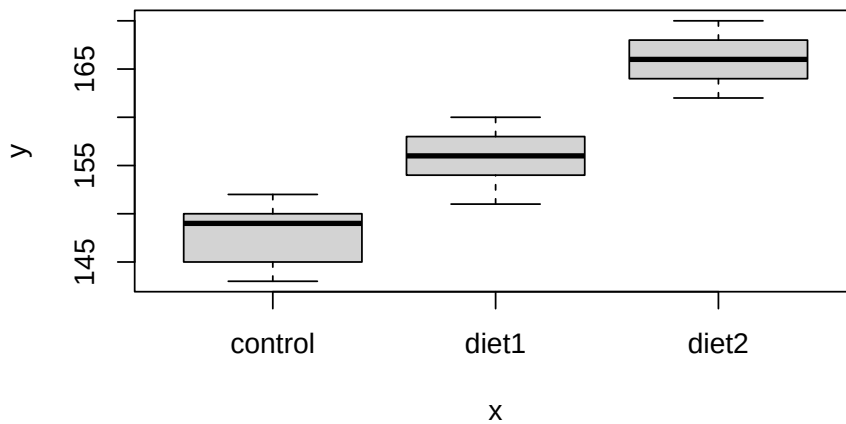
```
## Using a beeswarm like plot
df |> ggplot(aes(x = group, y = Hb)) + geom_quasirandom() +
  theme_bw() + xlab("Group") + ylab("Hb value")
```



```
## Using a boxplot  
df |> ggplot(aes(x = group, y = Hb)) + geom_boxplot() +  
  theme_bw() + xlab("Group") + ylab("Hb value")
```



```
## Can also be done using plot  
plot(df$group, df$Hb)
```

c) Perform one-way ANOVA in R:

```
fit <- aov(Hb ~ group, data = df)
summary(fit)
```

```

              Df Sum Sq Mean Sq F value    Pr(>F)
group           2  832.1    416.1    34.77 1.02e-05 ***
Residuals      12  143.6     12.0
---

```

Signif. codes: 0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

d) The ANOVA p-value is 1.0161078×10^{-5} . Because the p-value is below 0.05, we reject H_0 and conclude that there is evidence that not all group means are equal.

Exercise 15.22 (Post hoc comparisons). Continue with the data from Exercise 15.21.

- Why are post hoc tests needed after a significant ANOVA result?
- Perform Tukey's HSD test in R.
- Which pairs of groups differ significantly? Use the significance level 0.05.

i Solution

a) ANOVA is an omnibus test. A significant ANOVA result tells us that at least one group mean differs, but it does not tell us which groups differ. To identify the specific differences, post hoc pairwise comparisons are needed.

b) Perform Tukey's HSD test:

```
TukeyHSD(fit)
```

```

Tukey multiple comparisons of means
95% family-wise confidence level

```

```
Fit: aov(formula = Hb ~ group, data = df)
```

```

$group
      diff      lwr      upr      p adj
diet1-control  8.0  2.163127 13.83687 0.0085339
diet2-control 18.2 12.363127 24.03687 0.0000070
diet2-diet1   10.2  4.363127 16.03687 0.0014703

```

c) The output shows that the pairwise comparisons between all three pairs are significant, because the adjusted p-values are below 0.05.

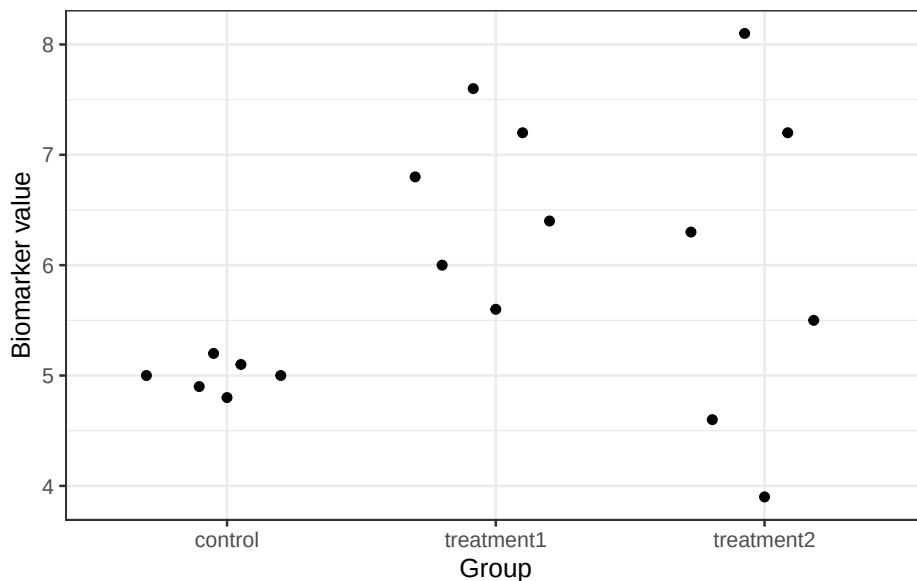
Exercise 15.23 (Kruskal-Wallis test). The following biomarker values were observed in three treatment groups:

```

ctrl1 <- c(4.8, 4.9, 5.0, 5.0, 5.1, 5.2)
trt1  <- c(5.6, 6.0, 6.4, 6.8, 7.2, 7.6)
trt2  <- c(3.9, 4.6, 5.5, 6.3, 7.2, 8.1)

df2 <- data.frame(
  biomarker = c(ctrl1, trt1, trt2),
  group = factor(rep(c("control", "treatment1", "treatment2"),
    ↪     each=6))
)

```



- Why might the Kruskal-Wallis test be a reasonable alternative to one-way ANOVA?
- State the null and alternative hypotheses.
- Perform the Kruskal-Wallis test in R.
- What do you conclude?

i Solution

- a) The Kruskal-Wallis test is a rank-based alternative to one-way ANOVA. It can be useful when the assumptions of ANOVA, such as approximate normality within groups, are doubtful.
- b) The null hypothesis is that the groups have the same distribution, or equivalently that the mean ranks are equal across groups.

H_0 : the group distributions are the same

H_1 : at least one group differs

- c) Perform the Kruskal-Wallis test in R:

```
kruskal.test(biomarker ~ group, data = df2)
```

Kruskal-Wallis rank sum test

data: biomarker by group

Kruskal-Wallis chi-squared = 6.6131, df = 2, p-value = 0.03664

- d) As the p-value is below 0.05, we reject H_0 and conclude that there is evidence that at least one group differs from the others.

Part III

Linear regression

Linear models allows us to answer questions such as:

- is there a relationship between exposure and outcome, e.g. height and weight?
- how strong is the relationship between the two variables?
- what will be a predicted value of the outcome given a new set of exposure values?
- how accurately can we predict outcome?
- which variables are associated with the response, e.g. is it only height that explains weight or could it be height and age that are both associated with the response?

Learning outcomes

- to understand what a linear model is and be familiar with the terminology
- to be able to state linear model in the general vector-matrix notation
- to be able to use the general vector-matrix notation to numerically estimate model parameters
- to be able to use `lm()` function for model fitting, parameter estimation, hypothesis testing and prediction
- to be able to evaluate model fit by interpreting R^2 and $R^2(adj)$ values
- to be able to check model assumptions
- to be able to use `glm()` for extending linear models into generalized linear models

Chapter 16

Introduction to linear models

16.1 Why linear models?

With linear models we can answer questions such as:

- is there a relationship between exposure and outcome, e.g. height and weight?
- how strong is the relationship between the two variables?
- what will be a predicted value of the outcome given a new set of exposure values?
- how accurately can we predict outcome?
- which variables are associated with the response, e.g. is it only height that explains weight or could it be height and age that are both associated with the response?

```
data_diabetes %>%  
  ggplot(aes(x = height, y = weight)) +  
  geom_point() +  
  geom_smooth(method = "lm", se = FALSE) +  
  my.ggtheme +  
  xlab("height [m]") +  
  ylab("weight [kg]")
```

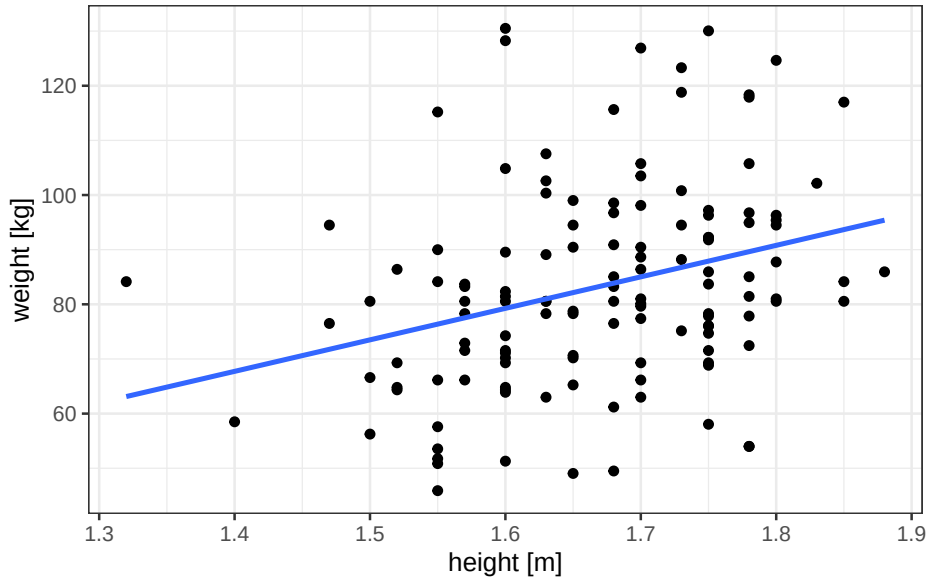


Figure 16.1: Scatter plot of weight vs. height for the 130 study participants based on the diabetes data set collected to understand the prevalence of obesity, diabetes, and other cardiovascular risk factors in central Virginia, USA.

16.2 Statistical vs. deterministic relationship

Relationships in probability and statistics can generally be one of three things: deterministic, random, or statistical:

- a **deterministic** relationship involves an **exact relationship** between two variables, for instance Fahrenheit and Celsius degrees is defined by an equation $Fahrenheit = \frac{9}{5} \cdot Celsius + 32$
- there is **no relationship** between variables in the **random relationship**, for instance number of succulents Olga buys and time of the year as Olga keeps buying succulents whenever she feels like it throughout the entire year
- a **statistical relationship** is a **mixture of deterministic and random relationship**, e.g. the savings that Olga has left in the bank account depend on Olga's monthly salary income (deterministic part) and the money spent on buying succulents (random part)

16.3 What linear models are and are not

- In an linear model we model (explain) the relationship between a single continuous variable Y and one or more variables X . The X variables can be numerical, categorical or a mixture of both.
- One very general form for the model would be:

$$Y = f(X_1, X_2, \dots, X_p) + \epsilon$$

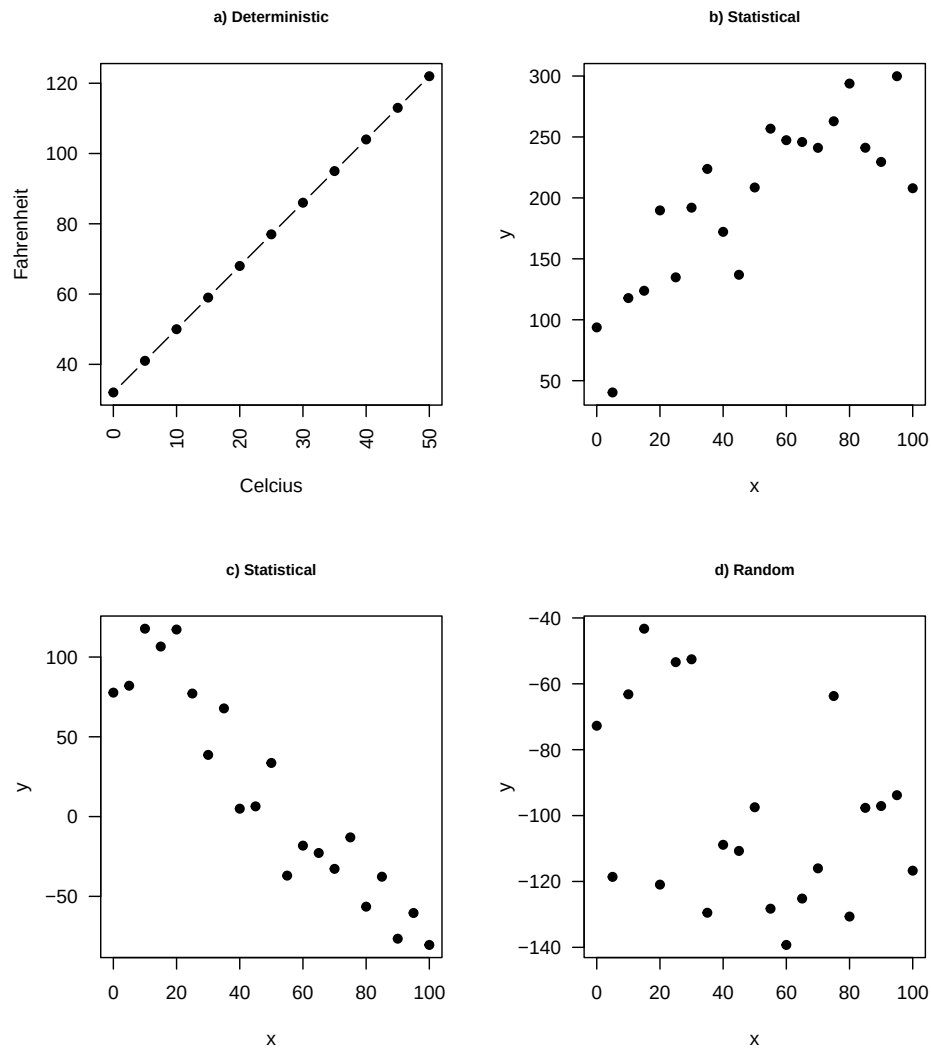


Figure 16.2: Deterministic vs. statistical relationship: a) deterministic: equation exactly describes the relationship between the two variables e.g. Fahrenheit and Celsius relationship, b) statistical relationship between x and y is not perfect (increasing relationship), c) statistical relationship between x and y is not perfect (decreasing relationship), d) random signal

where f is some unknown function and ϵ is the error in this representation.

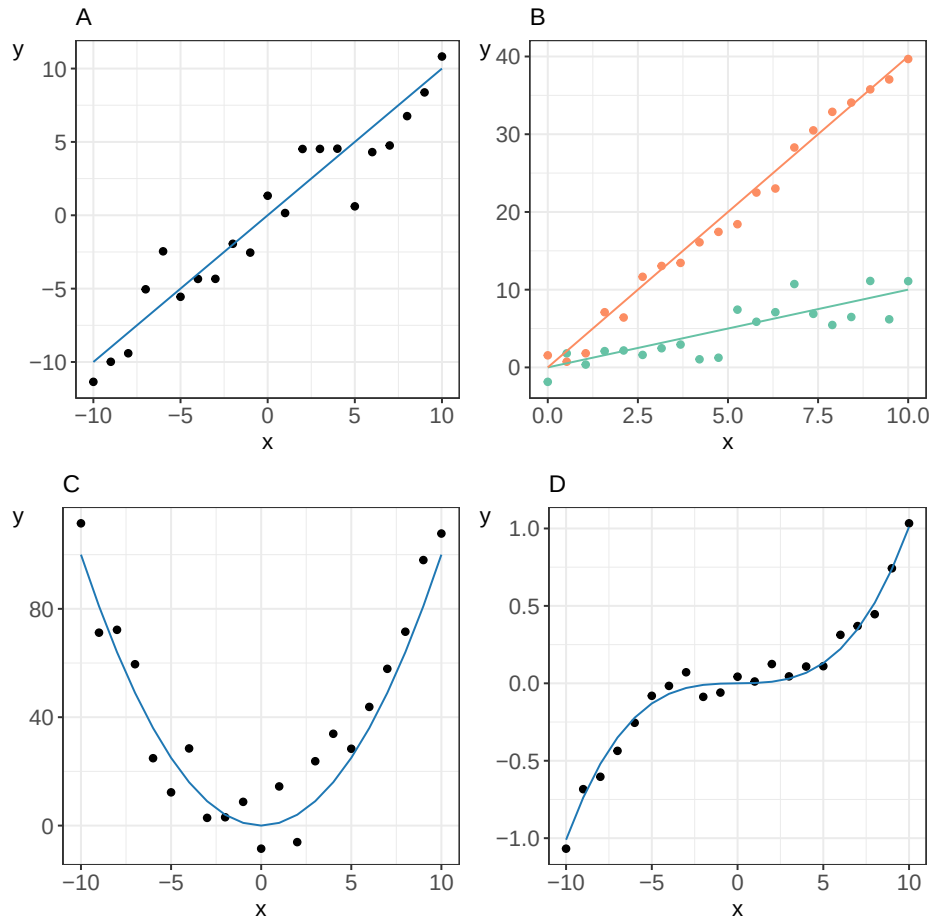
- For instance a **simple linear regression** through the origin is a simple linear model of the form

$$Y_i = \beta \cdot x + \epsilon$$

often used to express a relationship of **one numerical variable to another**, e.g. the calories burnt and the kilometers cycled.

- Linear models can become quite advanced by including **many variables**, e.g. the calories burnt could be a function of the kilometers cycled, road incline and status of a bike, or the **transformation of the variables**, e.g. a function of kilometers cycled squared
- Formally, linear models are a way of describing a response variable in terms of **linear combination** of predictor variables, i.e. expression constructed from a set of terms by multiplying each term by a constant and/or adding the results.
- For instance these are all models that can be constructed using linear combinations of predictors:

Figure 16.3: Examples of a linear models: A) $y_i = x_i + e_i$, B) $x_1 + I_{x_i} + e_i$ C) $y_i = x_i^2 + e_i$, D) $y_i = x + x_i^3 + e_i$ showing that linear models can get more complex and/or capture more than a straight line relationship.



- $Y_i = \alpha + \beta x_i + \gamma y_i + \epsilon_i$

- $Y_i = \alpha + \beta x_i^2 \epsilon$
- $Y_i = \alpha + \beta x_i^2 + \gamma x_i^3 + \epsilon$
- $Y_i = \beta_0 + \beta_1 x_i + \beta_2 x_i^2 + \beta_3 y_i + \beta_4 \sqrt{y_i} + \beta_5 x_i y_i + \epsilon$

vs. an example of a non-linear model where parameter β appears in the exponent of x_i

- $Y_i = \alpha + x_i^\beta + \epsilon$

16.4 Terminology

There are many terms and notations used interchangeably:

- y is being called:
 - response
 - outcome
 - dependent variable
- x is being called:
 - exposure
 - explanatory variable
 - independent variable
 - predictor
 - covariate

16.5 Simple linear regression

- It is used to check the association between **the numerical outcome and one numerical explanatory variable**
- In practice, we are finding the best-fitting straight line to describe the relationship between the outcome and exposure

Example 16.1 (Weight and plasma volume). Let's look at the example data containing body weight (kg) and plasma volume (liters) for eight healthy men to see what the best-fitting straight line is.

Example data:

```
weight <- c(58, 70, 74, 63.5, 62.0, 70.5, 71.0, 66.0) # body
  ↪ weight (kg)
plasma <- c(2.75, 2.86, 3.37, 2.76, 2.62, 3.49, 3.05, 3.12) #
  ↪ plasma volume (liters)
```

The equation for the red line is:

$$Y_i = 0.086 + 0.044 \cdot x_i \quad \text{for } i = 1 \dots 8$$

and in general:

$$Y_i = \alpha + \beta \cdot x_i \quad \text{for } i = 1 \dots n$$

- In other words, by finding the best-fitting straight line we are **building a statistical model** to represent the relationship between plasma volume (Y) and explanatory body weight variable (x)

Figure 16.4: Scatter plot of the data shows that high plasma volume tends to be associated with high weight and *vice versa*.

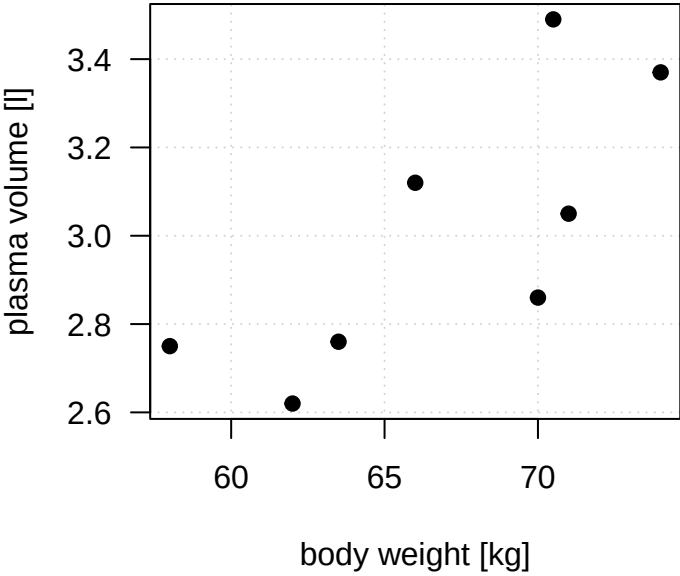
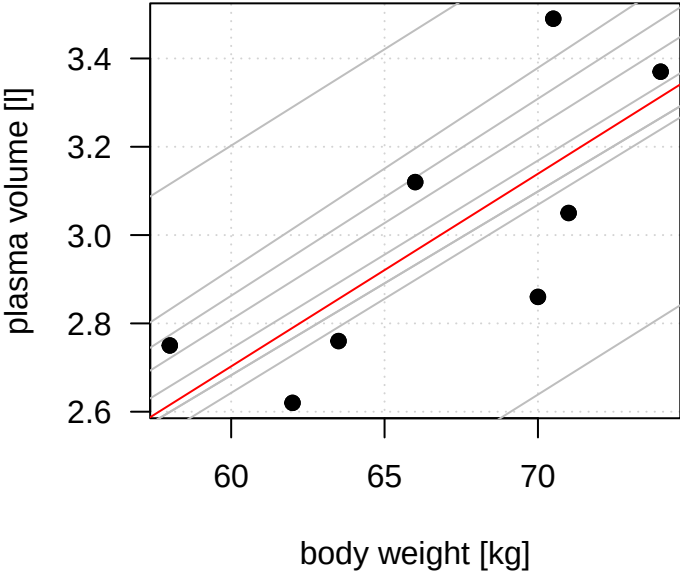


Figure 16.5: Scatter plot of the data shows that high plasma volume tends to be associated with high weight and *vice versa*. Linear regression gives the equation of the straight line (red) that best describes how the outcome changes (increase or decreases) with a change of exposure variable



- If we were to use our model $Y_i = 0.086 + 0.044 \cdot x_i$ to find plasma volume given a weight of 58 kg (our first observation, $i = 1$), we would notice that we would get $Y = 0.086 + 0.044 \cdot 58 = 2.638$, not exactly 2.75 as we have for our first man in our dataset that we started with, i.e. $2.75 - 2.638 = 0.112 \neq 0$.
- We thus add to the above equation an **error term** to account for this and now we can write our **simple regression model** more formally as:

$$Y_i = \alpha + \beta \cdot x_i + \epsilon_i \quad (16.1)$$

where:

- x : is called: exposure variable, explanatory variable, dependent variable, predictor, covariate
- y : is called: response, outcome, dependent variable
- α and β are **model coefficients**
- and ϵ_i is an **error terms**

16.6 Least squares

- in the above “**body weight - plasma volume**” example, the values of α and β have just appeared
- in practice, α and β values are unknown and we use data to **estimate these coefficients**, noting the estimates with a **hat**, $\hat{\alpha}$ and $\hat{\beta}$
- **least squares** is one of the methods of parameters estimation, i.e. finding $\hat{\alpha}$ and $\hat{\beta}$

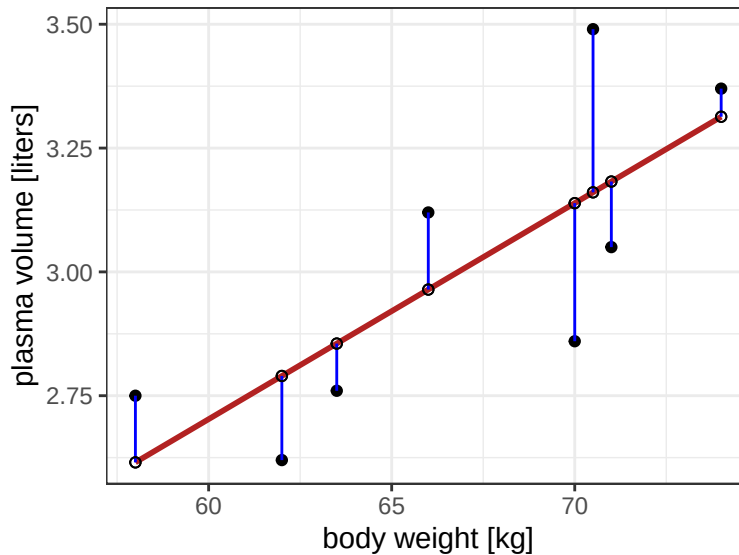


Figure 16.6: Scatter plot of the data shows that high plasma volume tends to be associated with high weight and *vice versa*. Linear regression gives the equation of the straight line (red) that best describes how the outcome changes with a change of exposure variable. Blue lines represent error terms, the vertical distances to the regression line

Let $\hat{y}_i = \hat{\alpha} + \hat{\beta}x_i$ be the prediction y_i based on the i -th value of x :

- Then $\epsilon_i = y_i - \hat{y}_i$ represents the i -th **residual**, i.e. the difference between the i -th observed response value and the i -th response value that is predicted by the linear model

- RSS, the **residual sum of squares** is defined as:

$$RSS = \epsilon_1^2 + \epsilon_2^2 + \dots + \epsilon_n^2$$

or equivalently as:

$$RSS = (y_1 - \hat{\alpha} - \hat{\beta}x_1)^2 + (y_2 - \hat{\alpha} - \hat{\beta}x_2)^2 + \dots + (y_n - \hat{\alpha} - \hat{\beta}x_n)^2$$

- the least squares approach chooses $\hat{\alpha}$ and $\hat{\beta}$ **to minimize the RSS**. With some calculus, a good video explanation for the interested ones is here, we get Theorem 16.1

Theorem 16.1 (Least squares estimates for a simple linear regression).

$$\hat{\beta} = \frac{S_{xy}}{S_{xx}}$$

$$\hat{\alpha} = \bar{y} - \frac{S_{xy}}{S_{xx}} \cdot \bar{x}$$

where:

- $\bar{x}$: mean value of x
- $\bar{y}$: mean value of y
- S_{xx} : sum of squares of X defined as $S_{xx} = \sum_{i=1}^n (x_i - \bar{x})^2$
- S_{yy} : sum of squares of Y defined as $S_{yy} = \sum_{i=1}^n (y_i - \bar{y})^2$
- S_{xy} : sum of products of X and Y defined as $S_{xy} = \sum_{i=1}^n (x_i - \bar{x})(y_i - \bar{y})$

Example 16.2 (Least squares). Let's try least squares method to find coefficient estimates in the “**body weight and plasma volume example**”

```
# initial data
weight <- c(58, 70, 74, 63.5, 62.0, 70.5, 71.0, 66.0) # body
↪ weight (kg)
plasma <- c(2.75, 2.86, 3.37, 2.76, 2.62, 3.49, 3.05, 3.12) #
↪ plasma volume (liters)

# rename variables for convenience
x <- weight
y <- plasma

# mean values of x and y
x.bar <- mean(x)
y.bar <- mean(y)

# Sum of squares
Sxx <- sum((x - x.bar)^2)
Sxy <- sum((x - x.bar)*(y - y.bar))
```



```
# Coefficient estimates
beta.hat <- Sxy / Sxx
alpha.hat <- y.bar - Sxy/Sxx*x.bar

# Print estimated coefficients alpha and beta
print(alpha.hat)
```

```
[1] 0.08572428
```

```
print(beta.hat)
```

```
[1] 0.04361534
```

In R we can use `lm()`, the built-in function, to fit a linear regression model and we can replace the above code with one line

```
lm(plasma ~ weight)
```

Call:

```
lm(formula = plasma ~ weight)
```

Coefficients:

(Intercept)	weight
0.08572	0.04362

16.7 The linear model describes an expected value

Another way to think about a linear regression model is that it describes the **expected value of the outcome** for a given value of the predictor.

For simple linear regression,

$$E(Y \mid X = x) = \alpha + \beta x.$$

This means that for individuals with the same value $X = x$, their observed outcomes Y do not all have to lie exactly on the regression line. Instead, the line describes their expected, or average, outcome.

An individual observation can therefore be written as

$$Y_i = E(Y_i \mid X_i = x_i) + \epsilon_i$$

or, for the linear model,

$$Y_i = \alpha + \beta x_i + \epsilon_i.$$

The error term ϵ_i represents how much an individual observation differs from its expected value.

When we fit the model from a sample, we estimate this expected value by

$$\hat{Y}_i = \hat{\alpha} + \hat{\beta}x_i,$$

and the difference between the observed and fitted value is the **residual**:

$$e_i = Y_i - \hat{Y}_i.$$

Thus, the regression line describes how the **expected outcome changes with the predictor**, while individual observations vary around that expectation.

16.8 Intercept and Slope

- Linear regression gives us estimates of model coefficient $Y_i = \alpha + \beta x_i + \epsilon_i$
- α is known as the **intercept**
- β is known as the **slope**

16.9 From simple regression to more general linear models

So far, we have considered **simple linear regression**, where a continuous outcome Y is described using one numerical predictor X :

$$Y_i = \beta_0 + \beta_1 x_i + \epsilon_i.$$

The same framework can be extended to include **more than one predictor**. For example, with two predictors:

$$Y_i = \beta_0 + \beta_1 x_{1i} + \beta_2 x_{2i} + \epsilon_i.$$

More generally, a linear model with p predictors can be written as:

$$Y_i = \beta_0 + \beta_1 x_{1i} + \beta_2 x_{2i} + \cdots + \beta_p x_{pi} + \epsilon_i.$$

Each coefficient describes the contribution of one predictor to the expected outcome, **while keeping the other predictors in the model constant**.

For example, if body weight is modelled using height and age,

$$E(\text{weight} \mid \text{height, age}) = \beta_0 + \beta_1 \text{height} + \beta_2 \text{age},$$

then β_1 describes the expected difference in weight associated with a one-unit difference in height **for individuals of the same age**.

Predictors in a linear model do not have to be numerical. They can also be **categorical**, such as treatment group, sex, genotype or experimental condition.

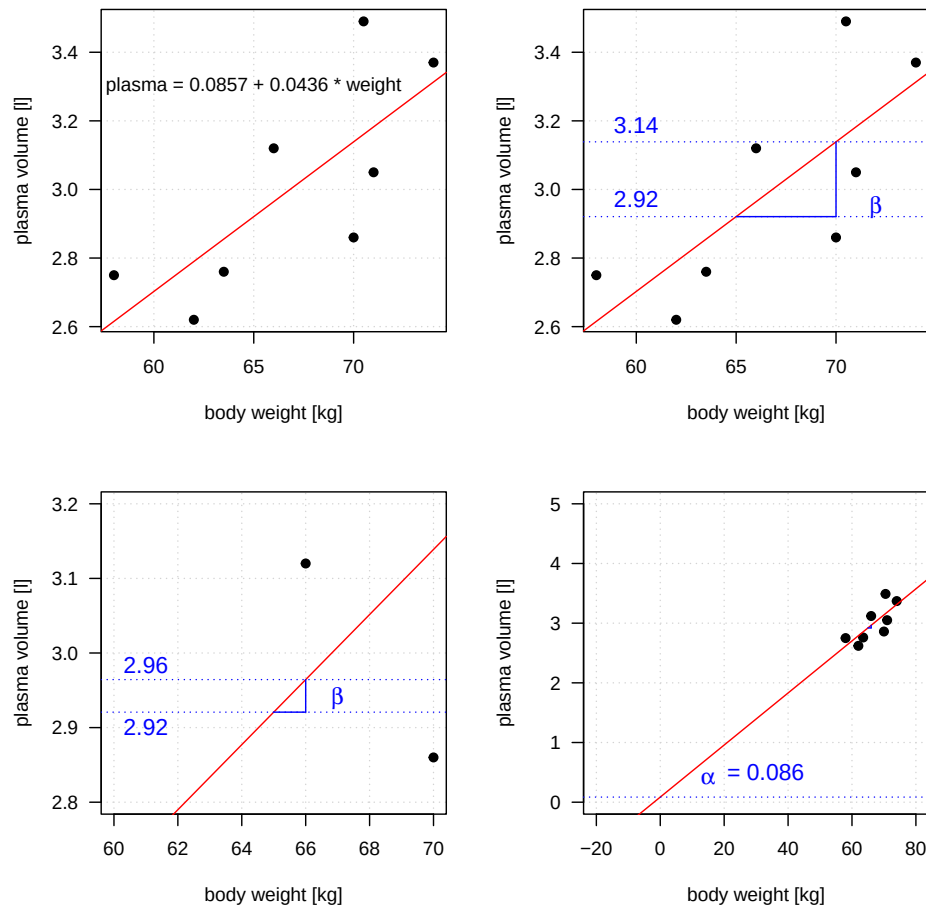


Figure 16.7: Scatter plot of the data shows that high plasma volume tends to be associated with high weight and *vice versa*. Linear regression gives the equation of the straight line that best describes how the outcome changes (increase or decreases) with a change of exposure variable (in red)

This allows the same linear-model framework to describe both relationships between continuous variables and differences between groups.

We will look at how numerical and categorical predictors, and combinations of several predictors, are used in practice in the next session.

16.10 Hypothesis testing

Is there a relationship between the response and the predictor?

- the calculated $\hat{\alpha}$ and $\hat{\beta}$ are **estimates of the population values** of the intercept and slope and are therefore subject to **sampling variation**
- their precision is measured by their **estimated standard errors**, $\text{e.s.e}(\hat{\alpha})$ and $\text{e.s.e}(\hat{\beta})$
- these estimated standard errors are used in **hypothesis testing** and in constructing **confidence and prediction intervals**

The **most common hypothesis test** involves testing the null hypothesis of:

- H_0 : There is no relationship between X and Y
- versus the **alternative hypothesis** H_a : there is some relationship between X and Y

Mathematically, this corresponds to testing:

- $H_0 : \beta = 0$
- versus $H_a : \beta \neq 0$
- since if $\beta = 0$ then the model $Y_i = \alpha + \beta x_i + \epsilon_i$ reduces to $Y = \alpha + \epsilon_i$

$$\frac{\hat{\beta} - \beta}{\text{e.s.e}(\hat{\beta})} \sim t(n - p)$$

Under the null hypothesis $H_0 : \beta = 0$

- n is number of observations
- p is number of model parameters
- $\frac{\hat{\beta} - \beta}{\text{e.s.e}(\hat{\beta})}$ is the ratio of the departure of the estimated value of a parameter, $\hat{\beta}$, from its hypothesized value, β , to its standard error, called **t-statistics**
- the **t-statistics** follows Student's t distribution with $n - p$ degrees of freedom

Example 16.3 (Hypothesis testing). Let's look again at our example data. This time we will not only fit the linear regression model but also look a bit more closely at the R **summary** of the model

```
weight <- c(58, 70, 74, 63.5, 62.0, 70.5, 71.0, 66.0) # body
↪ weight (kg)
plasma <- c(2.75, 2.86, 3.37, 2.76, 2.62, 3.49, 3.05, 3.12) #
↪ plasma volume (liters)

model <- lm(plasma ~ weight)
print(summary(model))
```

```

Call:
lm(formula = plasma ~ weight)

Residuals:
    Min       1Q   Median       3Q      Max
-0.27880 -0.14178 -0.01928  0.13986  0.32939

Coefficients:
              Estimate Std. Error t value Pr(>|t|)
(Intercept)  0.08572     1.02400   0.084   0.9360
weight       0.04362     0.01527   2.857   0.0289 *
---
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

Residual standard error: 0.2188 on 6 degrees of freedom
Multiple R-squared:  0.5763,    Adjusted R-squared:  0.5057
F-statistic:  8.16 on 1 and 6 DF,  p-value: 0.02893

```

- Under **Estimate** we see estimates of our model coefficients, $\hat{\alpha}$ (intercept) and $\hat{\beta}$ (slope, here weight), followed by their estimated standard errors, **Std. Errors**
- If we were to test if there is an **association between weight and plasma volume** we would write under the null hypothesis $H_0 : \beta = 0$

$$\frac{\hat{\beta} - \beta}{e.s.e(\hat{\beta})} = \frac{0.04362 - 0}{0.01527} = 2.856582$$

- and we would **compare t-statistics** to Student's **t distribution** with $n - p = 8 - 2 = 6$ degrees of freedom (as we have 8 observations and two model parameters, α and β)
- we can use **Student's t distribution table** or **R code** to obtain the associated *P*-value

```
2*pt(2.856582, df=6, lower=F)
```

```
[1] 0.02893095
```

- here the observed t-statistics is large and therefore yields a small *P*-value, meaning that **there is sufficient evidence to reject null hypothesis in favor of the alternative** and conclude that there is a significant association between weight and plasma volume

16.11 Overall model test: the F-test

The `summary()` output also reports an **F-statistic** and its corresponding *P*-value. The F-test evaluates the **linear model as a whole**.

For a linear model with several predictors,

$$Y_i = \beta_0 + \beta_1 x_{1i} + \cdots + \beta_{p-1} x_{(p-1)i} + \epsilon_i,$$

the overall null hypothesis is

$$H_0 : \beta_1 = \beta_2 = \cdots = \beta_{p-1} = 0.$$

In other words, under the null hypothesis none of the predictors contributes to explaining variation in the response, and the model reduces to an intercept-only model.

The **F-statistic** compares the variation explained by the regression model with the variation left unexplained by the model:

$$F = \frac{\text{explained variation per degree of freedom}}{\text{unexplained variation per degree of freedom}}.$$

Under the null hypothesis, the F-statistic follows an **F-distribution** with

$$(p - 1, n - p)$$

degrees of freedom, where n is the number of observations and p is the number of estimated model parameters, including the intercept.

For our plasma-volume example,

```
weight <- c(58, 70, 74, 63.5, 62.0, 70.5, 71.0, 66.0)
plasma <- c(2.75, 2.86, 3.37, 2.76, 2.62, 3.49, 3.05, 3.12)

model <- lm(plasma ~ weight)
summary(model)
```

Call:

```
lm(formula = plasma ~ weight)
```

Residuals:

	Min	1Q	Median	3Q	Max
	-0.27880	-0.14178	-0.01928	0.13986	0.32939

Coefficients:

	Estimate	Std. Error	t value	Pr(> t)
(Intercept)	0.08572	1.02400	0.084	0.9360
weight	0.04362	0.01527	2.857	0.0289 *

Signif. codes: 0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

Residual standard error: 0.2188 on 6 degrees of freedom

Multiple R-squared: 0.5763, Adjusted R-squared: 0.5057

F-statistic: 8.16 on 1 and 6 DF, p-value: 0.02893

Here we have $n = 8$ observations and $p = 2$ model parameters, the intercept α and slope β . Therefore,

$$F \sim F_{1,6}$$

under the null hypothesis.

The P-value is the probability, under H_0 , of observing an F-statistic at least as large as the one obtained:

$$P(F_{1,6} \geq 8.16) \approx 0.029.$$

Thus, there is sufficient evidence to reject the null hypothesis and conclude that the model containing weight explains more variation in plasma volume than an intercept-only model.

In simple linear regression, with only one predictor, the overall F-test tests exactly the same hypothesis as the t-test for the slope:

$$H_0 : \beta = 0.$$

In this case,

$$F = t^2,$$

so the two tests give the same P-value. With several predictors, the F-test is useful because it tests whether the predictors are jointly associated with the response.

16.12 Vector-matrix notations

While in simple linear regression it is feasible to arrive at the parameters estimates using calculus in more realistic settings of **multiple regression**, with more than one explanatory variable in the model, it is **more efficient to use vectors and matrices to define the regression model**.

Let's **rewrite** our simple linear regression model $Y_i = \alpha + \beta_i + \epsilon_i \quad i = 1, \dots, n$ **into vector-matrix notation** in **6 steps**.

1. First we rename our α to β_0 and β to β_1 as it is easier to keep tracking the number of model parameters this way
2. Then we notice that we actually have n equations such as:

$$y_1 = \beta_0 + \beta_1 x_1 + \epsilon_1$$

$$y_2 = \beta_0 + \beta_1 x_2 + \epsilon_2$$

$$y_3 = \beta_0 + \beta_1 x_3 + \epsilon_3$$

...

$$y_n = \beta_0 + \beta_1 x_n + \epsilon_n$$

3. We group all Y_i and ϵ_i into column vectors: $\mathbf{Y} = \begin{bmatrix} y_1 \\ y_2 \\ \vdots \\ y_n \end{bmatrix}$ and $\epsilon = \begin{bmatrix} \epsilon_1 \\ \epsilon_2 \\ \vdots \\ \epsilon_n \end{bmatrix}$
4. We stack two parameters β_0 and β_1 into another column vector:

$$\beta = \begin{bmatrix} \beta_0 \\ \beta_1 \end{bmatrix}$$

5. We append a vector of ones with the single predictor for each i and create a matrix with two columns called **design matrix**

$$\mathbf{X} = \begin{bmatrix} 1 & x_1 \\ 1 & x_2 \\ \vdots & \vdots \\ 1 & x_n \end{bmatrix}$$

6. We write our linear model in a vector-matrix notations as:

$$\mathbf{Y} = \mathbf{X}\beta + \epsilon$$

Definition 16.1 (vector matrix form of the linear model). The vector-matrix representation of a linear model with $p - 1$ predictors can be written as

$$\mathbf{Y} = \mathbf{X}\beta + \epsilon$$

where:

- $\mathbf{Y}$ is $n \times 1$ vector of observations
- $\mathbf{X}$ is $n \times p$ **design matrix**
- β is $p \times 1$ vector of parameters
- ϵ is $n \times 1$ vector of vector of random errors, indepedent and identically distributed (i.i.d) $N(0, \sigma^2)$

In full, the above vectors and matrix have the form:

$$\mathbf{Y} = \begin{bmatrix} y_1 \\ y_2 \\ \vdots \\ y_n \end{bmatrix} \quad \beta = \begin{bmatrix} \beta_0 \\ \beta_1 \\ \vdots \\ \beta_p \end{bmatrix} \quad \epsilon = \begin{bmatrix} \epsilon_1 \\ \epsilon_2 \\ \vdots \\ \epsilon_n \end{bmatrix} \quad \mathbf{X} = \begin{bmatrix} 1 & x_{1,1} & \dots & x_{1,p-1} \\ 1 & x_{2,1} & \dots & x_{2,p-1} \\ \vdots & \vdots & \ddots & \vdots \\ 1 & x_{n,1} & \dots & x_{n,p-1} \end{bmatrix}$$

Theorem 16.2 (Least squares in vector-matrix notation). *The least squares estimates for a linear regression of the form:*

$$\mathbf{Y} = \mathbf{X}\beta + \epsilon$$

is given by:

$$\hat{\beta} = (\mathbf{X}^T \mathbf{X})^{-1} \mathbf{X}^T \mathbf{Y}$$

Example 16.4 (vector-matrix-notation). Following the above definition we can write the “**weight - plasma volume model**” as:

$$\mathbf{Y} = \mathbf{X}\beta + \epsilon$$

where:

$$\mathbf{Y} = \begin{bmatrix} 2.75 \\ 2.86 \\ 3.37 \\ 2.76 \\ 2.62 \\ 3.49 \\ 3.05 \\ 3.12 \end{bmatrix} \quad \beta = \begin{bmatrix} \beta_0 \\ \beta_1 \end{bmatrix} \quad \epsilon = \begin{bmatrix} \epsilon_1 \\ \epsilon_2 \\ \vdots \\ \epsilon_8 \end{bmatrix} \quad \mathbf{X} = \begin{bmatrix} 1 & 58.0 \\ 1 & 70.0 \\ 1 & 74.0 \\ 1 & 63.5 \\ 1 & 62.0 \\ 1 & 70.5 \\ 1 & 71.0 \\ 1 & 66.0 \end{bmatrix}$$

and we can estimate model parameters using $\hat{\beta} = (\mathbf{X}^T \mathbf{X})^{-1} \mathbf{X}^T \mathbf{Y}$.

We can do it by hand or in R as follows:

```
n <- length(plasma) # no. of observation
Y <- as.matrix(plasma, ncol=1)
X <- cbind(rep(1, length=n), weight)
X <- as.matrix(X)

# print Y and X to double-check that the format is according to
# the definition
print(Y)
```

```
      [,1]
[1,] 2.75
[2,] 2.86
[3,] 3.37
[4,] 2.76
[5,] 2.62
[6,] 3.49
[7,] 3.05
[8,] 3.12
```

```
print(X)
```

```
      weight
[1,] 1    58.0
[2,] 1    70.0
```

```
[3,] 1 74.0
[4,] 1 63.5
[5,] 1 62.0
[6,] 1 70.5
[7,] 1 71.0
[8,] 1 66.0
```

```
# least squares estimate
# solve() finds inverse of matrix
beta.hat <- solve(t(X)%*%X)%*%t(X)%*%Y
print(beta.hat)
```

```
      [,1]
0.08572428
weight 0.04361534
```

16.13 Confidence intervals and prediction intervals

- when we estimate coefficients we can also find their **confidence intervals**, typically 95% confidence intervals, i.e. a range of values that contain the true unknown value of the parameter
- we can also use linear regression models to predict the response value given a new observation and find **prediction intervals**. Here, we look at any specific value of x_i , and find an interval around the predicted value y'_i for x_i such that there is a 95% probability that the real value of y (in the population) corresponding to x_i is within this interval

Example 16.5 (Prediction and intervals). Let's:

- find confidence intervals for our coefficient estimates
- predict plasma volume for a men weighting 60 kg
- find prediction interval
- plot original data, fitted regression model, predicted observation and prediction interval

```
# fit regression model
model <- lm(plasma ~ weight)
print(summary(model))
```

Call:

```
lm(formula = plasma ~ weight)
```

Residuals:

	Min	1Q	Median	3Q	Max
	-0.27880	-0.14178	-0.01928	0.13986	0.32939

Coefficients:

	Estimate	Std. Error	t value	Pr(> t)
(Intercept)	0.08572	1.02400	0.084	0.9360

```
weight      0.04362    0.01527    2.857    0.0289 *
```

```
---
```

```
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
```

```
Residual standard error: 0.2188 on 6 degrees of freedom
```

```
Multiple R-squared:  0.5763,    Adjusted R-squared:  0.5057
```

```
F-statistic:  8.16 on 1 and 6 DF,  p-value: 0.02893
```

```
# find confidence intervals for the model coefficients
confint(model)
```

```
                2.5 %      97.5 %
(Intercept) -2.419908594 2.59135716
weight      0.006255005 0.08097567
```

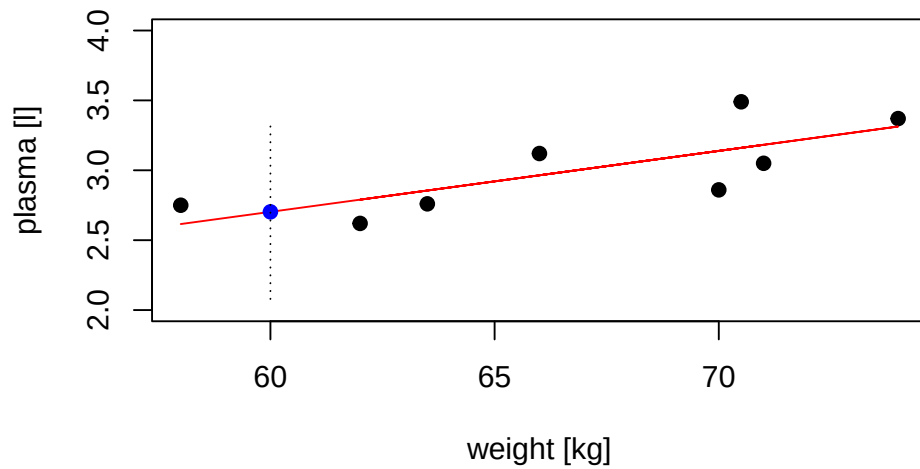
```
# predict plasma volume for a new observation of 60 kg
# we have to create data frame with a variable name matching the
  ↪ one used to build the model
new.obs <- data.frame(weight = 60)
predict(model, newdata = new.obs)
```

```
1
2.702645
```

```
# find prediction intervals
prediction.interval <- predict(model, newdata = new.obs,
  ↪ interval = "prediction")
print(prediction.interval)
```

```
      fit      lwr      upr
1 2.702645 2.079373 3.325916
```

```
# plot the original data, fitted regression and predicted value
plot(weight, plasma, pch=19, xlab="weight [kg]", ylab="plasma
  ↪ [l]", ylim=c(2,4))
lines(weight, model$fitted.values, col="red") # fitted model in
  ↪ red
points(new.obs, predict(model, newdata = new.obs), pch=19,
  ↪ col="blue") # predicted value at 60kg
segments(60, prediction.interval[2], 60, prediction.interval[3],
  ↪ lty = 3) # add prediction interval
```



16.14 Assessing model fit

- Earlier we learned how to estimate parameters in a linear model using least squares estimation.
- Now we will consider how to assess the goodness of fit of a model, i.e. how well does the model explain our data.
- We do that by calculating the amount of variability in the response that is explained by the model.

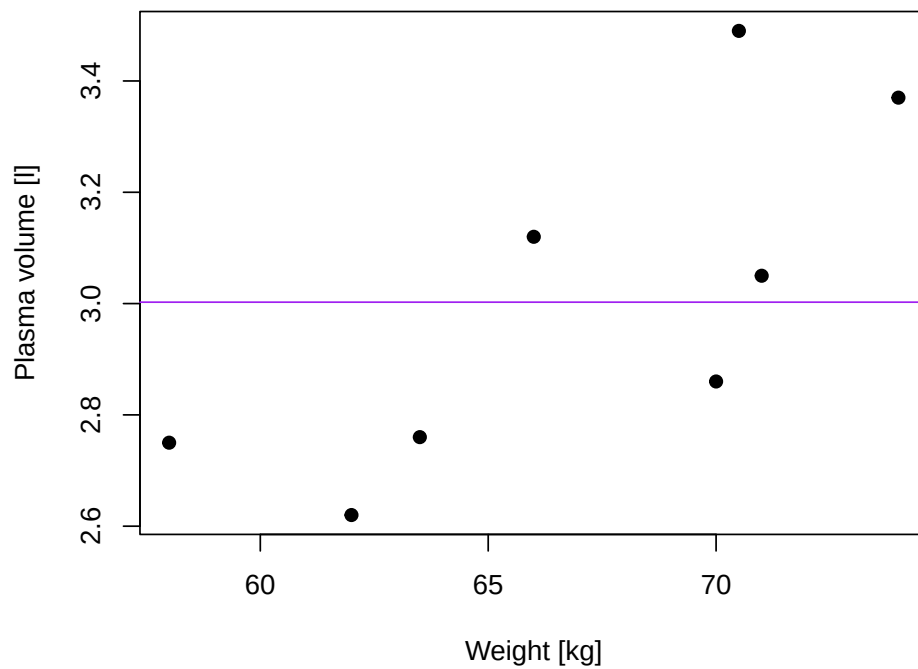
R^2 : summary of the fitted model

- considering a simple linear regression, the simplest model, **Model 0**, we could consider fitting is

$$Y_i = \beta_0 + \epsilon_i$$

that corresponds to a line that runs through the data but lies parallel to the horizontal axis

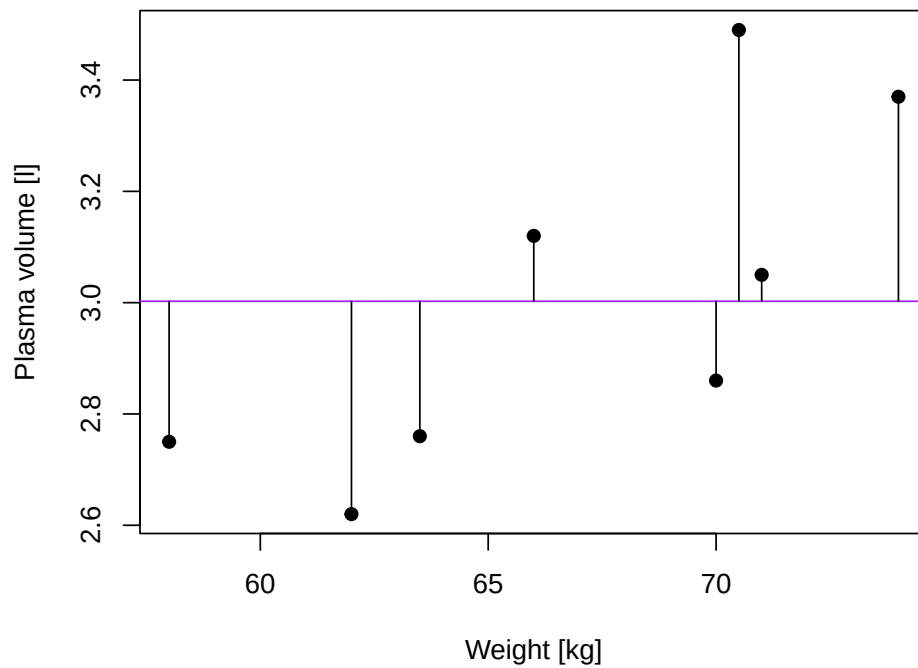
- in our plasma volume example that would correspond to the mean value of plasma volume being predicted for any value of weight (in purple)



- TSS, denoted **Total corrected sum-of-squares** is the residual sum-of-squares for Model 0

$$S(\hat{\beta}_0) = TSS = \sum_{i=1}^n (y_i - \bar{y})^2 = S_{yy}$$

corresponding the to the sum of squared distances to the purple line



- Fitting **Model 1** of the form

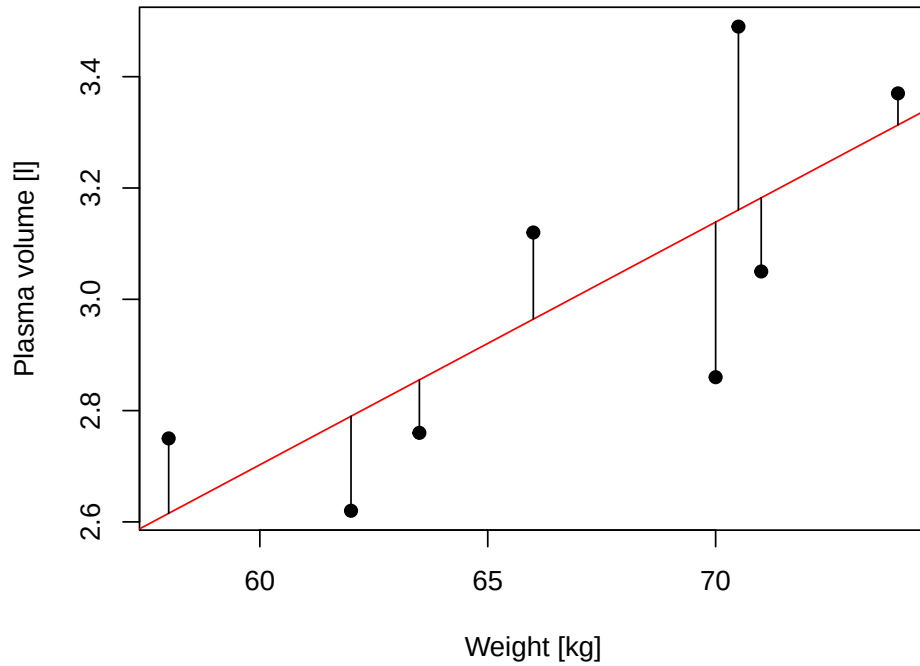
$$Y_i = \beta_0 + \beta_1 x + \epsilon_i$$

we have earlier defined

- **RSS**, the residual sum-of-squares as:

$$RSS = \sum_{i=1}^n (y_i - \{\hat{\beta}_0 + \hat{\beta}_1 x_{1i} + \dots + \hat{\beta}_p x_{pi}\}) = \sum_{i=1}^n (y_i - \hat{y}_i)^2$$

- that corresponds to the squared distances between the observed values $y_i, \dots, y_n$ to fitted values $\hat{y}_1, \dots, \hat{y}_n$, i.e. distances to the red fitted line



Definition 16.2 (R^2). A simple but useful measure of model fit is given by

$$R^2 = 1 - \frac{RSS}{TSS}$$

where:

- RSS is the residual sum-of-squares for Model 1, the fitted model of interest
- TSS is the sum of squares of the **null model**
- R^2 quantifies how much of a drop in the residual sum-of-squares is accounted for by fitting the proposed model
- R^2 is also referred as **coefficient of determination**
- It is expressed on a scale, as a proportion (between 0 and 1) of the total variation in the data
- Values of R^2 approaching 1 indicate the model to be a good fit
- Values of R^2 less than 0.5 suggest that the model gives rather a poor fit to the data

R^2 and correlation coefficient

Theorem 16.3 (R^2). *In the case of simple linear regression:*

Model 1: $Y_i = \beta_0 + \beta_1 x + \epsilon_i$

$$R^2 = r^2$$

where:

- R^2 is the coefficient of determination
- r^2 is the sample correlation coefficient

 $R^2(adj)$

- in the case of multiple linear regression, where there is more than one explanatory variable in the model
- we are using the adjusted version of R^2 to assess the model fit
- as the number of explanatory variables increase, R^2 also increases
- $R^2(adj)$ takes this into account, i.e. adjusts for the fact that there is more than one explanatory variable in the model

Theorem 16.4 ($R^2(adj)$). *For any multiple linear regression*

$$Y_i = \beta_0 + \beta_1 x_{1i} + \dots + \beta_{p-1} x_{(p-1)i} + \epsilon_i$$

$R^2(adj)$ is defined as

$$R^2(adj) = 1 - \frac{\frac{RSS}{n-p-1}}{\frac{TSS}{n-1}}$$

where

- p is the number of independent predictors, i.e. the number of variables in the model, excluding the constant

$R^2(adj)$ can also be calculated from R^2 :

$$R^2(adj) = 1 - (1 - R^2) \frac{n-1}{n-p-1}$$

16.15 The assumptions of a linear model

Up until now we were fitting models and discussed how to assess the model fit. Before making use of a fitted model for explanation or prediction, it is wise to check that the model provides an adequate description of the data. Informally we have been using box plots and scatter plots to look at the data. There are however formal definitions of the assumptions.

Assumption A: The deterministic part of the model captures all the non-random structure in the data

- This implies that the **mean of the errors** ϵ_i is zero.
- It applies only over the range of explanatory variables.

Assumption B: the scale of variability of the errors is constant at all values of the explanatory variables

- Practically we are looking at whether the observations are equally spread on both side of the regression line.

Assumption C: the errors are independent

- Broadly speaking this means that knowledge of errors attached to one observation does not give us any information about the error attached to another.

Assumptions D: the errors are normally distributed

- This will allow us to describe the variation in the model's parameters estimates and therefore make inferences about the population from which our sample was taken.

Assumption E: the values of the explanatory variables are recorded without error

- This one is not possible to check via examining the data, instead we have to consider the nature of the experiment.

16.15.1 Checking assumptions

Residuals, $\hat{\epsilon}_i = y_i - \hat{y}_i$ are the **main ingredient to check model assumptions**. We use plots such as:

1. Histograms or normal probability plots of $\hat{\epsilon}_i$
 - useful to check the assumption of normality
2. Plots of $\hat{\epsilon}_i$ versus the fitted values $\hat{y}_i$
 - used to detect changes in error variance
 - used to check if the mean of the errors is zero
3. Plots of $\hat{\epsilon}_i$ vs. an explanatory variable x_{ij}
 - this helps to check that the variable x_j has a linear relationship with the response variable
4. Plots of $\hat{\epsilon}_i$ vs. an explanatory variable x_{kj} that is **not** in the model
 - this helps to check whether the additional variable x_k might have a relationship with the response variable
4. Plots of $\hat{\epsilon}_i$ in the order of the observations were collected
 - this is useful to check whether errors might be correlated over time

Let's fit a simple model to predict BMI given **waist** for the diabetes study and see if the model meets the assumptions of linear models.

```
# fit simple linear regression model
model <- lm(BMI ~ waist, data = data_diabetes)

# plot diagnostic plots of the linear model
# by default plot(model) calls four diagnostics plots
# par() divides plot window in 2 x 2 grid
```



```
par(mfrow=c(2,2))
plot(model)
```

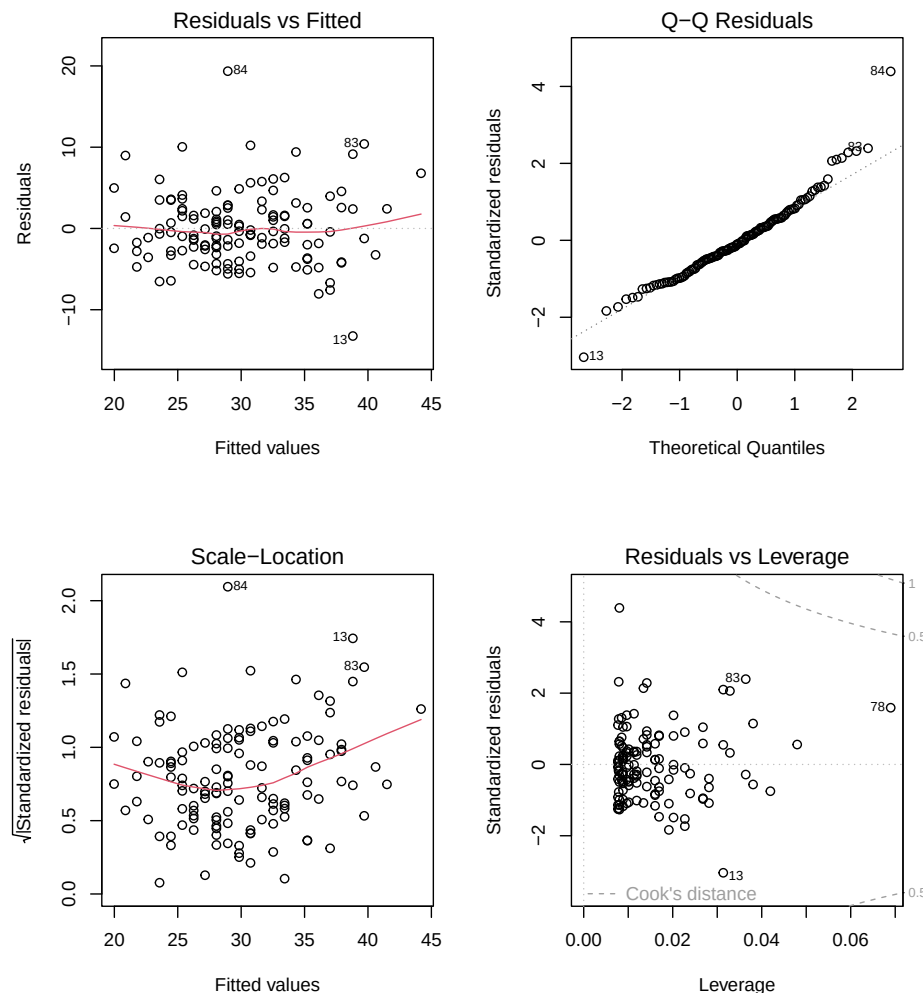


Figure 16.8: Default diagnostic residual plots based on the `lm()` model used to assess whether the assumptions of linear models are met. Simple regression to model BMI with waist explanatory variable.

- The residual plots provides examples of a situation where the assumptions appear to be met.
- The linear regression appears to describe data quite well.
- There is no obvious trend of any kind in the residuals vs. fitted values (the shape is scattered) with potential few outliers that we may want to decide to exclude later.
- Points lie reasonably well along the line in the normal probability plot, hence normality appears to be met.

Examples of assumptions not being met

The following simulated examples show some typical patterns that may appear in diagnostic plots when linear model assumptions are violated.

Typical warning signs include:

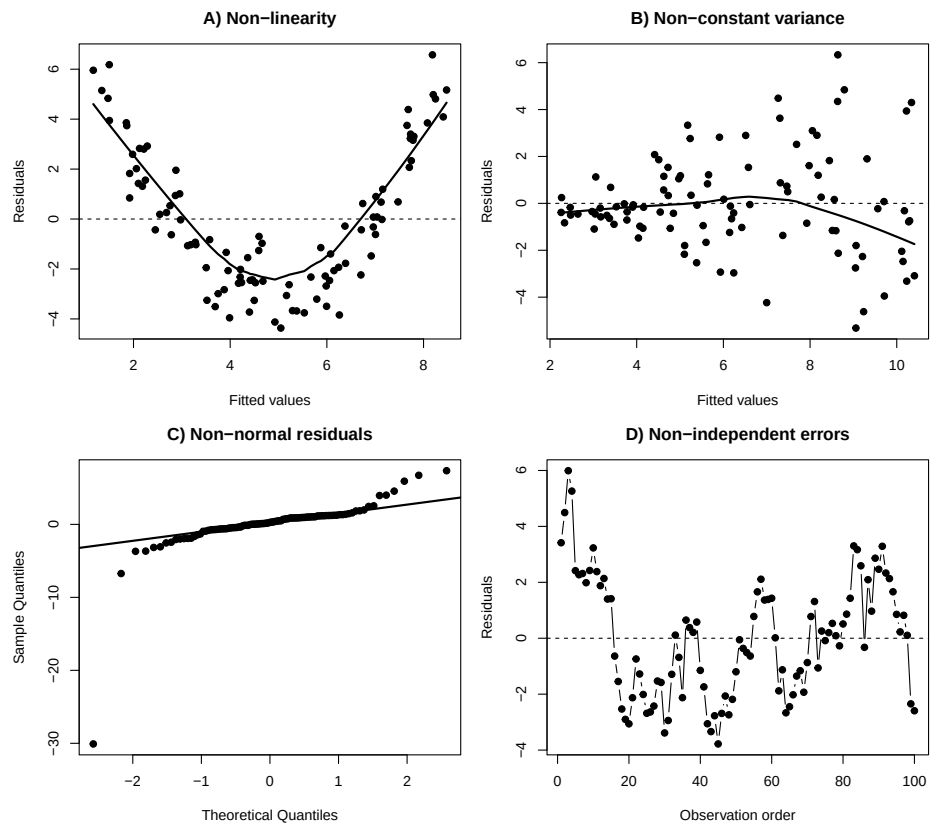


Figure 16.9: Examples of diagnostic patterns suggesting violations of linear model assumptions: A) non-linearity, B) non-constant variance, C) non-normal residuals, and D) dependence between observations.

- **Non-linearity:** a systematic curved pattern remains in the residuals.
- **Non-constant variance:** the spread of residuals changes with the fitted values, often forming a funnel shape.
- **Non-normality:** residuals deviate systematically from the straight line in the normal Q-Q plot.
- **Non-independence:** neighbouring observations tend to have similar residuals, producing runs or other structure when residuals are plotted in observation order.

The exact appearance will vary between data sets. Diagnostic plots should therefore be interpreted as tools for identifying **systematic patterns**, rather than as formal pass/fail tests.

16.16 Influential observations

- Sometimes individual observations can exert a great deal of influence on the fitted model.
- One routine way of checking for this is to fit the model n times, missing out each observation in turn.
- If we removed i -th observation and compared the fitted value from the full model, say $\hat{y}_i$ to those obtained by removing this point, denoted $\hat{y}_{j(i)}$ then
- observations with a high Cook's distance (measuring the effect of deleting a given observation) could be influential.

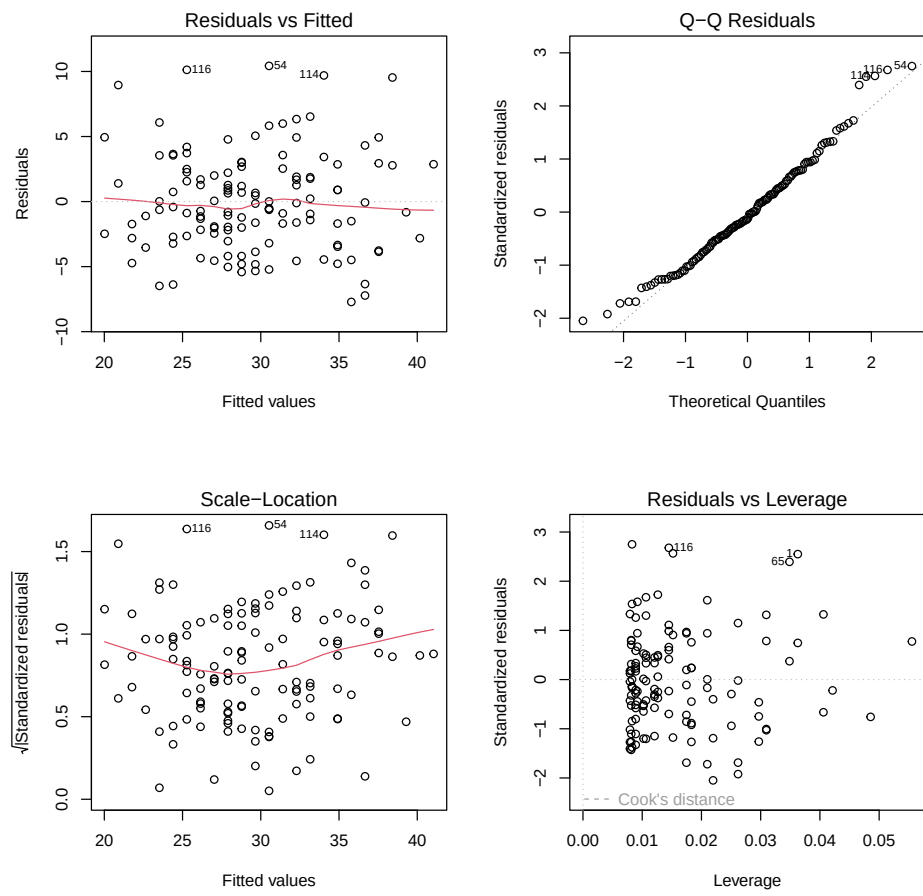
Let's remove some observation with higher Cook's distance from protein data set, re-fit our model and compare the diagnostics plots

```
# observations to be removed (based on Residuals vs. Leverage
  ↪ plot)
obs <- c(13, 78, 83, 84)

# fit models removing observations
data_diabetes_flr <- data_diabetes[-obs, ]

model_flr <- lm(BMI ~ waist, data = data_diabetes_flr)

# plot diagnostics plot
par(mfrow=c(2,2))
plot(model_flr)
```



Chapter 17

Generalized linear models

17.1 Why do we need GLM?

In linear regression, we described the **expected value of the outcome** for given values of the predictors.

For example, in simple linear regression,

$$E(Y \mid X = x) = \beta_0 + \beta_1 x.$$

More generally,

$$E(Y_i \mid \mathbf{x}_i) = \beta_0 + \beta_1 x_{1i} + \cdots + \beta_p x_{pi}.$$

This works well for many continuous outcomes, but it is not appropriate for all types of data. For example, probabilities must lie between 0 and 1, while expected counts must be non-negative.

Generalized linear models (GLMs) extend the same framework to such outcomes.

We still construct a **linear predictor**

$$\eta_i = \beta_0 + \beta_1 x_{1i} + \cdots + \beta_p x_{pi}.$$

However, instead of assuming that the expected outcome itself is equal to the linear predictor, we connect them through a **link function** $g(\cdot)$:

$$g(E(Y_i \mid \mathbf{x}_i)) = \eta_i.$$

Equivalently, if we denote the expected outcome by

$$\mu_i = E(Y_i \mid \mathbf{x}_i),$$

then

$$g(\mu_i) = \eta_i.$$

A GLM therefore specifies:

1. a probability distribution for the response Y_i , and
2. a link function connecting its expected value to the linear predictor.

Different choices give different models:

Outcome	Model	Distribution	Link function
Continuous	Linear regression	Normal	Identity
Binary	Logistic regression	Binomial	Logit
Counts	Poisson regression	Poisson	Log

The ordinary linear model is a special case. With the **identity link**,

$$g(\mu_i) = \mu_i,$$

so that

$$E(Y_i | \mathbf{x}_i) = \beta_0 + \beta_1 x_{1i} + \cdots + \beta_p x_{pi}.$$

For other types of outcomes, the link function allows us to retain the same linear predictor while modelling the expected outcome on an appropriate scale.

So briefly,

- GLMs extend linear model framework to outcome variables that do not follow normal distribution.
- They are most frequently used to model binary, categorical or count data.

For instance, consider a binary outcome, such as obesity status, coded as 0 (No) and 1 (Yes).

- If we fit an ordinary linear model to a binary outcome, the fitted values can fall below 0 or above 1 and therefore cannot always be interpreted as probabilities.
- In addition, the assumptions of the normal linear model are not appropriate for a binary response.

```
data_diabetes %>%
  mutate(obese = as.numeric(obese) - 1) %>%
  ggplot(aes(y=obese, x=waist)) +
  geom_jitter(width=0, height = 0) +
  geom_smooth(method="lm", se=FALSE, color=col.blue.dark) +
  my.ggtheme
```

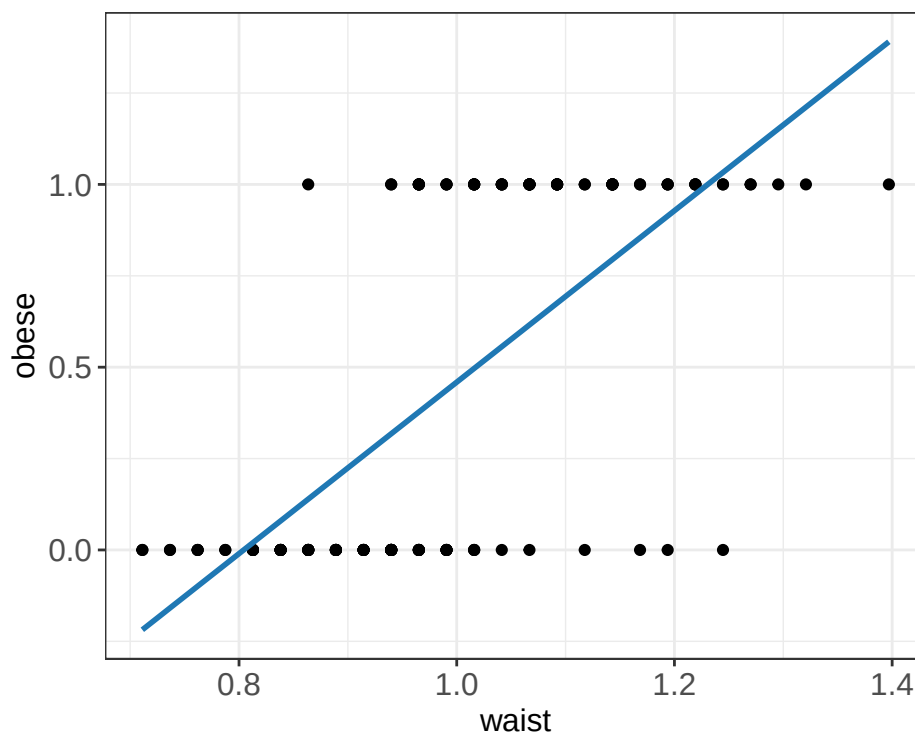


Figure 17.1: Example of fitting linear model to binary data, to model obesity status coded as 1 (Yes) and 0 (No) with waist variable. Linear model does not fit the data well in this case. The fitted values are not constrained to lie between 0 and 1 and therefore cannot always be interpreted as probabilities.

17.2 Logistic regression

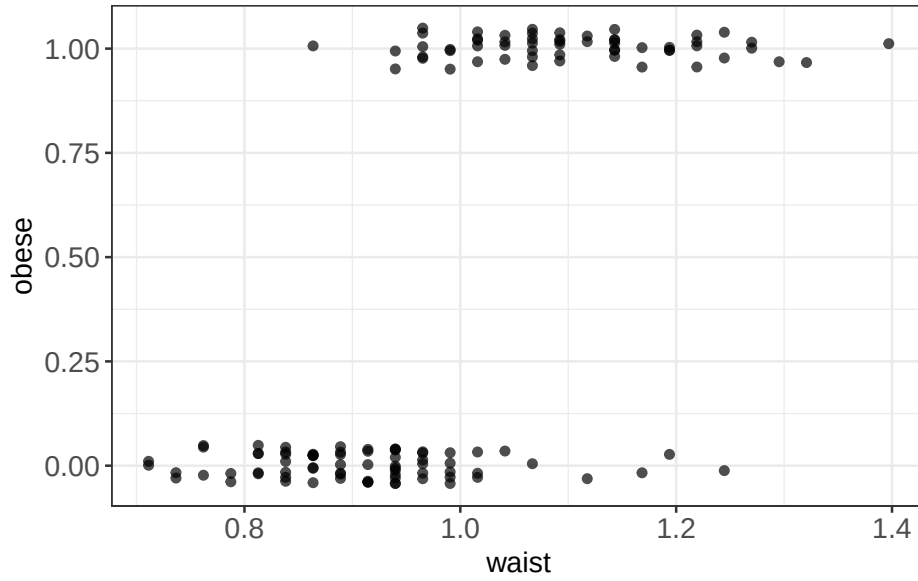
Let's look again at the binary obesity status data and try to fit logistic regression model using waist as explanatory variable instead of fitting inappropriate here simple linear model.

```
p1 <- data_diabetes %>%
  mutate(obese = as.numeric(obese) - 1) %>%
  ggplot(aes(y=obese, x=waist)) +
  geom_jitter(width=0, height = 0.05, alpha = 0.7) +
  my.ggtheme

# p2 <- data_diabetes %>%
#   ggplot(aes(x = obese, y = waist, fill = obese)) +
#   geom_boxplot() +
#   scale_fill_brewer(palette = "Set2") +
#   my.ggtheme

p1
```

Figure 17.2: Obesity status data: jittered plot (left) waist measurements and their obesity status for the 130 study participants.



Let's look again at the binary **obesity** status data. We will model obesity status (the probability of being obese) using **waist** as an explanatory variable, this time using logistic rather than simple linear regression.

For a binary response, let

$$Y_i = \begin{cases} 1 & \text{if the individual is obese,} \\ 0 & \text{otherwise.} \end{cases}$$

We denote the probability that individual i is obese, given their waist measurement x_i , by

$$p_i = P(Y_i = 1 \mid X_i = x_i).$$

Since Y_i is coded as 0 or 1, its expected value is equal to this probability:

$$E(Y_i \mid X_i = x_i) = p_i.$$

For a binary observation,

$$Y_i \sim \text{Bernoulli}(p_i),$$

which is equivalent to a binomial distribution with one trial.

In **logistic regression**, we use the **logit link** to relate the probability p_i to the linear predictor, thus rewriting linear model to:

$$\log \left(\frac{p_i}{1 - p_i} \right) = \beta_0 + \beta_1 x_i.$$

The quantity

$$\frac{p_i}{1 - p_i}$$

is called the **odds**, and its logarithm is the **log-odds** or **logit**.

Equivalently, we can express the probability as

$$p_i = \frac{\exp(\beta_0 + \beta_1 x_i)}{1 + \exp(\beta_0 + \beta_1 x_i)}.$$

This transformation ensures that the fitted probabilities always lie between 0 and 1.

And here the **link function** $\log(\frac{p_i}{1-p_i})$ provides the link between the binomial distribution of Y_i (probability of suffering from obesity) and the linear predictor (waist)

In R we can use `glm()` function to fit GLM models:

```
# re-code obese status from Yes/No to 1/0
data_diabetes <-
  data_diabetes %>%
  mutate(obese = as.numeric(obese) - 1)

# fit logistic regression model
logmodel_1 <- glm(obese ~ waist, family = binomial(link="logit"),
  ↪ data = data_diabetes)

# print model summary
print(summary(logmodel_1))
##
## Call:
## glm(formula = obese ~ waist, family = binomial(link =
  ↪ "logit"),
## data = data_diabetes)
##
## Coefficients:
##              Estimate Std. Error z value Pr(>|z|)
## (Intercept)  -17.357      2.973  -5.837 5.30e-09 ***
## waist         17.174      2.974   5.775 7.71e-09 ***
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## (Dispersion parameter for binomial family taken to be 1)
##
## Null deviance: 178.71  on 129  degrees of freedom
## Residual deviance: 102.79  on 128  degrees of freedom
## AIC: 106.79
##
```

```
## Number of Fisher Scoring iterations: 5

# plot
ggPredict(logmodel_1) +
  my.ggtheme
```

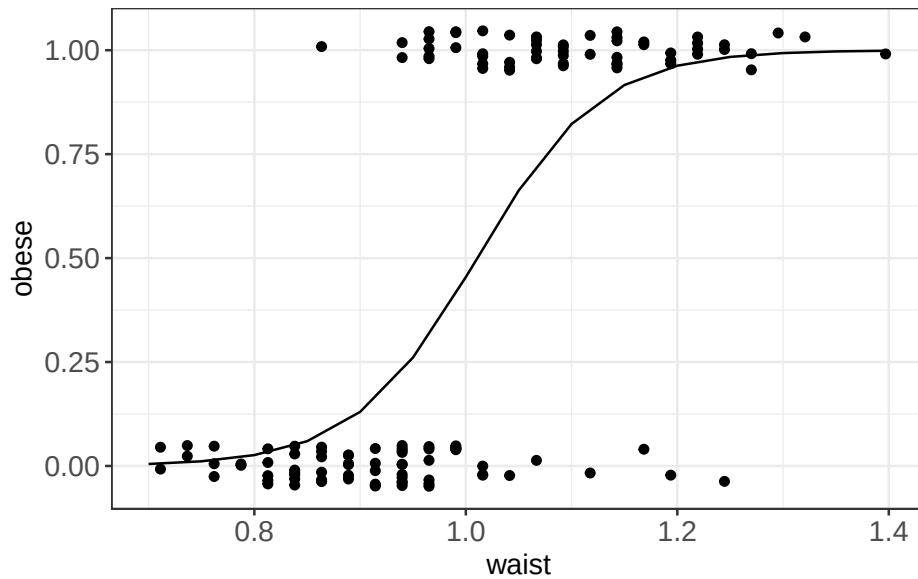


Figure 17.3: Fitted logistic model to diabetes data given the 130 study participants and using waist as explanatory variable to model obesity status.

```
# to get predictions use predict() functions
# if no new observations is specified predictions are returned
  ↳ for the values of exploratory variables used
# we specify response to return prediction on the probability
  ↳ scale
obese_predicted <- predict(logmodel_1, type="response")
print(head(obese_predicted))
##           3           7           9          11          16
  ↳ 21
## 0.98231495 0.93752915 0.07405337 0.41460606 0.22839680
  ↳ 0.72385967
```

- The regression equation for the fitted model is:

$$\log\left(\frac{\hat{p}_i}{1 - \hat{p}_i}\right) = -17.357 + 17.174 \cdot x_i$$

- We see from the output that $\hat{\beta}_0 = -17.357$ and $\hat{\beta}_1 = 17.174$.

The coefficient estimates are obtained using **maximum likelihood estimation**. The idea is to find the values of the model coefficients that make the observed

data most likely under the model. This is done using **numerical optimization**, where the algorithm searches over possible coefficient values until it finds the combination that gives the highest likelihood.

17.2.1 Hypothesis testing

As in linear regression, we can test whether an explanatory variable is associated with the outcome by testing whether its regression coefficient is zero.

For a coefficient β_j , we test

$$H_0 : \beta_j = 0$$

against

$$H_a : \beta_j \neq 0.$$

In logistic regression, individual coefficients are commonly tested using a **Wald test**. The Wald statistic is the estimated coefficient divided by its estimated standard error:

$$W = \frac{\hat{\beta}_j - 0}{e.s.e.(\hat{\beta}_j)}.$$

Under the null hypothesis, for sufficiently large samples, the Wald statistic approximately follows the standard normal distribution:

$$W \sim N(0, 1).$$

This is analogous to the t-test for a coefficient in linear regression. In the output from `summary(glm(...))`, this statistic is reported as the **z value**.

In our example, we want to test whether **waist** is associated with the probability of being obese:

$$H_0 : \beta_1 = 0.$$

From the fitted model,

$$\hat{\beta}_1 = 17.174$$

with estimated standard error

$$e.s.e.(\hat{\beta}_1) = 2.974.$$

Therefore,

$$W = \frac{17.174}{2.974} \approx 5.77.$$

We can calculate the corresponding two-sided P -value from the standard normal distribution:

```
2 * pnorm(abs(5.77), lower.tail = FALSE)
## [1] 7.927153e-09
```

The resulting P -value is very small, providing sufficient evidence to reject the null hypothesis and conclude that waist is associated with the probability of being obese.

17.2.2 Deviance

Another way to test whether an explanatory variable contributes to a GLM is to compare two **nested models**:

- a **reduced model**, without the explanatory variable of interest;
- a **full model**, including that variable.

For example, to test the contribution of **waist**, we compare

$$\text{reduced model: } \text{logit}(p_i) = \beta_0$$

with

$$\text{full model: } \text{logit}(p_i) = \beta_0 + \beta_1 x_i.$$

The comparison is based on the **deviance**, which measures lack of fit: smaller deviance indicates a better-fitting model.

The likelihood-ratio test uses the difference in deviance between the two models:

$$D_{\text{reduced}} - D_{\text{full}}.$$

Under the null hypothesis that the additional term does not improve the model, this difference approximately follows a chi-squared distribution. The degrees of freedom correspond to the difference in the number of estimated parameters between the models.

For our example, the intercept-only model has deviance

$$D_{\text{reduced}} = 178.71,$$

while the model including **waist** has deviance

$$D_{\text{full}} = 102.79.$$

The reduction in deviance is therefore

$$178.71 - 102.79 = 75.92.$$

Adding `waist` introduces one additional parameter. Under the null hypothesis, the deviance difference approximately follows a chi-squared distribution with 1 degree of freedom:

$$\Delta D \sim \chi_1^2.$$

For our data, the observed deviance difference is

$$\Delta D = 75.92.$$

The corresponding P -value is very small, providing strong evidence that including `waist` improves the fit of the model.

We can perform this comparison directly in R:

```
logmodel_0 <- glm(
  obese ~ 1,
  family = binomial(link = "logit"),
  data = data_diabetes
)

anova(logmodel_0, logmodel_1, test = "Chisq")
## Analysis of Deviance Table
##
## Model 1: obese ~ 1
## Model 2: obese ~ waist
##   Resid. Df Resid. Dev Df Deviance  Pr(>Chi)
## 1      129      178.71
## 2      128      102.79  1    75.915 < 2.2e-16 ***
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
```

17.2.3 Odds ratios

The coefficients in logistic regression describe changes in the **log-odds** of the outcome. Because log-odds are difficult to interpret directly, we often exponentiate the coefficients:

$$OR = \exp(\hat{\beta}).$$

The resulting quantity is an **odds ratio**.

For a numerical predictor, the odds ratio describes the multiplicative change in the odds of the outcome associated with a one-unit increase in the predictor, with other variables in the model held constant.

In our example, **waist** is measured in metres. Therefore, $\exp(\hat{\beta}_1)$ would describe the change in odds associated with a 1 metre increase in waist circumference, which is not a useful unit for interpretation.

Instead, we can consider an increase of 10 cm, corresponding to 0.1 metres:

$$\exp(0.1\hat{\beta}_1) = \exp(0.1 \times 17.174) \approx 5.6.$$

Thus, a 10 cm increase in waist circumference is associated with approximately **5.6 times higher odds of being obese**.

In general:

- an odds ratio of 1 indicates no change in the odds of the outcome;
- an odds ratio > 1 indicates higher odds of the outcome as the predictor increases;
- an odds ratio < 1 indicates lower odds of the outcome as the predictor increases.

The further an odds ratio is from 1, the larger the estimated association on the odds scale. However, the size of an odds ratio alone does **not** provide evidence that the relationship is causal.

17.2.4 Other covariates

As in multiple linear regression, we can extend a logistic regression model by including additional explanatory variables. These can be numerical, binary or categorical.

For example, we can investigate whether **gender** is associated with the probability of being obese after accounting for **waist**:

```
# fit logistic regression including waist and gender
logmodel_2 <- glm(
  obese ~ waist + gender,
  family = binomial(link = "logit"),
  data = data_diabetes
)

# print model summary
print(summary(logmodel_2))
##
## Call:
## glm(formula = obese ~ waist + gender, family = binomial(link =
##    "logit"),
##      data = data_diabetes)
##
## Coefficients:
##              Estimate Std. Error z value Pr(>|z|)
## (Intercept)  -18.2756     3.1077  -5.881 4.08e-09 ***
## waist         17.4401     3.0523   5.714 1.11e-08 ***
## genderfemale  1.2335     0.5228   2.359 0.0183 *
```

```
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## (Dispersion parameter for binomial family taken to be 1)
##
##      Null deviance: 178.708  on 129  degrees of freedom
## Residual deviance:  96.877  on 127  degrees of freedom
## AIC: 102.88
##
## Number of Fisher Scoring iterations: 6

# plot fitted probabilities
ggPredict(logmodel_2) +
  my.ggtheme +
  scale_color_brewer(palette = "Set2")
```

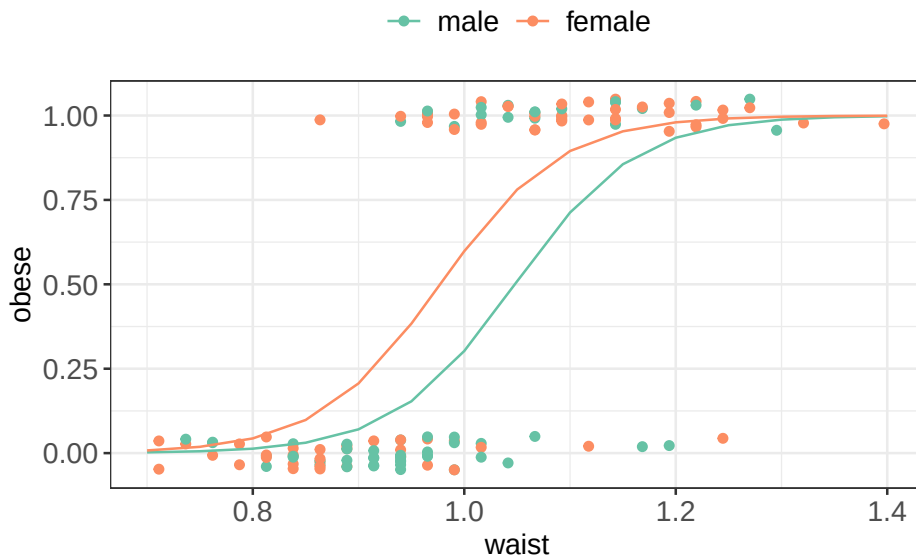


Figure 17.4: Fitted probability of obesity as a function of waist circumference, shown separately by gender.

The model can be written as

$$\text{logit}(p_i) = \beta_0 + \beta_1 \text{waist}_i + \beta_2 \text{gender}_i$$

The interpretation follows the same principle as in multiple linear regression: each coefficient describes the association with the outcome while the other variables in the model are held constant.

Thus, the coefficient for gender describes the association between gender and the probability of being obese for individuals with the same waist circumference.

17.3 Poisson regression

GLMs can also be used for **count data**, for example:

- number of hospital admissions due to respiratory disease;
- number of mutations observed in a sample;
- number of bird nests in a habitat.

For count data, a common starting point is to assume that

$$Y_i \sim \text{Poisson}(\mu_i),$$

where

$$\mu_i = E(Y_i \mid \mathbf{x}_i)$$

is the expected count for observation i .

In **Poisson regression**, we use the **log link** to relate the expected count to the linear predictor:

$$\log(\mu_i) = \beta_0 + \beta_1 x_{1i} + \cdots + \beta_p x_{pi}.$$

Equivalently,

$$\mu_i = \exp(\beta_0 + \beta_1 x_{1i} + \cdots + \beta_p x_{pi}).$$

The log link ensures that the fitted expected counts are always positive.

Hypothesis testing and comparison of nested models follow the same general logic as for logistic regression. Individual coefficients can be tested using Wald tests, while models can be compared using differences in deviance.

Example 17.1 (Number of cancer cases). Suppose we wish to model Y_i , the number of cancer cases in geographical region i in Glasgow.

We have data from 271 small regions, together with information on:

- **Y_all**: observed number of cases of all types of cancer in the region in 2013;
- **E_all**: expected number of cancer cases based on the population size and demographic structure of the region;
- **pm10**: air pollution;
- **smoke**: percentage of people who smoke;
- **ethnic**: percentage of people who are non-white;
- **log.price**: natural logarithm of average house price;
- **easting** and **northing**: geographical coordinates.

The regions differ in population size and demographic structure. We therefore do not want to compare the raw numbers of cancer cases directly.

Instead, we include **E_all** as an **offset**. The model becomes

$$\log(\mu_i) = \log(E_i) + \beta_0 + \beta_1 x_{1i} + \cdots + \beta_p x_{pi}.$$

Here, $\log(E_i)$ is included with its coefficient fixed at 1. It accounts for the number of cases that would already be expected in each region based on its population and demographic structure.

Equivalently,

$$\mu_i = E_i \exp(\beta_0 + \beta_1 x_{1i} + \cdots + \beta_p x_{pi}).$$

We can fit the model using `glm()` with the Poisson family:

```
# Read in and preview data
cancer <- read.csv("data/lm/cancer.csv")

head(cancer)
##           IG Y_all      E_all pm10 smoke ethnic log.price
  ↪ easting northing
## 1 S02000260    133 106.17907 17.8  21.9   5.58  11.59910
  ↪ 26.16245 66.96574
## 2 S02000261     38  62.43131 18.6  21.8   7.91  11.84940
  ↪ 26.29271 67.00278
## 3 S02000262     97 120.00694 18.6  20.8   9.58  11.74106
  ↪ 26.21429 67.04280
## 4 S02000263     80 109.10245 17.0  14.0  10.39  12.30138
  ↪ 25.45705 67.05938
## 5 S02000264    181 149.77821 18.6  15.2   5.67  11.88449
  ↪ 26.12484 67.09280
## 6 S02000265     77  82.31156 17.0  14.6   5.61  11.82004
  ↪ 25.37644 67.09826

# Fit Poisson regression
epid1 <- glm(
  Y_all ~ pm10 + smoke + ethnic + log.price +
    easting + northing + offset(log(E_all)),
  family = poisson,
  data = cancer
)

print(summary(epid1))
##
## Call:
## glm(formula = Y_all ~ pm10 + smoke + ethnic + log.price +
  ↪ easting +
##      northing + offset(log(E_all)), family = poisson, data =
  ↪ cancer)
##
## Coefficients:
##              Estimate Std. Error z value Pr(>|z|)
## (Intercept) -0.8592657  0.8029040  -1.070  0.284531
## pm10         0.0500269  0.0066724   7.498 6.50e-14 ***
## smoke        0.0033516  0.0009463   3.542 0.000397 ***
```

```
## ethnic      -0.0049388  0.0006354  -7.773 7.66e-15 ***
## log.price   -0.1034461  0.0169943  -6.087 1.15e-09 ***
## easting     -0.0331305  0.0103698  -3.195 0.001399 **
## northing    0.0300213  0.0111013   2.704 0.006845 **
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## (Dispersion parameter for poisson family taken to be 1)
##
##      Null deviance: 972.94  on 270  degrees of freedom
## Residual deviance: 565.18  on 264  degrees of freedom
## AIC: 2356.2
##
## Number of Fisher Scoring iterations: 4
```

Rate ratios

As in logistic regression, coefficients in Poisson regression are often easier to interpret after exponentiation:

$$RR = \exp(\hat{\beta}).$$

The resulting quantity is a **rate ratio**.

For a numerical predictor, the rate ratio describes the multiplicative change in the expected rate associated with a one-unit increase in the predictor, while the other variables in the model are held constant.

For example, suppose we are interested in the association between air pollution and cancer incidence. The estimated coefficient for `pm10` is

$$\hat{\beta}_{pm10} = 0.0500.$$

Since this coefficient is positive, higher `pm10` values are associated with a higher expected cancer rate.

Exponentiating the coefficient gives

$$\exp(0.0500) \approx 1.05.$$

Thus, a one-unit increase in `pm10` is associated with approximately a **1.05-fold**, or **5% higher**, expected cancer incidence rate, while the other variables in the model are held constant.

Chapter 18

Common cases

Let's go over some common cases of linear models to clarify their interpretation and usage. We will need to run this code to begin with:

```
library(tidyverse)
library(magrittr)
library(faraway)
library(ggiraphExtra)

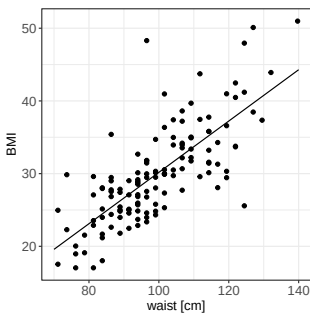
font.size <- 12
my.ggtheme <-
  theme_bw() +
  theme(axis.title = element_text(size = font.size),
        axis.text = element_text(size = font.size),
        legend.text = element_text(size = font.size),
        legend.title = element_blank(),
        legend.position = "top")

# add obesity and diabetes status to diabetes faraway data
inch2m <- 2.54/100
inch2cm <- 2.54
pound2kg <- 0.45
data_diabetes <- diabetes %>%
  mutate(height = height * inch2m, height = round(height, 2))
  ↪ %>%
  mutate(waist = waist * inch2cm) %>%
  mutate(weight = weight * pound2kg, weight = round(weight, 2))
  ↪ %>%
  mutate(BMI = weight / height^2, BMI = round(BMI, 2)) %>%
  mutate(obese= cut(BMI, breaks = c(0, 29.9, 100), labels =
    ↪ c("No", "Yes"))) %>%
  mutate(diabetic = ifelse(glyhb > 7, "Yes", "No"), diabetic =
    ↪ factor(diabetic, levels = c("No", "Yes"))) %>%
  na.omit()
```

18.1 Example: simple linear regression

```
m1 <- lm(BMI ~ waist, data = data_diabetes)
summary(m1)
##
## Call:
## lm(formula = BMI ~ waist, data = data_diabetes)
##
## Residuals:
##      Min       1Q   Median       3Q      Max
## -13.2374  -2.7689  -0.4532   2.4065  19.3549
##
## Coefficients:
##              Estimate Std. Error t value Pr(>|t|)
## (Intercept) -5.12445     2.73538  -1.873   0.0633 .
## waist        0.35298     0.02723   12.965 <2e-16 ***
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Residual standard error: 4.426 on 128 degrees of freedom
## Multiple R-squared:  0.5677, Adjusted R-squared:  0.5643
## F-statistic: 168.1 on 1 and 128 DF,  p-value: < 2.2e-16

ggPredict(m1) +
  my.ggtheme +
  xlab("waist [cm]")
```



Model (generic)

- $Y_i = \alpha + \beta \cdot x_i + \epsilon_i$

Model (fitted)

- $BMI_i = -5.12 + 0.35 \cdot waist_i + \epsilon_i$

Slope

- The value of slope tells us how and by much the outcome changes with a unit change in x
- If the **waist** increases by 1 unit, here in cm, what would be our expected change in BMI¹?
- And if the **waist** increases by 10 units what would be our expected change in BMI²?

Intercept

- The **intercept**, often labeled the **constant**, is the value of Y when $x_i = 0$.

¹If the waist increases by 1 cm we would expect our BMI to increase by ≈ 0.35 since $\hat{\beta} = 0.35298$

²If the waist increases by 10 cm we would expect BMI to increase by $0.35298 \cdot 10 \approx 3.53$

- In models where x_i can be equal 0, the intercept is simply the expected mean value of response.
- In models where x_i cannot be equal 0, like in our BMI example where it is not possible to have BMI equal to zero, the intercept has no intrinsic meaning.
- The intercept is thus quite often ignored in linear models, as it is the value of slope that dictates the association between exposure and outcome.

Hypothesis testing

- We've seen during the lecture that the check for association between exposure and outcome we check if the we have enough evidence to reject $H_0 : \beta = 0$ in favor of the alternative $H_a : \beta \neq 0$.
- Here, for the β coefficient we have $t - statistics = 0.35298/0.02723 = 12.965$ and a corresponding $p - value = 12.96291$, as $t - statistics \sim t(130 - 2) \ll 0.05$. Such large **t-statsitics** or small **p-value** means we have enough evidence to reject the null hypothesis and conclude that there is a significant association between waist and BMI.
- We can double-check R output by calculating p-value ourselves using the Student t distribution:

```
2*pt(12.96291, df=128, lower=F)
## [1] 4.605102e-25
```

- Is there enough evidence to reject the null hypothesis of $H_0 : \alpha = 0$ in favor of the alternative $H_a : \alpha \neq 0$ assuming 5% significance level?³.
- Is there enough evidence to reject the null hypothesis of $H_0 : \alpha = 0$ in favor of the alternative $H_a : \alpha \neq 0$ assuming 10% significance level?⁴.

Predictions

- Using the model we can predict the BMI value for a new observation of **waist**.
- For instance, we can find expected BMI value for someone who measures 100 cm in **waist** by:
- $BMI = -5.12445 + 0.35298 \cdot 100 = 30.17355$
- In R can use **predict()** function:

```
# predict BMI for a new value of 100
new_data <- data.frame(waist = 100)
predict(m1, newdata = new_data)
##      1
## 30.17348
```

- What would be BMI for someone with **waist** measurements of 75?⁵
- What would be BMI for someone with **waist** measurements of 200?⁶

Model fit

³No, as $p - value = 0.0633 \not< 0.05$

⁴Yes, as $p - value = 0.0633 < 0.1$

⁵ $BMI = -5.12445 + 0.35298 \cdot 75 = 21.349$

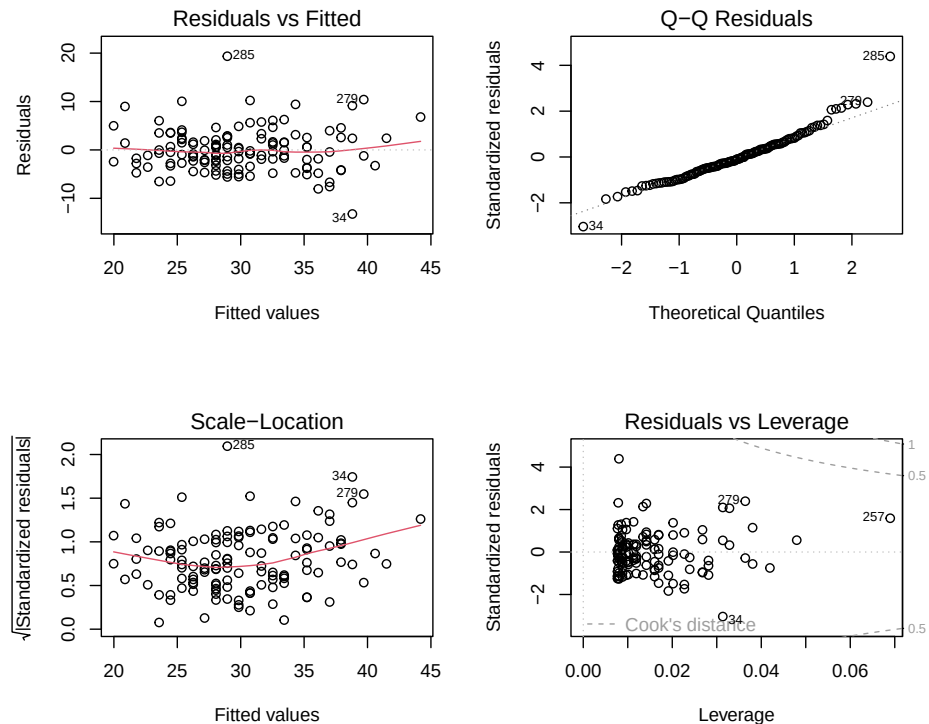
⁶ $BMI = -5.12445 + 0.35298 \cdot 200 = 65.47141$, however here we have to be careful in predicting outside the model range.

- In simple regression we can use R^2 to assess model fit, here $R^2 = 0.5677$.
- Do you think that the model fits the data well?⁷

Model assumptions

We should also not forget to look at the residual plots to check model assumptions:

```
par(mfrow = c(2,2))
plot(m1)
```



- Given the diagnostic plots can we comment about the assumptions of linear models being met?⁸

18.2 Example: multiple regression

Let's try to model BMI using more variables

Model (generic)

$$Y_i = \beta_0 + \beta_1 \cdot age_i + \beta_2 \cdot chol_i + \beta_3 \cdot hdl_i + \epsilon_i$$

⁷In simple linear regression R^2 is the same as r^2 and a value of 0.5677 indicates moderate fit, that agrees with the plot above. Since we have more variables in the data set we could try to improve the fit by including more variables.

⁸The diagnostics do not indicate a serious violation of model assumptions, with no obvious trends of any kind in the residuals plots. Few samples deviate from diagonal line on the Normal Q-Q plot and these could be removed to ensure that the residuals follow normal distribution.

```
# fit multiple linear regression and print model summary
m2 <- lm(BMI ~ age + chol + hdl, data = data_diabetes)
summary(m2)
##
## Call:
## lm(formula = BMI ~ age + chol + hdl, data = data_diabetes)
##
## Residuals:
##      Min       1Q   Median       3Q      Max
## -13.074  -4.833  -1.132   3.438  22.032
##
## Coefficients:
##              Estimate Std. Error t value Pr(>|t|)
## (Intercept) 35.456968   3.149661  11.257 < 2e-16 ***
## age        -0.027047   0.040304  -0.671  0.50340
## chol        0.002039   0.012701   0.161  0.87269
## hdl        -0.090023   0.032734  -2.750  0.00683 **
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Residual standard error: 6.552 on 126 degrees of freedom
## Multiple R-squared:  0.06763,    Adjusted R-squared:  0.04543
## F-statistic: 3.046 on 3 and 126 DF,  p-value: 0.03124
```

Coefficient interpretations

Using the model answer the questions:

- what would happen to BMI if hdl levels increase by 10?⁹
- what would happen to BMI if age increases by 1 year?¹⁰

Hypothesis testing

- overall, is there a relationship between the response Y (BMI) and predictors?¹¹

Not so easy: alternative model

⁹decreases by $-0.090023 \cdot 10 = 0.90023$

¹⁰decrease by 0.027047, however here we can see that the age coefficient is not significant and therefore we should be careful with our interpretations as there is no evidence that this coefficient is different than 0.

¹¹we have seen before that in the case of simple linear regression it was enough to test the null hypothesis of $H_0 : \beta = 0$ versus $H_0 : \beta \neq 0$ to answers the question whether there is an overall relationship between response and predictor. In case of multiple regression, with many predictors, we need to test the null hypothesis of

$$H_0 : \beta_1 = \beta_2 = \dots = \beta_p = 0$$

versus the alternative

$$H_a : \text{at least one } \beta_j \text{ is non-zero}$$

This hypothesis test is performed by computing **F-statistics** reported in the model summary and calculated as $F = \frac{(TSS-RSS)/p}{RSS/(n-p-1)}$ where $TSS = \sum (y_i - \bar{y})^2$ and $RSS = \sum (y_i - \hat{y}_i)^2$. Here, the F -statistics = 3.046 and the associated p -value < 0.05 so there is enough evidence to reject the null hypothesis in favor of the alternative and conclude that there is an overall significant relationship between response (BMI) and predictors.

Let's consider another multiple regression model:

$$\bullet Y_i = \beta_0 + \beta_1 \cdot \text{age}_i + \beta_2 \cdot \text{chol}_i + \beta_3 \cdot \text{hdl}_i + \beta_4 \cdot \text{waist}_i + \epsilon_i$$

We fit the model in R and look at the model summary:

```
m2_alt <- lm(BMI ~ age + chol + hdl + waist, data =
  ↪ data_diabetes)
summary(m2_alt)
##
## Call:
## lm(formula = BMI ~ age + chol + hdl + waist, data =
  ↪ data_diabetes)
##
## Residuals:
##      Min       1Q   Median       3Q      Max
## -13.0337  -3.0416  -0.6777   2.2711  18.2894
##
## Coefficients:
##              Estimate Std. Error t value Pr(>|t|)
## (Intercept) -0.921431   3.588473  -0.257   0.7978
## age          -0.050397   0.027016  -1.865   0.0645 .
## chol         -0.006250   0.008519  -0.734   0.4645
## hdl          -0.006199   0.022890  -0.271   0.7870
## waist        0.353256   0.028213  12.521 <2e-16 ***
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Residual standard error: 4.381 on 125 degrees of freedom
## Multiple R-squared:  0.5864, Adjusted R-squared:  0.5732
## F-statistic: 44.3 on 4 and 125 DF, p-value: < 2.2e-16
```

- What happens to BMI if hdl increases by 10?¹²
- What happens to BMI if hdl increases by 10 using the first model again?¹³
- How do you explain the difference in BMI changes given these two models?

Specific interpretation

- Obviously there is difference between decrease of 0.9 BMI and decrease of 0.06 in BMI (alternative model).
- Our interpretations need to be more specific and we say that **a unit increase in x with other predictors held constant will produce a change equal to $\hat{\beta}$ in the response y**
- Often it may be quite unrealistic to be able to control other variables and keep them constant and for our alternative model, a change in hdl would also imply a change in total cholesterol chol.
- Further, our explanation contains **no notation of causation**.
- We will learn later how to choose the best model by assessing its fit and including only relevant variable (feature selection), for now we focus on learning how to interpret the coefficients given a fitted model.

¹²decreases by ca. 0.06

¹³decreases by ca. 0.9

18.3 Example: categorical variable

- We want to **compare the average BMI of men and women**.
- We can do that using linear regression and including gender as **binary variable**

```
font.size <- 20
col.blue.light <- "#a6cee3"
col.blue.dark <- "#1f78b4"
my.ggtheme <-
  theme_bw() +
  theme(axis.title = element_text(size = font.size),
        axis.text = element_text(size = font.size),
        legend.text = element_text(size = font.size),
        legend.title = element_blank(),
        legend.position = "top")

# visualize the data with box plot
data_diabetes %>%
  ggplot(aes(x = gender, y = BMI, fill = gender)) +
  geom_boxplot() +
  scale_fill_brewer(palette = "Set2") +
  my.ggtheme
```

Model

$$Y_i = \beta_o + \beta_1 I_{x_1,i} + \epsilon_i$$

where

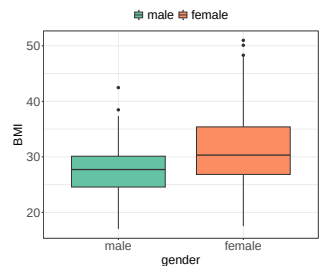
$$I_{x_1,i} = \begin{cases} 1 & \text{if } x_i = 1 \\ 0 & \text{if } x_i = 0 \end{cases} \quad (18.1)$$

for some coding, e.g. we choose to set “Female=1” and “Male=0” or vice versa.

In R we write:

```
# Note: check that Gender is indeed non-numeric
print(class(data_diabetes$gender))
## [1] "factor"

# fit linear regression and print model summary
m3 <- lm(BMI ~ gender, data = data_diabetes)
print(summary(m3))
##
## Call:
## lm(formula = BMI ~ gender, data = data_diabetes)
##
## Residuals:
##      Min       1Q   Median       3Q      Max
## -14.167  -4.117  -0.327   3.160  19.273
##
```



```
## Coefficients:
##              Estimate Std. Error t value Pr(>|t|)
## (Intercept)  27.7674      0.8527  32.566 < 2e-16 ***
## genderfemale  3.9396      1.1379   3.462 0.000729 ***
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Residual standard error: 6.437 on 128 degrees of freedom
## Multiple R-squared:  0.08563,    Adjusted R-squared:  0.07849
## F-statistic: 11.99 on 1 and 128 DF,  p-value: 0.0007286
```

Estimates

$$\hat{\beta}_0 = 27.7674$$

$$\hat{\beta}_1 = 3.9396$$

- The `lm()` function chooses automatically one of the category as baseline, here `females`.
- Model summary prints the output of the model with the baseline category “**hidden**”.
- Notice that the only label we have is “genderfemale”.
- Meaning that we ended-up having a model coded as below:

$$I_{x_i} = \begin{cases} 1 & \text{if } person_i \text{ is female} \\ 0 & \text{if } person_i \text{ is male} \end{cases} \quad (18.2)$$

- Consequently, if observation i is female then the expected value of BMI is:

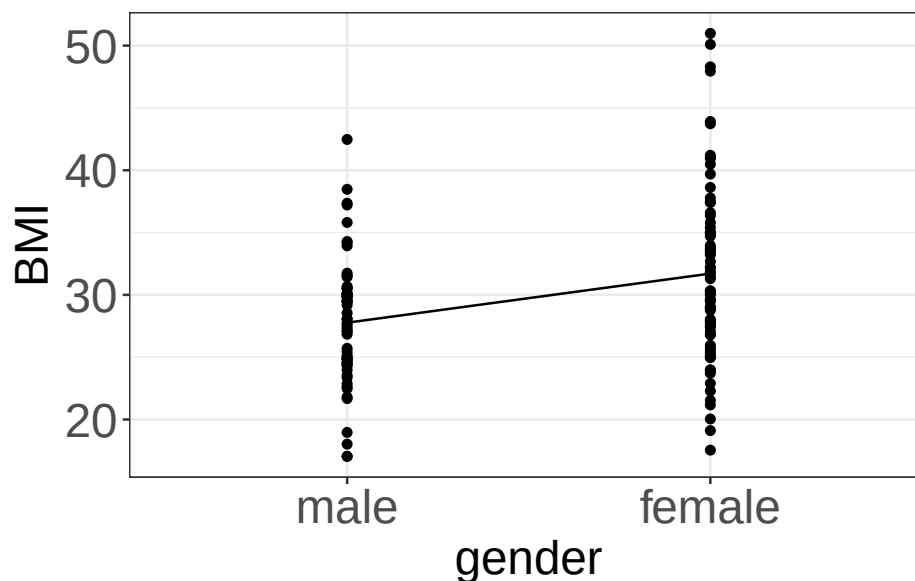
$$E(BMI_i | female) = 27.7674 + 3.9396 = 31.707$$

- and if observation i is male then the expected value of BMI is:

$$E(BMI_i | male) = 27.7674$$

We can plot the model in R:

```
ggPredict(m3) +
  my.ggtheme
```

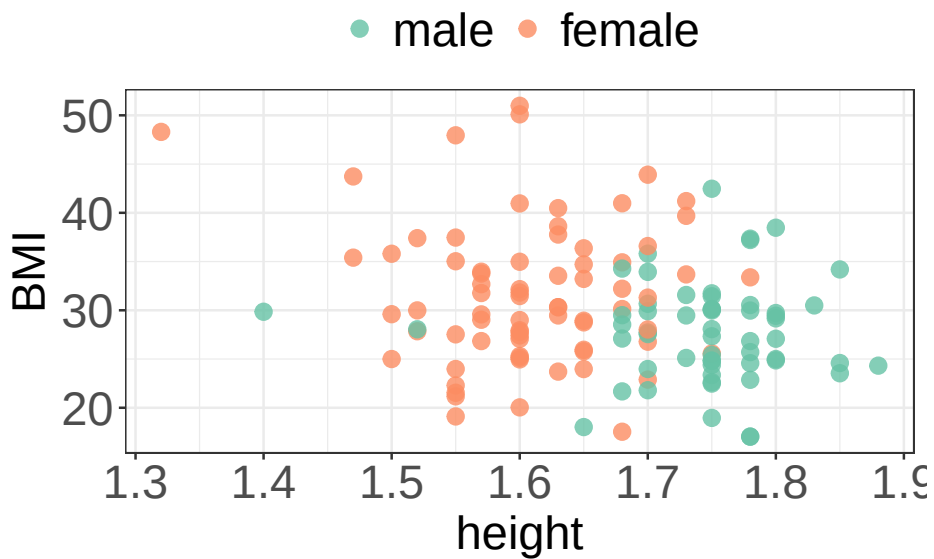


18.4 Example: categorical & numerical variables

- Above we observed a significant difference in average BMI between men and women among the study participants.
- Can we also observe a significant relationship between BMI and height?
- And if so, does this relationship depend on gender?

```
#|label: fig-htwtgen-plot
#|fig-cap: Scatter plot showing BMI measurements given height
  ↳ stratified by gender.
#|fig-cap-location: margin
#|collapse: true
#|code-fold: false
#|fig-width: 5
#|fig-height: 5

# plot the data separately for Male and Female
data_diabetes %>%
  ggplot(aes(x = height, y=BMI, col = gender)) +
  geom_point(alpha = 0.8, size = 3) +
  scale_color_brewer(palette = "Set2") +
  my.ggtheme
```



- From the plot we can see that BMI decreases slightly with `height`.
- On average, men are taller than women.
- On average, women have higher BMI than men.
- The relationship between `height` and BMI appears to be the same for males and females, i.e. BMI decreases with `height` for both men and women.

To assess the relationship we use a model containing `height` and `gender`.

Model

$$Y_i = \beta_0 + \beta_1 I_{x_1,i} + \beta_2 x_{2,i} + \epsilon_i$$

where

$$I_{x_1,i} = \begin{cases} 1 & \text{if } \text{person}_i \text{ is female} \\ 0 & \text{if } \text{person}_i \text{ is male} \end{cases} \quad (18.3)$$

and $x_{2,i}$ is the `height` of person i .

In R we write:

```
# fit linear model and print model summary
m4 <- lm(BMI ~ gender + height, data = data_diabetes)
print(summary(m4))
##
## Call:
## lm(formula = BMI ~ gender + height, data = data_diabetes)
##
## Residuals:
##      Min       1Q   Median       3Q      Max
## -13.7580  -4.2617  -0.3863   3.1646  19.2244
##
## Coefficients:
##              Estimate Std. Error t value Pr(>|t|)
## (Intercept)    37.743     13.294   2.839  0.00527 **
```

```
## genderfemale    3.163      1.538    2.057  0.04172 *
## height         -5.719      7.606   -0.752  0.45350
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Residual standard error: 6.448 on 127 degrees of freedom
## Multiple R-squared:  0.08969,    Adjusted R-squared:  0.07535
## F-statistic: 6.256 on 2 and 127 DF,  p-value: 0.002562
```

Model together with estimates

$$Y_i = \beta_0 + \beta_1 I_{x_1,i} + \beta_2 \cdot x_{2,i} + \epsilon_i$$

where

$$I_{x_i} = \begin{cases} 1 & \text{if } \text{person}_i \text{ is male} \\ 0 & \text{if } \text{person}_i \text{ is female} \end{cases} \quad (18.4)$$

and $x_{2,i}$ is the height of person i

Estimates

$$\hat{\beta}_0 = 37.743$$

$$\hat{\beta}_1 = 3.163$$

$$\hat{\beta}_2 = -5.719$$

- For instance, using our estimates, for a female who happens to 1.7 m tall we would predict BMI of:

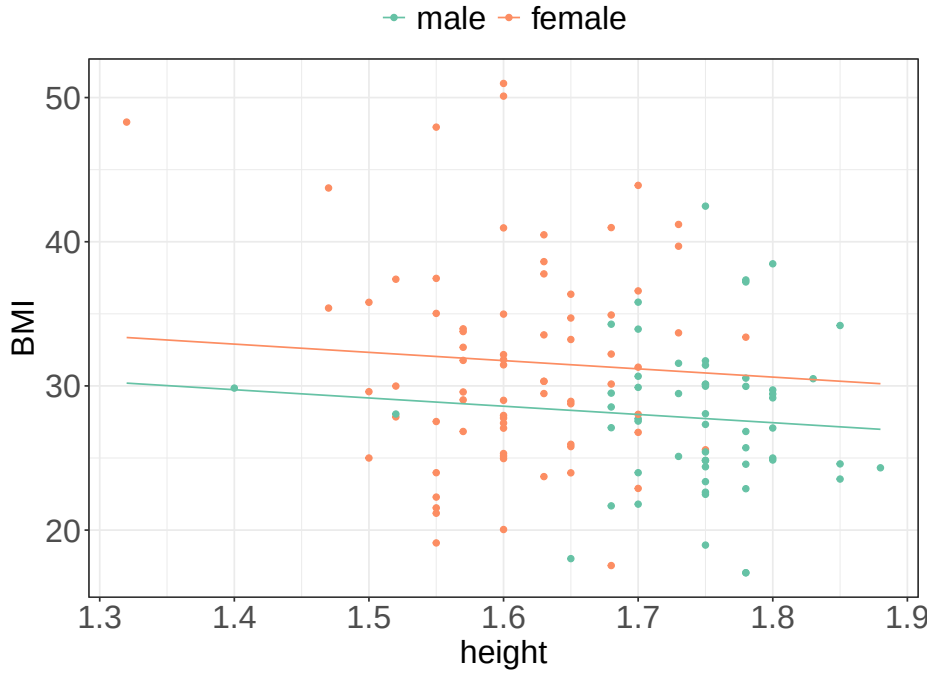
$$E(BMI_i | \text{female}, \text{height} = 1.7) = 37.743 + 3.163 + (-5.719 \cdot 1.7) = 31.1837$$

- and for a male of height 1.7 m tall we would predict BMI of

$$E(BMI_i | \text{male}, \text{height} = 1.7) = 37.743 + (-5.719 \cdot 1.7) = 28.0207$$

In R we can plot our data and the fitted model to verify our calculations:

```
# plot the data separately for men and women
# using ggplot() and geom_smooth()
ggPredict(m4) +
  scale_color_brewer(palette = "Set2") +
  my.ggtheme
```



18.5 Example: interactions

- The fitted lines in the above example are **parallel**, the **slope is modeled to be the same for men and women**, and the intercept denotes the group differences.
- It is also possible to allow for **both intercept and slope being fitted separately for each group**.
- This is done when we expect that the relationships are different in different groups, e.g. increasing in one group and decreasing in the other.
- And we then talk about including **interaction effect** since the two lines may interact (cross).

Model

$$Y_i = \beta_0 + \beta_1 I_{x_1,i} + \beta_2 \cdot x_{2,i} + \beta_3 \cdot I_{x_1,i} \cdot x_{2,i} + \epsilon_i$$

where:

- Y_i is the BMI of person i

$$I_{x_1,i} = \begin{cases} 1 & \text{if } \text{person}_i \text{ is female} \\ 0 & \text{if } \text{person}_i \text{ is male} \end{cases} \quad (18.5)$$

- $x_{1,i}$ is the height of person i
- and $\beta_3 \cdot I_{x_1,i} \cdot x_{2,i}$ is the interaction term

In R we define the interaction term with `*`:

```
# fit linear model with interaction
m5 <- lm(BMI ~ gender * height, data = data_diabetes)
print(summary(m5))
##
## Call:
## lm(formula = BMI ~ gender * height, data = data_diabetes)
##
## Residuals:
##      Min       1Q   Median       3Q      Max
## -13.5564  -4.1137  -0.3072   3.1057  19.2005
##
## Coefficients:
##              Estimate Std. Error t value Pr(>|t|)
## (Intercept)      31.222     20.318   1.537   0.127
## genderfemale      14.219     26.032   0.546   0.586
## height           -1.981     11.638  -0.170   0.865
## genderfemale:height -6.558     15.414  -0.425   0.671
##
## Residual standard error: 6.469 on 126 degrees of freedom
## Multiple R-squared:  0.09099,    Adjusted R-squared:  0.06935
## F-statistic: 4.204 on 3 and 126 DF,  p-value: 0.007155
```

Now, based on the regression output we would expect:

- for a woman of height x , a BMI value of:

$$E(BMI|female \text{ and } height = x) = 31.222 + 14.219 - 1.981 \cdot x - 6.558 \cdot x = 45.441 - 8.539 \cdot x$$

- for a man of height x , a BMI value of

$$E(BMI|male \text{ and } height = x) = 31.222 - 1.981 \cdot x$$

Estimates

$$\hat{\beta}_0 = 31.22$$

$$\hat{\beta}_1 = 14.22$$

$$\hat{\beta}_2 = -1.98$$

$$\hat{\beta}_3 = -6.558$$

To model relationship between BMI and height in males, the model reduces to:

$$Y_i = \beta_0 + \beta_2 \cdot x_{2,i} + \epsilon_i = 31.22 - 1.98 \cdot height_i$$

To model relationship between BMI and height in females, the models sums up to:

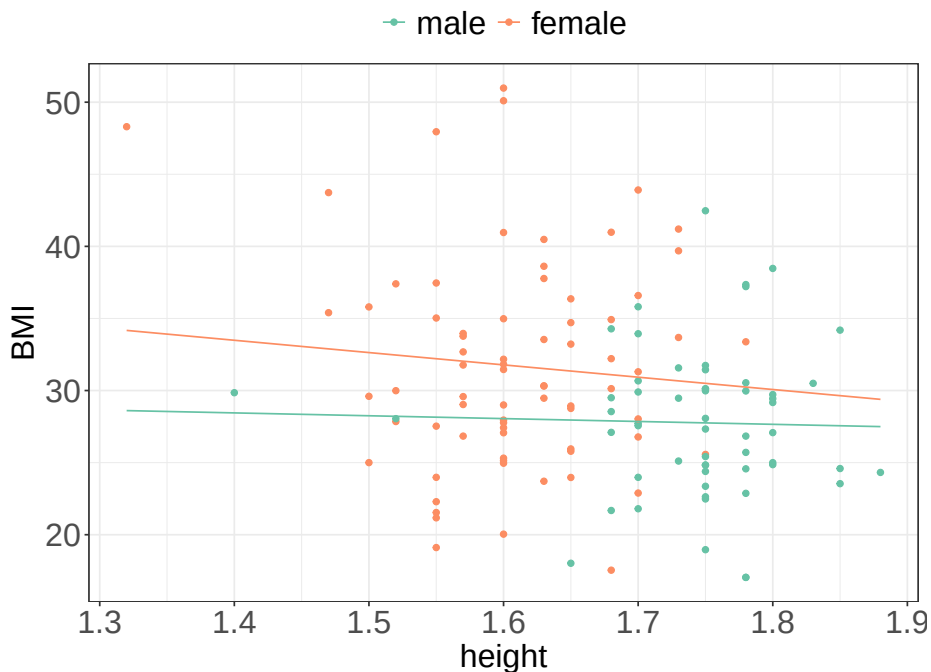
$$\begin{aligned} Y_i &= \beta_0 + \beta_1 I_{x_1,i} + \beta_2 \cdot x_{2,i} + \beta_3 \cdot I_{x_1,i} \cdot x_{2,i} + \epsilon_i = \\ &= (\beta_0 + \beta_1) + (\beta_2 + \beta_3) \cdot height_i = \\ &= (31.22 + 14.219) + (-1.98 - 6.558) \cdot height_i = \\ &= 45.44 - 8.539 \cdot height_i \end{aligned}$$

This lets us model different relationships of BMI and height in both groups, with **individual intercept and slope values**.

In addition:

- We can see from the regression output that there is no evidence to reject the null hypothesis that the interaction term “Genderfemale:height” is equal to zero.
- Therefore, there is no evidence that the relationship between BMI and height differs between men and women.
- However, when we plot the fitted model, the fitted lines are not exactly parallel. The estimated interaction is non-zero, but the data do not provide sufficient evidence that this difference is different from zero.

```
ggPredict(m5) +
  guides(color=guide_legend(override.aes=list(fill=NA))) +
  scale_color_brewer(palette = "Set2") +
  my.ggtheme
```



18.6 Example: logistic regression with categorical variable

```
# recode diabetic status to 1 and 0
data_diabetes <- data_diabetes %>%
  mutate(obese = ifelse(obese == "Yes", 1, 0))

# fit logistic regression using age and gender
```



```

m6 <- glm(obese ~ hdl + gender, family = binomial(link="logit"),
  ↪ data = data_diabetes)
summary(m6)
##
## Call:
## glm(formula = obese ~ hdl + gender, family = binomial(link =
  ↪ "logit"),
## data = data_diabetes)
##
## Coefficients:
##             Estimate Std. Error z value Pr(>|z|)
## (Intercept)  0.55047    0.58718   0.937   0.3485
## hdl         -0.02997    0.01197  -2.504   0.0123 *
## genderfemale  1.26586    0.40120   3.155   0.0016 **
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## (Dispersion parameter for binomial family taken to be 1)
##
## Null deviance: 178.71  on 129  degrees of freedom
## Residual deviance: 164.39  on 127  degrees of freedom
## AIC: 170.39
##
## Number of Fisher Scoring iterations: 4

```

- By how much change odds of suffering from obesity when hdl increases by 1?¹⁴
- What are the odds of suffering from obesity and being a women vs. suffering from obesity and being a man?¹⁵

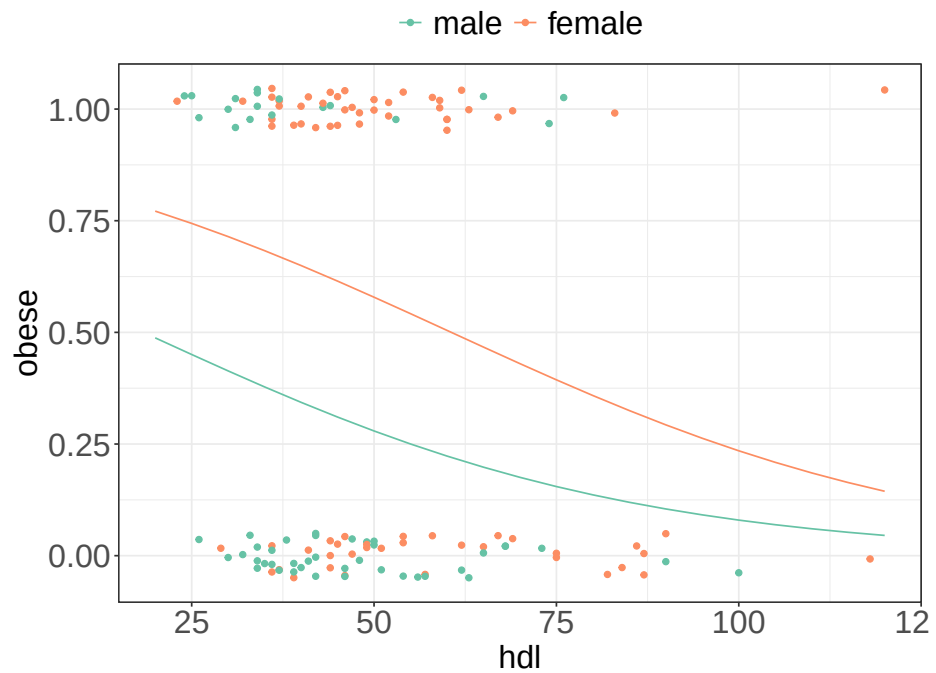
```

ggPredict(m6) +
  scale_color_brewer(palette = "Set2") +
  my.ggtheme

```

¹⁴the odds increase by $e^{\{-0.02997\}} = 0.97$

¹⁵The odds of suffering from obesity as a woman are $e^{\{1.26586\}} = 3.55$ times of that suffering from obesity and being a man.



We can predict obesity status in R for a man with hdl values of 50:

```
# define new observation
df <- data.frame(hdl = 50, gender = as.factor("male"))

# predict probability of suffering from obesity
prob_obese <- predict(m6, newdata = df, type = "response")
print(prob_obese)
##          1
## 0.2792396
```

Exercises (introduction to linear models)

Exercise 18.1 (Fitting linear model). Going back to the diabetes data, fit linear regression models using vector-matrix notations to model BMI based on age [years] and waist [m] measurements. In particular, define design matrix $\mathbf{X}$, vector of observations $\mathbf{Y}$ and vector of parameters β and use

$$\hat{\beta} = (\mathbf{X}^T \mathbf{X})^{-1} \mathbf{X}^T \mathbf{Y}$$

to find beta values estimates.

Check your calculations by fitting the model using `lm()` function.

The below code can get you started.

```
inch2m <- 2.54/100
pound2kg <- 0.45
data_diabetes <- diabetes %>%
  mutate(height = height * inch2m, height = round(height, 2))
  ↪ %>%
  mutate(waist = waist * inch2m) %>%
  mutate(weight = weight * pound2kg, weight = round(weight, 2))
  ↪ %>%
  mutate(BMI = weight / height^2, BMI = round(BMI, 2)) %>%
  mutate(obese= cut(BMI, breaks = c(0, 29.9, 100), labels =
    ↪ c("No", "Yes"))) %>%
  mutate(diabetic = ifelse(glyhb > 7, "Yes", "No"), diabetic =
    ↪ factor(diabetic, levels = c("No", "Yes"))) %>%
  na.omit()
```

💡 Solution

```

inch2m <- 2.54/100
pound2kg <- 0.45
data_diabetes <- diabetes %>%
  mutate(height = height * inch2m, height = round(height,
    ↪ 2)) %>%
  mutate(waist = waist * inch2m) %>%
  mutate(weight = weight * pound2kg, weight = round(weight,
    ↪ 2)) %>%
  mutate(BMI = weight / height^2, BMI = round(BMI, 2)) %>%
  mutate(obese= cut(BMI, breaks = c(0, 29.9, 100), labels =
    ↪ c("No", "Yes"))) %>%
  mutate(diabetic = ifelse(glyhb > 7, "Yes", "No"), diabetic
    ↪ = factor(diabetic, levels = c("No", "Yes"))) %>%
  na.omit()

# define Y
Y <- data_diabetes %>% select("BMI") %>% as.matrix()

# define X
X <- cbind(rep(1, nrow(data_diabetes)), data_diabetes$age,
  ↪ data_diabetes$waist)
X <- as.matrix(X)

# alternatively we can use model.matrix() to define X
X <- model.matrix(~ age + waist, data = data_diabetes)

# least square estimate
beta.hat <- solve(t(X)%*%X)%*%t(X)%*%Y
print(beta.hat)

```

	BMI
(Intercept)	-2.39192911
age	-0.05716479
waist	35.46856117

```

# check calculations using lm() function
reg_1 <- lm(BMI ~ age + waist, data = data_diabetes)
summary(reg_1)

```

Call:

```
lm(formula = BMI ~ age + waist, data = data_diabetes)
```

Residuals:

	Min	1Q	Median	3Q	Max
	-12.6952	-2.8803	-0.5864	2.2229	18.7443

Coefficients:

```

              Estimate Std. Error t value Pr(>|t|)
(Intercept) -2.39193    2.95646  -0.809   0.4200
age          -0.05716    0.02551  -2.241   0.0267 *
waist        35.46856    2.68192  13.225  <2e-16 ***
---
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

Residual standard error: 4.358 on 127 degrees of freedom
Multiple R-squared:  0.5841,    Adjusted R-squared:  0.5776
F-statistic: 89.19 on 2 and 127 DF,  p-value: < 2.2e-16

```

Exercise 18.2 (Hypothesis testing). Your colleague Anna is interested in association between BMI and cholesterol, both total cholesterol (chol) and high density lipoprotein fraction (hdl). She has correctly fitted linear model using `lm()` function but her computer broke and she only has the below output:

```

# Coefficients:
#              Estimate Std. Error t value Pr(>|t|)
# (Intercept)  3.471e+01  2.940e+00  11.808  < 2e-16 ***
# chol         1.965e-05  1.231e-02
# hdl          -9.371e-02  3.220e-02

```

Can you help Anna finding out whether `chol` and `hdl` are significantly associated with BMI? What are the t-value statistics and the corresponding p-values? Calculate these values without fitting the model and then fit the model to double check your calculations.

Solution

Under the null hypothesis $H_0 : \beta = 0$

$$\frac{\hat{\beta} - \beta}{e.s.e(\hat{\beta})} \sim t(n-p)$$

t-statistics Student's t distribution

- n is number of observations
- p is number of model parameters
- $\frac{\hat{\beta} - \beta}{e.s.e(\hat{\beta})}$ is the ratio of the departure of the estimated value of a parameter, $\hat{\beta}$, from its hypothesized value, β , to its standard error, called **t-statistics**
- the **t-statistics** follows Student's t distribution with $n-p$ degrees of freedom

This means that to check if there is an association between `chol` and BMI we check if there is enough evidence to reject the null hypothesis of $H_0 : \beta = 0$. Here, t-statistics equals to $\frac{\hat{\beta} - \beta}{e.s.e(\hat{\beta})} = \frac{1.965 \times 10^{-05} - 0}{1.231 \times 10^{-02}} = 0.001596263$ and a corresponding p-values can be found:

```

2*pt(0.001596263, df=130 - 3, lower.tail = F)
## [1] 0.9987289

```

Assuming 5% significance level, we do not have enough evidence to reject the null hypothesis in favor of the alternative as p-value is large $p = 0.99873 > 0.05$. This means we do not observe association between `chol` and `BMI`.

Analogously, for `age` we have t-statistics equal to $\frac{\hat{\beta} - \beta}{e.s.e(\hat{\beta})} = \frac{-9.371 \times 10^{02} - 0}{3.220 \times 10^{02}} = -2.910248$ and a corresponding p-value equals to:

```
2*pt(-2.910248, df=130 - 3, lower.tail = T)
## [1] 0.004265831
```

Here, p-value is small and we can thus reject the null hypothesis in favour of the alternative and conclude that there is an association between `hdl` and `BMI`.

To check our calculations we can re-fit the linear model and print the entire summary.

```
reg_2 <- lm(BMI ~ chol + hdl, data = data_diabetes)
summary(reg_2)
##
## Call:
## lm(formula = BMI ~ chol + hdl, data = data_diabetes)
##
## Residuals:
##      Min       1Q   Median       3Q      Max
## -12.8952  -4.8988  -0.9225   3.0629  21.7951
##
## Coefficients:
##              Estimate Std. Error t value Pr(>|t|)
## (Intercept)  3.471e+01  2.940e+00  11.808  < 2e-16 ***
## chol        1.965e-05  1.231e-02   0.002  0.99873
## hdl        -9.371e-02  3.220e-02  -2.910  0.00426 **
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1
##                  ' ' 1
##
## Residual standard error: 6.538 on 127 degrees of freedom
## Multiple R-squared:  0.0643, Adjusted R-squared:  0.04956
## F-statistic: 4.363 on 2 and 127 DF,  p-value: 0.0147
```

Exercise 18.3 (Evaluate model fit). After helping Anna you got interested in whether your initial model containing `age` and `waist` is a better fit to the data than Anna's model containing `chol` and `hdl`. Evaluate model fit by calculating $R^2(adj)$ based on the equation:

$$R^2(adj) = 1 - \frac{\frac{RSS}{n-p-1}}{\frac{TSS}{n-1}}$$

where

- p is the number of independent predictors, i.e. the number of variables in the model, excluding the constant and n is the number of observations.

Check your calculations using `lm()` function.

Hint: If `reg` holds a fitted linear regression model, you can assess the estimated BMI values by accessing `reg$fitted.values` and residuals via `reg$residuals`.

Solution

```
n <- nrow(data_diabetes)
p <- 2 # beta for age and beta for waist (model 1)
p <- 2 # beta for chol and beta for hdl (model 2)

# calculate TSS
TSS <- sum((data_diabetes$BMI - mean(data_diabetes$BMI))^2)

# calculate RSS and R2_adj (model 1)
# model BMI ~ age + waist
reg_1 <- lm(BMI ~ age + waist, data = data_diabetes)
reg <- reg_1
RSS <- sum((reg$residuals)^2)
R2_adj <- 1 - (RSS/(n-p-1))/(TSS/(n-1))
print(R2_adj)
## [1] 0.5775851

# calculate RSS and R2_adj (model 2)
# model BMI ~ chol + hdl
reg_2 <- lm(BMI ~ chol + hdl, data = data_diabetes)
reg <- reg_2
RSS <- sum((reg$residuals)^2)
R2_adj <- 1 - (RSS/(n-p-1))/(TSS/(n-1))
print(R2_adj)
## [1] 0.04956166
```

```
# check calculations
summary(reg_1)
##
## Call:
## lm(formula = BMI ~ age + waist, data = data_diabetes)
##
## Residuals:
##      Min       1Q   Median       3Q      Max
## -12.6952  -2.8803  -0.5864   2.2229  18.7443
##
## Coefficients:
##              Estimate Std. Error t value Pr(>|t|)
## (Intercept) -2.39193    2.95646  -0.809   0.4200
## age          -0.05716    0.02551  -2.241   0.0267 *
## waist        35.46856    2.68192  13.225  <2e-16 ***
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1
##                  ' ' 1
##
## Residual standard error: 4.358 on 127 degrees of freedom
## Multiple R-squared:  0.5841, Adjusted R-squared:  0.5776
## F-statistic: 89.19 on 2 and 127 DF,  p-value: < 2.2e-16
summary(reg_2)
##
## Call:
## lm(formula = BMI ~ chol + hdl, data = data_diabetes)
##
## Residuals:
##      Min       1Q   Median       3Q      Max
## -12.8952  -4.8988  -0.9225   3.0629  21.7951
##
## Coefficients:
##              Estimate Std. Error t value Pr(>|t|)
## (Intercept)  3.471e+01  2.940e+00  11.808  < 2e-16 ***
## chol         1.965e-05  1.231e-02   0.002  0.99873
## hdl          -9.371e-02  3.220e-02  -2.910  0.00426 **
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1
##                  ' ' 1
##
## Residual standard error: 6.538 on 127 degrees of freedom
## Multiple R-squared:  0.0643, Adjusted R-squared:  0.04956
##
## F-statistic: 4.363 on 2 and 127 DF,  p-value: 0.0147
```


Exercises (model diagnostics)

Data for exercises are on Canvas under Files -> data_exercises -> linear-models

Exercise 18.4 (Brozek score). Researchers collected age, weight, height and 10 body circumference measurements for 252 men in an attempt to find an alternative way of calculate body fat as oppose to measuring someone weight and volume, the latter one by submerging in a water tank. Is it possible to predict body fat using easy-to-record measurements?

Use `lm()` function and fit a linear method to model brozek, score estimate of percent body fat

- find R^2 and $R^2(adj)$
- assess the diagnostics plots to check for model assumptions
- delete observation #86 with the highest Cook's distance and re-fit the model (`model.clean`)
- look at the model summary. Are all variables associated with brozek score?
- try improving the model fit by removing variables with the highest p-value first and re-fitting the model until all the variables are significantly associated with the response (p value less than 0.1); note down the $R^2(adj)$ values while doing so
- compare the output models for `model.clean` and final model

To access and preview the data:

```
data(fat, package = "faraway")
```

Answers to selected exercises

Solution. Exercise 18.4

```
# access and preview data
data(fat, package = "faraway")
head(fat)
##   brozek siri density age weight height adipos  free neck
   ↪ chest abdom  hip
## 1   12.6 12.3  1.0708  23 154.25  67.75   23.7 134.9 36.2
   ↪ 93.1  85.2  94.5
```

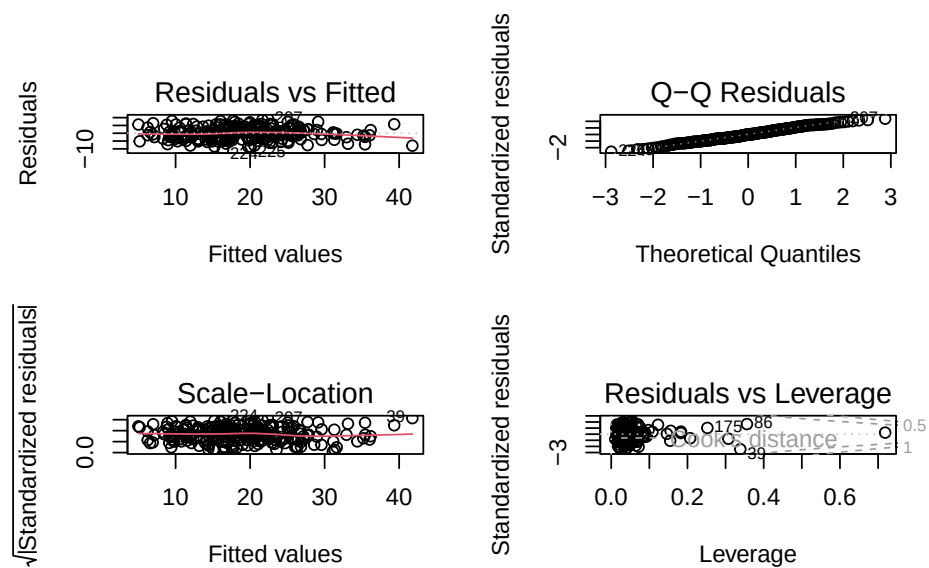
```
## 2    6.9  6.1  1.0853  22 173.25  72.25  23.4 161.3 38.5
  ↪  93.6  83.0  98.7
## 3    24.6 25.3  1.0414  22 154.00  66.25  24.7 116.0 34.0
  ↪  95.8  87.9  99.2
## 4    10.9 10.4  1.0751  26 184.75  72.25  24.9 164.7 37.4
  ↪  101.8  86.4 101.2
## 5    27.8 28.7  1.0340  24 184.25  71.25  25.6 133.1 34.4
  ↪  97.3 100.0 101.9
## 6    20.6 20.9  1.0502  24 210.25  74.75  26.5 167.0 39.0
  ↪  104.5  94.4 107.8
##   thigh knee ankle biceps forearm wrist
## 1  59.0 37.3  21.9   32.0   27.4  17.1
## 2  58.7 37.3  23.4   30.5   28.9  18.2
## 3  59.6 38.9  24.0   28.8   25.2  16.6
## 4  60.1 37.3  22.8   32.4   29.4  18.2
## 5  63.2 42.2  24.0   32.2   27.7  17.7
## 6  66.0 42.0  25.6   35.7   30.6  18.8

# fit linear regression model
model.all <- lm(brozek ~ age + weight + height + neck + abdom +
  ↪   hip + thigh + knee + ankle + biceps + forearm + wrist, data =
  ↪   fat)

# print model summary
print(summary(model.all))
##
## Call:
## lm(formula = brozek ~ age + weight + height + neck + abdom +
##     hip + thigh + knee + ankle + biceps + forearm + wrist,
##     data = fat)
##
## Residuals:
##      Min       1Q   Median       3Q      Max
## -10.2664  -2.5658  -0.0798   2.8976   9.3204
##
## Coefficients:
##              Estimate Std. Error t value Pr(>|t|)
## (Intercept) -17.063433   14.489336  -1.178  0.24011
## age          0.056520    0.029888   1.891  0.05983 .
## weight      -0.085513    0.045170  -1.893  0.05954 .
## height      -0.059703    0.086695  -0.689  0.49171
## neck        -0.439315    0.214802  -2.045  0.04193 *
## abdom        0.875779    0.070589  12.407 < 2e-16 ***
## hip         -0.192118    0.132655  -1.448  0.14885
## thigh        0.237304    0.131793   1.801  0.07303 .
## knee        -0.006595    0.222832  -0.030  0.97642
## ankle        0.164831    0.204681   0.805  0.42144
## biceps       0.149530    0.157693   0.948  0.34397
## forearm     0.424885    0.182801   2.324  0.02095 *
```

```
## wrist          -1.474317  0.494475 -2.982  0.00316 **
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Residual standard error: 3.98 on 239 degrees of freedom
## Multiple R-squared:  0.7489, Adjusted R-squared:  0.7363
## F-statistic: 59.4 on 12 and 239 DF, p-value: < 2.2e-16

# diagnostics plots
par(mfrow=c(2,2))
plot(model.all)
```



```
# remove potentially influential observations
obs <- c(86)
fat2 <- fat[-obs, ]

# re-fit the model
model.clean <- lm(brozek ~ age + weight + height + neck + abdom +
  ↪ hip + thigh + knee + ankle + biceps + forearm + wrist, data =
  ↪ fat)

# diagnostics plots
par(mfrow=c(2,2))
plot(model.clean)

# model summary
print(summary(model.clean))
##
## Call:
## lm(formula = brozek ~ age + weight + height + neck + abdom +
```

```
##      hip + thigh + knee + ankle + biceps + forearm + wrist,
  ↪ data = fat)
##
## Residuals:
##      Min       1Q   Median       3Q      Max
## -10.2664  -2.5658  -0.0798   2.8976   9.3204
##
## Coefficients:
##              Estimate Std. Error t value Pr(>|t|)
## (Intercept) -17.063433   14.489336  -1.178   0.24011
## age          0.056520    0.029888   1.891   0.05983 .
## weight      -0.085513    0.045170  -1.893   0.05954 .
## height      -0.059703    0.086695  -0.689   0.49171
## neck        -0.439315    0.214802  -2.045   0.04193 *
## abdom        0.875779    0.070589  12.407 < 2e-16 ***
## hip         -0.192118    0.132655  -1.448   0.14885
## thigh        0.237304    0.131793   1.801   0.07303 .
## knee        -0.006595    0.222832  -0.030   0.97642
## ankle        0.164831    0.204681   0.805   0.42144
## biceps       0.149530    0.157693   0.948   0.34397
## forearm      0.424885    0.182801   2.324   0.02095 *
## wrist       -1.474317    0.494475  -2.982   0.00316 **
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Residual standard error: 3.98 on 239 degrees of freedom
## Multiple R-squared:  0.7489, Adjusted R-squared:  0.7363
## F-statistic: 59.4 on 12 and 239 DF, p-value: < 2.2e-16

# re-fit the model (no height)
model.red1 <- lm(brozek ~ age + weight + neck + abdom + hip +
  ↪ thigh + knee + ankle + biceps + forearm + wrist, data = fat)
print(summary(model.red1))
##
## Call:
## lm(formula = brozek ~ age + weight + neck + abdom + hip +
  ↪ thigh +
##      knee + ankle + biceps + forearm + wrist, data = fat)
##
## Residuals:
##      Min       1Q   Median       3Q      Max
## -10.2830  -2.6162  -0.1017   2.8789   9.3713
##
## Coefficients:
##              Estimate Std. Error t value Pr(>|t|)
## (Intercept) -22.66569   11.97691  -1.892   0.05963 .
## age          0.05948    0.02954   2.013   0.04521 *
## weight      -0.09829    0.04114  -2.389   0.01765 *
## neck        -0.43444    0.21445  -2.026   0.04389 *
```

```
## abdom      0.88762    0.06839   12.979 < 2e-16 ***
## hip        -0.17180    0.12919   -1.330  0.18483
## thigh      0.25327    0.12960    1.954  0.05183 .
## knee       -0.02318    0.22128   -0.105  0.91665
## ankle      0.17300    0.20411    0.848  0.39752
## biceps     0.15695    0.15715    0.999  0.31894
## forearm    0.43091    0.18239    2.363  0.01895 *
## wrist     -1.51011    0.49120   -3.074  0.00235 **
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Residual standard error: 3.976 on 240 degrees of freedom
## Multiple R-squared:  0.7484, Adjusted R-squared:  0.7369
## F-statistic: 64.9 on 11 and 240 DF, p-value: < 2.2e-16

# re-fit the model (no knee)
model.red2 <- lm(brozek ~ age + weight + neck + abdom + hip +
  ↪ thigh + ankle + biceps + forearm + wrist, data = fat)
print(summary(model.red2))
##
## Call:
## lm(formula = brozek ~ age + weight + neck + abdom + hip +
  ↪ thigh +
##     ankle + biceps + forearm + wrist, data = fat)
##
## Residuals:
##      Min       1Q   Median       3Q      Max
## -10.2552  -2.5979  -0.1133   2.8693   9.3584
##
## Coefficients:
##              Estimate Std. Error t value Pr(>|t|)
## (Intercept) -23.08716    11.25781  -2.051  0.04137 *
## age          0.05875     0.02864   2.051  0.04134 *
## weight      -0.09965     0.03897  -2.557  0.01117 *
## neck        -0.43088     0.21131  -2.039  0.04253 *
## abdom       0.88875     0.06740  13.186 < 2e-16 ***
## hip        -0.17231     0.12884  -1.337  0.18234
## thigh       0.24942     0.12403   2.011  0.04544 *
## ankle       0.16946     0.20089   0.844  0.39974
## biceps      0.15847     0.15616   1.015  0.31123
## forearm     0.42946     0.18150   2.366  0.01876 *
## wrist     -1.51470     0.48823  -3.102  0.00215 **
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Residual standard error: 3.968 on 241 degrees of freedom
## Multiple R-squared:  0.7484, Adjusted R-squared:  0.738
## F-statistic: 71.69 on 10 and 241 DF, p-value: < 2.2e-16
```

```
# re-fit the model (no ankle)
model.red3 <- lm(brozek ~ age + weight + neck + abdom + hip +
  ↪ thigh + biceps + forearm + wrist, data = fat)
print(summary(model.red3))
##
## Call:
## lm(formula = brozek ~ age + weight + neck + abdom + hip +
  ↪ thigh +
##     biceps + forearm + wrist, data = fat)
##
## Residuals:
##      Min       1Q   Median       3Q      Max
## -10.0740  -2.5615  -0.1021   2.7999   9.3199
##
## Coefficients:
##              Estimate Std. Error t value Pr(>|t|)
## (Intercept) -20.61247   10.86240  -1.898   0.0589 .
## age          0.05727    0.02857   2.004   0.0461 *
## weight      -0.09141    0.03770  -2.424   0.0161 *
## neck        -0.45458    0.20931  -2.172   0.0308 *
## abdom        0.88098    0.06673  13.203 <2e-16 ***
## hip         -0.17575    0.12870  -1.366   0.1733
## thigh        0.25504    0.12378   2.061   0.0404 *
## biceps       0.15178    0.15587   0.974   0.3311
## forearm      0.42805    0.18138   2.360   0.0191 *
## wrist       -1.40948    0.47175  -2.988   0.0031 **
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Residual standard error: 3.965 on 242 degrees of freedom
## Multiple R-squared:  0.7477, Adjusted R-squared:  0.7383
## F-statistic: 79.67 on 9 and 242 DF, p-value: < 2.2e-16

# re-fit the model (no biceps)
model.red4 <- lm(brozek ~ age + weight + neck + abdom + hip +
  ↪ thigh + forearm + wrist, data = fat)
print(summary(model.red4))
##
## Call:
## lm(formula = brozek ~ age + weight + neck + abdom + hip +
  ↪ thigh +
##     forearm + wrist, data = fat)
##
## Residuals:
##      Min       1Q   Median       3Q      Max
## -10.0574  -2.7411  -0.1912   2.6929   9.4977
##
## Coefficients:
##              Estimate Std. Error t value Pr(>|t|)
```

```
## (Intercept) -20.06213    10.84654   -1.850    0.06558 .
## age          0.05922     0.02850    2.078    0.03876 *
## weight      -0.08414     0.03695   -2.277    0.02366 *
## neck        -0.43189     0.20799   -2.077    0.03889 *
## abdom       0.87721      0.06661   13.170 < 2e-16 ***
## hip         -0.18641     0.12821   -1.454    0.14727
## thigh       0.28644      0.11949    2.397    0.01727 *
## forearm     0.48255      0.17251    2.797    0.00557 **
## wrist       -1.40487     0.47167   -2.978    0.00319 **
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Residual standard error: 3.965 on 243 degrees of freedom
## Multiple R-squared:  0.7467, Adjusted R-squared:  0.7383
## F-statistic: 89.53 on 8 and 243 DF,  p-value: < 2.2e-16

# re-fit the model (no hip)
model.red5 <- lm(brozek ~ age + weight + neck + abdom + thigh +
  ↪ forearm + wrist, data = fat)
print(summary(model.red5))
##
## Call:
## lm(formula = brozek ~ age + weight + neck + abdom + thigh +
  ↪ forearm +
##     wrist, data = fat)
##
## Residuals:
##      Min       1Q   Median       3Q      Max
## -10.0193  -2.8016  -0.1234   2.9387   9.0019
##
## Coefficients:
##              Estimate Std. Error t value Pr(>|t|)
## (Intercept) -30.17420     8.34200  -3.617 0.000362 ***
## age          0.06149     0.02852   2.156 0.032047 *
## weight      -0.11236     0.03151  -3.565 0.000437 ***
## neck        -0.37203     0.20434  -1.821 0.069876 .
## abdom       0.85152     0.06437  13.229 < 2e-16 ***
## thigh       0.20973     0.10745   1.952 0.052099 .
## forearm     0.51824     0.17115   3.028 0.002726 **
## wrist       -1.40081     0.47274  -2.963 0.003346 **
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Residual standard error: 3.974 on 244 degrees of freedom
## Multiple R-squared:  0.7445, Adjusted R-squared:  0.7371
## F-statistic: 101.6 on 7 and 244 DF,  p-value: < 2.2e-16

# compare model.clean and final model
print(summary(model.clean))
```

```
##
## Call:
## lm(formula = brozek ~ age + weight + height + neck + abdom +
##      hip + thigh + knee + ankle + biceps + forearm + wrist,
##      data = fat)
##
## Residuals:
##      Min       1Q   Median       3Q      Max
## -10.2664  -2.5658  -0.0798   2.8976   9.3204
##
## Coefficients:
##              Estimate Std. Error t value Pr(>|t|)
## (Intercept) -17.063433   14.489336  -1.178  0.24011
## age          0.056520    0.029888   1.891  0.05983 .
## weight      -0.085513    0.045170  -1.893  0.05954 .
## height      -0.059703    0.086695  -0.689  0.49171
## neck        -0.439315    0.214802  -2.045  0.04193 *
## abdom        0.875779    0.070589  12.407 < 2e-16 ***
## hip         -0.192118    0.132655  -1.448  0.14885
## thigh        0.237304    0.131793   1.801  0.07303 .
## knee        -0.006595    0.222832  -0.030  0.97642
## ankle        0.164831    0.204681   0.805  0.42144
## biceps       0.149530    0.157693   0.948  0.34397
## forearm      0.424885    0.182801   2.324  0.02095 *
## wrist       -1.474317    0.494475  -2.982  0.00316 **
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Residual standard error: 3.98 on 239 degrees of freedom
## Multiple R-squared:  0.7489, Adjusted R-squared:  0.7363
## F-statistic: 59.4 on 12 and 239 DF, p-value: < 2.2e-16
print(summary(model.red5))
##
## Call:
## lm(formula = brozek ~ age + weight + neck + abdom + thigh +
##      forearm +
##      wrist, data = fat)
##
## Residuals:
##      Min       1Q   Median       3Q      Max
## -10.0193  -2.8016  -0.1234   2.9387   9.0019
##
## Coefficients:
##              Estimate Std. Error t value Pr(>|t|)
## (Intercept) -30.17420    8.34200  -3.617 0.000362 ***
## age          0.06149    0.02852   2.156 0.032047 *
## weight      -0.11236    0.03151  -3.565 0.000437 ***
## neck        -0.37203    0.20434  -1.821 0.069876 .
## abdom        0.85152    0.06437  13.229 < 2e-16 ***
```



```
## thigh      0.20973    0.10745    1.952 0.052099 .
## forearm    0.51824    0.17115    3.028 0.002726 **
## wrist     -1.40081    0.47274   -2.963 0.003346 **
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Residual standard error: 3.974 on 244 degrees of freedom
## Multiple R-squared:  0.7445, Adjusted R-squared:  0.7371
## F-statistic: 101.6 on 7 and 244 DF,  p-value: < 2.2e-16
```

Note: we have just run a very simple feature selection using stepwise regression. In this method, using backward elimination, we build a model containing all the variables and remove them one by one based on defined criteria (here we have used p-values) and we stop when we have a justifiable model or when removing a predictor does not change the chosen criterion significantly.

Exercises (regression coefficients)

Exercise 18.5 (Dose, genotype and enzyme activity). An experiment investigates how the activity of an enzyme changes with drug dose. Measurements were obtained from two genotypes, WT and *Mutant*, and samples were processed in three experimental batches.

Copy and run the code below to create the data.

```
genotype <- factor(
  rep(c("WT", "Mutant"), each = 24),
  levels = c("WT", "Mutant")
)

dose <- rep(
  rep(c(0, 2, 4, 6, 8, 10), each = 4),
  2
)

batch <- factor(
  rep(c("B1", "B2", "B3"), length.out = 48),
  levels = c("B1", "B2", "B3")
)

activity <- c(
  13.43, 13.44, 10.60, 10.36,
  12.34, 13.12, 12.75, 13.67,
  14.61, 15.53, 13.64, 16.10,
  14.77, 20.82, 17.47, 16.79,
  21.95, 18.53, 20.10, 20.34,
  21.73, 22.38, 20.82, 19.50,
  14.65, 14.93, 10.53, 16.83,
  15.47, 13.89, 15.64, 19.03,
  22.03, 20.09, 20.89, 19.93,
  23.02, 24.61, 21.32, 24.07,
  28.11, 22.73, 26.46, 27.81,
  29.73, 28.64, 28.97, 27.76
)
```

```
enzyme <- data.frame(  
  activity,  
  dose,  
  genotype,  
  batch  
)  
  
head(enzyme)
```

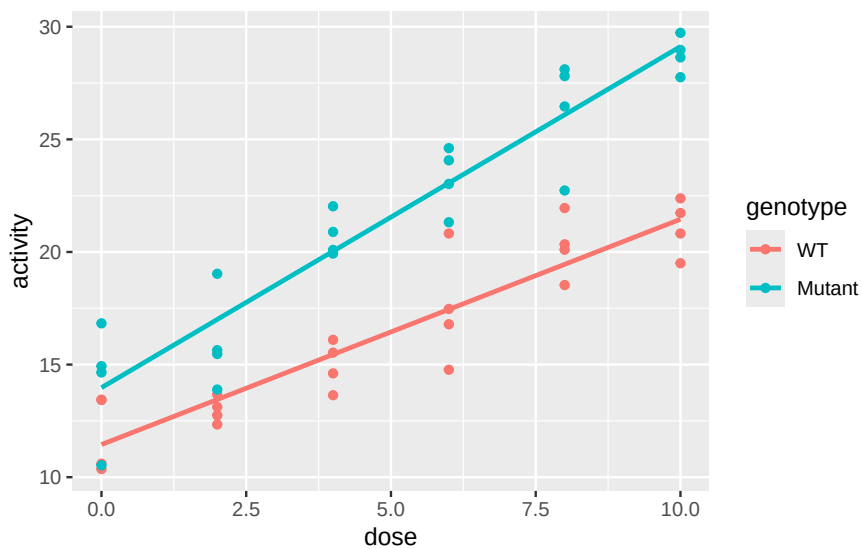
	activity	dose	genotype	batch
1	13.43	0	WT	B1
2	13.44	0	WT	B2
3	10.60	0	WT	B3
4	10.36	0	WT	B1
5	12.34	2	WT	B2
6	13.12	2	WT	B3

- Make an exploratory plot of **activity** against **dose**, using different colours for the two genotypes. What patterns do you see?
- Fit a linear model in R to model enzyme activity given **dose**, **genotype** and **batch**.
- What are the reference categories for **genotype** and **batch**?
- Given the fitted model, what is the expected enzyme activity for a **Mutant** sample with a **dose** of 5 from batch B2? Calculate using the model and check your calculations using `predict()` function.
- Which model coefficients show evidence of an association with enzyme activity at the 5% significance level?

💡 Solution

a)

```
ggplot(enzyme, aes(x = dose, y = activity, colour =  
  ↪ genotype)) +  
  geom_point() +  
  geom_smooth(method = "lm", se = FALSE)  
  
`geom_smooth()` using formula = 'y ~ x'
```



Enzyme activity appears to increase with increasing dose. The Mutant samples also tend to have higher enzyme activity than the WT samples.

b)

```
model1 <- lm(
  activity ~ dose + genotype + batch,
  data = enzyme
)
```

```
summary(model1)
```

Call:

```
lm(formula = activity ~ dose + genotype + batch, data = enzyme)
```

Residuals:

Min	1Q	Median	3Q	Max
-3.7464	-0.8783	0.1982	1.0280	3.2338

Coefficients:

	Estimate	Std. Error	t value	Pr(> t)
(Intercept)	10.19616	0.61019	16.710	< 2e-16 ***
dose	1.27206	0.07497	16.968	< 2e-16 ***
genotypeMutant	5.09792	0.50846	10.026	7.97e-13 ***
batchB2	0.69710	0.62386	1.117	0.270
batchB3	-1.01768	0.62722	-1.623	0.112

Signif. codes: 0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

Residual standard error: 1.761 on 43 degrees of freedom

Multiple R-squared: 0.9015, Adjusted R-squared: 0.8924

F-statistic: 98.4 on 4 and 43 DF, p-value: < 2.2e-16

c)

The reference categories are: WT for genotype and B1 for batch. The coefficients for genotypeMutant, batchB2 and batchB3 therefore describe differences relative to these reference categories.

d)

From the fitted model,

$$\widehat{activity} = 10.196 + 1.272 \cdot dose + 5.098 \cdot I_{\text{Mutant}} + 0.697 \cdot I_{\text{B2}} - 1.018 \cdot I_{\text{B3}}.$$

For a Mutant sample with dose = 5 from batch B2:

$$\widehat{activity} = 10.196 + 1.272 \cdot 5 + 5.098 + 0.697 \approx 22.35.$$

We can also obtain the prediction in R:

```
new_sample <- data.frame(
  dose = 5,
  genotype = factor("Mutant", levels =
    ↪ levels(enzyme$genotype)),
  batch = factor("B2", levels = levels(enzyme$batch))
)

predict(model1, newdata = new_sample)
```

```
      1
22.35146
```

e)

At the 5% significance level:

- dose shows evidence of an association with enzyme activity ($p < 2 \times 10^{-16}$);
- genotypeMutant shows evidence of an association with enzyme activity ($p = 7.97 \times 10^{-13}$);
- there is no evidence that batchB2 differs from the reference batch B1 ($p = 0.270$);
- there is no evidence that batchB3 differs from the reference batch B1 ($p = 0.112$).

Thus, dose and genotype are clearly associated with enzyme activity in this fitted model, while the individual batch coefficients are not significantly different from the reference batch.

Exercise 18.6 (Treatment response). A clinical study investigates whether a biomarker and treatment are associated with the probability of responding to treatment.

The outcome **response** is coded as:

- 1: responded to treatment
- 0: did not respond

The data also contain:

- **marker**: a continuous biomarker measurement
- **treatment**: either Control or Drug

- **age**: age in years

Copy and run the code below to create the data.

```
response <- c(
  0,1,0,0,1,1,1,1,0,1,
  0,0,0,1,1,1,1,1,0,1,
  0,0,0,1,0,1,1,1,0,1,
  0,0,0,1,0,1,1,1,1,1,
  1,1,1,1,0,1,0,1,0,1,
  0,0,0,0,0,1,1,1,0,1
)

marker <- rep(
  c(0.5, 1, 1.5, 2, 2.5, 3, 3.5, 4, 1.2, 2.8),
  6
)

treatment <- factor(
  rep(c("Control", "Drug"), 30),
  levels = c("Control", "Drug")
)

age <- rep(
  c(35, 42, 51, 60, 67, 48, 55, 39, 63, 45),
  6
)

trial <- data.frame(
  response,
  marker,
  treatment,
  age
)

head(trial)
```

	response	marker	treatment	age
1	0	0.5	Control	35
2	1	1.0	Drug	42
3	0	1.5	Control	51
4	0	2.0	Drug	60
5	1	2.5	Control	67
6	1	3.0	Drug	48

- Fit a logistic regression model to model the probability of **response** given **marker**, **treatment** and **age**.
- What is the reference category for **treatment**? What comparison does the coefficient **treatmentDrug** therefore represent?
- Using the Wald tests reported in the model summary, which coefficients

show evidence of an association with the probability of response at the 5% significance level?

- d) The estimated coefficient for **marker** is approximately 1.75.
- Calculate the corresponding odds ratio.
 - Interpret this odds ratio in the context of the study.
- e) The estimated coefficient for **treatmentDrug** is approximately 1.51.
- Calculate the corresponding odds ratio.
 - Interpret this odds ratio in the context of the study.
- f) Using the fitted model, estimate the probability of response for a 50-year-old patient with a **marker** value of 2.5 who receives the **Drug** treatment. Check your answer using `predict()` with `type = "response"`.

💡 Solution

a)

Fit the logistic regression model:

```
model2 <- glm(
  response ~ marker + treatment + age,
  family = binomial(link = "logit"),
  data = trial
)
```

```
summary(model2)
```

Call:

```
glm(formula = response ~ marker + treatment + age, family = binomial(link = "logit",
  data = trial)
```

Coefficients:

	Estimate	Std. Error	z value	Pr(> z)
(Intercept)	-0.96994	2.18446	-0.444	0.657031
marker	1.75274	0.51477	3.405	0.000662 ***
treatmentDrug	1.50666	0.76777	1.962	0.049717 *
age	-0.05776	0.04477	-1.290	0.197067

Signif. codes: 0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

(Dispersion parameter for binomial family taken to be 1)

Null deviance: 82.108 on 59 degrees of freedom
 Residual deviance: 47.317 on 56 degrees of freedom
 AIC: 55.317

Number of Fisher Scoring iterations: 6

b)

The reference category for treatment is Control.

Therefore, the coefficient `treatmentDrug` compares patients receiving the Drug treatment with patients receiving the Control treatment, while marker and age are held constant.

c)

At the 5% significance level:

- marker shows evidence of an association with the probability of response ($p = 0.000662$);
- `treatmentDrug` shows evidence of an association with the probability of response ($p = 0.0497$);
- age does not show evidence of an association with the probability of response ($p = 0.197$).

d) The estimated coefficient for marker is

$$\hat{\beta}_{\text{marker}} = 1.753.$$

The corresponding odds ratio is

$$\exp(1.753) \approx 5.77.$$

Thus, for a one-unit increase in marker, the odds of treatment response are estimated to be approximately 5.8 times higher, with treatment and age held constant.

e) The estimated coefficient for `treatmentDrug` is

$$\hat{\beta}_{\text{Drug}} = 1.507.$$

The corresponding odds ratio is

$$\exp(1.507) \approx 4.51.$$

Thus, patients receiving the Drug have approximately 4.5 times the odds of response compared with patients receiving Control, with marker and age held constant.

The odds ratios can also be calculated in R:

```
#|code-fold: false
exp(coef(model2))
```

(Intercept)	marker	treatmentDrug	age
0.3791070	5.7703864	4.5116284	0.9438804

f)

For a 50-year-old patient with marker = 2.5 receiving the Drug, the fitted log-odds are

$$-0.970 + 1.753(2.5) + 1.507 - 0.058(50) \approx 2.03.$$

Converting the log-odds to a probability gives

$$p = \frac{\exp(2.03)}{1 + \exp(2.03)} \approx 0.884.$$

Thus, the fitted model predicts a probability of response of approximately 88%.

We can check this directly in R:

```
new_patient <- data.frame(  
  marker = 2.5,  
  treatment = factor(  
    "Drug",  
    levels = levels(trial$treatment)  
  ),  
  age = 50  
)  
  
predict(  
  model2,  
  newdata = new_patient,  
  type = "response"  
)
```

```
      1  
0.8839909
```

Part IV

Mixed models

In this short introduction we are going to learn about linear mixed models. We will start with the issues with modelling hierarchical data using simple linear regression. After that, we will formally introduce mixed models and show some applications of them in the context of biological data analysis.

i Note

This chapter offers an introduction to linear mixed models, focusing on their applications within biological data analysis. It is important to understand that our exploration of mixed models here is not exhaustive. The field of mixed models is extensive and complex, with a variety of techniques and considerations that go beyond the scope of this text. If you seek a comprehensive understanding of the subject, please consult additional resources and literature dedicated to statistical modelling and mixed models. Our goal is to provide a foundational understanding and illustrate the potential of mixed models in addressing hierarchical data challenges in biological research.

Environment

Packages

In order to run the code in this tutorial we will need to install a few packages.

```
install.packages(c("lme4", "emmeans", "ggplot2", "performance"))
```


Chapter 19

Simple linear regression

Imagine we are given a dataset of only two variables (measurements) for a number of people. This could be the expression of a single gene or a protein, body mass index (BMI), blood pressure, age, or any other quantifiable trait.

```
simulate_grouped_trend <- function(group_count = 5,  
  ↪ points_per_group = 10, global_slope = -10, global_intercept =  
  ↪ 30, group_slope = 2, noise_sd = 50) {  
  set.seed(123) # Setting a seed for reproducibility  
  
  # Initialize an empty data frame to store the simulated data  
  data <- data.frame(x = numeric(), y = numeric())  
  
  # Loop to create each group  
  for (i in 1:group_count) {  
    x_start <- 12 + (i - 1) * (10 / group_count) # Stagger the  
    ↪ start of x for each group  
    x <- runif(points_per_group, min = x_start, max = x_start +  
    ↪ (10 / group_count))  
  
    # Apply a local positive trend within the group, but maintain  
    ↪ the global negative trend  
    local_intercept <- global_intercept + global_slope * (x_start  
    ↪ + (10 / (2 * group_count))) + rnorm(1, mean = 0, sd =  
    ↪ noise_sd)  
    y <- local_intercept + group_slope * (x - x_start) +  
    ↪ rnorm(points_per_group, mean = 0, sd = noise_sd)  
  
    # Combine this group with the overall dataset  
    group_data <- data.frame(x = x, y = y, group=i)  
    data <- rbind(data, group_data)  
  }  
  
  return(data)
```

```

}

# generate simulated data
data_int <- simulate_grouped_trend(group_count =
  ↪ 4, points_per_group = 10, global_slope = -2, global_intercept =
  ↪ 100, group_slope = 4, noise_sd = 5)

# plot the data
plot(data_int$x, data_int$y, xlab="Age", ylab="Protein expression")

```

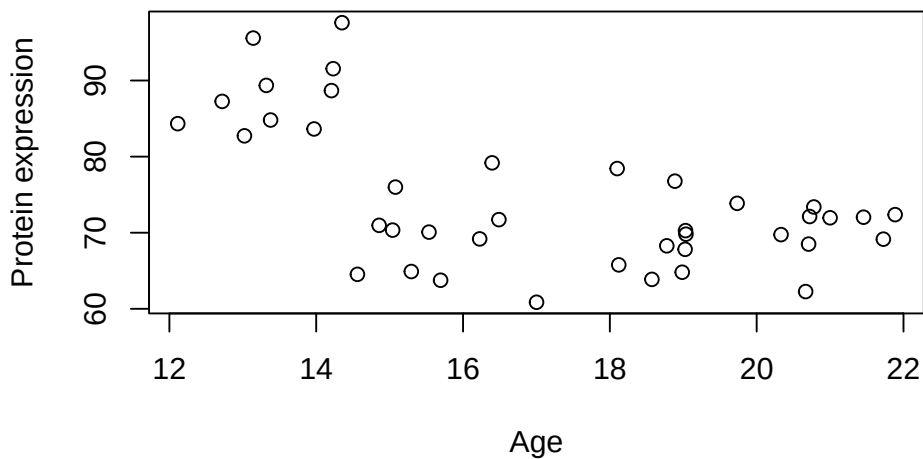


Figure 19.1: A scatter plot of Age vs. Protein expression

Our aim here is to examine the relationship between Age and Protein expression, we would apply a simple linear regression model. This statistical method allows us to quantify the relationship between an independent variable (Age) and a dependent variable (Protein expression), assuming a linear relationship between them. The goal here is to determine how Protein expression changes with Age.

```

# Fit a linear regression model
model <- lm(y ~ x, data = data_int)

# Plot the data with the regression line
plot(data_int$x, data_int$y, xlab="Age", ylab="Protein
  ↪ expression", main="Linear Regression: Age vs. Protein
  ↪ Expression")
abline(model, col="red") # Add the regression line in red

```


Linear Regression: Age vs. Protein Expression

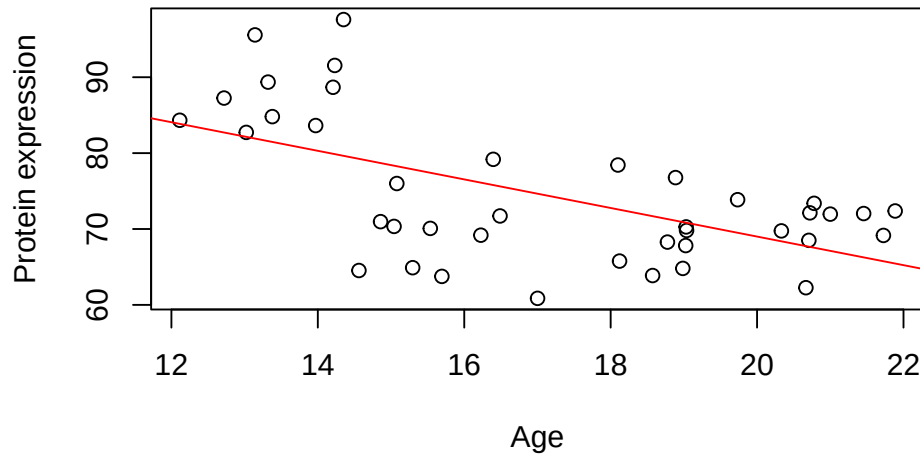


Figure 19.2: Scatter plot of Age vs. Protein expression including the regression line

The `lm()` function fits a linear model to the data, allowing us to see the relationship between Age and Protein expression. What this plot suggests is that there is a global decrease in Protein expression as Age increases, which is depicted by the negative slope of the red regression line. The scatter of points around the line indicates variability in Protein expression that is not explained by Age alone.

Based on the output from the `lm()` function, we can extract the coefficients and examine the fit of our linear regression model.

```
# Summary of the model to see the coefficients
summary(model)
```

Call:

```
lm(formula = y ~ x, data = data_int)
```

Residuals:

Min	1Q	Median	3Q	Max
-14.7166	-6.2725	0.7094	5.0500	17.9565

Coefficients:

	Estimate	Std. Error	t value	Pr(> t)
(Intercept)	106.6762	7.2948	14.62	< 2e-16 ***
x	-1.8836	0.4204	-4.48	6.63e-05 ***

Signif. codes: 0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

Residual standard error: 7.761 on 38 degrees of freedom

Multiple R-squared: 0.3457, Adjusted R-squared: 0.3285

F-statistic: 20.07 on 1 and 38 DF, p-value: 6.627e-05

The intercept, estimated at 106.6762, represents the expected Protein expression when Age is zero. Although having an age of zero is not meaningful in this context, the intercept gives us a starting reference point for the model.

The slope coefficient for Age (x) is -1.8836, indicating that for each one-unit increase in Age, Protein expression is expected to decrease by approximately 1.88 units. This negative association is significant, as indicated by the p-value of 6.63e-05, which is much lower than the conventional threshold of 0.05 for statistical significance.

The standard error of the slope tells us the uncertainty in the estimated slope. Here, the standard error for the slope is 0.4204, which is relatively small compared to the estimate itself, suggesting a precise estimate of the slope. So we are pretty confident in what we have estimated. However, while the regression line provides a clear visual indication of the overall trend, it does not account for the potential clustering of data points.

If you look more closely, you can see four different clusters of the data.

```
# Plot the data with different colors for each group
plot(data_int$x, data_int$y, xlab="Age", ylab="Protein
  ↪ expression", main="Linear Regression: Age vs. Protein
  ↪ Expression", col=data_int$group)

# Add a linear regression line
abline(lm(y ~ x, data = data_int), col="red")

# Function to draw ellipse
draw_ellipse <- function(x, y, level = 0.95, col){
  # Calculate the means of x and y
  xbar <- mean(x)
  ybar <- mean(y)

  # Calculate the standard deviations
  std_x <- sd(x)*2
  std_y <- sd(y)*2

  # Calculate the correlation
  correlation <- cor(x, y)

  # Create a sequence of angles
  t <- seq(0, 2*pi, length.out = 100)

  # Calculate the ellipse points
  a <- std_x * sqrt(1 + correlation)
  b <- std_y * sqrt(1 - correlation)
  ellipse_x <- xbar + a * cos(t)
  ellipse_y <- ybar + b * sin(t)

  # Draw the ellipse
```

```

    lines(ellipse_x, ellipse_y, col=col, lwd=2)
  }

# Draw ellipses for each group
unique_groups <- unique(data_int$group)
colors <- rainbow(length(unique_groups))
for (group in unique_groups) {
  group_data <- data_int[data_int$group == group, ]
  draw_ellipse(group_data$x, group_data$y, col=group)
}

```

Linear Regression: Age vs. Protein Expression

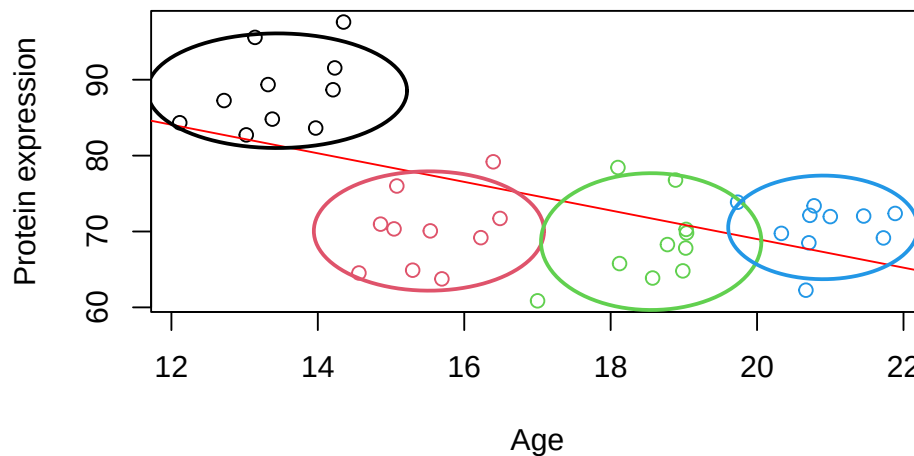


Figure 19.3: Scatter plot of Age vs. Protein expression including the regression line. The clusters are shown by colors

While the global pattern seems decreasing, the local pattern per cluster of data increases with Age.

```

# Plot the data with different colors for each group
plot(data_int$x, data_int$y, xlab="Age", ylab="Protein
  ↪ expression", main="Linear Regression: Age vs. Protein
  ↪ Expression", pch=19, col=data_int$group)

# Draw ellipses and regression lines for each group
unique_groups <- unique(data_int$group)
colors <- rainbow(length(unique_groups))

for (group in unique_groups) {
  group_data <- data_int[data_int$group == group, ]
  # Draw ellipse
  draw_ellipse(group_data$x, group_data$y, col=group)
}

```

```
# Fit linear model for the group
group_model <- lm(y ~ x, data=group_data)

# Get regression line points
reg_line_x <- range(group_data$x)
reg_line_y <- predict(group_model,
  ↪ newdata=data.frame(x=reg_line_x))

# Draw the regression line for the group
lines(reg_line_x, reg_line_y, col=group, lwd=2)
}
```

Linear Regression: Age vs. Protein Expression

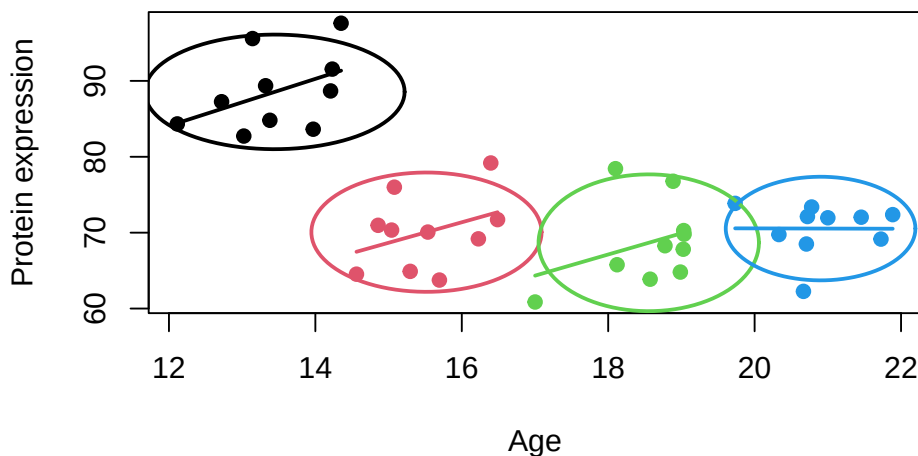


Figure 19.4: Scatter plot of Age vs. Protein expression including the regression line. The clusters are shown by colors

More precisely, there is a positive trend in each of the clusters that has not been captured by the model. Instead the model focused on the global pattern. The example that we just saw was a case when both Protein expression and Age are correlated within each cluster.

You could of course fit a separate model for each group, or include group as a **covariate** in the model. Both of these options can be OK in circumstances that we don't have too many groups and we have enough data in each of groups. Below we include cluster as a covariate to see what happens to the coefficients.

```
# Fit a linear regression model
data_int$group <- as.factor(data_int$group)
model_cov <- lm(y ~ x+group, data = data_int)

# Get the predictions
```

```

data_int$predicted <- predict(model_cov)

# Plotting
plot(data_int$x, data_int$y, xlab="Age", ylab="Protein
  ↪ expression", main="Linear Regression: Age vs. Protein
  ↪ Expression", pch=19, col=data_int$group)

# Add predicted lines
for(i in seq_along(unique(data_int$group))){
  group_subset <- data_int[data_int$group == unique_groups[i], ]
  lines(group_subset$x, group_subset$predicted, col = i, lwd = 2)
}

```

Linear Regression: Age vs. Protein Expression

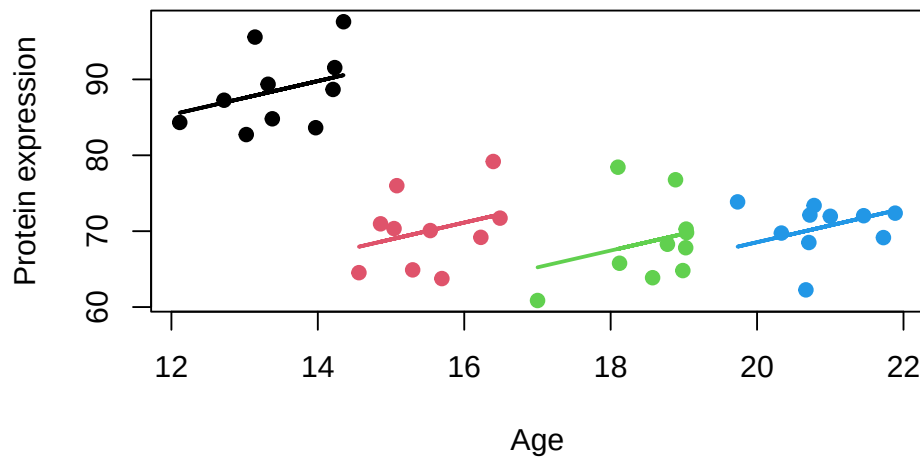


Figure 19.5: Scatter plot of Age vs. Protein expression including the regression line including group covariate

As it is clear in the figure, the function `lm(y ~ x+group, data = data_int)` has fitted four different lines (one for each group). These lines are parallel to each other while each group has a different intercept.

To make this even more clear we can have a look at the coefficients of the the model:

```
print(model_cov)
```

Call:

```
lm(formula = y ~ x + group, data = data_int)
```

Coefficients:

(Intercept)	x	group2	group3	group4
58.798	2.213	-23.072	-31.187	-34.509

Starting with the intercept, which is 58.798, this tells us the expected value of y when x is at 0 and when we are considering the baseline group (`group1`). It's like saying, "If we don't move x and if we're looking at `group1`, y would be around 58.798."

Next, we have the coefficient for x , which is 2.213. This number shows how y changes with a one-unit increase in x , across all groups. It's a universal slope, implying that regardless of which group we're looking at, if x goes up by one, y tends to increase by about 2.213 units, holding everything else constant.

Now, let's talk about the interesting part - the different intercepts for each group beyond the first. The coefficients for `group2` (-23.072), `group3` (-31.187), and `group4` (-34.509) adjust the baseline level of y for each group compared to `group1`. What this means is that if we're in `group2` and x is 0, y is expected to be 23.072 units lower than the baseline of `group1`. Similarly, for `group3` and `group4`, y would be 31.187 and 34.509 units lower than `group1`'s baseline, respectively, when x is 0.

These differences highlight that our groups have unique starting points for y , which aren't explained by x alone. There are underlying characteristics or conditions specific to each group that affect y , independent of x . By including these group differences in our model, we can accurately capture the unique relationship between x and y for each group, showing that not every group begins from the same place.

Having explored how different groups can have unique starting points or intercepts in our model, it's natural to wonder if the relationship between x and y —our slope—might also vary across these groups. Until now, we've assumed that a one-unit increase in x has the same impact on y for all groups. But what if this isn't the case? What if the effect of x on y is stronger for some groups than for others?

To explore this possibility, we introduce the concept of interaction terms in our model. By fitting a new model, `model_cov <- lm(y ~ x*group, data = data_int)`, we're not just looking at how x and `group` independently predict y , but also how the effect of x on y might change depending on the group. This model allows x to have a different slope for each group, providing a richer understanding of our data. It's a step towards uncovering more complex patterns and interactions that might exist, shedding light on the nuanced ways in which our variables interplay.

```
# Fit a linear regression model
data_int$group <- as.factor(data_int$group)
model_cov <- lm(y ~ x*group, data = data_int)

# Get the predictions
data_int$predicted <- predict(model_cov)

# Plotting
plot(data_int$x, data_int$y, xlab="Age", ylab="Protein
  ↪ expression", main="Linear Regression: Age vs. Protein
  ↪ Expression", pch=19, col=data_int$group)
```

```
# Add predicted lines
for(i in seq_along(unique(data_int$group))){
  group_subset <- data_int[data_int$group == unique_groups[i], ]
  lines(group_subset$x, group_subset$predicted, col = i, lwd = 2)
}

print(model_cov)
```

Call:

```
lm(formula = y ~ x * group, data = data_int)
```

Coefficients:

(Intercept)	x	group2	group3	group4	x:group2
46.9913	3.0911	-18.8314	-30.0940	24.0594	-0.3905
x:group3	x:group4				
-0.3006	-3.1154				

Linear Regression: Age vs. Protein Expression

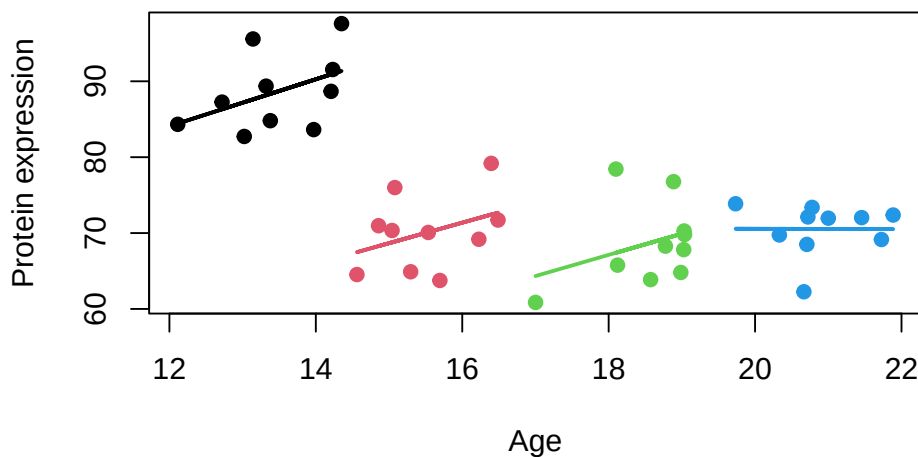


Figure 19.6: Scatter plot of Age vs. Protein expression including the regression line with interaction

The interaction terms in our new model `model_cov <- lm(y ~ x*group, data = data_int)` reveal yet another layer of complexity. Now, not only do we recognize that each group starts at a different point, but we also see that the relationship between x and y —how y changes as x increases—is not a one-size-fits-all across the groups.

Now, let's interpret the coefficients:

1. **(Intercept) 46.9913:** This is the expected protein expression when age is zero for the baseline group (`group1`).

2. **x 3.0911**: This coefficient represents the slope of the line for the baseline group. For each additional year of age, the protein expression is expected to increase by 3.0911 units for **group1**.
3. **group2 -18.8314**: This coefficient adjusts the intercept for **group2**. So, the expected starting point for protein expression in **group2**, when age is zero, is 46.9913 (the baseline intercept) minus 18.8314, giving us a starting point of 28.1599.
4. **group3 -30.0940**: Similarly, for **group3**, the expected starting point for protein expression when age is zero is 46.9913 minus 30.0940, resulting in 16.8973.
5. **group4 24.0594**: For **group4**, the expected starting point is 46.9913 plus 24.0594, which is 71.0507, indicating that **group4** has a higher starting point than **group1**.
6. **x:group2 -0.3905**: This is the interaction term for age and **group2**, indicating how the slope for **group2** differs from the baseline group's slope. The slope for **group2** is 3.0911 (baseline slope) minus 0.3905, which is approximately 2.7006. This suggests that for **group2**, for each additional year of age, the protein expression increases by about 2.7006 units, which is less than the increase for **group1**.
7. **x:group3 -0.3006**: For **group3**, the slope is 3.0911 minus 0.3006, approximately 2.7905. So the rate of increase in protein expression per year of age for **group3** is slightly less than for **group1** but not as much less as for **group2**.
8. **x:group4 -3.1154**: This interaction term indicates a significant change in the slope for **group4** compared to **group1**. The slope for **group4** is 3.0911 minus 3.1154, which is approximately -0.0243. This would suggest a slight decrease in protein expression with age for **group4**, which is contrary to the positive association seen in the other groups.

The interaction terms, specifically the coefficients for **x:group2**, **x:group3**, and **x:group4**, tell us just how much the slope of the relationship between **x** and **y** differs for each group compared to our baseline **group1**. For example, **group2** has a slightly lesser slope than **group1**, indicating that as **x** increases, **y** increases but not as rapidly as in **group1**. **Group3** follows a similar pattern, albeit the difference is more modest.

Most intriguing is **group4**, where the interaction term is quite pronounced, suggesting that the increase in **y** with **x** is significantly less for **group4** compared to **group1** in fact, it's almost negligible, indicating a nearly flat line. This could mean that for individuals in **group4**, changes in **x** do not translate into changes in **y** as they do for the other groups.

What we just saw was a simple example of how to handle clustered data in a simple linear regression. Without accounting for these cluster-related differences, the assumption of independence among observations—a cornerstone of linear regression—is violated, as the outcome within a cluster are inherently correlated. This correlation persists even when attempting to control for the cluster's effect, due to other unobservable factors.

The consequence of ignoring cluster-level effects might lead to incorrect inferences, as standard errors are usually underestimated, making them appear more precise than they actually are. While the regression coefficients themselves might remain unbiased, the underestimation of standard errors can falsely enhance the significance of findings. Furthermore, we often tend to overestimate the degrees of freedom, which can lead to too low p-values.

Addressing this issue often involves introducing indicator variables (similar to what we have done above) to account for cluster effects, allowing for different intercepts or even slope for each cluster. However, this approach becomes impractical with a large number of clusters or when dealing with longitudinal data. Furthermore, we might not even be interested in modeling clusters parameters, we are just interested in some specific questions. Think about the simple dataset we just analyzed, the main effect of interest was **Age**, however, we had to estimate a lot more parameters in the model to account for the cluster effect. More specifically, in `lm(y~x+group)` we estimate *five* parameters and in `lm(y~x*group,data_int)` we had *eight* parameters while we basically needed *two*. These parameters are all degrees of freedom that we spend while we might not even be interested in the cluster level parameters. Now imagine we might have hundreds of nested clusters with limited data, such modeling attempts will quickly become infeasible and may lead to overfitting. In such cases, the model would consume degrees of freedom excessively, reducing the power of statistical tests and potentially leading to unreliable estimates due to the curse of dimensionality. Moreover, with a high number of parameters relative to the number of observations, standard errors of the estimates increase, which can make the model's predictions less precise.

To address these issues, we often resort to mixed-effects models which is the topic of the next chapter.

Chapter 20

Mixed models

As briefly mentioned in the previous chapter, mixed-effects models (also known as multilevel or hierarchical models) are used when observations are grouped and hence not independent. This approach allows us to model the dependency structure due to clusters without having to estimate a separate parameter for each cluster effect. Mixed-effects models incorporate the clustering in the variance-covariance structure of the data, which provides a more parsimonious and interpretable model, especially when our primary interest lies in the fixed effects, such as the main effect of **Age** in the previous example.

Correlated observations are more common in biology than one might think. For example:

- **Repeated Measurements:** Suppose you're measuring a patient's blood pressure over time. Measurements from the same patient will naturally be more similar to each other than those from different patients. A mixed-effects model handles this by modeling the variation between patients.
- **Nested Designs:** Imagine studying the effectiveness of a new drug on mice housed within different cages. Mice within the same cage might be more alike due to shared environments. A mixed-effects model accounts for both variation between mice and variation between cages.
- **Longitudinal Studies:** When tracking the growth of trees over time, measurements from the same tree will be more correlated than those from different trees. Here, a mixed-effects model can account for the individual tree's growth patterns.
- **Multi-omics Studies:** Omics data (genomics, transcriptomics, proteomics, etc.) from the same individual will show correlations due to underlying biological pathways. A mixed-effects model can tease apart the effects of different omic layers while accounting for their relationships within a person.
- **Family-based Disease Studies:** Genetic variations and environmental exposures tend to cluster within families. Mixed-effects models can effectively model these familial factors when investigating how they contribute

to disease risk.

Mixed-effects models are a powerful tool for analyzing complex biological datasets, offering improved accuracy and providing valuable insights into the sources of variability in disease studies.

20.1 Why “Mixed”?

Mixed models get their name because they combine two types of effects: fixed effects and random effects. This combination is what makes them so valuable for analyzing complex data.

20.1.1 Fixed Effects

The statistical models you might be familiar with, like linear regression or ANOVA, focus on “fixed effects.” These are the direct relationships between your chosen variables and the outcome you’re interested in. Fixed effects are the factors you control or are primarily interested in. If you’re studying the effect of age on protein expression, age is your fixed effect. A fixed effect will contain all possible levels of a factor in the experiment, measuring a few specific instances of interest. The key premise of the fixed-effect model is that there is one true effect size that is common to all the studies being analyzed. This model assumes that any observed variation among study results is solely due to sampling error within studies, and not due to actual differences in effect sizes across studies. Therefore, the goal of a fixed-effect analysis is to estimate this common true effect size.

Let’s think about a research question asking if a specific vitamin supplement reduces the duration of common cold symptoms. Multiple clinical trials have been conducted to evaluate the effect of this vitamin supplement on the duration of common cold symptoms. Each trial uses the same dosage of the supplement and has similar participant demographics (e.g., age, health status). The researchers conducting the analysis believe that, given the standardized intervention and population, the supplement should have a consistent effect across all trials.

Studies Reviewed: - Study 1: 100 participants, average reduction in symptom duration of 1.5 days. - Study 2: 150 participants, average reduction in symptom duration of 1.4 days. - Study 3: 120 participants, average reduction in symptom duration of 1.6 days. - Study 4: 130 participants, average reduction in symptom duration of 1.3 days.

Fixed-Effect Model Assumptions in this Example:

- **Single True Effect Size:** The assumption here is that there is one true effect size reflecting the average reduction in the duration of common cold symptoms due to the vitamin supplement. This is based on the controlled administration of the supplement and the homogeneous nature of the participant groups across the trials.
- **Variation Due to Sampling Error:** The slight differences in observed effects among the studies (e.g., some showing a 1.3-day reduction while others show a 1.6-day reduction) are attributed to sampling error—random variation due to the different samples of participants in each study.

- **Goal:** To estimate the common true effect size of the vitamin supplement on reducing the duration of cold symptoms, a fixed-effect analysis is employed. This involves calculating a weighted average of the effect sizes from the individual trials, with greater weight given to larger trials since they are less prone to sampling error.

Through the fixed-effect model analysis, it might be concluded that the specific vitamin supplement leads to an average reduction of approximately 1.4 days in the duration of common cold symptoms. This conclusion is based on the premise that the variation in results across the trials is solely due to sampling error, without accounting for potential underlying differences in participant response to the supplement, as such variability is assumed to be negligible given the standardized conditions of the trials.

The primary limitation of using a fixed-effect model, lies in its underlying assumptions about the homogeneity of effect sizes across studies. This model assumes that there is one true effect size that applies to all studies, and any observed variability in outcomes is solely due to sampling error. However, this assumption can be overly simplistic and may not always hold true, especially in biological and medical research where variability is the norm rather than the exception.

20.1.2 Random effect

The random-effects offer a more flexible and realistic approach, especially in contexts where between-study variability is expected or observed. The defining feature of the random-effects model is the assumption that there is a distribution of true effect sizes across studies, and the goal is to estimate the mean of this distribution. This approach accounts for variation not only within studies (due to sampling error) but also between studies, recognizing that different studies may inherently have different true effect sizes due to various factors (e.g., differences in populations, interventions, outcomes measured).

Returning to our scenario of evaluating the impact of a vitamin supplement on the duration of common cold symptoms, let's consider that the clinical trials are conducted across various geographical locations, each with a distinct demographic and environmental profile. These location-based groups could inherently influence how participants respond to the supplement, independent of the supplement's effect itself. We would introduce random intercepts for each geographical location. This means that while we are still interested in estimating the overall effect of the vitamin supplement on symptom duration, we also acknowledge that each location starts from a different baseline in terms of average symptom duration without the supplement. However, we actually don't directly estimate these baselines but rather assume that these baselines are randomly sampled from a distribution of baselines in the population with certain statistical properties. Therefore, we are not interested in each single baseline but we want to know statistical properties of the distribution of baselines.

In summary, the fixed-effect model's generalization is limited to the “levels” within the experiment, meaning it applies to populations or conditions that exactly match those of the included studies. It treats the effect as if it were a fixed property of the specific scenarios tested. In contrast, the random-effects

model allows for generalization to “levels beyond those that took part in the study,” acknowledging that the effect sizes are part of a distribution across a wider population. This model treats the effect as a variable property reflecting a range of possible scenarios, including those not directly observed in the analysis.

This might sound too complicated or abstract. We are going to clarify this with a few examples.

20.2 Ingredients

In this section, we will try to clarify what information do we need in order to fit a mixed model.

20.2.1 Grouping Structure of the Data

The first and most important thing is to understand the data we are dealing with. Before we can make any attempt to use mixed models, we need to know what kind of grouping structure we want to capture. Mixed models are particularly useful when data is collected in groups or clusters, such as measurements from different subjects, schools, geographical locations, or time points within subjects. Identifying the hierarchy or nested structure where observations are grouped within higher-level units is crucial. This grouping structure can significantly influence the correlations among observations, as data points within the same group are likely to be more similar to each other than to data points in different groups. Understanding this structure allows us to correctly model the random effects, which account for the variability at different levels of the data hierarchy, thereby improving the accuracy and interpretability of our results. By incorporating these random effects, mixed models enable us to make inferences about both the fixed effects, which are consistent across groups, and the random effects, which vary across groups.

Without having this information, we cannot use standard mixed models. This grouping must be categorical, not continuous, as mixed models rely on categorical variables to define the levels of the hierarchy or nested structure within the data. Categorical grouping allows us to classify observations into distinct categories or groups that share common characteristics.

The grouping factor identifies the clusters. The random effect is the model component allowed to vary across these clusters. Random effects are calculated based on the assumption that data points within the same group may share more similarities with each other than with data points from other groups, thus accounting for the within-group correlation. Therefore, the identification of categorical groupings is not just a step in the analysis but a prerequisite for accurately calculating and interpreting the random effects that are fundamental to the mixed model’s approach to data analysis.

! Important

A grouping variable in standard mixed models should have repeated levels defining the clusters such as patient, clinic, cage or family. The grouping

cannot be defined by unique continuous values such as age or BMI.

i Note

Sometimes it is not clear what grouping of data exist. Or even what should be used as grouping. Often (but not always), the grouping is something different than what we are interested in. For example, let's say, we have gathered data from patients with a disease and attempted to find a matched control based on certain characteristics such as age, BMI etc. The group of interest here is disease vs. healthy and a possible grouping variable for random effect would be a match ID for each patient and its matched control.

Very small numbers of groups can make the estimation unstable and mixed models based on only a few groups should be interpreted with caution. There is no no golden consensus, but more than five group levels are usually needed for a stable estimate.

20.2.2 Design for fixed and random effects

Now that we have identified the grouping structure of the data, we need to decide what effects are fixed and what effects are random with respect to the grouping structure of the data. This is again might be confusing a bit but if we start asking ourselves a few questions, it might clarify the whole concept.

It is important to stress that the main reason we have chosen to go with mixed models is that we believe there is a grouping in our data that somehow affects some of the effects in our study. And by saying effect we mean the influence or impact that a particular variable has on the outcome of interest.

So here comes the first and a key question:

1. **Do we believe a particular effect varies across the grouping structure of the data?** if we do not believe that a particular effect varies across the grouping structure of the data, then we may consider modeling this effect as a fixed effect **without taking care of the grouping structure**.

If the answer to this question is yes. Then we ask ourselves another key question.

2. **Are we interested in estimating the effect specific to the groups?** Now that we have decided to take care of the grouping structure. We might be interested in estimating the effect of interest in each of groups or even compare them across the groups. You might want to even calculate p-values for these. If this is so then you might want to model these as fixed effect for example by including them as covariate (or blocking/control variable or even interaction) in the model. Doing so however, will force the model to estimate these effect thus spending degrees of freedom.

If however, we are still interested in taking care of the grouping but instead of estimating the effect in each group, we are happy just to know how

much variability is in the effect of interest across the groups, then we are going to consider modeling that effect as random. This will instruct the modeling application to instead of estimating effects for each group (could be hundred of groups), just come up with estimates of few parameters showing the variability. Please note that these effects can be anything from baseline, slopes, interaction etc.

3. **Do we have a large number of groups, and are some of these groups potentially unobserved?** Random effects models are particularly useful when dealing with a large number of groups, especially when some groups in the population might not be represented in the sample. Random effects assume that the observed groups are a random sample from a larger population of groups, allowing for generalization beyond the specific groups in the study.

However, as said before, instead of estimating parameters for each group, then calculate variance across the groups. If the number of levels in each group is too few (like two or something like that), the estimated variance is not going to be accurate. So you might want to actually get back and consider using classical models.

By asking these questions, we can better clarify the roles of fixed and random effects. These things are going to be further clarified when we start working on a few examples.

20.3 Examples (Mixed models in R)

We get back our simulated data in the previous chapter and try to model it using mixed models. Before proceeding make sure that you have installed `lme4` package as this is what we are going to use throughout this section.

Just to remind you, the idea was to examine the relationship between Age and Protein expression.

```
simulate_grouped_trend <- function(group_count = 5,
  ↪ points_per_group = 10, global_slope = -10, global_intercept =
  ↪ 30, group_slope = 2, noise_sd = 50, noise_sd2=2) {
  set.seed(123) # Setting a seed for reproducibility

  # Initialize an empty data frame to store the simulated data
  data <- data.frame(x = numeric(), y = numeric())

  # Loop to create each group
  for (i in 1:group_count) {
    x_start <- 12 + (i - 1) * (10 / group_count) # Stagger the
    ↪ start of x for each group
    x <- runif(points_per_group, min = x_start, max = x_start +
    ↪ (10 / group_count))

    # Apply a local positive trend within the group, but maintain
    ↪ the global negative trend
```



```

    local_intercept <- global_intercept + global_slope * (x_start
↪ + (10 / (2 * group_count))) + rnorm(1, mean = 0, sd =
↪ noise_sd)
    y <- local_intercept + group_slope[i] * (x - x_start) +
↪ rnorm(points_per_group, mean = 0, sd = noise_sd2)

    # Combine this group with the overall dataset
    group_data <- data.frame(x = x, y = y, group=i)
    data <- rbind(data, group_data)
  }

  return(data)
}

# generate simulated data
data_int <- simulate_grouped_trend(group_count =
↪ 4, points_per_group = 10, global_slope = -2, global_intercept =
↪ 100, group_slope = c(6, 4, 2, 1), noise_sd = 5, noise_sd2=2)

# set group to factor
data_int$group <- factor(data_int$group)
# plot the data
plot(data_int$x, data_int$y, xlab="Age", ylab="Protein
↪ expression", col=data_int$group, pch=as.numeric(data_int$group))

```

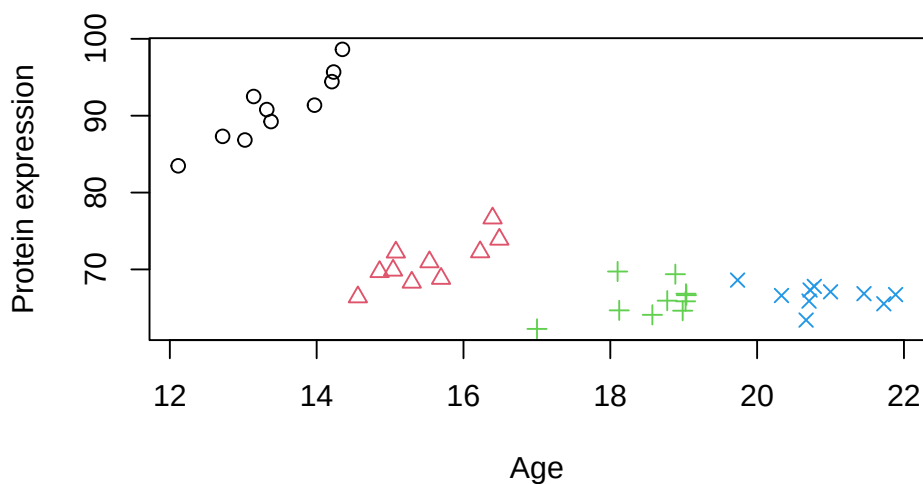


Figure 20.1: A scatter plot of Age vs. Protein expression

Because there are some grouping structure in data, we decide to take care of it. Let's say that each group is a random clinic that we decided to get some sample from. So we now have identified the grouping structure in our data that might influence the effect of interest (Age).

What we believe is that the effect of Age on the protein expression is constant in each of the groups. However, due to either technical issues with the instruments or even demographics, each group might have different baselines (starting point) for the protein expression. This gives us a hint that we might have to model **intercept** differently for each group. This decision gives us two choices, model this intercept as fixed or random effect.

Me as a researcher tell you in addition to the effect of age, I would like to know how much differences exactly are between each of these clinics. Therefore i want to use fixed effect to exactly pinpoint the estimated baseline for each of the clinic and compare them against each other.

You however, argue that there is no reason to do that. We just picked a few random clinics, there is no point in knowing by how much clinic 1 is different to clinic 2 because we could have selected and other random clinics in the world. So let's model this as random effect to get the variance of the intercepts across clinics. By modeling the intercept as a random effect, we acknowledge that each clinic has its own baseline level of protein expression due to various unmeasured factors such as technical differences in equipment or demographic variations. This approach allows us to account for the inherent variability between clinics without focusing on the specific differences between any two clinics. Instead, we aim to understand the general variability of baseline protein expression levels across clinics.

We start using the classical approach (as we saw in the previous chapter) using `lm` function with the following formula `y ~ 1+x+group`:

i Note

We have included 1 in the model just to stress that there is an intercept in the model. Obviously, `y ~ 1+x+group` is identical to `y ~ x+group`

```
# Fit a linear regression model
model_lm <- lm(y ~ 1+x+group, data = data_int)
summary(model_lm)
```

Call:

```
lm(formula = y ~ 1 + x + group, data = data_int)
```

Residuals:

	Min	1Q	Median	3Q	Max
	-3.9184	-1.7771	-0.1619	1.3793	5.2085

Coefficients:

	Estimate	Std. Error	t value	Pr(> t)
(Intercept)	54.3746	8.0727	6.736	8.41e-08 ***
x	2.7260	0.5977	4.561	5.99e-05 ***
group2	-25.7602	1.6487	-15.625	< 2e-16 ***
group3	-38.9476	3.2411	-12.017	5.62e-14 ***
group4	-44.7675	4.5867	-9.760	1.59e-11 ***

Signif. codes: 0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

Residual standard error: 2.434 on 35 degrees of freedom
 Multiple R-squared: 0.954, Adjusted R-squared: 0.9488
 F-statistic: 181.6 on 4 and 35 DF, p-value: < 2.2e-16

This model has four different intercepts, one for each group and a single slope that is identical for each group.

Given this result, we have estimated the group coefficients and we can go ahead with comparing the groups or do whatever we wanted to do with these estimates. It is important that here we don't have a global intercept. We have four intercepts, one for each group.

Now it is time to do the same analysis using the mixed model approach. The model that we are going to fit using `lmer` function from `lme4` is of form $y \sim 1+x+(1|\text{group})$. This part of the model, $y \sim 1+x$, we already know what it is. However, what this part $(1|\text{group})$ is new. The right part of $|$ shows what variable we want to use as grouping factor by which we are going to fit our random effect (That is the `group` in our case). The left part of $|$ is the effect we want to consider random. The left part is like a formula, similar to classical regression. which in this case, we just used 1. This means for our main model, $y \sim 1+x$ which has an intercept (1) and a slope x , we would like to consider the intercept as random effect so we use 1.

Now that we know the structure of the model, let's go ahead and do the modeling:

```
library(lme4)
# Fit a mixed linear regression model
model_lmm <- lmer(y ~ 1+x+(1|group), data = data_int)
summary(model_lmm)
```

```
Linear mixed model fit by REML ['lmerMod']
Formula: y ~ 1 + x + (1 | group)
Data: data_int
```

REML criterion at convergence: 201.4

Scaled residuals:

	Min	1Q	Median	3Q	Max
	-1.68459	-0.76282	-0.09731	0.60068	2.18875

Random effects:

Groups	Name	Variance	Std.Dev.
group	(Intercept)	378.606	19.458
Residual		5.931	2.435

Number of obs: 40, groups: group, 4

Fixed effects:

	Estimate	Std. Error	t value
(Intercept)	29.9105	14.0125	2.135

```
x          2.5561      0.5892   4.339
```

Correlation of Fixed Effects:

```
(Intr)
x -0.719
```

This output has some key differences to the previous approach. The first thing is in the fixed effect part:

```
as.list(summary(model_lmm))[['coefficients']]
```

```
              Estimate Std. Error  t value
(Intercept) 29.910511  14.012502  2.134559
x            2.556149   0.589159  4.338640
```

Unlike `lm`, here there is no estimate for each group. There is a single intercept and a single slope. These are **global** trends in the data with their corresponding **Std. Error** and **t value**. But what happen with random effect and grouping of the data. This is what we can see in the **Random effects** part of the output:

```
as.list(summary(model_lmm))[['varcor']]
```

```
Groups   Name          Std.Dev.
group    (Intercept) 19.4578
Residual                2.4355
```

Depending of what type of random effect we have been fitting this table can have different headers. But generally, **Groups**, **Name**, **Variance**, **Std.Dev.** and **Corr** are part of it.

Each row of this table correspond to a source of variation in our data that is random because of a certain reason. In our model, we decided to consider only **Intercept** as a random effect for **group**. Therefore, the first row of the table tells us that there is **random** variability in **Intercept** between different group. This variability is quantified by a variance of 378.606 and a standard deviation of 19.458. This means that the average effect (or intercept) across groups can deviate from the overall (**global**) intercept by about 19.458 units, indicating substantial between-group variability. We did not fit separate fixed intercept parameters for each group, but we we have estimated the variability of intercepts across different groups, which allows us to account for the fact that each group may have a different starting point or baseline level in the response variable, even though these specific baseline levels are not directly calculated. You can think about this as the variance of the distribution of intercepts across the groups. Essentially, we're modeling the distribution from which each group's intercept is drawn.

The second row, labeled **Residual**, refers to the variability within each group that is not explained by the model predictors (because of random sampling). The variance here is 5.931, with a standard deviation of 2.435. This residual variance represents the individual differences or noise within groups after accounting for the modeled effects, including the random intercepts.

So before moving on let's summarize what we got so far.

We used mixed models with a **random intercept** to model the effect of age on protein expression. We got an estimated global intercept (baseline) and slope (main effect of interest). We took care of the grouping structure of our data without directly estimating any additional parameters. In fact, what we estimated was **intercept**, **slope**, **variance of Intercept** and **variance of residuals**. One of the coolest thing here is that we would have estimated the exact same number of parameters if our grouping structure had a lot more levels. But if we want to use grouping as covariate (in a classic model), we should have modeled all of those levels!

20.3.1 Predicting the random intercept

Despite that we have not estimated the intercept for each of the group, we can still predict what would the intercept be for each of the groups. We are going to see in the math section how that can be done, but for now, we skip the details and directly use the function **ranef** from **lme4** package.

```
print(ranef(model_lmm))
```

```
$group
  (Intercept)
1    26.705636
2     1.337016
3   -11.314930
4   -16.727722
```

with conditional variances for "group"

In this particular case, it gave us the differences from the overall intercept estimated in the mixed effects model. By extracting the random effects using the **ranef** function, we obtain the specific adjustments needed for each group. We can now use the global intercept and these adjustments to visualize the predicted intercept.

```
# Plotting the initial scatter plot with points colored by group
↪ and shape determined by group
plot(data_int$x, data_int$y, xlab="Age", ylab="Protein
↪ expression",
      col=data_int$group, pch=as.numeric(data_int$group))

# Looping through each unique group in the dataset
for (group in unique(data_int$group)) {
  # Subset data for the current group
  group_data <- data_int[data_int$group == group,]

  # Calculate the intercept for the group by adding the fixed
  ↪ intercept to the group's random effect
  intercept <- fixef(model_lmm)[1] + ranef(model_lmm)[[1]][group,
  ↪ 1]

  # Extract the fixed effect slope from the model
```

```

slope <- fixef(model_lmm)[2]
# Draw a line for the group using the calculated intercept and
  ↪ slope
lines(y = intercept + slope * group_data$x, x = group_data$x,
  ↪ col = "red", lty = 1)

}

# Draw a global effect line using the fixed effect intercept
  ↪ and slope from the model
abline(a = fixef(model_lmm)[1], b = fixef(model_lmm)[2], col =
  ↪ "blue")

# Adding a legend to the plot
legend_labels <- c("Global Effect", "Random Intercept")
legend("topright", legend = legend_labels, col = c("blue",
  ↪ "red"), lty = 1)

```

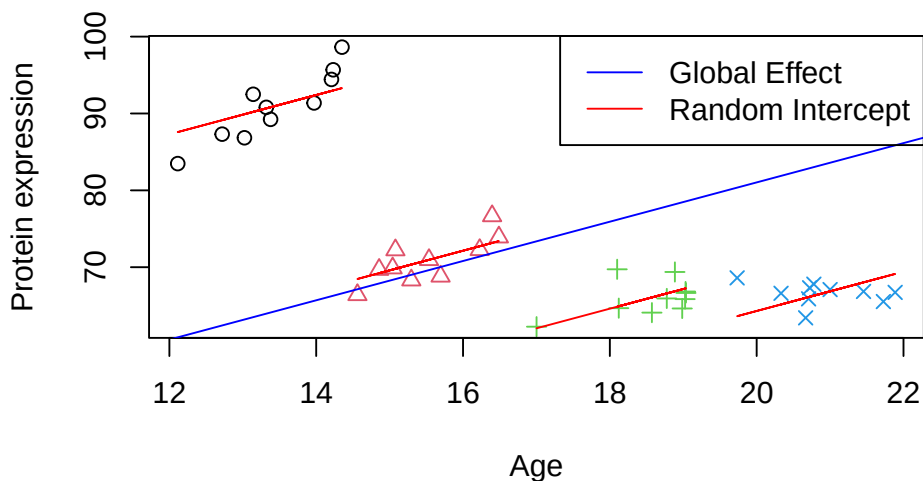


Figure 20.2: Scatter plot of Age vs. Protein expression per group with random intercept

Here we have plotted predicted group-specific lines and also the global trend. We can see that all the lines are parallel (as we still did not model slope per group) and there is a deviation of intercept in each of the groups compared to the global trend.

Looking at Figure 20.2, we can see that a lot of variation is because of the differences between the groups. This is exactly what the variance component of the mixed model showed us. The ratio of random intercept and the total variance quantifies how much of the total variability in the outcome is due to variability between groups: $\frac{378.606}{5.931+378.606} = 98.46\%$. So most of the variation in the data is because of the differences between the groups.

20.3.2 Random slope

So far we have the random intercept model, however, one can argue that the effect of age on protein expression might not be uniform across all clinics. This variability suggests that we should also consider modeling the slope as a random effect, allowing the relationship between age and protein expression to vary among the groups. This leads us to a more complex but potentially more accurate model: $y \sim 1+x+(1+x|group)$. In this model, not only the intercepts are allowed to vary randomly across clinics (1 part of the model), but also the slopes (x). This basically says that each clinic might not only have a different starting point of protein expression but also a different rate at which protein expression changes with age.

Implementing a random slope model accommodates the hypothesis that the effect of age could be influenced by factors specific to each clinic, such as demographic characteristics, environmental factors, or other clinic-specific variables not captured in the dataset.

To fit this model, we modify our `lmer` function call as follows:

```
# Fit a mixed linear regression model with random slopes
model_lmm_rs <- lmer(y ~ 1+x+(1+x|group), data = data_int)
summary(model_lmm_rs)
```

Linear mixed model fit by REML ['lmerMod']

Formula: $y \sim 1 + x + (1 + x | group)$

Data: data_int

REML criterion at convergence: 186.8

Scaled residuals:

	Min	1Q	Median	3Q	Max
	-1.56057	-0.74656	-0.02922	0.26277	2.22754

Random effects:

Groups	Name	Variance	Std.Dev.	Corr
group	(Intercept)	344.655	18.565	
	x	5.141	2.267	-0.93
Residual		3.672	1.916	

Number of obs: 40, groups: group, 4

Fixed effects:

	Estimate	Std. Error	t value
(Intercept)	35.321	11.977	2.949
x	2.538	1.220	2.080

Correlation of Fixed Effects:

(Intr)
x -0.904

In the output, we see not just the variance and standard deviation for the intercepts across groups, but also for the slopes. This allows us to quantify how

much variation there is in the relationship between age and protein expression across the different clinics.

Looking at fixed effect, we still have the global intercept and slope. There is a small change compared to the previous model but the major change now is in the random effect part.

The **Random effects** section of the model summary now includes information about the variability in slopes in addition to the intercepts.

```
print(VarCorr(model_lmm_rs))
```

Groups	Name	Std.Dev.	Corr
group	(Intercept)	18.5649	
	x	2.2675	-0.933
Residual		1.9164	

We still have the major source of variation in our data caused by **Intercept** and a smaller, but still present, variability in how age affects protein expression across these clinics (the slope **x**).

A high correlation between the intercept and slope within the random effects suggests a linear relationship in the random part of the model. This indicates that clinics with higher baseline protein expression also have a stronger positive relationship between age and protein expression.

```
# Plotting the initial scatter plot with points colored by group
↪ and shape determined by group
plot(data_int$x, data_int$y, xlab="Age", ylab="Protein
↪ expression",
     col=data_int$group, pch=as.numeric(data_int$group))

# Looping through each unique group in the dataset
for (group in unique(data_int$group)) {
  # Subset data for the current group
  group_data <- data_int[data_int$group == group,]

  # Calculate the intercept for the group by adding the fixed
  ↪ intercept to the group's random effect
  intercept <- fixef(model_lmm_rs)[1] +
  ↪ ranef(model_lmm_rs)[[1]][group, 1]

  # Extract the fixed effect slope from the model
  slope <- fixef(model_lmm_rs)[2]+ranef(model_lmm_rs)[[1]][group,
  ↪ 2]

  # Draw a line for the group using the calculated intercept and
  ↪ slope
  lines(y = intercept + slope * group_data$x, x = group_data$x,
  ↪ col = "green", lty = 1)

  intercept <- fixef(model_lmm)[1] + ranef(model_lmm)[[1]][group,
  ↪ 1]
```



```

# Extract the fixed effect slope from the model
slope <- fixef(model_lmm)[2]

lines(y = intercept + slope * group_data$x, x = group_data$x,
      col = "red", lty = 1)

}

# Draw a global effect line using the fixed effect intercept
# and slope from the model
abline(a = fixef(model_lmm_rs)[1], b = fixef(model_lmm_rs)[2],
      col = "blue")

# Adding a legend to the plot
legend_labels <- c("Global Effect", "Random Intercept", "Random
  Intercept and slope")
legend("topright", legend = legend_labels, col = c("blue",
  "red", "green"), lty = 1)

```

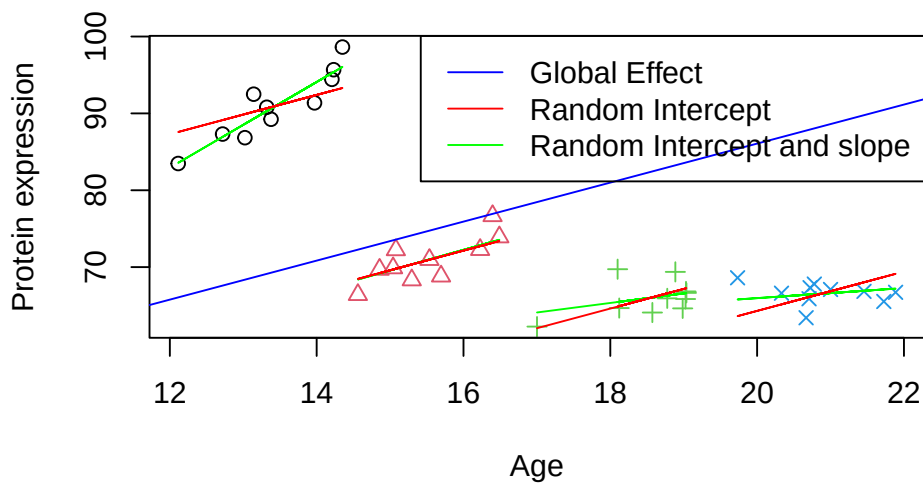


Figure 20.3: Scatter plot of Age vs. Protein expression per group with random intercept and slope

The plot above illustrates the variability in protein expression as a function of age across different clinics. The blue line indicates the overall trend across all clinics (Global Effect), while the red and green lines represent individual clinic trends. The red lines show the clinic-specific adjustments to the overall intercept (Random Intercept), indicating that some clinics, on average, have higher or lower protein expression regardless of age. The green lines show both clinic-specific intercepts and slopes (Random Intercept and Slope), suggesting that the rate at which protein expression changes with age also varies by clinic. This visual representation underscores the importance of accounting for random

effects in the model, as it captures the ways in which age influences protein expression across diverse clinical settings.

In this specific case, the global estimated intercept and slope did not change that much compared to the random intercept model. However, in the cases that there is a very large deviation between slopes of different groups, the global coefficient can be much more impacted.

20.3.3 Nested and crossed designs

So far we have seen pretty simple cases of two level models. However, mixed models can be used to handle pretty complex experimental designs. Here i will try to give you a few examples of what you can do with these models and how to interpret them.

! Important

We are going to repeat this later, but the type of design is not something that should be inferred based on the data. Despite that this is possible and might seem reasonable to do so, especially nested and crossed designs must be supported by the experimental design.

20.3.3.1 Nested designs

Nested designs occur when one level of grouping is entirely contained within another level, without any overlap between groups at the same level. In the context of mixed models, this translates to data where, for instance, patients are grouped within clinics, and clinics are grouped within regions. Each patient belongs to one and only one clinic, and each clinic belongs to one and only one region. The key characteristic of nested designs is that the groups at one level do not intersect or overlap with groups at another level.

```
graph TD
  A[Region 1]
  A --> B[Clinic 1]
  A --> C[Clinic 2]
  A --> D[Clinic 3]

  B --> M1[Patient 1]
  B --> N2[Patient 2]
  B --> O3[Patient 3]

  C --> P4[Patient 4]
  C --> Q5[Patient 5]
  C --> R6[Patient 6]

  D --> S7[Patient 7]
  D --> T8[Patient 8]
  D --> U9[Patient 9]
```

```

E[Region 2]
E --> F[Clinic 4]
E --> G[Clinic 5]
E --> H[Clinic 6]

F --> V10[Patient 10]
F --> W11[Patient 11]
F --> X12[Patient 12]

G --> Y13[Patient 13]
G --> Z14[Patient 14]
G --> A15[Patient 15]

H --> B16[Patient 16]
H --> C17[Patient 17]
H --> D18[Patient 18]

I[Region 3]
I --> J[Clinic 7]
I --> K[Clinic 8]
I --> L[Clinic 9]

J --> E19[Patient 19]
J --> F20[Patient 20]
J --> G21[Patient 21]

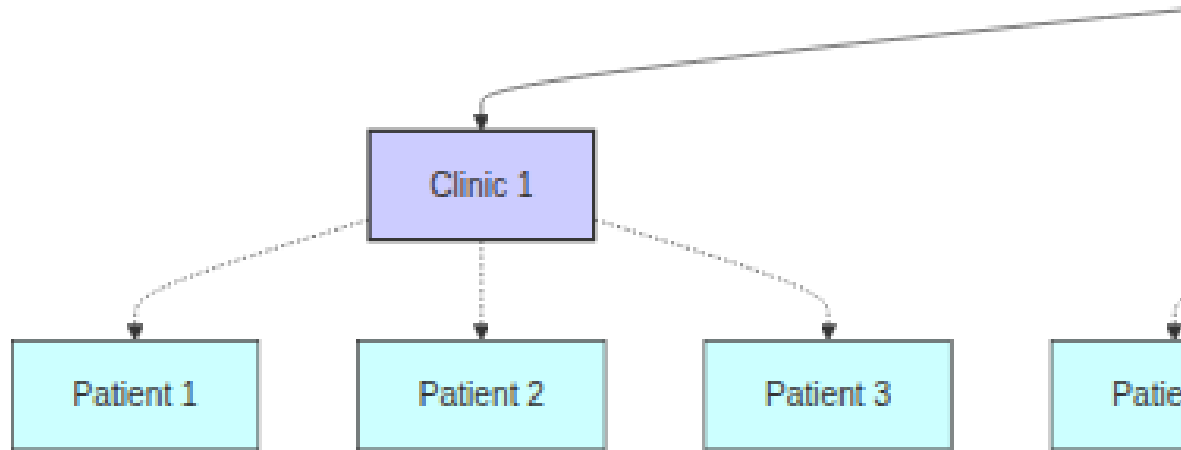
K --> H22[Patient 22]
K --> I23[Patient 23]
K --> J24[Patient 24]

L --> K25[Patient 25]
L --> L26[Patient 26]
L --> M27[Patient 27]

classDef region fill:#f9f,stroke:#333,stroke-width:2px;
classDef clinic fill:#ccf,stroke:#333,stroke-width:2px;
classDef patient fill:#cff,stroke:#333,stroke-width:1px;

class A,E,I region;
class B,C,D,F,G,H,J,K,L clinic;
class
↪ M1,N2,O3,P4,Q5,R6,S7,T8,U9,V10,W11,X12,Y13,Z14,A15,B16,C17,D18,E19,F20,G21,H22,I23,J24,K25,
↪ patient;

```



20.3.3.2 Interpreting Nested Designs

In nested designs, the analysis aims to account for the variability at each level of the hierarchy. For example, when assessing the effect of age on protein expression, a nested model allows for the separation of the variability attributable to differences between regions, differences between clinics within regions, and individual differences between patients within clinic. This separation is crucial for accurately estimating the effects of interest (e.g., age) while controlling for the hierarchical structure of the data.

Nested models are particularly powerful for:

- **Identifying Variability:** They help in identifying where the most significant sources of variability lie within the hierarchical structure. Is it between regions, between clinics within regions, or within regions?
- **Improving Estimates:** By correctly modeling the nested structure, we obtain more accurate estimates of group-level effects and individual effects. This is because the model accounts for the correlation within groups, leading to better estimates of the standard errors for the fixed effects.
- **Customizing Interventions:** In life science research, for example, understanding the level at which interventions are most effective (regions vs. clinics) can inform targeted strategies for improvement.

20.3.3.3 Example

In R (`lmer`), we can use `(1 | region/clinic)` for samples nested within clinic, which are in turn nested within regions. The notation `region/clinic` is shorthand for `region + region:clinic`.

Let's see one example:

```
simulate_grouped_trend_nested <- function(region_count = 3,
  ↪ clinics_per_region = 5,
```

```

measurements_per_clinic = 10,
global_slope = -10, global_intercept = 30,
region_effects = c(), region_slopes = c(),
clinic_effects = list(), clinic_slopes = list()) {
  set.seed(123) # Setting a seed for reproducibility

  # Initialize an empty data frame to store the simulated data
  data <- data.frame(x = numeric(), y = numeric(), clinic =
↪ integer(), region = integer())

  clinic_id_counter = 1 # Initialize a counter for clinic IDs
↪ across regions

  # Loop to create data for each region
  for (region in 1:region_count) {
    # Use the specific region effect and slope
    region_effect_adj = region_effects[region]
    region_slope_adj = region_slopes[region]

    # Loop to create data for each clinic within a region
    for (clinic in 1:clinics_per_region) {
      x_start = runif(1, min = 0, max = 10) + 10 * (clinic - 1) #
↪ More continuous x values across clinics
      x = runif(measurements_per_clinic, min = x_start, max =
↪ x_start + 10) # Continuous x for measurements

      # Use the specific clinic effect and slope
      clinic_effect = clinic_effects[[region]][clinic]
      clinic_slope = clinic_slopes[[region]][clinic]

      # Simulate measurements for each clinic
      for (i in 1:measurements_per_clinic) {
        # Model y incorporating both global and specific slopes
        ↪ and effects
        y = (global_intercept + region_effect_adj +
↪ clinic_effect) +
        (global_slope + region_slope_adj + clinic_slope) *
↪ x[i] +
        rnorm(1, mean = 0, sd = 1) # Assuming
        ↪ measurement_noise_sd is constant for simplicity

        # Combine this measurement with the overall dataset
        data = rbind(data, data.frame(x = x[i], y = y, clinic =
↪ clinic_id_counter, region = region))
      }

      clinic_id_counter <- clinic_id_counter + 1 # Increment
↪ clinic ID for unique identification across regions
    }
  }

```

```

}

return(data)
}

set.seed(10)
data_int_nested<-simulate_grouped_trend_nested(region_count = 4,
  clinics_per_region = 3,
  measurements_per_clinic = 20,
  global_slope = 0,global_intercept = 100,
  region_effects = rnorm(4,mean = 0,sd = 20),
  region_slopes=rnorm(4,mean = 0.1,sd = 2),
  clinic_effects = lapply(1:4,function(i){rnorm(3,mean = 0,sd =
    ↪ 10)}),
  clinic_slopes = lapply(1:4,function(i){rnorm(3,mean = 0.1,sd =
    ↪ 1)}))

plot(data_int_nested$x,data_int_nested$y,col=data_int_nested$region,pch=data_int_nest
  ↪ = "Age",ylab = "Protein expression")
legend("top", legend = paste("Region",
  ↪ unique(data_int_nested$region)),
  fill = unique(data_int_nested$region), cex = 0.8, title =
  ↪ "Regions",horiz=T)

legend("topleft", legend = paste("Clinic",
  ↪ unique(data_int_nested$clinic)),
  pch = unique(data_int_nested$clinic), col = "black", cex =
  ↪ 0.8, title = "Clinics", inset = c(0.05, 0))

```

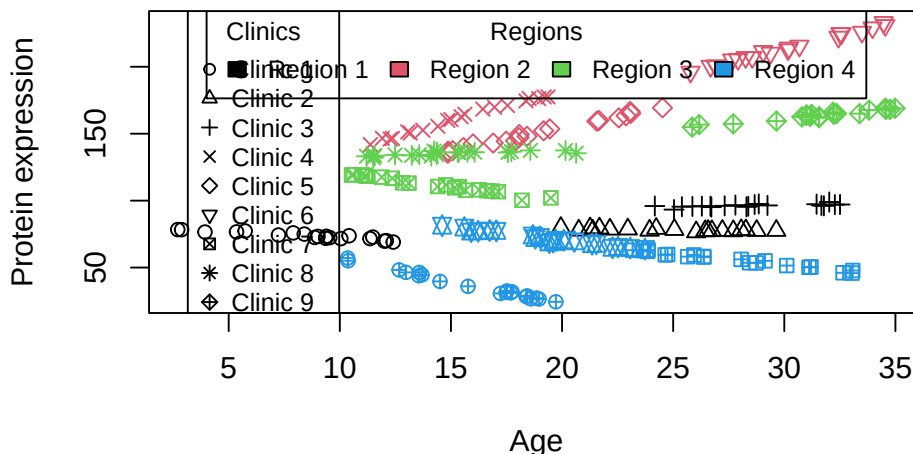


Figure 20.4: Scatter plot of Age vs. Protein expression for nested design

In this example, we simulated a number of samples from a few clinics from different regions. From the beginning the experimental design was nested in a

way that, a single sample was from a single clinic and a single clinic was from one and only one region. Looking at Figure 20.4, we can already see that there is a very big baseline difference between regions. Within each region, there is also some baseline differences between clinics but it is less dominant than the region differences. Furthermore, the effect of Age on Protein expression seems to be different slightly across different regions (some have positive effect, some negative and some close to zero) and to a lesser extend within each region but across clinics. Given this informal interpretation of the data we go forward and do the modeling considering both the intercept and slope as random effects.

```
model_lmm_nested <- lmer(y~x+(x|region/clinic),data_int_nested)

print(model_lmm_nested)
```

```
Linear mixed model fit by REML ['lmerMod']
Formula: y ~ x + (x | region/clinic)
Data: data_int_nested
REML criterion at convergence: 859.4597
Random effects:
Groups          Name          Std.Dev. Corr
clinic:region   (Intercept)  8.124
                x            1.104   -0.31
region          (Intercept) 19.610
                x            2.572   -0.25
Residual                        1.042
Number of obs: 240, groups:  clinic:region, 12; region, 4
Fixed Effects:
(Intercept)          x
    100.915         0.337
```

This result is slightly different to what we have seen before using only a single grouping factor. Now we have **clinic within region (clinic:region)** in which the model estimates random intercepts and slopes for clinics nested within regions. This reflects the variability in baseline Protein expression and the effect of Age on Protein expression at the clinic level, indicating that clinics within the same region may still exhibit unique characteristics. The model also estimates random intercepts and slopes at the **region** level, capturing broader variations across regions. This indicates how different regions can have distinct baseline Protein expressions and different relationships between Age and Protein expression.

The model identifies substantial variability in baseline Protein expression levels across regions, as indicated by the standard deviation of the intercepts for regions. This variability signifies that different regions start off with different baseline Protein expressions. There's additional variability at the clinic level within each region, though this effect is smaller compared to the variability between regions.

The slopes, representing the effect of Age on Protein expression, also vary by region and by clinic within regions. This variation in slopes indicates that the relationship between Age and Protein expression is not uniform; it changes from one region to another and even among clinics within the same region. Some re-

gions or clinics might show a positive relationship (increasing Age associated with higher Protein expression), while others show a negative or negligible relationship.

Similar to what we have seen before, the model provides a correlation coefficient between the intercept and slope within clinics nested in regions. A negative correlation suggests that clinics with higher baseline Protein expression tend to have a less steep (or more negative) Age effect, or vice versa.

Finally, the fixed effect intercept represents the average baseline level of Protein expression when Age is zero, pooled across all regions and clinics. This gives a general starting point for Protein expression across the dataset. And the fixed effect slope indicates the average effect of Age on Protein expression, again pooled across the entire dataset. This effect sizes up the general tendency of how Protein expression changes with Age, irrespective of the region or clinic.

20.3.3.4 Crossed designs

The last experimental design I want to mention is the full crossed design, which contrasts with the nested structure we've explored so far. In a crossed design, each level of one grouping factor can be combined with each level of another grouping factor, creating a matrix-like structure of group combinations. This design is particularly relevant in experiments where the interaction effects between two or more independent grouping factors are of interest, and these factors are not hierarchically related but rather orthogonal or independent of each other.

graph TD

```

A[Clinic 1]
E[Clinic 2]
I[Clinic 3]

B[Doctor 1]
C[Doctor 2]
D[Doctor 3]

M1[Sample 1]
M2[Sample 2]
M3[Sample 3]
M4[Sample 4]
M5[Sample 5]

```

```

A --> B
A --> C
A --> D
E --> B
E --> C
E --> D
I --> B
I --> C

```



```

I -. -> D

%% Patients connected to clinics to show they can come from
↪ any clinic, implying crossed with regions
B --> M1
B --> M2
B --> M3
B --> M4
B --> M5

C --> M1
C --> M2
C --> M3
C --> M4
C --> M5

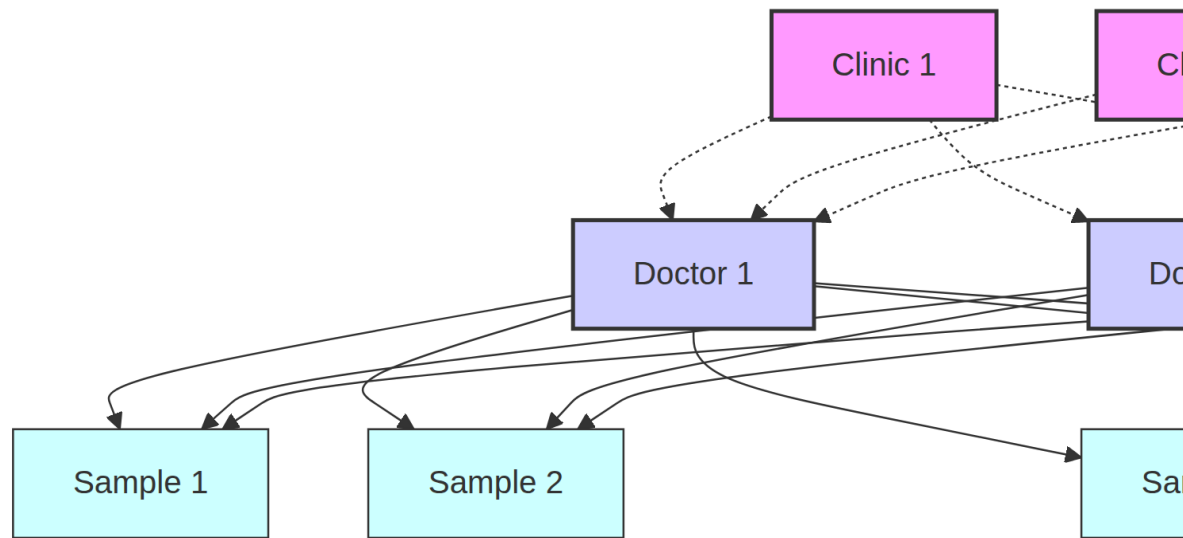
D --> M1
D --> M2
D --> M3
D --> M4
D --> M5

%% Additional patient connections omitted for brevity

classDef region fill:#f9f,stroke:#333,stroke-width:2px;
classDef clinic fill:#ccf,stroke:#333,stroke-width:2px;
classDef patient fill:#cff,stroke:#333,stroke-width:1px;

class A,E,I region;
class B,C,D,F,G,H,J,K,L clinic;
class M1,M2,M3,M4,M5 patient;

```



One hypothetical study (probably very rare!) is a study designed to explore the effects of age on protein expression within a single patient, our research employs a unique crossed design where the patient visits multiple doctors across various clinics at different ages. Recognizing the potential for variability introduced by the differing methodologies and environments inherent to each clinic, as well as the distinct clinical approaches of each doctor. Specifically, we want to analyze the data, treating age as a continuous fixed effect to investigate its influence on protein expression. Concurrently, we model doctors and clinics as random effects to account for the variability they introduce into the protein expression measurements. This allows us not only to assess the general trend of protein expression changes over time but also to explore how these changes might be modulated differently across the various clinical settings and by different medical professionals.

```
simulate_grouped_trend_crossed <- function(clinic_count = 3,
  ↪  doctor_count = 5, measurements_per_combination = 10,
  ↪  global_slope = -10, global_intercept = 30,
  ↪  clinic_effects = c(), clinic_slopes = c(),
  ↪  doctor_effects = c(), doctor_slopes = c()) {
  set.seed(123) # Setting a seed for reproducibility

  # Initialize an empty data frame to store the simulated data
  data <- data.frame(x = numeric(), y = numeric(), doctor =
  ↪  integer(), clinic = integer())

  # Validate or adjust lengths of effects and slopes arrays to
  ↪  match counts
  if (length(clinic_effects) != clinic_count) clinic_effects <-
  ↪  rep(clinic_effects[1], clinic_count)
  if (length(clinic_slopes) != clinic_count) clinic_slopes <-
  ↪  rep(clinic_slopes[1], clinic_count)
  if (length(doctor_effects) != doctor_count) doctor_effects <-
  ↪  rep(doctor_effects[1], doctor_count)
```

```

if (length(doctor_slopes) != doctor_count) doctor_slopes <-
  ↪ rep(doctor_slopes[1], doctor_count)

# Loop to create data for each combination of clinic and doctor
for (clinic in 1:clinic_count) {
  for (doctor in 1:doctor_count) {
    x_start = runif(1, min = 0, max = 10) # Continuous x values
    x = runif(measurements_per_combination, min = x_start, max
  ↪ = x_start + 10) # Continuous x for measurements

    # Use the specific effect and slope for the current clinic
    ↪ and doctor
    clinic_effect_adj = clinic_effects[clinic]
    clinic_slope_adj = clinic_slopes[clinic]
    doctor_effect = doctor_effects[doctor]
    doctor_slope = doctor_slopes[doctor]

    # Simulate measurements for each combination of clinic and
    ↪ doctor
    for (i in 1:measurements_per_combination) {
      # Model y incorporating both global and specific slopes
      ↪ and effects
      y = (global_intercept + clinic_effect_adj +
  ↪ doctor_effect) +
      ↪ (global_slope + clinic_slope_adj + doctor_slope) *
  ↪ x[i] +
      ↪ rnorm(1, mean = 0, sd = 1) # Assuming
      ↪ measurement_noise_sd is constant for simplicity

      # Combine this measurement with the overall dataset
      data = rbind(data, data.frame(x = x[i], y = y, doctor =
  ↪ doctor, clinic = clinic))
    }
  }
}

return(data)
}

data_int_crossed<-simulate_grouped_trend_crossed(clinic_count =
  ↪ 4, doctor_count = 3,
  measurements_per_combination = 20,
  global_slope = 1, global_intercept = 100,
  clinic_effects = c(1, 20, 40, 60),
  clinic_slopes = c(1, 1.5, 0.9, 1.1),
  doctor_effects = c(1.3, 1.2, 2),
  doctor_slopes = c(-3, 1, 2))

  ↪ plot(data_int_crossed$x, data_int_crossed$y, col = data_int_crossed$clinic, pch = data_int_cross
  ↪ = "Age", ylab = "Protein expression")

```

```

legend("top", legend = paste("Clinic",
  ↪ unique(data_int_crossed$clinic)),
      fill = unique(data_int_crossed$clinic), cex = 0.8, title =
  ↪ "Clinics",horiz=T)

legend("topleft", legend = paste("Doctor",
  ↪ unique(data_int_crossed$doctor)),
      pch = unique(data_int_crossed$doctor), col = "black", cex
  ↪ = 0.8, title = "Doctors", inset = c(0.05, 0))

```

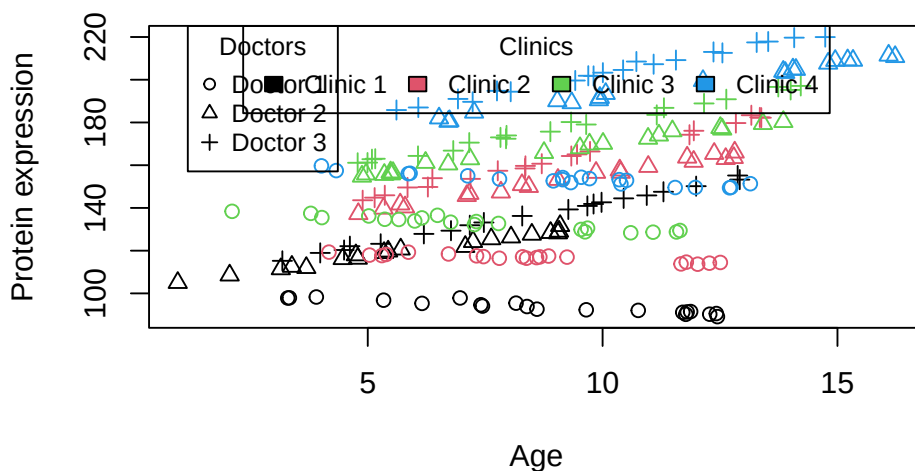


Figure 20.5: Scatter plot of Age vs. Protein expression for crossed design

Ok! Now it is a bit more complicated than before. What Figure 20.5 tells us is that there appears to be quite a big differences in baseline between different clinics (colors). The effect of age on protein expression seems to be consistently go up in each clinic. However, looking at the doctors (shapes), we see that all three of them are quite consistent when it comes to baseline but one of them has a big slope differences compare to the other two.

Basically, the results of modeling tells us the same thing:

```

model_lmm_crossed <-
  ↪ lmer(y~x+(x|clinic)+(x|doctor),data_int_crossed)
print(model_lmm_crossed)

```

```

Linear mixed model fit by REML ['lmerMod']
Formula: y ~ x + (x | clinic) + (x | doctor)
Data: data_int_crossed
REML criterion at convergence: 732.3083
Random effects:
Groups   Name             Std.Dev. Corr
clinic   (Intercept) 25.4828

```

```

              x          0.2434 -0.17
doctor (Intercept) 0.4461
              x          2.6332 0.82
Residual              0.9559
Number of obs: 240, groups: clinic, 4; doctor, 3
Fixed Effects:
(Intercept)              x
      131.832          2.108

```

The interpretation is the same as before, we have three kind of variations in both slopes and intercept. Between doctors, between clinics and of course the residuals. The largest variance is between the clinics in their baseline followed by the slope of doctors.

That of more or less what the crossed design looks like. It is important to note that, you **should not** model the crossed designs using the nested formula. Using a nested random effects notation like `(x | clinic:doctor)` implies a nested structure where each “doctor” is considered to be unique within each “clinic” and does not account for the possibility of the same doctor working across multiple clinics. This notation inherently assumes that each doctor-clinic combination is distinct and treats “doctor” identities as separate within each clinic, without recognizing that a single doctor could contribute data across different clinics. Modeling crossed factors using nested notation can inadvertently obscure the independence of those factors and the data’s true structure. As a result, it introduces structural biases in the analysis, potentially leading to incorrect conclusions about the effects of interest and the variability within and across the groups.

So to summarize, the crossed design does not assume any particular relationship between region and clinic. They are just two separate sources of random variability. This could be appropriate in a design where doctors from the same clinic could be more similar to each other, but a doctor could theoretically belong to any clinic.

The nested design assumes that doctors are nested within clinics. This introduces additional structure to the random effects, where the variability at the doctor level is specific to each clinic. This model is used in designs, where the data from doctors are grouped by clinics, and you expect that doctors within the same clinic share some commonality that is different from doctors in other clinics.

i Note

One way to make sure that you are using the correct formulation of the random effect is to ask yourself whether this design is nested or not. If the design is not nested you should use the crossed formulation.

For example for smaller designs we can check using `xtabs`

```

print(xtabs(~region+clinic,data_int_nested))

      clinic
region 1  2  3  4  5  6  7  8  9 10 11 12

```

```

1 20 20 20 0 0 0 0 0 0 0 0 0
2 0 0 0 20 20 20 0 0 0 0 0 0
3 0 0 0 0 0 0 0 20 20 20 0 0
4 0 0 0 0 0 0 0 0 0 0 20 20 20

```

and compare to a crossed design

```
print(xtabs(~clinic+doctor,data_int_crossed))
```

```

      doctor
clinic 1  2  3
1 20 20 20
2 20 20 20
3 20 20 20
4 20 20 20

```

We can see that in the first example, each lower level grouping factor (in this case `clinic`) is only associated to one and only one **region**. But a **region** can accommodate multiple clinics. This is nested. However, in the second example, we have each **doctor** (lower level) associated with multiple **clinics**. This violates the assumptions of nesting and therefore is crossed.

There exist much more complex designs, but we are not going to cover them here.

20.3.4 Understand what you are doing!

Statistical models can become very complex especially when many variables of different types (countinous, categorical etc) are of interest or one is to look at interactions. What we want to do here in this section is to explore a few cases that involve creating models of more than one variable.

Consider a simple scenario where some patients have been given three treatments by different doctors.

```

simulatePatientData_treatment_doctor <- function(numDoctors = 30,
  ↪ numPatients = 10, seed = 12345) {
  set.seed(seed)

  # Parameters
  intercept = 50 # Baseline recovery time
  treatmentEffect = c(A = 0, B = -50, C = -20) # Treatment
  ↪ effect on recovery time
  # Simulate data
  doctors <- rep(1:numDoctors, each = numPatients)
  treatment <- sample(c("A", "B", "C"), size = numDoctors *
  ↪ numPatients, replace = TRUE)

  # Random effects for doctors
  doctorIntercept <- rnorm(numDoctors, 0, 2) # Variation in
  ↪ doctor baseline recovery times
  doctorTreatmentBEffct <- rnorm(numDoctors, 0, 2) # Variation
  ↪ in treatment B effect by doctor

```

```

doctorTreatmentCEffect <- rnorm(numDoctors, 0, 2) # Variation
↪ in treatment C effect by doctor

# Calculate recovery time
doctor_var<-list(c(1,1.4,0),c(1.2,1,0),c(0.9,1.3,20))
tr<-treatmentEffect[treatment]
  for(i in unique(doctors))
  {
    tr[treatment=="A" & doctors==i]<-tr[treatment=="A"
↪ &doctors==i]+doctor_var[[i]][1]
    tr[treatment=="B" & doctors==i]<-tr[treatment=="B"
↪ &doctors==i]+doctor_var[[i]][2]
    tr[treatment=="C" & doctors==i]<-tr[treatment=="C"
↪ &doctors==i]+doctor_var[[i]][3]

  }

recoveryTime <- intercept+tr +
  #(treatment == "B") *
  ↪ doctorTreatmentBEffect[doctors] +
  #(treatment == "C") *
  ↪ doctorTreatmentCEffect[doctors] +
  doctorIntercept[doctors] +
  rnorm(numDoctors * numPatients, 0, 2) # Add
  ↪ some observation noise

data <- data.frame(doctor = doctors, treatment = treatment,
↪ recoveryTime = recoveryTime)
return(data)
}

data_int_treatment <-
  ↪ simulatePatientData_treatment_doctor(numDoctors =
  ↪ 3,numPatients = 200)
data_int_treatment$treatment <-
  ↪ factor(data_int_treatment$treatment)
plot(data_int_treatment$recoveryTime ~
  ↪ jitter(as.numeric(data_int_treatment$treatment), 1),
  ↪ col=as.factor(data_int_treatment$treatment),pch=as.numeric(as.factor(data_int_treatment$doc
  ↪ = "Recovery Time"))
axis(1, at=c(1,2,3), labels=c("A","B","C"))
legend("top", legend = paste("Doctor",
  ↪ unique(data_int_treatment$doctor)),
  pch = unique(data_int_treatment$doctor), cex = 0.8, title
  ↪ = "Doctor",horiz=T)

```

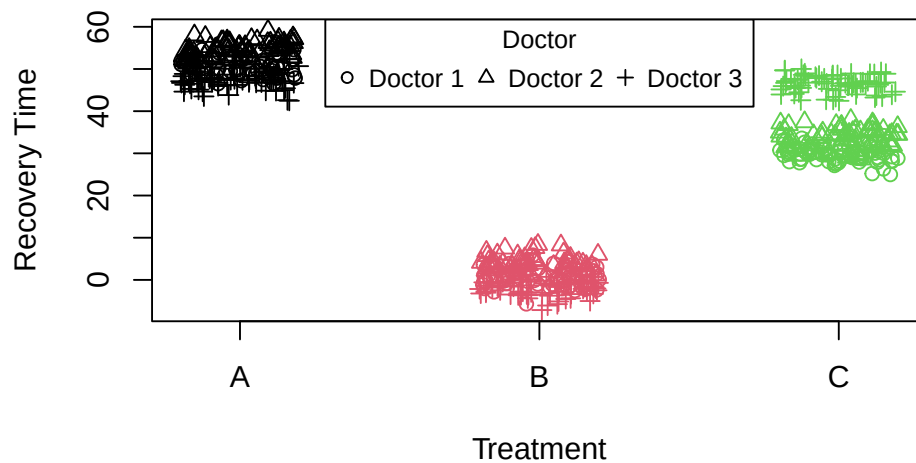


Figure 20.6: Scatter plot of treatment vs recovery time

In this example, we have three treatments each given by three different doctors. What we are interested in is the treatment effect. However, what we believe is that patients going to different doctors might experience slightly different outcomes not just because of the treatment itself but also due to the specific practices or approaches of each doctor. Therefore, to accurately assess the treatment effect, we need to account for the variability introduced by the doctors. We decide to use random intercept model `lmer(recoveryTime~treatment+(1|doctor),data_int_treatment)`.

```
model_lmm_treatment <- lmer(recoveryTime ~ treatment +
  ↪ (1|doctor), data_int_treatment)
print(model_lmm_treatment)
```

```
Linear mixed model fit by REML ['lmerMod']
Formula: recoveryTime ~ treatment + (1 | doctor)
Data: data_int_treatment
REML criterion at convergence: 3586.159
Random effects:
 Groups   Name      Std.Dev.
 doctor  (Intercept) 2.298
 Residual                    4.782
Number of obs: 600, groups:  doctor, 3
Fixed Effects:
(Intercept)  treatmentB  treatmentC
      50.75      -49.81      -14.33
```

In the linear mixed model we’ve fitted, it’s essential to note that due to the categorical nature of the treatment variable, there is no “slope” for the treatment variable in the conventional sense. Instead, the model estimates the differences in the intercept (mean recovery time) for treatments B and C relative to treatment A which is the same as the slope of the regression line in this case. Therefore, In the model, treatment effects are represented as shifts from the baseline (treatment A) rather than as distinct slopes or lines. If we look

at the estimated fixed effects, we see that we have an intercept and two more coefficients for treatment B and treatment C. The intercept represents the expected recovery time for treatment A, and the coefficients for treatment B and treatment C represent the deviation in recovery time from treatment A.

Let's plot these effects first before interpreting the random effects:

```
plot(data_int_treatment$recoveryTime ~
  ↪ jitter(as.numeric(data_int_treatment$treatment), 1),
  ↪ col=as.factor(data_int_treatment$treatment), pch=as.numeric(as.factor(data_int_treatment$doctor)),
  ↪ = "Recovery Time")
axis(1, at=c(1,2,3), labels=c("A","B","C"))
legend("top", legend = paste("Doctor",
  ↪ unique(data_int_treatment$doctor)),
  ↪ pch = unique(data_int_treatment$doctor), cex = 0.8, title
  ↪ = "Doctor", horiz=T)

abline(fixef(model_lmm_treatment)[1], b = 0, col=1)
abline(fixef(model_lmm_treatment)[1]+fixef(model_lmm_treatment)[2], b
  ↪ = 0, col=2)
abline(fixef(model_lmm_treatment)[1]+fixef(model_lmm_treatment)[3], b
  ↪ = 0, col=3)
```

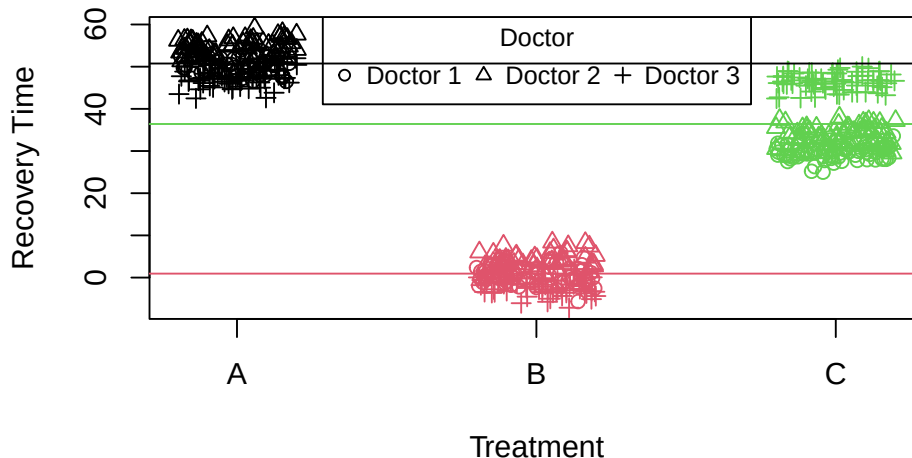


Figure 20.7: Scatter plot of treatment vs recovery time (fixed effects)

Given such a fit, what do you think the random intercept model actually does?

The answer to this question is actually quite simple. What we asked `lmer` to do, was to assume that the **Intercept** is the random effect. In this particular example the intercept is the baseline level of the treatment A. Therefore, the random intercept captures variability at the doctor level around the baseline effect of treatment A:

```
print(coef(model_lmm_treatment))

$doctor
  (Intercept) treatmentB treatmentC
1    48.12258   -49.81091   -14.33386
2    52.07763   -49.81091   -14.33386
3    52.04323   -49.81091   -14.33386

attr(,"class")
[1] "coef.mer"
```

Despite that we saw that there is three lines (three intercepts) in this model, only the first one is assume to be random. The coefficients for treatments B and C are relative to this random baseline and have not been allowed to change depending on the doctor. This model is sufficient if we assume that the effect of the treatments is consistent across all doctors, meaning that the difference in recovery times between treatments does not vary from one doctor to another. Essentially, it assumes that each treatment's effectiveness is uniform regardless of who administers it.

Let's plot these effects before proceeding:

```
doctor_effects <-
  ↪ cbind(doctor=c(1:3),coef(model_lmm_treatment)$doctor)
colnames(doctor_effects)[2]<-c("intercept")
# Calculate adjusted means for treatments B and C
doctor_effects$treatmentB_effect = doctor_effects$intercept +
  ↪ doctor_effects$treatmentB
doctor_effects$treatmentC_effect = doctor_effects$intercept +
  ↪ doctor_effects$treatmentC

# Plot
plot(NULL, xlim=c(1, 3), ylim=range(c(doctor_effects$intercept,
  ↪ doctor_effects$treatmentB_effect,
  ↪ doctor_effects$treatmentC_effect)), xlab="Doctor",
  ↪ ylab="Recovery Time", main="Effect of Doctor and Treatment on
  ↪ Recovery Time",xaxt="n")
axis(1, at=c(1,2,3), labels=c("1","2","3"))

points(doctor_effects$doctor, doctor_effects$intercept, pch=19,
  ↪ col="blue") # Baseline (A)
points(doctor_effects$doctor, doctor_effects$treatmentB_effect,
  ↪ pch=19, col="red") # Treatment B
points(doctor_effects$doctor, doctor_effects$treatmentC_effect,
  ↪ pch=19, col="green") # Treatment C

# Connect points with lines to show the constant effect across
  ↪ doctors
lines(doctor_effects$doctor, doctor_effects$intercept,
  ↪ col="blue", lty=2)
```

```

lines(doctor_effects$doctor, doctor_effects$treatmentB_effect,
      col="red", lty=2)
lines(doctor_effects$doctor, doctor_effects$treatmentC_effect,
      col="green", lty=2)

legend("right", legend=c("Baseline (A)", "Treatment B",
      "Treatment C"), col=c("blue", "red", "green"), pch=19, lty=2)

```

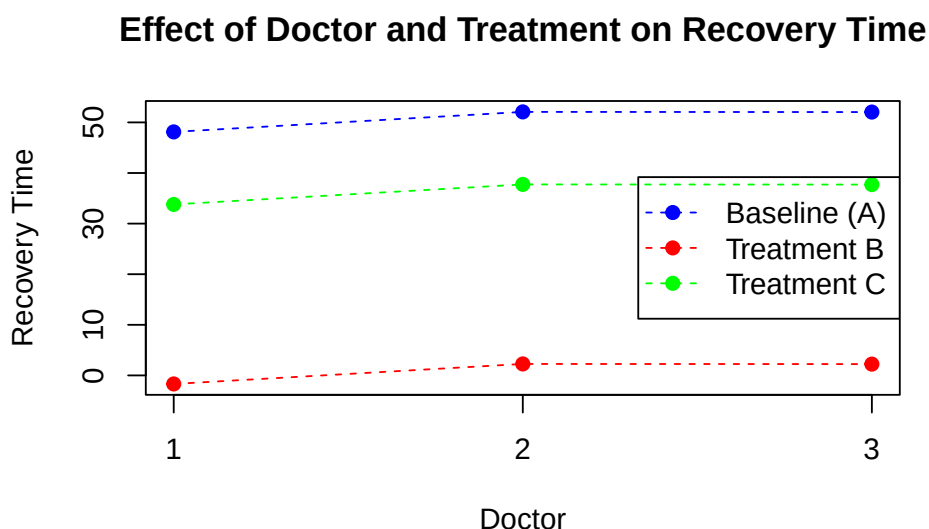


Figure 20.8: Scatter plot of treatment vs recovery time (random intercept)

What this plot tells us is that, each doctor might have certain way of treating patient which affect the overall recovery time. However, what we assume is that the treatment specific effects are constant across the doctors. That means that no matter which doctor a patient go to, there is a **constant** shift in recovery time when the patient switch between the treatments. But what if some doctors are much more skilled that others to administrate a certain type of treatments? The previous model does not allow for the examination of how the effect of **each treatment** might vary by doctor, as it assumes a fixed effect of treatments across all doctors.

We have to specifically clarify for `lmer` which effects we want to be random. In this case, we are going to use the treatment effect as random and check what would happen

```

model_lmm_treatment_rnd <- lmer(recoveryTime ~ treatment +
      (1+treatment|doctor), data_int_treatment)

```

Warning in checkConv(attr(opt, "derivs"), opt\$par, ctrl = control\$checkConv, : Model failed to
See ?lme4::convergence and ?lme4::troubleshooting.

```
print(model_lmm_treatment_rnd)
```

Linear mixed model fit by REML ['lmerMod']

```

Formula: recoveryTime ~ treatment + (1 + treatment | doctor)
Data: data_int_treatment
REML criterion at convergence: 2545.443
Random effects:
Groups   Name             Std.Dev. Corr
doctor  (Intercept)    3.4988
        treatmentB   0.4936 -0.84
        treatmentC  11.2470 -0.80  0.34
Residual                1.9710
Number of obs: 600, groups: doctor, 3
Fixed Effects:
(Intercept)  treatmentB  treatmentC
      50.89      -50.02      -14.27
optimizer (nloptwrap) convergence code: 0 (OK) ; 0 optimizer warnings; 1 lme4 warning

```

If we look at the random effect part of the model, it is now more clear that There is not only a big difference between the doctors when it comes to treatment A but also an even larger difference in treatment C.

```

doctor_effects <-
  ↪ cbind(doctor=c(1:3),coef(model_lmm_treatment_rnd)$doctor)
colnames(doctor_effects)[2]<-c("intercept")
# Calculate adjusted means for treatments B and C
doctor_effects$treatmentB_effect = doctor_effects$intercept +
  ↪ doctor_effects$treatmentB
doctor_effects$treatmentC_effect = doctor_effects$intercept +
  ↪ doctor_effects$treatmentC

# Plot
plot(NULL, xlim=c(1, 3), ylim=range(c(doctor_effects$intercept,
  ↪ doctor_effects$treatmentB_effect,
  ↪ doctor_effects$treatmentC_effect)), xlab="Doctor",
  ↪ ylab="Recovery Time", main="Effect of Doctor and Treatment on
  ↪ Recovery Time",xaxt="n")
axis(1, at=c(1,2,3), labels=c("1","2","3"))

points(doctor_effects$doctor, doctor_effects$intercept, pch=19,
  ↪ col="blue") # Baseline (A)
points(doctor_effects$doctor, doctor_effects$treatmentB_effect,
  ↪ pch=19, col="red") # Treatment B
points(doctor_effects$doctor, doctor_effects$treatmentC_effect,
  ↪ pch=19, col="green") # Treatment C

# Connect points with lines to show the constant effect across
  ↪ doctors
lines(doctor_effects$doctor, doctor_effects$intercept,
  ↪ col="blue", lty=2)
lines(doctor_effects$doctor, doctor_effects$treatmentB_effect,
  ↪ col="red", lty=2)

```

```
lines(doctor_effects$doctor, doctor_effects$treatmentC_effect,
      col="green", lty=2)

legend("right", legend=c("Baseline (A)", "Treatment B",
      ↪ "Treatment C"), col=c("blue", "red", "green"), pch=19, lty=2)
```

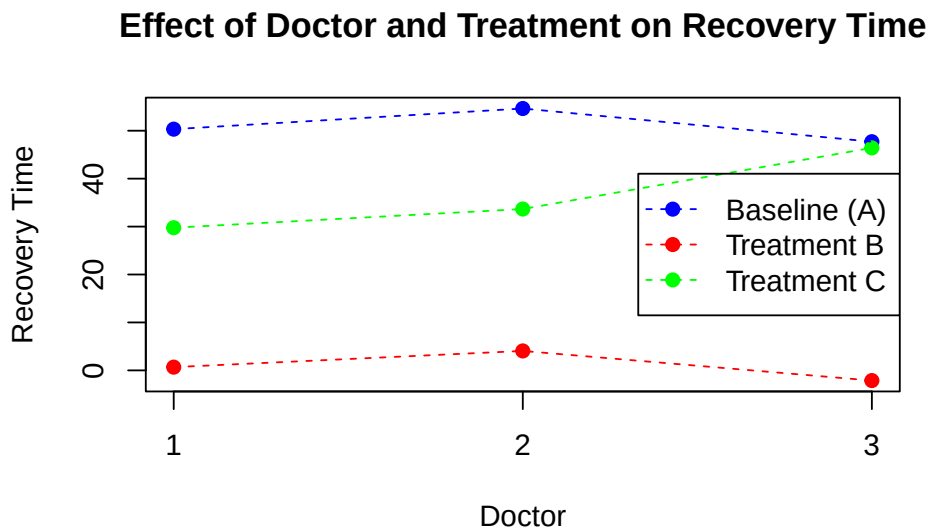


Figure 20.9: Scatter plot of treatment vs recovery time (random intercept and deviation)

So despite that there is no slope for continuous variable in a traditional sense, we can still model the variations in the deviations from the baseline. This model structure is particularly useful for highlighting the overall effectiveness of each treatment while controlling for variability in outcomes attributed to individual doctor differences.

We can now explore a bit more complex model that involves the interaction.

20.3.4.1 Models with interaction

In this example, our aim is to explore how the effectiveness of various treatments may vary with changes in dosage, and whether these effects are consistent across different doctors.

```
simulatePatientData <- function(numDoctors = 30, numPatients =
  ↪ 10, doctor_dosage, doctor_var, seed = 12345) {
  set.seed(seed)

  # Parameters
  intercept = 50 # Baseline recovery time
  treatmentEffect = c(A = 0, B = -50, C = -20) # Treatment
  ↪ effect on recovery time
  dosageEffect = c(A = 3, B = -2, C = 1) # Effect of dosage on
  ↪ recovery time
```

```

# Simulate data
doctors <- rep(1:numDoctors, each = numPatients)
treatment <- sample(c("A", "B", "C"), size = numDoctors *
↪ numPatients, replace = TRUE)
dosage <- runif(numDoctors * numPatients, min = 1, max = 10) #
↪ Dosage levels between 1 and 10
dosageEffect<-rep(dosageEffect,each=numPatients)
# Random effects for doctors
doctorIntercept <- rnorm(numDoctors, 0, 2) # Variation in
↪ doctor baseline recovery times

# Calculate recovery time
tr<-treatmentEffect[treatment]
for(i in unique(doctors))
{
  tr[treatment=="A" & doctors==i]<-tr[treatment=="A"
↪ &doctors==i]+doctor_var[[i]][1]
  tr[treatment=="B" & doctors==i]<-tr[treatment=="B"
↪ &doctors==i]+doctor_var[[i]][2]
  tr[treatment=="C" & doctors==i]<-tr[treatment=="C"
↪ &doctors==i]+doctor_var[[i]][3]

}

recoveryTime <- intercept+tr +

↪ unlist(doctor_dosage)[paste(doctors,".",treatment,sep="")]
↪ * dosage + doctorIntercept[doctors] +
rnorm(numDoctors * numPatients, 0, 2) # Add
↪ some observation noise

data <- data.frame(doctor = doctors, treatment = treatment,
↪ dosage = dosage, recoveryTime = recoveryTime)
return(data)
}

# Generate the data
data_int_interaction <- simulatePatientData(numDoctors =
↪ 3,numPatients = 200,doctor_dosage=list("1"=c(A = 3, B = -2, C
↪ = 1),
↪ "2"=c(A = 3, B = -2, C = 1),"3"=c(A = 3, B
↪ = -2, C =
↪ -1)),doctor_var=list(c(1,1.4,0),c(1.2,1,0),c(0.9,1,50)))

data_int_interaction$treatment<-factor(data_int_interaction$treatment,levels
↪ = c("A","B","C"))

```

```

↪ plot(data_int_interaction$dosage,data_int_interaction$recoveryTime,col=as.factor(data_int_
      xlab="Dosage",ylab="Recovery time")

legend("top", legend = paste("Doctor",
↪ unique(data_int_interaction$doctor)),
      pch = unique(data_int_interaction$doctor), cex = 0.8,
      ↪ title = "Doctor",horiz=T)
legend("topleft", legend = paste("Treatment",
↪ levels(data_int_interaction$treatment)),
      pch=1,col =
      ↪ 1:length(levels(data_int_interaction$treatment)), cex
      ↪ = 0.8, title = "Treatment",horiz=F)

```

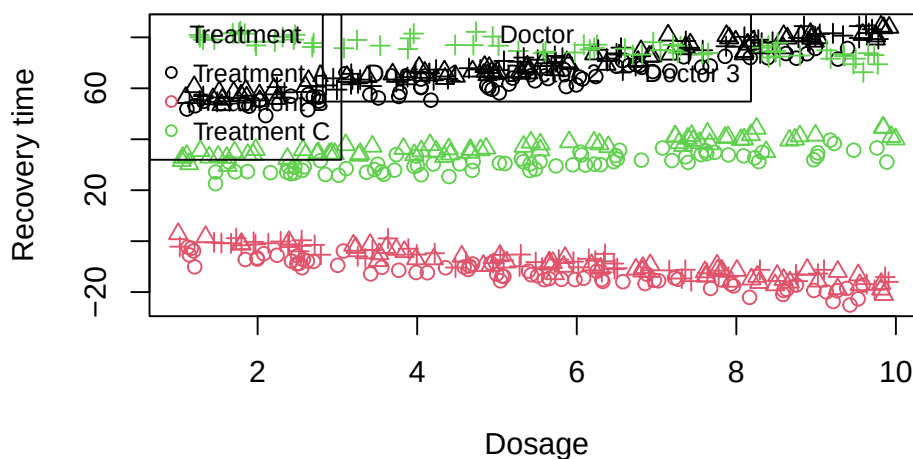


Figure 20.10: Scatter plot of treatment vs recovery time vs dosage

What we believe is that there is dose-response relationship that is not uniform across all treatments. In other words, the impact of increasing dosage on treatment effectiveness might depend on the specific type of treatment being administered. To investigate this hypothesis, we incorporate interaction terms into our mixed model.

```

model_lmm_treatment_dosage <- lmer(recoveryTime ~
  ↪ treatment*dosage + (1|doctor), data_int_interaction)
print(model_lmm_treatment_dosage)

```

```

Linear mixed model fit by REML ['lmerMod']
Formula: recoveryTime ~ treatment * dosage + (1 | doctor)
Data: data_int_interaction
REML criterion at convergence: 4366.567
Random effects:
Groups   Name             Std.Dev.
doctor   (Intercept)  9.874
Residual                      9.170

```

Number of obs: 600, groups: doctor, 3

Fixed Effects:

(Intercept)	treatmentB	treatmentC	dosage
50.905	-51.009	-7.324	2.950
treatmentB:dosage	treatmentC:dosage		
-4.708	-2.210		

Similar to the previous case, we start assuming a random intercept in the model. What we see is that there is a big variation across different doctors when it comes to the baseline of the treatment A. However, here we only assumed that there is only the level of treatment A is random across the doctors. Let's look at the fitted lines:

```

↪ plot(data_int_interaction$dosage,data_int_interaction$recoveryTime,
↪ col=as.factor(data_int_interaction$treatment),pch=as.numeric(as.factor(data_int_
  xlab="Dosage",ylab="Recovery time")

for(dr in unique(data_int_interaction$doctor))
{
  abline(coef(model_lmm_treatment_dosse)[[1]][dr,1],
        coef(model_lmm_treatment_dosse)[[1]][dr,4],lty=dr,col=1)

  abline(coef(model_lmm_treatment_dosse)[[1]][dr,1]+coef(model_lmm_treatment_dosse)[[1]
    ↪ coef(model_lmm_treatment_dosse)[[1]][dr,4]+coef(model_lmm_treatment_dosse)

  abline(coef(model_lmm_treatment_dosse)[[1]][dr,1]+coef(model_lmm_treatment_dosse)[[1]
    ↪ coef(model_lmm_treatment_dosse)[[1]][dr,4]+coef(model_lmm_treatment_dosse)

}

```

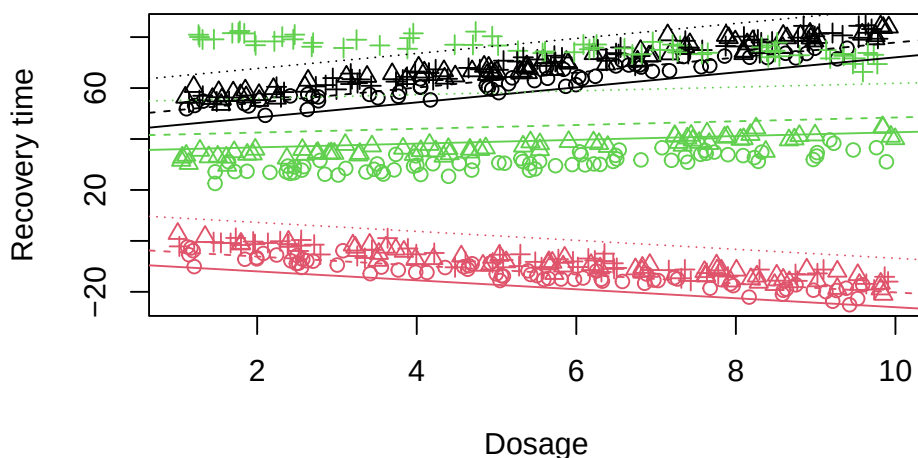


Figure 20.11: Scatter plot of treatment vs recovery time vs dosage, fitted lines

What is clear is that the fit does not look that promising specially when it comes to the Doctor 3 (treatment C). One might suspect that the effect of dosage administration can also depend on the doctor. In this case, we need to model the dosage as random effect:

```
model_lmm_treatment_doss_rnd <-
  ↪ lmer(recoveryTime~treatment*dosage+(1+dosage|doctor),data_int_interaction)
print(model_lmm_treatment_doss_rnd)
```

Linear mixed model fit by REML ['lmerMod']

Formula: recoveryTime ~ treatment * dosage + (1 + dosage | doctor)

Data: data_int_interaction

REML criterion at convergence: 4363.593

Random effects:

Groups	Name	Std.Dev.	Corr
doctor	(Intercept)	11.5778	
	dosage	0.3272	-0.96
Residual		9.1444	

Number of obs: 600, groups: doctor, 3

Fixed Effects:

(Intercept)	treatmentB	treatmentC	dosage
50.807	-50.937	-7.268	2.975
treatmentB:dosage	treatmentC:dosage		
-4.737	-2.218		

```
↪ plot(data_int_interaction$dosage,data_int_interaction$recoveryTime,col=as.factor(data_int_
  xlab="Dosage",ylab="Recovery time")

for(dr in unique(data_int_interaction$doctor))
{
  abline(coef(model_lmm_treatment_doss_rnd)[[1]][dr,1],

    ↪ coef(model_lmm_treatment_doss_rnd)[[1]][dr,4],lty=dr,col=1)

  abline(coef(model_lmm_treatment_doss_rnd)[[1]][dr,1]+coef(model_lmm_treatment_doss_rnd)[[1]][dr,4],lty=dr,col=1)

    ↪ coef(model_lmm_treatment_doss_rnd)[[1]][dr,4]+coef(model_lmm_treatment_doss_rnd)[[1]][dr,1],lty=dr,col=1)

  abline(coef(model_lmm_treatment_doss_rnd)[[1]][dr,1]+coef(model_lmm_treatment_doss_rnd)[[1]][dr,4],lty=dr,col=1)

    ↪ coef(model_lmm_treatment_doss_rnd)[[1]][dr,4]+coef(model_lmm_treatment_doss_rnd)[[1]][dr,1],lty=dr,col=1)

}
```

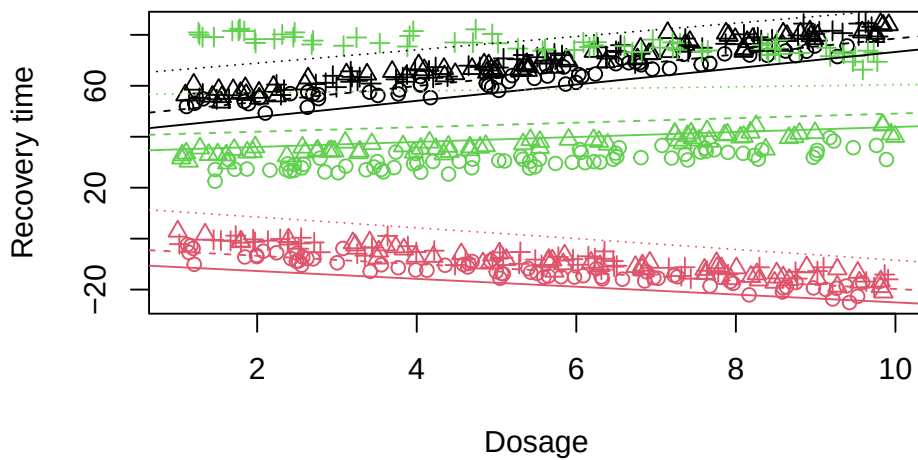


Figure 20.12: Scatter plot of treatment vs recovery time vs dosage, fitted lines

Looking at the plot above and the output of the mixed model, the fit does not look much better than before. `Dosage` alone also does not seem to explain much of the variability. So what are we missing?

Similar to the previous scenario, not only the baseline treatment level (treatment A) is changing across the doctors but also the rate at which the doctors perform the treatment change. So let's incorporate this:

```
model_lmm_treatment_doss_trt_rnd <-
  ↪ lmer(recoveryTime~treatment*dosage+(1+treatment+dosage|doctor),data_int_interacti
```

```
boundary (singular) fit: see help('isSingular')
```

```
print(model_lmm_treatment_doss_trt_rnd)
```

```
Linear mixed model fit by REML ['lmerMod']
```

```
Formula: recoveryTime ~ treatment * dosage + (1 + treatment + dosage |
```

```
  doctor)
```

```
Data: data_int_interaction
```

```
REML criterion at convergence: 2791.263
```

```
Random effects:
```

Groups	Name	Std.Dev.	Corr
doctor	(Intercept)	4.8010	
	treatmentB	0.3782	-1.00
	treatmentC	22.4142	0.85 -0.85
	dosage	0.3983	-0.84 0.84 -1.00

```
Residual
```

```
Number of obs: 600, groups: doctor, 3
```

```
Fixed Effects:
```

	(Intercept)	treatmentB	treatmentC	dosage
	50.462	-49.589	-4.396	3.096
treatmentB:dosage		-5.068		
treatmentC:dosage			-2.727	

```
optimizer (nloptwrap) convergence code: 0 (OK) ; 0 optimizer warnings; 1 lme4 warning
```

```

↪ plot(data_int_interaction$dosage,data_int_interaction$recoveryTime,col=as.factor(data_int_
      xlab="Dosage",ylab="Recovery time")

for(dr in unique(data_int_interaction$doctor))
{
  abline(coef(model_lmm_treatment_doss_trt_rnd)[[1]][dr,1],

      ↪ coef(model_lmm_treatment_doss_trt_rnd)[[1]][dr,4],lty=dr,col=1)

  abline(coef(model_lmm_treatment_doss_trt_rnd)[[1]][dr,1] +
      ↪ coef(model_lmm_treatment_doss_trt_rnd)[[1]][dr,2],
        coef(model_lmm_treatment_doss_trt_rnd)[[1]][dr,4] +
        ↪ coef(model_lmm_treatment_doss_trt_rnd)[[1]][dr,5],lty=dr,col=2)

  abline(coef(model_lmm_treatment_doss_trt_rnd)[[1]][dr,1] +
      ↪ coef(model_lmm_treatment_doss_trt_rnd)[[1]][dr,3],
        coef(model_lmm_treatment_doss_trt_rnd)[[1]][dr,4] +
        ↪ coef(model_lmm_treatment_doss_trt_rnd)[[1]][dr,6],lty=dr,col=3)
}

```

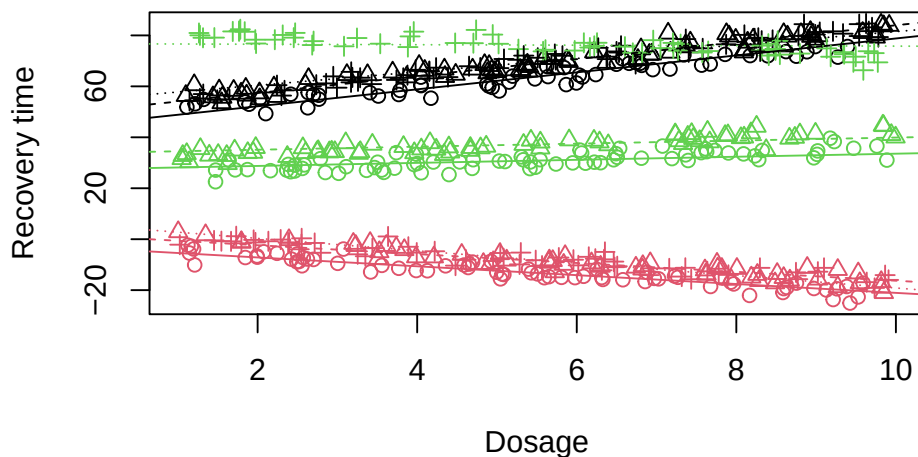


Figure 20.13: Scatter plot of treatment vs recovery time vs dosage, fitted lines

Now the fit looks much better than before. We can see in the output that the treatment C is now explaining a lot of variability across different doctors. At this point the main question is given that we have assumed the differences in the treatment levels can be changing across the doctors could it be that the differences between the slopes are also random?

What this means is that so far, what we have done is modeling (Intercept), `treatmentB`, `treatmentC`, `dosage` as random effects. But `treatmentB:dosage` and `treatmentC:dosage` are still fixed. This translate to a scenario that we

believe there is a general differences between the doctors in a way that they administrate a *specific* treatment and also how they generally work with doses. But what if a doctor is much more skilled in administrating a dosage for a specific treatment?

In this case, we have to model the interaction as random effect:

```
model_lmm_treatment_int_rnd <-
  ↪ lmer(recoveryTime~treatment*dosage+(1+treatment*dosage|doctor),data_int_interaction)
```

boundary (singular) fit: see help('isSingular')

```
print(model_lmm_treatment_int_rnd)
```

Linear mixed model fit by REML ['lmerMod']

Formula: recoveryTime ~ treatment * dosage + (1 + treatment * dosage | doctor)

Data: data_int_interaction

REML criterion at convergence: 2580.411

Random effects:

Groups	Name	Std.Dev.	Corr
doctor	(Intercept)	2.4120	
	treatmentB	0.9624	1.00
	treatmentC	29.8022	0.43 0.43
	dosage	0.1633	0.86 0.86 0.84
	treatmentB:dosage	0.2238	-1.00 -1.00 -0.47 -0.88
	treatmentC:dosage	1.3254	-0.46 -0.46 -1.00 -0.85 0.50
Residual		2.0081	

Number of obs: 600, groups: doctor, 3

Fixed Effects:

	(Intercept)	treatmentB	treatmentC	dosage
	50.644	-49.623	-4.641	3.050
treatmentB:dosage				
treatmentC:dosage				
	-5.027	-2.642		

optimizer (nloptwrap) convergence code: 0 (OK) ; 0 optimizer warnings; 1 lme4 warning

```
↪ plot(data_int_interaction$dosage,data_int_interaction$recoveryTime,col=as.factor(
  xlabel="Dosage",ylab="Recovery time")
```

```
for(dr in unique(data_int_interaction$doctor))
```

```
{
  abline(coef(model_lmm_treatment_int_rnd)[[1]][dr,1],
         coef(model_lmm_treatment_int_rnd)[[1]][dr,4],lty=dr,col=1)
```

```
abline(coef(model_lmm_treatment_int_rnd)[[1]][dr,1] +
  ↪ coef(model_lmm_treatment_int_rnd)[[1]][dr,2],
       coef(model_lmm_treatment_int_rnd)[[1]][dr,4] +
  ↪ coef(model_lmm_treatment_int_rnd)[[1]][dr,5],lty=dr,col=2)
```

```
abline(coef(model_lmm_treatment_int_rnd)[[1]][dr,1] +
  ↪ coef(model_lmm_treatment_int_rnd)[[1]][dr,3],
  coef(model_lmm_treatment_int_rnd)[[1]][dr,4] +
  ↪ coef(model_lmm_treatment_int_rnd)[[1]][dr,6], lty=dr, col=3)
}
```

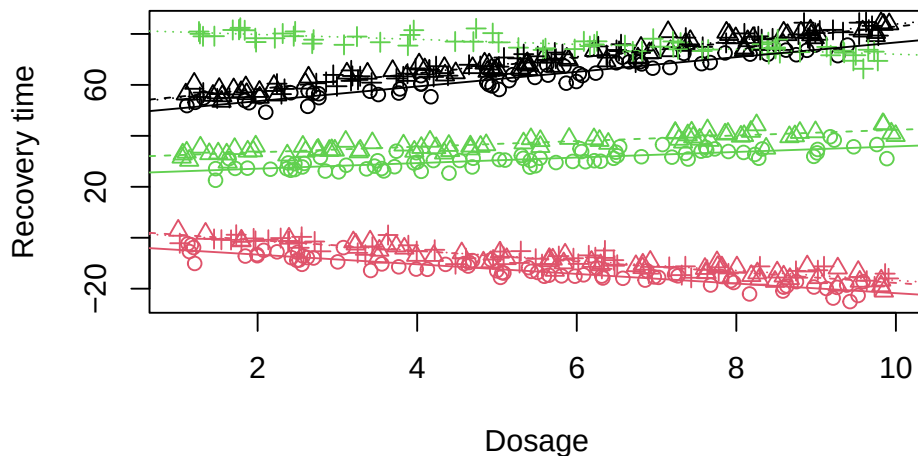


Figure 20.14: Scatter plot of treatment vs recovery time vs dosage, fitted lines

This final model tells that that the three main sources of variation in our data is `treatmentC`, (`Intercept`) and `treatmentC:dosage`. The fit also looks much better than before. This result translates to a scenario in which there is a variability between doctors in how they treat patients, there is a significant variability between then in the way they administrate the treatment C and also how they administrate dosage for treatment C.

To summarize this section, I want to point out that despite that there are some general terms such as random intercept or random slope models, one might often end up specifically model certain effects as random or fixed depending on the experimental design and characteristic of the samples and population. These effect should have been ideally studied prior to setting up the statistical model to avoid bias. In the next section we will have a look at some automated way of selecting fixed and random effect.

20.4 Model selection and comparisons using LRT

A critical step in using mixed models effectively is determining which model provides the best fit to the data without overcomplicating the structure. ANOVA for mixed models serves as a tool for hypothesis testing between models, specifically testing if a more complex model significantly improves the explanation of data variability over a simpler one. Without going too much into the

details (theory is covered in the math section), what we are going to do is likelihood ratio test (LRT) between different models. The Likelihood Ratio Test (LRT) is a statistical procedure used to compare the fit of two models to a set of data, where one model (the null model) is a special case of the other model (the alternative model). The LRT evaluates whether the more complex model significantly improves the fit to the data over the simpler model. The LRT is based on the likelihood ratio, which is a measure of how much more likely the data is under one model compared to another. Specifically, it compares the maximum likelihood of the data under the null hypothesis (H_0), a simpler model with constraints on parameters) against the maximum likelihood under the alternative hypothesis (H_1), a more complex model without those constraints).

In the case of mixed models, as you noticed, we have two sets of parameters to consider: fixed effects, which are common to all observations, and random effects, which vary across levels of a grouping factor (e.g., subjects, experiments, clusters). This dual nature adds a layer of complexity to model selection and comparison. Specifically, when conducting an LRT, we're not only interested in how adding or removing fixed effects influences the model but also in how variations in the random effects structure contribute to explaining the data variability.

There are different approaches on how to apply model selection on mixed models. Here we are going to use a simple approach.

First we are going to use LRT to select the random part of the model. With random part of the model chosen, we will proceed with selecting the fixed part. `lme4` package provides `anova` function for doing so. As suggested by some authors, we are going to fit the models as complex as we believe they are and see whether a simpler model is preferred.

! Important

When we apply model selection for the random part, all the models must have the same fixed structure. Similar, when working with fixed part, the random structure of the model must remain the same.

So let's start with the random part of the model that included the interaction.

```
m1 <- lmer(recoveryTime ~ treatment*dosage + (1|doctor),
  ↪ data_int_interaction, REML = TRUE)
m2 <- lmer(recoveryTime ~ treatment*dosage +
  ↪ (1+treatment|doctor), data_int_interaction, REML = TRUE)
m3 <- lmer(recoveryTime ~ treatment*dosage +
  ↪ (1+treatment+dosage|doctor), data_int_interaction, REML =
  ↪ TRUE)
m4 <- lmer(recoveryTime ~ treatment*dosage +
  ↪ (1+treatment*dosage|doctor), data_int_interaction, REML =
  ↪ TRUE)
print(anova(m1,m2,m3,m4,refit=FALSE))
```

Data: data_int_interaction

```
Models:
m1: recoveryTime ~ treatment * dosage + (1 | doctor)
m2: recoveryTime ~ treatment * dosage + (1 + treatment | doctor)
m3: recoveryTime ~ treatment * dosage + (1 + treatment + dosage | doctor)
m4: recoveryTime ~ treatment * dosage + (1 + treatment * dosage | doctor)
      npar    AIC    BIC  logLik -2*log(L)   Chisq Df Pr(>Chisq)
m1      8 4382.6 4417.7 -2183.3    4366.6
m2     13 2887.0 2944.2 -1430.5    2861.0 1505.55   5 < 2.2e-16 ***
m3     17 2825.3 2900.0 -1395.6    2791.3   69.75   4 2.563e-14 ***
m4     28 2636.4 2759.5 -1290.2    2580.4   210.85  11 < 2.2e-16 ***
---
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
```

The ANOVA table provides essential metrics for model comparison.

ANOVA table columns

Number of Parameters (npa):

The ‘npa’ indicates the total number of parameters estimated by the model, reflecting its complexity.

Akaike Information Criterion (AIC):

AIC assesses the model quality, penalizing excessive complexity. Models with lower AIC values are generally preferred, as they give a balance between fitting the data well and being parsimonious.

Bayesian Information Criterion (BIC):

Similar to AIC, the BIC also rewards model fit but imposes a stricter penalty on the number of parameters. BIC is particularly useful for model selection in scenarios where overfitting is a concern, as its heavier penalty discourages unnecessarily complex models.

Log-Likelihood (logLik):

Log-likelihood is a direct measure of model fit, representing the logarithm of the likelihood function at its maximum point. Higher values indicate better fit.

Deviance:

Derived from the log-likelihood, deviance is a measure of model fit that allows direct comparison between models. It is defined as twice the difference between the maximum log-likelihood and the log-likelihood under the model being tested. Lower deviance values indicate a better fit to the data.

Chi-Squared Test (Chisq) and Degrees of Freedom (Df):

These metrics are used to test whether the addition of parameters (and thus, complexity) significantly improves the model fit. The chi-squared statistic tests the null hypothesis that both models fit the data equally well. The degrees of freedom, calculated as the difference in parameters between models, help determine the critical value for this test.

Probability Value (Pr(>Chisq)):

A low p-value (typically <0.05) indicates that the more complex model provides a significantly better fit to the data, justifying its additional complexity.

The table is sorted based on the complexity of the model (the number of parameters). The first row is the least and the last row is the most complex model. The columns used for comparisons the `Pr(>Chisq)` (p-value) is comparisons the current row to the previous one. So let's start interpreting the results from the table, it's evident that each successive, more complex model significantly improves the fit to the data over its predecessor, as indicated by the `Pr(>Chisq)` column. This p-value compares the current model to the previous one in terms of how much more likely the data is under the more complex model.

Starting with model m1, which includes a random intercept for doctors, we observe a baseline for comparison. This model sets the foundation with a certain number of parameters (`npar = 8`), Akaike Information Criterion (AIC), Bayesian Information Criterion (BIC), log-likelihood (`logLik`), and deviance scores.

When we move to model m2, which adds treatment as a random slope alongside the doctor, we see a substantial improvement in model fit. This is evidenced by the dramatic reduction in AIC and BIC values and a significant chi-squared statistic (`Chisq = 1505.55`) with a p-value less than $2.2e-16$, indicating a highly significant improvement over m1. The addition of treatment as a random effect accounts for the variability in treatment effects across doctors, enhancing the model's explanatory power.

The transition from m2 to m3, where dosage is added as another random slope, continues to show significant model improvement, albeit the magnitude of improvement is less dramatic than the previous step. This is reflected in the `Chisq` value of 69.75 with a p-value of $2.563e-14$, indicating that incorporating dosage variability across doctors further refines our model's accuracy.

Finally, model m4, which introduces an interaction between treatment and dosage as a random slope, marks the most substantial complexity increase, as shown by the jump in the number of parameters to 28. This model shows another significant leap in fit, with a `Chisq` value of 210.85 and a p-value less than $2.2e-16$. The substantial decrease in AIC and BIC values alongside this indicates that considering the interaction between treatment and dosage in the model's random effects structure captures a critical aspect of the data's underlying structure, providing the best fit among the compared models.

In summary, the progression from m1 to m4 illustrates a clear trajectory of increasing model complexity leading to significantly improved fits to the data, as demonstrated by decreasing AIC/BIC values and highly significant p-values in the likelihood ratio tests.

! Important

Please note that in order to compare the random effect part, we HAVE to set `refit=FALSE` in the `anova` function. The reason for this is that using `refit=FALSE` would use the original estimation method, which in our case was Restricted Maximum Likelihood (REML). REML adjusts for the loss of degrees of freedom when estimating fixed effects, providing unbiased estimates of variance components. We must always use `REML = TRUE` when fitting and reporting mixed models. However, REML assumes

that the fixed effects structure is correct. So in order to compare fixed effect part of the model, we need to use `refit=TRUE` or fit the original mixed model using `REML = FALSE` only for the purpose of comparing the fixed effects. The repoted/final model must always be based on `REML = TRUE`. More about this in the math section.

Give we now know that our interaction model was the best one, we can now proceed with refining our model by selecting the most appropriate fixed effects. We will keep the random part constant across the models and change the fixed part:

```
m1 <- lmer(recoveryTime ~ 1 + (1+treatment*dosage|doctor),
  ↪ data_int_interaction, REML = FALSE)
m2 <- lmer(recoveryTime ~ treatment+(1+treatment*dosage|doctor),
  ↪ data_int_interaction, REML = FALSE)
m3 <- lmer(recoveryTime ~ treatment+dosage +
  ↪ (1+treatment*dosage|doctor), data_int_interaction, REML =
  ↪ FALSE)
m4 <- lmer(recoveryTime ~ treatment*dosage +
  ↪ (1+treatment*dosage|doctor), data_int_interaction, REML =
  ↪ FALSE)
print(anova(m1,m2,m3,m4,refit=TRUE))
```

Data: data_int_interaction

Models:

```
m1: recoveryTime ~ 1 + (1 + treatment * dosage | doctor)
m2: recoveryTime ~ treatment + (1 + treatment * dosage | doctor)
m3: recoveryTime ~ treatment + dosage + (1 + treatment * dosage | doctor)
m4: recoveryTime ~ treatment * dosage + (1 + treatment * dosage | doctor)
```

	npars	AIC	BIC	logLik	-2*log(L)	Chisq	Df	Pr(>Chisq)
m1	23	2657.9	2759.0	-1305.9	2611.9			
m2	25	2646.3	2756.2	-1298.1	2596.3	15.6026	2	0.0004092 ***
m3	26	2647.7	2762.0	-1297.8	2595.7	0.6007	1	0.4383114
m4	28	2632.4	2755.5	-1288.2	2576.4	19.2922	2	6.468e-05 ***

 Signif. codes: 0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

Starting with model m1, which serves as our baseline model including only the intercept in the fixed part, we assess the improvement in fit as we incrementally add fixed effects. Model m1 establishes the groundwork with specific values for the number of parameters (npars), AIC, BIC, log-likelihood (logLik), and deviance.

The progression to model m2, which introduces treatment as a fixed effect, gives a big improvement in the model's explanatory power. This is clear by a decrease in both AIC and BIC scores and a significant chi-squared statistic (Chisq = 15.6092) with a p-value of 0.0004078. The statistical significance indicates that the inclusion of treatment as a fixed effect significantly enhances the model's fit over the baseline model, which is consistent with the expectation that treatment would have a systematic effect on recovery time.

Going to model m3, which further includes dosage alongside treatment in the fixed effects, we observe a marginal change. The chi-squared statistic ($\text{Chisq} = 0.5915$) with a p-value of 0.4418240 suggests that the addition of dosage as a fixed effect does not significantly improve the model's fit beyond what is achieved by including treatment alone.

Model m4 reintroduces the treatment by dosage interaction in the fixed effects, reflecting the most complex fixed-effects structure among the evaluated models. The significant reduction in AIC and BIC values, coupled with a chi-squared statistic ($\text{Chisq} = 19.2892$) and a p-value of 6.477e-05, shows the critical importance of this interaction term. The substantial improvement in model fit indicates that the interaction between treatment and dosage is a key factor in explaining variations in recovery time, warranting its inclusion in both the fixed and random components of the model.

We now know that our final model should be in the form of `lmer(recoveryTime~treatment*dosage+(1|REML = TRUE))`. Please note that we use `REML=TRUE` in the final model.

! Important

It is always a good practice to design the model based on the population/sample structure and experimental design. Despite model selection being a powerful tool, it cannot replace the necessity for substantive knowledge and theoretical understanding of the phenomena under study. Relying solely on automated procedures or statistical criteria (like AIC or BIC) for model selection can lead to overfitting, misinterpretation of model parameters, and conclusions that do not accurately reflect the underlying processes. Moreover, important variables might be overlooked if they are not considered in the model selection process. Hence, it's crucial to integrate domain expertise with statistical approaches, ensuring that the chosen model is both statistically sound and theoretically justified.

20.5 Model diagnostic

The assumptions of mixed models are very similar to those of linear models. By far the most informative way of checking the model assumptions is the residuals vs fitted plot. This can be easily plotted using the `plot` function.

```
plot(model_lmm_treatment_int_rnd)
```

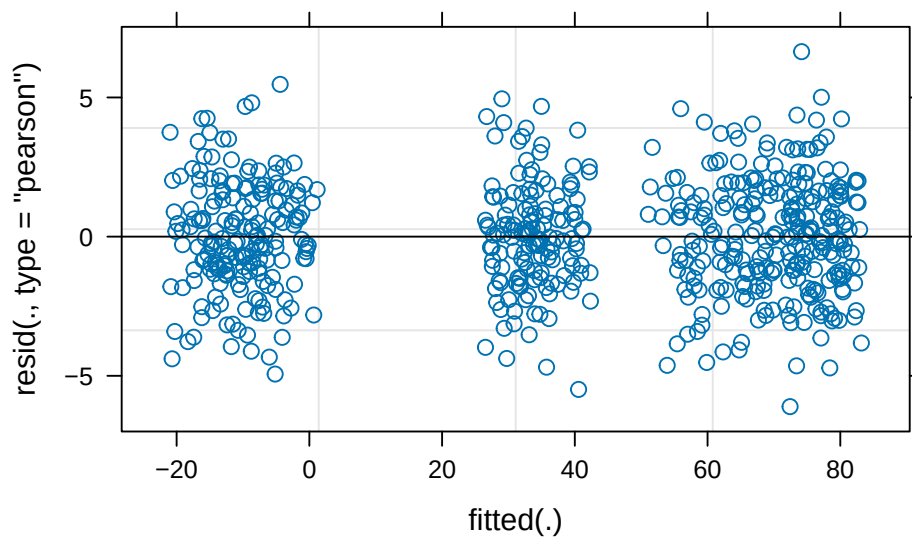


Figure 20.15: Scatter plot of residuals vs fitted values

A residuals versus fitted values plot provides insights into several assumptions underlying linear and mixed models. When interpreting this plot, there are a few key scenarios and patterns to look out for, each indicating different potential violations of model assumptions or areas for model improvement:

1. Homoscedasticity vs. Heteroscedasticity

- **Homoscedasticity** is assumed in linear models, meaning the variance of the residuals should be constant across all levels of the fitted values. In the plot, this should appear as a random scatter of points, evenly spread across the range of fitted values.
- **Heteroscedasticity** manifests as a fan-shaped or funnel-shaped pattern, where the spread of residuals increases or decreases as a function of the fitted values. This indicates that the variance of the errors is not constant and may necessitate transformations of variables or the use of models that allow for non-constant variance, such as generalized least squares.

2. Linearity

- The relationship between predictors and the response variable in linear models is assumed to be linear. A properly specified model should show no systematic patterns in the residuals plot. If the residuals appear to follow a curved pattern, this suggests a nonlinear relationship that the model has not captured, indicating the need for model revision, such as adding polynomial terms or considering non-linear models.

3. Independence of Errors

- Residuals should be independent of each other. In the context of time series data or spatial data, this assumption might be violated, often visible as “clumps” or “streaks” of residuals in the plot, indicating autocorrelation. Special models like ARIMA for time series or spatial models might be needed in these cases.

4. Outliers

- Outliers can be identified as points that stand far away from the cloud of points in the residuals vs. fitted plot. These outliers can have a disproportionate influence on the model fit and may need to be investigated further to determine if they should be removed or treated differently.

5. Missing Nonlinear Effects or Interactions

- If the residuals plot shows patterns, this might suggest that important predictors or interactions between predictors are missing from the model. Investigating the nature of these patterns can guide the inclusion of additional terms in the model.

There are other types of diagnostics tools and plots that can be used in different scenarios. **performance** package provides an easy to use function to check the assumptions of linear models. You need to install **performance** package to run the following code:

```
performance::check_model(model_lmm_treatment_int_rnd, panel =
  ↪ T, check = "all")
```

20.6 P-value calculation and marginal means

Without going too much into the details, obtaining accurate p-values in mixed models requires sophisticated statistical techniques. The issue is in determining the correct denominator degrees of freedom for significance tests in mixed models. Unlike simpler models where the denominator degrees of freedom can be straightforwardly calculated, mixed models introduce ambiguity due to their structure and data hierarchies. With more complex, crossed, or unbalanced designs, the choice of denominator degrees of freedom becomes less clear. Should it be based on the number of observations, the number of subjects or items, the number of random effects, or some combination thereof? This uncertainty complicates the calculation of p-values, on top of that in (many) hierarchical models the cumulative distribution function of the test statistic when the null hypothesis is true is simply not known. Consequently, **lme4** package does not directly give us a p-value.

There are some approximations methods that can be used to find the degrees of freedom such as Satterthwaite and Kenward-Roger and use F-distribution to calculate p-values. We are going to use those here using the package **emmeans**. This package gives us estimated marginal means (EMMs) for different types of models including mixed models.

Before proceeding with the actual analysis i want to briefly tell you about marginal means. Marginal means are designed to calculate the average expected outcome on a dependent variable—take recovery time as an example—across varying levels of a factor, such as different treatment types, while accounting for the influence of other variables in the model. This adjustment is there to ensure that the marginal means reflect the isolated effect of each factor, free from confounding variables.

One of the most important aspects of EMMs is their ability to provide clear, comparable insights even in the face of unbalanced data. Unlike simpler analytical methods that may be biased when data lack symmetry across groups or conditions, EMMs adjust for these differences, offering an accurate representation of group effects. There are great guidelines on how to use `emmeans` here so please read them if you want to get more complex analysis done.

In our case however, we are interested in two comparisons. First we are going to compare the treatments and then the dosage effect.

```
library(emmeans)
```

Welcome to emmeans.

Caution: You lose important information if you filter this package's results. See '? untidy'

```
em_object <- emmeans(model_lmm_treatment_int_rnd, specs =
  ↪ "treatment", by = "dosage", lmer.df = "satterthwaite")
print(pairs(em_object))
```

Warning: Model failed to converge with 1 negative eigenvalue: -1.7e-03

Warning: Model failed to converge with 1 negative eigenvalue: -1.7e-03

Warning: Model failed to converge with 1 negative eigenvalue: -1.7e-03

dosage = 5.5:

	contrast	estimate	SE	df	t.ratio	p.value
A - B		77.3	0.259	3.28	298.533	<0.0001
A - C		19.2	13.000	2.04	1.476	0.4530
B - C		-58.1	13.100	2.04	-4.439	0.0819

Degrees-of-freedom method: satterthwaite

P value adjustment: tukey method for comparing a family of 3 estimates

In the code above the `emmeans` function is called with the fitted mixed model object (`model_lmm_treatment_int_rnd`) to estimate marginal means for the `treatment` factor. The `specs` argument specifies that we are focusing on the treatment effect. The `by = "dosage"` argument indicates that these comparisons will be made within levels or average of the dosage variable, allowing for an examination of how treatment effects change with different dosages. The `lmer.df = "satterthwaite"` argument specifies that the Satterthwaite approximation should be used to estimate the degrees of freedom for the tests, which is crucial for mixed models where the degrees of freedom are not straightforward.

The `pairs` function is used to compare every pair of treatment levels within average dosage level. This results in a table showing the contrast (difference) between each pair of treatments, the estimate of this difference, its standard error (SE), degrees of freedom (df), t-ratio, and the p-value for the significance of the difference.

For a dosage of 5.5, the results of the pairwise comparisons between treatments are as follows:

- **A vs. B:** The estimated difference in the response variable between treatments A and B is 77.3 units, with a very small standard error (SE =

0.259), leading to a highly significant p-value ($<.0001$). This suggests a strong evidence of a significant difference between treatments A and B at this dosage level.

- **A vs. C:** The difference between treatments A and C is 19.2 units, but with a much larger standard error ($SE = 13.003$) and a p-value of 0.4477. This indicates that the difference between treatments A and C at this dosage level is not statistically significant.
- **B vs. C:** The difference here is -58.1 units, with a standard error of 13.097. The p-value is 0.0758, which suggests that while there is a noticeable difference between treatments B and C, it does not reach the conventional threshold for statistical significance (usually $p < 0.05$).

In summary, at a dosage of 5.5, there is strong evidence that treatment A significantly differs from treatment B, with A being more effective or yielding higher response values. The comparisons between A and C, and between B and C, do not show statistically significant differences at the conventional level, although the trend suggests treatment A might be more effective than C, and B less effective than both A and C.

We can visualize this using `pwpp` function

```
pwpp(em_object)
```

Warning: Model failed to converge with 1 negative eigenvalue: -1.7e-03
 Warning: Model failed to converge with 1 negative eigenvalue: -1.7e-03
 Warning: Model failed to converge with 1 negative eigenvalue: -1.7e-03
 Warning: Model failed to converge with 1 negative eigenvalue: -1.7e-03
 Warning: Model failed to converge with 1 negative eigenvalue: -1.7e-03
 Warning: Model failed to converge with 1 negative eigenvalue: -1.7e-03

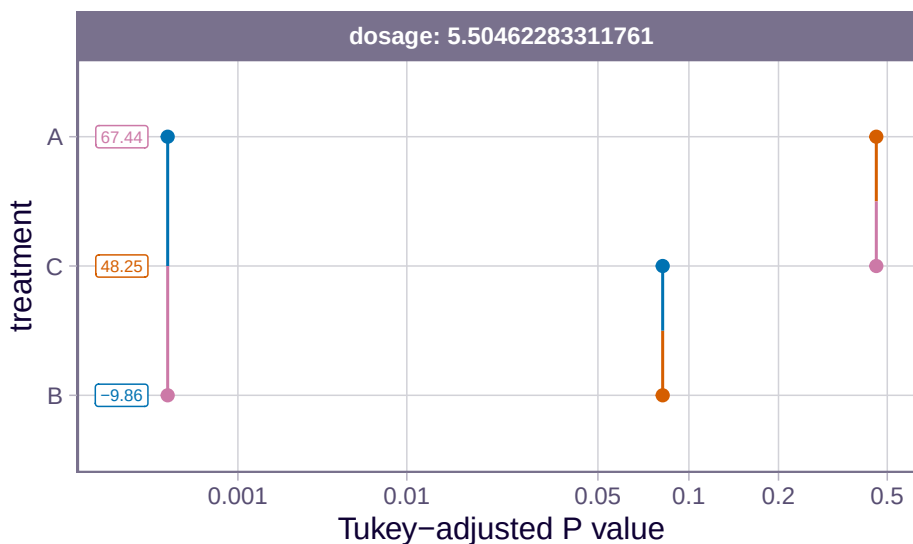


Figure 20.16: Example of Pairwise P-value plot

Each comparison is associated with a vertical line segment that joins the scale positions of the two EMMs being compared, and whose horizontal position is determined by the P value of that comparison.

The second comparison we are interested in is the effect of dosage between treatments. We can use `emtrends` for comparing numerical variables (slopes) across different groups:

```
em_object_trend <- emtrends(model_lmm_treatment_int_rnd, pairwise
  ~ treatment, var = "dosage", lmer.df = "satterthwaite")
print(em_object_trend)
```

```
Warning: Model failed to converge with 1 negative eigenvalue: -1.7e-03
Warning: Model failed to converge with 1 negative eigenvalue: -1.7e-03
Warning: Model failed to converge with 1 negative eigenvalue: -1.7e-03
Warning: Model failed to converge with 1 negative eigenvalue: -1.7e-03
Warning: Model failed to converge with 1 negative eigenvalue: -1.7e-03
Warning: Model failed to converge with 1 negative eigenvalue: -1.7e-03
```

\$emtrends

treatment	dosage.trend	SE	df	lower.CL	upper.CL
A	3.050	0.1100	2.13	2.60	3.50
B	-1.977	0.0871	2.61	-2.28	-1.67
C	0.408	0.6890	2.04	-2.50	3.31

Degrees-of-freedom method: satterthwaite

Confidence level used: 0.95

\$contrasts

contrast	estimate	SE	df	t.ratio	p.value
A - B	5.03	0.152	2.10	33.014	0.0004
A - C	2.64	0.769	2.05	3.435	0.1299
B - C	-2.38	0.714	2.04	-3.339	0.1378

Degrees-of-freedom method: satterthwaite

P value adjustment: tukey method for comparing a family of 3 estimates

The code above calculates and compares the slopes (rate of change) of the response variable with respect to dosage for each treatment group. By specifying `pairwise ~ treatment`, the code requests pairwise comparisons of these trends between treatments. The `var = "dosage"` argument indicates that the trend of interest is how the response variable changes with dosage. The output includes two parts: `emtrends`, which lists the slope (`dosage.trend`) for each treatment along with its standard error, degrees of freedom, and confidence interval, and `contrasts`, which presents the pairwise comparisons of these slopes between treatments, including estimates, standard errors, degrees of freedom, t-ratios, and p-values, adjusted using the Tukey method for multiple comparisons.

Dosage Trends per Treatment:

- **Treatment A:** Shows a positive trend of 3.050 with dosage, with a standard error (SE) of 0.110 and a 95% confidence interval (CI) from 2.59 to 3.51. This indicates that for treatment A, the response variable increases

by approximately 3.050 units with each unit increase in dosage, which is statistically significant and suggests a strong positive dosage effect.

- **Treatment B:** Displays a negative dosage trend of -1.977, with an SE of 0.087 and a 95% CI from -2.29 to -1.66. This suggests that for treatment B, the response variable decreases by about 1.977 units for each unit increase in dosage, indicating a significant negative effect of dosage on the response variable.
- **Treatment C:** Has a dosage trend of 0.408 with an SE of 0.689 and a 95% CI stretching from -2.36 to 3.18. The wide confidence interval and the proximity of the trend to zero suggest that the dosage effect for treatment C is not statistically significant, implying little to no clear effect of dosage on the response variable for this treatment.

Pairwise Comparisons of Dosage Trends:

- **A vs. B:** The estimated difference in dosage trends between treatments A and B is 5.03, with an SE of 0.152, resulting in a highly significant p-value of 0.0004. This indicates a substantial difference in how dosage affects the response variable between these two treatments, with A increasing and B decreasing in response to higher dosages.
- **A vs. C:** Shows an estimate of 2.64 with an SE of 0.769. Despite the positive difference, the p-value of 0.1234 suggests that this difference is not statistically significant at the conventional 0.05 level, though it indicates a trend towards a stronger positive effect of dosage for treatment A compared to C.
- **B vs. C:** The difference in dosage trends between treatments B and C is -2.38, with an SE of 0.714. The p-value of 0.1285, while not conventionally significant, suggests a moderate difference in the effect of dosage between these treatments, with B showing a negative trend and C showing an insignificant positive trend.

We can also use `pwpp` and `plot` function to visualize the results.

There is a lot more that can be done using `emmeans` package for example if we have two interacting factors instead of one. You should really check out the `emmeans` package homepage to learn more about how different analysis can be performed.

20.6.1 ANOVA type I, II, and III

Sometimes we are only interested in if a variable in the model is significant or not. This is normally simple when it comes to numerical variables but it can get a bit complicated with categorical variables, especially if they have multiple levels. ANOVA provides a type of analysis that allows us to compare the means of the different levels of a categorical variable simultaneously to determine if there is a statistically significant difference among them. This is particularly useful because it accounts for the variability within each group and between groups, enabling a comprehensive assessment of how the categorical variable, as a whole, influences the response variable.

There different types of ANOVA analysis but we are going to explain type 1 to 3 here.

Type I ANOVA, also known as sequential sum of squares (SS), tests each predictor's effect on the outcome variable in the order specified in the model. It's sensitive to the order of variables in the model because it evaluates the additional contribution of each variable after accounting for the variables entered before it. Type I ANOVA is most suitable for models where predictors have a natural hierarchical order, or for simple designs without interactions or with only one main effect. It's often used in purely observational studies where the order of variables can reflect the temporal or causal sequence of their introduction into the system. This is however, rarely used in practice in many applications so we are going to leave it here and proceed to Type II ANOVA.

Type II ANOVA examines the main effects of each factor after accounting for other main effects in the model but does not consider interactions involving the factor being assessed. It's less sensitive to the order of variables than Type I ANOVA. It is useful in factorial designs where interaction effects are **NOT** the main focus or when you want to assess the unique contribution of each main effect after accounting for other main effects.

Finally, **Type III ANOVA**, or marginal sum of squares, evaluates the effect of each predictor while accounting for all other predictors in the model, including interactions. This type does not depend on the order of variables and is considered the most general approach.

We can use `joint_tests` from `emmeans` package to perform type II and type III ANOVA:

Type II:

```
joint_tests(model_lmm_treatment_int_rnd, lmer.df =
  ↪ "satterthwaite") ## type 3
```

```
Warning: Model failed to converge with 1 negative eigenvalue: -1.7e-03
Warning: Model failed to converge with 1 negative eigenvalue: -1.7e-03
Warning: Model failed to converge with 1 negative eigenvalue: -1.7e-03
Warning: Model failed to converge with 1 negative eigenvalue: -1.7e-03
Warning: Model failed to converge with 1 negative eigenvalue: -1.7e-03
Warning: Model failed to converge with 1 negative eigenvalue: -1.7e-03
Warning: Model failed to converge with 1 negative eigenvalue: -1.7e-03
Warning: Model failed to converge with 1 negative eigenvalue: -1.7e-03
Warning: Model failed to converge with 1 negative eigenvalue: -1.7e-03
Warning: Model failed to converge with 1 negative eigenvalue: -1.7e-03
Warning: Model failed to converge with 1 negative eigenvalue: -1.7e-03
Warning: Model failed to converge with 1 negative eigenvalue: -1.7e-03
Warning: Model failed to converge with 1 negative eigenvalue: -1.7e-03
Warning: Model failed to converge with 1 negative eigenvalue: -1.7e-03
Warning: Model failed to converge with 1 negative eigenvalue: -1.7e-03
Warning: Model failed to converge with 1 negative eigenvalue: -1.7e-03
```

model term	df1	df2	F.ratio	p.value
treatment	2	2.04	51012.650	<0.0001
dosage	1	2.04	6.160	0.1284

```
treatment:dosage    2 2.05    624.940  0.0014
```

Type III:

```
joint_tests(model_lmm_treatment_int_rnd, lmer.df =
  ↪ "satterthwaite") ## type 3
```

```
Warning: Model failed to converge with 1 negative eigenvalue: -1.7e-03
Warning: Model failed to converge with 1 negative eigenvalue: -1.7e-03
Warning: Model failed to converge with 1 negative eigenvalue: -1.7e-03
Warning: Model failed to converge with 1 negative eigenvalue: -1.7e-03
Warning: Model failed to converge with 1 negative eigenvalue: -1.7e-03
Warning: Model failed to converge with 1 negative eigenvalue: -1.7e-03
Warning: Model failed to converge with 1 negative eigenvalue: -1.7e-03
Warning: Model failed to converge with 1 negative eigenvalue: -1.7e-03
Warning: Model failed to converge with 1 negative eigenvalue: -1.7e-03
Warning: Model failed to converge with 1 negative eigenvalue: -1.7e-03
Warning: Model failed to converge with 1 negative eigenvalue: -1.7e-03
Warning: Model failed to converge with 1 negative eigenvalue: -1.7e-03
Warning: Model failed to converge with 1 negative eigenvalue: -1.7e-03
Warning: Model failed to converge with 1 negative eigenvalue: -1.7e-03
Warning: Model failed to converge with 1 negative eigenvalue: -1.7e-03
Warning: Model failed to converge with 1 negative eigenvalue: -1.7e-03
```

model term	df1	df2	F.ratio	p.value
treatment	2	2.04	51012.650	<0.0001
dosage	1	2.04	6.160	0.1284
treatment:dosage	2	2.05	624.940	0.0014

The interpretation is the key here. in the type II model the assumption is that the interaction is tested first and is not significant; if interaction is significant, Type II is generally not recommended. ANOVA type II is more powerful than Type III when interactions are not significant due to its focus on main effects in isolation from interaction effects.

Please recall that Type III tests for each main effect after accounting for all other factors and interactions, making it robust in the presence of significant interactions. However, while Type III allows for the testing of main effects even when interactions are present, interpreting these main effects can be complex and potentially misleading due to the significance of interactions.

Generally speaking, if the interactions are not significant, type II gives a more powerful test but if an interaction is significant Type III can be used but it is not really interesting to interpret a main effect to start with.

20.7 Power calculation

Statistical power is defined as the probability of correctly rejecting the null hypothesis when it is false, or in other words, the likelihood that a study will detect an effect when there is an effect to be detected.

One way to calculate power is through data simulation. This process involves generating a large number of datasets that resemble the expected structure and

variation of your real-world data, under both the null hypothesis (where the effect you're testing for is absent) and the alternative hypothesis (where the effect is present). In the case of power analysis, we simulate data assuming the effect we're testing for does exist at a specific magnitude. This allows us to see how often we can expect to correctly detect the effect, given the true difference and our study design. The key to these simulations is the introduction of randomness in a controlled manner. By introducing randomness in this way, each simulated dataset gives us one possible outcome of our study if it were conducted under the exact same conditions. Running thousands of these simulations provides a robust overview of the range of possible outcomes we might expect. We can use `simulate` function to generate simulated data:

For speeding up the process we are going to use only 10 simulations. In reality this number must be much higher.

```
simple_model <- lmer(recoveryTime ~ treatment*dosage +
  ↪ (1+treatment*dosage|doctor),data_int_interaction)
```

boundary (singular) fit: see help('isSingular')

```
simulated_data<-simulate(object = simple_model,nsim = 10,seed =
  ↪ 10)
```

Each of the simulated recovery times is now in a column of `simulated_data`. We can fit a model for each of these new responses followed by statistical analysis.

```
tmp_data<-data_int_interaction
stat_information<-lapply(simulated_data,function(x){
  simple_model_refit <- refit(simple_model,x)
  em_obj <- emmeans(simple_model_refit,specs = "treatment",by =
  ↪ "dosage",lmer.df = "satterthwaite")
  data.frame(pairs(em_obj,adjust="none"))
})

cp_A_B<-sapply(stat_information,function(x){x[1,"p.value"]})
cp_A_C<-sapply(stat_information,function(x){x[2,"p.value"]})
cp_B_C<-sapply(stat_information,function(x){x[3,"p.value"]})
```

After extracting the p-values for each comparisons and for each simulation. We can define power as proportion of the p-values lower than our alpha level. We can even calculate confidence intervals etc:

```
cat("A - B power: ",mean(cp_A_B<0.05)*100,"%\n")
```

```
A - B power: 100 %
```

```
cat("A - B conf.level: ",binom.test(x = sum(cp_A_B<0.05), n =
  ↪ length(cp_A_B),
  conf.level = .95)$conf.int*100,"%\n")
```

```
A - B conf.level: 69.15029 100
```

```
cat("A - C power: ",mean(cp_A_C<0.05)*100,"%\n")
```

A - C power: 10 %

```
cat("A - C conf.level: ",binom.test(x = sum(cp_A_C<0.05), n =
↪ length(cp_A_C),
    conf.level = .95)$conf.int*100,"%\n")
```

A - C conf.level: 0.2528579 44.50161

```
cat("B - C power: ",mean(cp_B_C<0.05)*100,"%\n")
```

B - C power: 30 %

```
cat("B - C conf.level: ",binom.test(x = sum(cp_B_C<0.05), n =
↪ length(cp_B_C),
    conf.level = .95)$conf.int*100,"%\n")
```

B - C conf.level: 6.673951 65.24529

There are also some packages such as `simr` that can be used to perform power analysis based on simulation.

Chapter 21

Mixed models (mathematical details)

In this section we are going to explain some of the key mathematical properties of mixed model. Before we begin please note that as said before mixed models can become fairly complex, with a variety of parametrization techniques that go beyond the scope and time frame of this chapter. Here I assume that the reader has relatively a good understanding of Ordinary Least Squares regression (OLS) as we are going to use that to build mixed models.

Chapter 22

Ordinary Least Squares regression

Ordinary Least Squares (OLS) is a fundamental method for linear regression analysis. It aims to find the line (or hyperplane) that best fits a set of data points by minimizing the sum of the squared differences (residuals) between observed values and the values predicted by the linear model.

22.1 Mathematical Formulation

Consider a dataset with n observations and p predictors. The model can be expressed as:

$$y = X\beta + \epsilon$$

Where:

- y is an $n \times 1$ vector of observed values.
- X is an $n \times p$ matrix of predictor variables, often augmented with a column of ones to include the intercept in the model.
- β is a $p \times 1$ vector of coefficients that we want to estimate.
- ϵ is an $n \times 1$ vector of the model's residuals or errors.

The objective of OLS is to minimize the sum of squared residuals, given by:

$$S(\beta) = (y - X\beta)^\top (y - X\beta)$$

To find the value of β that minimizes $S(\beta)$, set the gradient of S with respect to β to zero. This yields the normal equations:

$$X^\top X\beta = X^\top y$$

Assuming $X^\top X$ is non-singular (invertible), the solution for β is:

$$\beta = (X^\top X)^{-1} X^\top y$$

This solution provides the best linear unbiased estimates of the coefficients under the Gauss-Markov theorem, provided the residuals ϵ are uncorrelated with constant variance (homoscedasticity) and mean zero.

OLS has some highly desired statistical properties:

- **Unbiasedness:** $E(\beta) = \beta$, meaning that the expected value of the OLS estimator equals the true parameter value.
- **Efficiency:** Under the assumptions of the Gauss-Markov theorem, the OLS estimator has the smallest variance among all linear unbiased estimators.
- **Consistency:** As the sample size n approaches infinity, the OLS estimator converges in probability to the true parameter value, assuming the predictors X are non-stochastic.

22.2 Correlated residuals

The classical OLS method is designed under the assumption that the residuals ϵ are uncorrelated and homoscedastic. However, when these assumptions are violated, particularly in the presence of correlated residuals, OLS estimates remain unbiased but lose efficiency, meaning they no longer have the smallest possible variance among all linear unbiased estimators.

Consider the linear model:

$$y = X\beta + \epsilon, \quad \text{where } \epsilon \sim N(0, \sigma^2 V)$$

Assuming that the residuals are not correlated, V is an identity (The diagonal entries are 1, the rest are zero). However, when the correlation exist we are dealing with a more complex covariance matrix V . The presence of correlated residuals necessitates a transformation of the model so that the assumptions of OLS are effectively restored. This transformation can be achieved through the use of a matrix C , such that when applied to the residuals, results in uncorrelated residuals with uniform variance.

So formally what we are interested in is:

$$\text{Cov}(C\epsilon) = \sigma^2 I$$

So let's start working with $\text{Cov}(C\epsilon)$. Please remember that the mean of the residuals are assumed to be zero. Therefore we can write the covariance equation as

$$\text{Cov}(C\epsilon) = C\epsilon(C\epsilon)^T$$

If we rewrite this and apply the transpose we end up with

$$\text{Cov}(C\epsilon) = C\epsilon\epsilon^T C^T$$

As we said the original covariance of ϵ is $\sigma^2 V$ so we can replace $\epsilon\epsilon^T$ by $\sigma^2 V$:

$$\text{Cov}(C\epsilon) = C\sigma^2VC^T = \sigma^2I$$

If we define both side by σ^2

$$CVC^T = I$$

We assume that C exhibits complex conjugate symmetry then

$$\text{Cov}(C\epsilon) = CVC$$

We take C to the other side of the equation:

$$CV = C^{-1}$$

We do the same thing for the other C :

$$V = C^{-1}C^{-1}$$

Multiply two C^{-1} :

$$V = C^{-2}$$

Take the inverse of both side

$$V^{-1} = C^2$$

and finally we take the square root of both side:

$$C = V^{-1/2}$$

So let's put this back in the original equation

$$\text{Cov}(C\epsilon) = C\sigma^2VC^T = V^{-1/2}\sigma^2VV^{-1/2} = \sigma^2I$$

Now that we have our transformation matrix $V^{-1/2}$, we should apply it to the OLS equation. In order to effectively “whitens” the errors we are going to premultiply OLS equation with the transformation $V^{-1/2}$:

$$V^{-1/2}y = V^{-1/2}X\beta + V^{-1/2}\epsilon$$

With the transformed model $V^{-1/2}y = V^{-1/2}X\beta + V^{-1/2}\epsilon$ and the transformed errors having covariance I , we can now apply OLS directly:

$$\beta_{\text{transformed OLS}} = ((V^{-1/2}X)^\top (V^{-1/2}X))^{-1} (V^{-1/2}X)^\top (V^{-1/2}y)$$

This is equivalent to estimating beta for Generalized Least Squares (GLS):

$$\beta_{\text{GLS}} = (X^\top V^{-1}X)^{-1}X^\top V^{-1}y$$

with the following minimization problem:

$$\min_{\beta} (y - X\beta)^{\top} V^{-1} (y - X\beta)$$

We can also try to verify our derivation of beta by expanding the quadratic form:
 $(y - X\beta)^{\top} V^{-1} (y - X\beta) = y^{\top} V^{-1} y - \beta^{\top} X^{\top} V^{-1} y - y^{\top} V^{-1} X\beta + \beta^{\top} X^{\top} V^{-1} X\beta$

Taking the derivative of this expression with respect to β and setting it to zero gives:

$$-2X^{\top} V^{-1} y + 2X^{\top} V^{-1} X\beta = 0$$

$$X^{\top} V^{-1} X\beta = X^{\top} V^{-1} y$$

Solving for β yields:

$$\beta = (X^{\top} V^{-1} X)^{-1} X^{\top} V^{-1} y$$

This is again the GLS estimator, confirming that it minimizes the quadratic form.

Chapter 23

Generalized Least Squares (GLS)

Generalized Least Squares (GLS) is a statistical technique used to estimate the unknown parameters in a linear regression model when the error terms exhibit heteroscedasticity (varying variance) or are correlated. Unlike ordinary least squares (OLS) which assumes that the error terms are homoscedastic (constant variance) and uncorrelated, GLS adjusts for these irregularities by pre-multiplying the model by the inverse square root of the covariance matrix of the errors, $V^{-1/2}$, or directly incorporating the inverse of the covariance matrix, V^{-1} , into the estimation process. This transformation normalizes the errors, leading to more efficient and unbiased estimates even in the presence of non-ideal error structures. GLS is particularly useful in time series analysis, panel data models, and other contexts where observations are not independent.

23.1 Example

Before going forward we want to try whether the equation we derived actually work on not.

Let's get back to our previous example:

```
simulate_grouped_trend <- function(group_count = 5,
  ↪ points_per_group = 10, global_slope = -10, global_intercept =
  ↪ 30, group_slope = 2, noise_sd = 50, noise_sd2=2) {
  set.seed(123) # Setting a seed for reproducibility

  # Initialize an empty data frame to store the simulated data
  data <- data.frame(x = numeric(), y = numeric())

  # Loop to create each group
  for (i in 1:group_count) {
    x_start <- 12 + (i - 1) * (10 / group_count) # Stagger the
    ↪ start of x for each group
```

```

x <- runif(points_per_group, min = x_start, max = x_start +
↪ (10 / group_count))

# Apply a local positive trend within the group, but maintain
↪ the global negative trend
local_intercept <- global_intercept + global_slope * (x_start
↪ + (10 / (2 * group_count))) + rnorm(1, mean = 0, sd =
↪ noise_sd)
y <- local_intercept + group_slope[i] * (x - x_start) +
↪ rnorm(points_per_group, mean = 0, sd = noise_sd2)

# Combine this group with the overall dataset
group_data <- data.frame(x = x, y = y, group=i)
data <- rbind(data, group_data)
}

return(data)
}

# generate simulated data
data_int <- simulate_grouped_trend(group_count =
↪ 4, points_per_group = 10, global_slope = -2, global_intercept =
↪ 100, group_slope = c(6, 4, 2, 1), noise_sd = 5, noise_sd2=2)

# set group to factor
data_int$group <- factor(data_int$group)
# plot the data
plot(data_int$x, data_int$y, xlab="Age", ylab="Protein
↪ expression", col=data_int$group, pch=as.numeric(data_int$group))

```

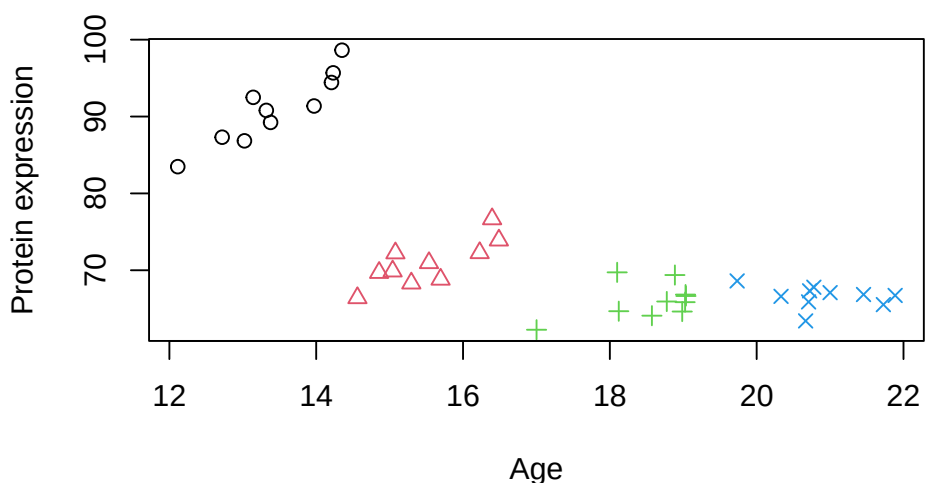


Figure 23.1: A scatter plot of Age vs. Protein expression

Let's say that we know the structure of V using some prior knowledge (we will cover how V is exactly derived later).

```
# Create a diagonal matrix 'var.d' with equal diagonal elements
↪ set to 378.6062 (covariance). The dimensions are based on the
↪ number of levels in 'data_int$group'.
var.d <- diag(378.6062, ncol = length(levels(data_int$group)),
↪ nrow=length(levels(data_int$group)))

# Transpose of the model matrix for the 'group' variable in the
↪ dataset 'data_int', excluding intercept (hence '~0+').
Zt <- t(model.matrix(~0+data_int$group))

# Compute the variance-covariance matrix 'V' for the random
↪ effects using the matrix 'Zt' and the variance diagonal
↪ matrix 'var.d'.
V <- (t(Zt) %*% var.d %*% Zt)

# Set the diagonal elements of the variance-covariance matrix 'V'
↪ to 384.5376 (total variance).
diag(V) <- 384.5376

# Construct the model matrix 'X' for the fixed effect 'x' using
↪ data from 'data_int'.
X <- model.matrix(~x, data=data_int)

# Extract the response variable 'y' from 'data_int'.
y <- data_int$y

# Compute the mixed model solution using generalized least
↪ squares, solving the equation  $(X'V^{-1}X)^{-1}X'V^{-1}y$  for the
↪ fixed effects estimates.
print(solve(t(X) %*% solve(V) %*% X) %*% t(X) %*% solve(V) %*% y)
```

```
      [,1]
(Intercept) 29.910468
x           2.556151
```

Let's compare this to when we use mixed model:

```
library(lme4)
model_lmm <- lmer(y ~ 1+x+(1|group), data = data_int)
print(model_lmm)
```

```
Linear mixed model fit by REML ['lmerMod']
Formula: y ~ 1 + x + (1 | group)
Data: data_int
REML criterion at convergence: 201.3982
Random effects:
Groups   Name             Std.Dev.
group    (Intercept) 19.458
```

```

Residual                2.435
Number of obs: 40, groups: group, 4
Fixed Effects:
(Intercept)              x
                29.911      2.556

```

We have calculated the exact same coefficients using both approaches. We can now see how V changes our data using $V^{-1/2}y = V^{-1/2}X\beta + V^{-1/2}\epsilon$. But before that we need to cover another small algebraic operation *Square root of a matrix* or specifically how to calculate $V^{-1/2}$.

Assuming V is a symmetric matrix, it can be decomposed into:

$$V = P\Lambda P^T$$

where:

- P is an orthogonal matrix whose columns are the eigenvectors of V .
- Λ is a diagonal matrix whose diagonal elements are the eigenvalues of V , denoted as λ_i (i.e., $\Lambda = \text{diag}(\lambda_1, \lambda_2, \dots, \lambda_n)$).

For V to have a real square root, all its eigenvalues should be non-negative ($\lambda_i \geq 0$ for all i). This is because the square root of a negative number is not defined in the real numbers, which would lead to complex values.

The square root of the diagonal matrix Λ is straightforward to compute. The square root of a diagonal matrix is another diagonal matrix $\Lambda^{1/2}$ with diagonal entries equal to the square roots of the diagonal entries of Λ . Thus:

$$\Lambda^{1/2} = \text{diag}(\sqrt{\lambda_1}, \sqrt{\lambda_2}, \dots, \sqrt{\lambda_n})$$

The square root of the matrix V , denoted $V^{1/2}$, can now be constructed using the eigenvectors and the square root of the eigenvalues:

$$V^{1/2} = P\Lambda^{1/2}P^T$$

This expression utilizes the original matrix of eigenvectors P , the square root of the eigenvalue matrix $\Lambda^{1/2}$, and the transpose of P .

To verify that $V^{1/2}$ is the square root of V , compute $V^{1/2}V^{1/2}$ and check that it equals V :

$$V^{1/2}V^{1/2} = (P\Lambda^{1/2}P^T)(P\Lambda^{1/2}P^T)$$

Using the orthogonality of P (i.e., $P^TP = I$ where I is the identity matrix):

$$V^{1/2}V^{1/2} = P\Lambda^{1/2}(P^TP)\Lambda^{1/2}P^T = P\Lambda^{1/2}\Lambda^{1/2}P^T = P\Lambda P^T = V$$

So, $V^{1/2}$ is the square root of V .

Let's create a small function in R to calculate the square root of a matrix:

```

matrix_sqrt <- function(Sigma) {
  # Ensure the matrix is symmetric
  if (!all.equal(Sigma, t(Sigma))) {
    stop("The input matrix must be symmetric.")
  }
}

```

```

}

# Compute the eigenvalues and eigenvectors
eigen_decomp <- eigen(Sigma)

# Extract eigenvalues and eigenvectors
eigenvalues <- eigen_decomp$values
eigenvectors <- eigen_decomp$vectors

# Check for negative eigenvalues
if (any(eigenvalues < 0)) {
  stop("The matrix must be positive definite.")
}

# Compute Lambda^1/2
Lambda_half <- diag(1/sqrt(eigenvalues))

# Compute Sigma^1/2
Sigma_half <- eigenvectors %%% Lambda_half %%% t(eigenvectors)

↪ return(list(Sigma_half=Sigma_half,eigenvectors=eigenvectors,eigenvalues=eigenvalues))
}

```

Now what we are going to do is to transform X (the design matrix) and y (our response) using the $V^{1/2}$ and only perform OLS on it:

```

V_inv_sqrt <- matrix_sqrt(V)$Sigma_half
X_transformed <- V_inv_sqrt%%X
y_transformed <- V_inv_sqrt%%y

print(lm(y_transformed~0+X_transformed))

```

Call:

```
lm(formula = y_transformed ~ 0 + X_transformed)
```

Coefficients:

X_transformed(Intercept)	X_transformedx
29.910	2.556

You see that this again gave us the exact same coefficients as mixed model.

Let's look at the original data and compare it with the transformed one:

```

par(mfrow=c(1,2))
plot(X[,2],y,col=data_int$group,main="Original data")
abline(model_lmm@beta)
plot(X_transformed[,2],y_transformed,col=data_int$group,main="Transformed
↪ data")
lines(x=X_transformed[,2],y=predict(lm(y_transformed~0+X_transformed)))

```

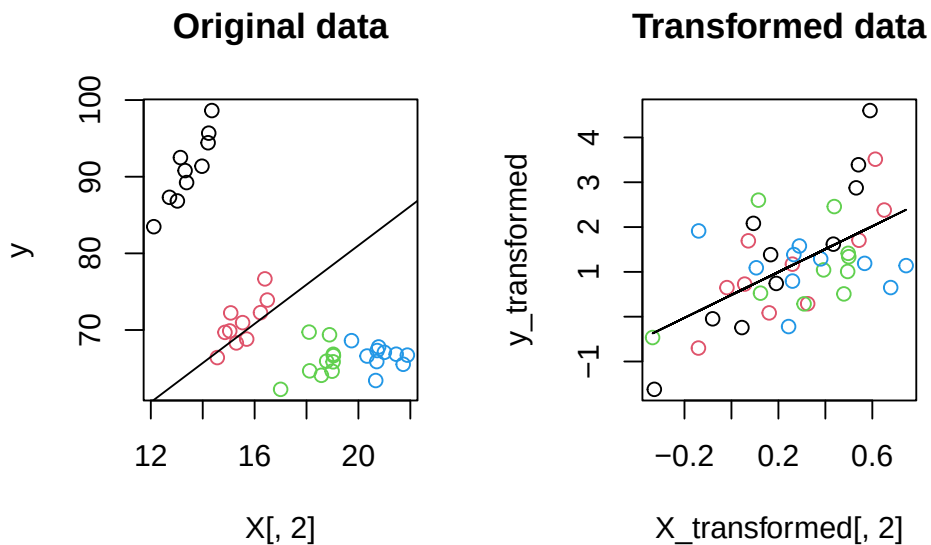


Figure 23.2: A scatter plot of of transformed X and Y

It is clear that the difference between groups has been removed in the transformed data. We can appreciate this procedure more if we try to increase the variance between the groups and see what happens to the transformation:

```
# fit the model
library(ggplot2)
library(gganimate)
library(dplyr)
library(tidyr)
library(mvtnorm)

global_intercept_slop <- model_lmm@beta
max_cov <- 378.6062
total_variance <- 384.5376

# Animation Data Preparation
results <- list()
for (co in seq(0, max_cov, length.out = 20)) {
  var.d <- diag(co, ncol = length(levels(data_int$group)), nrow =
    ↪ length(levels(data_int$group)))
  Zt <- t(model.matrix(~0 + data_int$group))
  V <- (t(Zt) %*% var.d %*% Zt)
  diag(V) <- total_variance
  V_inv_sqrt <- matrix_sqrt(V)$Sigma_half

  X <- model.matrix(~x, data = data_int)
  y <- data_int$y

  X_transformed <- V_inv_sqrt %*% X
  y_transformed <- V_inv_sqrt %*% y
}
```



```

transformed_data <- data.frame(X_transformed = X_transformed[,
↪ 2], y_transformed, Group = data_int$group, co = co)
results[[length(results) + 1]] <- transformed_data
}

# Combine all data frames for animation
animation_data <- do.call(rbind, results)

# Create the animation using ggplot2 and gganimate
ggplot(animation_data, aes(x = X_transformed, y = y_transformed,
↪ color = Group)) +
  geom_point()+ stat_ellipse(aes(x=X_transformed,
↪ y=y_transformed,color=Group),type = "norm")+
  labs(title = "Variance scaling factor: {closest_state}", x =
↪ "Transformed X", y = "Transformed Y") +
  transition_states(co, transition_length = 2, state_length = 1)
↪ +
  ease_aes('linear')

```

By now it should be clear that as the variance across the group is low that transformation does basically nothing (because there is nothing to correct for) but as we increase this variance we keep rescaling and stretching the data so that they are aligned and therefore the differences between the groups are minimized.

So this transformation ensures that each dimension (principal component) of the space now has equal variance (specifically, variance of 1), removing the effect of the original variances encoded by V . If the components of the data were correlated (as indicated by the off-diagonal elements of V), the transformation effectively decorrelates these components. In the new space, the covariance between any pair of different components of $V^{-1/2}data$ is zero. Thus, geometrically, $V^{-1/2}data$ transforms the data into a form where the scale differences and correlations introduced by V are neutralized, making the transformed data suitable for analysis under standard assumptions of independence and uniform scale.

The last thing to note here is that here we are dealing with a random intercept model, the covariance matrix V used for the transformation typically involves scaling and centering the group data without any rotation. This is because the model structure we are using doesn't incorporate random slopes or other types of covariance structures that might induce rotation. Instead, it predominantly influences the scaling and translation of the groups in our dataset.

In a random intercept model, each group has its own intercept but shares the slope (if any), leading to vertical shifts in the data but maintaining the same orientation (no rotation) regarding the predictor axis. So geometrically:

1. **Scaling:** As we increase the variance parameter (co in our model), the amount of scaling increases. This scaling affects how tightly or loosely the data points within each group are clustered around their group mean.
2. **Centering:** The transformation tends to draw group data towards a cen-

tral axis or point because each group's variance contributes to pulling the group's mean towards the overall mean of the dataset.

In terms of matrix operations: - The matrix V in this case is primarily influencing the variance within each group and does not introduce any off-diagonal terms that would typically cause a rotation. Rotation in the data space would occur if there were covariances between different dimensions (such as would be introduced with random slopes), which would manifest as off-diagonal elements in V .

In order to truly see what the transformation does, we now incorporate a random slope and show it affects the transformation here.

```
# fit the model
model_lmm <- lmer(y ~ 1+x+(1+x|group), data = data_int)

# custome function to extract the V matrix
getV1 <-
  ↪ function(x,var1=344.65503,var2=5.141458,cov=-39.26625,times=4){
    Zt <- getME(x, "Zt")
    vr <- sigma(x)^2
    original_matrix <- matrix(c(var1, cov, cov, var2), nrow=2,
    ↪ byrow=TRUE)
    var.d <- bdiag(rep(list(original_matrix),times))
    var.b <- (t(Zt) %*% var.d %*% Zt)
    sI <- vr * Matrix::Diagonal(nobs(x)) #for a sparse matrix
    var.y <- var.b + sI
  }

# Animation Data Preparation
results <- list()

varr1<-seq(0, 344.65503, length.out = 20)
varr2<-seq(0, 5.141458, length.out = 20)
corr<-seq(0, -39.26625, length.out = 20)
for (ii in 1:20) {
  V<-getV1(x = model_lmm,var2 = varr2[ii],cov = corr[ii],var1 =
  ↪ varr1[20])

  V_inv_sqrt <- matrix_sqrt(V)$Sigma_half

  X <- model.matrix(~x, data = data_int)
  y <- data_int$y

  X_transformed <- V_inv_sqrt %*% X
  y_transformed <- V_inv_sqrt %*% y
  transformed_data <- data.frame(X_transformed = X_transformed[,
  ↪ 2], y_transformed, Group = data_int$group, co = varr2[ii])
  results[[length(results) + 1]] <- transformed_data
}
```

```

V<-getV1(x = model_lmm,var2 = varr2[20],cov = corr[20],var1 =
↪ varr1[20])

V_inv_sqrt <- matrix_sqrt(V)$Sigma_half

X <- model.matrix(~x, data = data_int)
y <- data_int$y

X_transformed <- V_inv_sqrt %*% X
y_transformed <- V_inv_sqrt %*% y
transformed_data <- data.frame(X_transformed = X_transformed[,
↪ 2], y_transformed, Group = data_int$group, co = varr2[ii])
# Combine all data frames for animation
animation_data <- do.call(rbind, results)

ggplot(animation_data, aes(x = X_transformed, y = y_transformed,
↪ color = Group)) +
  geom_point()+ stat_ellipse(aes(x=X_transformed,
↪ y=y_transformed,color=Group),type = "norm")+
  labs(title = "Variance of slope scaling factor:
↪ {closest_state}", x = "Transformed X", y = "Transformed Y")
↪ +
  transition_states(co, transition_length = 1, state_length = 6)
↪ +
  ease_aes('linear')+geom_smooth(method = "lm",se = F)

```

In the plot above, i have already applied the random intercept transformation to make it a bit easier to see the rotation.

The slope transformation basically draws the overall slope towards the global one and aligns different groups.

So to summarize V in GLS transforms the data into a form where the scale differences and correlations introduced by grouping structure (or any other known structure) are neutralized, making the transformed data suitable for analysis under standard assumptions of independence and uniform scale.

23.2 Structure of V

I want to remind you here is that the purpose of matrix V is to encode the correlation structure within each group in the data. Given this, as we said before, in an OLS model, it is typically assumed that the residuals are independently and identically distributed (i.i.d.) with a variance of σ^2 . This again implies that the covariance matrix of the residuals is:

$$\text{Cov}(\epsilon) = \sigma^2 I$$

where $\mathbf{I}$ is the identity matrix, indicating no correlation between residuals of different observations.

Suppose we now believe that there is an underlying group structure that impacts the variance of the observations. This is a very important statement. If we don't believe or know that there is a grouping structure in our experiment as prior knowledge, then our model has the total residual variance σ^2 . In this context even if the true covariance matrix of the residuals is not diagonal, we will have no way of addressing this simply because we don't know what grouping structure has caused the dependencies between the residuals. So from now on, we are going to assume that we know there is a grouping in our data so that the difference between these groups whether it is on the slope or intercept adds an additional variance component. For simplicity we will be only talking about random intercept from now but the concept will apply to any sort of structure we decide to add to the model.

So let's say that we think there is a grouping structure where each group has its specific baseline level of y . The differences between these baselines adds an additional variance component σ_d^2 that reflects the variability across the groups beyond what is captured by the base residual variance σ^2 . As a result our total variance is now $\sigma_d^2 + \sigma^2$. In this case, σ^2 is now the variance of residuals after accounting for grouping structure and σ_d^2 is the variance of between the groups.

It should be obvious to see that the relation between σ_d^2 and σ^2 should effectively reflect how tightly the data points are clustered within each group. If the variance within groups (i.e., σ^2) is relatively small compared to the variance between groups (i.e., σ_d^2), it indicates that the groups are distinctly different from each other, and that the grouping structure is a significant factor in explaining the overall variability in the data. This scenario would suggest strong clustering within groups, meaning that the observations within each group are similar to each other but differ significantly from observations in other groups. Conversely, if σ^2 is large relative to σ_d^2 , it suggests that the within-group differences are significant, and the grouping structure might not be as influential in explaining the variance in the data.

This relationship helps us define intra-class correlation coefficient (ICC).

$$\text{ICC} = \frac{\sigma_d^2}{\sigma_d^2 + \sigma^2}$$

In our case ICC describes the degree to which units within the same group resemble each other. It is considered a type of correlation, but it applies to data organized into groups rather than to data consisting of paired observations.

The ICC formula calculates the proportion of total variance ($\sigma_d^2 + \sigma^2$) that is due to differences between groups. It essentially measures the ratio of between-group variance to the total variance in the data.

- If ICC is close to 1, it indicates that most of the variation in the data is due to differences between groups (high between-group variability).
- If ICC is close to 0, it indicates that most of the variation in the data is due to differences within group (high within-group variability or noise).

We can clarify this using a simple example:

```

#| code-fold: true
#| label: fig-scatter-plot-ICC
#| fig-cap: "A scatter plot of of X and Y showing the effect of
↪ ICC"
# fit the model
library(ggplot2)
library(gganimate)
simulate_data <- function(n_groups, n_per_group, slope, icc,
↪ sigma_within) {
  # Parameters
  mu <- 0 # Overall intercept (can be set to other values as
↪ needed)

  # Compute the variance components
  sigma_between <- icc * sigma_within / (1 - icc)

  # Simulate random effects for each group (intercepts)
  group_effects <- rnorm(n = n_groups, mean = mu, sd =
↪ sqrt(sigma_between))

  # Prepare data frame
  data <- data.frame(
    group = rep(1:n_groups, each = n_per_group),
    x = rep(runif(n = n_per_group, min = 0, max = 10), times =
↪ n_groups)
  )

  # Add the fixed effect (slope) and random effects
  data$y <- slope * data$x + rep(group_effects, each =
↪ n_per_group) +
    rnorm(n = n_groups * n_per_group, mean = 0, sd =
↪ sqrt(sigma_within))

  return(data)
}

results<-list()

data_int <- simulate_data(4,10,10,0.1,100)
ICCs<-seq(0.01, 0.99, length.out = 20)
for (ICC in seq(0.01, 0.99, length.out = 20)) {
  set.seed(123)
  data_int <- simulate_data(4,10,10,ICC,100)
  transformed_data <- cbind(data_int,ICC=ICC)
  results[[length(results) + 1]] <- transformed_data
}

```

```

animation_data <- do.call(rbind, results)
animation_data$group<-as.factor(animation_data$group)
ggplot(animation_data, aes(x = x, y = y, color = group)) +
  geom_point()+ stat_ellipse(aes(x=x, y=y,color=group),type =
    ↪ "norm")+
  labs(title = "ICC: {closest_state}", x = "X", y = "Y") +
  transition_states(ICC, transition_length = 1, state_length = 6)
    ↪ +
  ease_aes('linear')

```

Warning: No renderer available. Please install the gifski, av, or magick package to create animated output

It should be clear that as ICC goes up, the data points starts forming more distinct clusters thus increasing the correlation. We have now a method to measure the correlation within a cluster of data points. We can now proceed with defining the structure of V .

We start with the diagonal elements. The diagonal elements must reflect the total variance for each observation within the group, which includes both the baseline residual variance and the additional variance due to the group effect. Thus:

$$\text{Diagonal elements} = \sigma^2 + \sigma_d^2$$

The off-diagonal elements are a little bit more involved. Remember that, off-diagonal elements show the **covariance** between data points within each group.

We have so far calculated the correlation:

$$\text{ICC} = \rho = \frac{\sigma_d^2}{\sigma_d^2 + \sigma^2}$$

Recal that $\rho = \frac{\text{Cov}(X,Y)}{\sigma_X \sigma_Y}$. We can use this to transform ICC into covariance:

$$\text{Cov}(W_{ij}, W_{ik}) = \rho \times \sigma_{W_{ij}} \times \sigma_{W_{ik}}$$

Also recall that each observation within a group shares the same variance components therefore $\sigma_{W_{ij}} = \sigma_{W_{ik}}$. Our total variance was $\sqrt{\sigma^2 + \sigma_d^2}$ so

$$\sigma_{W_{ij}} = \sigma_{W_{ik}} = \sqrt{\sigma^2 + \sigma_d^2}$$

Substitute the expressions for ρ and $\sigma_{W_{ij}}$ into the covariance formula:

$$\text{Cov}(W_{ij}, W_{ik}) = \frac{\sigma_d^2}{\sigma^2 + \sigma_d^2} \times (\sigma^2 + \sigma_d^2)$$

$$\text{Cov}(W_{ij}, W_{ik}) = \sigma_d^2$$

Here, the factor $\sigma^2 + \sigma_d^2$ in the denominator and the standard deviations cancel out, leaving σ_d^2 .

Given the calculations and the framework we've discussed, we can now construct the covariance matrix V for our generalized least squares model.

In the context of GLS where we recognize the presence of group effects, the covariance matrix V takes on a block diagonal form. Each block corresponds to a group of data points that share a common variance structure due to their grouping. This form is important as it reflects the within-group correlations while assuming independence between groups.

The matrix V is composed of blocks V_i along the diagonal, where each block V_i is an $n_i \times n_i$ matrix for group i (assuming n_i observations per group). Each block V_i can be described as follows:

$$V_i = \begin{bmatrix} \sigma^2 + \sigma_d^2 & \sigma_d^2 & \cdots & \sigma_d^2 \\ \sigma_d^2 & \sigma^2 + \sigma_d^2 & \cdots & \sigma_d^2 \\ \vdots & \vdots & \ddots & \vdots \\ \sigma_d^2 & \sigma_d^2 & \cdots & \sigma^2 + \sigma_d^2 \end{bmatrix}$$

Or more compactly as

$$V_i = \sigma^2 \mathbf{I} + \sigma_d^2 \mathbf{1}\mathbf{1}'$$

and factor out σ^2 it gives us a general form of

$$\sigma^2 \left(\mathbf{I} + \frac{\sigma_d^2}{\sigma^2} \mathbf{1}\mathbf{1}' \right)$$

Giving us the form of

$$\sigma^2 V_i = \begin{bmatrix} 1 + \sigma_d^2/\sigma^2 & \sigma_d^2/\sigma^2 & \cdots & \sigma_d^2/\sigma^2 \\ \sigma_d^2/\sigma^2 & 1 + \sigma_d^2/\sigma^2 & \cdots & \sigma_d^2/\sigma^2 \\ \vdots & \vdots & \ddots & \vdots \\ \sigma_d^2/\sigma^2 & \sigma_d^2/\sigma^2 & \cdots & 1 + \sigma_d^2/\sigma^2 \end{bmatrix}$$

Each diagonal element of V_i is $\sigma^2 + \sigma_d^2$, representing the total variance for each observation within the group. This includes both the baseline residual variance (σ^2) and the additional variance σ_d^2 due to the group effect. Each off-diagonal element is σ_d^2 , which we derived from the intra-class correlation ρ . These elements represent the covariance between any two different observations within the same group, quantifying how much observations within a group are expected to covary due to the shared group effect.

It is important to note that again V has the block diagonal form in which each block is V_i . The block diagonal form of V is appropriate because it reflects that: - Observations within the same group are correlated due to shared group effects, as captured by the off-diagonal elements. - Observations from different groups are assumed to be independent of each other, justifying the separation into different blocks for each group.

We will now finish construction of V based on V_i . Please note that the structure of V_i ($\sigma^2(\mathbf{I} + \frac{\sigma_d^2}{\sigma^2} \mathbf{1}\mathbf{1}')$) seems to be identical for any i . This is actually true for our simple cases where we have only a single varying baseline (*intercept*) between

the groups. In this case we can manually count how many data points are in each group and just create a matrix that looks like this:

$$V = \begin{bmatrix} V_1 & 0 & 0 \\ 0 & \ddots & 0 \\ 0 & 0 & V_n \end{bmatrix}$$

where n is the number of groups. We practically just copy and paste $\sigma^2(\mathbf{I} + \frac{\sigma_d^2}{\sigma^2} \mathbf{1}\mathbf{1}')$ in each diagonal entire. This however is not an elegant solution. If we have more complex grouping structure with many components, we will have a hard time doing that manually. Here we can use design matrix. You should be familiar with a design matrix but I will try to give a very short overview.

23.2.1 Design matrix

The design matrix, Z , specifies how grouping and effects are associated with observations. Essentially, Z allocates the influence of effects to the observational data.

Each row of Z corresponds to an observation in the dataset. Each column of Z represents an effect (like intercept or slope). The number of columns in Z is equal to the number of effects in the model. The elements of Z indicate the degree to which a given effect influences a particular observation. These elements can be binary (0 or 1) in simple cases or continuous values representing the strength or magnitude of the effect.

To illustrate the use of Z more concretely, let's consider a couple of examples from typical applications:

Suppose we have a study measuring blood pressure in patients from different clinics, where each clinic is considered a group. If we model the intercept for each clinic, indicating that each clinic might inherently have higher or lower blood pressure readings:

Each row corresponds to a patient's measurement, and there is one column for each clinic. Each element in Z is: - 1 if the observation (patient) belongs to the clinic (group) - 0 otherwise

Form of Z :

If there are three clinics and six patients (two in each clinic), Z might look like:

$$Z = \begin{bmatrix} 1 & 0 & 0 \\ 1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \\ 0 & 0 & 1 \end{bmatrix}$$

Each column represents a clinic, and each row assigns a patient to their respective clinic.

For multiple individuals, Z would block these vectors accordingly, potentially stacking them if each has a separate slope effect.

If we had model both intercept and slope (let's say age) together we might end up having:

$$Z = \begin{bmatrix} 1 & 30 & 0 & 0 & 0 & 0 \\ 1 & 35 & 0 & 0 & 0 & 0 \\ 0 & 0 & 1 & 20 & 0 & 0 \\ 0 & 0 & 1 & 21 & 0 & 0 \\ 0 & 0 & 0 & 0 & 1 & 43 \\ 0 & 0 & 0 & 0 & 1 & 53 \end{bmatrix}$$

The design matrix Z is tailored to the structure of the study and the specific effects being modeled. Here we are going to automate building V using design matrix.

Please Note that Z has also a block diagonal form. Each block Z_i continues the matrix format by isolating specific subgroups or experimental conditions, which allows for different intercepts and slopes to be applied to separate segments of the data. This design is particularly useful for mixed-effects models where we might want to estimate both fixed effects (common to all groups) and random effects (varying by group or individual).

For example if we consider the matrix V :

$$Z = \begin{bmatrix} 1 & 0 & 0 \\ 1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \\ 0 & 0 & 1 \end{bmatrix}$$

The structure suggests that we are dealing with three distinct groups. Here's how each block Z_i in the block diagonal matrix Z is formed and its role in the model:

- **Block Z_i :**

$$Z_i = \begin{bmatrix} 1 \\ 1 \end{bmatrix}$$

This block corresponds to the group i . In the second example where Z is:

$$Z = \begin{bmatrix} 1 & 30 & 0 & 0 & 0 & 0 \\ 1 & 35 & 0 & 0 & 0 & 0 \\ 0 & 0 & 1 & 20 & 0 & 0 \\ 0 & 0 & 1 & 21 & 0 & 0 \\ 0 & 0 & 0 & 0 & 1 & 43 \\ 0 & 0 & 0 & 0 & 1 & 53 \end{bmatrix}$$

- **First Block Z_1 :**

$$Z_1 = \begin{bmatrix} 1 & 30 \\ 1 & 35 \end{bmatrix}$$

This block corresponds to the first group. The first column of ones represents the intercept, and the second column represents the slope with respect to the covariate ‘age’. This group could represent a specific demographic or experimental condition.

- **Second Block Z_2 :**

$$Z_2 = \begin{bmatrix} 1 & 20 \\ 1 & 21 \end{bmatrix}$$

Similar to the first, this block represents another group but with different age values. The structure again includes an intercept and a slope component, indicating a similar analysis but for a different subset of the population or a different experimental condition.

- **Third Block Z_3 :**

$$Z_3 = \begin{bmatrix} 1 & 43 \\ 1 & 53 \end{bmatrix}$$

This block deals with yet another group, possibly older individuals based on the age values shown. The model structure for this group is consistent with the others, providing the ability to compare across groups while accounting for within-group variations.

23.3 Structure of V (cont.)

Given that we now know about the design matrix, we can go ahead and reformulate the structure of V based on the design matrix. Please again assume that we are only dealing with intercept here so our design matrix looks like

$$Z = \begin{bmatrix} 1 & 0 & 0 \\ 1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \\ 0 & 0 & 1 \end{bmatrix}$$

and Z_i :

$$Z_i = \begin{bmatrix} 1 \\ 1 \end{bmatrix}$$

If you remember the structure of $V_i = \sigma^2(\mathbf{I} + \frac{\sigma_d^2}{\sigma^2}\mathbf{1}\mathbf{1}')$.

If Z_i is only a column matrix of 1 then $Z_i Z_i^T = \mathbf{1}\mathbf{1}'$. Let's define matrix D as a block diagonal matrix where each block, D_i , represents the variance components associated with the intercepts and possibly slopes for each group. Given the structure of $V_i = \sigma^2(\mathbf{I} + \frac{\sigma_d^2}{\sigma^2}\mathbf{1}\mathbf{1}')$, and assuming Z_i is structured to capture different types of effects (like intercept and slope as previously mentioned), the form of D and each D_i needs to reflect the appropriate variance scaling for these effects.

The matrix D is structured as:

$$D = \begin{bmatrix} D_1 & 0 & \cdots & 0 \\ 0 & D_2 & \cdots & 0 \\ \vdots & \vdots & \ddots & \vdots \\ 0 & 0 & \cdots & D_N \end{bmatrix}$$

where each block D_i is specific to a group and contains the variance components for the intercept and slope within that group. Assuming a simple case where each group has one intercept each Z_i block consists of one column for intercept.

$$D_i = \begin{bmatrix} \frac{\sigma_{\text{int},i}^2}{\sigma^2} \end{bmatrix}$$

Therefore

$$D = \begin{bmatrix} \frac{\sigma_{\text{int},1}^2}{\sigma^2} & 0 & \cdots & 0 \\ 0 & \frac{\sigma_{\text{int},2}^2}{\sigma^2} & \cdots & 0 \\ \vdots & \vdots & \ddots & \vdots \\ 0 & 0 & \cdots & \frac{\sigma_{\text{int},N}^2}{\sigma^2} \end{bmatrix}$$

Since we consider all the groups to share the same variance covariance component:

$$D = \begin{bmatrix} \frac{\sigma_{\text{int}}^2}{\sigma^2} & 0 & 0 \\ 0 & \ddots & 0 \\ 0 & 0 & \frac{\sigma_{\text{int}}^2}{\sigma^2} \end{bmatrix}$$

If we had intercept and slope, each D_i could be represented as a 2x2 matrix, because each Z_i block consists of two columns (one for intercepts and one for slopes).

$$D_i = \begin{bmatrix} \frac{\sigma_{\text{int}}^2}{\sigma^2} & 0 \\ 0 & \frac{\sigma_{\text{slope}}^2}{\sigma^2} \end{bmatrix}$$

Then

$$D = \begin{bmatrix} \begin{bmatrix} \frac{\sigma_{\text{int}}^2}{\sigma^2} & 0 \\ 0 & \frac{\sigma_{\text{slope}}^2}{\sigma^2} \end{bmatrix} & 0 & 0 \\ 0 & \ddots & 0 \\ 0 & 0 & \begin{bmatrix} \frac{\sigma_{\text{int}}^2}{\sigma^2} & 0 \\ 0 & \frac{\sigma_{\text{slope}}^2}{\sigma^2} \end{bmatrix} \end{bmatrix}$$

Give D and Z_i we can now write $V_i = \sigma^2(\mathbf{I} + \frac{\sigma^2}{\sigma^2}\mathbf{1}\mathbf{1}')$ as

$$V_i = \sigma^2(\mathbf{I} + Z_i D Z_i^T)$$

The complete matrix form of V can be written as

$$V = \sigma^2(\mathbf{I} + ZDZ^T)$$

Chapter 24

Estimation of parameters

Given everything we have said so far, we now have complete ingredients to perform GLS. This includes our response variable y , design matrix for the effect of interest X (fixed effect), a design matrix for grouping structure in the data Z and variance covariance matrix D which encodes σ^2 (residual variance) and σ_d^2 (variance of the grouping structure) which gives us $V = \sigma^2(\mathbf{I} + ZDZ^T)$.

What we have assumed so far is that we know V but in reality V is unknown. Since V is from using σ^2 and $\frac{\sigma_d^2}{\sigma^2}$, we need to estimate these two which we are going to denote as $\theta = [\sigma^2, \frac{\sigma_d^2}{\sigma^2}] = [\sigma^2, \lambda]$.

To begin, in order to find the parameters for our model, we employ Maximum Likelihood Estimation (MLE). The goal is to estimate the θ that best fit our model to the observed data.

The GLS model assumes that the response variable y is linearly dependent on multiple predictors, captured in matrix X , with random errors ϵ that are normally distributed but with a non-constant variance structure. This variance structure is modeled by $\sigma^2 V$, where V depends on another matrix Z and a variance-covariance matrix D . The model can be written as:

$$y = X\beta + \epsilon, \quad \text{where } \epsilon \sim N(0, \sigma^2 V).$$

Since $\epsilon \sim N(0, \sigma^2 V)$, and $\epsilon = y - X\beta$, we can say that y , the response variable, is also normally distributed. This follows from the properties of the normal distribution where a linear transformation of a normally distributed variable is also normally distributed. Thus:

$$y \sim N(X\beta, \sigma^2 V).$$

The probability density function for a multivariate normal distribution with mean μ and covariance matrix Σ is given by:

$$f(x) = \frac{1}{(2\pi)^{k/2} |\Sigma|^{1/2}} \exp \left(-\frac{1}{2} (x - \mu)^T \Sigma^{-1} (x - \mu) \right),$$

We substitute our model notation and it gives us

$$L(\theta; y) = \frac{1}{(2\pi)^{n/2} |V|^{1/2}} \exp \left(-\frac{1}{2} (y - X\beta)^T (V)^{-1} (y - X\beta) \right),$$

where n is the number of observations in y . Here $|V|$ denotes the determinant of V .

Taking the natural logarithm of the likelihood function simplifies calculations and helps in deriving estimators. The log-likelihood (ℓ) is:

$$\ell(\theta; y) = -\frac{n}{2} \log(2\pi) - \frac{1}{2} \log |V| - \frac{1}{2} (y - X\beta)^T V^{-1} (y - X\beta).$$

What we know from GLS (see above) is that

$$\beta = (X^T V^{-1} X)^{-1} X^T V^{-1} y.$$

Ignore the constant $-n \log(2\pi)$ as it does not depend on β or θ .

3. Substituting $\tilde{\beta}$ and acknowledging V as $\tilde{V}(\theta)$ gives:

$$-2\ell(\theta; Y) = \log |\tilde{V}(\theta)| + (Y - X\tilde{\beta}(\theta))^T \tilde{V}(\theta)^{-1} (Y - X\tilde{\beta}(\theta)).$$

Given this function we can go ahead and write a small piece of code to calculate this.

```
simulate_grouped_trend <- function(group_count = 5,
  ↪ points_per_group = 10, global_slope = -10, global_intercept =
  ↪ 30, group_slope = 2, noise_sd = 50, noise_sd2=2) {
  set.seed(123) # Setting a seed for reproducibility

  # Initialize an empty data frame to store the simulated data
  data <- data.frame(x = numeric(), y = numeric())

  # Loop to create each group
  for (i in 1:group_count) {
    x_start <- 12 + (i - 1) * (10 / group_count) # Stagger the
    ↪ start of x for each group
    x <- runif(points_per_group, min = x_start, max = x_start +
    ↪ (10 / group_count))

    # Apply a local positive trend within the group, but maintain
    ↪ the global negative trend
    local_intercept <- global_intercept + global_slope * (x_start
    ↪ + (10 / (2 * group_count))) + rnorm(1, mean = 0, sd =
    ↪ noise_sd)
    y <- local_intercept + group_slope[i] * (x - x_start) +
    ↪ rnorm(points_per_group, mean = 0, sd = noise_sd2)

    # Combine this group with the overall dataset
```

```

    group_data <- data.frame(x = x, y = y, group=i)
    data <- rbind(data, group_data)
  }

  return(data)
}

# generate simulated data
data_int <- simulate_grouped_trend(group_count =
  ↪ 4, points_per_group = 10, global_slope = -2, global_intercept =
  ↪ 100, group_slope = c(6, 4, 2, 1), noise_sd = 5, noise_sd2=2)

# set group to factor

data_int$group <- factor(data_int$group)

# theta is [sigmal, lambda]

# Design matrix of random effect
Z <- model.matrix(~0+data_int$group)

# Construct the model matrix 'X' for the fixed effect 'x' using
  ↪ data from 'data_int'.
X <- model.matrix(~x, data=data_int)

# get groups (random effects)
groups <- unique(data_int$group)

log1 <- function(theta) {
  D <- diag(x = theta[2], ncol = length(groups), nrow =
  ↪ length(groups))
  I <- diag(1, nrow = nrow(X), ncol = nrow(X))
  V <- theta[1] * (I + Z %*% D %*% t(Z))
  Beta <- solve(t(X) %*% solve(V) %*% X) %*% t(X) %*% solve(V)
  ↪ %*% y
  return(c(log(det(V)) + t(y - X %*% Beta) %*% solve(V) %*% (y -
  ↪ X %*% Beta)))
}

theta <- optimization::optim_nm(fun = log1, k = 2, start =
  ↪ c(1, 1), maximum = FALSE, tol = 0.000000000001)$par

cat("Residual = ", sqrt(theta[1]), " group = ",
  ↪ sqrt(prod(theta)), "\n")

```

```
Residual = 2.40449 group = 16.61624
```

if we compare the results to that of `lmer` we see they are identical:

```
print(VarCorr(lmer(y~x+(1|group),data=data_int,REML = FALSE)))
```

Groups	Name	Std.Dev.
group	(Intercept)	16.6162
Residual		2.4045

24.1 Bias of MLE

While Maximum Likelihood Estimators are efficient and consistent under many conditions, they can be biased, particularly in estimating the variance in a model.

Consider a random sample $y_1, y_2, \dots, y_n$ from a normal distribution $N(\mu, \sigma^2)$. We want to estimate the variance σ^2 using MLE.

The probability density function for a single observation y_i given σ^2 is:

$$f(y_i|\sigma^2) = \frac{1}{\sqrt{2\pi\sigma^2}} \exp\left(-\frac{(y_i - \mu)^2}{2\sigma^2}\right).$$

The likelihood function for all observations $y_1, \dots, y_n$, assuming independence, is the product of individual densities:

$$L(\sigma^2|y_1, \dots, y_n) = \prod_{i=1}^n \frac{1}{\sqrt{2\pi\sigma^2}} \exp\left(-\frac{(y_i - \mu)^2}{2\sigma^2}\right).$$

The log-likelihood function, $\ell(\sigma^2)$, is the natural logarithm of the likelihood function:

$$\ell(\sigma^2) = \sum_{i=1}^n \log\left(\frac{1}{\sqrt{2\pi\sigma^2}}\right) - \sum_{i=1}^n \frac{(y_i - \mu)^2}{2\sigma^2} = -\frac{n}{2} \log(2\pi\sigma^2) - \frac{1}{2\sigma^2} \sum_{i=1}^n (y_i - \mu)^2.$$

To find the maximum likelihood estimator (MLE) for σ^2 , differentiate $\ell(\sigma^2)$ with respect to σ^2 and set this derivative to zero:

$$\frac{d\ell(\sigma^2)}{d\sigma^2} = -\frac{n}{2\sigma^2} + \frac{\sum_{i=1}^n (y_i - \mu)^2}{2(\sigma^2)^2} = 0.$$

Solving for σ^2 , we rearrange and simplify:

$$\frac{n}{2\sigma^2} = \frac{\sum_{i=1}^n (y_i - \mu)^2}{2(\sigma^2)^2} \Rightarrow \sigma^2 = \frac{1}{n} \sum_{i=1}^n (y_i - \mu)^2.$$

When μ is estimated from the data, the sample mean $\bar{y}$ is:

$$\bar{y} = \frac{1}{n} \sum_{i=1}^n y_i.$$

The MLE for σ^2 when the mean is estimated from the data becomes:

$$\hat{\sigma}^2 = \frac{1}{n} \sum_{i=1}^n (y_i - \bar{y})^2,$$

We now need to take the expectation of $\hat{\sigma}^2$. To find the expectation of $\hat{\sigma}^2$. To do that we express it in terms of $y_i - \mu$ and $\bar{y} - \mu$.

First, rewrite the squared term using the identity $a - b = (a - c) + (c - b)$ where $a = y_i$, $b = \bar{y}$, and $c = \mu$:

$$y_i - \bar{y} = (y_i - \mu) - (\bar{y} - \mu).$$

Thus, the squared term becomes:

$$(y_i - \bar{y})^2 = [(y_i - \mu) - (\bar{y} - \mu)]^2.$$

Expanding this, we get:

$$(y_i - \bar{y})^2 = (y_i - \mu)^2 - 2(y_i - \mu)(\bar{y} - \mu) + (\bar{y} - \mu)^2.$$

Summing over all i from 1 to n , we obtain:

$$\sum_{i=1}^n (y_i - \bar{y})^2 = \sum_{i=1}^n (y_i - \mu)^2 - 2 \sum_{i=1}^n (y_i - \mu)(\bar{y} - \mu) + \sum_{i=1}^n (\bar{y} - \mu)^2.$$

Since $\bar{y} - \mu$ does not depend on i , it can be factored out of the sums:

$$\sum_{i=1}^n (y_i - \bar{y})^2 = \sum_{i=1}^n (y_i - \mu)^2 - 2(\bar{y} - \mu) \sum_{i=1}^n (y_i - \mu) + n(\bar{y} - \mu)^2.$$

Given that $\sum_{i=1}^n (y_i - \mu) = n(\bar{y} - \mu)$, the middle term simplifies to:

$$-2n(\bar{y} - \mu)^2,$$

and the equation becomes:

$$\sum_{i=1}^n (y_i - \bar{y})^2 = \sum_{i=1}^n (y_i - \mu)^2 - n(\bar{y} - \mu)^2.$$

We take the expectation:

$$E[\hat{\sigma}^2] = E\left[\frac{1}{n} \sum_{i=1}^n (y_i - \bar{y})^2\right] = \frac{1}{n} E\left[\sum_{i=1}^n (y_i - \mu)^2 - n(\bar{y} - \mu)^2\right].$$

Since $E[\sum_{i=1}^n (y_i - \mu)^2] = n\sigma^2$ and $E[n(\bar{y} - \mu)^2] = \sigma^2$ (by the properties of variance), we get:

$$E[\hat{\sigma}^2] = \frac{1}{n}[n\sigma^2 - \sigma^2] = \frac{n-1}{n}\sigma^2.$$

Thus, the bias $B(\hat{\sigma}^2)$ is:

$$B(\hat{\sigma}^2) = E(\hat{\sigma}^2) - \sigma^2 = \frac{n-1}{n}\sigma^2 - \sigma^2 = \frac{n-1-n}{n}\sigma^2 = \frac{-1}{n}\sigma^2.$$

24.2 Restricted Maximum Likelihood (REML)

As said before MLE has a significant limitation: it tends to underestimate variance components due to its non-adjustment for the degrees of freedom consumed by the fixed effects. This underestimation arises because MLE maximizes a likelihood function using all available data, which inadvertently causes an overfitting of fixed effects, leading to biased and sometimes misleading estimates of variance.

This bias in MLE is known to result from too large a denominator in the variance formula, which does not account for the degrees of freedom used up by estimating the mean (fixed effects). Intuitively, this comes from how the sample mean is based on the same data, making the data points appear more clustered around the mean than they actually are, hence underestimating the true variability that a correct denominator would account for.

To address this issue, Restricted Maximum Likelihood (REML) was developed. REML refines the approach used by MLE by first fitting the fixed effects and then adjusting the likelihood function used to estimate variance components, specifically compensating for the degrees of freedom used in estimating the fixed effects. This adjustment ensures that the estimation of variance components is based on the correct amount of independent information.

A key aspect of REML is the process of “integrating out” the fixed effects. This approach involves focusing on the likelihood of the residuals after removing the influence of the fixed effects, rather than the likelihood of the entire dataset. By integrating out these effects, REML effectively separates the estimation of variance parameters from the estimation of the fixed effects, which can reduce bias in the variance estimates. The integration is achieved through a mathematical transformation where the fixed effects are marginalized over (integrated out), usually requiring the calculation of integrals over these parameters. This process results in a likelihood function that depends only on the random effects and the variance components, which REML then maximizes to obtain more accurate and less biased estimates.

Let's have a look at our previous likelihood function

$$\log[\ell(\theta; y)] = -\frac{n}{2}\log(2\pi) - \frac{1}{2}\log|V| + \log\left[e^{-\frac{(\mathbf{y}-\mathbf{x})^T\mathbf{V}^{-1}(\mathbf{y}-\mathbf{x})}{2}}\right]$$

Integrating out the fixed effects in REML involves focusing on the part of the log-likelihood that pertains only to the variance components, adjusting for the estimation of β :

$$\log \left[\int L(\beta, V) d\beta \right] = -\frac{1}{2} \log(2\pi) - \frac{1}{2} \log(|\mathbf{V}|) + \log \left[\int e^{-\frac{(\mathbf{Y}-\mathbf{X}\beta)^T \mathbf{V}^{-1} (\mathbf{Y}-\mathbf{X}\beta)}{2}} d\beta \right]$$

Let's start from $(\mathbf{Y} - \mathbf{X}\beta)^T \mathbf{V}^{-1} (\mathbf{Y} - \mathbf{X}\beta)$

Expanding this, we get:

$$= \mathbf{Y}^T \mathbf{V}^{-1} \mathbf{Y} - \mathbf{Y}^T \mathbf{V}^{-1} \mathbf{X} \beta - \beta^T \mathbf{X}^T \mathbf{V}^{-1} \mathbf{Y} + \beta^T \mathbf{X}^T \mathbf{V}^{-1} \mathbf{X} \beta$$

To rewrite this in the form of a completed square, it's helpful to recognize the general form: $\beta^T \mathbf{A} \beta - 2\beta^T \mathbf{b}$. Here, $\mathbf{A} = \mathbf{X}^T \mathbf{V}^{-1} \mathbf{X}$ and $\mathbf{b} = \mathbf{X}^T \mathbf{V}^{-1} \mathbf{Y}$.

We can rewrite it by completing the square in β :

$$\begin{aligned} & \beta^T \mathbf{X}^T \mathbf{V}^{-1} \mathbf{X} \beta - 2\beta^T \mathbf{X}^T \mathbf{V}^{-1} \mathbf{Y} \\ &= (\beta - \mathbf{A}^{-1} \mathbf{b})^T \mathbf{A} (\beta - \mathbf{A}^{-1} \mathbf{b}) - \mathbf{b}^T \mathbf{A}^{-1} \mathbf{b} \end{aligned}$$

Where $\mathbf{A}^{-1} = (\mathbf{X}^T \mathbf{V}^{-1} \mathbf{X})^{-1}$.

Substituting Back: Thus, substituting back $\mathbf{A}^{-1} \mathbf{b}$ and rearranging, we find:

$$= (\beta - (\mathbf{X}^T \mathbf{V}^{-1} \mathbf{X})^{-1} \mathbf{X}^T \mathbf{V}^{-1} \mathbf{Y})^T \mathbf{X}^T \mathbf{V}^{-1} \mathbf{X} (\beta - (\mathbf{X}^T \mathbf{V}^{-1} \mathbf{X})^{-1} \mathbf{X}^T \mathbf{V}^{-1} \mathbf{Y}) + \mathbf{Y}^T \mathbf{V}^{-1} \mathbf{Y} - \mathbf{b}^T \mathbf{A}^{-1} \mathbf{b}$$

Simplifying:

$$= (\beta - \mathbf{A} \mathbf{Y})^T \mathbf{X}^T \mathbf{V}^{-1} \mathbf{X} (\beta - \mathbf{A} \mathbf{Y}) + \mathbf{Y}^T \mathbf{V}^{-1} \mathbf{Y} - \mathbf{Y}^T \mathbf{V}^{-1} \mathbf{X} \mathbf{A} \mathbf{Y}$$

where $\mathbf{A} = (\mathbf{X}^T \mathbf{V}^{-1} \mathbf{X})^{-1} \mathbf{X}^T \mathbf{V}^{-1}$.

The final expression for the quadratic form then becomes:

$$(\beta - \mathbf{A} \mathbf{Y})^T \mathbf{X}^T \mathbf{V}^{-1} \mathbf{X} (\beta - \mathbf{A} \mathbf{Y}) + \mathbf{Y}^T (\mathbf{V}^{-1} - \mathbf{V}^{-1} \mathbf{X} \mathbf{A}) \mathbf{Y}$$

Given:

$$\int e^{-\frac{(\mathbf{Y}-\mathbf{X}\beta)^T \mathbf{V}^{-1} (\mathbf{Y}-\mathbf{X}\beta)}{2}} d\beta$$

From the earlier completion of the square, we know the integral's argument can be expressed as:

$$(\beta - \mathbf{A} \mathbf{Y})^T \mathbf{X}^T \mathbf{V}^{-1} \mathbf{X} (\beta - \mathbf{A} \mathbf{Y}) + \mathbf{Y}^T (\mathbf{V}^{-1} - \mathbf{V}^{-1} \mathbf{X} \mathbf{A}) \mathbf{Y}$$

The integral of a Gaussian (quadratic form in the exponent) over β is:

$$e^{-\frac{1}{2} \mathbf{Y}^T (\mathbf{V}^{-1} - \mathbf{V}^{-1} \mathbf{X} \mathbf{A}) \mathbf{Y}} \cdot \int e^{-\frac{1}{2} (\beta - \mathbf{A} \mathbf{Y})^T \mathbf{X}^T \mathbf{V}^{-1} \mathbf{X} (\beta - \mathbf{A} \mathbf{Y})} d\beta$$

For the integral over β , since it's a Gaussian integral, it simplifies to:

$$(2\pi)^{\frac{k}{2}} |\mathbf{X}^T V^{-1} \mathbf{X}|^{-\frac{1}{2}}$$

Where k is the number of parameters (dimension of β).

Substituting back:

$$\begin{aligned} & \log \left[e^{-\frac{1}{2} \mathbf{Y}^T (V^{-1} - V^{-1} \mathbf{X} \mathbf{A}) \mathbf{Y}} (2\pi)^{\frac{k}{2}} |\mathbf{X}^T V^{-1} \mathbf{X}|^{-\frac{1}{2}} \right] \\ &= -\frac{1}{2} \mathbf{Y}^T (V^{-1} - V^{-1} \mathbf{X} \mathbf{A}) \mathbf{Y} + \frac{k}{2} \log(2\pi) - \frac{1}{2} \log |\mathbf{X}^T V^{-1} \mathbf{X}| \end{aligned}$$

$$\log \left[\int L(\beta, V) d\beta \right] = -\frac{1}{2} \log(2\pi) - \frac{1}{2} \log(|V|) + \left(-\frac{1}{2} \mathbf{Y}^T (V^{-1} - V^{-1} \mathbf{X} \mathbf{A}) \mathbf{Y} + \frac{k}{2} \log(2\pi) - \frac{1}{2} \log |\mathbf{X}^T V^{-1} \mathbf{X}| \right)$$

Therefore:

$$-2\ell_{REML}(\theta; Y) = \log |V| + \log |\mathbf{X}^T V^{-1} \mathbf{X}| + \mathbf{Y}^T V^{-1} \mathbf{Y} - \mathbf{Y}^T V^{-1} \mathbf{X} \mathbf{A} \mathbf{Y}$$

where $\mathbf{A} = (\mathbf{X}^T V^{-1} \mathbf{X})^{-1} \mathbf{X}^T V^{-1}$

Rewriting the last part sing complete squares and simplifying ($\mathbf{A} \mathbf{Y} = \tilde{\beta}(\theta)$):

$$\mathbf{Y}^T V^{-1} \mathbf{Y} - \mathbf{Y}^T V^{-1} \mathbf{X} \mathbf{A} \mathbf{Y} = (\mathbf{Y} - \mathbf{X} \tilde{\beta}(\theta))^T V^{-1} (\mathbf{Y} - \mathbf{X} \tilde{\beta}(\theta))$$

Combining the rearranged terms, the full expression is:

$$-2\ell_{REML}(\theta; Y) = \log |\tilde{V}(\theta)| + \log |X^T \tilde{V}^{-1} X| + (Y - X \tilde{\beta}(\theta))^T \tilde{V}^{-1} (Y - X \tilde{\beta}(\theta))$$

This is our final equation for REML and can be implemented easily:

```
# theta is [signal,lambda]

# Design matrix of random effect
Z <- model.matrix(~0+data_int$group)

# Construct the model matrix 'X' for the fixed effect 'x' using
# data from 'data_int'.
X <- model.matrix(~x, data=data_int)

# get groups (random effects)
groups <- unique(data_int$group)

log_reml <- function(theta) {
  D <- diag(x = theta[2], ncol = length(groups), nrow =
# length(groups))
  I <- diag(1, nrow = nrow(X), ncol = nrow(X))
```

```

V<-theta[1]*(I+Z%*%D%*%t(Z))
Beta <- solve(t(X) %*% solve(V) %*% X) %*% t(X) %*% solve(V)
↪ %*% y
return(c(log(det(V)) + log(det(t(X) %*% solve(V) %*% X))+ t(y -
↪ X %*% Beta) %*% solve(V) %*% (y - X %*% Beta)))
}

theta<-optimization::optim_nm(fun = log_reml, k = 2, start =
↪ c(1,1),maximum= FALSE, tol = 0.000000000001)$par

cat("Residual = ",sqrt(theta[1])," group = ",
↪ sqrt(prod(theta)), "\n")

```

```
Residual = 2.435465 group = 19.45779
```

which gives the same results as

```
print(VarCorr(lmer(y~x+(1|group),data=data_int,REML = TRUE)))
```

Groups	Name	Std.Dev.
group	(Intercept)	19.4578
Residual		2.4355

24.3 Other methods of fitting

In practice, when dealing with this kind of models, we do not directly optimize the log-likelihood function due to its complexity, particularly when random effects are included. Instead, more tractable approaches such as the Expectation-Maximization (EM) algorithm, Newton-Raphson, and quasi-likelihood methods are commonly employed. These techniques facilitate the optimization process by breaking it down into simpler steps or by approximating the log-likelihood function. For instance, the EM algorithm efficiently handles missing data or latent variables by iteratively estimating expected values and maximizing the likelihood. Newton-Raphson and related methods, on the other hand, provide robust mechanisms for dealing with non-linear optimization through successive approximations. These methods are essential for ensuring convergence and computational feasibility in the analysis of mixed models.

Chapter 25

Best Linear Unbiased Estimators (BLUE)

Now that we have estimated the components of V , we can show that B is in fact Best Linear Unbiased Estimators (BLUE). This involves proving that the GLS estimator is both unbiased and has the smallest variance among all linear unbiased estimators.

Recall the GLS estimator is defined as:

$$\hat{\beta}_{GLS} = (X^T V^{-1} X)^{-1} X^T V^{-1} y$$

where V is the known covariance matrix of the errors e in the model $y = X\beta + e$.

The model equation is:

$$y = X\beta + e$$

where e is the error term with $E(e) = 0$ and $Var(e) = \sigma^2 V$.

Plug the expression for y into the GLS estimator:

$$\hat{\beta}_{GLS} = (X^T V^{-1} X)^{-1} X^T V^{-1} (X\beta + e)$$

Break the expression into two parts by distributing $X^T V^{-1}$:

$$\hat{\beta}_{GLS} = (X^T V^{-1} X)^{-1} X^T V^{-1} X\beta + (X^T V^{-1} X)^{-1} X^T V^{-1} e$$

Notice that $X^T V^{-1} X$ and $(X^T V^{-1} X)^{-1} X^T V^{-1} X$ simplify as follows:

$$(X^T V^{-1} X)^{-1} X^T V^{-1} X = I$$

where I is the identity matrix, thus:

$$\hat{\beta}_{GLS} = \beta + (X^T V^{-1} X)^{-1} X^T V^{-1} e$$

To show unbiasedness, we take the expectation of $\hat{\beta}_{GLS}$:

$$E(\hat{\beta}_{GLS}) = E(\beta + (X^T V^{-1} X)^{-1} X^T V^{-1} e)$$

$$E(\hat{\beta}_{GLS}) = \beta + E((X^T V^{-1} X)^{-1} X^T V^{-1} e)$$

Since $E(e) = 0$:

$$E((X^T V^{-1} X)^{-1} X^T V^{-1} e) = (X^T V^{-1} X)^{-1} X^T V^{-1} E(e) = 0$$

Therefore:

$$E(\hat{\beta}_{GLS}) = \beta$$

The expectation of the GLS estimator is equal to the true parameter vector β , so $\hat{\beta}_{GLS}$ is an unbiased estimator of β . That means covariance structure V of the errors do not introduce any bias into the estimator.

Lastly we need to show that variance of the GLS estimator $\hat{\beta}_{GLS}$ is lower than or equal to the variance of any other linear unbiased estimator.

Let's first derive the variance of $\hat{\beta}_{GLS}$

Recall that

$$\hat{\beta}_{GLS} = (X^T V^{-1} X)^{-1} X^T V^{-1} y$$

Since $y = X\beta + e$ and the errors e have covariance $\sigma^2 V$, the variance of y is:

$$Var(y) = \sigma^2 V$$

Give the linear transformation rule for variance ($Var(Ay) = AVar(y)A^T$), which states that for any linear transformation

$$\begin{aligned} Var(\hat{\beta}_{GLS}) &= Var((X^T V^{-1} X)^{-1} X^T V^{-1} y) \\ &= (X^T V^{-1} X)^{-1} X^T V^{-1} Var(y) V^{-1} X (X^T V^{-1} X)^{-1} \\ &= (X^T V^{-1} X)^{-1} X^T V^{-1} (\sigma^2 V) V^{-1} X (X^T V^{-1} X)^{-1} \end{aligned}$$

Simplifying:

$$\begin{aligned} &= \sigma^2 (X^T V^{-1} X)^{-1} X^T V^{-1} V V^{-1} X (X^T V^{-1} X)^{-1} = \sigma^2 (X^T V^{-1} X)^{-1} (X^T V^{-1} X) (X^T V^{-1} X)^{-1} \\ &= \sigma^2 (X^T V^{-1} X)^{-1} \end{aligned}$$

The variance of the GLS estimator $\hat{\beta}_{GLS} = Var(\hat{\beta}_{GLS}) = \sigma^2 (X^T V^{-1} X)^{-1}$

Any other linear unbiased estimator $\hat{\beta}_L$ can be expressed as:

$$\hat{\beta}_L = CY$$

where C must satisfy $CX = I$ for unbiasedness. Specifically, we can define C as:

$$C = (X^T V^{-1} X)^{-1} X^T V^{-1} + M$$

where M is a matrix such that $MX = 0$. This ensures that $\hat{\beta}_L$ is unbiased:

$$E[\hat{\beta}_L] = \beta + M X E[\beta] = \beta$$

since $E[y] = X\beta$.

Given the model $y = X\beta + e$ with $\text{Var}(e) = \sigma^2 V$, the variance of y is $\sigma^2 V$, and hence:

$$\text{Var}(\hat{\beta}_L) = C\sigma^2 V C^T$$

Substituting C into the variance formula, we get:

$$\text{Var}(\hat{\beta}_L) = ((X^T V^{-1} X)^{-1} X^T V^{-1} + M) \sigma^2 V ((X^T V^{-1} X)^{-1} X^T V^{-1} + M)^T$$

Expanding and using $MX = 0$, we have:

$$\text{Var}(\hat{\beta}_L) = \sigma^2 ((X^T V^{-1} X)^{-1} + MVM^T)$$

Since $\sigma^2 (X^T V^{-1} X)^{-1} = \text{Var}(\hat{\beta}_{GLS})$ then $\text{Var}(\hat{\beta}_L) = \text{Var}(\hat{\beta}_{GLS}) + \sigma^2 MVM^T$

MVM^T is a positive semi-definite matrix since V is positive definite (or semi-definite if V is not strictly positive). This implies $\sigma^2 MVM^T \geq 0$ which gives us $\text{Var}(\hat{\beta}_L) \geq \text{Var}(\hat{\beta}_{GLS})$.

This shows that any deviation from the GLS estimator in the form of an additional matrix M results in an increase in variance. Therefore, the GLS estimator $\hat{\beta}_{GLS}$ is the Best Linear Unbiased Estimator (BLUE) for models where the errors have a covariance structure described by $\sigma^2 V$.

Chapter 26

Random Effects

I want to briefly review the components we've discussed so far before proceeding with random effects

1. **Design Matrix, Z :** This matrix is called the design matrix for the random effect and encodes how each observation in our dataset is related to other observations within the same group or category. It effectively organizes our data according to predetermined groupings.
2. **Covariance Matrix, D :** D is the covariance matrix associated with the groupings defined by Z . It quantifies the variance within each group as well as the covariance between groups.
3. **Error Terms:** Often denoted as σ^2 , this represents the variance of the error terms in our models. It captures the amount of variability in our observations that isn't explained by the model.

What we tried to model so far is that we believe there is certain grouping in the data which we encoded by Z . What we believe is that for one or more of our primary effects just as intercept, slope etc there exist variabilities that are specific to these groups and not fully captured by these primary effects alone. This variability, which may come from unique characteristics or influences within each group. We modeled this variability using D . If we suspected that only a single effect such as intercept was affected by groups, D will be diagonal, meaning that it will have a single value repeating on its diagonal element. If we suspect 2 or more effects are random, then D is block diagonal. Each block's size corresponds to the number of effects considered to be random within a group, effectively capturing the covariance between these effects across different groupings.

I guess you realized that our aim has never been to **estimate** the exact influence of each single group on the effects but rather to say that they are from a distribution that captures the inherent variability among the groups. If we have thousands of data points, we might as well consider estimating directly each group's effect; however, such an approach can quickly become unwieldy and prone to overfitting, especially in cases where groups are many but each group has few observations. By using a model that incorporates random effects, we

effectively balance the need for specificity in modeling group influences against the practical constraints of model complexity and the risk of fitting noise rather than signal. Also have this in mind that, we might not be interested in the effects of each group but rather just quantifying the influence collectively would be sufficient.

Having said that, we know that these groupings affect the observed data, yet unlike fixed effects, random effects are not directly observable. They represent underlying influences or variables that affect the outcome but are not included as direct measurements in the data. For example, in a study on student performance across schools, random effects might include inherent school qualities like teaching methods or school climate, which are not directly measured but still influence student outcomes. Despite being unseen, we can predict these random effects. This is different to directly estimating a parameter in that prediction focuses on the likely values of random variables for specific instances or groups, given the observed data and the model structure. We are going to denote these prediction as u here.

We we said is that u is coming from a distribution D as covariance parameter and mean of zero. so $u \sim N(0, \sigma^2 D)$ and we also said that our residuals are distributed as $e \sim N(0, \sigma^2)$.

The errors e and the random effects u are assumed independent.

Given the assumptions and linearity, the observations y are also normally distributed with: - Mean: $E(y) = X\beta$ (since $E(u) = 0$ and $E(e) = 0$) - Covariance: $Var(y) = Z\sigma^2 DZ^T + \sigma^2 I$

Please note that this is because our total variation is variation of Zu and e so $Var(Zu) = ZDZ^T$ and $Var(e) = \sigma^2$, utilizing the independence of u and e .

Therefore $y \sim N(X\beta, \sigma^2(I + ZDZ^T))$

When we have a pair of random variables X and Y , and we know the outcome of Y , the conditional expectation $E(X|Y)$ serves as the best predictor of X based on Y , in the mean squared error sense. This is because it minimizes the expected squared difference between the predicted values and the actual values of X , compared to any other function of Y .

The conditional expectation of X given Y in a bivariate normal distribution can be expressed as:

$$E(X|Y = y) = \mu_X + \frac{\sigma_{XY}}{\sigma_Y^2}(y - \mu_Y)$$

To apply conditional expectation, recognize that u and y have a joint normal distribution. We can reformulate conditional expectation in a form suitable for our model:

$$E(u|y) = E(u) + Cov(u, y)Var(y)^{-1}(y - E(y))$$

- $E(u) = 0$ and $E(y) = X\beta$ - $Cov(u, y)$ needs to be calculated:

$$Cov(u, y) = Cov(u, Zu + e)$$

$$\begin{aligned}
&= \text{Cov}(u, Zu) + \text{Cov}(u, e) \\
&= \sigma^2 DZ^T + 0 \\
&= \sigma^2 DZ^T
\end{aligned}$$

The last equality is because $\text{Cov}(u, e) = 0$ due to independence and $\text{Cov}(u, Zu) = \sigma^2 DZ^T$ because

$$\text{Cov}(u, Zu) = E[(u - E[u])(Zu - E[Zu])^T]$$

Given that $E[u] = 0$ (since u is assumed to have mean zero), the expression simplifies to:

$$\text{Cov}(u, Zu) = E[u(Zu)^T]$$

$$E[u(Zu)^T] = E[u(u^T Z^T)]$$

Here, $u^T Z^T$ is simply a rearrangement of the multiplication. T

The inner operation $u(u^T Z^T)$ results in a matrix where each element is a product of elements from u and u^T , transformed by Z^T .

$$E[uu^T] = \sigma^2 D$$

So, applying Z^T to each element effectively results in:

$$E[u(u^T Z^T)] = E[uu^T]Z^T = \sigma^2 DZ^T$$

Using the formula for conditional expectation in multivariate normal distributions:

$$\begin{aligned}
E(u|y) &= 0 + \sigma^2 DZ^T (Z\sigma^2 DZ^T + \sigma^2)^{-1} (y - X\beta) \\
&= \sigma^2 DZ^T V^{-1} (y - X\beta)
\end{aligned}$$

where $V = Z\sigma^2 DZ^T + \sigma^2$ is the covariance matrix of y .

The final predictor $\hat{u}$ is:

$$\hat{u} = \sigma^2 DZ^T V^{-1} (y - X\beta)$$

Let's give this a try to see if it gives us correct results:

```
# Design matrix of random effect
Z <- model.matrix(~0+data_int$group)

# Construct the model matrix 'X' for the fixed effect 'x' using
  ↪ data from 'data_int'.
X <- model.matrix(~x, data=data_int)

# get groups (random effects)
groups <- unique(data_int$group)
```

```

log_reml <- function(theta) {
  D <- diag(x = theta[2], ncol = length(groups), nrow =
    ↪ length(groups))
  I <- diag(1, nrow = nrow(X), ncol = nrow(X))
  V <- theta[1] * (I + Z %>% D %>% t(Z))
  Beta <- solve(t(X) %>% solve(V) %>% X) %>% t(X) %>% solve(V)
    ↪ %>% y
  return(c(log(det(V)) + log(det(t(X) %>% solve(V) %>% X)) + t(y -
    ↪ X %>% Beta) %>% solve(V) %>% (y - X %>% Beta)))
}

theta <- optimization::optim_nm(fun = log_reml, k = 2, start =
  ↪ c(1, 1), maximum = FALSE, tol = 0.0000000000001)$par

D <- diag(x = theta[2], ncol = length(groups), nrow =
  ↪ length(groups))
I <- diag(1, nrow = nrow(X), ncol = nrow(X))
V <- theta[1] * (I + Z %>% D %>% t(Z))
Beta <- solve(t(X) %>% solve(V) %>% X) %>% t(X) %>% solve(V) %>%
  ↪ y

u <- theta[1] * D %>% t(Z) %>% solve(V) %>% (y - X %>% Beta)

print(u)

```

```

      [,1]
[1,] 26.705634
[2,]  1.337016
[3,] -11.314929
[4,] -16.727721

```

compare this to lmer:

```
print(ranef(lmer(y~x+(1|group), data=data_int, REML = TRUE)))
```

```

$group
  (Intercept)
1    26.705636
2     1.337016
3   -11.314930
4   -16.727722

```

with conditional variances for "group"

We get identical results! These values are also called to be Best Linear Unbiased Prediction (BLUP) because Unbiasedness ($E(E(u|y)) = E(u) = 0$) and minimization of the variance of the prediction error $E[(u - \hat{u})^2]$ among all linear unbiased predictors. We are not going to prove them here as we did similar

analysis up there fore BLUEs.

Chapter 27

Final mixed model form

So all together, we now know that each observation in our data can be described by our fixed effects and predicted random effects plus some random noise.

Putting all together we have

$$y = X\beta + Zu + e$$

This describes our data model, where:

- y is the vector of observed data.
- β represents fixed effects, which may or may not be of interest for the derivation.
- u represents the random effects we wish to predict.
- e is the error term.
- X and Z are known design matrices.

We also assume: - $u \sim N(0, \sigma^2 D)$, where D is the covariance matrix of the random effects. - $e \sim N(0, \sigma^2)$, where σ^2 is the covariance matrix of the residuals. - u and e are independent.

This is the final form of the mixed model which we derived starting from OLS.

Chapter 28

p-value for mixed models (Satterthwaite approximation)

This should be clear by now that mixed models are inherently more complex than OLS. This complexity goes also to statistical testing. In this section, I want to write a bit about statistical testing both from the classical perspective but also from linear models and mixed models. Again we are not going to go through all details but just demonstrate some of the key concepts.

28.1 How rare is this value?!

Let's assume we have a specific value from our dataset that we're curious about its rarity. The only way to accurately assess how rare this value is involves understanding where it falls within an overall distribution of data. This compares the value against a bunch of expected values. If it deviates significantly from what's typical, then we can consider it rare. To do this, we first need to define the distribution that best describes our dataset. A common assumption to make about a dataset is that it follows a normal distribution. Many natural phenomena and human-made processes tend to distribute their values in a way that can be symmetrically modeled around a central value, with fewer occurrences appearing as one moves further away from this central point.

If we assume that our data follows a normal distribution, we can use its mathematical properties to check the rarity of specific observations within our dataset.

In probability, the “density” of a value in a continuous distribution refers to the relative likelihood of this value occurring. For a normal distribution values closer to the mean are more densely packed and so more likely to occur. As we move away from the mean, the density decreases, indicating that these values are less likely and rarer.

The density function doesn't give the probability of the event occurring at ex-

actly a specific point (which in continuous distributions is technically zero), but rather it helps us understand how dense the distribution is around that point, which is indicative of the probability of observing values in a small interval around that point.

The probability density function for a normal distribution is given by:

$$f(x|\mu, \sigma^2) = \frac{1}{\sqrt{2\pi\sigma^2}} e^{-\frac{(x-\mu)^2}{2\sigma^2}}$$

Here: - x is the value for which we are calculating the probability density. - μ is the mean of the distribution. - σ is the standard deviation, which measures the dispersion of the data around the mean. - σ^2 is the variance.

To determine how rare or unusual a particular value x is within a normal distribution, we need to calculate the probability of observing x or a value more extreme (larger or smaller, depending on the context). This is typically done using the cumulative distribution function (CDF), which gives the probability that a normally distributed random variable is less than or equal to a certain value.

CDF is represented by the integral of the probability density function from negative infinity to x . Mathematically, it's expressed as:

$$F(x|\mu, \sigma^2) = \int_{-\infty}^x \frac{1}{\sqrt{2\pi\sigma^2}} e^{-\frac{(t-\mu)^2}{2\sigma^2}} dt$$

For instance, if we want to calculate the probability of observing a value less than or equal to 60 from a normal distribution with a mean of 50 and a standard deviation of 10:

```
# Parameters
mu <- 50
sigma <- 10

# Value to calculate the cumulative probability for
x <- 60

# Calculate the probability
probability <- pnorm(x, mean = mu, sd = sigma)
print(probability)
```

```
[1] 0.8413447
```

I guess you realized that we have an infinite number of normal distributions defined by particular parameters (μ and σ). However, it is often nice to have a standard one from which we can derive a table of probabilities. This distribution is called the **standard normal distribution**.

28.2 Standard normal distribution

The standard normal distribution is a specific instance of the normal distribution where the mean (μ) is 0 and the standard deviation (σ) is 1. This standardization simplifies many statistical methods and allows us to use a single, universally applicable table (commonly called the Z-table) for finding probabilities

To convert any normal distribution to the standard normal distribution, values are transformed into what are known as Z-scores. This transformation process is called **standardization**. A Z-score tells us how many standard deviations away a point is from the mean of the distribution.

The formula to calculate the Z-score of a value x is:

$$Z = \frac{(x - \mu)}{\sigma}$$

Where:

- x is the value from the original distribution.
- μ is the mean of the original distribution.
- σ is the standard deviation of the original distribution.

Let's consider an example where we calculate the Z-scores:

```
# Sample data values
data_values <- c(40, 55, 60, 45, 70)

# Mean and standard deviation of the dataset
mu <- 50
sigma <- 10

# Calculate Z-scores
z_scores <- (data_values - mu) / sigma

# Output Z-scores
print(z_scores)
```

```
[1] -1.0  0.5  1.0 -0.5  2.0
```

We can now use a standard normal distribution for finding probabilities.

For any random variable we can write the transformation as

$$Z = \frac{(X - \mu)}{\sigma}$$

Now suppose that we went and collected a few samples and measured its mean $\bar{X}$ and we want to check how likely is that to get a value larger or smaller than $\bar{X}$. We can do a z transformation but with a small difference.

There is however a small difference here. Eventhough the sample can be used to calculate unbiased estimates of the population property, the sample estimate will not be perfect. The standard deviation of the sampling distribution is called

the standard error or SE . If each X_i is normally distributed with mean μ (mean of population) and standard deviation σ , then:

- **Expected Value:** $E[\bar{X}] = \mu$
- **Variance:** Since X_i are independent, the variance of the sum is the sum of the variances: $Var(\bar{X}) = Var(\frac{1}{n} \sum_{i=1}^n X_i) = \frac{1}{n^2} \sum_{i=1}^n Var(X_i) = \frac{1}{n^2} \cdot n\sigma^2 = \frac{\sigma^2}{n}$

The standard error of the mean is:

$$SEM = \frac{\sigma}{\sqrt{n}}$$

So we change the definition of Z for sample mean to

$$Z = \frac{(\bar{X} - \mu)}{\frac{\sigma}{\sqrt{n}}}$$

Which can be used to derive the probability. However, there is a challenge! So far, we have assumed that we know the variance of the population. In many practical situations σ is not known and must be estimated from the sample data we call it SE .

$$SE = \frac{s}{\sqrt{n}}$$

Here, s is the sample standard deviation, and n is the sample size. The standard error provides a measure of how much error is expected in the estimate of the mean. As the sample size increases, the standard error decreases, indicating a more accurate estimate of the population mean.

Our standardized value now becomes

$$\frac{\bar{X} - \mu}{SE}$$

There is again a small little question here: Is this actually normal distribution?

Let's see if we can write $\frac{\bar{X} - \mu}{SE}$ in terms of distributions that we already know such as Z .

We know that

$$\frac{\bar{X} - \mu}{SE} = \frac{\bar{X} - \mu}{S/\sqrt{n}}$$

Where S is the standard deviation of samples.

Recall that $Z = \frac{\bar{X} - \mu}{\sigma/\sqrt{n}}$.

So we can write

$$\bar{X} - \mu = Z \cdot \frac{\sigma}{\sqrt{n}}$$

Plugging this into the expression $\frac{\bar{X} - \mu}{S/\sqrt{n}}$, we have:

$$\frac{\bar{X} - \mu}{S/\sqrt{n}} = \frac{Z \cdot \frac{\sigma}{\sqrt{n}}}{S/\sqrt{n}} = Z \cdot \frac{\sigma}{S}$$

Now we should work our way around $\frac{\sigma}{S}$ and write it in a form that has known distribution.

Remember that sample mean $\bar{X}$ and sample variance S^2 are defined as follows:

$$\bar{X} = \frac{1}{n} \sum_{i=1}^n X_i$$

$$S^2 = \frac{1}{n-1} \sum_{i=1}^n (X_i - \bar{X})^2$$

First, let's express $X_i - \bar{X}$:

$$\begin{aligned} X_i - \bar{X} &= (X_i - \mu) - (\bar{X} - \mu) \\ &= (X_i - \mu) - \left(\frac{1}{n} \sum_{j=1}^n (X_j - \mu) \right) \end{aligned}$$

Now consider the sum of squared deviations from the mean:

$$\sum_{i=1}^n (X_i - \bar{X})^2 = \sum_{i=1}^n [(X_i - \mu) - (\bar{X} - \mu)]^2$$

Using the property of variance and the independence of X_i , we know:

$$\sum_{i=1}^n (X_i - \bar{X})^2 = \sum_{i=1}^n (X_i - \mu)^2 - n(\bar{X} - \mu)^2$$

Since X_i are i.i.d. $N(\mu, \sigma^2)$, $(X_i - \mu)/\sigma$ are i.i.d. $N(0, 1)$, we know that:

$$\sum_{i=1}^n \left(\frac{X_i - \mu}{\sigma} \right)^2 \sim \chi_n^2$$

However, we are interested in $\sum_{i=1}^n (X_i - \bar{X})^2$. Note that $\bar{X}$ itself is $N(\mu, \sigma^2/n)$.

By Cochran's theorem, the sum of squared deviations can be split into independent parts:

$$\sum_{i=1}^n (X_i - \mu)^2 = \sum_{i=1}^n (X_i - \bar{X})^2 + n(\bar{X} - \mu)^2$$

where $\sum_{i=1}^n (X_i - \bar{X})^2$ and $n(\bar{X} - \mu)^2$ are independent.

Now, $\sum_{i=1}^n (X_i - \bar{X})^2$ follows a chi-squared distribution with $n - 1$ degrees of freedom:

$$\sum_{i=1}^n (X_i - \bar{X})^2 \sim \sigma^2 \chi_{n-1}^2$$

We have:

$$S^2 = \frac{1}{n-1} \sum_{i=1}^n (X_i - \bar{X})^2$$

So, we can write:

$$(n-1)S^2 = \sum_{i=1}^n (X_i - \bar{X})^2$$

So:

$$\frac{(n-1)S^2}{\sigma^2} = \frac{\sum_{i=1}^n (X_i - \bar{X})^2}{\sigma^2} \sim \chi_{n-1}^2$$

Thus, we have shown that:

$$\frac{(n-1)S^2}{\sigma^2} \sim \chi_{n-1}^2$$

Let's call this $U = \frac{(n-1)S^2}{\sigma^2} \sim \chi_{n-1}^2$.

We can express σ in terms of S and U using the relation $\sigma^2 = \frac{(n-1)S^2}{U}$, then:

$$\sigma = S \sqrt{\frac{n-1}{U}}$$

Substituting back into the equation, we get:

$$Z \cdot \frac{S \sqrt{\frac{n-1}{U}}}{S} = Z \cdot \sqrt{\frac{n-1}{U}} = \frac{Z}{\sqrt{\frac{U}{n-1}}}$$

Therefore:

$$\frac{\bar{X} - \mu}{S/\sqrt{n}} = \frac{Z}{\sqrt{\frac{U}{n-1}}}$$

28.3 Putting it all together (T)

Based on

$$\frac{\bar{X} - \mu}{S/\sqrt{n}} = \frac{Z}{\sqrt{\frac{U}{n-1}}}$$

Let's just assume we call $n-1 = d$ (degrees of freedom) so

$$\frac{Z}{\sqrt{\frac{U}{n-1}}} = \frac{Z}{\sqrt{\frac{U}{d}}} \sim \chi_n^2$$

We know that a random variable Z that is standard normally distributed, denoted $Z \sim N(0, 1)$, has a probability density function (PDF):

$$f_Z(z) = \frac{1}{\sqrt{2\pi}} e^{-\frac{z^2}{2}}$$

Also a chi-squared distribution with d degrees of freedom, denoted $U \sim \chi_d^2$, can be expressed as the sum of squares of d independent standard normal variables. Its PDF is given by:

$$f_U(u) = \frac{u^{(d/2)-1} e^{-u/2}}{2^{d/2} \Gamma(d/2)}$$

Let's make things a bit tidy here by defining:

$$T = \frac{Z}{\sqrt{\frac{U}{d}}} = \frac{Z}{\sqrt{V}}$$

where $V = \frac{U}{d}$.

We now have to find PDF of T which a joint PDF if we assume that Z and V are independent. The joint PDF of Z and V , $f_{Z,V}(z, v)$, is the product of their marginal PDFs:

$$f_{Z,V}(z, v) = f_Z(z) f_V(v) = \left(\frac{1}{\sqrt{2\pi}} e^{-\frac{z^2}{2}} \right) \left(\frac{d^{d/2} v^{d/2-1} e^{-dv/2}}{2^{d/2} \Gamma(d/2)} \right)$$

Directly working with $f_{Z,V}(z, v)$ to find the PDF of T involves solving an integral where T is a function of both Z and V . Instead we are going to write things in terms of T using change variables method.

Let's start with

$$Z = T\sqrt{W}, \quad V = W$$

where W is just a placeholder to help in calculation. This is an easier form to work with but the problem is when working with transformation of random variables, the volume that the probability density covers can change. In order to conserve the total probability between the original and the transformed space we need to derive how much we have to stretch or compress the transformed space. determinant of Jacobian gives us a way to calculate this scaling factor.

Given:

$$Z = T\sqrt{W}, \quad V = W$$

To find the Jacobian determinant, we compute the partial derivatives:

$$\begin{aligned}\frac{\partial Z}{\partial T} &= \sqrt{W} \\ \frac{\partial Z}{\partial W} &= \frac{T}{2\sqrt{W}} \\ \frac{\partial V}{\partial T} &= 0 \\ \frac{\partial V}{\partial W} &= 1\end{aligned}$$

These partial derivatives form the Jacobian matrix:

$$J = \begin{bmatrix} \sqrt{W} & \frac{T}{2\sqrt{W}} \\ 0 & 1 \end{bmatrix}$$

The determinant of this matrix is:

$$\det(J) = \sqrt{W} \times 1 - 0 \times \frac{T}{2\sqrt{W}} = \sqrt{W}$$

Using the Jacobian, we can transform the joint PDF of Z and V into the joint PDF of T and W :

$$f_{T,W}(t, w) = f_{Z,V}(t\sqrt{w}, w) \times |\det(J)|$$

Given $f_{Z,V}(z, v) = f_Z(z)f_V(v)$, we substitute $z = t\sqrt{w}$ and $v = w$, and include the Jacobian:

$$f_{T,W}(t, w) = f_Z(t\sqrt{w})f_V(w)\sqrt{w}$$

which becomes

$$f_{T,W}(t, w) = \left(\frac{1}{\sqrt{2\pi}} e^{-\frac{t^2 w}{2}} \right) \left(\frac{d^{d/2} w^{d/2-1} e^{-dw/2}}{2^{d/2} \Gamma(d/2)} \right) \cdot \sqrt{w}$$

Finally, to find $f_T(t)$, the marginal PDF of T , we integrate out W :

$$f_T(t) = \int_0^\infty f_{T,W}(t, w) dw$$

I'm not going to solve this here but the results of this integration is

$$f_T(t) = \frac{\Gamma\left(\frac{d+1}{2}\right)}{\sqrt{d\pi}\Gamma\left(\frac{d}{2}\right)} \left(1 + \frac{t^2}{d}\right)^{-\frac{d+1}{2}}$$

This gives us the Student's t-distribution, matching the formula for the PDF of the t-distribution.

We came all the way here to just to prove that

$$\frac{Z}{\sqrt{\frac{U}{d}}} \sim t_d$$

with d degrees of freedom and $Z \sim N(0, 1)$ and $U \sim \text{chi}_d^2$.

28.4 t-statistics for OLS

Consider a linear model given by:

$$Y = X\beta + \epsilon$$

In order to do a t-statistics, we need to first create a hypothesis. To create such a hypothesis we need a **linear contrast** that is a specific linear combination of the estimated regression coefficients. It's typically represented as $c'\beta$

where:

- c is a known $p \times 1$ vector, specifying the coefficients for the linear combination.
- β is the $p \times 1$ vector of true regression coefficients.

The purpose of c is to focus on specific hypotheses about combinations of the parameters β . For example, if we are interested in testing whether the sum of two coefficients is zero or some other value, c would be constructed accordingly to weight these coefficients. Similarly, if we have a bunch coefficients and we want to test whether one of them is different from zero or not we can construct a $p \times 1$ vector which has all the elements zero except for the coefficient that we want to test.

$c'\beta$ in fact gives us the numerator of the t-statistics. So we have

$$t = \frac{c'\beta}{SE}$$

We still need to figure out what SE is.

Recall that SE is the standard error of $c'\beta$. So we need to derive the variance of the estimated regression coefficients ($\hat{\beta}$). Although we have done that before, let's go through it one more time.

The variance of a linear transformation A of a random vector b (where A is a matrix and b is a vector of random variables with variance-covariance matrix Σ) is given by:

$$\text{Var}(Ab) = A\Sigma A'$$

Applying this to $\hat{\beta}$:

$$\hat{\beta} = (X'X)^{-1}X'Y$$

$$Y = X\beta + \epsilon$$

$$\text{Var}(\hat{\beta}) = \text{Var}((X'X)^{-1}X'Y)$$

Since $Y = X\beta + \epsilon$ and $X\beta$ is non-random (deterministic), the variance part comes only from ϵ :

$$\text{Var}(\hat{\beta}) = \text{Var}((X'X)^{-1}X'(X\beta + \epsilon))$$

$$\text{Var}(\hat{\beta}) = \text{Var}((X'X)^{-1}X'\epsilon)$$

Given $\text{Var}(\epsilon) = \sigma^2 I$:

$$\text{Var}(\hat{\beta}) = (X'X)^{-1}X'(\sigma^2 I)X(X'X)^{-1}$$

$$\text{Var}(\hat{\beta}) = \sigma^2(X'X)^{-1}X'IX(X'X)^{-1}$$

$$\text{Var}(\hat{\beta}) = \sigma^2(X'X)^{-1}$$

The unbiased estimator of σ^2 (variance of the errors) is given by the Residual Sum of Squares (RSS) divided by the degrees of freedom, which is typically the number of observations minus the number of parameters estimated (including intercept). If n is the number of observations and p is the number of parameters (including the intercept), the degrees of freedom are $n - p$:

$$\hat{\sigma}^2 = \frac{\text{RSS}}{n - p}$$

$$\text{RSS} = (Y - \hat{Y})'(Y - \hat{Y})$$

Given:

$$\hat{Y} = X\hat{\beta} = X(X'X)^{-1}X'Y$$

The RSS becomes:

$$\text{RSS} = (Y - X(X'X)^{-1}X'Y)'(Y - X(X'X)^{-1}X'Y)$$

$$\text{RSS} = Y'Y - Y'X(X'X)^{-1}X'Y$$

Given this the formula for t-statistics for testing coefficients of linear regression will be

$$t = \frac{c'\beta}{\frac{Y'Y - Y'X(X'X)^{-1}X'Y}{n-p} \sqrt{(X'X)^{-1}}}$$

```
calculate_t_statistic <- function(X, Y, c) {
  # Calculate the OLS estimates for beta
  beta_hat <- solve(t(X) %*% X) %*% t(X) %*% Y

  # Calculate RSS
  Y_hat <- X %*% beta_hat
  rss <- sum((Y - Y_hat)^2)

  # Degrees of freedom
```

```

n <- nrow(X) # Number of observations
p <- ncol(X) # Number of parameters
df <- n - p

# Variance estimate
sigma_squared_hat <- rss / df

# Calculating c'beta
c_beta_hat <- as.numeric(c %*% beta_hat)

# Calculating the variance of c'beta_hat
var_c_beta_hat <- sigma_squared_hat * (c %*% solve(t(X) %*% X)
↪ %*% c)

# t-statistic
t_statistic <- c_beta_hat / sqrt(var_c_beta_hat)

return(t_statistic)
}

# Construct the model matrix 'X' for the fixed effect 'x' using
↪ data from 'data_int'.
X <- model.matrix(~x, data=data_int)

y<-data_int$y

print(data.frame(t=calculate_t_statistic(X = X,Y = y,c =
↪ c(0,1)),df=nrow(X)-ncol(X),
  pvalue=2*pt(abs(calculate_t_statistic(X = X,Y = y,c =
↪ c(0,1))),df=nrow(X)-ncol(X),lower.tail = FALSE)))

```

```

      t df      pvalue
1 -6.932677 38 3.033548e-08

```

Compare to lm function

```
print(summary(lm(y~0+X)))
```

Call:

```
lm(formula = y ~ 0 + X)
```

Residuals:

```

      Min       1Q   Median       3Q      Max
-14.1384  -4.3018   0.0277   4.2264  17.5140

```

Coefficients:

```

              Estimate Std. Error t value Pr(>|t|)
X(Intercept) 120.1292     6.8039  17.656 < 2e-16 ***
Xx           -2.7184     0.3921  -6.933 3.03e-08 ***

```

Signif. codes: 0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

Residual standard error: 7.238 on 38 degrees of freedom

Multiple R-squared: 0.991, Adjusted R-squared: 0.9905

F-statistic: 2094 on 2 and 38 DF, p-value: < 2.2e-16

28.4.1 degrees of freedom of GLS

In mixed models, determining the degrees of freedom (df) is complex due to the presence of both fixed and random effects. Unlike simple linear models where df is straightforward (number of observations minus number of parameters), mixed models must account for the hierarchical structure of the data and correlations within random effects. Methods like Satterthwaite and Kenward-Roger approximations are often used to estimate df. We are again going to just use the definition of GLS and not $y = X\beta + Zb + \epsilon$ and demonstrate how Satterthwaite approximation works.

We know from the very first sections the estimated fixed effects $\hat{\beta}$ are given by:

$$\hat{\beta} = (X'V^{-1}X)^{-1}X'V^{-1}y$$

where $V = ZDZ' + \sigma^2$ is the variance-covariance matrix of y .

If you remember we also proved that the variance of the β is $(X'V^{-1}X)^{-1}$.

But what we are interested is the test of coefficient $c'\hat{\beta}$. From the OLS part above we can show that

$$\text{Var}(c'\hat{\beta}) = c'\text{Var}(\hat{\beta})c = c'(X'V^{-1}X)^{-1}c$$

The test statistic for testing $c'\beta = 0$ is constructed as:

$$T = \frac{c'\hat{\beta}}{\sqrt{\text{Var}(c'\hat{\beta})}}$$

Substitute the variance expression:

$$\text{Var}(c'\hat{\beta}) = c'(X'V^{-1}X)^{-1}c$$

So, the test statistic becomes:

$$T = \frac{c'\hat{\beta}}{\sqrt{c'(X'V^{-1}X)^{-1}c}}$$

This test statistic follows a t-distribution with degrees of freedom that can be approximated using the Satterthwaite approximation. However unlike OLS we don't want to use $n - p$ as degrees of freedom. We need to estimate it.

If you remember from the definition of T-stats we derived $\sqrt{c'(X'V^{-1}X)^{-1}c} \sim ch_d^2$ (we still don't know d).

However, just earlier we proved that:

$$\frac{(n-1)S^2}{\sigma^2} \sim \chi_{n-1}^2$$

Let's call the DF here $(n-1)$ is d so we have

$$\frac{(d)S^2}{\sigma^2} \sim \chi_d^2$$

We know the definition $S^2 = c'(X'V^{-1}X)^{-1}c$. But σ^2 is true but unknown variance of the population. Let's call this $\sigma^2 = c'(X'V_p^{-1}X)^{-1}c$

We can now say that $S^2 \approx \frac{\sigma^2}{d} \chi_d^2$. Given this we are going to match the first two moments of S^2 and $\frac{\sigma^2}{d} \chi_d^2$ to see if we can derive d

Let's start with expectation (first moment)

$$\mathbb{E}(S^2) = \mathbb{E}\left(\frac{\sigma^2}{d} \chi_d^2\right) = \frac{\sigma^2}{d} \mathbb{E}(\chi_d^2) = \frac{\sigma^2}{d} d = \sigma^2$$

Unfortunately d did not stay in this moment. Let's go to the second moment (variance).

$$\text{Var}(S^2) = \text{Var}\left(\frac{\sigma^2}{d} \chi_d^2\right) = \left(\frac{\sigma^2}{d}\right)^2 \text{Var}(\chi_d^2) = \left(\frac{\sigma^2}{d}\right)^2 \cdot 2d = \frac{2\sigma^4}{d}$$

This is much better. We have d left in the formula. So we have

$$\text{Var}(S^2) = \frac{2\sigma^4}{d}$$

We can write:

$$d = \frac{2\sigma^4}{\text{Var}(S^2)}$$

We can estimate σ^2 as $c'(X'V^{-1}X)^{-1}c$ which is S^2 . so we can write

$$d = \frac{2(c'(X'V^{-1}X)^{-1}c)^2}{\text{Var}(c'(X'V^{-1}X)^{-1}c)}$$

The numerator is simple to calculate but denominator is more complex.

Remember that $c'(X'V^{-1}X)^{-1}c$ involves estimating V which itself is a function of a parameter θ . To fix this we are going to use the delta method. The delta method can be used to approximate the variance of a function of a random variable.

If $g(\theta)$ is a differentiable function of θ , and θ has an approximate normal distribution $N(\hat{\theta}, \Sigma)$ where Σ is the covariance matrix of θ , the variance of $g(\theta)$ can be approximated as:

$$\text{Var}(g(\theta)) \approx \nabla g(\hat{\theta})' \Sigma \nabla g(\hat{\theta})$$

where $\nabla g(\hat{\theta})$ is the gradient (vector of first partial derivatives) of g evaluated at $\hat{\theta}$ and $g(\theta) = c'(X'V^{-1}X)^{-1}c$.

Therefore our final df becomes

$$d = \frac{2g(\theta)}{g(\hat{\theta})' \Sigma \nabla g(\hat{\theta})}$$

We need to compute $\nabla g(\theta)$, the gradient of g with respect to θ . This involves differentiating $c'(X'V^{-1}X)^{-1}c$ with respect to each element of θ . We are just going to estimate this here but in practice this can be found using Jacobian. and Σ is going to be estimated using the Hessian of the log-likelihood. Let's see how we can implement this

```
library(numDeriv)

simulate_grouped_trend <- function(group_count = 5,
  ↪ points_per_group = 10, global_slope = -10, global_intercept =
  ↪ 30, group_slope = 2, noise_sd = 50, noise_sd2=2) {
  set.seed(123) # Setting a seed for reproducibility

  # Initialize an empty data frame to store the simulated data
  data <- data.frame(x = numeric(), y = numeric())

  # Loop to create each group
  for (i in 1:group_count) {
    x_start <- 12 + (i - 1) * (10 / group_count) # Stagger the
    ↪ start of x for each group
    x <- runif(points_per_group, min = x_start, max = x_start +
    ↪ (10 / group_count))

    # Apply a local positive trend within the group, but maintain
    ↪ the global negative trend
    local_intercept <- global_intercept + global_slope * (x_start
    ↪ + (10 / (2 * group_count))) + rnorm(1, mean = 0, sd =
    ↪ noise_sd)
    y <- local_intercept + group_slope[i] * (x - x_start) +
    ↪ rnorm(points_per_group, mean = 0, sd = noise_sd2)

    # Combine this group with the overall dataset
    group_data <- data.frame(x = x, y = y, group=i)
    data <- rbind(data, group_data)
```



```

}

return(data)
}

# generate simulated data
data_int <- simulate_grouped_trend(group_count =
  ↪ 4, points_per_group = 10, global_slope = -2, global_intercept =
  ↪ 100, group_slope = c(6, 4, 2, 1), noise_sd = 5, noise_sd2 = 2)

# set group to factor
data_int$group <- factor(data_int$group)

# Design matrix of random effect
Z <- model.matrix(~0+data_int$group)

# Construct the model matrix 'X' for the fixed effect 'x' using
  ↪ data from 'data_int'.
X <- model.matrix(~x, data=data_int)

# get groups (random effects)
groups <- unique(data_int$group)

y <- data_int$y

c <- matrix(c(0, 1), ncol = 2)

# Define the log-likelihood function (as negative log-likelihood)
log_reml <- function(theta) {
  D <- diag(x = theta[2], ncol = length(groups), nrow =
  ↪ length(groups))
  I <- diag(1, nrow = nrow(X), ncol = nrow(X))
  V <- theta[1] * (I + Z %*% D %*% t(Z))
  Beta <- solve(t(X) %*% solve(V) %*% X) %*% t(X) %*% solve(V)
  ↪ %*% y
  return(c(log(det(V)) + log(det(t(X) %*% solve(V) %*% X)) + t(y
  ↪ - X %*% Beta) %*% solve(V) %*% (y - X %*% Beta)))
}

theta <- optimization::optim_nm(fun = log_reml, k = 2, start =
  ↪ c(1, 1), maximum = FALSE, tol = 0.000000000001)$par

theta <- matrix(theta, ncol = 2)
# Compute the Hessian at the estimates
hessian_matrix <- hessian(log_reml, theta)
Sigma <- 2*solve(hessian_matrix)

# define g matrix

```

```

g <- function(theta) {
  D <- diag(x = theta[2], ncol = length(groups), nrow =
    ↪ length(groups))
  I <- diag(1, nrow = nrow(X), ncol = nrow(X))
  V <- theta[1] * (I + Z %*% D %*% t(Z))
  return((c) %*% solve(t(X) %*% solve(V) %*% X) %*% t(c))
}

# Calculate the gradient of a function by numerical approximation
g_grad<-as.matrix(grad(g,theta))
var_S2<-t(g_grad)%*%Sigma%*%g_grad
S2<-g(theta)
d <- (2 * S2^2) / var_S2

D <- diag(x = theta[2],ncol = length(groups),nrow =
  ↪ length(groups))
I <- diag(1,nrow = nrow(X),ncol = nrow(X))
V<-theta[1]*(I+Z%*%D%*%t(Z))
Beta <- solve(t(X) %*% solve(V) %*% X) %*% t(X) %*% solve(V) %*%
  ↪ y

print(data.frame(df=d,t=Beta[2]/sqrt(S2),pvalue=2*pt(Beta[2]/sqrt(S2),df=d,lower.tail
  ↪ = FALSE)))

```

```

      df      t      pvalue
1 36.7423 4.33864 0.000107513

```

Compare this to `lmerTest` results:

```

model<-lmer(y~x+(1|group),data=data_int,REML = TRUE)
library(emmeans)

```

Welcome to emmeans.

Caution: You lose important information if you filter this package's results.
See '? untidy'

```

trends <- emtrends(model, var = "x", lmer.df = "satterthwaite")
print(summary(trends, infer = TRUE))

```

```

      x x.trend    SE   df lower.CL upper.CL t.ratio p.value
17.1    2.56 0.589 36.7    1.36    3.75    4.339 0.0001

```

Degrees-of-freedom method: satterthwaite

Confidence level used: 0.95

We get similar results and this concludes our chapter on math behind mixed models.

Chapter 29

Summary

In this section, we went through how mixed models works starting from OLS all the way to GLS and estimation and hypothesis testing. Please note that some of the derivations are very in efficient way of solving the equations in practice there are highly optimized methods to solve the mixed models.

Chapter 30

Exercises: Mixed Effects Models

Mixed-effects models are powerful statistical tools for analyzing data with hierarchical or grouped structures. In these exercises, we will explore the use of mixed models in R.

You will use the `lme4` package for fitting mixed models, so make sure it is installed. In the examples we also use `tidyverse` and `ggplot2` for data manipulation and visualization, but you are welcome to use other alternatives.

```
library(lme4)
library(tidyverse)
library(ggplot2)
```

30.1 Random intercepts and random slopes

In these exercises we cover both models with **random intercepts** and **random slopes** using the `lme4` package in R.

Exercise 30.1 (Drug effect on blood pressure). A study investigates how a drug affects blood pressure. A total of 200 patients are recruited from 10 hospitals. We suspect that average blood pressure differs between hospitals, mainly due to different measurement routines, but we expect the drug effect to be consistent across hospitals.

```
## Generate the data
databp <- data.frame(hospital = factor(rep(1:10, each = 20)),
  drug = rep(c("placebo", "treatment"), times = 100),
  bp =
    ↪ c(123.9, 114.6, 119.8, 113.3, 115.0, 121.7, 120.2, 102.9, 121.3, 110.4, 112.4, 111.7, 112.6, 109.1, 114.5,
    )
```

- Visualize the data using, for example, boxplots or jitter plots, both overall and by hospital. Based on the plots, do you think there is variability in

blood pressure across hospitals that should be accounted for?

💡 Solution

```
## Blood pressure vs drug treatment, treat each measurement
  ↪ as independent
databp %>%
  ggplot(aes(x = drug, y = bp)) +
    geom_boxplot() +
    geom_jitter(width = 0.2, alpha = 0.5) +
    theme_bw()
```

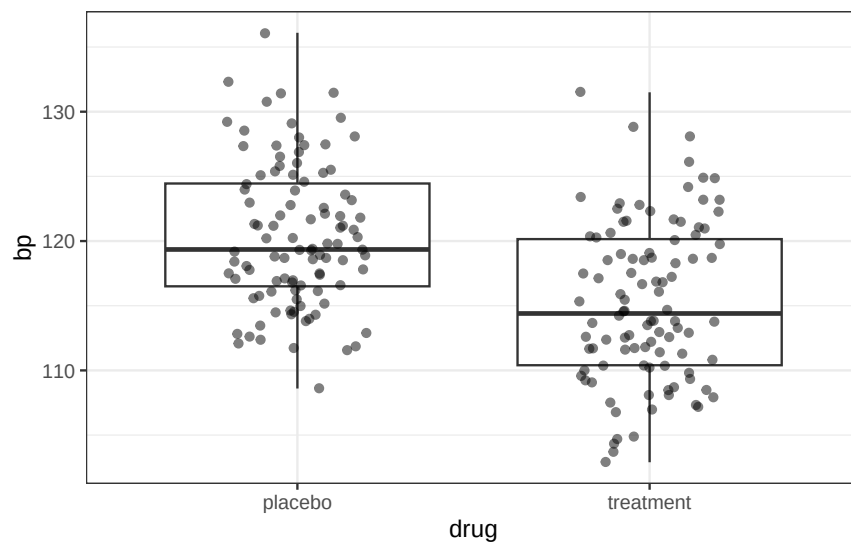


Figure 30.1: Blood pressure vs drug treatment, overall

```
databp |>
  ggplot(aes(x = hospital, y = bp, color = drug)) +
    geom_boxplot(outlier.shape = NA) +
    geom_jitter(width = 0.2, alpha = 0.5) +
    theme_bw()
```

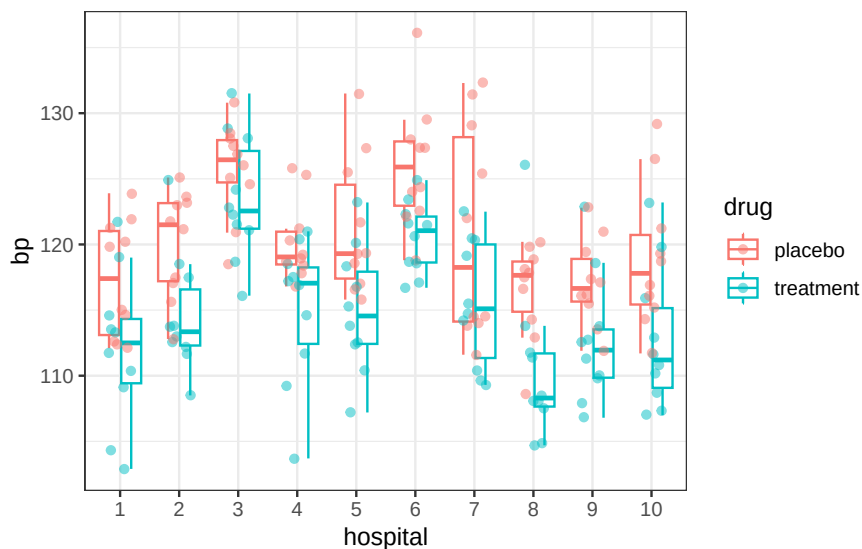


Figure 30.2: Blood pressure vs hospital and drug treatment

Yes, the plots suggest that there is variability in blood pressure across hospitals. The hospital-specific boxplots show different baseline levels per hospital, which indicates that the grouping structure should be taken into account.

- b. If we ignore the grouping structure, we can fit an ordinary linear regression model. Fit a linear regression model with blood pressure as the response and drug treatment as the independent variable. According to this model, is the average blood pressure higher in the treatment group or in the placebo group? By how much does the drug change blood pressure on average?

Solution

```
lm_model <- lm(bp ~ drug, data = databp)
summary(lm_model)
```

Call:

```
lm(formula = bp ~ drug, data = databp)
```

Residuals:

	Min	1Q	Median	3Q	Max
Residuals	-12.329	-4.302	-1.016	4.603	16.271

Coefficients:

	Estimate	Std. Error	t value	Pr(> t)
(Intercept)	120.4020	0.5835	206.338	< 2e-16 ***
drugtreatment	-5.1730	0.8252	-6.269	2.24e-09 ***

```

---
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

Residual standard error: 5.835 on 198 degrees of freedom
Multiple R-squared:  0.1656,    Adjusted R-squared:  0.1614 
F-statistic: 39.3 on 1 and 198 DF,  p-value: 2.236e-09
The average blood pressure is lower in the treatment group than in the
placebo group. The estimated treatment effect is approximately -5.17,
meaning that the treatment reduces blood pressure by about 5.17 units
on average.

```

c. Is this linear model valid given the data structure? Why or why not?

Solution

No, not really. The model treats all observations as independent, but patients from the same hospital may be more similar to each other than patients from different hospitals. This violates the independence assumption and may lead to incorrect standard errors and inference.

d. To account for variability in blood pressure across hospitals, fit a mixed model with a random intercept for hospital. Random intercepts allow each hospital to have its own baseline blood pressure. Investigate the model using the functions `summary`, `VarCorr`, and `fixef`.

Solution

```

model1 <- lmer(bp ~ drug + (1 | hospital), data = databp)
summary(model1)

```

Linear mixed model fit by REML ['lmerMod']

Formula: `bp ~ drug + (1 | hospital)`

Data: `databp`

REML criterion at convergence: 1215.7

Scaled residuals:

	Min	1Q	Median	3Q	Max
	-2.36147	-0.65012	-0.08265	0.56163	3.07473

Random effects:

Groups	Name	Variance	Std.Dev.
--------	------	----------	----------

hospital	(Intercept)	11.90	3.449
----------	-------------	-------	-------

	Residual	23.23	4.820
--	----------	-------	-------

Number of obs: 200, groups: hospital, 10

Fixed effects:

	Estimate	Std. Error	t value
--	----------	------------	---------


```
(Intercept) 120.4020 1.1925 100.968
drugtreatment -5.1730 0.6817 -7.589
```

```
Correlation of Fixed Effects:
      (Intr)
drugtremnt -0.286
```

```
VarCorr(model1)
```

```
Groups   Name             Std.Dev.
hospital (Intercept) 3.4492
Residual                      4.8202
```

```
fixef(model1)
```

```
(Intercept) drugtreatment
120.402      -5.173
```

`VarCorr()` gives the variance components, including the variance of the random intercept for hospital. `fixef()` extracts the estimated fixed effects.

- e. Based on the mixed model results, what is the estimated effect of the drug treatment on blood pressure?

Solution

The estimated fixed effect of treatment is about -5.17, so the drug is estimated to lower blood pressure by approximately 5.17 units on average.

- f. Based on the mixed model, how much variability in blood pressure is due to differences between hospitals?

Solution

We can quantify the between-hospital variability using the variance components and compute the **intraclass correlation coefficient (ICC)**:

```
vc <- as.data.frame(VarCorr(model1))
vc
```

```
      grp      var1 var2    vcov    sdcov
1 hospital (Intercept) <NA> 11.89669 3.449158
2 Residual          <NA> <NA> 23.23435 4.820202
```

```
icc <- vc$vcov[1] / (vc$vcov[1] + vc$vcov[2])
icc
```

```
[1] 0.3386376
```

The ICC gives the proportion of total variance attributable to differences between hospitals.

- g. Based on the estimated between-hospital variability, do you think it is appropriate to use an ordinary linear regression model instead of a mixed

model? Why or why not?

💡 Solution

The estimated between-hospital variability is substantial, and the ICC is clearly above 0. This means that a non-negligible part of the total variation is due to hospital-level differences. Hence, it is more appropriate to use a mixed model, since ordinary linear regression ignores this dependence structure.

- h. The random intercepts for each hospital can be obtained using the `ranef()` function. This gives the deviation of each hospital from the overall intercept.

Compute the random effects using `ranef()`. What is the estimated average blood pressure at hospital 3 for a patient receiving the placebo?

💡 Solution

```
## Note that `ranef` gives the deviations from the overall  
↪ mean for each hospital.
```

```
ranef(model1)
```

```
$hospital  
(Intercept)  
1   -2.8246697  
2   -0.4104221  
3    6.1809293  
4   -0.1462214  
5    0.2865211  
6    4.9829159  
7    0.3821799  
8   -3.9498001  
9   -2.6880141  
10  -1.8134188
```

with conditional variances for "hospital"

```
## The average blood pressure at a specific hospital is the  
↪ sum of the fixed and random effects
```

```
fixef(model1)[1] + ranef(model1)$hospital[3,1]
```

```
(Intercept)  
126.5829
```

This gives the estimated mean blood pressure for placebo patients at hospital 3.

Exercise 30.2 (Sleep study). The `sleepstudy` dataset from the `lme4` package contains reaction times for subjects measured over several days of sleep deprivation.

Days 0 and 1 correspond to adaptation and training, and will be excluded. Day

2 is the baseline day, but for simplicity we rename it to 0.

We will initially work with a subset containing only 5 subjects.

```
##Remove the first two days (before baseline) and adjust Days to
  ↪ be 0 at baseline.
sleepall <- sleepstudy |> filter(Days>=2) |> mutate(Days=Days-2)
##Select a subset of 5 individuals
sleep <- sleepstudy |> filter(Subject %in% c(333, 334, 369, 349,
  ↪ 372), Days>=2) |> mutate(Days=Days-2)
```

- a. Investigate the dataset. What columns are available? Plot the data, e.g. using `ggplot2` and add a linear regression line to visualize the relationship between reaction time and days of sleep deprivation, both overall and separately per subject.

Solution

```
## Print the first 12 rows
head(sleep, 12)
```

	Reaction	Days	Subject
1	276.7693	0	333
2	299.8097	1	333
3	297.1710	2	333
4	338.1665	3	333
5	332.0265	4	333
6	348.8399	5	333
7	333.3600	6	333
8	362.0428	7	333
9	243.3647	0	334
10	254.6723	1	334
11	279.0244	2	334
12	284.1912	3	334

```
##Plot Reaction time vs days of sleep deprivation, treat all
  ↪ measurements as independent
sleep |> ggplot(aes(x=Days, y=Reaction)) + geom_point() +
  ↪ geom_smooth(method="lm", se=FALSE) + theme_bw() +
  labs(x = "Days of sleep deprivation",
       y = "Reaction time (ms)")
```

``geom_smooth()`` using formula = 'y ~ x'

```
##Plot Reaction time vs days of sleep deprivation, treating
  ↪ each subject as a separate group
sleep |> ggplot(aes(x=Days, y=Reaction, color=Subject)) +
  ↪ geom_point() + geom_smooth(method="lm", se=FALSE) +
  theme_bw() + theme(legend.position = "none") +
  labs(x = "Days of sleep deprivation",
       y = "Reaction time (ms)")
```

```
`geom_smooth()` using formula = 'y ~ x'
```

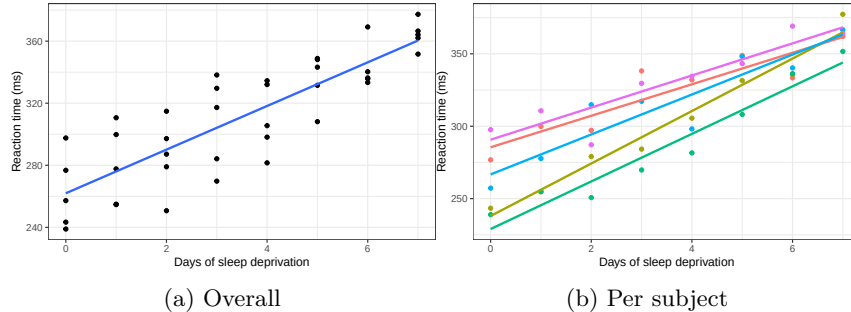


Figure 30.3: Reaction time vs Days of sleep deprivation

b. Fit a linear model to the data, ignoring individual differences.

According to this model, by how many milliseconds does reaction time increase per day of sleep deprivation?

Solution

```
lm_sleep <- lm(Reaction ~ Days, data = sleep)
summary(lm_sleep)
```

Call:

```
lm(formula = Reaction ~ Days, data = sleep)
```

Residuals:

Min	1Q	Median	3Q	Max
-39.385	-14.384	0.434	15.952	35.621

Coefficients:

	Estimate	Std. Error	t value	Pr(> t)
(Intercept)	261.976	5.905	44.36	< 2e-16 ***
Days	14.060	1.412	9.96	3.82e-12 ***

Signif. codes: 0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

Residual standard error: 20.46 on 38 degrees of freedom

Multiple R-squared: 0.723, Adjusted R-squared: 0.7157

F-statistic: 99.2 on 1 and 38 DF, p-value: 3.816e-12

The estimated slope is approximately 14.06, so reaction time increases by about 14.06 milliseconds per day of sleep deprivation.

c. Fit a mixed model with a random intercept for each subject to account for individual differences in baseline reaction time.

According to this model, by how many milliseconds does reaction time increase per day of sleep deprivation?

💡 Solution

```
mm1_sleep <- lmer(Reaction ~ Days + (1 | Subject), data =
  ↳ sleep)
summary(mm1_sleep)
```

Linear mixed model fit by REML ['lmerMod']

Formula: Reaction ~ Days + (1 | Subject)

Data: sleep

REML criterion at convergence: 325.3

Scaled residuals:

	Min	1Q	Median	3Q	Max
	-1.79871	-0.71049	-0.07413	0.64819	1.91336

Random effects:

Groups	Name	Variance	Std.Dev.
Subject	(Intercept)	278.9	16.70
Residual		183.6	13.55

Number of obs: 40, groups: Subject, 5

Fixed effects:

	Estimate	Std. Error	t value
(Intercept)	261.9758	8.4307	31.07
Days	14.0599	0.9351	15.04

Correlation of Fixed Effects:

(Intr)

Days -0.388

The fixed effect of `Days` gives the average increase in reaction time per day. In this subset, it is approximately the same as in the ordinary linear model, around 14.06 ms/day.

- d. How much variability in reaction time is due to differences between subjects?

💡 Solution

```
VarCorr(mm1_sleep)
```

Groups	Name	Std.Dev.
Subject	(Intercept)	16.699
Residual		13.551

The random-intercept variance from `VarCorr(mm1_sleep)` gives the between-subject variability in baseline reaction times. You can also compute an ICC if you want the proportion of total variance attributable to subject differences.

- e. Now include the full dataset and repeat the analysis: visualize the data and fit a mixed model with a random intercept for subject.

What is the estimated fixed effect of sleep deprivation on reaction time in the full dataset?

What is the estimated variability in reaction time due to subject differences?

💡 Solution

```
sleepall |> ggplot(aes(x=Days, y=Reaction, color=Subject)) +
  ↪ geom_point() + geom_smooth(method="lm", se=FALSE) +
  theme_bw() + theme(legend.position = "none") +
  labs(x = "Days of sleep deprivation",
       y = "Reaction time (ms)")
```

`geom\_smooth()` using formula = 'y ~ x'

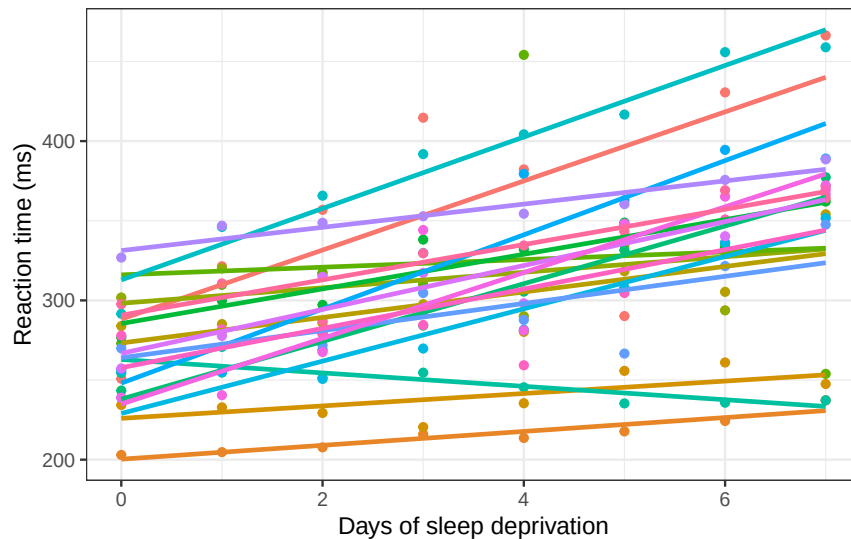


Figure 30.4: Reaction time vs Days of sleep deprivation

```
mm_sleepall <- lmer(Reaction ~ Days + (1 | Subject), data =
  ↪ sleepall)
summary(mm_sleepall)
```

Linear mixed model fit by REML ['lmerMod']
 Formula: Reaction ~ Days + (1 | Subject)
 Data: sleepall

REML criterion at convergence: 1430

Scaled residuals:

Min	1Q	Median	3Q	Max
-----	----	--------	----	-----

```
-3.6261 -0.4450 0.0474 0.5199 4.1378
```

Random effects:

Groups	Name	Variance	Std.Dev.
Subject	(Intercept)	1746.9	41.80
Residual		913.1	30.22

Number of obs: 144, groups: Subject, 18

Fixed effects:

	Estimate	Std. Error	t value
(Intercept)	267.967	10.871	24.65
Days	11.435	1.099	10.40

Correlation of Fixed Effects:

(Intr)
Days -0.354

`VarCorr(mm_sleepall)`

Groups	Name	Std.Dev.
Subject	(Intercept)	41.796
Residual		30.217

The fixed effect of **Days** gives the average increase in reaction time per day in the full dataset. The random-intercept variance gives the variability in baseline reaction time due to differences between subjects.

- f. The plot suggests that the slope may also vary between subjects, i.e., some subjects are more affected by sleep deprivation than others. Investigate this further by including a random slope for **Days** within **Subject**.

Solution

```
mm2_sleepall <- lmer(Reaction ~ Days + (Days | Subject),
  ↪ data = sleepall)
summary(mm2_sleepall)
```

Linear mixed model fit by REML ['lmerMod']
Formula: Reaction ~ Days + (Days | Subject)
Data: sleepall

REML criterion at convergence: 1404.1

Scaled residuals:

Min	1Q	Median	3Q	Max
-4.0157	-0.3541	0.0069	0.4681	5.0732

Random effects:

Groups	Name	Variance	Std.Dev.	Corr
Subject	(Intercept)	958.35	30.957	

```

          Days          45.78    6.766    0.18
Residual          651.60    25.526
Number of obs: 144, groups: Subject, 18

```

Fixed effects:

```

          Estimate Std. Error t value
(Intercept) 267.967    8.266  32.418
Days        11.435    1.845   6.197

```

Correlation of Fixed Effects:

```

(Intr)
Days -0.062

```

```
VarCorr(mm2_sleepall)
```

```

Groups   Name          Std.Dev. Corr
Subject (Intercept) 30.9573
          Days        6.7659  0.178
Residual                25.5264

```

Based on the model with a random slope, what is the average effect of sleep deprivation on reaction time? How does it compare with the model with only a random intercept?

Compute the random effects for the random-slope model using `ranef(mm2_sleepall)`. What is the estimated effect of sleep deprivation on reaction time for subject 309?

Solution

```
ranef(mm2_sleepall)$Subject
```

```

      (Intercept)      Days
308 24.4992891    8.602000
309 -59.3723102   -8.127753
310 -39.4762764   -7.429236
330  1.3500428   -2.384597
331 18.4576169   -3.747734
332 30.5270040   -4.893689
333 13.3682027    0.288863
334 -18.1583020    3.843668
335 -16.9737887  -12.070233
337 44.5850842   10.176083
349 -26.6839022    2.194669
350 -5.9657957    8.175861
351 -5.5710355   -2.371849
352 46.6347253   -0.561637
369  0.9616395    1.738513
370 -18.5216778    5.631753
371 -7.3431320    0.272928

```



```
372 17.6826159 0.6623897
```

```
fixef(mm2_sleepall)["Days"] +  
↪ ranef(mm2_sleepall)$Subject["309", "Days"]
```

Days

```
3.307675
```

The subject-specific slope for subject 309 is the sum of: - the fixed slope for Days, and - the random slope deviation for subject 309.

Exercise 30.3 (Orthodontic measurements over time). The `Orthodont` dataset from the `nlme` package contains orthodontic measurements on patients over time. The dataset includes measurements of the distance from the pituitary to the pterygomaxillary fissure (mm).

```
library(nlme)  
data("Orthodont", package = "nlme")
```

Use this dataset to study how the distance changes with age, while accounting for individual differences in growth patterns. Also, take any relevant covariates into account.

- Start by looking at the data. What columns are available?

Solution

```
summary(Orthodont)
```

	distance	age	Subject	Sex
Min.	:16.50	Min. : 8.0	M16 : 4	Male :64
1st Qu.	:22.00	1st Qu.: 9.5	M05 : 4	Female:44
Median	:23.75	Median :11.0	M02 : 4	
Mean	:24.02	Mean :11.0	M11 : 4	
3rd Qu.	:26.00	3rd Qu.:12.5	M07 : 4	
Max.	:31.50	Max. :14.0	M08 : 4	
			(Other):84	

```
head(Orthodont)
```

Grouped Data: distance ~ age | Subject

	distance	age	Subject	Sex
1	26.0	8	M01	Male
2	25.0	10	M01	Male
3	29.0	12	M01	Male
4	31.0	14	M01	Male
5	21.5	8	M02	Male
6	22.5	10	M02	Male

- We can visualize the data using `ggplot2`.

```
Orthodont |>
  ggplot(aes(x = age, y = distance, group = Subject, color =
    ↪ Sex)) +
  geom_line() +
  geom_point() +
  xlab("Age (years)") +
  ylab("Distance (mm)") +
  theme_bw()
```

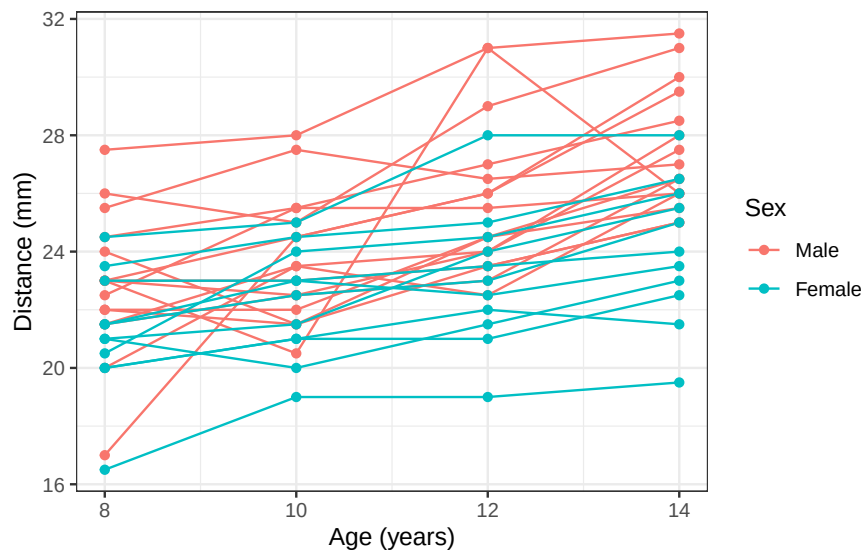


Figure 30.5

- b. Based on the plot and what you now know of the available variables, what grouping structure could be relevant to consider?

💡 Solution

The groups would be the individual subjects.

What covariates could be relevant to include in the model?

💡 Solution

The covariates could be **age** and **Sex**.

- c. Fit a linear model to the data, ignoring the grouping structure, but include any relevant covariates.

Then, fit a mixed model taking the grouping structure into account. Use a random intercept for **Subject** to account for individual differences in growth patterns.

💡 Solution

```
lmortho <- lm(distance ~ age + Sex, data=Orthodont)
summary(lmortho)
```

Call:

```
lm(formula = distance ~ age + Sex, data = Orthodont)
```

Residuals:

Min	1Q	Median	3Q	Max
-5.9882	-1.4882	-0.0586	1.1916	5.3711

Coefficients:

	Estimate	Std. Error	t value	Pr(> t)
(Intercept)	17.70671	1.11221	15.920	< 2e-16 ***
age	0.66019	0.09776	6.753	8.25e-10 ***
SexFemale	-2.32102	0.44489	-5.217	9.20e-07 ***

Signif. codes: 0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

Residual standard error: 2.272 on 105 degrees of freedom

Multiple R-squared: 0.4095, Adjusted R-squared: 0.3983

F-statistic: 36.41 on 2 and 105 DF, p-value: 9.726e-13

```
mmortho <- lmer(distance ~ age + Sex + (1 | Subject),
  data=Orthodont)
summary(mmortho)
```

Linear mixed model fit by REML ['lmerMod']

Formula: distance ~ age + Sex + (1 | Subject)

Data: Orthodont

REML criterion at convergence: 437.5

Scaled residuals:

Min	1Q	Median	3Q	Max
-3.7489	-0.5503	-0.0252	0.4534	3.6575

Random effects:

Groups	Name	Variance	Std.Dev.
Subject	(Intercept)	3.267	1.807
	Residual	2.049	1.432

Number of obs: 108, groups: Subject, 27

Fixed effects:

	Estimate	Std. Error	t value
(Intercept)	17.70671	0.83392	21.233
age	0.66019	0.06161	10.716
SexFemale	-2.32102	0.76142	-3.048

```
Correlation of Fixed Effects:
              (Intr) age
age          -0.813
SexFemale -0.372  0.000
```

```
VarCorr(mmortho)
```

Groups	Name	Std.Dev.
Subject	(Intercept)	1.8074
Residual		1.4316

- d. What is the estimated effect of age on distance in the linear model? How does it compare to the mixed model?

What is the estimated variability in distance due to differences between subjects? Does this suggest that the random intercept is needed in the model?

Hint

The estimated effect of age is given by the coefficient of **age** in each model. In many repeated-measures settings, the estimated slope is similar in the linear and mixed models, but the mixed model gives more appropriate standard errors because it accounts for the dependence within subject. The estimated variability due to differences between subjects is given by the random-intercept variance in `VarCorr(mmortho)`. If this variance is clearly larger than 0, that suggests that a random intercept is needed.

These exercises focus on more complex grouped structures with more than one grouping level as well as statistical tests for mixed models.

30.2 Nested design

Exercise 30.4 (Mice in litters). We simulate data for mice nested within litters, which are themselves nested within breeding lines. Our aim is to evaluate the effect of a drug on mouse weight while accounting for the hierarchical grouping structure.

```
## Data simulation
set.seed(42)

# Parameters
n_lines <- 5
n_litters_per_line <- 4
n_mice_per_litter <- 6

# Create data
nested_data <- expand_grid(
  line = factor(1:n_lines),
  litter = factor(1:n_litters_per_line),
  mouse = 1:n_mice_per_litter
```

```

) %>%
  mutate(
    litter_id = factor(paste(line, litter, sep=".")),
    drug = sample(c("control", "treated"), n(), replace = TRUE),
    line_effect = rnorm(n_lines, 0, 1)[as.integer(line)],
    litter_effect = rnorm(n_lines * n_litters_per_line, 0,
      ↪ 0.8)[as.integer(litter_id)],
    residual = rnorm(n(), 0, 1),
    drug_effect = ifelse(drug == "treated", -2, 0),
    weight = 25 + drug_effect + line_effect + litter_effect +
      ↪ residual
  )

```

Visualize the data, e.g. using boxplots and jitter plots, both overall and showing the line and litter.

Solution

```

ggplot(nested_data, aes(x = drug, y = weight)) +
  geom_boxplot() +
  geom_jitter() +
  theme_bw()

```

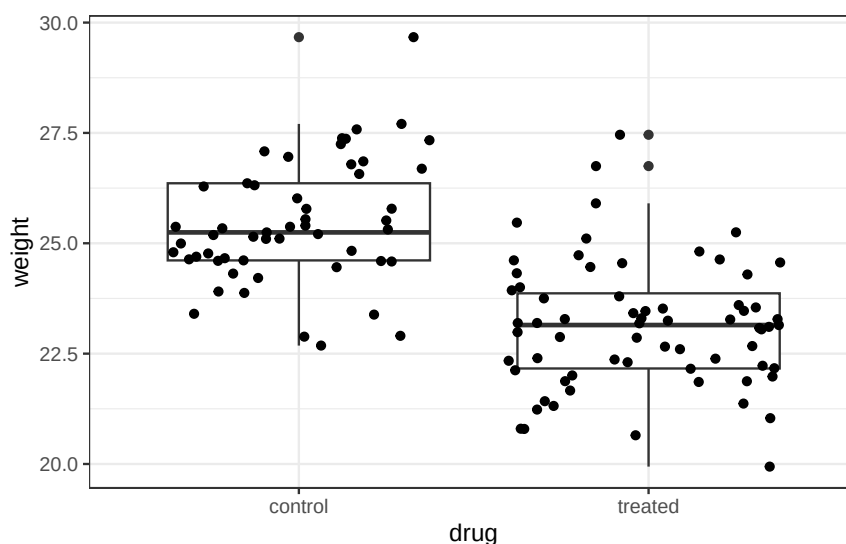


Figure 30.6: Weight by Drug Treatment

```
ggplot(nested_data, aes(x = litter_id, y = weight, color =
  ↪ drug, shape=line)) +
  geom_jitter() +
  labs(title = "Weight by Litter", x = "Line.Litter") +
  theme_bw()
```

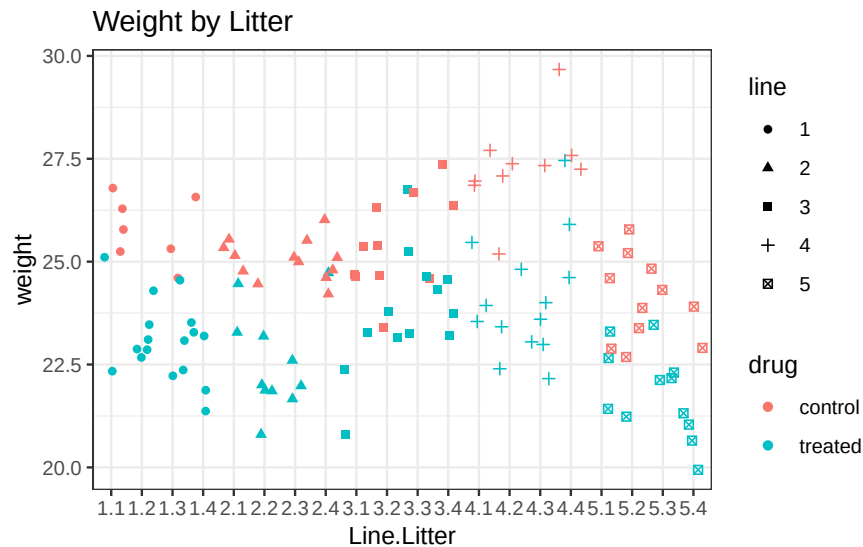


Figure 30.7: Weight by Litter and Drug Treatment

Fit three different models:

1. A linear regression model ignoring the grouping structure.
2. A linear mixed model with a random intercept for litter nested within line.
3. A linear mixed model with a random intercept and random slope for drug treatment for litters nested within lines.

 Solution

```
# Model 1: Ignore nesting
m0 <- lm(weight ~ drug, data = nested_data)

# Model 2: Random intercept for litter (nested within line)
m1 <- lmer(weight ~ drug + (1 | line/litter_id), data =
  ↪ nested_data)
# Compare to this notation
m1_alt <- lmer(weight ~ drug + (1 | line) + (1 |
  ↪ litter_id:line), data = nested_data)
# or
m1_alt2 <- lmer(weight ~ drug + (1 | line) + (1 |
  ↪ litter_id), data = nested_data)

# Model 3: Add random slope for drug per litter within line
m2 <- lmer(weight ~ drug + (1 + drug | line/litter), data =
  ↪ nested_data)

boundary (singular) fit: see help('isSingular')
```

Use `summary()` to study the different models and `anova()` to compare the linear mixed models with different random-effects structures.

 Solution

```
summary(m0)
```

Call:

```
lm(formula = weight ~ drug, data = nested_data)
```

Residuals:

	Min	1Q	Median	3Q	Max
	-3.1747	-0.8468	-0.0684	0.8763	4.3433

Coefficients:

	Estimate	Std. Error	t value	Pr(> t)
(Intercept)	25.4423	0.1947	130.65	< 2e-16 ***
drugtreated	-2.3272	0.2606	-8.93	6.75e-15 ***

Signif. codes: 0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

Residual standard error: 1.418 on 118 degrees of freedom

Multiple R-squared: 0.4032, Adjusted R-squared: 0.3982

F-statistic: 79.74 on 1 and 118 DF, p-value: 6.745e-15

```
summary(m1)
```

Linear mixed model fit by REML ['lmerMod']

```
Formula: weight ~ drug + (1 | line/litter_id)
Data: nested_data
```

REML criterion at convergence: 365.4

Scaled residuals:

	Min	1Q	Median	3Q	Max
	-2.06105	-0.62779	-0.05877	0.59189	2.72304

Random effects:

Groups	Name	Variance	Std.Dev.
litter_id:line	(Intercept)	0.4566	0.6757
	line	0.8129	0.9016
Residual		0.9092	0.9535

Number of obs: 120, groups: litter_id:line, 20; line, 5

Fixed effects:

	Estimate	Std. Error	t value
(Intercept)	25.4609	0.4520	56.33
drugtreated	-2.3605	0.1904	-12.40

Correlation of Fixed Effects:

	(Intr)
drugtreated	-0.235

```
summary(m2)
```

Linear mixed model fit by REML ['lmerMod']

```
Formula: weight ~ drug + (1 + drug | line/litter)
```

```
Data: nested_data
```

REML criterion at convergence: 360

Scaled residuals:

	Min	1Q	Median	3Q	Max
	-2.2294	-0.5673	-0.0806	0.6434	2.4779

Random effects:

Groups	Name	Variance	Std.Dev.	Corr
litter:line	(Intercept)	0.15865	0.3983	
	drugtreated	0.20968	0.4579	1.00
line	(Intercept)	1.17439	1.0837	
	drugtreated	0.08797	0.2966	-1.00
Residual		0.84284	0.9181	

Number of obs: 120, groups: litter:line, 20; line, 5

Fixed effects:

	Estimate	Std. Error	t value
(Intercept)	25.4883	0.5096	50.01


```

drugtreated  -2.3550      0.2471   -9.53

Correlation of Fixed Effects:
      (Intr)
drugtreated -0.580
optimizer (nloptwrap) convergence code: 0 (OK)
boundary (singular) fit: see help('isSingular')

anova(m1, m2, refit = FALSE)

Data: nested_data
Models:
m1: weight ~ drug + (1 | line/litter_id)
m2: weight ~ drug + (1 + drug | line/litter)
      npar    AIC    BIC logLik -2*log(L)  Chisq Df Pr(>Chisq)
m1      5 375.44 389.37 -182.72   365.44
m2      9 377.96 403.04 -179.98   359.96 5.4815  4    0.2414

```

1. Why is model 1 more appropriate than model 0?
2. For the random intercept model 1, does line or litter account for more variation in weight?
3. Does adding a random slope for `drug` improve the model fit?
4. Based on the best model, what is the estimated fixed effect of the drug treatment? Interpret it.

Solution

1. **Model m1 accounts for the nested structure**—ignoring it can underestimate standard errors and inflate Type I error.
2. The `VarCorr(m1)` output shows variance estimates for line and litter. These tell us how much of the total variance in weight is due to those grouping levels.

```

# Variance components for model m1
print(VarCorr(m1), comp="Variance")

```

```

Groups      Name      Variance
litter_id:line (Intercept) 0.45659
line         (Intercept) 0.81293
Residual                                0.90917

```

3. **Compare AIC and p-value from the ANOVA.** If model m2 doesn't improve AIC much or the p-value is large, the added complexity is not justified.
4. From `summary(m1)`, the fixed effect of `drugtreated` is approximately -2.3, indicating that treatment lowers weight by 2.3 g on average.

30.3 Lab, technician and sample

We will use the `faraway` package to analyze a dataset with a hierarchical structure involving laboratory, technician and sample. The dataset `eggs` contains

measurements of fat content in egg powder. Four samples, 2 G samples and 2 H samples, were analyzed by six laboratories, each with two technicians. The samples were each divided into two parts and fat content measured in each. The goal is to assess the effect of lab and technician on the fat content.

```
# Load the faraway package and the eggs dataset
library(faraway)
```

Attaching package: 'faraway'

The following object is masked from 'package:lme4':

```
toenail
data("eggs", package = "faraway")
# Display the first few rows of the dataset
head(eggs)
```

	Fat	Lab	Technician	Sample
1	0.62	I	one	G
2	0.55	I	one	G
3	0.34	I	one	H
4	0.24	I	one	H
5	0.80	I	two	G
6	0.68	I	two	G

Visualize the data.

Solution

```
eggs |> ggplot(aes(x= Lab, y = Fat, color = Technician,
  ↪ shape=Sample)) +
  geom_jitter() +
  theme_bw()
```

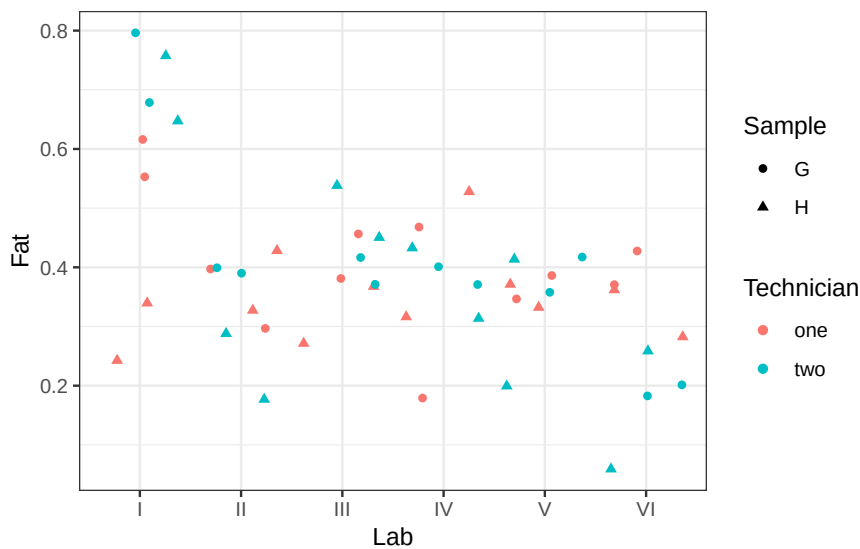


Figure 30.8: Fat content by Lab and Technician

What is the grouping structure in the data, and how is it nested? Using this information, fit a linear mixed model.

Solution

Replicate measurements are made on samples, samples are handled by technicians, and technicians are nested within laboratories. Thus, the grouping structure is hierarchical, with technician nested within lab, and sample nested within technician within lab.

```
##These two models are equivalent, try it!
mfat <- lmer(Fat ~ 1 + (1|Lab/Technician/Sample), data =
  ↪ eggs)
mfat <- lmer(Fat ~ 1 + (1|Lab) + (1|Lab:Technician) +
  ↪ (1|Lab:Technician:Sample), data = eggs)
summary(mfat)
```

Linear mixed model fit by REML ['lmerMod']

Formula:

Fat ~ 1 + (1 | Lab) + (1 | Lab:Technician) + (1 | Lab:Technician:Sample)
Data: eggs

REML criterion at convergence: -64.2

Scaled residuals:

	Min	1Q	Median	3Q	Max
	-2.04098	-0.46576	0.00927	0.59713	1.54276

Random effects:

```

Groups              Name      Variance Std.Dev.
Lab:Technician:Sample (Intercept) 0.003065 0.05536
Lab:Technician        (Intercept) 0.006980 0.08355
Lab                   (Intercept) 0.005920 0.07694
Residual                                0.007196 0.08483
Number of obs: 48, groups:
Lab:Technician:Sample, 24; Lab:Technician, 12; Lab, 6

Fixed effects:
              Estimate Std. Error t value
(Intercept)  0.38750    0.04296   9.019

```

`VarCorr(mfat)`

```

Groups              Name      Std.Dev.
Lab:Technician:Sample (Intercept) 0.055359
Lab:Technician        (Intercept) 0.083548
Lab                   (Intercept) 0.076941
Residual                                0.084828

```

Examine the model output and interpret the random effects. At what level is the most variation in fat content?

Hint

Use `VarCorr(mfat)` to compare the variance components at the lab, technician-within-lab, and sample-within-technician-within-lab levels. The level with the largest estimated variance contributes the most to the total variability in fat content.

30.4 Model comparisons

Exercise 30.5 (Orthodont). Let's get back to the `Orthodont` dataset in the `nlme` package, which contains measurements of 27 children's teeth growth over time.

```

library(nlme)
data("Orthodont", package = "nlme")
summary(Orthodont)

```

When assessing the change in `distance` with increasing `age`, it is important to take the grouping structure into account. Each child is measured multiple times, so we need to account for the correlation of measurements within subjects. Add a random intercept for subject and fit a linear mixed model to assess the association between `distance` and `age`.

```

mo1 <- lmer(distance ~ age + (1|Subject), data = Orthodont)
summary(mo1)

```

Linear mixed model fit by REML [`'lmerMod'`]

```
Formula: distance ~ age + (1 | Subject)
Data: Orthodont
```

```
REML criterion at convergence: 447
```

```
Scaled residuals:
```

Min	1Q	Median	3Q	Max
-3.6645	-0.5351	-0.0129	0.4874	3.7218

```
Random effects:
```

Groups	Name	Variance	Std.Dev.
Subject	(Intercept)	4.472	2.115
	Residual	2.049	1.432

Number of obs: 108, groups: Subject, 27

```
Fixed effects:
```

	Estimate	Std. Error	t value
(Intercept)	16.76111	0.80240	20.89
age	0.66019	0.06161	10.72

```
Correlation of Fixed Effects:
```

```
(Intr)
age -0.845
```

There are both boys and girls in the dataset. Your PI wants to estimate the effect of sex on the distance. How could you do this? With a random intercept per subject, build models with different sets of fixed effects. Start with only intercept and age, then add also sex and finally also the interaction between age and sex.

```
mof2 <- lmer(distance ~ age + Sex + (1|Subject), data =
  ↪ Orthodont)
mof3 <- lmer(distance ~ age*Sex + (1|Subject), data = Orthodont)
```

Compare the models using a likelihood ratio test. When comparing models with different fixed effects, the comparison should be based on ML rather than REML. Use the `anova` function and remember to set `refit = TRUE`, as `anova` will refit the models with ML.

What do the results tell you? Which model would you choose?

Solution

```
anova(mo1, mof2, mof3, refit = TRUE)

refitting model(s) with ML (instead of REML)
Data: Orthodont
Models:
mo1: distance ~ age + (1 | Subject)
mof2: distance ~ age + Sex + (1 | Subject)
```

```

mof3: distance ~ age * Sex + (1 | Subject)
      npar    AIC    BIC logLik -2*log(L)  Chisq Df Pr(>Chisq)
mof1     4 451.39 462.12 -221.69   443.39
mof2     5 444.86 458.27 -217.43   434.86 8.5331  1  0.003488 **
mof3     6 440.64 456.73 -214.32   428.64 6.2174  1  0.012650 *
---
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
Choose the simplest model that adequately describes the data. If adding
Sex improves the fit, keep mof2. If the interaction age:Sex also improves
the fit, keep mof3. Otherwise, prefer the simpler model.

```

30.5 Computing p-values for mixed models

In this section you will use the `lmerTest` package to compute p-values for mixed models using Satterthwaite's approximation. This package extends the `lme4` package and provides functions to compute p-values for fixed effects in linear mixed models.

Go back to the exercises “Drug effect on blood pressure” and “Sleep study” and use the `lmerTest` package to compute p-values for the fixed effects in the models you fitted.

Solution

```
library(lmerTest)
```

```
Attaching package: 'lmerTest'
```

```
The following object is masked from 'package:lme4':
```

```
lmer
```

```
The following object is masked from 'package:stats':
```

```
step
```

```

modell_test <- lmer(bp ~ drug + (1 | hospital), data =
  ↳ databp)
summary(modell_test)

```

```
Linear mixed model fit by REML. t-tests use Satterthwaite's method [
lmerModLmerTest]
```

```
Formula: bp ~ drug + (1 | hospital)
```

```
Data: databp
```

```
REML criterion at convergence: 1215.7
```

```
Scaled residuals:
```

```

      Min       1Q   Median       3Q      Max
-2.36147 -0.65012 -0.08265  0.56163  3.07473

```

```

Random effects:
  Groups   Name      Variance Std.Dev.
hospital (Intercept) 11.90    3.449
Residual                23.23    4.820
Number of obs: 200, groups:  hospital, 10

Fixed effects:
              Estimate Std. Error      df t value Pr(>|t|)
(Intercept)  120.4020    1.1925  10.6685 100.968 < 2e-16 ***
drugtreatment -5.1730    0.6817 189.0000  -7.589 1.43e-12 ***
---
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

Correlation of Fixed Effects:
      (Intr)
drugtretmnt -0.286

mm1_sleep_test <- lmer(Reaction ~ Days + (1 | Subject), data
  ↪ = sleep)
summary(mm1_sleep_test)

Linear mixed model fit by REML. t-tests use Satterthwaite's method [
lmerModLmerTest]
Formula: Reaction ~ Days + (1 | Subject)
Data: sleep

REML criterion at convergence: 325.3

Scaled residuals:
      Min       1Q   Median       3Q      Max
-1.79871 -0.71049 -0.07413  0.64819  1.91336

Random effects:
  Groups   Name      Variance Std.Dev.
Subject (Intercept) 278.9    16.70
Residual                183.6    13.55
Number of obs: 40, groups:  Subject, 5

Fixed effects:
              Estimate Std. Error      df t value Pr(>|t|)
(Intercept)  261.9758    8.4307    5.5251  31.07 2.04e-07 ***
Days          14.0599    0.9351   34.0000  15.04 < 2e-16 ***
---
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

Correlation of Fixed Effects:
      (Intr)
Days -0.388

```

```
mm_sleepall_test <- lmer(Reaction ~ Days + (1 | Subject),
  ↪ data = sleepall)
summary(mm_sleepall_test)
```

Linear mixed model fit by REML. t-tests use Satterthwaite's method [lmerModLmerTest]
 Formula: Reaction ~ Days + (1 | Subject)
 Data: sleepall

REML criterion at convergence: 1430

Scaled residuals:

	Min	1Q	Median	3Q	Max
	-3.6261	-0.4450	0.0474	0.5199	4.1378

Random effects:

Groups	Name	Variance	Std.Dev.
Subject	(Intercept)	1746.9	41.80
	Residual	913.1	30.22

Number of obs: 144, groups: Subject, 18

Fixed effects:

	Estimate	Std. Error	df	t value	Pr(> t)
(Intercept)	267.967	10.871	22.152	24.65	<2e-16 ***
Days	11.435	1.099	125.000	10.40	<2e-16 ***

 Signif. codes: 0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

Correlation of Fixed Effects:

	(Intr)
Days	-0.354

The `summary()` output from `lmerTest` includes approximate p-values for the fixed effects.

30.6 Logistic mixed model

Here we will use the `ohio` dataset in the `faraway` package. This dataset contains information on 537 children and their wheezing status, a binary variable indicating if the child has a pulmonary problem or not. Each child has multiple observations (one for each year from ages 7 to 10, where age 9 is coded as 0). There is also an indicator variable for whether the mother of the child is a smoker.

Take a look at the first two children:

```
library(faraway)
data("ohio", package = "faraway")
head(ohio, 8)
```



```

  resp id age smoke
1    0  0 -2     0
2    0  0 -1     0
3    0  0  0     0
4    0  0  1     0
5    0  1 -2     0
6    0  1 -1     0
7    0  1  0     0
8    0  1  1     0

```

What would the grouping structure be in this case?

 Solution

The grouping variable is `id`, since each child has repeated observations over time.

If we ignore the grouping structure, we can fit a logistic regression model using `glm()`:

```

mohio_glm <- glm(resp ~ age + smoke, family = binomial, data =
  ↪ ohio)
summary(mohio_glm)

```

Call:

```
glm(formula = resp ~ age + smoke, family = binomial, data = ohio)
```

Coefficients:

	Estimate	Std. Error	z value	Pr(> z)
(Intercept)	-1.88373	0.08384	-22.467	<2e-16 ***
age	-0.11341	0.05408	-2.097	0.0360 *
smoke	0.27214	0.12347	2.204	0.0275 *

 Signif. codes: 0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

(Dispersion parameter for binomial family taken to be 1)

Null deviance: 1829.1 on 2147 degrees of freedom
 Residual deviance: 1819.9 on 2145 degrees of freedom
 AIC: 1825.9

Number of Fisher Scoring iterations: 4

To compute the odds ratios (OR), we exponentiate the coefficients:

```
exp(coef(mohio_glm))
```

	age	smoke
(Intercept)	0.1520213	1.3127689

According to this model, do the odds of wheezing increase or decrease with age? What about smoking? Are any of the effects statistically significant?

 Hint

The sign of the coefficient for `age` tells us whether the odds of wheezing increase or decrease with age. The coefficient for `smoke` tells us how maternal smoking is associated with the odds of wheezing. Statistical significance can be assessed from the p-values in `summary(mohio_glm)`, and the odds ratios are obtained from `exp(coef(mohio_glm))`.

The above model is however incorrect as it ignores the fact that each child has multiple observations. To take this grouping into account, we can use a mixed model with a random intercept for each child.

```
mohio <- glmer(resp ~ age + smoke + (1|id), family=binomial,
  ↪ data=ohio)
summary(mohio)
```

```
Generalized linear mixed model fit by maximum likelihood (Laplace
Approximation) [glmerMod]
Family: binomial ( logit )
Formula: resp ~ age + smoke + (1 | id)
Data: ohio
```

AIC	BIC	logLik	-2*log(L)	df.resid
1597.9	1620.6	-794.9	1589.9	2144

Scaled residuals:

Min	1Q	Median	3Q	Max
-1.4027	-0.1802	-0.1577	-0.1321	2.5176

Random effects:

Groups	Name	Variance	Std.Dev.
id	(Intercept)	5.49	2.343

Number of obs: 2148, groups: id, 537

Fixed effects:

	Estimate	Std. Error	z value	Pr(> z)
(Intercept)	-3.37395	0.27496	-12.271	<2e-16 ***
age	-0.17676	0.06797	-2.601	0.0093 **
smoke	0.41478	0.28705	1.445	0.1485

Signif. codes: 0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

Correlation of Fixed Effects:

	(Intr) age
age	0.227
smoke	-0.419 -0.010

We compute the ORs:

```
exp(fixef(mohio))
```

```
(Intercept)          age          smoke  
0.03425393  0.83797708  1.51403850
```

According to this model, do the odds of wheezing increase or decrease with age? What about smoking? What does the model output suggest about the age and smoking effects?

 Hint

The sign of the fixed effects tells us whether the odds of wheezing increase or decrease with age and with maternal smoking. The exponentiated fixed effects, `exp(fixef(mohio))`, give the corresponding odds ratios. Comparing this model with the ordinary logistic regression allows us to see how accounting for repeated observations within child affects the estimates and inference.

Part V

Survival Analysis

Chapter 31

Introduction

31.1 Why survival analysis?

- The term **survival analysis** refers to statistical methods developed to study time-to-event data, originally applied to time until death in 17th-century mortality studies, which is where the term “survival” originates.
- Over the decades its scope has broadened to include a wide range of applications in both medical studies and beyond, in fields such as marketing, engineering or criminology and many more.
- The common factor in survival analysis is the study of **censored data** that arise when studying a unique kind of outcome variable: **time until an event occurs**.
- This event, can be death, as previously mentioned, but also disease onset or relapse in medical studies. In other fields, we can study time until next purchase (marketing), time until machine failure (engineering), time until a released prisoner commits another crime (criminology), and many more.
- In fact, survival analysis is applicable whenever censored data occurs, and these cases are **not limited to studying time-based events**. We can use survival analysis methods to study the number of treatment sessions until a patient recovers or the number of attempts until a student passes an exam.
- **Censoring** occurs when the event of interest has not yet been observed by the end of the study. This makes survival analysis especially useful for incomplete data sets, where we can still make use of the available information to study time to event patterns.
- With survival analysis, we can compare groups (e.g., treatment vs. control) and determine which factors are associated with an increase or decrease of the hazard of the event occurring over time, even when adjusting for other variables. Here, the **hazard** can be thought of as the instantaneous event rate at a given time among individuals who have not yet experienced the

event. Informally, it can be thought of as the risk of the event happening “right now”.

- We can incorporate time-dependent variables, allowing the analysis to account for changing risk factors, such as fluctuating health conditions or environmental factors, and include multiple states and/or events. For example, a patient may progress through different stages of a disease (multiple states), or a patient dying from different causes may prevent the occurrence of the primary event being studied (multiple events, competing risks).

31.2 Survival and censoring times

Let’s imagine a mortality study of terminally ill patients receiving a certain medical treatment. We follow a group of patients over 24 months follow-up period and record the time until death or until the end of the study. For each patient, we suppose there is:

- a true **survival time** T and
- a **censoring time** C .

The survival time represents the time at which the event of interest occurs, here death. The censoring time represents the time at which the patient is lost to follow-up (e.g. withdraws from the study) or the study ends.

We observe the **random variable**

$$Y = \min(T, C)$$

If the event occurs before censoring (i.e. $T \leq C$), then we observe the true survival time T . Otherwise, if censoring occurs before the event (i.e. $T > C$), we observe the censoring time C .

We also observe a **status indicator**

$$\delta = \begin{cases} 1 & \text{if } T \leq C \\ 0 & \text{if } T > C \end{cases} \quad (31.1)$$

The figure below illustrates the data for 10 patients in this fictitious study.

```
# Example data for 10 patients
data <- data.frame(
  patient = factor(1:10),      # Patient IDs
  time = c(23, 24, 9, 7, 14, 16, 19, 20, 24, 24), # Time to
  ↪ event or censoring
  status = c(1, 0, 1, 0, 1, 1, 0, 1, 0, 1)      # Event status
  ↪ (1 = event, 0 = censored)
)

# Plot the data with horizontal lines
ggplot(data, aes(x = time, y = patient)) +
  geom_segment(aes(x = 0, xend = time, y = patient, yend =
  ↪ patient), color = mycols[2]) + #
```



```
geom_point(aes(shape = factor(status)), size = 3) +
geom_vline(xintercept = 24, linetype = "dashed") +
scale_shape_manual(values = c(1, 16),
                   labels = c("Censored", "Event")) +
labs(x = "Time (months)", y = "Patient", shape = "Status") +
theme_minimal() +
theme(legend.position = "top")
```

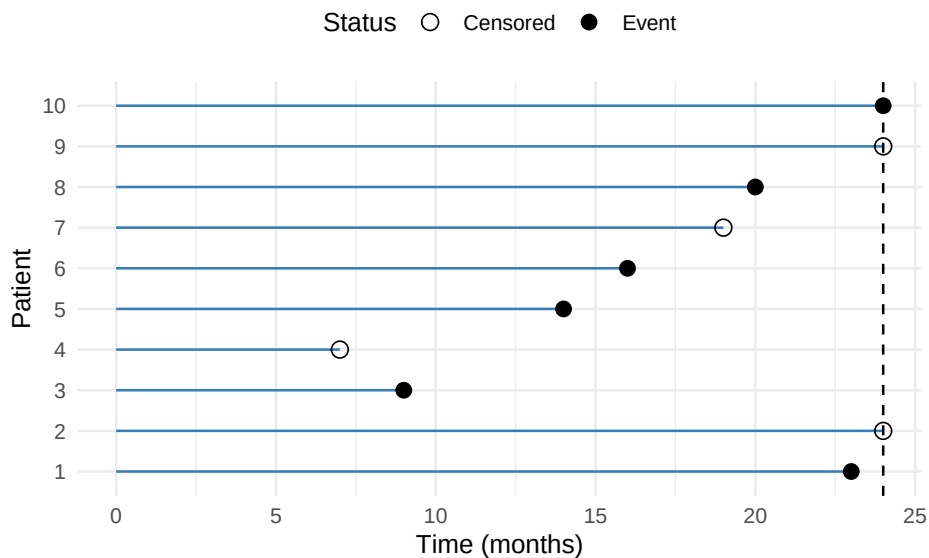


Figure 31.1: Example of right-censored data in a mortality study lasting 24 months. The horizontal lines represent the time to event or censoring for each patient. The points indicate whether the event occurred (event) or not (censored).

We observe 10 (Y, δ) pairs, which we denote as $(y_1, \delta_1), \dots, (y_n, \delta_n)$, where $n = 10$. From Figure 31.1, we can see that:

- for patient #1 we get $(y_1 = t_1, \delta_1 = 1)$ since we observe event (death) as ca. 23 months, before the end of the follow-up at 24 months.
- for patient #2 we get $(y_2 = c_2, \delta_2 = 0)$ since the patient is still alive at the end of the study at 24 months, meaning that the patient has survived at least 24 months (censored)
- for patient #3 we get $(y_3 = t_3, \delta_3 = 1)$ since we observe event (death) at 9 months
- for patient #4, we get $(y_4 = c_4, \delta_4 = 0)$, another censored data point. The study is not over yet, the patient has not died but is **lost to follow-up**, e.g. due to withdrawing from the study
- analogously, for patients #5, #6, #8 and #10 we observe survival times, and for patients #7 and #9 we observe censoring times.

31.3 Independent censoring

In order to analyze survival data, we need to make some **assumptions** about the censoring process. One of the key assumptions in survival analysis is **independent censoring**. This assumption states that the reason for **censoring is unrelated to the likelihood of the event occurring**. In other words, censored individuals should have the same probability of experiencing the event as those who remain in the study.

For example, in a cancer study, if a number of patients leave the study for reasons unrelated to their health status, their data can be treated as censored without biasing the results, since censoring does not provide any information about the outcome. However, if a number of patients drop out early because they are very sick, this would violate the assumption of the independent censoring.

31.4 Left and interval censoring

In addition to right-censored data, we may encounter left- and/or interval-censored data. **Left censoring** occurs when the event happened before the observation period started, whereas **interval censoring** occurs when the event is known to have happened between two specific time points within the study.

Imagine an age study in which **left censoring** occurs when a patient was diagnosed with a disease before the study began, but the exact age of diagnosis is unknown. For example, if a patient is known to have been diagnosed before age 25, but the exact age is not recorded, they are left-censored. **Interval censoring** occurs when a patient's diagnosis happens between two observed ages, but the exact age of diagnosis is unknown. For instance, if a patient was healthy at age 45 and diagnosed by age 55, but the precise age of diagnosis isn't known, their data is interval-censored.

```
# Create a data frame for 8 patients
# Left-censored patients were diagnosed before the study, and
# interval-censored patients were diagnosed between visits
data <- data.frame(
  patient_id = 1:8,
  start_age = c(NA, 30, 40, 35, NA, 25, 45, 50), # Start of
  # observation or known age for interval censoring (NA for
  # left-censored)
  end_age = c(25, 35, 45, 50, 20, 40, 55, 60), # Age of
  # diagnosis or end of observation
  type = c("Left Censored", "Fully Observed", "Fully Observed",
  # "Fully Observed",
  # "Left Censored", "Interval Censored",
  # "Interval Censored", "Fully Observed")
)

# Plot the data
ggplot(data, aes(x = end_age, y = factor(patient_id), color =
  type)) +
```

```

# Add points for observed ages (diagnosis age or censoring)
geom_point(size = 4) +

# For interval-censored patients, add a horizontal line to show
  ↪ the interval
geom_segment(data = subset(data, type == "Interval Censored"),
  ↪ aes(x = start_age, xend = end_age, y = factor(patient_id),
  ↪ yend = factor(patient_id)),
    color = mycols[2], linetype = "dashed") +

# For left-censored patients, add an arrow showing the
  ↪ uncertainty before diagnosis
geom_segment(data = subset(data, type == "Left Censored"),
  ↪ aes(x = 0, xend = end_age, y = factor(patient_id), yend =
  ↪ factor(patient_id)),
    color = mycols[1], linetype = "dotted", arrow =
      ↪ arrow(length = unit(0.2, "cm")))) +

# Customize the plot labels and appearance
labs(x = "Age", y = "Patient ID") +
scale_color_manual(values = c("Left Censored" = mycols[1],
  ↪ "Fully Observed" = "black", "Interval Censored" =
  ↪ mycols[2])) +
theme_minimal() +
theme(legend.position = "top", legend.title = element_blank())

```

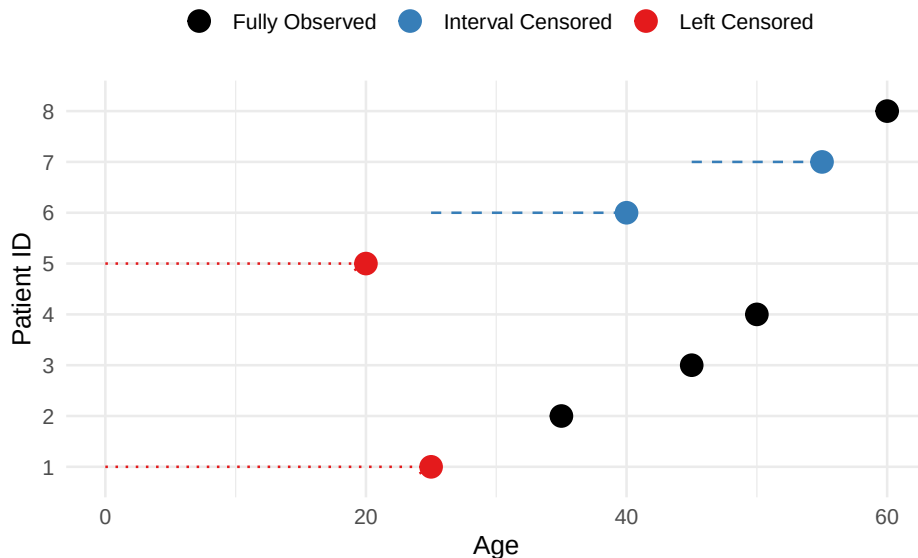


Figure 31.2: Example of left and interval censoring in an age study. Left-censored patients were diagnosed before the study, and interval-censored patients were diagnosed between visits.

In this chapter, we will focus on right-censored data.

31.5 Kaplan-Meier estimator

The **survival curve**, or survival function, is a decreasing function that quantifies the probability of surviving past time t and is defined as

$$S(t) = \Pr(T > t) \quad (31.2)$$

The **Kaplan-Meier (KM) estimator** is a non-parametric statistic used to estimate the survival function from lifetime data and is given by:

$$\hat{S}(t) = \prod_{t_j \leq t} \left(1 - \frac{d_j}{n_j}\right) \quad (31.3)$$

where:

- d_j : number of events (failures) at time t_j
- n_j : number of patients at risk just before time t_j
- and the product is taken over all event times up to and including t

The Kaplan-Meier estimator is also known as the **product-limit** estimator.

31.5.1 Tooth filling example

31.5.1.1 Complete follow-up data

Suppose we have 10 patients who receive a tooth filling, and we want to track how long these fillings last before they fail (fall out or need replacement). The failure times (in months) for each patient are recorded:

```
data_tooth <- data.frame(
  patientID = 1:10,
  time = c(10, 8, 11, 5, 4, 3, 7, 6, 12, 8),
  status = 1
)

str(data_tooth)
```

```
'data.frame':  10 obs. of  3 variables:
 $ patientID: int  1 2 3 4 5 6 7 8 9 10
 $ time      : num  10 8 11 5 4 3 7 6 12 8
 $ status    : num  1 1 1 1 1 1 1 1 1 1
```

Note that the **status** column is a binary event indicator of survival status (1 = event, 0 = censored). In this example, we assume all patients have experienced the event (failure) and have complete follow-up data.

To calculate the Kaplan-Meier curve:

1. We **sort** the data by time to failure.
2. For each event time, we calculate the **survival probabilities**, i.e. the probability the filling lasting beyond that time.

3. We apply the Kaplan-Meier formula (Equation 31.3) to calculate the probability of surviving up to time t , as the product of all individual survival probabilities at each time up to time t :

In our example:

Time (months)	Patients at Risk n_i	Failures d_i	Survival Probability at t_i	Cumulative Survival $S(t)$
3	10	1	$1 - \frac{1}{10} = 0.9$	0.9
4	9	1	$1 - \frac{1}{9} = 0.888$	$0.9 \times 0.888 =$ 0.8
5	8	1	$1 - \frac{1}{8} = 0.875$	$0.8 \times 0.875 =$ 0.7
6	7	1	$1 - \frac{1}{7} = 0.857$	$0.7 \times 0.857 =$ 0.6
7	6	1	$1 - \frac{1}{6} = 0.833$	$0.6 \times 0.833 =$ 0.5
8	5	2	$1 - \frac{2}{5} = 0.6$	$0.5 \times 0.6 = 0.3$
10	3	1	$1 - \frac{1}{3} = 0.667$	$0.3 \times 0.667 =$ 0.2
11	2	1	$1 - \frac{1}{2} = 0.5$	$0.2 \times 0.5 = 0.1$
12	1	1	$1 - \frac{1}{1} = 0$	$0.1 \times 0 = 0$

We can see that:

- At 3 months: 10 patients are at risk, and 1 failure occurs, so the survival probability for this period is 0.9.
- At 4 months: 9 patients are at risk (since one patient failed at 3 months), and the survival probability is now cumulative (0.9 from the first interval multiplied by 0.888 from this interval).
- At time 8 months: two failures occur and the survival probability drops more sharply.
- At time 12 months: the survival probability reaches 0, indicating that all patients experienced the event (tooth filling failure) by the end of the study.

We can visualize the Kaplan-Meier curve for this data:

```
fit.surv <- survfit(Surv(time, status) ~ 1, data = data_tooth)
plot(fit.surv, conf.int=FALSE, xlab = "Months", ylab = "Estimated
  ↪ Probability of Survival", las = 1)
```

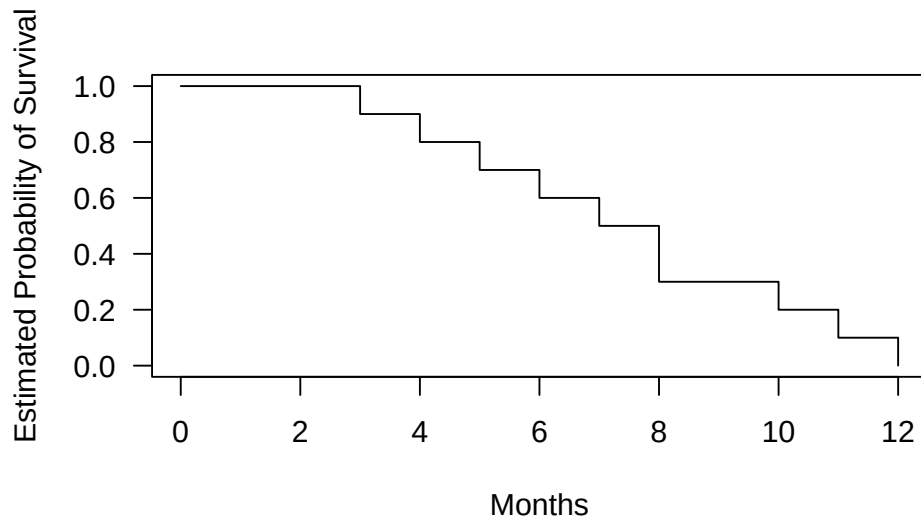


Figure 31.3: Kaplan-Meier survival curve for tooth filling data. The curve shows the estimated probability of survival (filling intact) over time.

31.5.1.2 Including censored data

In real life, not all patients may have complete follow-up data. Some patients might drop out of the study (e.g., move away or lose contact). This is called **censoring**, and these patients are still considered “alive” (their fillings intact) at the time they are censored, but we do not know what happens to them afterward. The filling can also be intact at the end of the study, but we do not know how long it will last.

Let’s update the data with censoring for two patients:

Patient	Time to Failure (months)	Censored
1	10	Yes
2	8	No
3	11	No
4	5	Yes
5	4	No
6	3	No
7	7	No
8	6	No
9	12	No
10	8	No

Now, we can adjust the KM calculations to account for censored data. For censored patients, we do not count them in the number of events (failures), but they are still included in the “at-risk” population until they are censored.

Time(months)	Patients at Risk n_i	Failures d_i	Censored c_i	Survival Probability at t_i	Cumulative Survival $S(t)$
3	10	1	0	$1 - \frac{1}{10} = 0.9$	0.9
4	9	1	0	$1 - \frac{1}{9} = 0.888$	$0.9 \times 0.888 = 0.8$
5	8	0	1		0.8
6	7	1	0	$1 - \frac{1}{7} = 0.857$	$0.8 \times 0.857 = 0.686$
7	6	1	0	$1 - \frac{1}{6} = 0.833$	$0.686 \times 0.833 = 0.571$
8	5	2	0	$1 - \frac{2}{5} = 0.6$	$0.571 \times 0.6 = 0.343$
10	3	0	1		0.343
11	2	1	0	$1 - \frac{1}{2} = 0.5$	$0.343 \times 0.5 = 0.171$
12	1	1	0	$1 - \frac{1}{1} = 0$	$0.171 \times 0 = 0$

Censoring affects the calculations:

- At 5 months, one patient is censored, so we do not record a failure, and the cumulative survival rate remains unchanged.
- At 10 months, one more patient is censored. Again, there is no effect on the failure probability, but the number of at-risk patients decreases.
- In other words, if a patient is censored, they contribute to the number of at-risk patients until their censoring time, but they do not contribute to the event (failure) counts.

We can see how the censoring data affect the KM curve:

```
data_tooth[1, "status"] <- 0
data_tooth[4, "status"] <- 0
fit.surv <- survfit(Surv(time, status) ~ 1, data = data_tooth)
plot(fit.surv, mark.time=TRUE, conf.int=FALSE, xlab = "Months",
     ylab = "Estiamted Probabiliy of Survival", las = 1)
```

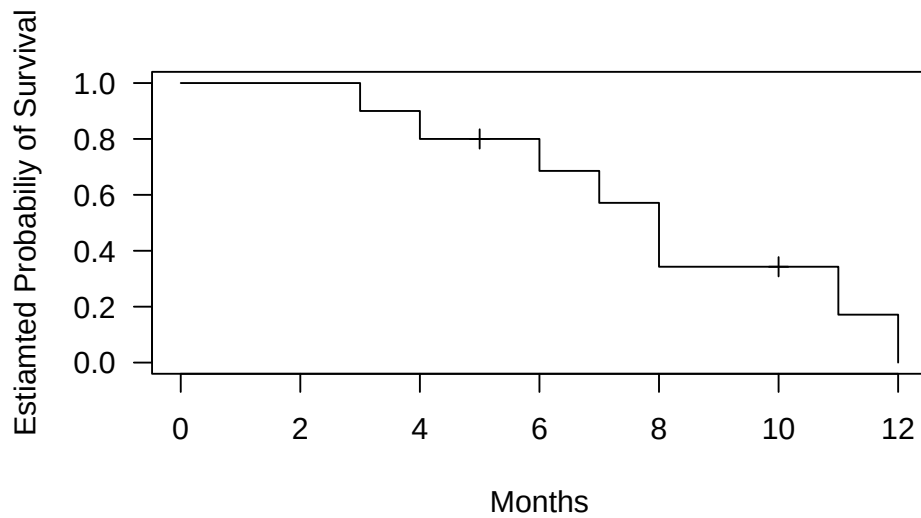


Figure 31.4: Kaplan-Meier survival curve for tooth filling data with censoring. The curve shows the estimated probability of survival (filling intact) over time. Censoring times are indicated with tick marks.

31.5.2 Interpretations

The Kaplan-Meier curve shows the estimated cumulative survival probabilities. A steeper slope indicates a higher event rate (death rate) and therefore a worse survival prognosis. A flatter slope indicates a lower event rate and therefore a better survival prognosis. The curve may have plateaus or flat areas, indicating periods of relatively stable survival.

At specific time points, you can estimate the survival probability by locating the time point on the horizontal axis and dropping a vertical line to the curve. Then, read the corresponding survival probability from the vertical axis.

- So, what is the probability that a tooth filling will last 10 months? We can estimate this by locating 10 months on the x-axis and dropping a vertical line to the curve. The estimated survival probability at 10 months is approximately 0.343, meaning there is a 34.3% chance that a tooth filling will last 10 months or longer.

```
plot(fit.surv, conf.int=FALSE, xlab = "Months", ylab =
  ↪ "Estimated Probability of Survival", las = 1)
abline(v = 10, h = 0.343, col = "red", lty = 2)
```

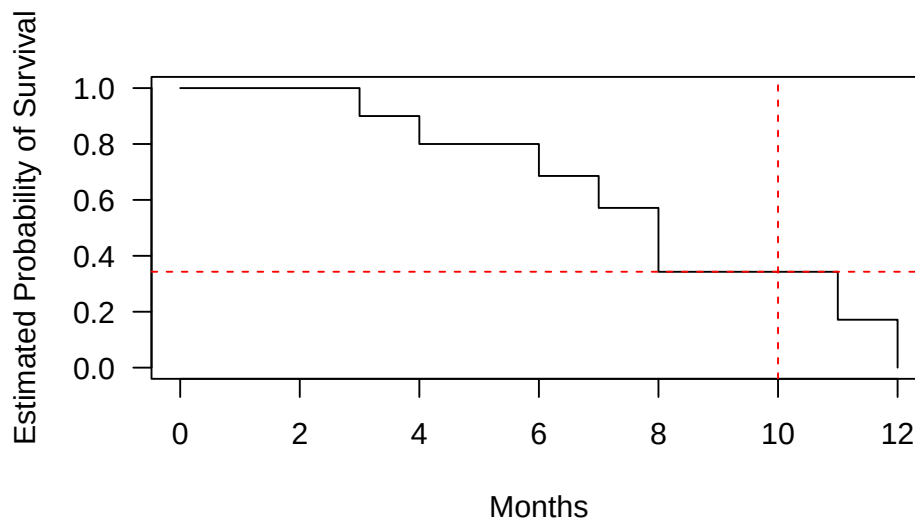



Figure 31.5: Kaplan-Meier survival curve for tooth filling data. Red lines indicate the estimated survival probability (tooth being intact) at 10 months.

If the interest is the **median survival time**, this may be estimated from the survival curve as the time corresponding to a cumulative survival probability of 0.5.

If there are multiple curves representing different groups, you can compare their shapes and patterns. If the curves are similar, it suggests that the groups have similar survival experiences. Large differences between the curves, diverging or crossing curves, suggest differences in survival between the groups, although formal inference requires an appropriate statistical test.

```
# Create data frame for survival data
data_tooth_ext <- data.frame(
  time = c(3, 5, 6, 7, 9, 10, 12, 15, 18, 20,
           4, 6, 7, 9, 11, 13, 14, 16, 18, 22),
  status = c(1, 1, 1, 1, 1, 0, 1, 1, 0, 1,
             1, 1, 0, 1, 1, 1, 1, 0, 1, 1),
  gender = c(rep("M", 10), rep("F", 10))
)

# Create a survival object and
surv_object <- Surv(time = data_tooth_ext$time, event =
  ↪ data_tooth_ext$status)

# Fit Kaplan-Meier curves for gender
fit <- survfit(surv_object ~ gender, data = data_tooth_ext)

# Plot Kaplan-Meier survival curves
ggsurvplot(fit,
  data = data_tooth_ext,
  pval = FALSE,           # Show p-value for
  ↪ difference in survival
```

```

conf.int = FALSE,      # Add confidence interval
risk.table = TRUE,     # Show risk table at the
  ↪ bottom
ggtheme = theme_minimal(), # Use a minimal theme
xlab = "Time (Months)",
ylab = "Survival Probability",
palette = mycols[1:2])

```

Ignoring unknown labels:
 * colour : "Strata"

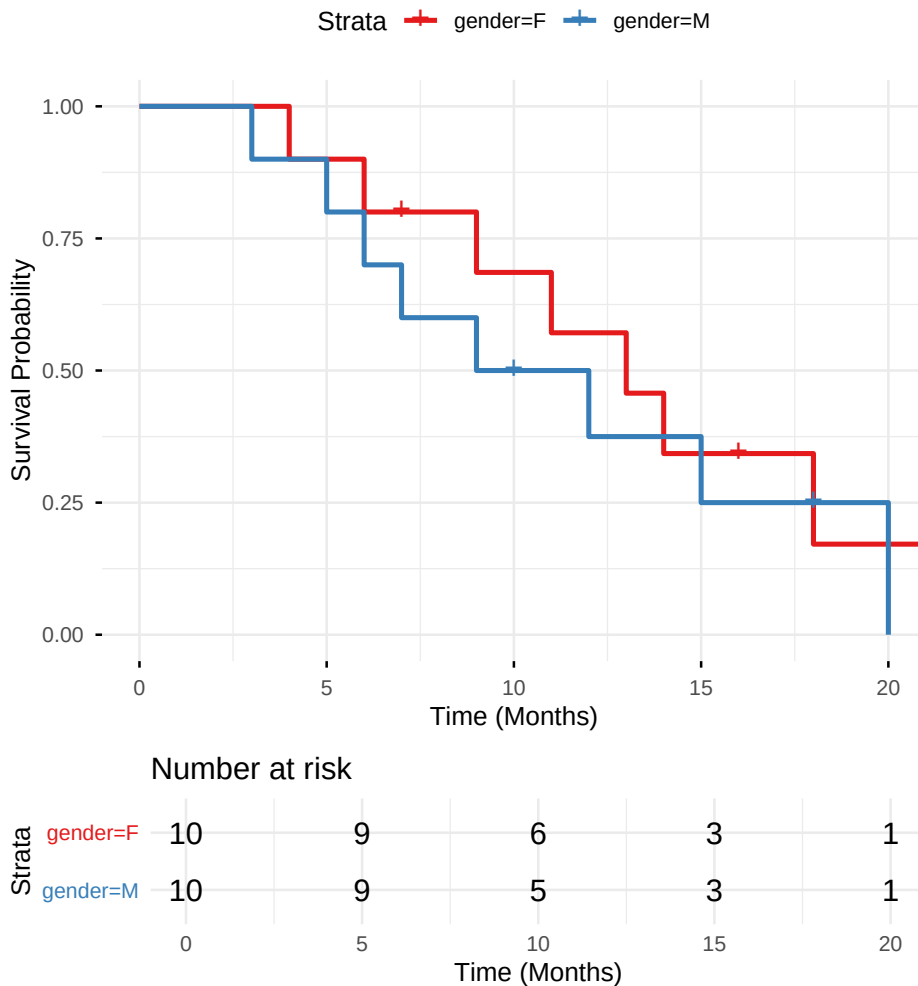


Figure 31.6: Kaplan-Meier survival curves for a new extended study of tooth filling data, including 20 participants (10 men and 10 women).

31.6 Comparisons of hazards

Often, in survival analysis we would like to compare the survival patterns of different groups, e.g. is there a group difference between survival times (tooth filling intact) between men and women in the tooth filling study?

The differences between the survival curves are not constant over time. For example, both groups start at 1 and the survival probability decreases, with lower survival probability for men at 10 months, but then again higher survival probability at 15 months.

One common way to compare groups is through their **hazards** over the duration of the follow-up. In the Cox proportional hazards framework, we assume that the **ratio of the hazards** in the two groups remains constant over time, even if the underlying hazards change. That is, at all times t :

$$\frac{h_1(t)}{h_0(t)} = \text{constant}$$

where:

- $h_1(t)$ and $h_0(t)$ are the hazards in the exposed and unexposed groups at time t (or in one group and the other).

This assumption is called the **proportional hazards assumption**.

It is difficult to estimate the hazard directly from the data, since this would give a series of “spikes” when an event occurs, interspersed with zeros when there is no disease event. Instead we use the **cumulative hazard function**, $H(t)$. This is the total hazard experienced up to time t , and is estimated by

$$H(t) = \sum_{t_j \leq t} \frac{d_j}{n_j}$$

If the ratio of the hazards in both groups is constant over time, it follows that the ratio of the cumulative hazard functions must also equal this constant

$$\frac{H_1(t)}{H_0(t)} = \frac{h_1(t)}{h_0(t)} = \text{constant}$$

and applying the rules of logarithms:

$$\log(H_1(t)) - \log(H_0(t)) = \log(\text{constant})$$

Therefore, if the proportional hazards assumption holds, then graphs of the log of the cumulative hazard functions should be approximately parallel.

```
# Plot Kaplan-Meier survival curves
ggsurvplot(fit,
  data = data_tooth_ext,
  fun = "cumhaz",          # Add cumulative hazard
  ↪ function
  pval = FALSE,            # Show p-value for
  ↪ difference in survival
```

```

conf.int = FALSE,      # Add confidence interval
risk.table = FALSE,    # Show risk table at the
  ↪ bottom
ggtheme = theme_minimal(), # Use a minimal theme
xlab = "Time (Months)",
ylab = "Cumulative hazard",
palette = mycols[1:2])

```

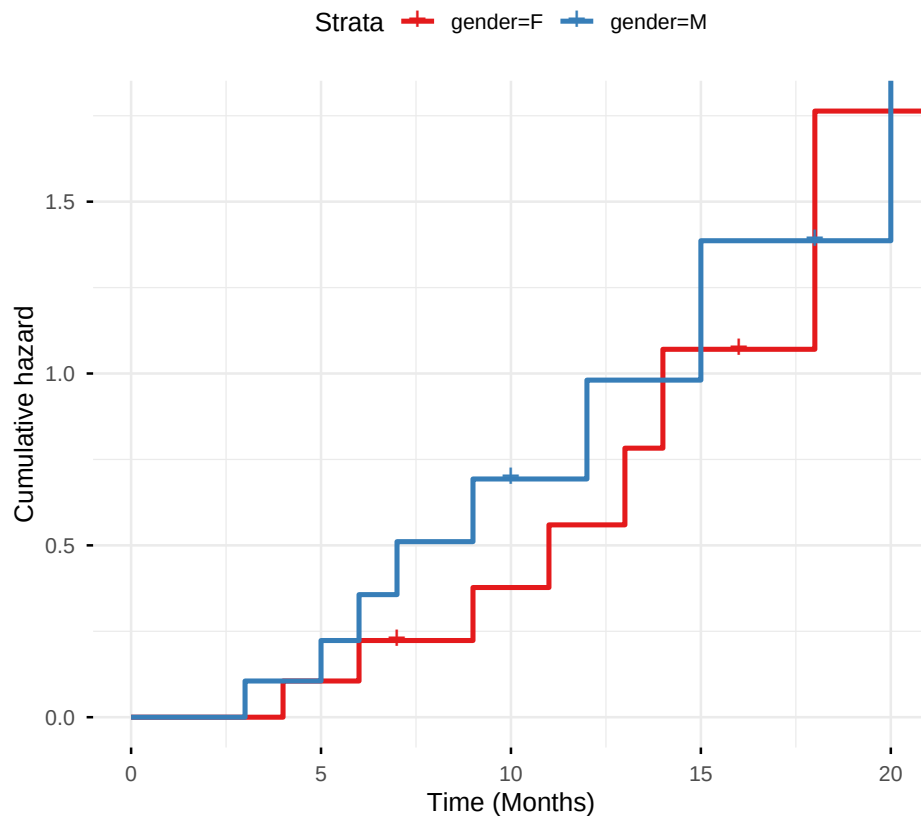


Figure 31.7: Cumulative hazard functions for the two groups in the extended tooth filling study.

Note that it can be shown mathematically that the cumulative hazard is related to the survival function by $H(t) = -\log(S(t))$.

31.7 Mantel-Cox (log-rank test)

Assuming the proportional hazards assumption holds, we can test for differences in survival between groups using the **Mantel-Cox test**, also known as the **log-rank test**. It tests the null hypothesis of no difference in survival between two or more independent groups. The test compares the entire survival experience between groups and can be thought of as a test of whether the survival curves are the same.

The **log-rank test statistic** is constructed by comparing the **observed** number of events (e.g. deaths, failures) in each group to the **expected** number of events, assuming that the survival experiences of the groups are the same. The test statistic is based on a **chi-square distribution**, which tests whether the difference between the observed and expected numbers of events is larger than what we would expect by chance.

Let's start with two groups, A and B. If at time t_j there were d_j events, and there were n'_{jA} and n'_{jB} subjects at risk just before t_j in groups A and B respectively, then the data can be arranged in a 2×2 table:

	Died	Survived	Total
Group A	d_{jA}	$r_{jA} = n'_{jA} - d_{jA}$	n'_{jA}
Group B	d_{jB}	$r_{jB} = n'_{jB} - d_{jB}$	n'_{jB}
Total	$d_j = d_{jA} + d_{jB}$	$r_j = r_{jA} + r_{jB}$	$n'_j = n'_{jA} + n'_{jB}$

where:

- **Group A** and **Group B** represent the two groups being compared (e.g., two different treatments, genders, etc.).
- **Died** (d_{jA} , d_{jB}): The number of individuals who experienced the event (e.g., death, tooth filling falling out) in each group.
- **Survived** (r_{jA} , r_{jB}): The number of individuals who did not experience the event (survivors, with tooth filling intact) in each group. This is calculated as the total number in the group minus the number of deaths.
- **Total** (n_A , n_B): The total number of individuals in each group at risk just before the time of an event

The log-rank test compares the **observed** number of events to the **expected** number of events if the survival curves were the same for all groups. The **expected number of events** in group A at each event time is denoted by:

$$E(d_{jA}) = \frac{n'_{jA}d_j}{n'_j}$$

and analogously for group B

$$E(d_{jB}) = \frac{n'_{jB}d_j}{n'_j}$$

The difference between observed and expected number of events is evidence against the null hypothesis. The Mantel-Cox chi-squared (log-rank) its simple for is the combination of these differences **over all the times at which deaths (events) occurred**:

$$\chi_{MC}^2 = \sum_j \left(\frac{(O_{jA} - E_{jA})^2}{E_{jA}} + \frac{(O_{jB} - E_{jB})^2}{E_{jB}} \right)$$

with 1 df .

The test can be easily expanded to more than two groups, by covering more groups in the summation and having $k - 1$ degrees of freedom, where k is the number of groups.

In more advanced version, the log-rank test incorporates variance into the test statistics. This ensures more accurate and reliable comparisons of survival distributions, particularly when groups differ in size, have varying numbers of individuals at risk, or experience uneven censoring:

$$\chi_{MC}^2 = \frac{(O_A - E_A)^2}{V_A} + \frac{(O_B - E_B)^2}{V_B}$$

where V stands for total variance across all times at which event occurs, with variance at these points calculated as

$$V(d_{jA}) = \frac{d_j(n'_j - d_j)n'_{jA}n'_{jB}}{n_j'^2(n'_j - 1)}$$

for group A (and analogously for other groups in the study).

In our extended tooth filling study, we can compare the survival experiences between men and women:

```
# Perform the log-rank test
log_rank_test <- survdiff(surv_object ~ gender, data =
  ↪ data_tooth_ext)

# Display the result of the log-rank test
print(log_rank_test)
```

Call:

```
survdiff(formula = surv_object ~ gender, data = data_tooth_ext)
```

	N	Observed	Expected	(O-E) ² /E	(O-E) ² /V
gender=F	10	8	9.05	0.121	0.302
gender=M	10	8	6.95	0.158	0.302

Chisq= 0.3 on 1 degrees of freedom, p= 0.6

In this example, the chi-square values is 0.3, with 1 degrees of freedom. The critical value for χ^2 at $\alpha = 0.05$ is approximately 3.84 and since $0.3 < 3.84$ we would conclude that we cannot reject the null hypothesis of no difference in survival between the groups (indicated also by the obtained p-value 0.58).

Chapter 32

Regression with survival response

32.1 Cox proportional hazards model

If survival time is of interest, such as in mortality studies, a common approach is to postulate a distribution for survival time and estimate the parameters of this distribution from the data. For instance, the exponential distribution can be used if the death rate is independent of time, the Weibull distribution allows for increasing or decreasing hazard rates, and the Gompertz distribution models hazard rates that exponentially increase or decrease over time.

More commonly, it is the relationship between survival time and one or more predictor variables (covariates) that is of interest. The Cox proportional hazards model is a widely used semi-parametric model to study this relationship. It is used to estimate the hazard ratio for individuals based on their covariates, without needing to specify the baseline hazard function.

Estimation of β and inferences are developed by considering the information supplied at each time that a death (event) occurred.

Consider:

- a death occurring at time t_j , and
- suppose that there were n_j subjects alive just before t_j ,
- that the values of x for these subjects are $x_1, x_2, \dots, x_{n'_j}$
- and that the subject that dies is denoted, by the subscript 1

Then:

- The set of n'_j subjects at risk is referred to as the *risk set*.
- The risk of death at time t_j for each subject in the risk set is given by:

$$h(t) = h_0(t)\exp(\beta^T x)$$

where:

- $\beta^T x$ is the matrix representation of the regression function, $\beta_1 x_1 + \beta_2 x_2 + \dots + \beta_p x_p$
- and $h_0(t)$ is **baseline hazard function**

The risk of death at time t_j in the risk set does not supply absolute measures of risk, but does supply the relative risks for each subject, since, although $h_0(t)$ is unknown, it is the same for each subject. Thus the probability that the death observed at t_j was of the subject who did die at that time is:

$$p_j = \frac{\exp(\beta^T x_j)}{\sum_{i \in \text{risk set at } t_j} \exp(\beta^T x_i)}$$

where summation is over all members of the risk set.

Similar terms are derived for each time that a death occurred and are combined to form a likelihood (product of these probabilities over all event times t_j):

$$PL(\beta) = \prod_{j=1}^k \frac{\exp(\beta^T x_j)}{\sum_{i \in \text{risk set at } t_j} \exp(\beta^T x_i)}$$

Technically this is called *partial likelihood*, since the component terms are derived conditionally on the times that deaths occurred and the composition of the risk set at these times. This partial likelihood is used to estimate the regression coefficients β in the Cox model. Maximizing this partial likelihood gives the best estimates for β , quantifying the effect of the covariates on the hazard.

To estimate β , we maximize the partial likelihood function with respect to β . No closed-form solution is available, and so iterative algorithms are used.

32.2 Example with brain cancer data

The `BrainCancer` data set from the `ISLR2` package contains survival times for patients with primary brain tumors undergoing treatment with radiation therapy. Variables included in the study are:

- **sex**: male or female
- **diagnosis**: meningioma, LG glioma, HG glioma, or other
- **loc**: tumor location, infratentorial or supratentorial
- **gtv**: gross tumor volume cm^3
- **ki**: Karnofsky index
- **stereo**: stereotactic radiosurgery (SRS) or fractionated stereotactic (SRT) radiotherapy

```
# preview data
data_brain <- BrainCancer

data_brain %>%
  str()
```

```
'data.frame':  88 obs. of  8 variables:
 $ sex      : Factor w/ 2 levels "Female","Male": 1 2 1 1 2 1 2 2 1 2 ...
```



```

$ diagnosis: Factor w/ 4 levels "Meningioma","LG glioma",...: 1 3 1 2 3 1 1 2 1 3 ...
$ loc       : Factor w/ 2 levels "Infratentorial",...: 1 2 1 2 2 2 2 2 2 2 ...
$ ki        : int   90 90 70 80 90 80 80 80 70 100 ...
$ gtv       : num   6.11 19.35 7.95 7.61 5.06 ...
$ stereo    : Factor w/ 2 levels "SRS","SRT": 1 2 1 2 2 1 2 2 2 2 ...
$ status    : int    0 1 0 1 1 0 0 0 0 0 ...
$ time      : num   57.64 8.98 26.46 47.8 6.3 ...

# Cox model with multiple predictors
fit.cox_multi <- coxph(Surv(time, status) ~ sex + diagnosis + ki,
  ↪ data = data_brain)
summary(fit.cox_multi)

```

Call:

```
coxph(formula = Surv(time, status) ~ sex + diagnosis + ki, data = data_brain)
```

```

n= 87, number of events= 35
(1 observation deleted due to missingness)

```

	coef	exp(coef)	se(coef)	z	Pr(> z)
sexMale	0.11149	1.11795	0.35670	0.313	0.754608
diagnosisLG glioma	1.01441	2.75774	0.62159	1.632	0.102687
diagnosisHG glioma	2.22962	9.29629	0.44520	5.008	5.5e-07 ***
diagnosisOther	0.82773	2.28813	0.57751	1.433	0.151774
ki	-0.06624	0.93591	0.01753	-3.778	0.000158 ***

Signif. codes: 0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

	exp(coef)	exp(-coef)	lower .95	upper .95
sexMale	1.1179	0.8945	0.5556	2.2493
diagnosisLG glioma	2.7577	0.3626	0.8156	9.3251
diagnosisHG glioma	9.2963	0.1076	3.8847	22.2467
diagnosisOther	2.2881	0.4370	0.7377	7.0967
ki	0.9359	1.0685	0.9043	0.9686

Concordance= 0.782 (se = 0.038)

Likelihood ratio test= 36.59 on 5 df, p=7e-07

Wald test = 34.11 on 5 df, p=2e-06

Score (logrank) test = 40.79 on 5 df, p=1e-07

32.2.1 Coefficient interpretation

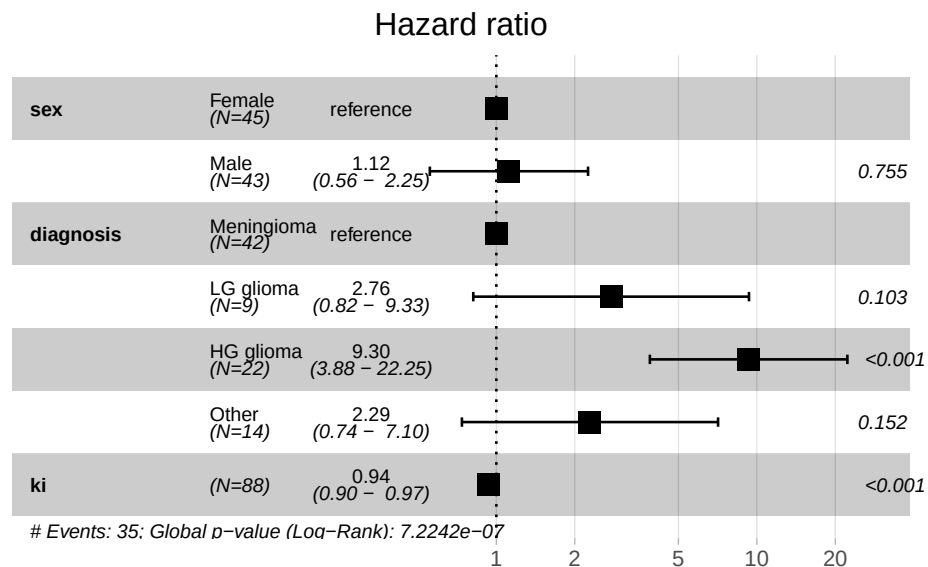
Above, we fitted Cox model using `sex` and `diagnosis` as predictors. The results, indicate, for instance that:

- The estimated hazard ratio (HR) for a male patient is $e^{0.11} = 1.11$. This means that men have 1.11 times the hazard of dying than women while keeping other variables constant (sometimes referred to as adjusted hazard ration, AHR).
- However the associated Z value is small, resulting in large p-value of 0.75, which indicates that this difference is not statistically significant.

- We can also see that one-unit increase in the Karnofsky index is corresponds to a multiplier of $e^{-0.06} = 0.94$ in the chance of dying. The higher the Karnofsky index score, the lower the chance of dying at any given point in time, with this effect being highly significant p -value of 0.000158.
- Note that if you want to find HR associated with more than 1-unit difference, e.g. for patients with 10 units higher Karnofsky index, we would calculate HR as $e^{-0.06 \times 10} = 0.54$. This means that patients with 10 units higher Karnofsky index have 0.54 times the hazard of dying compared to those with lower Karnofsky index.

Sometimes, in addition to reporting the numeric results of a Cox regression, **forest plots** are used to visualize HRs and their 95% confidence intervals.

```
# make forest plot with ggforest()
ggforest(fit.cox_multi, data = data_brain)
```



32.2.2 Global statistical significance

The output of the Cox regression contains also three methods for an overall test of whether the predictors in the model have a significant effect on the hazard (or risk) of the event occurring. Essentially, these check whether the model as a whole provides a better fit than a null model (a model without any covariates).

Likelihood Ratio Test (LRT) compares the log-likelihoods of two models:

- The **null model**, which includes only the baseline hazard (i.e., no covariates).
- The **full model**, which includes the covariates of interest.

The test statistic is computed as:

$$2 \times (\log\text{-likelihood of the full model} - \log\text{-likelihood of the null model})$$

This statistic follows a chi-squared distribution, and the associated p-value indi-

cates whether the full model (with covariates) is significantly better at explaining the data than the null model.

Wald Test

The Wald test checks whether the estimated regression coefficients are significantly different from zero. For each predictor, the test examines whether its coefficient, β_i , is significantly different from 0, implying that the predictor has a significant effect on the hazard. The Wald test can also be used as a global test of significance.

Score Test

The Score test (also called the log-rank test in this context) evaluates the contribution of each predictor to the model's fit, considering the expected number of events. Like the likelihood ratio test, the Score test also compares the null and full models and uses a chi-squared distribution to compute significance.

32.2.3 Concordance

In the context of a **Cox proportional hazards model**, **concordance** (also known as the **C-index** or **concordance index**) is a measure of how well the model predicts the order of events. Specifically, it assesses the ability of the Cox model to correctly predict which of two individuals will experience an event first, based on their risk scores.

The **concordance index** (C-index) ranges between 0.5 and 1:

- A **C-index of 0.5** indicates that the model's predictions are no better than random chance. This means the model cannot distinguish between individuals who experience the event earlier and those who experience it later.
- A **C-index of 1** means perfect prediction, indicating that the model always correctly predicts the order of events.

The C-index is similar to the **area under the ROC curve (AUC)** used in binary classification models, but it is adapted to survival analysis where the goal is to rank individuals by their risk of experiencing the event.

In our example, a concordance of 0.782 means that the Cox model can correctly predict the ordering of survival times 78.2% of the time. In other words, if you randomly select two individuals and compare their predicted risk scores, the model will correctly predict which individual experiences the event earlier in 78.2% of cases. The standard error is 0.038 which provides a measure of uncertainty around the concordance estimate. A smaller standard error indicates more confidence in the estimate.

32.2.4 Predictions

In Cox regression, we can estimate:

- the survival probability at a specific time, $S(t|X = x)$

- and the hazard ratio for an individual relative to a reference individual, $h(t|X = x)/h(t|X = x_{ref})$, where X_{ref} is a reference individual with known covariate values.

For predicted survival, the choice of time matters. A predicted HR will not depend on time due to proportional hazards assumption.

32.2.4.1 Survival

Let's predict the probability of survival at 40 days, for a new patient (man, with LG glioma, Karnofsky index of 80).

```
# new patient data
new_pat <- data.frame(time = 40,
                      sex = "Male",
                      diagnosis = "LG glioma",
                      ki = 80,
                      status = 0)

predict(fit.cox_multi, newdata = new_pat, type = "survival",
        se.fit = T)
```

```
$fit
[1] 0.4824421
```

```
$se.fit
[1] 0.1834607
```

The probability of surviving through 40 days is 0.48. To get confidence interval we could use:

```
unlist(summary(survfit(fit.cox_multi,
                      newdata = new_pat,
                      se.fit = T, conf.int = 0.95),
        times=40)[c("surv", "std.err", "lower", "upper")])
```

```
      surv      std.err      lower      upper
0.4824421 0.1834607 0.2289573 1.0000000
```

So the probability of surviving through 40 days is 0.48 with 95% CI [0.23, 1].

32.2.4.2 HR

To predict the adjusted hazard ratio for a new patient, we use again the `predict()` function, this type with `type = "risk"`. We also need to specify the reference individual. Some options include:

- `reference = "zero"`: the reference individual has continuous predictor values set to 0 and categorical predictors each at their reference level. This is not a good choice if a value of 0 is not plausible for continuous variables in the model, e.g. age.
- `reference = "sample"`: the reference individual has continuous predictors each their respective sample mean and categorical predictors at their

reference level.

- **reference = "strata"**: this leads to the same results as in **reference = "sample"** unless the model includes a **strata()**, in which case continuous variables will be set their within-stratum means.

```
# predict HR for our new patient
predict(fit.cox_multi,
        newdata = new_pat,
        type = "risk",
        reference = "strata",
        se.fit = T)
```

```
$fit
```

```
      1
3.276632
```

```
$se.fit
```

```
      1
1.201369
```

Our new patient 3.27 times the hazard of dying compared to a reference individual with level of each categorical predictor and the mean of each continuous predictor, which in our case are (Female for gender, Meningioma for diagnosis, and mean Karnofsky index of 81.02).

32.3 Time-dependent covariates

In all above example the predictors did not vary over time. Each individual had only one value for each predictor. Since survival data is the result of follow-up over time, it is possible to have predictors that do vary over time. For example, in a study of time to heart attack, researchers could record various other conditions' occurrence over time, such as hypertension or angina. In a study of juvenile recidivism, researchers could record how education or employment status change over time. Cox regression is able to handle such time-varying predictors (also known as "time dependent covariates").

For time-varying covariates, the hazard function is extended to account for changes in the covariates over time:

$$h(t|x(t)) = h_0(t) \cdot \exp(\beta^T x(t))$$

where:

- $h(t|x(t))$ is the hazard at time t , conditional on the covariate values at that time.
- $x(t)$ represents the covariate vector at time t . In this case, one or more covariates can change over time.
- β is the vector of regression coefficients.
- $h_0(t)$ is the baseline hazard function, which remains time-dependent but does not depend on the covariates.

In practice, time-varying covariates are handled by breaking the follow-up time into intervals where the covariate values are assumed to remain constant, and then updating the covariate values at each interval. Suppose a covariate (e.g., blood pressure, treatment status, or exposure) varies over time. For an individual, we divide their follow-up time into periods where the covariate value is constant, and then treat each period as a separate “observation” in the Cox model. The Cox model then uses these updated covariate values when estimating the hazard at each time point.

The partial likelihood function in the Cox model is modified to incorporate the updated covariate values $x(t)$. For each event time t_j , the likelihood is based on the covariate values that are valid at that specific time.

The partial likelihood becomes:

$$PL(\beta) = \prod_{j=1}^k \frac{\exp(\beta^T x_1(t_j))}{\sum_{i \in \text{risk set at } t_j} \exp(\beta^T x_i(t_j))}$$

where:

- $x_1(t_j)$ is the covariate vector for the subject who experienced the event at time t_j ,
- and $x_i(t_j)$ is the covariate vector for each subject in the risk set at time t_j .

A dataset with time-varying predictors will have multiple rows per individual, with different rows having different values for the time-varying predictors, reflecting how they change over time. Additionally, rather than having a single event time variable, each row will have two time variables indicating the beginning and end of the time interval represented by that row of data.

For instance, in the study of heroin usage, we can see a male participant who started using non-prescribed pharmaceutical opioids (NPPOs) at age 19 (time independent covariates). He did not meet the criteria for lifetime opioid dependence at baseline (wave 1), but did at the next interview (wave2), and he first reported using heroin at his 6th interview (both time varying variables).

```
data_opioid <- readRDS("data/data-opioid.rds")

data_opioid %>%
  filter(RANDID == 10) %>%
  select(RANDID, wave, START, STOP, heroin, age_at_init, sex,
         dep_lifetime)
```

	RANDID	wave	START	STOP	heroin	age_at_init	sex	dep_lifetime
1	10	1	3.78	4.26	0	19	Male	0
2	10	2	4.26	4.78	0	19	Male	1
3	10	3	4.78	5.29	0	19	Male	1
4	10	4	5.29	5.84	0	19	Male	1
5	10	5	5.84	6.27	0	19	Male	1
6	10	6	6.27	6.79	1	19	Male	1

32.4 Multiple events: competing risks

In survival analysis, **competing risks** occur when an individual is at risk of more than one mutually exclusive event, and the occurrence of one event **precludes** the occurrence of the others. Traditional survival models, such as the **Cox proportional hazards model**, typically focus on a single type of event (e.g., death or failure), but in many real-world scenarios, individuals may face different types of events that compete with each other.

A **competing risk** refers to events that prevent the event of primary interest from happening. For example:

- In a study of heart disease, **death due to cancer** is a competing risk if the primary event of interest is **death due to heart disease**.
- In a study of cancer treatment, **death from any cause** is a competing risk if the event of interest is **relapse**.

In these situations, standard survival analysis methods may overestimate the probability of the primary event because they do not account for the possibility of competing events.

The Cumulative Incidence Function (CIF) is commonly used in competing risks analysis. It represents the probability of experiencing a specific event (e.g., death from heart disease) by a certain time, accounting for the presence of competing risks (e.g., death from cancer).

CIF is a product of two estimates.

1. The estimate of hazard at ordered failure time t_j for the event of interest:

$$h_c(t_j) = \frac{d_{cj}}{n_j}$$

where:

- m_{cj} denotes the number of events for risk c at time t_j
- and n_j is the number of subjects at risk at time t_j

2. The estimate of overall probability of surviving previous time:

$$\hat{S}(t_{j-1})$$

We consider the overall survival as a subject must have survived all other competing events in order to fail from event type c at time t_j .

The estimated incidence probability of failing from even type c at time t_j is then:

$$\hat{I}_c(t_j) = \hat{S}(t_{j-1}) \times h_c(t_j)$$

The probability of failing form even type c at time t_j is a product of surviving the previous time periods and the cause specific hazard at time t_j .

The CIF for event type c at time t_j is then the cumulative sum up to time t_j , i.e. from $f' = 1$ to $f' = f$, of these incidence probabilities over all event type c

failure times, which is expressed:

$$CIF_c(t_j) = \sum_{f'=1}^f \hat{I}_c(t_j) = \sum_{f'=1}^f \hat{S}(t_{f'-1}) \times \hat{h}_c(t_{f'})$$

In 1999 Gray (1998) proposed a non-parametric test to compare two or more CIFs. The test is analogous to the log-rank test comparing KM curves, using a modified Chi-squared test statistic. This test does not require the independent censoring assumption.

In Fine and Gray Fine and Gray (1999) proposed a proportional hazards model aims at modeling the CIF with covariates, by treating the CIF curve as a subdistribution function. The subdistribution function is analogous to the Cox proportional hazard model, except that it models a hazard function (as known as subdistribution hazard) derived from a CIF.

Chapter 33

Exercises: Survival analysis

```
library(tidyverse)
library(gtsummary) # for tables
library(survival) #survival functions
library(survminer) #for ggsurvplot
library(cmprsk) #competing risks
```

33.1 Kaplan-Meier curves and log-rank test

Exercise 33.1 (Brain cancer survival). The dataset `data_cancer_brain.rds` contains survival data for patients with primary brain tumors.

Load the data and inspect the variables. The variable `status` indicates whether the event was observed (1) or the observation was censored (0).

```
data_brain <- readRDS("data/data_cancer_brain.rds")
tbl_summary(data_brain)
```

- Plot a Kaplan-Meier curve for the full dataset.
- Plot Kaplan-Meier curves stratified by sex.
- Based on the plot, do men and women appear to have similar survival patterns?
- Use a log-rank test to formally compare the survival curves.

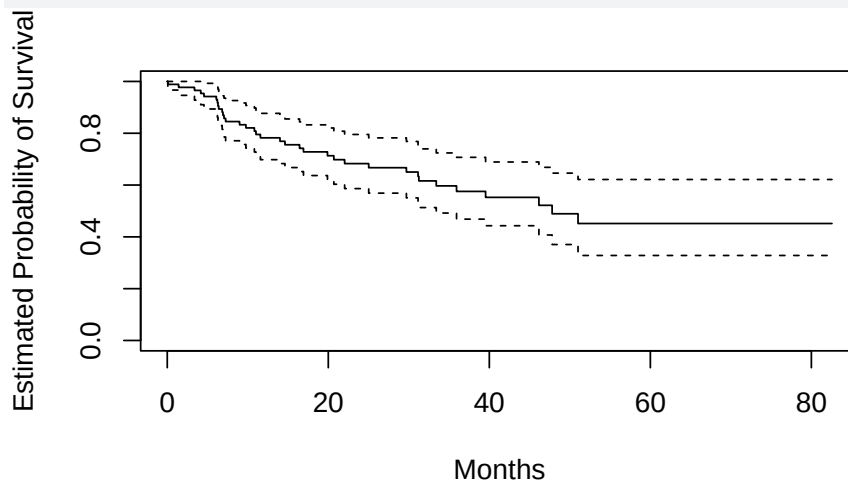
i Solution

- Kaplan-Meier curve for all patients:

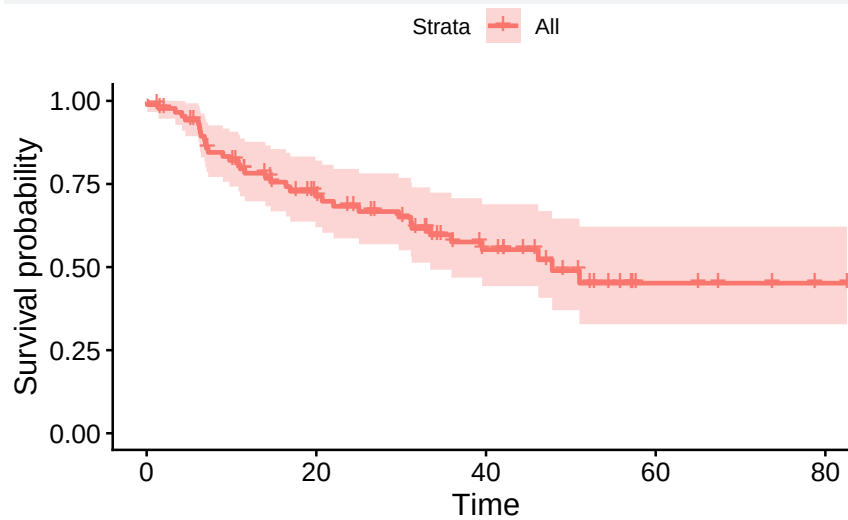
Table 33.1: Summary of the brain data set.

Characteristic	N = 88 ^{<i>1</i>}
sex	
Female	45 (51%)
Male	43 (49%)
diagnosis	
Meningioma	42 (48%)
LG glioma	9 (10%)
HG glioma	22 (25%)
Other	14 (16%)
Unknown	1
loc	
Infratentorial	19 (22%)
Supratentorial	69 (78%)
ki	
40	1 (1.1%)
60	5 (5.7%)
70	14 (16%)
80	36 (41%)
90	27 (31%)
100	5 (5.7%)
gtv	7 (3, 12)
stereo	
SRS	23 (26%)
SRT	65 (74%)
status	35 (40%)
time	24 (10, 42)
^{<i>1</i>} n (%); Median (Q1, Q3)	

```
fit.surv <- survfit(Surv(time, status) ~ 1, data =
  ↪ data_brain)
plot(fit.surv, xlab = "Months", ylab = "Estimated
  ↪ Probability of Survival")
```

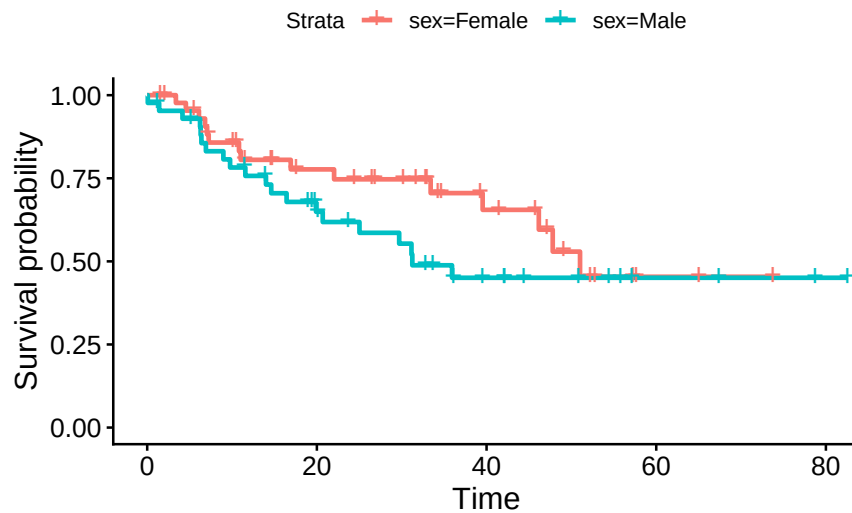


```
##or plot using ggsurvplot
ggsurvplot(fit.surv)
```



b) Kaplan-Meier curves by sex:

```
fit.sex <- survfit(Surv(time, status) ~ sex, data =
  ↪ data_brain)
ggsurvplot(fit.sex)
```

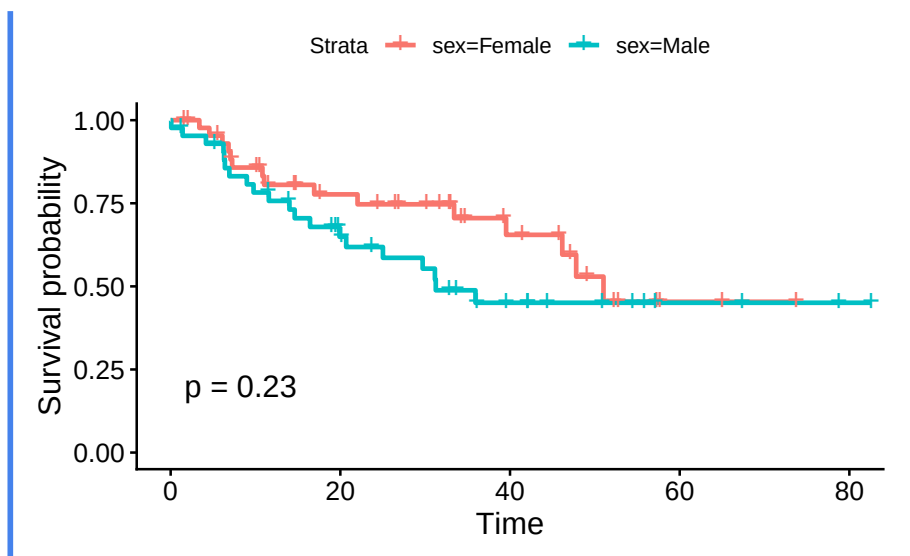


c) The curves can be compared visually. If they are clearly separated, this suggests possible differences in survival, although a formal statistical test is needed.

Select an appropriate significance level and perform the log-rank test. Is there a difference in survival time between males and females?

```
logrank_test <- survdiff(Surv(time, status) ~ sex, data =
  ↪ data_brain)
logrank_test
## Call:
## survdiff(formula = Surv(time, status) ~ sex, data =
  ↪ data_brain)
##
##           N Observed Expected (O-E)^2/E (O-E)^2/V
## sex=Female 45         15      18.5      0.676      1.44
## sex=Male   43         20      16.5      0.761      1.44
##
## Chisq= 1.4 on 1 degrees of freedom, p= 0.2
## Compute p-value
pchisq(logrank_test$chisq, df = length(logrank_test$n) - 1,
  ↪ lower.tail = FALSE)
## [1] 0.2300592

## The logrank p-value can be printed easily in the
  ↪ ggsurvplot
ggsurvplot(fit.sex, pval=TRUE)
```



33.2 Cox regression

Exercise 33.2 (Cox proportional hazards model). Continue with the brain cancer dataset.

- Fit a Cox proportional hazards model with `sex` as predictor.
- Interpret the hazard ratio for `sex`.
- Test the proportional hazards assumption.
- Fit a Cox model with several predictors: `sex`, `diagnosis`, `loc`, `ki`, `gtv`, and `stereo`.
- Which predictors appear to be important?

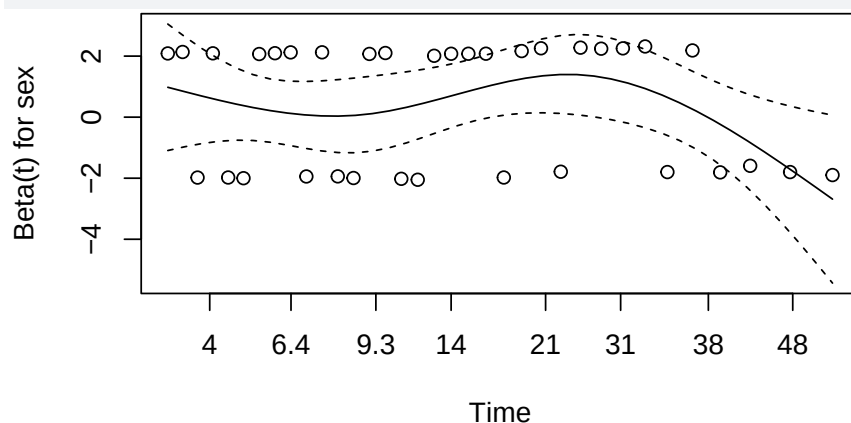
i Solution

- Cox model with `sex` only:

```
fit.cox <- coxph(Surv(time, status) ~ sex, data =
  ↪ data_brain)
summary(fit.cox)
## Call:
## coxph(formula = Surv(time, status) ~ sex, data =
  ↪ data_brain)
##
##      n= 88, number of events= 35
##
##              coef exp(coef) se(coef)      z Pr(>|z|)
## sexMale 0.4077    1.5033   0.3420 1.192   0.233
##
##              exp(coef) exp(-coef) lower .95 upper .95
## sexMale      1.503      0.6652    0.769    2.939
##
## Concordance= 0.565 (se = 0.045 )
## Likelihood ratio test= 1.44 on 1 df,  p=0.2
## Wald test               = 1.42 on 1 df,  p=0.2
## Score (logrank) test = 1.44 on 1 df,  p=0.2
```

- b) The hazard ratio for sex is obtained as `exp(coef)`. It compares the hazard of the event between the sex groups. If the HR is above 1, the hazard is higher in the coded comparison group; if it is below 1, the hazard is lower.
- c) Test the proportional hazards assumption:

```
ph_test <- cox.zph(fit.cox)
print(ph_test)
##           chisq df    p
## sex      0.588  1 0.44
## GLOBAL 0.588  1 0.44
plot(ph_test)
```



If the p-value is small, this suggests evidence against the proportional hazards assumption.

The proportional hazards assumption appears reasonable, we may proceed

with this Cox model.

d) Cox model with multiple predictors:

```

fit.cox_multi <- coxph(Surv(time, status) ~ sex + diagnosis
  ↪ + loc + ki + gtv + stereo, data = data_brain)
summary(fit.cox_multi)
## Call:
## coxph(formula = Surv(time, status) ~ sex + diagnosis +
  ↪ loc +
##      ki + gtv + stereo, data = data_brain)
##
##      n= 87, number of events= 35
##      (1 observation deleted due to missingness)
##
##              coef exp(coef) se(coef)      z
##      Pr(>|z|)
## sexMale          0.18375    1.20171  0.36036  0.510
##      ↪ 0.61012
## diagnosisLG glioma 0.91502    2.49683  0.63816  1.434
##      ↪ 0.15161
## diagnosisHG glioma 2.15457    8.62414  0.45052  4.782
##      ↪ 1.73e-06 ***
## diagnosisOther     0.88570    2.42467  0.65787  1.346
##      ↪ 0.17821
## locSupratentorial  0.44119    1.55456  0.70367  0.627
##      ↪ 0.53066
## ki                -0.05496    0.94653  0.01831 -3.001
##      ↪ 0.00269 **
## gtv                0.03429    1.03489  0.02233  1.536
##      ↪ 0.12466
## stereoSRT          0.17778    1.19456  0.60158  0.296
##      ↪ 0.76760
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1
##      ↪ ' ' 1
##
##              exp(coef) exp(-coef) lower .95 upper
##      ↪ .95
## sexMale          1.2017      0.8321  0.5930
##      ↪ 2.4352
## diagnosisLG glioma 2.4968      0.4005  0.7148
##      ↪ 8.7215
## diagnosisHG glioma 8.6241      0.1160  3.5664
##      ↪ 20.8546
## diagnosisOther     2.4247      0.4124  0.6678
##      ↪ 8.8031
## locSupratentorial  1.5546      0.6433  0.3914
##      ↪ 6.1741
## ki                0.9465      1.0565  0.9132
##      ↪ 0.9811
## gtv                1.0349      0.9663  0.9906
##      ↪ 1.0812
## stereoSRT          1.1946      0.8371  0.3674
##      ↪ 3.8839
##
## Concordance= 0.794 (se = 0.04 )
## Likelihood ratio test= 41.37 on 8 df,  p=2e-06
## Wald test              = 38.7 on 8 df,  p=6e-06
## Score (logrank) test = 46.59 on 8 df,  p=2e-07

```


- e) The diagnosisHG glioma and ki seems important, based on their low p-values.

33.3 Adjusted survival curves from a Cox model

Exercise 33.3 (Predicted survival curves by diagnosis). Continue with the Cox model with many predictors from Exercise 33.2.

We now want to compare the predicted survival curves for the different diagnosis groups while keeping the other predictors fixed.

Construct a small data frame with one row for each diagnosis category.

Set `sex = "Female"`, `loc = "Supratentorial"`, `stereo = "SRT"`, and use the sample means of `ki` and `gtv`.

```
modeldata <- data.frame(
  diagnosis = levels(data_brain$diagnosis),
  sex = rep("Female", length(levels(data_brain$diagnosis))),
  loc = rep("Supratentorial",
    length(levels(data_brain$diagnosis))),
  ki = rep(mean(data_brain$ki),
    length(levels(data_brain$diagnosis))),
  gtv = rep(mean(data_brain$gtv),
    length(levels(data_brain$diagnosis))),
  stereo = rep("SRT", length(levels(data_brain$diagnosis)))
)
modeldata
```

##	diagnosis	sex	loc	ki	gtv	stereo
## 1	Meningioma	Female	Supratentorial	81.02273	8.660795	SRT
## 2	LG glioma	Female	Supratentorial	81.02273	8.660795	SRT
## 3	HG glioma	Female	Supratentorial	81.02273	8.660795	SRT
## 4	Other	Female	Supratentorial	81.02273	8.660795	SRT

- Use `survfit()` and plot the predicted survival curves for the diagnosis groups using e.g. `ggsurvplot`.
- Which diagnosis group appears to have the best predicted survival under the chosen covariate values?

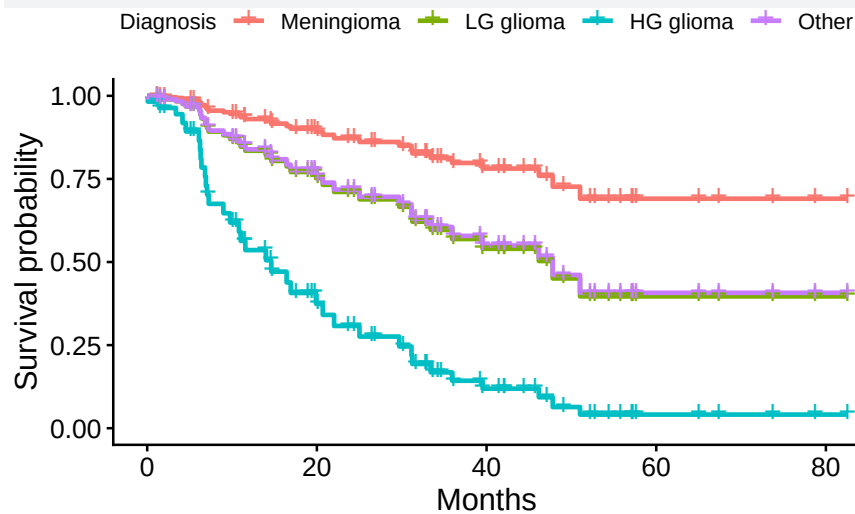
i Solution

- Predicted survival curves:

```

survplots <- survfit(fit.cox_multi, newdata = modeldata)
ggsurvplot(
  survplots,
  data = modeldata,
  legend.labs = modeldata$diagnosis,
  legend.title = "Diagnosis",
  xlab = "Months",
  conf.int = FALSE
)

```



b) The diagnosis group Meningioma with the highest curve has the best predicted survival under the chosen covariate values.

33.4 Time-varying predictors

adpoted from <https://www.bookdown.org/rwnahhas/RMPH/>

A dataset with time-varying predictors will have multiple rows per individual, with different rows having different values for the time-varying predictors, reflecting how they change over time. Additionally, rather than having a single event time variable, each row will have two time variables indicating the beginning and end of the time interval represented by that row of data.

Use the opioid dataset: `data/data-opioid.rds`.

```

data_opioid <- readRDS("data/data-opioid.rds")

data_opioid %>%
  filter(RANDID == 10) %>%
  select(RANDID, wave, START, STOP, heroin, age_at_init, sex,
         dep_lifetime)
##   RANDID wave START STOP heroin age_at_init sex dep_lifetime
## 1      10   1  3.78 4.26     0         19 Male             0

```

## 2	10	2	4.26	4.78	0	19 Male	1
## 3	10	3	4.78	5.29	0	19 Male	1
## 4	10	4	5.29	5.84	0	19 Male	1
## 5	10	5	5.84	6.27	0	19 Male	1
## 6	10	6	6.27	6.79	1	19 Male	1

Here, we see a preview of the `data_opioid` dataset containing longitudinal information for 362 individuals, who at baseline had used non-prescribed pharmaceutical opioids (NPPO, “pain pills”), but were not dependent on NPPOs and had never used heroin.

- Each row contains the time variables `START` and `STOP` which define the time interval (years from initiation of NPPO use) associated with that row.
- Time-invariant variables in the dataset are constant over all rows for the same individual, while time-varying variables can change between rows.
- Each `(START, STOP]` interval defines a period of time during which no variables changed.
- Two time-varying variables in the dataset are heroin use (`heroin`) (the event indicator variable) and lifetime opioid dependence (`dep_lifetime`)
- For instance, for this particular individual, male, started using NPPOs at age 19 (time invariant variables). He did not meet the criteria for lifetime opioid dependence at baseline (`wave = 0`), but did at the next interview (`wave = 1`) and he first reported using heroin at his 6th interview.

To model the heroin usage using both time-invariant and time-varying predictors, we can use the `coxph` function:

```
cox.timevar <- coxph(Surv(START, STOP, heroin) ~
  age_at_init + sex + dep_lifetime, cluster =
  RANDID,
  data = data_opioid)

summary(cox.timevar)
## Call:
## coxph(formula = Surv(START, STOP, heroin) ~ age_at_init + sex
  +
  dep_lifetime, data = data_opioid, cluster = RANDID)
##
## n= 1853, number of events= 27
##
##               coef exp(coef) se(coef) robust se      z
## Pr(>|z|)
## age_at_init -0.3932    0.6749   0.1350    0.1247 -3.153
## 0.00161 **
## sexMale     0.1515    1.1635   0.3925    0.3831  0.395
## 0.69254
## dep_lifetime 1.0580    2.8807   0.3991    0.3850  2.748
## 0.00600 **
```

```
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
##              exp(coef) exp(-coef) lower .95 upper .95
## age_at_init      0.6749      1.4817      0.5286      0.8617
## sexMale          1.1635      0.8594      0.5492      2.4652
## dep_lifetime     2.8807      0.3471      1.3544      6.1271
##
## Concordance= 0.716 (se = 0.042 )
## Likelihood ratio test= 15.24 on 3 df,  p=0.002
## Wald test              = 18.11 on 3 df,  p=4e-04
## Score (logrank) test = 15.45 on 3 df,  p=0.001,  Robust =
↪ 11.63 p=0.009
##
## (Note: the likelihood ratio and score tests assume
↪ independence of
## observations within a cluster, the Wald and robust score
↪ tests do not).
```

Whether a predictor is time-varying or time-invariant, its HR can be interpreted as a comparison of the hazard between groups of individuals with different values of that predictor. Thus, in this example, we could conclude that after adjusting for age at NPPO initiation and sex, those with lifetime opioid dependence have 2.88 times the hazard of using heroin as those who do not (AHR = 2.88; 95% CI = 1.35, 6.12; $p = 0.006$). The HR for a time-varying predictor, however, can also be interpreted as the effect of within-individual change on the hazard. If an individual without opioid dependence transitions to dependence, their hazard of transitioning to heroin is multiplied by 2.88.

33.5 Competing risks

33.5.1 Comparing CIFs

Let's demonstrate how to analyze competing risks using the **cmprsk** package in R. This package includes functions for estimating the cumulative incidence function (CIF) in the presence of competing risks.

Bone marrow transplant (BMT) data from the **cmprsk** package includes information for 35 leukemia cancer patients who underwent bone marrow transplantation. The data can be downloaded from here: `data_bmt`. The data includes the following variables:

- `dis`: disease; 0 = ALL; 1 = AML (ALL, Acute lymphoblastic leukemia; AML, Acute myeloid leukemia)
- `ftime`: follow-up time
- `status`: 0 = censored (survival); 1 = relapse; 2 = Transplant-related mortality;

```
library(cmprsk)
library(survminer)
```

```

# preview data
data_bmt <- readRDS("data/data-bmt.rds")
str(data_bmt)
## 'data.frame': 35 obs. of 3 variables:
## $ dis : int 0 0 0 0 0 1 0 0 1 1 ...
## $ ftime : int 13 1 72 7 8 67 9 5 70 4 ...
## $ status: int 2 1 0 2 2 0 2 2 0 0 ...

# label levels
data_bmt <- data_bmt %>%
  mutate(dis = factor(dis, levels = c(0,1), labels = c("ALL",
    ↪ "AML")))

str(data_bmt)
## 'data.frame': 35 obs. of 3 variables:
## $ dis : Factor w/ 2 levels "ALL","AML": 1 1 1 1 1 2 1 1 2 2
## ↪ ...
## $ ftime : int 13 1 72 7 8 67 9 5 70 4 ...
## $ status: int 2 1 0 2 2 0 2 2 0 0 ...

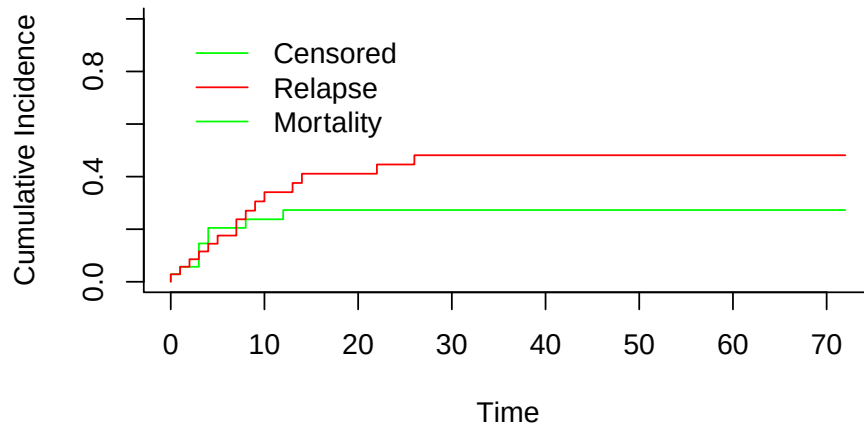
# Estimate the cumulative incidence functions for each event type
cif <- cuminc(ftime = data_bmt$ftime, # Failure time variable
  fstatus = data_bmt$status) # Codes for different
  ↪ causes of failure

# Print the CIF object
print(cif)
## Estimates and Variances:
## $est
##           20           40           60
## 1 1 0.2728390 0.2728390 0.2728390
## 1 2 0.4110148 0.4812696 0.4812696
##
## $var
##           20           40           60
## 1 1 0.006337835 0.006337835 0.006337835
## 1 2 0.008179798 0.008604776 0.008604776

# Plot the CIFs for all even types
plot(cif, lty = 1, col = c("green", "red", "blue"), xlab =
  ↪ "Time", ylab = "Cumulative Incidence",
  main = "Cumulative Incidence Functions for Competing Risks",
  ↪ curvlab = c("Censored", "Relapse", "Mortality"))

```

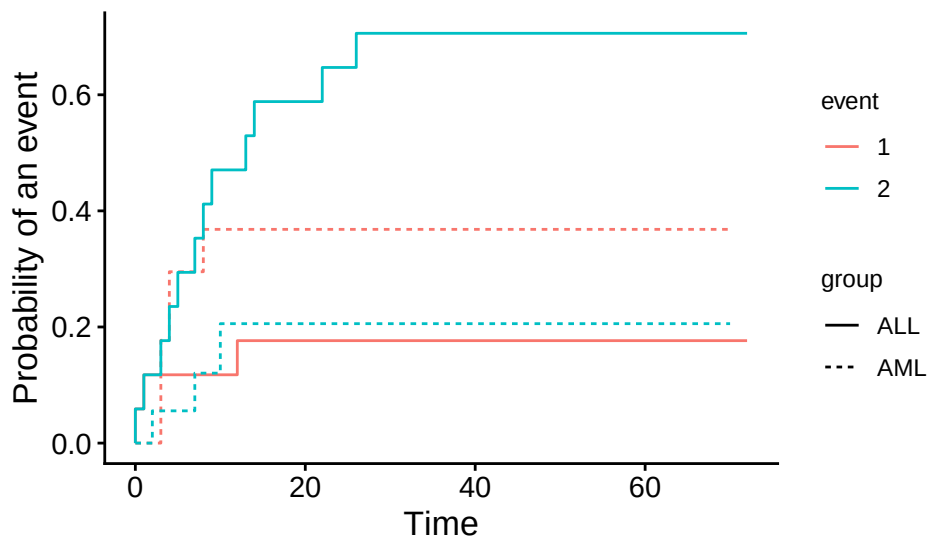
Cumulative Incidence Functions for Competing Risks



```
# Estimate the cumulative incidence functions for each event type
# per group
cif <- cuminc(ftime = data_bmt$ftime,
              fstatus = data_bmt$status,
              group = data_bmt$dis) # Estimates will be calculated
# within groups

# Plot the CIFs for both event types using survminer package
ggcompetingrisks(cif, multiple_panels = FALSE, legend = "right")
```

Cumulative incidence functions



```
# Show Gray's test for equality of CIFs
print(cif$Tests)
```

```
##          stat          pv df
## 1 1.301635 0.253914871 1
## 2 7.082278 0.007784998 1
```

Gray's test for equality of CIFs across groups (ALL vs. AML) is split across different types of events, here (Censored, Mortality, and Relapse).

- For **Censored**, the p-value is small and suggest that there is a **statistically significant difference** in the CIF for censored events between the groups.
- For **Mortality**, the p-value is greater than 0.05, suggesting that there is **no statistically significant difference** in the CIF for mortality between the groups.
- For **Relapse**, the p-value is less than 0.05, indicating a **statistically significant difference** in the CIF for relapse between the groups.
- The above suggest that the groups are behaving differently in terms of the time to relapse and censored events, but not for mortality.

33.5.2 Competing Risks Regression

adopted from <https://www.nature.com/articles/bmt2009359>

Suppose that the BMT study was extended to include more participants and additional covariates and now includes:

- 177 observations
- Sex: gender of the individual
- D: disease; 0 = ALL; 1 = AML
- Phase: phase at transplant (Relapse, CR1, CR2, CR3)
- Age: age at the beginning of follow-up
- Status: 0 = censored; 1 = relapse; 2 = Transplant-related mortality
- Source: source of stem cells (BM+PB, PB)
- ftime: failure time

We are interested in modeling time to relapse in the presence of transplant-related death (competing event). We want to study the effect on relapse of sex, disease type, phase at transplant, source of stem cells and age.

The data can be downloaded from here: `data/data-bmtcrr.rds`.

```
# load additional libraries
library(fastDummies)

# load data
data_bmtcrr <- readRDS("data/data-bmtcrr.rds")

# preview data
str(data_bmtcrr)
## 'data.frame': 177 obs. of 7 variables:
## $ Sex : Factor w/ 2 levels "F","M": 2 1 2 1 1 2 2 1 2 1 ...
## $ D : Factor w/ 2 levels "ALL","AML": 1 2 1 1 1 1 1 1 1 1
## ...
```

```
## $ Phase : Factor w/ 4 levels "CR1","CR2","CR3",...: 4 2 3 2 2
  ↪ 4 1 1 1 4 ...
## $ Age : int 48 23 7 26 36 17 7 17 26 8 ...
## $ Status: int 2 1 0 2 2 2 0 2 0 1 ...
## $ Source: Factor w/ 2 levels "BM+PB","PB": 1 1 1 1 1 1 1 1 1 1
  ↪ 1 ...
## $ ftime : num 0.67 9.5 131.77 24.03 1.47 ...

# prepare matrix of covariates
# factor variables need to be coded as dummy variables
x_sex <- dummy_cols(data_bmtcrr$Sex) %>% dplyr::select(.data_F)
x_phase <- dummy_cols(data_bmtcrr$Phase) %>%
  ↪ dplyr::select(.data_CR1, .data_CR2, .data_CR3)
x_d <- dummy_cols(data_bmtcrr$D) %>% dplyr::select(.data_AML)
x_source <- dummy_cols(data_bmtcrr$Source) %>%
  ↪ dplyr::select(.data_PB)
x <- data.frame(age = data_bmtcrr$Age, sex = x_sex, x_d, x_phase,
  ↪ x_source)
colnames(x) <- c("age", "sex_F", "D_AML", "P:CR1", "P:CR2",
  ↪ "P:CR3", "source:PB")

# The first regression model for relapse can be produced by
  ↪ typing
mod1 <- crr(ftime = data_bmtcrr$ftime,
  fstatus = data_bmtcrr$Status,
  cov1 = x,
  failcode = 1) # code for relapse

summary(mod1)
## Competing Risks Regression
##
## Call:
## crr(ftime = data_bmtcrr$ftime, fstatus = data_bmtcrr$Status,
## cov1 = x, failcode = 1)
##
##          coef exp(coef) se(coef)      z p-value
## age      -0.0185    0.982   0.0119 -1.554  0.1200
## sex_F     -0.0352    0.965   0.2900 -0.122  0.9000
## D_AML     -0.4723    0.624   0.3054 -1.547  0.1200
## P:CR1     -1.1018    0.332   0.3764 -2.927  0.0034
## P:CR2     -1.0200    0.361   0.3558 -2.867  0.0041
## P:CR3     -0.7314    0.481   0.5766 -1.268  0.2000
## source:PB  0.9211    2.512   0.5530  1.666  0.0960
##
##          exp(coef) exp(-coef)  2.5% 97.5%
## age          0.982      1.019 0.959 1.005
## sex_F         0.965      1.036 0.547 1.704
## D_AML         0.624      1.604 0.343 1.134
## P:CR1         0.332      3.009 0.159 0.695
```



```
## P:CR2      0.361      2.773 0.180 0.724
## P:CR3      0.481      2.078 0.155 1.490
## source:PB  2.512      0.398 0.850 7.426
##
## Num. cases = 177
## Pseudo Log-likelihood = -267
## Pseudo likelihood ratio test = 24.4 on 7 df,
```

The first part of the output shows for each term in the design matrix the estimated coefficient $\hat{\beta}_j$, the relative risk $\exp(\hat{\beta}_j)$, the standard error, the z-value and the corresponding P-value for assessing significance.

Here, **Sex** is not significant, followed by **Age** and **D** (disease type), whereas **Source** is only marginally significant. **Phase** is a factor with relapse as baseline, so each P-value provides a test for the difference of each level with respect to the baseline.

An overall P-value for Phase can be obtained through the Wald test via **aod** R package:

```
library(aod)
wald.test(mod1$var, mod1$coef, Terms = 4:6)
## Wald test:
## -----
##
## Chi-squared test:
## X2 = 14.0, df = 3, P(> X2) = 0.0029
```

The first argument to the function `wald.test()` is the estimated covariance matrix for the coefficients, followed by the vector of coefficients estimates, and the position of coefficients for which we want to assess significance. In our case, the P-value indicates that Phase is statistically significant.

The second part of the output for competing risks regression shows the relative risk for each term, and a 95% confidence interval. The relative risk or subdistribution hazard ratio for a categorical covariate is the ratio of subdistribution hazards for the actual group with respect to the baseline, with all other covariates being equal. If the covariate is continuous then the relative risk refers to the effect of a one unit increase in the covariate, with all other covariates being equal. In our data, $\exp(-0.0352)=0.965$ is the relative risk of a woman with respect to a man, and $\exp(-0.0185)=0.982$ is the relative risk for a 1 year increase in age.

The last part of the output shows the pseudo log-likelihood at maximum and the pseudo likelihood ratio test, that is, the difference in the objective function at the global null and at the final estimates. As this objective function is not a true likelihood, this test statistic is not asymptotically distributed as a χ^2 . As a consequence, model comparison based on likelihood ratio approach cannot be performed directly, but significance must be evaluated through simulations. However, a model selection criterion can be easily adopted as described in the following section.

For an example on model selection, model diagnostics or adding time-varying covariates in presence of competing events, please refer to the original tutorial article Scrucca, Santucci, and Aversa (2010) <https://www.nature.com/articles/bmt2009359>.

Part VI

Principal Component Analysis

Learning outcomes

- Understand the concept of principal component analysis (PCA)
- Understand and be able to perform PCA
- Understand the loading/score plot

Chapter 34

Background

```
nci_url <-  
  ↪ "https://web.stanford.edu/~hastie/ElemStatLearn/datasets/nci.data.csv"  
nci <- read.table(url(nci_url), sep = ",", row.names = 1, header  
  ↪ = TRUE) |>  
  as.matrix() |>  
  t()
```

Studies in biology have become data-intensive as high throughput experiments in Omics have produced data sets of massive volume and complexity. One example is shown in the figure below. It shows a part of the gene expression data for 6830 genes for 64 cell lines obtained from <https://web.stanford.edu/~hastie/ElemStatLearn/datasets/nci.data.csv>. Many illustrations here are generated from the data.

```
# table displayed as text  
round(nci, 2)[, sample(1000, 50)] |>  
  as_tibble(rownames = "sample") |>  
  pivot_longer(-sample, names_to = "gene") |>  
  ggplot(aes(gene, sample)) +  
  geom_text(aes(label = value), hjust = 0) +  
  theme_void(base_size = 5)
```

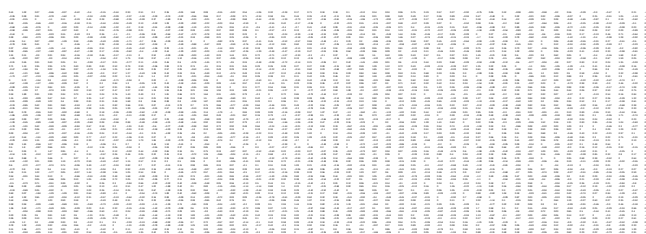


Figure 34.1: A part of gene expression data for 6830 genes for 64 cell lines

It makes difficult to get an idea about the contents of the data. Dimensional

reduction is frequently used for this challenge. It reduces vast dimensions of a data to a manageable number of dimensions and helps us get a simplified overview of the data. One of the popular methods for the dimensional reduction is principal component analysis (PCA).

PCA focuses on dispersion or variance of data. Let's take one of the variables and see how the values of it are dispersed around average.

```
corr <- cor(nci)
corr[upper.tri(corr, diag = TRUE)] <- NA # to use arrayInd
corr[corr > 0.95] <- NA # avoid too much correlated

# highly correlated gene pair
idx <- arrayInd(which.max(corr), dim(corr))[1, ]
idx <- sort(idx) # switch x and y in PCA plot
# correlated variables
correlated_vs <- as.list(as.data.frame(nci[1:30, idx]))
names(correlated_vs) <- sub("g", "Variable ",
  ↪ names(correlated_vs))

fig.cap <- paste(
  "The red numbers show the index of each sample.",
  "The red line shows the mean and the dashed lines show the
  ↪ distance between each observation and mean."
)

# Select a variable
v1 <- correlated_vs[[1]]
v1_name <- names(correlated_vs)[1]

# plot the values of the variable
plot(v1, xlab = "", ylab = v1_name, ylim = range(v1) * 1.1, xaxt
  ↪ = 'n')
title(xlab = "Sample index", mgp = c(1, 0, 0))

m <- mean(v1)
# below or above the points, determined by the comparison with
  ↪ mean
pos <- (v1 > m) * 2 + 1
# sample index
text(seq(v1), v1, seq(v1), cex = 0.65, col = "red", pos = pos)
# draw the mean line
abline(h = m, col = "red")
# for each observation draw a line from that point to the mean
for(ii in seq(v1)) {
  segments(x0 = ii, x1 = ii, y0 = v1[ii], y1 = m, lty = "dashed")
}
```

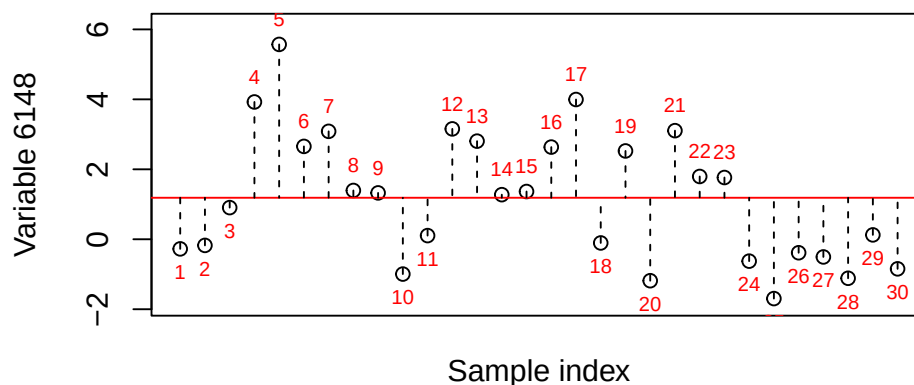



Figure 34.2: The red numbers show the index of each sample. The red line shows the mean and the dashed lines show the distance between each observation and mean.

The values of individual variables are dispersed as above. Interestingly, multiple variables often have dispersed in the similar pattern.

```
fig.cap <- paste(
  "The red numbers show the index of each sample.",
  "The red line shows the mean and the dashed lines show the
  ↪ distance between each observation and mean."
)

# Two correlated variables
vs <- correlated_vs
gap <- 0.1

# plot the values of the variable
plot(seq(vs[[1]] - gap, vs[[1]]),
     xlab = "", ylab = paste("Variables", paste(idx, collapse = "
     ↪ & ")),
     ylim = range(vs[[1]]) * 1.1, xaxt = 'n')
title(xlab = "Sample index", mgp = c(1, 0, 0))
points(seq(vs[[2]] + gap, vs[[2]]), col = "blue", pch = 16)

m <- sapply(vs, mean)
# below or above the points, determined by the comparison with
↪ mean
pos <- (vs[[1]] > m[1]) * 2 + 1
# sample index
text(seq(vs[[1]]), vs[[1]], seq(vs[[1]]), cex = 0.65, col =
  ↪ "red", pos = pos)
# draw the mean line
abline(h = m[1], col = "red")
abline(h = m[2], col = "blue")
# for each observation draw a line from that point to the mean
for(ii in seq(vs[[1]])) {
```

```
segments(x0 = ii - gap, x1 = ii - gap, y0 = vs[[1]][ii], y1 =
  ↪ m[1], lty = "dashed")
segments(x0 = ii + gap, x1 = ii + gap, y0 = vs[[2]][ii], y1 =
  ↪ m[2], col = "blue", lty = "dashed")
}
```

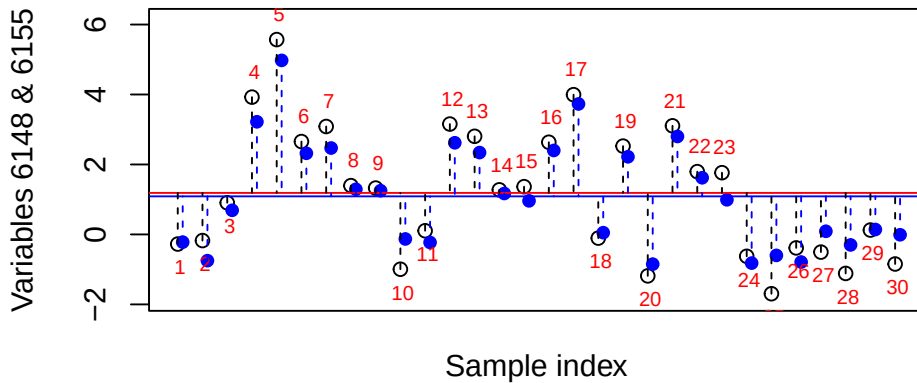
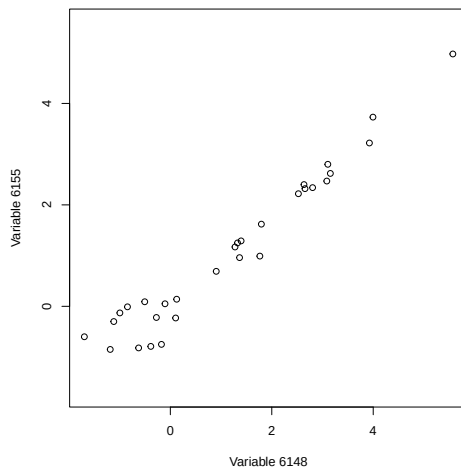


Figure 34.3: The red numbers show the index of each sample. The red line shows the mean and the dashed lines show the distance between each observation and mean.

In the scatter plot of those two variables, the correlation is more obvious.

```
opa <- par(pty = "s") # square plot
rg <- range(vs)
plot(vs[[1]], vs[[2]], xlab = names(vs)[1], ylab = names(vs)[2],
  ↪ xlim = rg, ylim = rg)
```



```
par(opa)
```

As both variables provide almost identical information redundantly in terms of dispersion across objects (or observations), we can attempt to reduce the

number of variables, i.e. the dimension.

Chapter 35

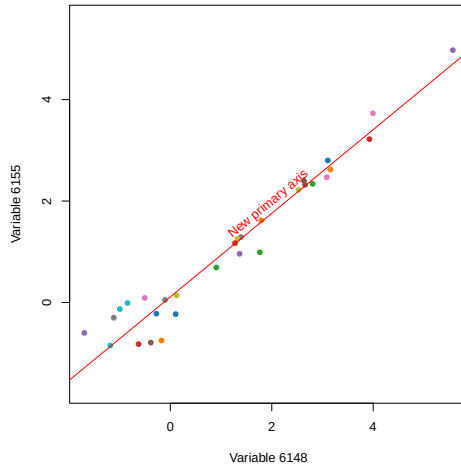
Two-dimensional data

PCA finds a new variable that explains most of the dispersion or variance of the two variables of interest. The remaining smaller portion of variance in the data is stored in an orthogonal variable to the primary variable. It is achieved by new axes and rotation graphically, which are mathematically expressed by linear combinations of the variables.

```
vs <- correlated_vs
pca <- prcomp(bind_rows(vs))
s <- pca$rotation[2, 1] / pca$rotation[1, 1] # slope

opa <- par(pty = "s") # square plot
col <- grDevices::palette.colors(palette = "Classic Tableau") |>
  rep_len(length(vs[[1]])) # individual colors of points

rg <- range(vs)
plot(vs[[1]], vs[[2]], xlab = names(vs)[1], ylab = names(vs)[2],
      pch = 16, col = col, xlim = rg, ylim = rg)
# PC1
abline(b = s, a = pca$center[2] - s * pca$center[1], col = "red")
# indicate "New axis"
text(mean(rg), mean(rg), "New primary axis", col = "red", srt =
  ↪ atan(s) / pi * 180)
```



```
par(opa)
```

Let x_{ij} denote an observation i for a variable j , and y_{i1} be a linear combination of x_{i1} and x_{i2} .

$$y_{i1} = a_{11}x_{i1} + a_{21}x_{i2} = \sum_{j=1}^2 a_{j1}x_{ij} = \sum_{j=1}^2 x_{ij}a_{j1}$$

By adjusting a_{11} and a_{21} , let the $\mathbf{y}_1^T = [y_{11}, y_{21}, \dots, y_{n1}]$ explain largest amount of variance of the data. Then, the $\mathbf{y}_1$ is called the **first principal component** (PC). In a matrix equation for all n observations,

$$\mathbf{y}_1 = \mathbf{X}\mathbf{a}_1$$

, where

$$\mathbf{y}_1 = \begin{bmatrix} y_{11} \\ y_{21} \\ \vdots \\ y_{n1} \end{bmatrix}_{n \times 1}, \quad \mathbf{X} = \begin{bmatrix} x_{11} & x_{12} \\ x_{21} & x_{22} \\ \vdots & \vdots \\ x_{n1} & x_{n2} \end{bmatrix}_{n \times 2} \quad \text{and} \quad \mathbf{a}_1 = \begin{bmatrix} a_{11} \\ a_{21} \end{bmatrix}_{2 \times 1}$$

The values of the 1st PC, often called PC1 scores, are colorfully highlighted in the scatter plot of the two original variables.

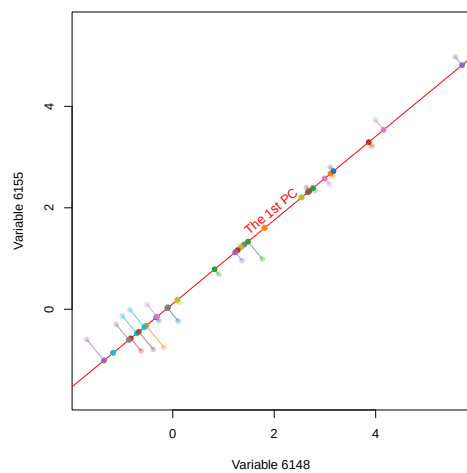
```
# coordinates after projection onto PC1
x <- pca$x
x[, 2] <- 0 # PC2 = 0
on_pc1 <- t(t(x %*% solve(pca$rotation)) + pca$center)

opa <- par(pty = "s") # square plot
plot(vs[[1]], vs[[2]],
     xlab = names(vs)[1], ylab = names(vs)[2], xlim = rg, ylim =
     ↪ rg, type = "n")
# show the projection
segments(x0 = vs[[1]], x1 = on_pc1[, 1],
```

```

        y0 = vs[[2]], y1 = on_pc1[, 2],
        col = scales::alpha(col, alpha = 0.7))
# indicate "PC1"
text(mean(rg), mean(rg), "The 1st PC", col = "red", srt = atan(s)
     ↪ / pi * 180)
# before projection
points(vs[[1]], vs[[2]], pch = 16, col = scales::alpha(col, alpha
     ↪ = 0.3))
# PC1
abline(b = s, a = pca$center[2] - s * pca$center[1], col = "red")
# after projection
points(on_pc1[, 1], on_pc1[, 2], pch = 16, col = col)

```



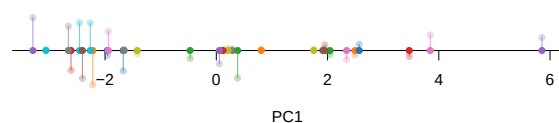
```
par(opa)
```

The scores are presented in the coordinate system of the 1st PC (PC1). As shown below, it is graphically a rotation of the plot above.

```

rg <- apply(pca$x, 2, range)
plot(pca$x, axes = FALSE, type = "n", ylab = "", mgp = c(1, 0,
     ↪ 0))
axis(1, pos = c(0, 0))
abline(h = 0)
# before projection
points(pca$x, pch = 16, col = scales::alpha(col, alpha = 0.3))
# show the projection
segments(x0 = pca$x[, 1], x1 = x[, 1],
        y0 = pca$x[, 2], y1 = 0,
        col = scales::alpha(col, alpha = 0.7))
points(x, pch = 16, col = col)

```



If $\mathbf{S}$ is the covariance matrix of $\mathbf{X}$ having **eigenvalue-eigenvector** pairs $(\hat{\lambda}_1, \hat{\mathbf{e}}_1)$ and $(\hat{\lambda}_2, \hat{\mathbf{e}}_2)$ where $\hat{\lambda}_1 \geq \hat{\lambda}_2 \geq 0$, it has been proved that when

$$\mathbf{a}_1 = \hat{\mathbf{e}}_1$$

, the $\mathbf{y}_1$ maximizes the proportion explained by it of the variance of $\mathbf{X}$. Hence, the first principal component is

$$\mathbf{y}_1 = \mathbf{X}\hat{\mathbf{e}}_1$$

As the total sample variance is $\sum_k \hat{\lambda}_k$, the explained proportion by the first principal component is

$$\frac{\hat{\lambda}_1}{\sum_k \hat{\lambda}_k}$$

Chapter 36

Multiple dimensional data

Extension of PCA from 2-D to multiple dimension is rather simple. The matrices are extended as

$$\mathbf{X}_{n \times p} = \begin{bmatrix} x_{11} & x_{12} & \cdots & x_{1p} \\ x_{21} & x_{22} & \cdots & x_{2p} \\ \vdots & \vdots & \ddots & \vdots \\ x_{n1} & x_{n2} & \cdots & x_{np} \end{bmatrix} \quad \text{and} \quad \mathbf{a}_1 = \begin{bmatrix} a_{11} \\ a_{21} \\ \vdots \\ a_{p1} \end{bmatrix}$$

PCA finds an axis or a linear combination that maximizes the explained variances. Because we can repeat the process more than once, we can have the 2nd, 3rd, and 4th principal components. Similar to the 1st PC, the 2nd PC is chosen such that it explains the largest variances after excluding the variances already explained by the 1st PC.

If $\mathbf{S}$ is the covariance matrix of $\mathbf{X}$ having **eigenvalue-eigenvector** pairs $(\hat{\lambda}_1, \hat{\mathbf{e}}_1)$, $(\hat{\lambda}_2, \hat{\mathbf{e}}_2)$, ..., $(\hat{\lambda}_p, \hat{\mathbf{e}}_p)$ where $\hat{\lambda}_1 \geq \hat{\lambda}_2 \geq \cdots \geq \hat{\lambda}_p \geq 0$, the k th principal component is

$$\mathbf{y}_k = \mathbf{X}\hat{\mathbf{e}}_k$$

The proportion of variance explained by the k th principal component is

$$\frac{\hat{\lambda}_k}{\sum_l \hat{\lambda}_l}$$

Chapter 37

Notes for PCA

37.1 Missing

Like many other multiple variable analysis, any missing value is not allowed. You may need to impute or trim incomplete cases out.

37.2 Scaling

Scaling ahead of PCA is often recommended. If one variable has substantially larger variance than other variables, basically that variable will be selected as the first principal component. But, we don't usually want to give a higher weight to one variable or a few, especially when units of the variables are different or arbitrary.

Chapter 38

Output from PCA

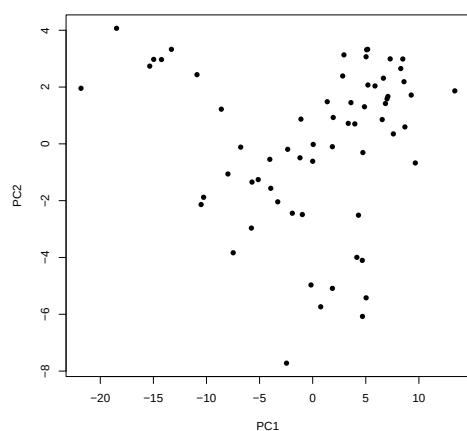
```
# somewhat correlated variables only, otherwise too little
  ↳ variance explained by PC1
corr <- cor(nci)
diag(corr) <- NA      # to use arrayInd
is_min_corr <- apply(corr, 1, \(x) max(abs(x), na.rm = TRUE) >
  ↳ 0.8 & mean(abs(x), na.rm = TRUE) > 0.21)
min_corr <- nci[, is_min_corr]

pca <- prcomp(min_corr, scale. = TRUE)
```

38.1 Principal components

The output presented and investigated most frequently from PCA is the first two principal components (PCs). The values of those two PCs, PC1 and PC2 scores, are often presented in a scatter plot. It is called “PCA score plot” or simply “PCA plot”.

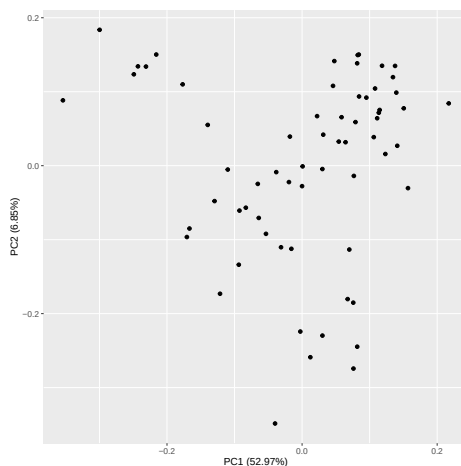
```
plot(pca$x[, 1:2], pch = 16)
```



The x and y axes of the plot often are modified. The labels include how much proportion of the variance was explained by the axis. The axes are standardized to have a similar range.

```
library(ggfortify)
autoplot(pca)
```

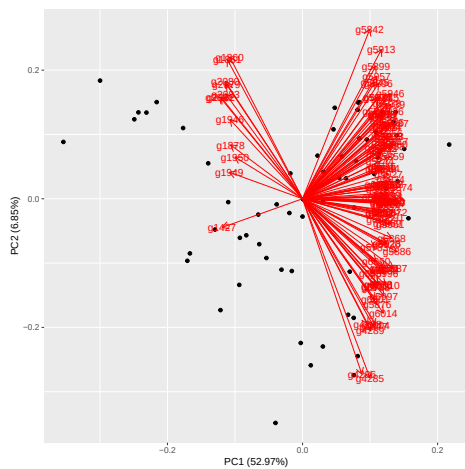
```
Warning: `aes_string()` was deprecated in ggplot2 3.0.0.
i Please use tidy evaluation idioms with `aes()`.
i See also `vignette("ggplot2-in-packages")` for more information.
i The deprecated feature was likely used in the ggfortify package.
  Please report the issue at <https://github.com/sinhrks/ggfortify/issues>.
```



38.2 Biplot (Scores + Loadings)

How much each variable contribute to each principal component is stored in vectors. They are called **loadings**, which are often presented together with the scores. The plot is called “Biplot”.

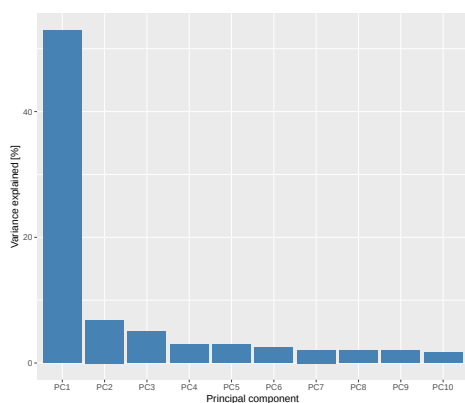
```
autoplot(pca, loadings = TRUE, loadings.label = TRUE)
```



38.3 Scree plot

The proportion explained by each principal component is presented in a bar or line plot. It is called “scree plot”.

```
scree <- tibble(
  pc = paste0("PC", seq(pca$sdev)),
  s = pca$sdev^2,
  expl = s / sum(s) * 100
) |>
mutate(pc = factor(pc, levels = pc)) |>
slice(1:10)
ggplot(scree, aes(pc, expl)) +
  geom_col(fill = "steelblue") +
  xlab("Principal component") +
  ylab("Variance explained [%]")
```



Exercises: Principal component analysis

You have found an older gene-expression dataset generated using microarray technology. It contains measurements from 22,215 probes in 189 samples from seven tissues: kidney, hippocampus, cerebellum, colon, liver, endometrium and placenta.

The data file can be loaded directly in R. It is also available from the original workshop repository.

Load the file and inspect the expression matrix and sample annotation.

```
data_file <- file.path("data", "tissuesGeneExpression.rda")
load(data_file)
```

```
# Expression matrix: probes x samples
dim(e)
```

```
[1] 22215 189
```

```
e[1:4, 1:5]
```

	GSM11805.CEL.gz	GSM11814.CEL.gz	GSM11823.CEL.gz	GSM11830.CEL.gz
1007_s_at	10.19	10.51	10.27	10.25
1053_at	6.04	6.70	6.14	6.58
117_at	7.45	7.78	7.70	8.48
121_at	12.03	12.01	11.63	11.08

	GSM12067.CEL.gz
1007_s_at	10.16
1053_at	6.61
117_at	8.12
121_at	10.83

```
# Sample information
str(tab)
```

```
'data.frame': 189 obs. of 6 variables:
 $ filename      : chr  "GSM11805.CEL.gz" "GSM11814.CEL.gz" "GSM11823.CEL.gz" "GSM11830.CEL.gz"
 $ DB_ID         : chr  "GSM11805" "GSM11814" "GSM11823" "GSM11830" ...
```

```
$ ExperimentID : chr  "GSE781" "GSE781" "GSE781" "GSE781" ...
$ Tissue       : chr  "kidney" "kidney" "kidney" "kidney" ...
$ SubType      : chr  "normal" "cancer" "normal" "cancer" ...
$ ClinicalGroup: chr  NA NA NA NA ...
```

`e` contains probes in rows and samples in columns, while `tab` contains the sample information. Match the rows of `tab` to the columns of `e` before running PCA.

```
stopifnot(
  ncol(e) == nrow(tab),
  !anyDuplicated(colnames(e)),
  !anyDuplicated(tab$filename),
  setequal(colnames(e), tab$filename)
)

tab <- tab[match(colnames(e), tab$filename), , drop = FALSE]
stopifnot(identical(colnames(e), tab$filename))
```

The file contains four expression profiles that exactly repeat earlier columns. Keeping both copies would give those liver samples twice the weight in the PCA, so we keep the first copy of each profile.

```
repeated_profile <- duplicated(e, MARGIN = 2)

repeated_samples <- tab[
  repeated_profile,
  c("filename", "ExperimentID", "Tissue", "SubType"),
  drop = FALSE
]
rownames(repeated_samples) <- NULL
repeated_samples
```

	filename	ExperimentID	Tissue	SubType
1	GSM299110.CEL.gz	GSE11819	liver	ihca
2	GSM299111.CEL.gz	GSE11819	liver	ihca
3	GSM299112.CEL.gz	GSE11819	liver	ihca
4	GSM299113.CEL.gz	GSE11819	liver	ihca

```
e <- e[, !repeated_profile, drop = FALSE]
tab <- tab[!repeated_profile, , drop = FALSE]

dim(e)
```

```
[1] 22215 185
```

Exercise 1: PCA of all samples

`prcomp()` expects observations in rows and variables in columns.

1. Transpose `e` so that samples are rows and probes are columns. Call the new matrix `x`.

2. How many observations and variables does `x` contain after removing the repeated profiles?
3. What is the maximum number of principal components that can have non-zero variance after centring? Explain why.
4. Run PCA with centring but without scaling.
5. Calculate the proportion of variance explained by each component. How much do PC1, PC2 and the two together explain?
6. Draw a scree plot for the first 20 components.
7. Plot PC1 against PC2 without using the tissue labels. What groups or unusual samples can you see?
8. Colour the same points by `Tissue`. How does this change your reading of the plot?

Tip

Use `t(e)` to put samples in rows and `dim()` to check the result. Run the PCA with `prcomp(x, center = TRUE, scale. = FALSE)`. The score matrix is stored in `pca$x`, while the component standard deviations are in `pca$sdev`. Square `pca$sdev` and divide by the sum of the squared values to obtain the proportion of variance explained. For plotting, combine `as.data.frame(pca$x)` with `tab` using `cbind()`.

Solution

```
x <- t(e)
dim(x)
```

```
[1] 185 22215
```

There are 185 observations and 22,215 variables. Centring makes the rows of `x` linearly dependent, so its rank cannot exceed $n - 1$. The maximum is therefore

$$\min(p, n - 1) = \min(22215, 184) = 184.$$

```
pca <- prcomp(x, center = TRUE, scale. = FALSE)
```

```
pve <- pca$sdev^2 / sum(pca$sdev^2)
```

```
round(100 * c(
  PC1 = pve[1],
  PC2 = pve[2],
  PC1_PC2 = sum(pve[1:2])
), 1)
```

PC1	PC2	PC1_PC2
32.9	13.0	45.8

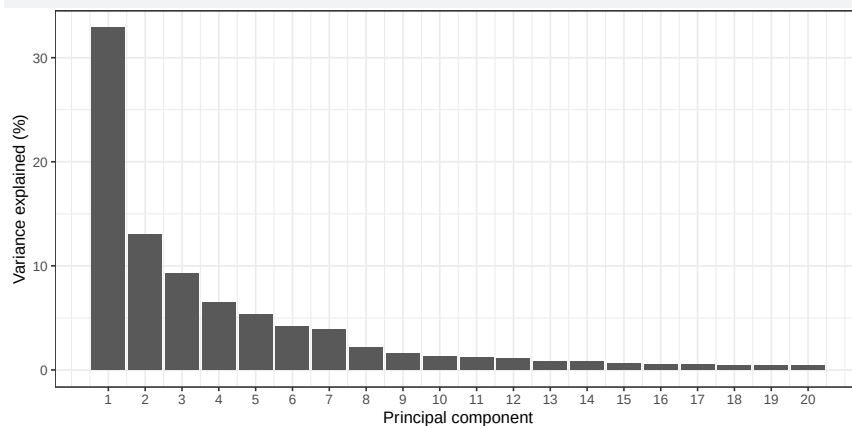
```
pc_axis <- function(pc, variance) {
  sprintf("PC%d (%.1f%%)", pc, 100 * variance[pc])
}
```

The probes are measured on the same log-expression scale, so this PCA leaves them unscaled. Probes with greater variance therefore have more influence on the components.

```
screes <- data.frame(
  PC = seq_along(pve),
  variance = 100 * pve
)

first_pcs <- seq_len(min(20, nrow(screes)))

ggplot(screes[first_pcs, , drop = FALSE], aes(x = PC, y =
  ↪ variance)) +
  geom_col() +
  scale_x_continuous(breaks = first_pcs) +
  labs(
    x = "Principal component",
    y = "Variance explained (%)"
  ) +
  theme_bw()
```

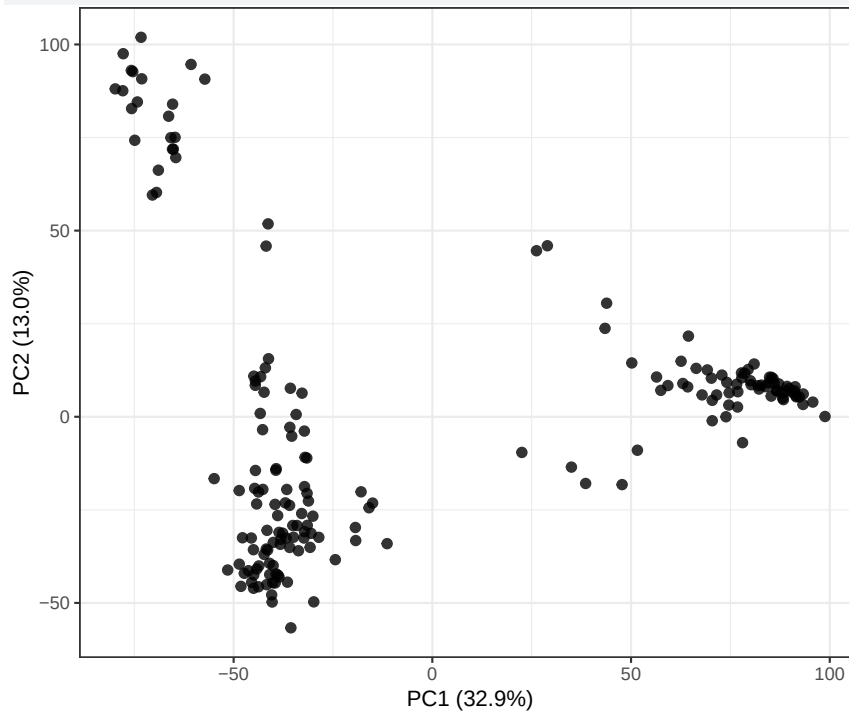


The rows of `pca$x` and `tab` are in the same order because the metadata were aligned above.

```
scores <- cbind(as.data.frame(pca$x), tab)
```

First plot the scores without adding sample information.

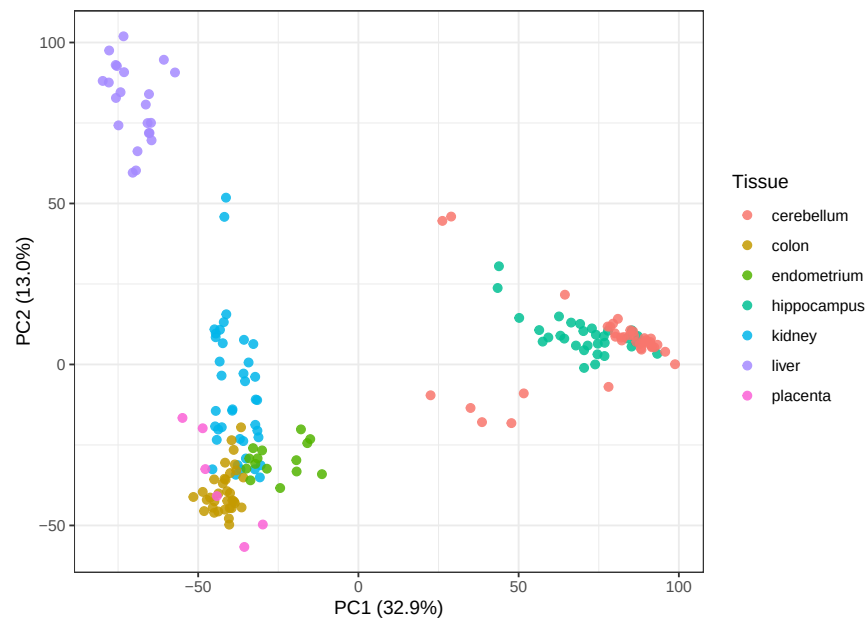
```
ggplot(scores, aes(x = PC1, y = PC2)) +
  geom_point(size = 2, alpha = 0.8) +
  labs(
    x = pc_axis(1, pve),
    y = pc_axis(2, pve)
  ) +
  theme_bw()
```



Without labels, the points form several compact groups: one at high PC2, a broad group at negative PC1 and another at positive PC1. A few samples lie between or apart from these groups, but this plot alone does not tell us what they represent.

Then colour the same points by tissue.

```
ggplot(scores, aes(x = PC1, y = PC2, color = Tissue)) +
  geom_point(size = 2, alpha = 0.85) +
  labs(
    x = pc_axis(1, pve),
    y = pc_axis(2, pve)
  ) +
  theme_bw()
```



Tissue labels were not used to calculate the principal components. The groups that appear after adding the labels show that tissue is associated with a large part of the expression variation. This is an association, not evidence that tissue alone caused the separation.

Exercise 2: Looking beyond PC1 and PC2

PC1 and PC2 describe the two largest directions of total variation, but later components can still contain clear patterns.

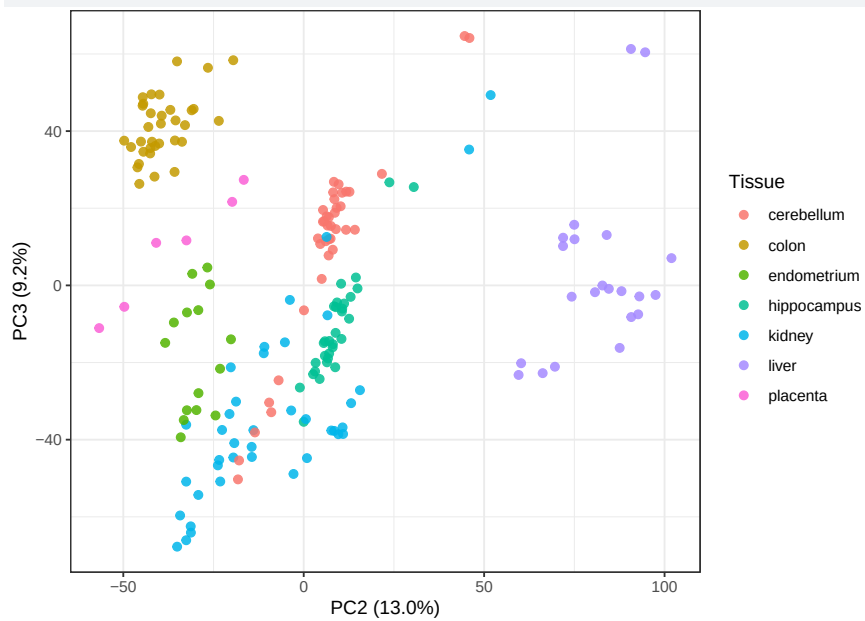
1. For the full dataset, plot PC2 against PC3 and PC3 against PC4. Colour the points by tissue.
2. Which tissue groups become easier to distinguish in these views?
3. Why might a component with modest overall variance still be useful?
4. Select the kidney, colon and liver samples and fit a new PCA from the expression matrix.
5. How many samples remain?
6. Plot PC1 against PC2 and PC3 against PC4 for the new PCA.
7. What happens to the kidney samples along PC4?

💡 Tip

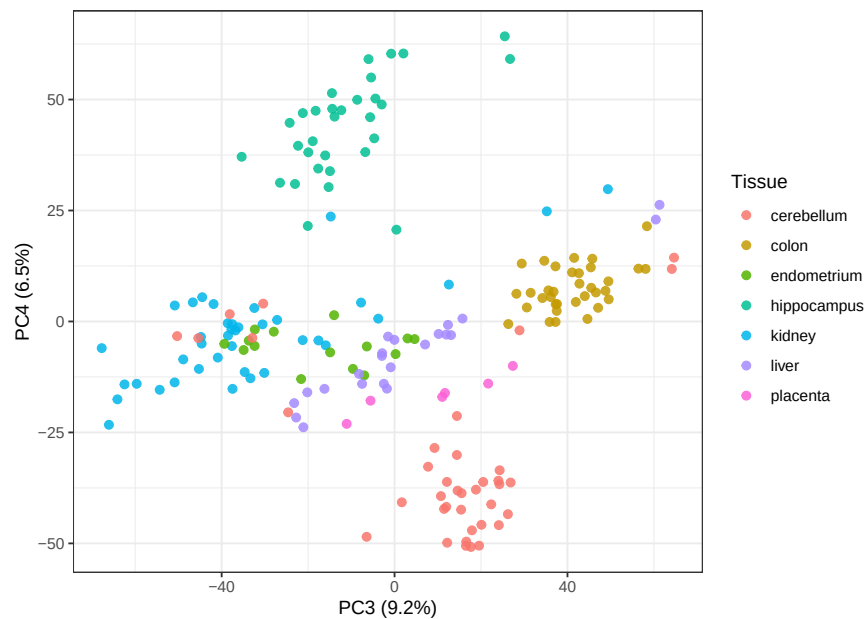
The full-data scores are already in `scores`, so PC2, PC3 and PC4 can be used directly in `ggplot()`. Select samples with `tab$Tissue %in% c("kidney", "colon", "liver")`. Apply that logical vector to both `e` and `tab`, keep matrix dimensions with `drop = FALSE`, transpose the selected expression matrix and run a new `prcomp()`. The new scores are in `pca3$x`.

💡 Solution

```
ggplot(scores, aes(x = PC2, y = PC3, color = Tissue)) +
  geom_point(size = 2, alpha = 0.85) +
  labs(
    x = pc_axis(2, pve),
    y = pc_axis(3, pve)
  ) +
  theme_bw()
```



```
ggplot(scores, aes(x = PC3, y = PC4, color = Tissue)) +
  geom_point(size = 2, alpha = 0.85) +
  labs(
    x = pc_axis(3, pve),
    y = pc_axis(4, pve)
  ) +
  theme_bw()
```



In the PC2–PC3 plot, colon and liver occupy distinct regions. The PC3–PC4 plot separates hippocampus from cerebellum more clearly along PC4. A component can describe a strong pattern in a small group of samples without accounting for much of the variance across the full dataset. Fit a new PCA after selecting kidney, colon and liver samples. Removing the other tissues changes the centre and covariance structure, so the principal change.

```
keep3 <- tab$Tissue %in% c("kidney", "colon", "liver")
```

```
x3 <- t(e[, keep3, drop = FALSE])
tab3 <- tab[keep3, , drop = FALSE]
```

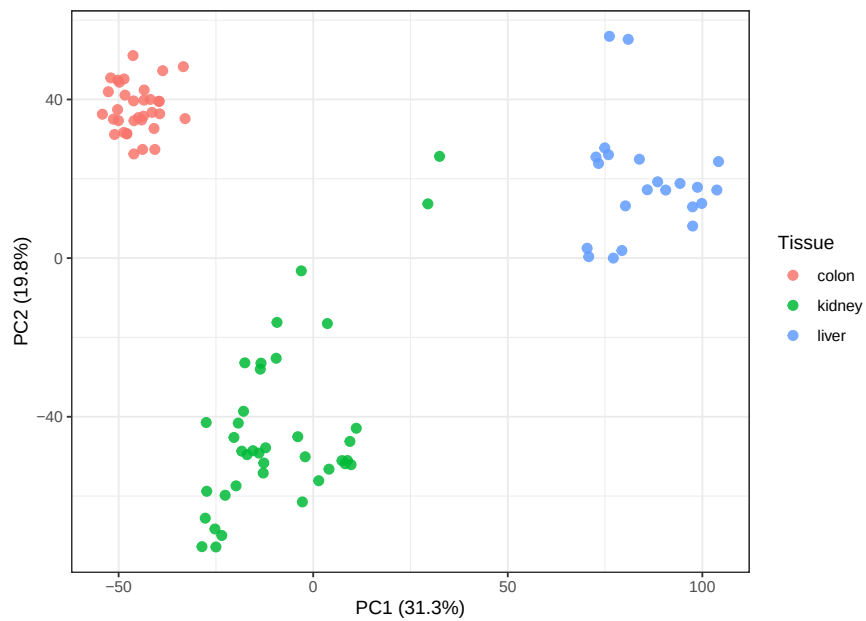
```
pca3 <- prcomp(x3, center = TRUE, scale. = FALSE)
pve3 <- pca3$sdev^2 / sum(pca3$sdev^2)
```

```
scores3 <- cbind(as.data.frame(pca3$x), tab3)
```

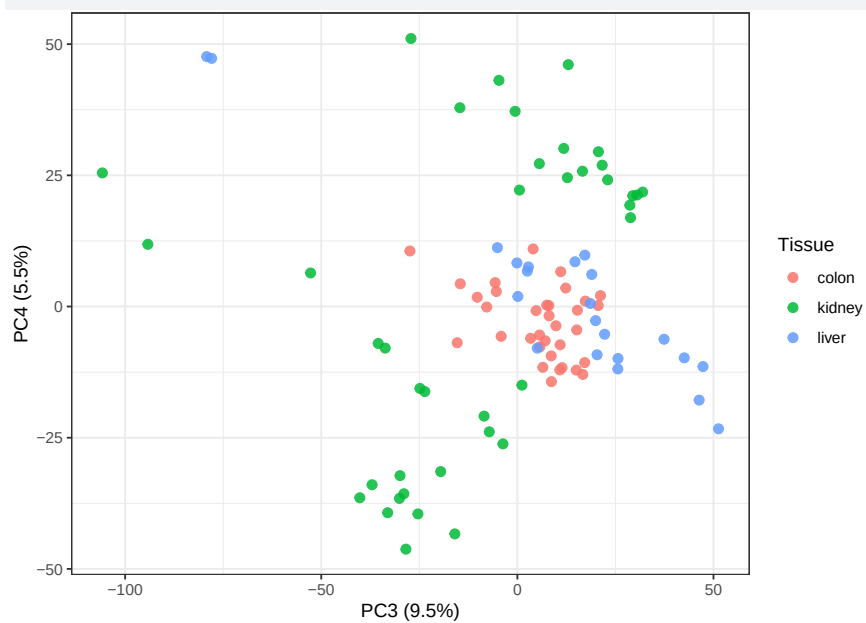
```
dim(x3)
```

```
[1] 95 22215
```

```
ggplot(scores3, aes(x = PC1, y = PC2, color = Tissue)) +
  geom_point(size = 2.3, alpha = 0.85) +
  labs(
    x = pc_axis(1, pve3),
    y = pc_axis(2, pve3)
  ) +
  theme_bw()
```

```
ggplot(scores3, aes(x = PC3, y = PC4, color = Tissue)) +
  geom_point(size = 2.3, alpha = 0.85) +
  labs(
    x = pc_axis(3, pve3),
    y = pc_axis(4, pve3)
  ) +
  theme_bw()
```



The kidney samples form several groups along PC4. This pattern is diffi-

cult to see in the full analysis. Once the broad differences among all seven tissues are removed, variation within the remaining tissues accounts for more of the total variation.

Exercise 3: Biology, experiment, or both?

A pattern in a PCA plot may be related to biology, study design, or both. Use the sample metadata to examine the visible groups.

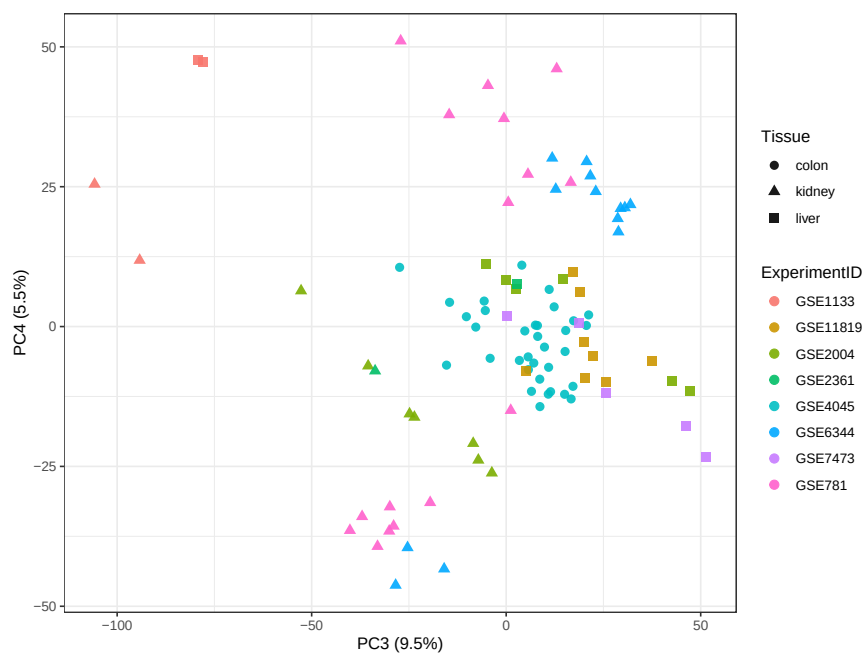
1. In the three-tissue PCA, plot PC3 against PC4 using colour for `ExperimentID` and shape for `Tissue`.
2. Examine `table(Tissue, ExperimentID)`. Are tissue and experiment independently balanced?
3. Does an association with `ExperimentID` prove that a component is a batch effect? What information would you need before making that claim?
4. Look at the kidney samples again, this time using colour for `SubType` and shape for `ExperimentID`. Does the separation on PC4 follow experiment or subtype more closely?
5. Fit a separate PCA using only hippocampus and cerebellum samples.
6. Plot PC1 against PC2 using shape for `Tissue` and colour for `ExperimentID`.
7. Do the brain samples follow tissue or experiment more closely? Can those effects be separated with this study design?

💡 Tip

Use `with(tab3, table(Tissue, ExperimentID))` to inspect the study design. In `ggplot()`, map one metadata column to `color` and another to `shape`. Subset kidney rows with `scores3$Tissue == "kidney"`. For the brain analysis, select the two tissues from `e` and `tab`, transpose the expression matrix and run a separate `prcomp()`. Match `pca_brain$x` to `tab_brain` with `cbind()` before plotting.

💡 Solution

```
ggplot(
  scores3,
  aes(x = PC3, y = PC4, color = ExperimentID, shape =
    ↪ Tissue)
) +
  geom_point(size = 2.5, alpha = 0.9) +
  labs(
    x = pc_axis(3, pve3),
    y = pc_axis(4, pve3)
  ) +
  theme_bw()
```



```
with(tab3, table(Tissue, ExperimentID))
```

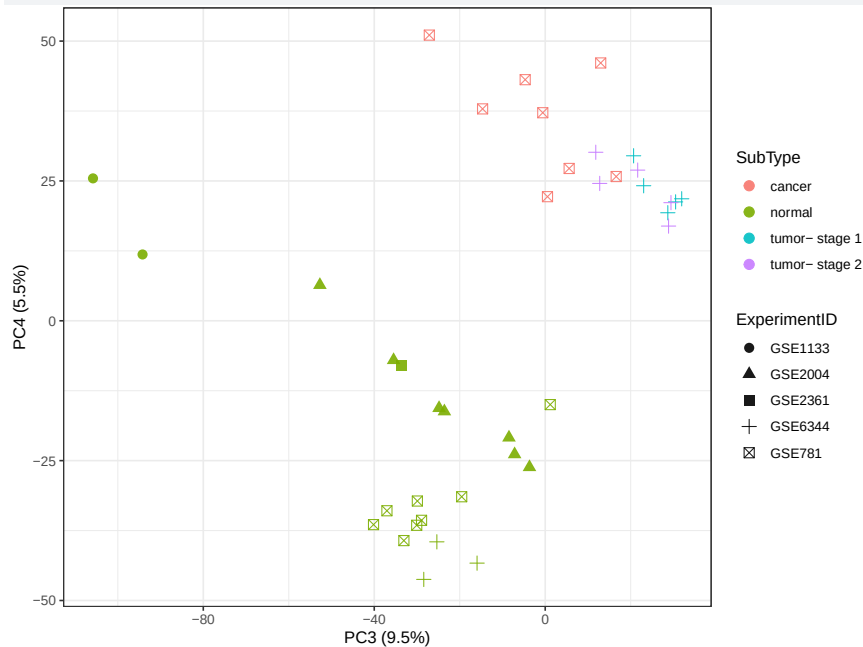
	ExperimentID							
Tissue	GSE1133	GSE11819	GSE2004	GSE2361	GSE4045	GSE6344	GSE7473	GSE781
colon	0	0	0	0	34	0	0	0
kidney	2	0	7	1	0	13	0	16
liver	2	8	6	1	0	0	5	0

Colon samples come from one experiment, while kidney and liver samples are spread across several experiments. Tissue and experiment are not independently balanced. An apparent tissue pattern may contain study-specific variation, and an **ExperimentID** pattern is not by itself proof of a batch effect. To make that claim, we would need information about sample collection, laboratory and processing dates, platform, normalisation and any technical replicates.

Compare experiment and subtype within the kidney samples.

```
kidney3 <- scores3$Tissue == "kidney"

ggplot(
  scores3[kidney3, , drop = FALSE],
  aes(x = PC3, y = PC4, color = SubType, shape =
    ExperimentID)
) +
  geom_point(size = 2.8, alpha = 0.9) +
  labs(
    x = pc_axis(3, pve3),
    y = pc_axis(4, pve3)
  ) +
  theme_bw()
```



Most normal kidney profiles lie lower on PC4. The GSE781 cancer samples and the GSE6344 tumour samples lie higher, so the separation follows subtype more closely than experiment. The variables are still partly confounded, and the plot does not establish a cause.

For the brain comparison:

```

keep_brain <- tab$Tissue %in% c("hippocampus", "cerebellum")

x_brain <- t(e[, keep_brain, drop = FALSE])
tab_brain <- tab[keep_brain, , drop = FALSE]

pca_brain <- prcomp(x_brain, center = TRUE, scale. = FALSE)
pve_brain <- pca_brain$sdev^2 / sum(pca_brain$sdev^2)

scores_brain <- cbind(as.data.frame(pca_brain$x), tab_brain)

with(tab_brain, table(Tissue, ExperimentID))

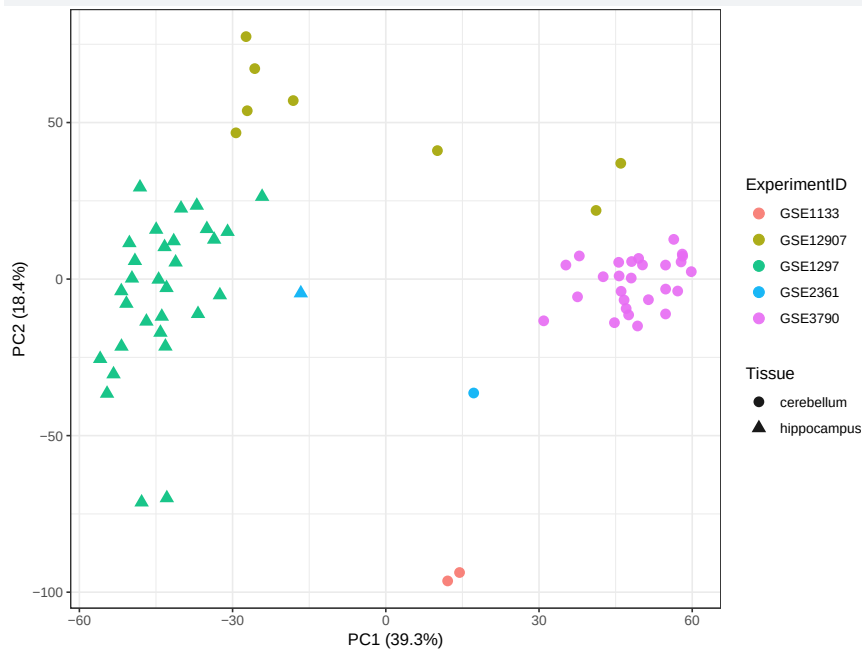
```

	ExperimentID				
Tissue	GSE1133	GSE12907	GSE1297	GSE2361	GSE3790
cerebellum	2	8	0	1	27
hippocampus	0	0	30	1	0

```

ggplot(
  scores_brain,
  aes(x = PC1, y = PC2, color = ExperimentID, shape =
    ↪ Tissue)
) +
  geom_point(size = 3, alpha = 0.9) +
  labs(
    x = pc_axis(1, pve_brain),
    y = pc_axis(2, pve_brain)
  ) +
  theme_bw()

```



The visible brain groups follow `ExperimentID` more closely, especially along PC2. However, 30 of the 31 hippocampus samples come from GSE1297, while none of the cerebellum samples do. The study design does not let us separate tissue from experiment cleanly.

Exercise 4: Which probes contribute to a component?

A biplot with 22,215 probes would be unreadable. Instead, use the rotation coefficients to find the probes that contribute most strongly to a selected component.

1. Find the 15 probes with the largest absolute coefficients for PC1 of the full PCA.
2. Why should the ranking use the absolute coefficient rather than only positive coefficients?
3. Plot the expression of the highest-ranking PC1 probe across tissues.
4. Repeat the calculation for PC4 of the three-tissue PCA.
5. For the highest-ranking PC4 probe, plot expression in kidney samples by `SubType` and colour the points by `ExperimentID`.
6. What does a large absolute rotation coefficient tell you? Does it identify a causal probe?
7. Would the PCA result change if every score and coefficient for one component changed sign?

Tip

The probe coefficients are in `pca$rotation`; for example, `pca$rotation[, 1]` contains the PC1 coefficients. Use `abs()`, `order(..., decreasing = TRUE)` and integer indexing to find the largest values. A probe's expression values can be extracted with `e[probe_name, ]`. For the kidney plot, use the matching column of `x3` and map `SubType` to one axis and `ExperimentID` to colour.

Solution

Define a helper function that returns the largest absolute coefficients.

```

top_coefficients <- function(pca_object, pc = 1, n = 15) {
  z <- pca_object$rotation[, pc]
  n <- min(n, length(z))
  idx <- order(abs(z), decreasing = TRUE)[seq_len(n)]

  data.frame(
    probe = names(z)[idx],
    coefficient = unname(z[idx]),
    abs_coefficient = unname(abs(z[idx]))
  )
}

top_pc1 <- top_coefficients(pca, pc = 1)
top_pc1

```

	probe	coefficient	abs_coefficient
1	203485_at	0.0538	0.0538
2	209469_at	0.0526	0.0526
3	209167_at	0.0512	0.0512
4	203540_at	0.0510	0.0510
5	209072_at	0.0510	0.0510
6	210198_s_at	0.0507	0.0507
7	209170_s_at	0.0506	0.0506
8	202508_s_at	0.0488	0.0488
9	209470_s_at	0.0485	0.0485
10	205358_at	0.0477	0.0477
11	203130_s_at	0.0459	0.0459
12	203296_s_at	0.0447	0.0447
13	203001_s_at	0.0435	0.0435
14	209169_at	0.0433	0.0433
15	206373_at	0.0432	0.0432

Large positive and negative coefficients both contribute strongly to a component, so the ranking uses their absolute values.

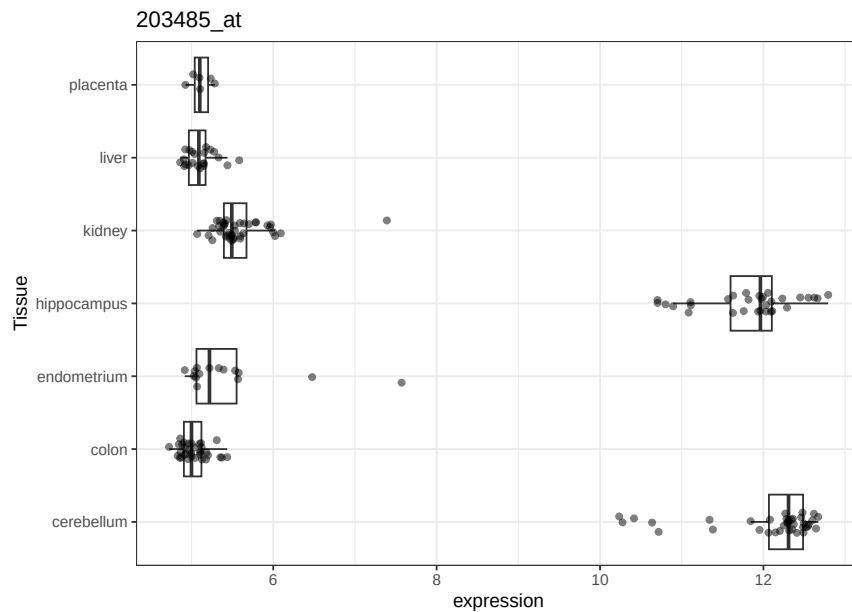
```

top_probe_pc1 <- top_pc1$probe[1]

probe_pc1 <- data.frame(
  expression = as.numeric(e[top_probe_pc1, ]),
  Tissue = tab$Tissue
)

ggplot(probe_pc1, aes(x = Tissue, y = expression)) +
  geom_boxplot(outlier.shape = NA) +
  geom_jitter(width = 0.15, alpha = 0.5) +
  coord_flip() +
  labs(title = top_probe_pc1) +
  theme_bw()

```



Repeat the calculation for PC4 of the three-tissue PCA.

```
top_pc4_3 <- top_coefficients(pca3, pc = 4)
top_pc4_3
```

	probe	coefficient	abs_coefficient
1	205625_s_at	-0.0686	0.0686
2	205626_s_at	-0.0639	0.0639
3	205030_at	0.0618	0.0618
4	218484_at	0.0614	0.0614
5	202238_s_at	0.0583	0.0583
6	206054_at	-0.0582	0.0582
7	202237_at	0.0576	0.0576
8	205719_s_at	-0.0572	0.0572
9	206025_s_at	0.0563	0.0563
10	204704_s_at	-0.0548	0.0548
11	206716_at	-0.0535	0.0535
12	211298_s_at	-0.0530	0.0530
13	205029_s_at	0.0523	0.0523
14	201839_s_at	-0.0508	0.0508
15	217238_s_at	-0.0487	0.0487

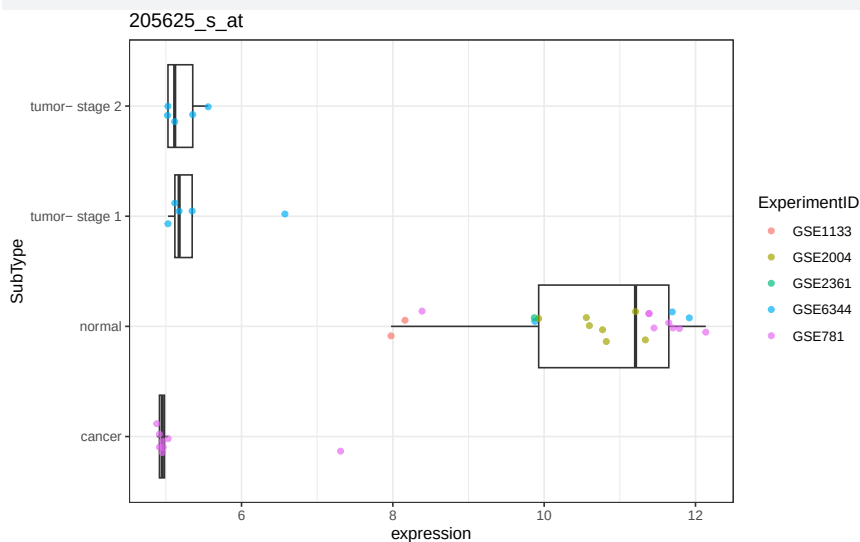

```

top_probe_pc4 <- top_pc4_3$probe[1]
kidney <- tab3$Tissue == "kidney"

probe_pc4_kidney <- data.frame(
  expression = as.numeric(x3[kidney, top_probe_pc4]),
  SubType = tab3$SubType[kidney],
  ExperimentID = tab3$ExperimentID[kidney]
)

ggplot(probe_pc4_kidney, aes(x = SubType, y = expression)) +
  geom_boxplot(outlier.shape = NA) +
  geom_jitter(
    aes(color = ExperimentID),
    width = 0.15,
    height = 0,
    alpha = 0.7
  ) +
  coord_flip() +
  labs(title = top_probe_pc4) +
  theme_bw()

```



The probe is expressed more strongly in the normal kidney profiles and more weakly in the GSE781 cancer group and the GSE6344 tumour groups. This agrees with the PC4 pattern, but the probe was selected from the same PCA and does not provide independent evidence.

A large absolute rotation coefficient means that a probe contributes strongly to the direction of that component. It does not show that the probe caused the pattern. The sign of a PCA axis is arbitrary: reversing both its scores and coefficients gives the same solution.

Exercise 5: A simple biplot

A biplot shows samples and variables together. It quickly becomes unreadable with 22,215 probes, so this exercise uses four probes with contrasting loadings.

1. From `pca$rotation`, find the probes with the most positive and most negative loadings on PC1 and PC2.
2. Combine the four probe names and inspect their PC1 and PC2 loadings.
3. Extract these probes from `x` and run a new PCA using only these columns. Centre and scale the columns.
4. Draw the result with base R `biplot()`.
5. Which arrows point in similar or opposite directions?
6. Calculate the correlation matrix for the four probes and draw a scatterplot matrix.
7. Do the correlations agree with the angles in the biplot?

💡 Tip

The loading matrix is `pca$rotation`. Use `which.max()` and `which.min()` on its PC1 and PC2 columns, then use the selected names to subset `x`. Run `prcomp(..., center = TRUE, scale. = TRUE)` on the four-column matrix. Base R `biplot()` draws the samples and probe arrows. Use `cor()` for the exact correlations and `pairs()` to inspect them graphically.

💡 Solution

Select one probe from each end of PC1 and PC2 in the full PCA.

```
loading12 <- pca$rotation[, c("PC1", "PC2"), drop = FALSE]

selected_probes <- unique(c(
  rownames(loading12)[which.max(loading12[, "PC1"])],
  rownames(loading12)[which.min(loading12[, "PC1"])],
  rownames(loading12)[which.max(loading12[, "PC2"])],
  rownames(loading12)[which.min(loading12[, "PC2"])])
))

selected_loadings <- data.frame(
  probe = selected_probes,
  loading12[selected_probes, , drop = FALSE],
  row.names = NULL
)
selected_loadings
```

	probe	PC1	PC2
1	203485_at	0.0538	0.0134
2	201596_x_at	-0.0390	-0.0127
3	203400_s_at	0.0191	0.0652
4	201650_at	-0.0246	-0.0448

These probes were chosen because their loadings point towards different

ends of the first two component axes. The selection is meant to make the biplot readable; it is not a list of the most biologically important probes. Fit a small PCA using only the four selected probes. Scaling is useful here because the purpose is to compare their correlations.

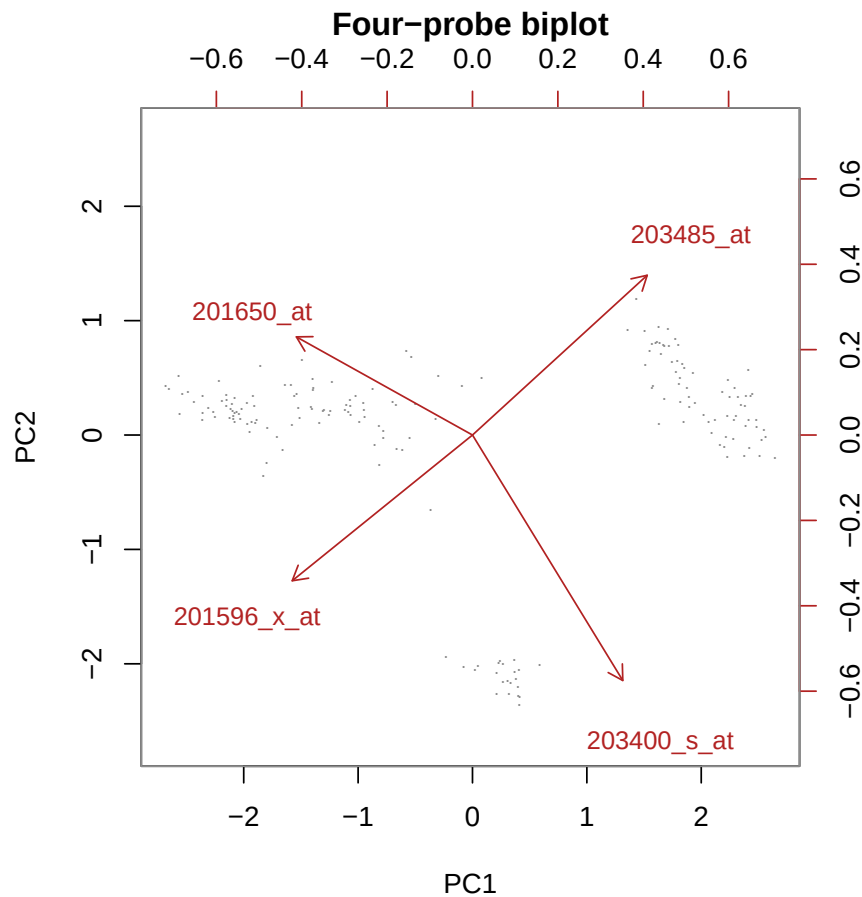
```
x_selected <- x[, selected_probes, drop = FALSE]

pca_selected <- prcomp(
  x_selected,
  center = TRUE,
  scale. = TRUE
)

round(100 * summary(pca_selected)$importance[2, 1:2], 1)
```

```
PC1 PC2
75.4 17.0
```

```
biplot(
  pca_selected,
  scale = 0,
  xlabs = rep(".", nrow(x_selected)),
  col = c("grey55", "firebrick"),
  cex = c(0.7, 0.9),
  main = "Four-probe biplot"
)
```



The grey dots are samples and the red arrows are probes. Arrows pointing in the same direction suggest positive correlation, arrows at about 90 degrees suggest little linear correlation, and arrows pointing in opposite directions suggest negative correlation. A 45-degree angle corresponds roughly to a correlation of $\cos(45^\circ) = 0.71$ when the two displayed components represent both probes well.

The first two components explain most of the variance in these four standardised probes, so the arrow angles give a useful summary. The correlation matrix gives the exact pairwise values.

```
round(cor(x_selected), 2)
```

	203485_at	201596_x_at	203400_s_at	201650_at
203485_at	1.00	-0.91	0.49	-0.64
201596_x_at	-0.91	1.00	-0.49	0.74
203400_s_at	0.49	-0.49	1.00	-0.73
201650_at	-0.64	0.74	-0.73	1.00

Use a scatterplot matrix to see the same relationships in the original expression values. The upper panels report Pearson correlations; the lower panels show the observations and a smooth trend.

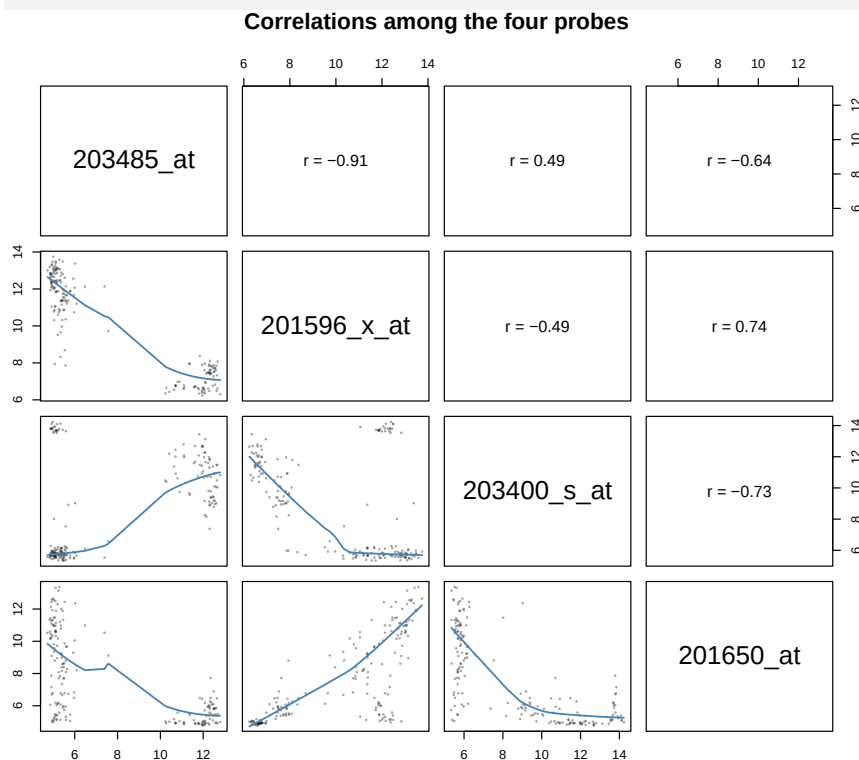
```

panel_cor <- function(x, y) {
  old_par <- par("usr")
  on.exit(par(usr = old_par))
  par(usr = c(0, 1, 0, 1))
  text(0.5, 0.5, sprintf("r = %.2f", cor(x, y)), cex = 1.3)
}

panel_points <- function(x, y) {
  points(
    x, y,
    pch = 16,
    cex = 0.45,
    col = adjustcolor("black", alpha.f = 0.35)
  )
  lines(lowess(x, y), col = "steelblue", lwd = 1.5)
}

pairs(
  x_selected,
  upper.panel = panel_cor,
  lower.panel = panel_points,
  main = "Correlations among the four probes"
)

```



The two nearly opposite arrow pairs match strong negative correla-

tions: 203485_at with 201596_x_at ($r = -0.91$), and 203400_s_at with 201650_at ($r = -0.73$). The acute angle between 203485_at and 203400_s_at matches their moderate positive correlation ($r = 0.49$). The matrix is the exact result; the biplot angle is a visual approximation based on two components.

Part VII

Clustering

Aims

- to introduce cluster analysis with focus on partitioning and hierarchical methods

Learning outcomes

- to be able to explain what clustering is
- to be able to run k-means, PAM and HCL cluster analysis
- to be able to evaluate cluster solutions

Chapter 39

Clustering: art of finding groups

```
library(tidyverse)
library(dendextend)
library(patchwork)
library(knitr)
library(kableExtra)
library(cluster)
library(RColorBrewer)
library(ggalt)
library(factoextra)
library(NbClust)
library(pvclust)
library(ComplexHeatmap)
library(circlize)

dir_images <- "images/"
```

39.1 Introduction

- Clustering is about finding structure in the data by identifying natural groupings (clusters). In other words, clustering aims to group object according to similarity. As a results, **a cluster** is a collection of objects (samples, genes etc.) that are more “similar” to each other that they are to cases in other clusters.
- Clustering for a set of n objects is usually performed based on a selection of p variables. We can for instance cluster n blood samples based on p gene expression measurements or we can cluster n genes based on the p samples blood profiles.
- Clustering is **unsupervised learning**, where we do not use any data la-

bels to perform grouping. It is commonly used in EDA to reveal structures in the data.

```
include_graphics("images/fruits.png")
```

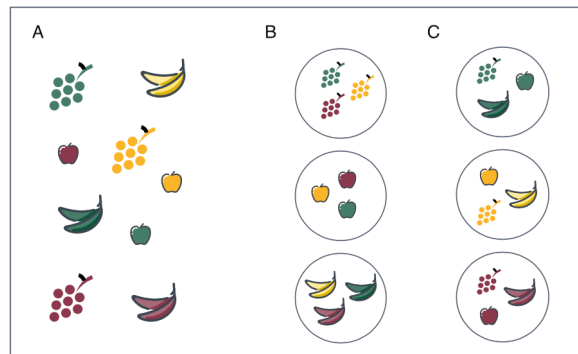


Figure 39.1: Grouping fruits (A) can be done in more than one natural way. For instance, we can group by fruit type (B) or by fruit color (C). Often, the usefulness of the grouping is in the eye of the beholder, something that should not be forgotten when promoting a particular grouping over other possibilities.

39.2 Common usage

- Grouping samples based on clinical data
- Grouping samples based on gene expression profiles
- Grouping cells by cell types, e.g. in scRNA-seq
- Biomarker studies
- Disease subtyping and stratification
- Clustering metagenomics contigs
- Species classification and biodiversity studies

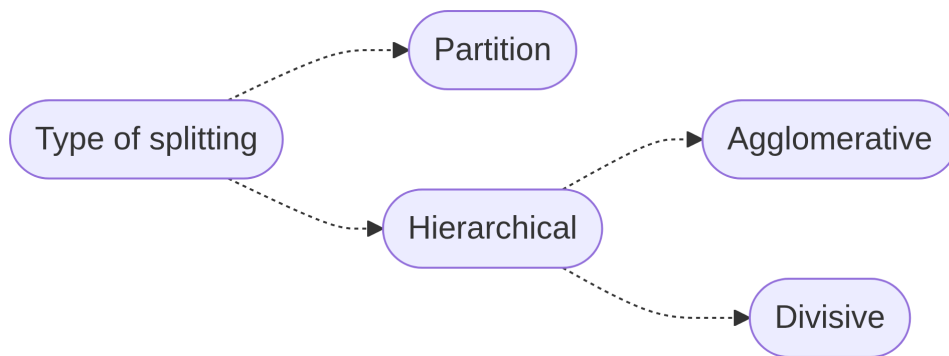
39.3 Partition methods vs. hierarchical

```
flowchart LR
```

```

A([Type of splitting]) --> B([Partition])
A([Type of splitting]) --> C([Hierarchical])
C([Hierarchical]) --> D([Agglomerative])
C([Hierarchical]) --> E([Divisive])

```



- The partitioning and hierarchical methods are two general classes of clustering methods.
- In **partitioning** methods address the problem of dividing n objects, described by p variables, into a small number (k) of discrete classes.
- For instance, considering only two genes A and B ($p = 2$) we can group 70 blood samples ($n = 70$) into three clusters ($k = 3$).

```

# simulate data
set.seed(190)
df <- tibble(class = rep(c("c11", "c12", "c13"), c(30, 15, 25)))
  <- |>
  mutate(
    x = rnorm(n(), c(c11 = 1, c12 = 3, c13 = 2)[class], sd =
      <- .35),
    y = rnorm(n(), c(c11 = 1, c12 = 1, c13 = 2)[class], sd =
      <- .35),
  )

# plot data
text_size <- 12
mytheme <- theme(legend.title = element_blank(),
  legend.position = "top",
  legend.text = element_text(size = text_size),
  panel.grid.major = element_blank(),
  panel.grid.minor = element_blank(),
  axis.text = element_text(size = text_size),
  axis.title = element_text(size = text_size))

mycols <- brewer.pal(6, "Set1")

p1 <- ggplot(df, aes(x, y)) +
  geom_point(size = 2) +
  theme_bw() +
  xlab("gene A") +
  ylab("gene B") +
  mytheme
  
```

```
# show clusters
p2 <- ggplot(df, aes(x, y, color = class)) +
  geom_point(size = 2) +
  theme_bw() +
  xlab("gene A") +
  ylab("gene B") +
  mytheme +
  scale_color_brewer(palette = "Set1")

p1 + p2
```

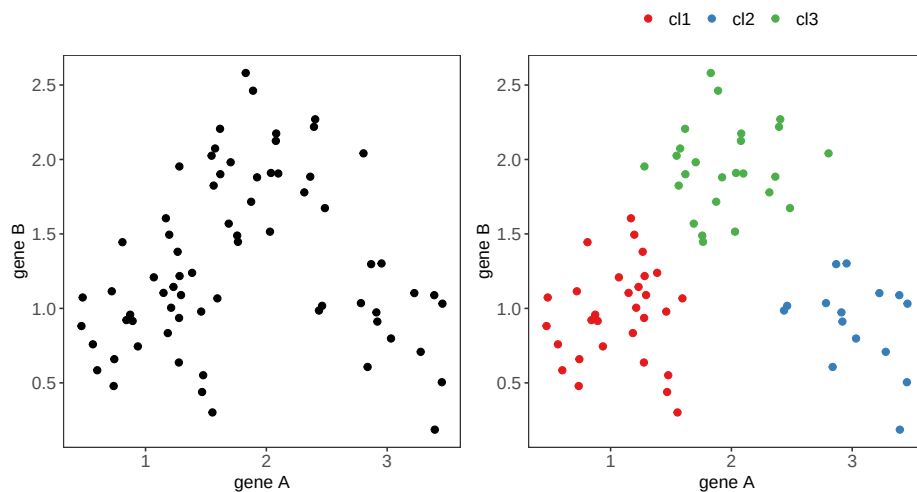


Figure 39.2: Illustration of partition data splitting. A) Mock-up data for 70 blood samples for which expression of two genes were measured (left). B) Following cluster analysis, samples are grouped into three clusters (right).

- **Divisive hierarchical** methods begin with all of the objects in one cluster which is broken down into sub-cluster until all objects are separated. **The agglomerative approach** works in the opposite direction as single-member clusters are increasingly fused (joint) until all objects are in one cluster.

```
include_graphics("images/hclust-ann.png")
```

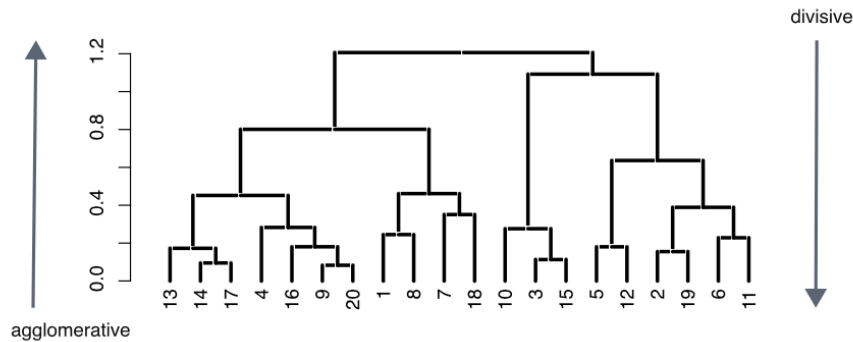


Figure 39.3: A dendrogram representation for hierarchical clustering of data objects.

39.4 Distance and similarity measures

- All clustering algorithms start by measuring the **similarity** between the objects to be clustered. Similar cases will be placed into the same cluster.
- It is also possible to view similarity by its inverse, the **distance** between the objects, with distance declining as similarity increases.
- There are large number of **distance metrics** by which distance can be measured. Some are restricted to particular data types.
- All distance metrics share the property that distance declines with increasing similarity. However, they **do not** share a common distance between the same two cases, i.e. **distance changes with the similarity measure**, leading to potential alternative clustering results.
- Three general classes of distance measures are **Euclidean metrics**, **Non-Euclidean metrics** and **Semi-metrics**.

Euclidean metrics

- These metrics measure true straight-line distances in Euclidean space.
- In a univariate example the Euclidean distance between two values is the arithmetic difference. In a bivariate case the minimum distance between two points is the hypotenuse of a triangle formed from the points (Pythagoras theorem). For three variables the hypotenuse extends through three-dimensional space. In n-dimensional space, the Euclidean distance between two points $A = (a_1, a_2, \dots, a_n)$ and $B = (b_1, b_2, \dots, b_n)$

$$distance(A, B) = \sqrt{\sum_{i=1}^n (a_i - b_i)^2}$$

- For instance, given two objects, red circle at $(5, 3)$ and yellow cylinder at $(3, 1)$ the Euclidean distance is $d = \sqrt{(3 - 5)^2 + (1 - 3)^2} \approx 2.82$

```
include_graphics("images/distances.png")
```

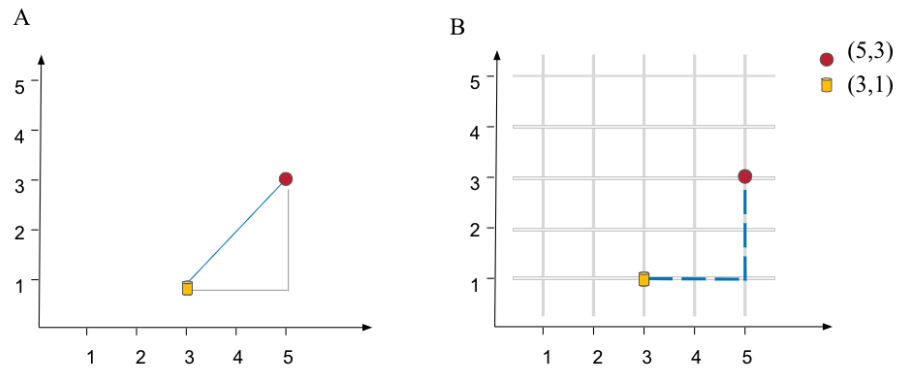


Figure 39.4: Illustration of distance metrics for a bivariate case. A) Euclidean metric, a distance between two points is the hypotenuse of a triangle formed from the points B) Non-Euclidean Manhattan metric.

Non-Euclidean metrics

- These are distances that are not straight lines, but which obey four rules. Let d_{ij} be the distance between two objects, i and j .
 1. d_{ij} must be 0 or positive (objects are identical, $d_{ij} = 0$, or they are different, $d_{ij} > 0$)
 2. $d_{ij} = d_{ji}$, distance from A to B is the same as from B to A
 3. $d_{jj} = 0$, an object is identical to itself
 4. $d_{ik} \leq d_{ij} + d_{jk}$, when considering three objects the distance between any two of them cannot exceed the sum of the distances between the other two pairs. In other words, the distances can construct a triangle.
- The **Manhattan** or City Block metric is an example of this type of distance metrics.

$$\text{distance}(A, B) = \sum |a_i - b_i|$$

- In our red circle and yellow cylinder, we get: $d = |3 - 5| + |1 - 3| = 4$

Semi metric

- Semi metric distances obey the first three aforementioned rules, but may not obey the last “triangle rule”.
- The **cosine** measure is an example of semi metric distance class. It is often used to compare vectors in text mining, where documents are often represented as vectors of numbers that represent word/term frequencies.


```
include_graphics("images/cosine.png")
```

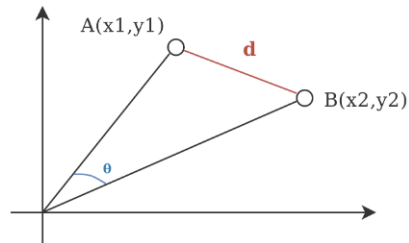


Figure 39.5: Illustration of cosine distance metric, used for comparing vectors and text queries.

39.4.1 Distances for interval variables

Euclidean distance

- Already mentioned above, normal Pythagoras theorem extended to the appropriate number of dimensions.
- Can be used when measuring similarity between points in Euclidean space, e.g. working with spatial data such as geographic coordinates or points on a map.
- It may not be the best choice for high-dimensional data due to “distance concentration” phenomena. In high-dimensional space, the pair-wise distances between different data points converge to the same value as the dimensionality increases, making it hard to distinguish points that are near and far away.

Squared Euclidean distance

- The sum of the squared differences between scores for two cases on all variables, i.e. the squared length of the hypotenuse. This measure magnifies distances between cases that are further apart.
- Note that the squared Euclidean distance is not a metric as it does not fulfill the triangle inequality.

Chebychev

- The absolute maximum difference, on one variable, between two cases.

$$distance(A, B) = \max |a_i - b_i|$$

- This measures examines distances across all of the variables, but only uses the maximum distance. This one dimensional difference need not be constant across cases. For example, age could be used for one pair and height for another.

- Interesting application of the distance is by Ghaderyan and Beyrami (2020) who tried many distance metric in analysis of gait rhythm fluctuations for automatic neurodegenerative disease detection. They achieved good results using Chebyshev and cosine distance to distinguish patients with Amyotrophic Lateral Sclerosis, Parkinson's disease (PD) and and Huntington's disease (HD).

City block or Manhattan distance

- A distance that follows a route along the non-hypotenuse sides of a triangle. The name refers to the grid-like layout of most American cities which makes it impossible to go directly between two points.
- This metric is hence less affected by outliers than the Euclidean and squared Euclidean distances.

$$distance(A, B) = \sum |a_i - b_i|$$

- Is preferred over Euclidean distance when there is a high dimensionality in the data.
- May not be the best choice, if one wants to perform space rotation or wants a smooth and differentiable function (converge more easily).

Mahalanobis distance

- It is a generalized version of a Euclidean distance which weights variables using the **sample variance-covariance matrix**. Because the covariance matrix is used this also means that correlations between variables are taken into account.

$$distance(A, B) = [(a_i - b_i)^t S^{-1} (a_i - b_i)]^{1/2}$$

where S^{-1} is the inverse covariance matrix

Minkowski

- Minkowski distance is a generalization of both the Euclidean distance and the Manhattan distance in a normed vector space.
- It can be thought of as a way of measuring the length of the path between two points when moving along axes aligned with the coordinate system.
- The formula for the Minkowski distance of order p between two points $A = (a_1, a_2, \dots, a_n)$ and $B = (b_1, b_2, \dots, b_n)$ in an n -dimensional space is given by

$$distance(A, B) = \left(\sum_{i=1}^n |a_i - b_i|^p \right)^{1/p}$$

where p determines the form of the distance: for $p = 1$, it becomes the Manhattan distance (sum of the absolute differences of their coordinates); for $p = 2$, it yields the Euclidean distance (the shortest direct line between two points); and for $p = \infty$, it approaches the Chebyshev distance (the maximum difference along any coordinate dimension).

- The versatility of the Minkowski distance makes it applicable across various fields and data types, allowing adjustments for different spatial concepts and scales.

Table 39.1: Binary metrics use two or more of the values obtained from a two by two matrix of agreement.

		Case	i
		+	-
Case j	+	a	b
	-	c	d

Canberra

- The Canberra is a weighted version of the Manhattan distance, as it calculates the absolute difference between two vectors and normalizes it by dividing it by the absolute sum of their values

$$distance(A, B) = \sum_{i=1}^n \frac{|a_i - b_i|}{|a_i| + |b_i|}$$

- Blanco-Mallo et al. (2023) compared a series of distance metrics for machine learning purposes and showed that Canberra distance had the best overall performance and the highest tolerance to noise.

39.4.2 Distances for binary data

- There is large number of metrics available for binary data, and many have more than one name, with Dice and Jaccard measures used probably most often in biology. All of the distance measures for binary data use two or three values obtained from a simple two by two matrix of agreement.

```
x <- matrix(data = c("", "", "Case", "i",
                    "", "", "+", "-"),
            ncol = 4, nrow = 4,
            byrow = TRUE)

x %>%
  as_tibble() %>% #print()
  kbl(col.names = NULL, row.names = FALSE, align = c("c", "c",
    ↪ "c", "c")) %>%
  kable_paper(full_width = F)
```

where:

- a is the number of cases which both share the attribute
- d number of cases which neither have the attribute
- b and c are the number of cases in which only one of the pair has the attribute.
- Note: $a + b + c + d = n$, the sample size

Dice

- A similarity measure in which joint absences d are excluded from consideration, and matches a are weighted double.

- Also known as Czekanowski or Sorensen measure

$$distance(A, B) = 1 - 2a/(2a + b + c)$$

```
include_graphics("images/dice.png")
```

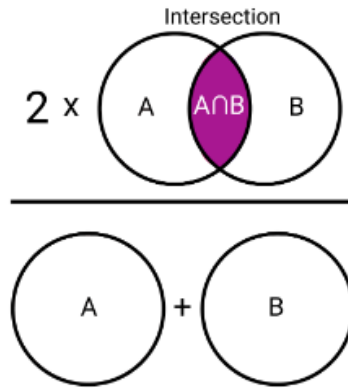


Figure 39.6: Illustration of dice distance metric, used for comparing binary data.

Jaccard

- A similarity measure in which joint absences d are excluded from consideration.
- Equal weight is given to matches and mismatches.

$$distance(A, B) = 1 - \frac{|A \cap B|}{|A \cup B|} = 1 - a/(a + b + c)$$

39.5 Partition methods: k-means & PAM

39.5.1 k-means

- The k-means clustering aims to divide all n objects, described by p variables, into exactly k clusters of discrete classes.
- The value of k , i.e. number of clusters, has to be given to the algorithm.
- The algorithm minimizes the variance within clusters, by iteratively assigning each object to the cluster with the closest **centroid**, where centroid of cluster k th is the arithmetic mean of all n_k objects in the cluster.

$$\mathbf{m}_k = \frac{1}{n_k} \sum_{i=1}^{n_k} \mathbf{x}_i$$

- The simplest version of k-means algorithm is by Lloyd and Forgy's Lloyd (1982). In R, `kmeans` functions implements a modified more effective version of this algorithm by Hartigan and Wong (1979).

The Lloyd and Forgy's algorithm

1. Initialization. Select k initial centroids. The initial centroids can be selected in several different ways. Two common methods are
 - Select k data points as initial centroids.
 - Randomly assign each data point to one out of k clusters and compute the centroids for these initial clusters.
2. Assign each object to the closest centroid (in terms of squared Euclidean distance). The squared Euclidean distance between an object (a data point) and a cluster centroid m_k is $d_i = \sum_{j=1}^p (x_{ij} - m_{kj})^2$. By assigning each object to closest centroid the total within cluster sum of squares (WSS) is minimized.

$$WSS = \sum_{k=1}^K \sum_{i \in C_k} \sum_{j=1}^p (x_{ij} - m_{kj})^2$$

3. Update the centroids for each of the k clusters by computing the centroid for all the objects in each of the clusters.
4. Repeat steps 2 and 3 until convergence.

kmeans

Figure 39.7: Interactive visualization of k-means algorithm can be found [here](#).

When to use and not to use k-means

- k-means is relatively fast and scalable, making it a good choice for large data sets. It works well with continuous, numeric data since it relies on Euclidean distances. Results are easy to interpret due to the simple assignment of data points to clusters. It works good if one knows the number of clusters or can get that the number of clusters for internal or external validation.
- On the downside, k-means is sensitive to the initial placement of cluster centroids. It can sometimes converge to local minima, resulting in sub-optimal cluster assignments. Outliers can also significantly impact the position of centroids and lead to poor cluster assignments.

39.5.2 PAM, partition around medoids

- PAM is very similar to k-means, but instead of using centroids we use **medoids** to represent clusters.
- **Medoid** is centrally located objects within the cluster.
- This makes PAM more robust when compare to k-means, but with higher computational complexity.

The algorithm can be described in few steps:

- Initialization: randomly select k objects as the initial medoids.
- Assignment: assign each object to the nearest medoid, forming k clusters.
- Update: for each cluster, find the new medoid by selecting the object that minimizes the sum of distances to all other objects in the cluster.

- Check for convergence: if none of the medoids have changed, the algorithm has converged. Otherwise, return to step 2 (Assignment) and repeat the process.

```
# example of running k-means and PAM in R (k = 3)

# simulate data
set.seed(190)
df <- tibble(class = rep(c("cl1", "cl2", "cl3"), c(30, 15, 25)))
  ↪ |>
  mutate(
    x = rnorm(n(), c(cl1 = 1, cl2 = 3, cl3 = 2)[class], sd =
      ↪ .35),
    y = rnorm(n(), c(cl1 = 1, cl2 = 1, cl3 = 2)[class], sd =
      ↪ .35),
  )

# extract x and y coordinates, convert to matrix
x <- df %>%
  dplyr::select(x, y) %>%
  as.matrix() %>%
  round(2)

# run k-means and PAM
res_kmeans <- kmeans(x, centers = 3)
res_pam <- pam(x, k = 3)

# prepare data frames for plotting
df_kmeans <- df %>%
  mutate(cluster = res_kmeans$cluster) %>%
  mutate(cluster = as.factor(cluster))

df_pam <- df %>%
  mutate(cluster = res_pam$clustering) %>%
  mutate(cluster = as.factor(cluster))

df_centroids <- res_kmeans$centers %>%
  as_tibble() %>%
  mutate(cluster = 1:3) %>%
  mutate(cluster = as.factor(cluster))

df_medoids <- res_pam$medoids %>%
  as_tibble() %>%
  mutate(cluster = 1:3) %>%
  mutate(cluster = as.factor(cluster))

# points with k-means clusters and centroids
p1 <- df_kmeans %>%
  ggplot(aes(x = x, y = y, color = cluster)) +
  geom_point(size = 2) +
```

```

geom_point(data = df_centroids, color = "black", size = 5, pch
  ↪ = 10) +
theme_bw() +
mytheme +
xlab("gene A") +
ylab("gene B") +
scale_color_brewer(palette = "Set1")

# points with pam clusters and medoids
p2 <- df_pam %>%
  ggplot(aes(x = x, y = y, color = cluster)) +
  geom_point(size = 2) +
  geom_point(data = df_medoids, color = "black", size = 5, pch =
    ↪ 21) +
  theme_bw() +
  mytheme +
  xlab("gene A") +
  ylab("gene B") +
  scale_color_brewer(palette = "Set1")

p1 + p2

```

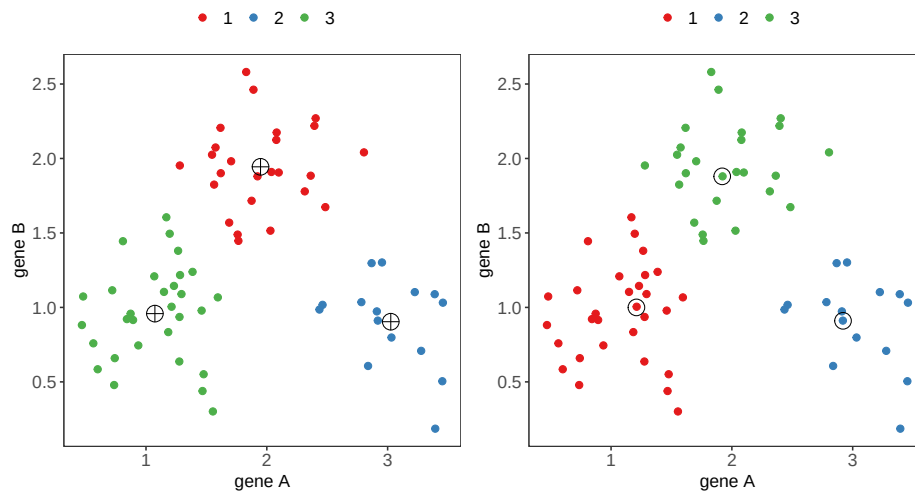


Figure 39.8: Illustration of k-means clusters with centroids marked (left) and PAM clusters with medoids marked (right) for $k = 3$.

39.6 HCL, hierarchical clustering

- Hierarchical clustering does not require the number of clusters to be specified. Instead of creating a single set of clusters it creates a hierarchy of clusterings based on pairwise dissimilarities.
- This hierarchy of clusters is often represented as a **dendrogram**.

```
set.seed(19)
df <- tibble(x = rnorm(10), y = rnorm(10))
h <- hclust(dist(df))

dend <- as.dendrogram(h) %>% set("branches_lwd", 3)
plot(dend)
```

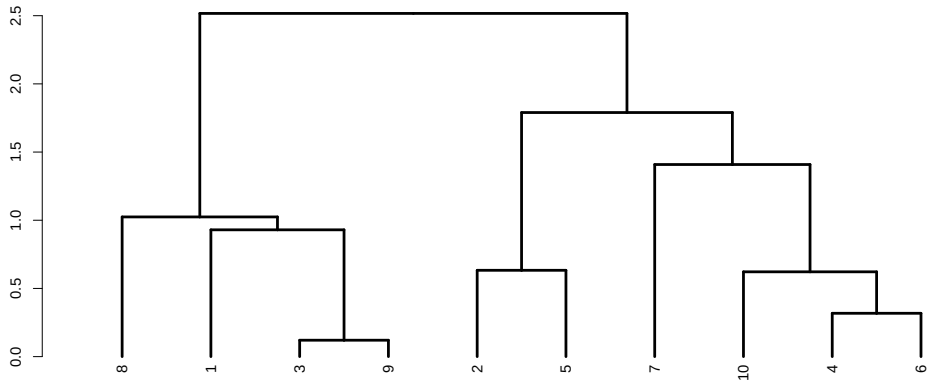


Figure 39.9: Hierarchical clustering creates a hierarchy of clustering based on pairwise dissimilarities, that are often represented as dendrograms.

- There are two strategies for hierarchical clustering *agglomerative* (bottom-up) and *divisive* (top-down).
- The agglomerative strategy starts at the bottom with all objects in separate clusters and at each level merge a pair of clusters.
- The merged pair of clusters are those with the smallest dissimilarity.

```
# illustrate HCL agglomerative

df_cl <- map_dfr(
  1:10, # step-by-step tree cuts
  \(ii) df |>
    mutate(step = ii,
           Cluster = as.character(cutree(h, 11 - ii))) |>
    group_by(Cluster) |>
    mutate(is_dup = n() > 1) |>
    ungroup()
)

ggplot(df_cl, aes(x, y, color = Cluster)) +
  geom_point(size = 2) +
  ggalt::geom_encircle(data = filter(df_cl, is_dup)) +
  facet_wrap(~step, nrow = 2) +
  guides(color = "none") +
  theme_bw() +
  mytheme +
  scale_color_brewer(palette = "Set1")
```

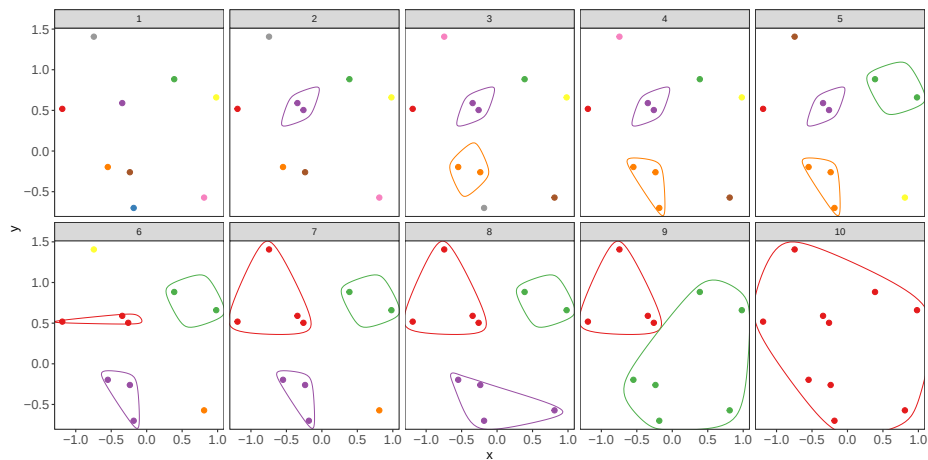



Figure 39.10: Illustration of hierarchical clustering agglomerative (bottom-up) approach.

- The divisive strategy starts at the top with all objects in a single cluster and at each level one cluster is split into two.
- The split is chosen to produce the two groups with the largest possible dissimilarity.

```
# illustrate HCL divisive
d <- cluster::diana(dist(df))
df_cl <- list()
df_cl0 <- pre <- mutate(df, step = 10, Cluster = -c(1:10), is_dup
  ↪ = FALSE)

for(ii in 1:9) {
  i_merge <- d$merge[ii, ] # output from divisive h-clustering
  cl <- pre$Cluster # copy previous
  cl[cl %in% i_merge] <- ii

  nxt <- mutate(df, step = 10 - ii, Cluster = cl)
  df_cl <- c(df_cl, list(pre <- nxt))
}

df_cl <- c(list(df_cl0), df_cl) |>
  map_dfr(
    ~.x |>
      mutate(Cluster = as.integer(factor(Cluster))) |> # drop
        ↪ disappeared cluster number
      # encircle
      group_by(Cluster) |>
      mutate(is_dup = n() > 1) |>
      ungroup()
  ) |>
  mutate(Cluster = as.character(Cluster))
```

```
ggplot(df_cl, aes(x, y, color = Cluster)) +
  geom_point(size = 2) +
  ggalt::geom_encircle(data = filter(df_cl, is_dup)) +
  facet_wrap(~step, nrow = 2) +
  guides(color = "none") +
  theme_bw() +
  mytheme +
  scale_color_brewer(palette = "Set1")
```

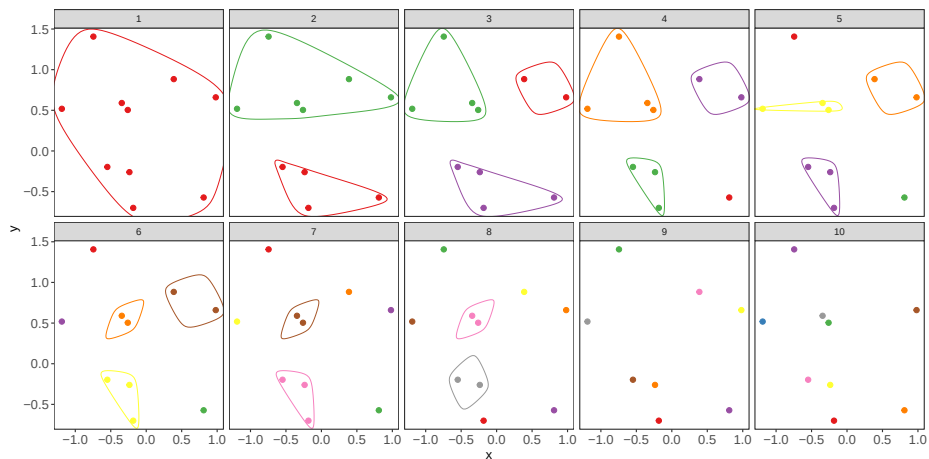


Figure 39.11: Illustration of hierarchical clustering divisive (top-down) approach.

- With n objects to cluster both strategies will produce a dendrogram representing the $n - 1$ levels in the hierarchy.
- Each level represent a specific clustering of the objects into disjoint clusters.
- The heights of the branches in the dendrogram are proportional to the dissimilarity of the merged/split clusters.
- The divisive approach is used less often due to the computational complexity. Here, we will focus more on the commonly used agglomerative approach.

39.6.1 Linkage & Agglomerative clustering

- Agglomerative clustering starts with all objects in separate clusters and at each level merge the pair of clusters with the smallest dissimilarity. The pairwise dissimilarities between objects can be computed according to the distance measures discussed above.
- We need one more ingredient for hierarchical clustering, that is a method for computing dissimilarity between clusters, known as **linkage method**. There are several linkage methods, as illustrated below.

```
include_graphics("images/linkage.png")
```

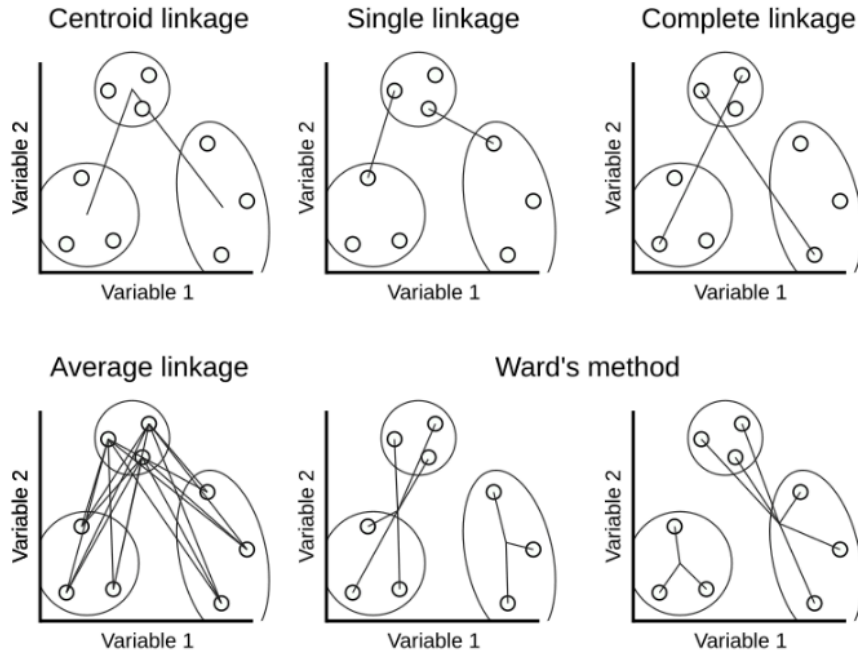


Figure 39.12: Illustration of linkage methods to calculate dissimilarities between clusters.

More formally, the dissimilarity between two clusters A and B with objects $a_1, \dots, a_{n_A}$ and $b_1, \dots, b_{n_B}$ are defined as follows.

Single linkage

- Single linkage takes as a cluster dissimilarity the distance between the two closest objects in the two clusters.

$$d_{sl}(A, B) = \min_{i,j} d(a_i, b_j)$$

Complete linkage

- Complete linkage takes as a cluster dissimilarity the distance between the two objects furthest apart in the two clusters.

$$d_{cl}(A, B) = \max_{i,j} d(a_i, b_j)$$

Average linkage

- Average linkage takes as a cluster dissimilarity the average distance between the objects in the two clusters.

$$d_{al}(A, B) = \frac{1}{n_A n_B} \sum_i \sum_j d(a_i, b_j)$$

Ward's linkage

- Ward's linkage method minimize the within variance, by merging clusters with the minimum increase in within sum of squares.

$$d_{wl}(A, B) = \sum_{i=1}^{n_A} (a_i - m_{A \cup B})^2 + \sum_{i=1}^{n_B} (b_i - m_{A \cup B})^2 - \sum_{i=1}^{n_A} (a_i - m_A)^2 - \sum_{i=1}^{n_B} (b_i - m_B)^2$$

where $m_A, m_B, m_{A \cup B}$ are the center of the clusters A, B and $A \cup B$, respectively.

- *Note* that Ward's linkage method should not be combined with any dissimilarity matrix as it is based on the squared Euclidean distance. In the R function `hclust` either the Euclidean or squared Euclidean distance can be used in combination with the linkage `method='ward.D2'` or `method='ward.D'`, respectively.

```
set.seed(19)
df <- tibble(x = rnorm(10), y = rnorm(10)) # simulate data
d <- dist(df) # calculate distance

h_single <- hclust(d, method = "single")
h_average <- hclust(d, method = "average")
h_complete <- hclust(d, method = "complete")
h_ward <- hclust(d, method = "ward.D2")
## Alternatively
## h_ward <- hclust(d^2, method = "ward.D")

dend_single <- as.dendrogram(h_single) %>% set("branches_lwd", 3)
dend_average <- as.dendrogram(h_average) %>% set("branches_lwd",
  ↪ 3)
dend_complete <- as.dendrogram(h_complete) %>%
  ↪ set("branches_lwd", 3)
dend_ward <- as.dendrogram(h_ward) %>% set("branches_lwd", 3)

par(mfrow=c(2,2))
plot(dend_single, main = "single")
plot(dend_average, main = "average")
plot(dend_complete, main = "complete")
plot(dend_ward, main = "Ward's")
```

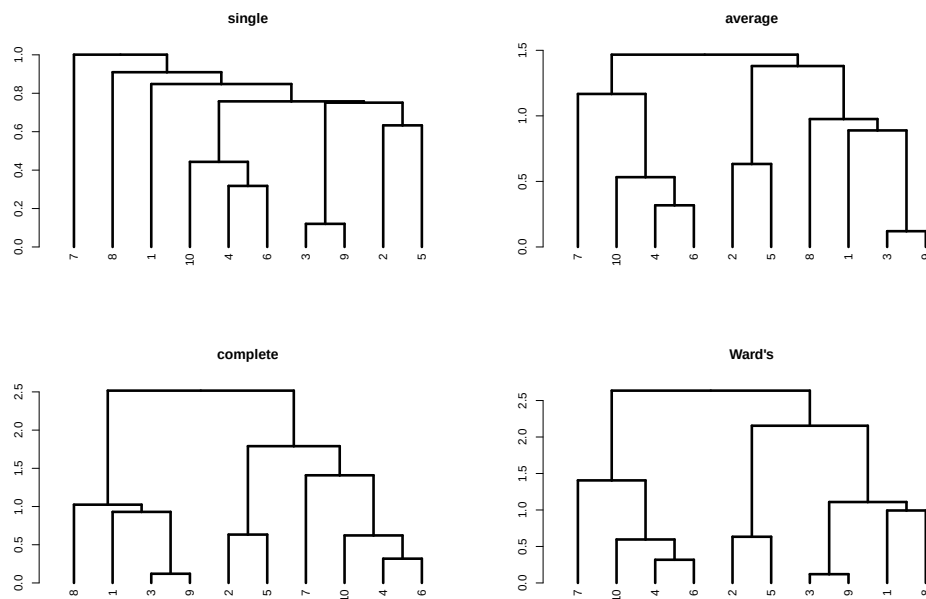


Figure 39.13: HCL agglomerative clustering with different linkage methods used.

```
# compare dendrograms side-by-side and highlight differences
dend_diff(dend_single, dend_complete)
```

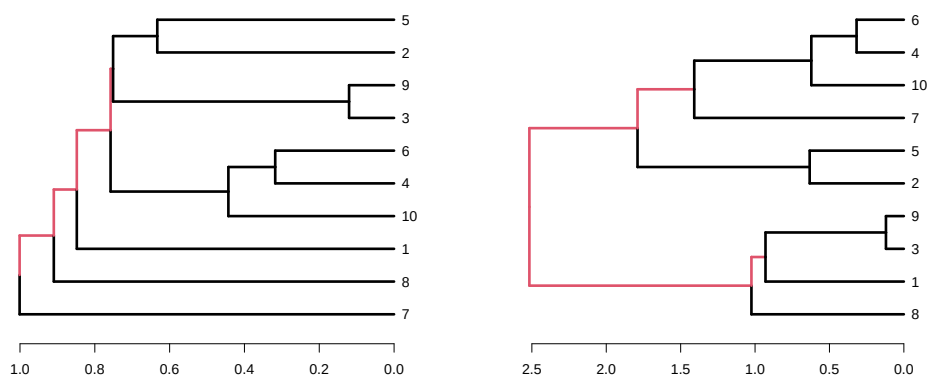


Figure 39.14: Side-by-side comparison of dendrograms with single (left) and complete linkage (right).

```
# compare dendrograms side-by-side and highlight differences
dend_diff(dend_average, dend_ward)
```

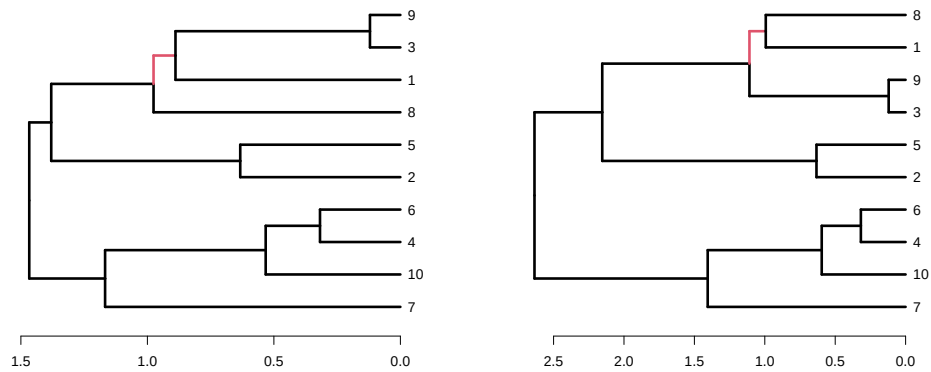


Figure 39.15: Side-by-side comparison of dendrograms with average (left) and Ward's linkage (right).

39.6.2 Getting clusters

To find concrete clusters based on HCL we can cut the tree either using height or using number of clusters.

```
par(mfrow = c(1,2))

dend_complete %>% set("labels_cex", 2) %>% set("labels_col",
  ↪ value = c(3,4), k=2) %>%
  plot(main = "Two clusters")
abline(h = 2, lty = 2)

dend_complete %>% set("labels_cex", 2) %>% set("labels_col",
  ↪ value = c(3,4,5), k=3) %>%
  plot(main = "Three clusters")
abline(h = 1.6, lty = 2)
```

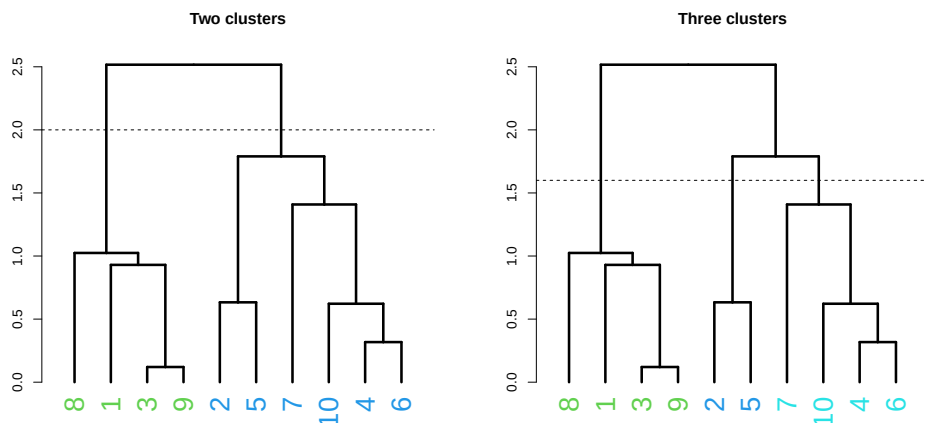
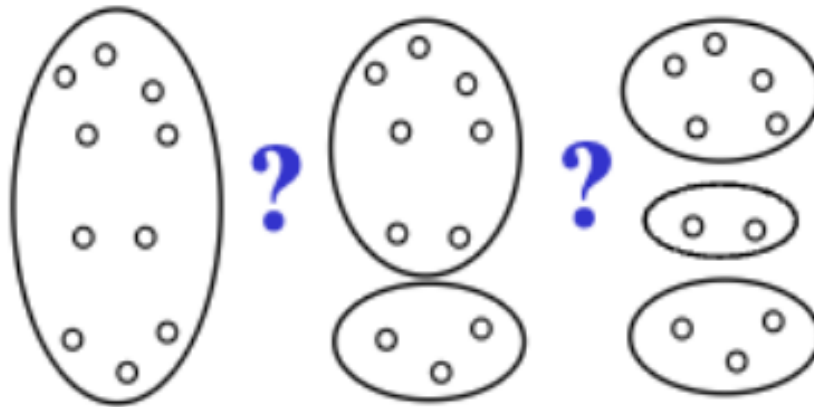


Figure 39.16: Example of cutting dendrogram, complete linkage, using height into two and three clusters.

39.7 How many clusters?



- As we have seen above clustering is subjective, and there is more than one way to think about groupings of objects. In addition, there are many clustering algorithms to choose from, each with its own set of parameters and assumptions. We need some ways to be able to say something about how good our clustering is, or how many clusters we have.
- We will focus on **internal validation** that evaluates clustering quality and estimates cluster count without external information. In **external validation** one compares clustering outcomes to known results, measuring label match to select the best algorithm for a dataset.
- Internal validation methods evaluate clustering often by measuring the **compactness** and **separation** of the clusters - objects within a cluster exhibit maximal similarity (compactness), while those in separate clusters maintain a high degree of distinction (separation).
- There are many ways to measure this, and some common methods include elbow method based on the total sum of squares WSS , Silhouette method or Gap statistic.
- `NbClust` package in R implements 30 indexes for determining the optimal number of clusters in a data set to offer the best clustering scheme.

39.7.1 Elbow method

- The Elbow method is based on the total within sum of squares, WSS , that the k-means algorithm tries to minimize.
- By running the algorithm with several different values of k , e.g. from 1 to 10, we can plot WSS as a function of k .
- The inflection (bend, elbow) on the curve indicates an optimal number of clusters.

```
# simulate data
set.seed(190)
```

```
df <- tibble(class = rep(c("cl1", "cl2", "cl3"), c(30, 15, 25)))
  <- |>
  mutate(
    x = rnorm(n(), c(cl1 = 1, cl2 = 3, cl3 = 2)[class], sd =
      <- .35),
    y = rnorm(n(), c(cl1 = 1, cl2 = 1, cl3 = 2)[class], sd =
      <- .35),
  )

factoextra::fviz_nbclust(df[, c("x", "y")], FUNcluster = kmeans,
  <- method="wss")
```

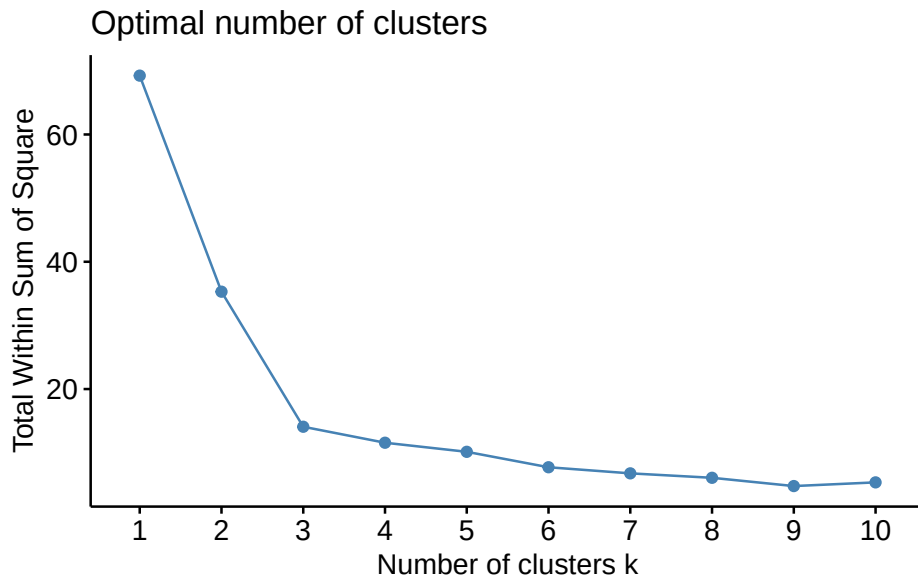
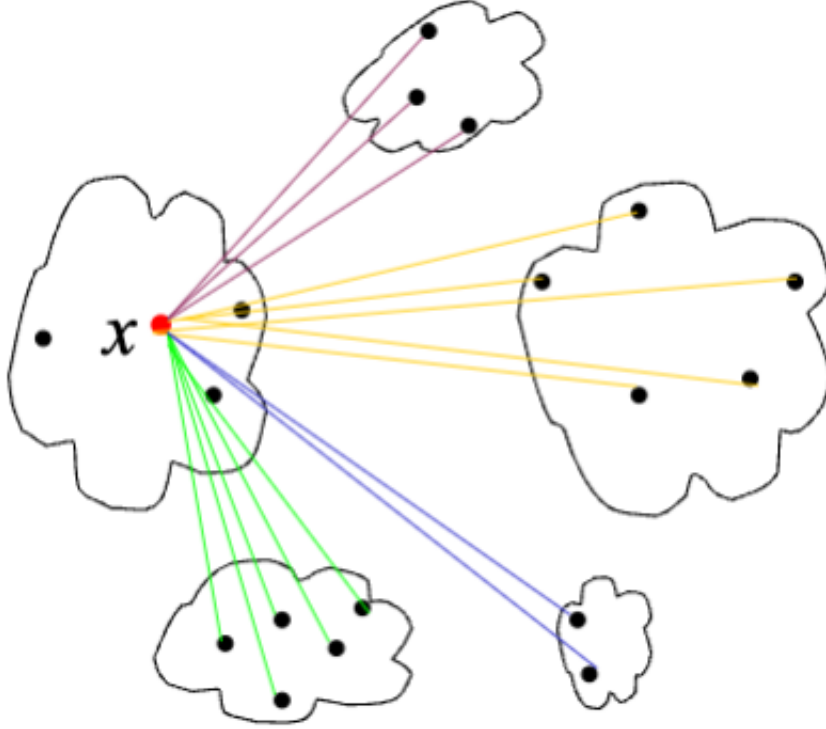


Figure 39.17: WSS as a function of the number of clusters. The inflection for $k = 3$ indicates that this is the best number of clusters for the data set.

39.7.2 Silhouette method



- The silhouette value for a single object i is a value between -1 and 1 that measures how similar the object is to other objects in the same cluster as compared to how similar it is to objects in other clusters.
- The average silhouette over all objects is a measure of how good the clustering is, the higher the value the better is the clustering.
- The silhouette value, $s(i)$, for an object i in cluster C_a is calculated as follows:

1. Average distance between i and all other objects in the same cluster C_a

$$a(i) = \frac{1}{|C_a| - 1} \sum_{j \neq i, j \in C_a} d(i, j)$$

2. Average distance between i and all objects in another cluster C_b , $d(i, C_b)$ and define the minimum;

$$b(i) = \min_{b \neq a} d(i, C_b)$$

3. The Silhouette index is defined as;

$$s(i) = \frac{b(i) - a(i)}{\max(a(i), b(i))}$$

- A silhouette value close to 1 means that the object is very well clustered, a value close to 0 means that the object lies between two clusters. A negative silhouette value means that the object is incorrectly clustered.
- The **average silhouette over all objects** is a measure of how good the clustering is, the higher the value the better is the clustering.

```
factoextra::fviz_nbclust(df[, c("x", "y")], FUNcluster = kmeans,
  ↪ method="silhouette")
```

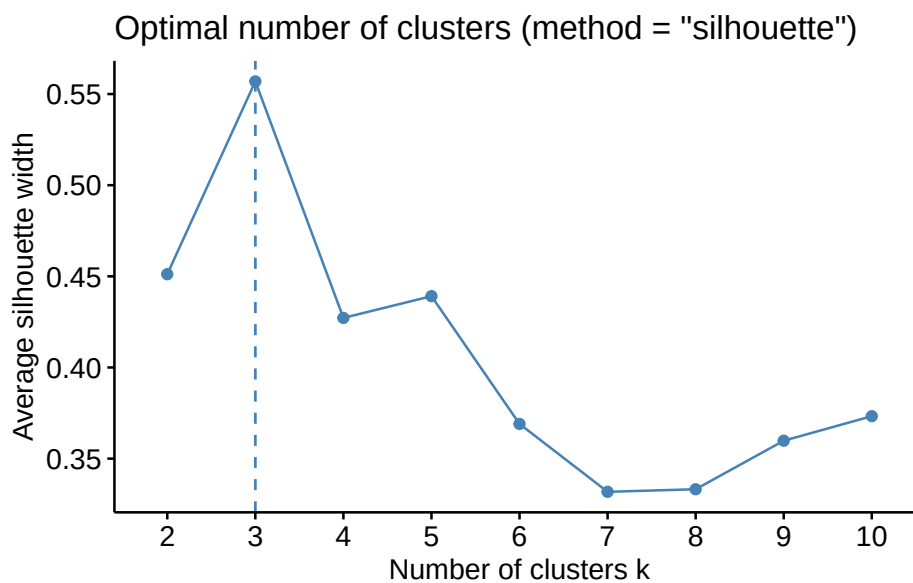


Figure 39.18: Average silhouette width as a function of the number of clusters. The maximum value for $k = 3$ indicates that this is the best number of clusters for the data set.

39.7.3 Gap

- The gap statistic compares the total within intra-cluster variation for different values of k with their expected values under null reference distribution of the data. The estimate of the optimal clusters will be value that maximize the gap statistic (i.e., that yields the largest gap statistic). This means that the clustering structure is far away from the random uniform distribution of points.

```
factoextra::fviz_nbclust(df[, c("x", "y")], FUNcluster = kmeans,
  ↪ method="gap_stat")
```

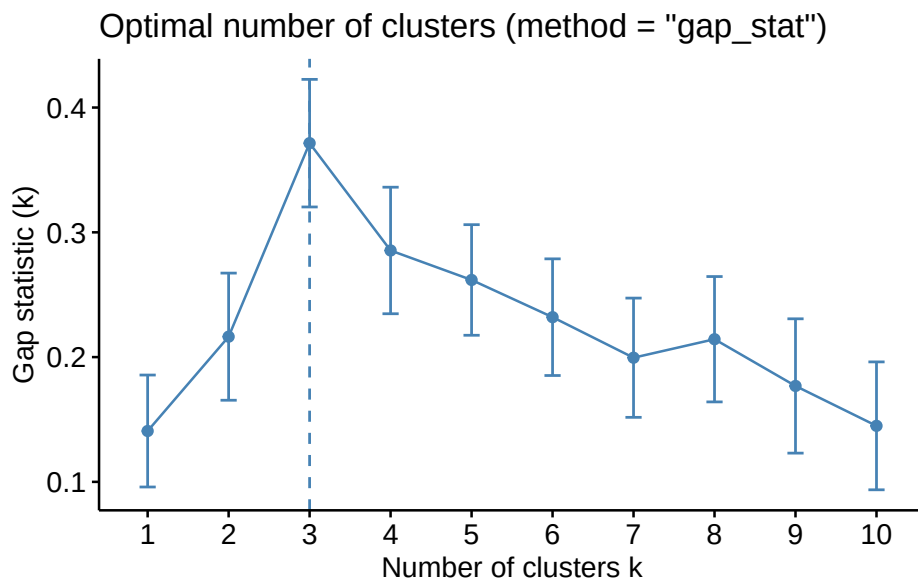


Figure 39.19: Gap statistics as a function of the number of clusters. The maximum value for $k = 3$ indicates that this is the best number of clusters for the data set.

39.7.4 pvclust

- Pvcust R package to assess the uncertainty in hierarchical cluster analysis (Suzuki and Shimodaira 2006)
- Pvcust calculates probability values (p-values) for each cluster using bootstrap resampling techniques.
- Two types of p-values are available: approximately unbiased (AU) p-value and bootstrap probability (BP) value. Multiscale bootstrap resampling is used for the calculation of AU p-value, which has superiority in bias over BP value calculated by the ordinary bootstrap resampling.

```
include_graphics("images/pvclust.png")
```

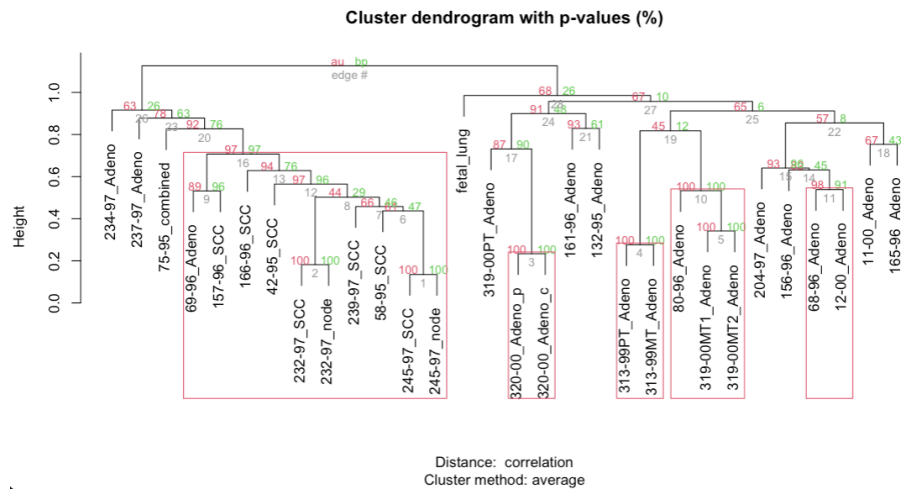


Figure 39.20: Bootstrapping for hierarchical clustering with pvclust. Clusters with AU > 95% indicated by the rectangles are considered to be strongly supported by data.

39.8 Two-way clustering

- When dealing with omics data we often perform two-way clustering, where there is simultaneous clustering, typically hierarchical, of the rows (e.g. gene expression) and columns (e.g. samples).
- This is often a default option when showing data on the heatmaps.
- We suggest checking out ComplexHeatmap package that provides rich functionalities for customizing, including splitting columns and rows by clustering solutions

```
load("data/tissuesGeneExpression.rda")

set.seed(123)
n <- 80
idx.keep <- sample(1:ncol(e), n, replace = FALSE)

data_genes <- e[, idx.keep]
data_samples <- tab[idx.keep, ]

# calculate variance
genes_var <- apply(data_genes, 1, var)
o <- order(genes_var, decreasing = T)

# select top genes with highest variance
n_ft <- 40
x <- data_genes[o[1:n_ft], ]

# define function to calculate z scores
```

```

func_cal_z_score <- function(x){
  (x - mean(x)) / sd(x)
}

data_zscore <- t(apply(x, 1, func_cal_z_score))
data_heatmap <- data_zscore
cols_wong <- c("#0072b2", "#e69f00")

# set color scale
col_fun = colorRamp2(c(-1, 0, 2), c(cols_wong[1], "white",
  ↪ cols_wong[2]))

# annotate columns
# annotations: columns
ha_col <- HeatmapAnnotation(tissue = data_samples$Tissue,
  col = list(tissue = c("kidney" =
    ↪ "darkgreen",
    "hippocampus" =
    ↪ "red",
    "colon" = "blue",
    "cerebellum" =
    ↪ "orange",
    "liver" =
    ↪ "yellow",
    "placenta" =
    ↪ "pink",
    "endometrium" =
    ↪ "black")))

Heatmap(data_heatmap,
  col = col_fun,
  top_annotation = ha_col,
  name = "h1",
  show_row_names = FALSE,
  show_column_names = FALSE,
  show_row_dend = TRUE,
  show_column_dend = TRUE,
  column_dend_height = unit(3, "cm"),
  row_dend_width = unit(3, "cm"))

```

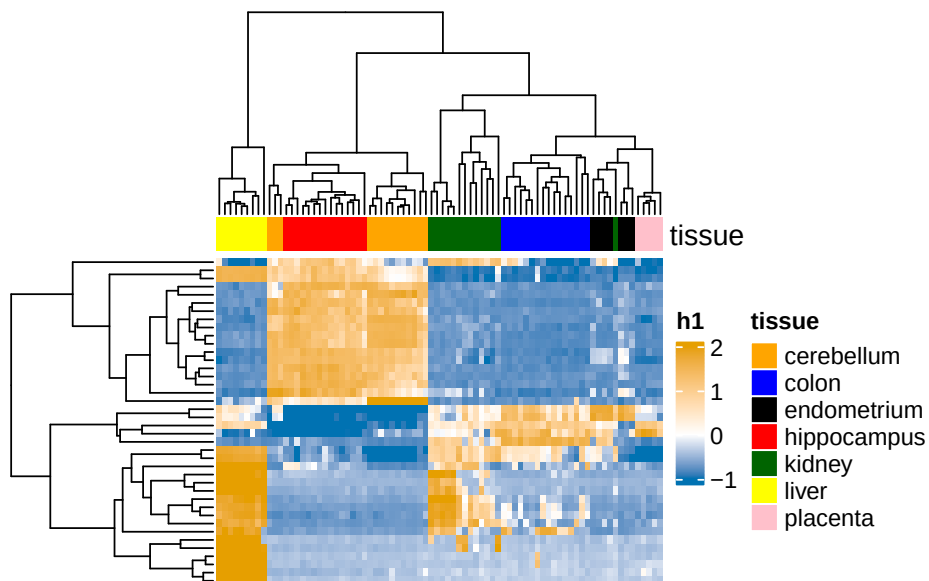


Figure 39.21: Example of a heatmap based on gene expression data collected for seven tissues. Two-way clustering of both genes and samples shows a good separation between the tissues.

39.9 Additional comments & resources

- In addition to partitioning and hierarchical clustering methods there is a wide range of other algorithms. Common ones include model-, density, and grid-based methods. Some of them are explained in this presentation. Here, you can also find more details about **external validation** of cluster analysis.
- Although historically it was not recommended to apply cluster analysis to **mixed data types** (e.g. both binary and continuous), some distance metrics have been developed, e.g. Gower's general coefficient to calculate similarity between different data types. Lately, the principles of decision trees have been also used for computing yet another metric for mixed data types, **unsupervised extra trees dissimilarity, UET**. For an interesting study comparing clustering methods for heterogeneous data, where more details about Gower's and UET can be found, see Preud'homme et al. (2021).
- For a more applied reading in health research, reviewing distance metrics and clustering algorithms see Gao et al. (2023).
- In scRNA-seq analysis it is common to use clustering based on the graph data. In particular, more details about Leiden algorithms can be found here or in the (Traag, Waltman, and Eck 2019).

References

Exercises

```
library(tidyverse)
library(dendextend)
library(patchwork)
library(knitr)
library(kableExtra)
library(cluster)
library(RColorBrewer)
library(ggalt)
library(factoextra)
library(NbClust)
library(pvclust)
library(ComplexHeatmap)
library(circlize)
```

You have found some older gene expression data, based on the microarray technology. They contains measurements for 22215 genes for 189 samples, across 7 tissue (kidney, hippocampus, cerebellum, colon, liver, endometrium and placenta).

Data can be loaded and preview:

```
load("data/tissuesGeneExpression.rda")
```

```
# expression data
print(dim(e))
```

```
[1] 22215 189
```

```
print(head(e[1:4, 1:5]))
```

	GSM11805.CEL.gz	GSM11814.CEL.gz	GSM11823.CEL.gz	GSM11830.CEL.gz
1007_s_at	10.191267	10.509167	10.272027	10.252952
1053_at	6.040463	6.696075	6.144663	6.575153
117_at	7.447409	7.775354	7.696235	8.478135
121_at	12.025042	12.007817	11.633279	11.075286

	GSM12067.CEL.gz
1007_s_at	10.157605
1053_at	6.606701
117_at	8.116336

```
121_at          10.832528
```

```
# corresponding sample information
str(tab)
```

```
'data.frame':  189 obs. of  6 variables:
 $ filename      : chr  "GSM11805.CEL.gz" "GSM11814.CEL.gz" "GSM11823.CEL.gz" "GSM11830.CEL.gz" ...
 $ DB_ID         : chr  "GSM11805" "GSM11814" "GSM11823" "GSM11830" ...
 $ ExperimentID  : chr  "GSE781" "GSE781" "GSE781" "GSE781" ...
 $ Tissue        : chr  "kidney" "kidney" "kidney" "kidney" ...
 $ SubType       : chr  "normal" "cancer" "normal" "cancer" ...
 $ ClinicalGroup: chr  NA NA NA NA ...
```

Exercise 39.1 (Partition methods). Use first two genes only and run k-means clustering.

1. Find optimal number of k using Silhouette method.
2. Plot samples using the first two genes as x and y coordinates and visualize your cluster results on one a scatter plot.
3. Use first 1000 genes and the same value of k . Is this clustering solution better now? How can you tell?

Exercise 39.2 (HCL). Select samples corresponding to two tissues of your choice. Run HCL and compare dendrograms:

1. with complete and ward linkage, distance measure Euclidean
2. with complete and ward linkage, distance measure Canberra

Exercise 39.3 (Pvclust). Try running pvclust on the samples you've chosen above. Which clusters are supported by bootstrapping?

Exercise 39.4 (Heatmap). Select top 100 genes based on variance (with highest variance). Make a heatmap using ComplexHeatmap package. Group columns (samples) by tissue and split rows (genes) using k-means ($k = 7$).

1. Do you see any interesting patterns?
2. How would you go about extracting genes belonging to a specific cluster, if you were interested in running functional annotations on those?

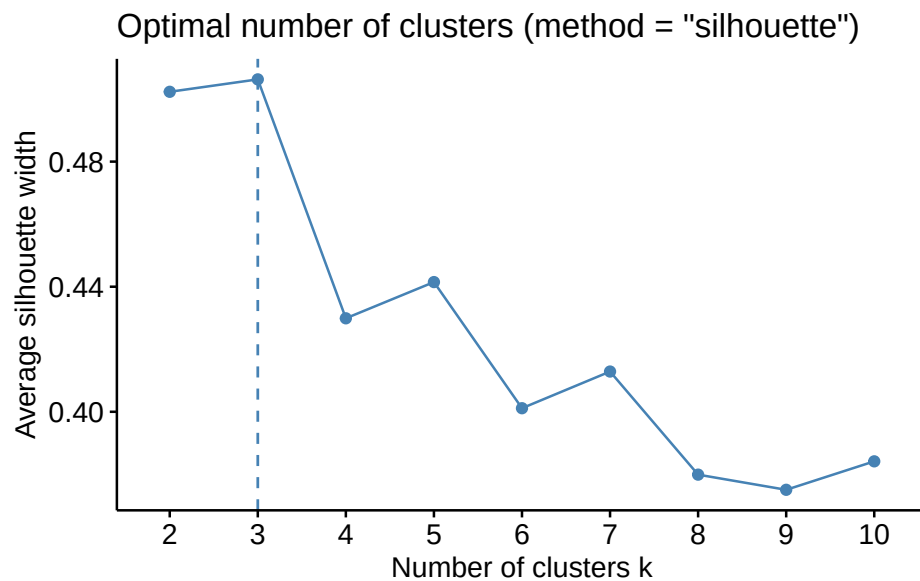
Answers to exercises

Solution. Exercise 39.1

```
# load data
load("data/tissuesGeneExpression.rda")

# subset gene expression data, two first genes
x2 <- t(e[1:2, ])

# find optimal number of cluster using WSS
factoextra::fviz_nbclust(x2, FUNcluster = kmeans, method="sil")
```

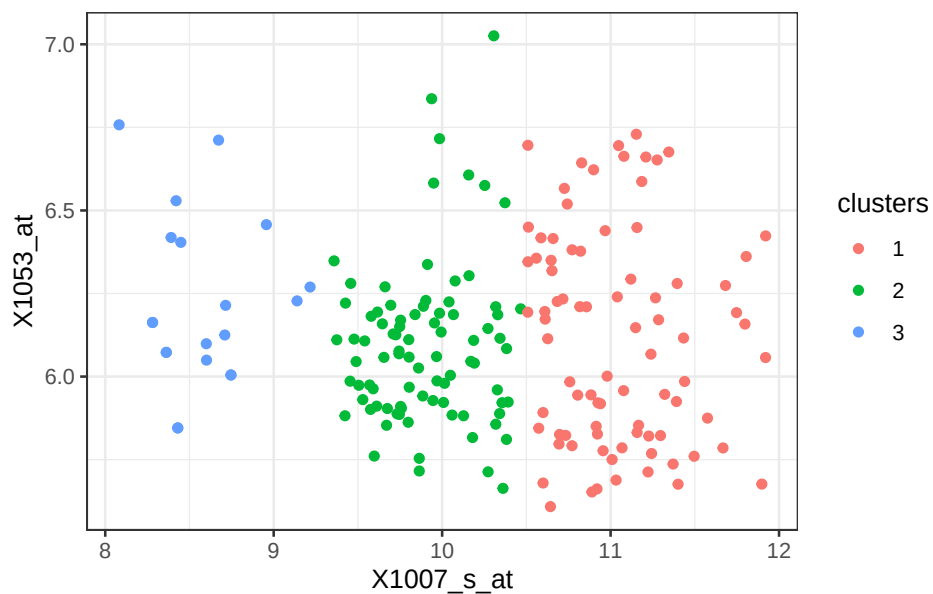


```
# decide on the best cluster
k_best <- 3

# run k-means with best k
res_kmeans2 <- kmeans(x2, centers = k_best)

# prepare data for plotting, add cluster solutions
df <- data.frame(x2, clusters = res_kmeans2$cluster) %>%
  mutate(clusters = as.factor(clusters))

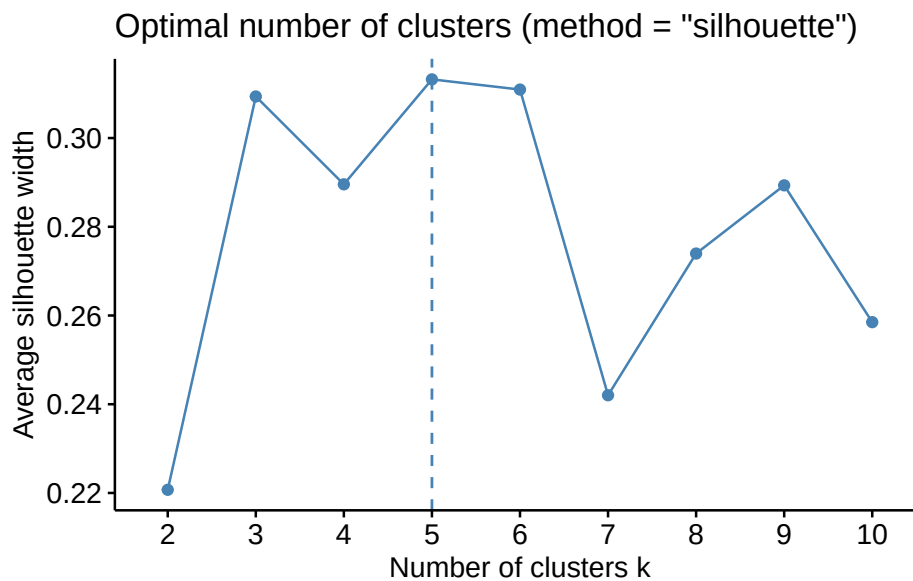
# plot data
df %>%
  ggplot(aes(x = X1007_s_at, y = X1053_at, color = clusters)) +
  geom_point() +
  theme_bw()
```



```
# run for 1000 genes
n <- nrow(e)
n <- 1000
x_n <- t(e[1:n, ])

# run k-means with best k
res_kmeans_n <- kmeans(x_n, centers = k_best)

# To compare clustering results, we can compare average
  ↳ Silhouette for k = 3.
# one way to do so is to plot average Silhouette as a function of
  ↳ different k again
factoextra::fviz_nbclust(x_n, FUNcluster = kmeans, method="sil")
```



```
# from the plot we can see now that the best value for k is 7
# Looking at the average Silhouette value at k = 3, it is just
  ↳ above 0.3 when using 1000 genes
# whereas it was above 0.5 when using 2 genes.
# This means that choosing k = 3 for 1000 genes would results in
  ↳ worse clustering solution than when choosing k = 3 for 2
  ↳ genes.
# We can also see that the best number of clusters for 1000 genes
  ↳ is now 7.
# Since we have some prior knowledge here, i.e. we know that the
  ↳ samples originate from 7 tissues and gene expression has been
  ↳ shown tissue specific, most likely using only two first genes
  ↳ is not enough to cluster samples into relevant groups.
```

Solution. Exercise 39.2

```
# see how many samples of each tissue we have
print(summary(as.factor(tab$Tissue)))
```

```
cerebellum      colon endometrium hippocampus      kidney      liver
           38           34           15           31           39           26
placenta
           6
```

```
# let's pick the fours tissues with smallest amount of samples
# so the dendrograms are smaller and easier to read
```

```
data_samples <- tab %>%
  as_tibble() %>%
  filter(Tissue %in% c("placenta", "endometrium"))
```

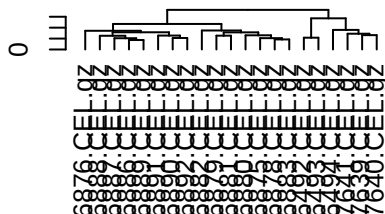
```
# subset gene expression data
```

```
x <- t(e[, data_samples$filename])

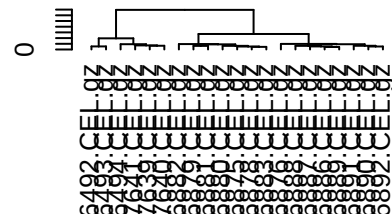
# make dendrograms
dend_euclidean_complete <- as.dendrogram(hclust(dist(x, method =
  ↪ "euclidean"), method = "complete"))
dend_euclidean_ward <- as.dendrogram(hclust(dist(x, method =
  ↪ "euclidean"), method = "ward.D"))
dend_canberra_complete <- as.dendrogram(hclust(dist(x, method =
  ↪ "canberra"), method = "complete"))
dend_canberra_ward <- as.dendrogram(hclust(dist(x, method =
  ↪ "canberra"), method = "ward.D"))

# plot all dendrograms
par(mfrow=c(2,2))
plot(dend_euclidean_complete, main = "euclidean, complete")
plot(dend_euclidean_ward, main = "euclidean, Ward's")
plot(dend_canberra_complete, main = "canberra, complete")
plot(dend_canberra_ward, main = "canberra, Ward's")
```

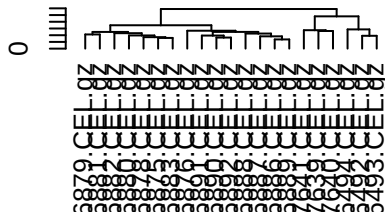
euclidean, complete



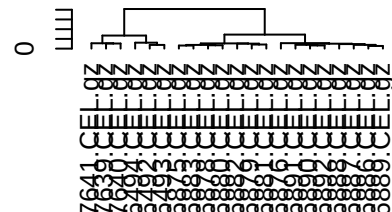
euclidean, Ward's



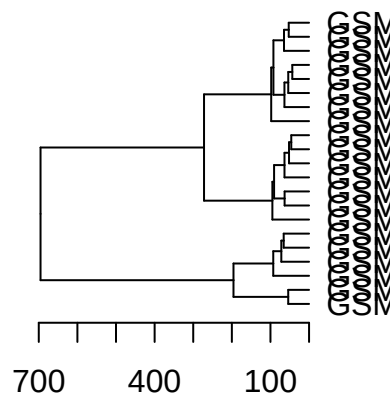
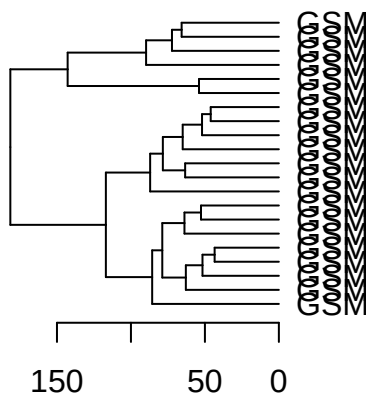
canberra, complete



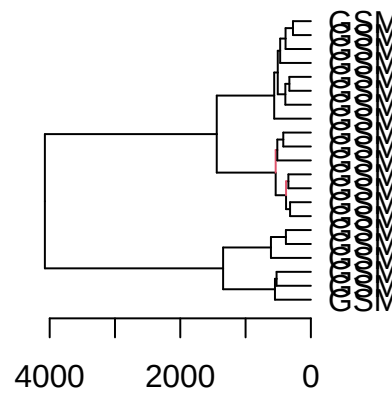
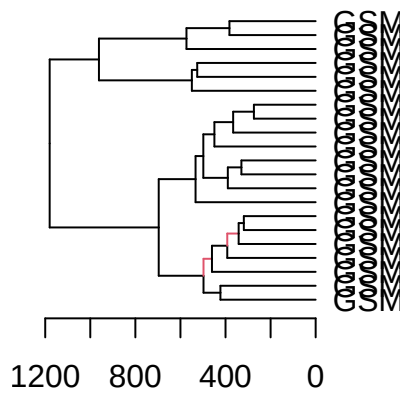
canberra, Ward's



```
# compare 2 side-by-side
dend_diff(dend_euclidean_complete, dend_euclidean_ward)
```



```
dend_diff(dend_canberra_complete, dend_canberra_ward)
```



Part VIII

From Linear Models to Machine Learning

Chapter 40

From inference to prediction

So far, we have mainly used statistical models for **inference**: to learn about relationships in the data and to quantify the effects of explanatory variables on an outcome. For example, in a linear regression we may ask whether an outcome is associated with age, treatment or another variable, and estimate the size and uncertainty of these associations.

Another goal is **prediction**. Here, we use observed data to build a model that can predict the outcome for **new observations**.

- If the outcome is continuous, such as age or gene expression, we have a **regression task**.
- If the outcome consists of categories, such as disease versus healthy, we have a **classification task**.

For example, we can train a model using biopsy images labelled as benign or malignant. Given an image from a new biopsy, the trained model can then predict whether it is benign or malignant (classification task).

Or we can train a model. For example, we can start with DNA methylation measurements at thousands of CpG sites from individuals of known chronological age. During model building, we may select a subset of CpG sites that are useful for prediction. Given methylation measurements from a new individual, the trained model can then predict their age (regression task).

When prediction is our goal, an important question is therefore not only how well the model describes the data used to fit it, but **how well it performs on new, unseen data**.

40.1 What is supervised learning?

In **supervised learning**, we use observations for which the outcome is already known to **train** a model. The measured variables are used as predictors, while

the known outcome provides the **label** or target that we want the model to learn to predict.

This is in contrast to **unsupervised learning**, such as clustering or Principal Component Analysis (PCA), where there is no predefined outcome. Instead, the aim is to identify patterns or structure in the data, for example groups of samples with similar molecular profiles.

```
library(knitr)

include_graphics("images/supervised.png")
```

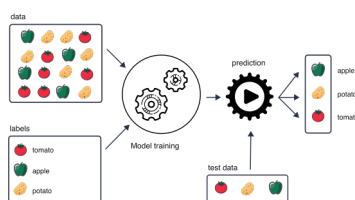


Figure 40.1: Illustration of supervised learning, where labelled data are used to train a model that can make predictions for new observations.

Training a model means using the observed data to estimate the model parameters so that the predictors are linked to the outcome. Depending on the type of outcome, this may be a **regression** or a **classification** task.

Once trained, the model can be used to make predictions for new observations.

40.2 Data splitting

A model can often be made to fit the observed data very well. However, good performance on the data used to train the model does not necessarily mean that it will perform equally well on new data.

If a model adapts too closely to the particular patterns, noise or peculiarities of the training data, it may fail to generalize to new observations. This is known as **overfitting**.

To obtain a more realistic estimate of predictive performance, we therefore separate the data used to train the model from the data used to evaluate it. This is the basic idea behind data splitting.

train, validation & test sets

- Common split strategies include 50%/25%/25% and 33%/33%/33% splits for training/validation/test respectively
- **Training data:** this is data used to fit (train) the classification or regression model, i.e. derive the classification rule
- **Validation data:** this is data used to select which parameters or types of model perform best, i.e. to validate the performance of model parameters

- **Test data:** this data is used to give an estimate of future prediction performance for the model and parameters chosen

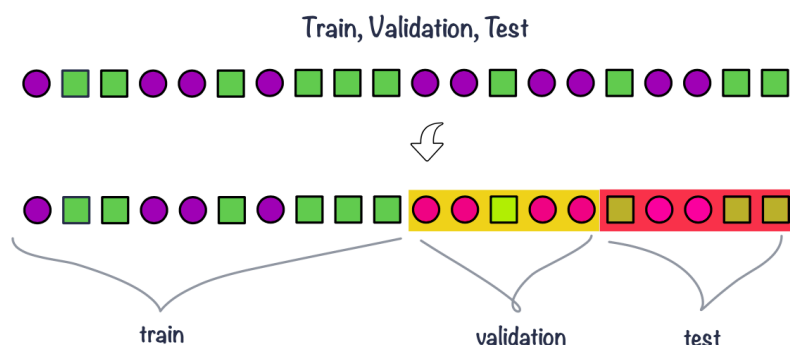


Figure 40.2: Example of splitting data into train (50%), validation (25%) and test (25%) set

cross validation

- It can happen that despite random splitting in train/validation/test dataset one of the subsets does not represent data. e.g. gets all the difficult observation to classify.
- Or that we do not have enough data in each subset after performing the split.
- In **k-fold cross-validation** we split data into k roughly equal-sized parts.
- We start by setting the validation data to be the first set of data and the training data to be all other sets.
- We estimate the validation error rate / correct classification rate for the split.
- We then repeat the process $k - 1$ times, each time with a different part of the data set to be the validation data and the remainder being the training data.
- We finish with k different error or correct classification rates.
- In this way, every data point has its class membership predicted once.
- The final reported error rate is usually the average of k error rates.

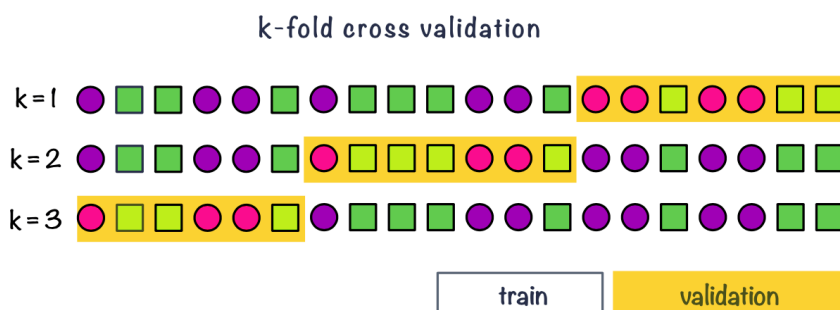
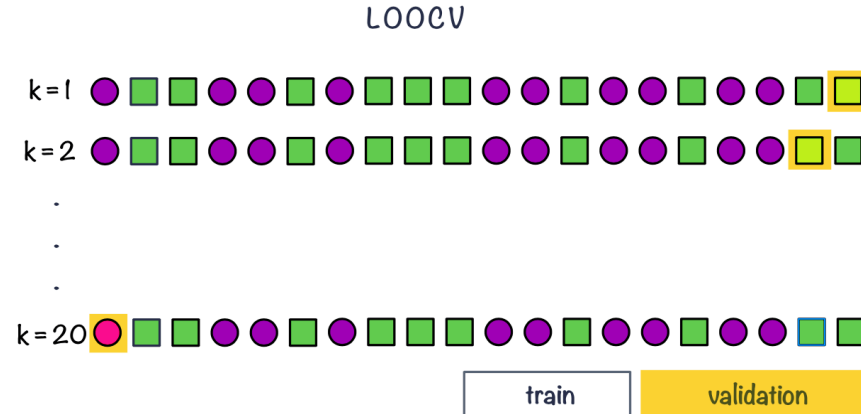


Figure 40.3: Example of k-fold cross validation split ($k = 3$)

Leave-one-out cross-validation

- Leave-one-out cross-validation is a special case of cross-validation where the number of folds equals the number of instances in the data set.

Figure 40.4: Example of LOOCV, leave-one-out cross validation



Nested cross-validation

Nested cross-validation uses two cross-validation loops:

- The **outer cross-validation** is used to estimate how well the final modelling procedure performs on new data.
- The **inner cross-validation** is used to select or tune the model.

For example, in **5-fold nested cross-validation**:

1. Split the data into 5 **outer folds**.
2. Keep one outer fold aside as the **test set**.
3. Use the remaining data as the outer **training set**.
4. Within this training set, perform another k -fold cross-validation to select the best model or hyperparameters.
5. Fit the selected model using all of the outer training data.
6. Evaluate its performance on the outer test set.
7. Repeat until every outer fold has been used once as the test set.

The final estimate of predictive performance is obtained by summarizing performance across the **outer test folds**.

40.3 Evaluating regression

For regression tasks, the outcome is numeric and the prediction is a number:

$$\hat{y}_i$$

For each observation, we can compare the observed value y_i with the predicted value $\hat{y}_i$. The difference between them is the prediction error:

$$e_i = y_i - \hat{y}_i$$

When we evaluate regression models, it is useful to distinguish between two related but different goals:

1. **Assessing model fit:** how well the model describes the data used to fit it.
2. **Assessing predictive performance:** how well the model predicts new, unseen observations.

In classical regression modelling, we often assess model fit using quantities such as R^2 , adjusted R^2 , AIC or BIC. In prediction-focused modelling, however, we usually want to estimate the prediction error on data that were **not used to train the model**. This is why regression performance is often evaluated using a validation set, a test set, or cross-validation.

Model fit

Measures of model fit summarize how well the model describes the observed data.

A familiar example is R^2 , which measures the proportion of variation in the outcome that is explained by the model:

$$R^2 = 1 - \frac{RSS}{TSS}$$

where

$$RSS = \sum_{i=1}^n (y_i - \hat{y}_i)^2$$

and

$$TSS = \sum_{i=1}^n (y_i - \bar{y})^2$$

Adjusted R^2 adds a penalty for the number of predictors in the model:

$$R_{adj}^2 = 1 - (1 - R^2) \frac{n-1}{n-p-1}$$

where n is the number of observations and p is the number of predictors.

This is useful because adding more predictors will usually increase R^2 , even if the additional predictors are not very useful. Adjusted R^2 therefore tries to balance goodness of fit against model complexity.

Other model comparison criteria include the **Akaike Information Criterion (AIC)** and the **Bayesian Information Criterion (BIC)**. For linear regression models with normally distributed errors, these can be written, up to constants, in terms of the residual sum of squares:

$$AIC = n \log(RSS/n) + 2k$$

$$BIC = n \log(RSS/n) + k \log(n)$$

where k is the number of fitted parameters in the model.

Both AIC and BIC reward better fit but penalize model complexity. A lower AIC or BIC indicates a better balance between fit and complexity. BIC usually gives a stronger penalty for complexity than AIC, especially when the sample size is large.

However, these are still measures of model fit or model comparison. They do not directly tell us how well the model will predict future observations.

Prediction error

When the goal is prediction, we usually evaluate the model on observations that were not used during training. These may come from a validation set, a test set, or cross-validation.

The most common regression metrics are based on the difference between observed and predicted values.

Mean Squared Error (MSE) is the average squared prediction error:

$$MSE = \frac{1}{N} \sum_{i=1}^N (y_i - \hat{y}_i)^2$$

Because errors are squared, large prediction errors receive a stronger penalty. However, the MSE is expressed in squared units, which can make it harder to interpret.

Root Mean Squared Error (RMSE) is the square root of the MSE:

$$RMSE = \sqrt{\frac{1}{N} \sum_{i=1}^N (y_i - \hat{y}_i)^2}$$

RMSE is in the same units as the outcome, which makes it easier to interpret. For example, if we are predicting age in years, RMSE is also measured in years.

Mean Absolute Error (MAE) is the average absolute prediction error:

$$MAE = \frac{1}{N} \sum_{i=1}^N |y_i - \hat{y}_i|$$

MAE is also in the same units as the outcome. Compared with RMSE, it is less strongly affected by large errors.

Mean Absolute Percentage Error (MAPE) expresses the error as a percentage of the observed value:

$$MAPE = \frac{100}{N} \sum_{i=1}^N \left| \frac{y_i - \hat{y}_i}{y_i} \right|$$

MAPE can be useful when percentage errors are easy to interpret. However, it is problematic when observed values are zero or close to zero.

Which metric should we use?

There is no single best regression metric. The choice depends on the scientific question and on what type of error matters most.

- Use **RMSE** when large errors should be penalized strongly.
- Use **MAE** when you want an error measure that is easy to interpret and less sensitive to very large errors.
- Use **MAPE** only when percentage errors make sense and the outcome is not close to zero.
- Use (R^2) or adjusted (R^2) when you want to describe model fit, but be careful not to confuse good fit with good prediction.

For prediction, the key idea is that the metric should be calculated on data that were not used to train the model.

A model that fits the training data very well may still perform poorly on new data. Therefore, when our goal is prediction, we care most about the error on validation, test or cross-validation data.

40.4 Evaluating classification

40.5 Evaluating classification models

For a classification task, the outcome consists of two or more **classes** rather than a continuous value. For example, we may want to predict whether a tumour is **benign or malignant**, or whether a patient has **disease or no disease**.

To evaluate a classifier, we compare the **true class labels** with the classes predicted by the model.

Many binary classification models first produce a **score or probability** for belonging to the positive class. A classification threshold is then used to convert this value into a predicted class. For example, using a threshold of 0.5:

- predicted probability $\geq 0.5 \rightarrow$ predict positive
- predicted probability $< 0.5 \rightarrow$ predict negative

The choice of threshold affects the numbers of correct and incorrect classifications and therefore affects many classification performance metrics.

As for regression, predictive performance should ideally be evaluated using observations that were **not used to train the model**, for example using validation data, test data or cross-validation.

Accuracy and misclassification rate

The simplest way to evaluate a classifier is to calculate the proportion of observations that were classified correctly.

For N observations,

$$Accuracy = \frac{\text{number of correct predictions}}{N}.$$

For a binary classifier, this can be written as

$$Accuracy = \frac{TP + TN}{TP + TN + FP + FN}.$$

The **misclassification rate** is the proportion of incorrect predictions:

$$Misclassification\ rate = 1 - Accuracy.$$

Accuracy is easy to interpret, but it can be misleading when the classes are **imbalanced**.

For example, suppose only 5% of individuals have a disease. A classifier that predicts *no disease* for every individual would have an accuracy of 95%, despite being unable to identify any diseased individuals.

We therefore often need to examine performance separately for the positive and negative classes.

Confusion matrix

A **confusion matrix** compares the true class labels with the predicted class labels.

For a binary classification task:

	Predicted positive	Predicted negative
Actual positive	True positive (TP)	False negative (FN)
Actual negative	False positive (FP)	True negative (TN)

The four possible outcomes are:

- **True positive (TP):** a positive observation is correctly classified as positive.
- **True negative (TN):** a negative observation is correctly classified as negative.

- **False positive (FP):** a negative observation is incorrectly classified as positive.
- **False negative (FN):** a positive observation is incorrectly classified as negative.

Several commonly used classification metrics can be calculated from these four quantities.

Sensitivity

Sensitivity measures how well the classifier identifies observations that truly belong to the positive class:

$$\text{Sensitivity} = \frac{TP}{TP + FN}.$$

It answers the question:

Of all observations that are truly positive, what proportion did the model identify correctly?

Sensitivity is also called the **true positive rate (TPR)** or **recall**.

A classifier with high sensitivity has relatively few **false negatives**.

For example, in disease detection, high sensitivity means that most individuals who truly have the disease are detected by the classifier.

Specificity

Specificity measures how well the classifier identifies observations that truly belong to the negative class:

$$\text{Specificity} = \frac{TN}{TN + FP}.$$

It answers the question:

Of all observations that are truly negative, what proportion did the model identify correctly?

Specificity is also called the **true negative rate (TNR)**.

A classifier with high specificity has relatively few **false positives**.

For example, in disease detection, high specificity means that most healthy individuals are correctly identified as healthy.

Precision

Precision measures how many of the observations predicted to be positive are actually positive:

$$\text{Precision} = \frac{TP}{TP + FP}.$$

It answers the question:

Of all observations predicted to be positive, what proportion are truly positive?

Precision is also known as the **positive predictive value (PPV)**.

A classifier with high precision makes relatively few **false-positive predictions**.

Sensitivity and precision therefore answer different questions:

- **Sensitivity:** among the actual positives, how many did we find?
- **Precision:** among the predicted positives, how many were actually positive?

Classification thresholds

Sensitivity, specificity, precision and accuracy usually depend on the **classification threshold**.

For example, lowering the threshold for predicting the positive class will generally classify more observations as positive. This tends to:

- increase **sensitivity**, because fewer positive observations are missed;
- decrease **specificity**, because more negative observations may be incorrectly classified as positive.

Increasing the threshold usually has the opposite effect.

There is therefore often a **trade-off between sensitivity and specificity**, and the appropriate threshold depends on the scientific or clinical context.

For example, if missing a disease is particularly costly, we may prefer a lower threshold and prioritize high sensitivity. If false-positive results lead to costly or invasive follow-up procedures, specificity may be particularly important.

ROC curve and ROC AUC

Rather than evaluating the classifier at only one threshold, we can examine its performance across many possible thresholds using the **Receiver Operating Characteristic (ROC) curve**.

The ROC curve plots:

Sensitivity

against

$1 - \textit{Specificity}$

for different classification thresholds.

Thus, the ROC curve shows the trade-off between correctly identifying positive observations and incorrectly classifying negative observations as positive.

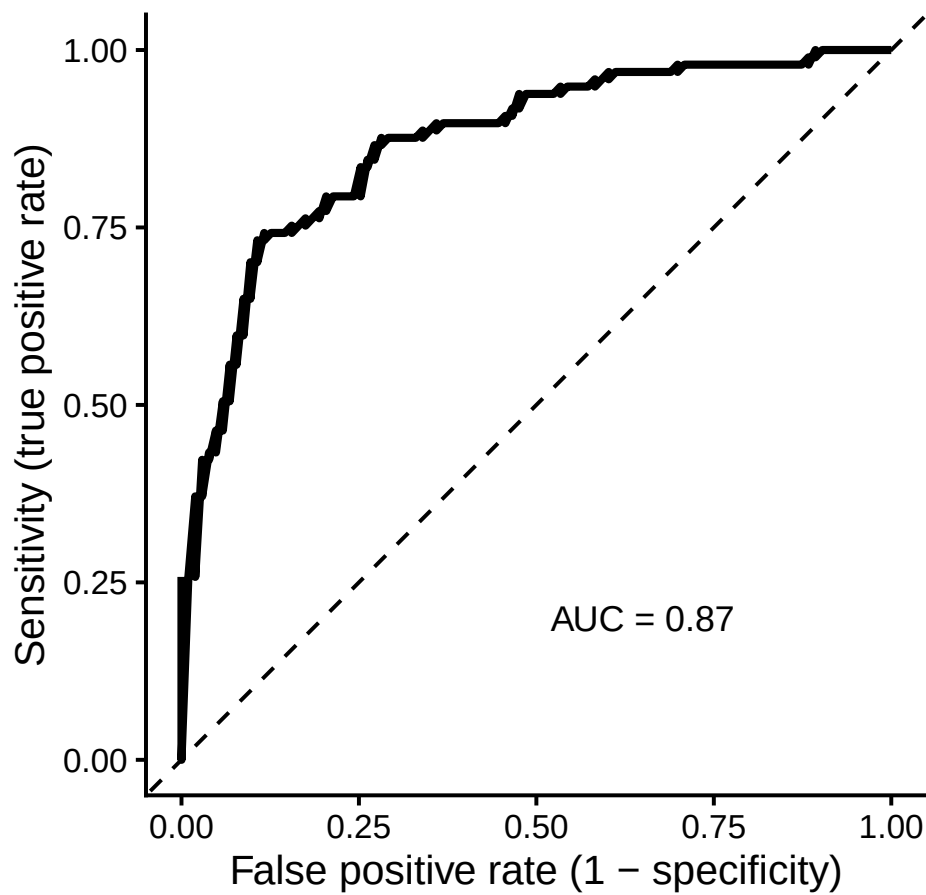


Figure 40.5: Example ROC curve. The curve shows sensitivity versus the false positive rate (1 - specificity) across different classification thresholds.

The **Area Under the ROC Curve (ROC AUC)** summarizes this curve using a single number.

ROC AUC ranges from 0 to 1:

- $AUC = 0.5$ corresponds approximately to random discrimination between the two classes.
- $AUC > 0.5$ indicates some ability to distinguish the classes.
- values closer to 1 indicate better discrimination.
- $AUC = 1$ represents perfect discrimination.

Unlike accuracy, sensitivity or specificity at a particular threshold, ROC AUC summarizes the model's ability to **rank positive observations above negative observations across all possible thresholds**.

Importantly, a high ROC AUC does not automatically mean that a particular classification threshold will be useful in practice. The threshold and the resulting sensitivity, specificity and precision still need to be considered for the application.

Which metric should we use?

There is no single best metric for every classification problem. The appropriate metric depends on the question and on the consequences of different types of errors.

- **Accuracy** gives the overall proportion of correct classifications, but can be misleading when classes are strongly imbalanced.
- **Sensitivity** is important when detecting as many positive observations as possible is the priority.
- **Specificity** is important when avoiding false-positive classifications is particularly important.
- **Precision** is useful when we want positive predictions themselves to be reliable.
- **ROC AUC** summarizes how well the model distinguishes between the classes across different classification thresholds.

As with regression, these metrics should be calculated using **held-out or cross-validation predictions** when the goal is to estimate performance on new data.

Chapter 41

Regularization

Linear models are widely used not only for inference, but also for prediction. In predictive modelling, we may have many potential predictors, and not all of them will necessarily contribute useful information about the outcome. Including irrelevant predictors can increase model complexity and may reduce performance on new data.

One way to address this is through feature selection or shrinkage methods, which aim to reduce the influence of less useful predictors. Here, we will focus on **regularized regression**, which extends the linear models we already know by adding a penalty to the model-fitting procedure. This leads to methods such as **Ridge regression**, **Lasso** and **Elastic Net**.

41.1 Regularized regression

Regularized regression expands on the regression by adding a penalty term(s) to shrink the model coefficients of less important features towards zero. This can help to prevent overfitting and improve the accuracy of the predictive model. Depending on the penalty added, we talk about **Ridge**, **Lasso** or **Elastic Nets** regression.

Previously when talking about regression, we saw that the least squares fitting procedure estimates model coefficients $\beta_0, \beta_1, \dots, \beta_p$ using the values that minimize the residual sum of squares:

$$RSS = \sum_{i=1}^n \left(y_i - \beta_0 - \sum_{j=1}^p \beta_j x_{ij} \right)^2 \quad (41.1)$$

In **regularized regression** the coefficients are estimated by minimizing slightly different quantity. In **Ridge regression** we estimate $\hat{\beta}^{Ridge}$ that minimizes

$$\sum_{i=1}^n \left(y_i - \beta_0 - \sum_{j=1}^p \beta_j x_{ij} \right)^2 + \lambda \sum_{j=1}^p \beta_j^2 = RSS + \lambda \sum_{j=1}^p \beta_j^2 \quad (41.2)$$

where:

$\lambda \geq 0$ is a **tuning parameter** to be determined separately e.g. via cross-validation

Equation 41.2 trades two different criteria:

- as with least squares, Ridge regression seeks coefficient estimates that fit the data well, by making RSS small
- however, the second term $\lambda \sum_{j=1}^p \beta_j^2$, called **shrinkage penalty** is small when $\beta_1, \dots, \beta_p$ are close to zero, so it has the effect of **shrinking** the estimates of β_j towards zero.
- the tuning parameter λ controls the relative impact of these two terms on the regression coefficient estimates
 - when $\lambda = 0$, the penalty term has no effect
 - as $\lambda \rightarrow \infty$ the impact of the shrinkage penalty grows and the ridge regression coefficient estimates approach zero

```
library(glmnet)
library(latex2exp)

# select subset data
# and scale: since regression puts constraints on the size of the
↪ coefficient
data_input <- data_diabetes %>%
  dplyr::select(BMI, chol, hdl, age, stab.glu) %>%
  na.omit()

# fit ridge regression for a series of lambda values
# note: lambda values were chosen by experimenting to show lambda
↪ effect on beta coefficient estimates
x <- model.matrix(BMI ~., data = data_input)
y <- data_input %>% pull(BMI)
model <- glmnet(x, y, alpha=0, lambda = seq(0, 100, 1))

# plot beta estimates vs. lambda
betas <- model$beta %>% as.matrix() %>% t()
data_plot <- tibble(data.frame(lambda = model$lambda, betas)) %>%
  dplyr::select(-"X.Intercept.") %>%
  pivot_longer(-lambda, names_to = "variable", values_to =
↪ "beta")

data_plot %>%
  ggplot(aes(x = lambda, y = beta, color = variable)) +
  geom_line(linewidth = 2, alpha = 0.7) +
  theme_classic() +
  xlab(TeX("$\\lambda$")) +
  ylab(TeX("Standardized coefficients")) +
  scale_color_brewer(palette = "Set1") +
  theme(legend.title = element_blank(), legend.position = "top",
↪ legend.text = element_text(size=12)) +
```

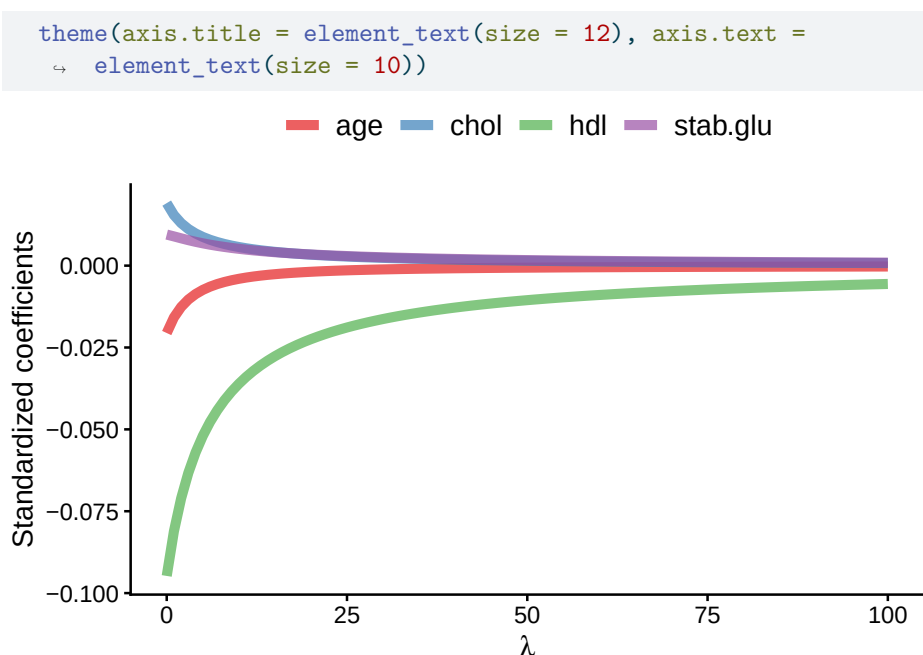



Figure 41.1: Example of Ridge regression to model BMI using age, chol, hdl and glucose variables: model coefficients are plotted over a range of lambda values, showing how initially for small lambda values all variables are part of the model and how they gradually shrink towards zero for larger lambda values.

41.2 Bias-variance trade-off

Ridge regression's advantages over least squares estimates stems from **bias-variance trade-off**, another fundamental concept in machine learning.

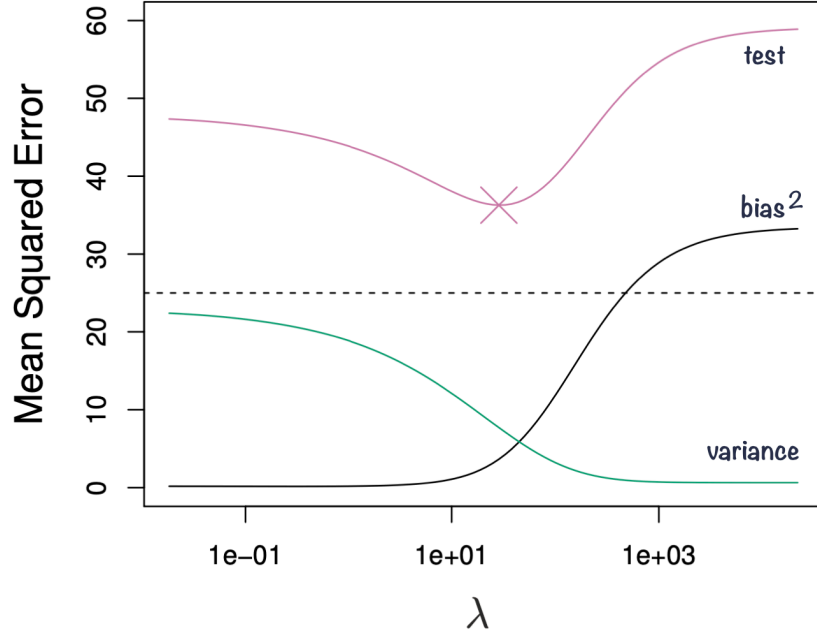
- The bias-variance trade-off describes the relationship between model complexity, prediction accuracy, and the ability of the model to generalize to new data.
- **Bias** refers to the error that is introduced by approximating a real-life problem with a simplified model. A high bias model is one that makes overly simplistic assumptions about the underlying data, resulting in *under-fitting* and poor accuracy.
- **Variance** refers to the sensitivity of a model to fluctuations in the training data. A high variance model is one that is overly complex and captures noise in the training data, resulting in *overfitting* and poor generalization to new data.
- The goal of machine learning is to find a model with **the right balance between bias and variance**, which can generalize well to new data.
- The bias-variance trade-off can be visualized in terms of MSE, means squared error of the model. The **MSE** can be decomposed into:

$$MSE(\hat{\beta}) := bias^2(\hat{\beta}) + Var(\hat{\beta}) + noise$$

- The irreducible error is the inherent noise in the data that cannot be reduced by any model, while the bias and variance terms can be reduced by choosing an appropriate model complexity. The trade-off lies in finding the right balance between bias and variance that minimizes the total MSE.
- In practice, this trade-off can be addressed by **regularizing the model**, selecting an appropriate model complexity, or by using ensemble methods

that combine multiple models to reduce the variance (e.g. Random Forest). Ultimately, the goal is to find a model that is both accurate and generalizing.

Figure 41.2: Squared bias, variance and test mean squared error for ridge regression predictions on a simulated data as a function of lambda demonstrating bias-variance trade-off. Based on Gareth James et. al, A Introduction to statistical learning



41.3 Ridge, Lasso and Elastic Nets

In **Ridge** regression we minimize:

$$\sum_{i=1}^n \left(y_i - \beta_0 - \sum_{j=1}^p \beta_j x_{ij} \right)^2 + \lambda \sum_{j=1}^p \beta_j^2 = RSS + \lambda \sum_{j=1}^p \beta_j^2 \quad (41.3)$$

where $\lambda \sum_{j=1}^p \beta_j^2$ is also known as **L2** regularization element or l_2 penalty

In **Lasso** regression, that is Least Absolute Shrinkage and Selection Operator regression we change penalty term to absolute value of the regression coefficients:

$$\sum_{i=1}^n \left(y_i - \beta_0 - \sum_{j=1}^p \beta_j x_{ij} \right)^2 + \lambda \sum_{j=1}^p |\beta_j| = RSS + \lambda \sum_{j=1}^p |\beta_j| \quad (41.4)$$

where $\lambda \sum_{j=1}^p |\beta_j|$ is also known as **L1** regularization element or l_1 penalty

Lasso regression was introduced to help model interpretation. With Ridge regression we improve model performance but unless $\lambda = \infty$ all beta coefficients are non-zero, hence all variables remain in the model. By using l_1 penalty we can force some of the coefficients estimates to be exactly equal to 0, hence perform **variable selection**

```

library(glmnet)
library(latex2exp)

# select subset data
# and scale: since regression puts constraints on the size of the
↪ coefficient
data_input <- data_diabetes %>%
  dplyr::select(BMI, chol, hdl, age, stab.glu) %>%
  na.omit()

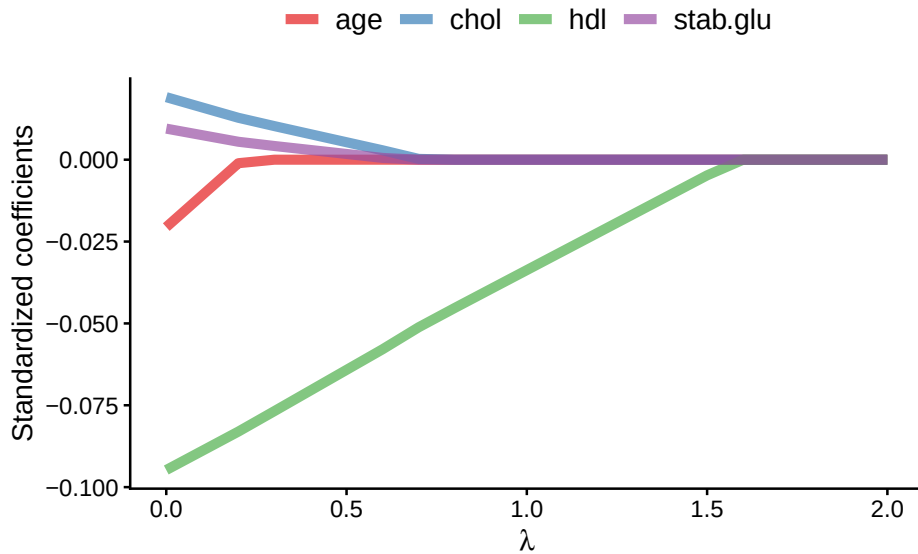
# fit ridge regression for a series of lambda values
# note: lambda values were chosen by experimenting to show lambda
↪ effect on beta coefficient estimates
x <- model.matrix(BMI ~ ., data = data_input)
y <- data_input %>% pull(BMI)
model <- glmnet(x, y, alpha=1, lambda = seq(0, 2, 0.1))

# plot beta estimates vs. lambda
betas <- model$beta %>% as.matrix() %>% t()
data_plot <- tibble(data.frame(lambda = model$lambda, betas)) %>%
  dplyr::select(-"X.Intercept.") %>%
  pivot_longer(-lambda, names_to = "variable", values_to =
    ↪ "beta")

data_plot %>%
  ggplot(aes(x = lambda, y = beta, color = variable)) +
  geom_line(linewidth = 2, alpha = 0.7) +
  theme_classic() +
  xlab(TeX("$\\lambda$")) +
  ylab(TeX("Standardized coefficients")) +
  scale_color_brewer(palette = "Set1") +
  theme(legend.title = element_blank(), legend.position = "top",
    ↪ legend.text = element_text(size=12)) +
  theme(axis.title = element_text(size = 12), axis.text =
    ↪ element_text(size = 10))

```

Figure 41.3: Example of Lasso regression to model BMI using age, chol, hdl and glucose variables: model coefficients are plotted over a range of lambda values, showing how initially for small lambda values all variables are part of the model and how they gradually shrink towards zero and are also set to zero for larger lambda values.



Elastic Net use both L1 and L2 penalties to try to find middle grounds by performing parameter shrinkage and variable selection.

$$\sum_{i=1}^n \left(y_i - \beta_0 - \sum_{j=1}^p \beta_j x_{ij} \right)^2 + \lambda \sum_{j=1}^p |\beta_j| + \lambda \sum_{j=1}^p \beta_j^2 = RSS + \lambda \sum_{j=1}^p |\beta_j| + \lambda \sum_{j=1}^p \beta_j^2 \quad (41.5)$$

In the `glmnet` library we can fit Elastic Net by setting parameters α . Actually, under the hood `glmnet` minimizes a cost function:

$$\sum_{i=1}^n (y_i - \hat{y}_i)^2 + \lambda \left((1 - \alpha) \sum_{j=1}^p \beta_j^2 + \alpha \sum_{j=1}^p |\beta_j| \right)$$

where:

- n is the number of samples
- p is the number of parameters
- λ , α hyperparameters control the shrinkage

When $\alpha = 0$ this corresponds to Ridge regression and when $\alpha = 1$ this corresponds to Lasso regression. A value of $0 < \alpha < 1$ gives us **Elastic Net regularization**, combining both L1 and L2 regularization terms.

```
library(glmnet)
library(latex2exp)

# select subset data
# and scale: since regression puts constraints on the size of the
# coefficient
data_input <- data_diabetes %>%
  dplyr::select(BMI, chol, hdl, age, stab.glu) %>%
  na.omit()
```

```

# fit ridge regression for a series of lambda values
# note: lambda values were chosen by experimenting to show lambda
# effect on beta coefficient estimates
x <- model.matrix(BMI ~., data = data_input)
y <- data_input %>% pull(BMI)
model <- glmnet(x, y, alpha=0.1, lambda = seq(0, 3, 0.05))

# plot beta estimates vs. lambda
betas <- model$beta %>% as.matrix() %>% t()
data_plot <- tibble(data.frame(lambda = model$lambda, betas)) %>%
  dplyr::select(-"X.Intercept.") %>%
  pivot_longer(-lambda, names_to = "variable", values_to =
    "beta")

data_plot %>%
  ggplot(aes(x = lambda, y = beta, color = variable)) +
  geom_line(linewidth = 2, alpha = 0.7) +
  theme_classic() +
  xlab(TeX("$\\lambda$")) +
  ylab(TeX("Standardized coefficients")) +
  scale_color_brewer(palette = "Set1") +
  theme(legend.title = element_blank(), legend.position = "top",
    legend.text = element_text(size=12)) +
  theme(axis.title = element_text(size = 12), axis.text =
    element_text(size = 10))

```

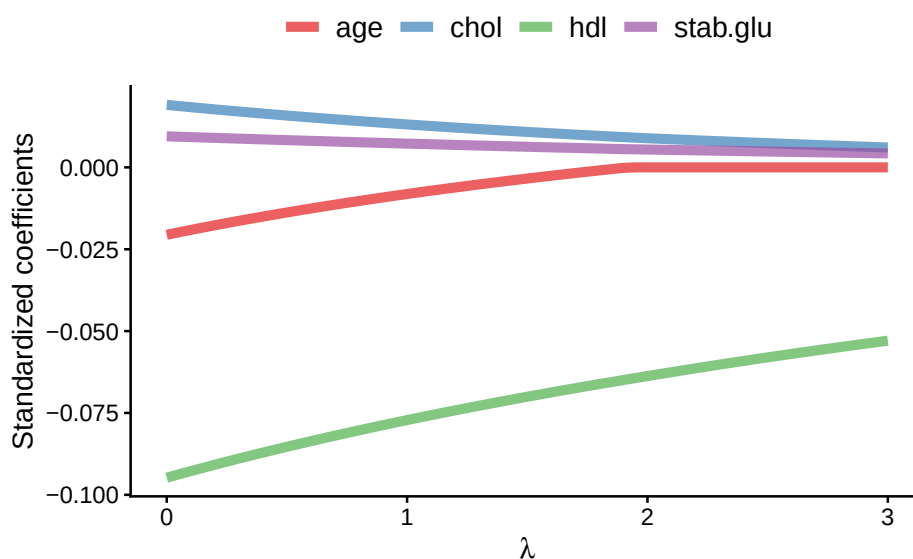


Figure 41.4: Example of Elastic Net regression to model BMI using age, chol, hdl and glucose variables: model coefficients are plotted over a range of lambda values and alpha value 0.1, showing the changes of model coefficients as a function of lambda being somewhere between those for Ridge and Lasso regression.

41.4 Logistic Lasso

- Logistic Lasso combines logistic regression with Lasso regularization to analyze binary outcome data while simultaneously performing variable selection and regularization.
- The equation for Logistic Lasso combines logistic regression with Lasso regularization. We estimate set of coefficients $\hat{\beta}$ that minimize the combined logistic loss function and the Lasso penalty:

$$\left(\sum_{i=1}^n [y_i \log(p_i) + (1 - y_i) \log(1 - p_i)] \right) + \lambda \sum_{j=1}^p |\beta_j|$$

where:

- y_i represents the binary outcome (0 or 1) for the (i)-th observation.
- p_i is the predicted probability of $y_i = 1$ given by the logistic model $p_i = \frac{1}{1 + e^{-(\beta_0 + \beta_1 x_{i1} + \dots + \beta_p x_{ip})}}$.
- $\beta_0, \beta_1, \dots, \beta_p$ are the coefficients of the model, including the intercept β_0 .
- $x_{i1}, \dots, x_{ip}$ are the predictor variables for the i -th observation.
- λ is the regularization parameter that controls the strength of the Lasso penalty $\lambda \sum_{j=1}^p |\beta_j|$, which encourages sparsity in the coefficients β_j by shrinking some of them to zero.
- n is the number of observations, and p is the number of predictors.

Exercises (Lasso)

Data for exercises are on Canvas under Files -> data_exercises -> linear-models

Exercise 41.1 (Lasso). Follow and try to run the code below for fitting Lasso model to predict BMI given all the numerical variables in the **diabetes** data set.

Exercise 41.2 (MAE).

- Add a code to calculate MAE, Mean Absolute Error to get a metric of model performance.
- Adjust the code, to get the best Lasso model again, but now exclude **weight** from the predictors. What value of MEA are you getting for this model, and what does it mean?

Exercise 41.3 (Logistic Lasso). Adjust the original code to model probability of becoming obese. You can define obesity as BMI > 30. Evaluate the final model using overall accuracy.

Load data & reformat data

```
# load libraries
library(tidyverse)
library(glmnet)
library(caret)
library(splitTools)

# import raw data
input_diabetes <- read_csv("data/data-diabetes.csv")

# preview data
glimpse(input_diabetes)
```

Rows: 403

Columns: 19

```
$ id      <dbl> 1000, 1001, 1002, 1003, 1005, 1008, 1011, 1015, 1016, 1022, 1~
$ chol    <dbl> 203, 165, 228, 78, 249, 248, 195, 227, 177, 263, 242, 215, 23~
$ stab.glu <dbl> 82, 97, 92, 93, 90, 94, 92, 75, 87, 89, 82, 128, 75, 79, 76, ~
$ hdl     <dbl> 56, 24, 37, 12, 28, 69, 41, 44, 49, 40, 54, 34, 36, 46, 30, 4~
```

```

$ ratio      <dbl> 3.6, 6.9, 6.2, 6.5, 8.9, 3.6, 4.8, 5.2, 3.6, 6.6, 4.5, 6.3, 6~
$ glyhb      <dbl> 4.31, 4.44, 4.64, 4.63, 7.72, 4.81, 4.84, 3.94, 4.84, 5.78, 4~
$ location    <chr> "Buckingham", "Buckingham", "Buckingham", "Buckingham", "Buck~
$ age        <dbl> 46, 29, 58, 67, 64, 34, 30, 37, 45, 55, 60, 38, 27, 40, 36, 3~
$ gender      <chr> "female", "female", "female", "male", "male", "male", "male", ~
$ height      <dbl> 62, 64, 61, 67, 68, 71, 69, 59, 69, 63, 65, 58, 60, 59, 69, 6~
$ weight      <dbl> 121, 218, 256, 119, 183, 190, 191, 170, 166, 202, 156, 195, 1~
$ frame       <chr> "medium", "large", "large", "large", "medium", "large", "medi~
$ bp.1s       <dbl> 118, 112, 190, 110, 138, 132, 161, NA, 160, 108, 130, 102, 13~
$ bp.1d       <dbl> 59, 68, 92, 50, 80, 86, 112, NA, 80, 72, 90, 68, 80, NA, 66, ~
$ bp.2s       <dbl> NA, NA, 185, NA, NA, NA, 161, NA, 128, NA, 130, NA, NA, NA, N~
$ bp.2d       <dbl> NA, NA, 92, NA, NA, NA, 112, NA, 86, NA, 90, NA, NA, NA, NA, ~
$ waist       <dbl> 29, 46, 49, 33, 44, 36, 46, 34, 34, 45, 39, 42, 35, 37, 36, 3~
$ hip         <dbl> 38, 48, 57, 38, 41, 42, 49, 39, 40, 50, 45, 50, 41, 43, 40, 4~
$ time.ppn    <dbl> 720, 360, 180, 480, 300, 195, 720, 1020, 300, 240, 300, 90, 7~

```

```

# run basic feature engineering of the data
# exclude bp.2s, bp.2d due to large number of missing data
# create BMI based on weight and height
# keep only numerical variables
# keep samples for which none missing data is present (complete
  ↳ case analysis)

# define numerical predictors
pred_numeric <- c("chol", "stab.glu", "hdl", "ratio", "glyhb",
  ↳ "age", "weight", "bp.1s", "bp.1d", "waist", "hip",
  ↳ "time.ppn")

# and only numerical predictors
conv_factor <- 703 # conversion factor to calculate BMI from
  ↳ inches and pounds BMI = weight (lb) / [height (in)]^2 x 703
data_diabetes <- input_diabetes %>%
  mutate(BMI = weight / height^2 * 703, BMI = round(BMI, 2)) %>%
  relocate(BMI, .after = id) %>%
  dplyr::select(-bp.2s, -bp.2d) %>%
  select(all_of(c("BMI", pred_numeric))) %>%
  na.omit()

# preview reformatted data
glimpse(data_diabetes)

```

Rows: 375

Columns: 13

```

$ BMI      <dbl> 22.13, 37.42, 48.37, 18.64, 27.82, 26.50, 28.20, 24.51, 35.78~
$ chol     <dbl> 203, 165, 228, 78, 249, 248, 195, 177, 263, 242, 215, 238, 19~
$ stab.glu <dbl> 82, 97, 92, 93, 90, 94, 92, 87, 89, 82, 128, 75, 76, 83, 78, ~
$ hdl      <dbl> 56, 24, 37, 12, 28, 69, 41, 49, 40, 54, 34, 36, 30, 47, 38, 6~
$ ratio    <dbl> 3.6, 6.9, 6.2, 6.5, 8.9, 3.6, 4.8, 3.6, 6.6, 4.5, 6.3, 6.6, 6~
$ glyhb    <dbl> 4.31, 4.44, 4.64, 4.63, 7.72, 4.81, 4.84, 4.84, 5.78, 4.77, 4~
$ age      <dbl> 46, 29, 58, 67, 64, 34, 30, 45, 55, 60, 38, 27, 36, 33, 50, 2~

```



```
$ weight <dbl> 121, 218, 256, 119, 183, 190, 191, 166, 202, 156, 195, 170, 1~
$ bp.1s <dbl> 118, 112, 190, 110, 138, 132, 161, 160, 108, 130, 102, 130, 1~
$ bp.1d <dbl> 59, 68, 92, 50, 80, 86, 112, 80, 72, 90, 68, 80, 66, 90, 100,~
$ waist <dbl> 29, 46, 49, 33, 44, 36, 46, 34, 45, 39, 42, 35, 36, 37, 37, 3~
$ hip <dbl> 38, 48, 57, 38, 41, 42, 49, 40, 50, 45, 50, 41, 40, 41, 43, 3~
$ time.ppn <dbl> 720, 360, 180, 480, 300, 195, 720, 300, 240, 300, 90, 720, 22~
```

split data and scale

```
# split data into train (70%) and test (30%)
set.seed(123)
inds <- partition(data_diabetes$BMI, p = c(train = 0.7, test =
  ↪ 0.3))

data_train <- data_diabetes %>%
  slice(inds$train)

data_test <- data_diabetes %>%
  slice(inds$test)

# standardize variables
# Note: the correct approach is to scale the non-test data
  ↪ (train)
# and then scale the test data using the parameters of the train
  ↪ data (e.g. mean or sd)
# Here we will use preProcess() function from caret package to do
  ↪ that
# We do not scale response (BMI)

scaling <- preProcess(data_train[, pred_numeric], method =
  ↪ c("center", "scale"))
data_train[, pred_numeric] <- predict(scaling, data_train[,
  ↪ pred_numeric])
data_test[, pred_numeric] <- predict(scaling, data_test[,
  ↪ pred_numeric])

# preview scaled data
glimpse(data_train)
```

Rows: 258

Columns: 13

```
$ BMI <dbl> 48.37, 27.82, 26.50, 28.20, 40.75, 33.20, 27.02, 26.12, 24.90~
$ chol <dbl> 0.4524462, 0.9164540, 0.8943584, -0.2767090, 0.1652032, 0.673~
$ stab.glu <dbl> -0.28652567, -0.32418417, -0.24886718, -0.28652567, 0.3913272~
$ hdl <dbl> -0.749786859, -1.261997828, 1.071407700, -0.522137539, -0.920~
$ ratio <dbl> 0.89155394, 2.35907876, -0.52161816, 0.13061530, 0.94590692, ~
$ glyhb <dbl> -0.42464460, 0.91116765, -0.35091467, -0.33790342, -0.2815218~
$ age <dbl> 0.70202683, 1.05915385, -0.72648126, -0.96456594, -0.48839658~
```

```
$ weight <dbl> 1.8615300087, 0.1191556829, 0.2862326730, 0.3101008145, 0.405~
$ bp.1s <dbl> 2.23602108, 0.03429933, -0.21974548, 1.00813780, -1.48996957,~
$ bp.1d <dbl> 0.62214432, -0.24471383, 0.18871524, 2.06690792, -1.11157199,~
$ waist <dbl> 1.8500049, 1.0090936, -0.3363645, 1.3454581, 0.6727291, -0.50~
$ hip <dbl> 2.42867737, -0.36396335, -0.18942330, 1.03235701, 1.20689706,~
$ time.ppn <dbl> -0.5033145, -0.1207312, -0.4554916, 1.2183102, -0.7902519, 1.~

glimpse(data_test)
```

Rows: 117

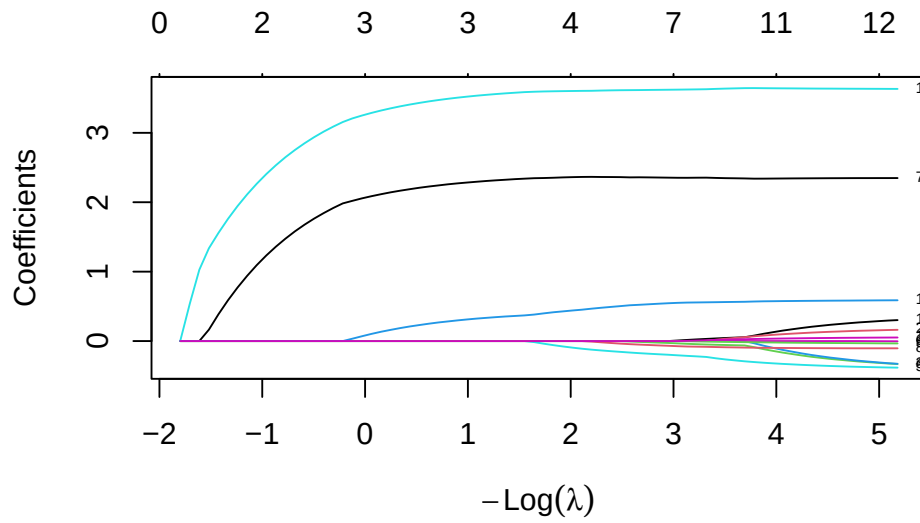
Columns: 13

```
$ BMI <dbl> 22.13, 37.42, 18.64, 24.51, 35.78, 25.96, 30.45, 21.63, 32.61~
$ chol <dbl> -0.09994410, -0.93957728, -2.86189536, -0.67442996, 1.2257925~
$ stab.glu <dbl> -0.47481814, -0.19237944, -0.26769642, -0.38067191, -0.343013~
$ hdl <dbl> 0.331547411, -1.489647148, -2.172595108, -0.066838899, -0.579~
$ ratio <dbl> -0.52161816, 1.27202352, 1.05461237, -0.52161816, 1.10896509,~
$ glyhb <dbl> -0.56776732, -0.51138558, -0.42898155, -0.33790342, 0.0697795~
$ age <dbl> -0.01222722, -1.02408711, 1.23771736, -0.07174839, 0.52346332~
$ weight <dbl> -1.3606691, 0.9545406, -1.4084054, -0.2866027, 0.5726504, -0.~
$ bp.1s <dbl> -0.81251672, -1.06656154, -1.15124315, 0.96579699, -1.2359247~
$ bp.1d <dbl> -1.7617156, -1.1115720, -2.4118592, -0.2447138, -0.8226193, 0~
$ waist <dbl> -1.5136404, 1.3454581, -0.8409113, -0.6727291, 1.1772759, 0.1~
$ hip <dbl> -0.88758348, 0.85781697, -0.88758348, -0.53850339, 1.20689706~
$ time.ppn <dbl> 1.21831021, 0.07056041, 0.45314368, -0.12073122, -0.31202285,~
```

```
# define x (predictors) and y (response)
x <- model.matrix(BMI ~.-1, data = data_train)
y <- data_train$BMI

# we use glmnet function to fit Lasso, alpha = 1 for Lasso, alpha
↪ = 0 for Ridge
fit <- glmnet(x, y, alpha=1)

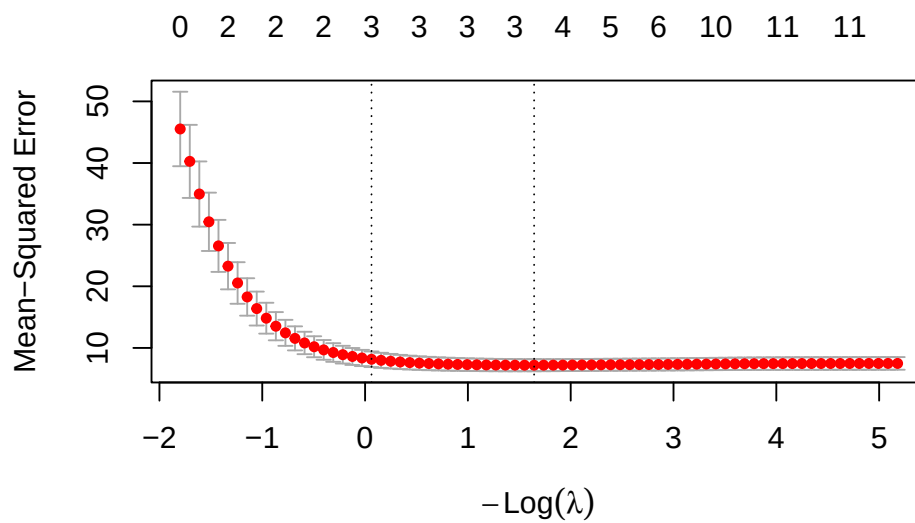
# by default this returns many Lasso models, fitted across a
↪ range of lambda values
# and we can plot a whole path of models, to see how coefficients
↪ change as a function of lambda
plot(fit, xvar="lambda", label=TRUE)
```



train Lasso model

```
# To train Lasso model we can use k-fold cross validation
# We can use cv.glmnet() function for that
# Alternatively, we could try writing our code from scratch doing
  ↪ more data splits...
cv <- cv.glmnet(x, y, alpha = 1)

# plot MSE as a function of the lambda
plot(cv)
```



create final model

```
# `lambda.min` gives the lowest cross-validation error.
# `lambda.1se` gives a more strongly regularized, simpler model
# whose error is still within one standard error of the minimum.
# lambda.1se is often preferred when a simpler, more regularized
# model is desired with little loss in predictive performance.
```

```
cv$lambda.1se
```

```
[1] 0.9393291
```

```
# fit the final model with the selected lambda value
fit_final <- glmnet(x, y, alpha = 1, lambda = cv$lambda.1se)

# check model coefficients. "." indicates 0
coef(fit_final)
```

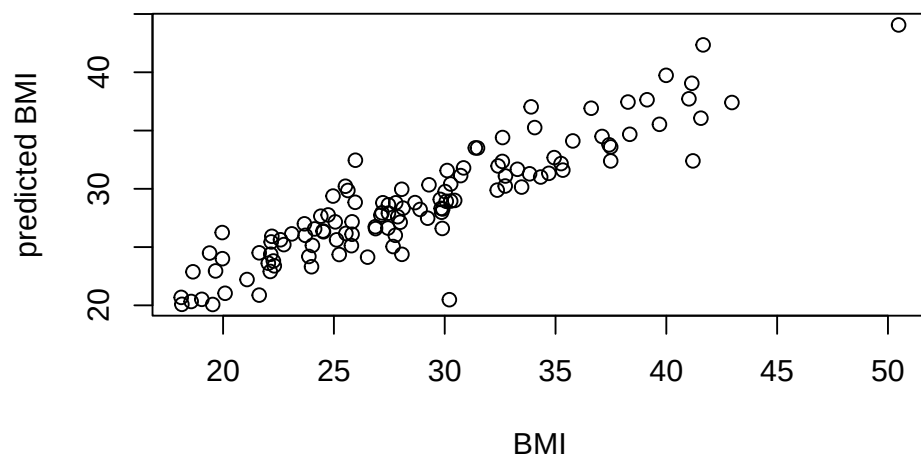
```
13 x 1 sparse Matrix of class "dgCMatrix"
              s0
(Intercept) 28.8212016
chol         .
stab.glu     .
hdl          .
ratio        .
glyhb        .
age          .
weight       2.0834135
bp.1s        .
bp.1d        .
waist        0.1062243
hip          3.2879569
time.ppn     .
```

We can see that the largest coefficients (in terms of absolute values) are for **weight** and **height**, which of course it makes sense as we are predicting BMI that was defined based on weight and height in the first place. This is naturally for demonstration purposes only.

predict BMI

```
# Let's see how the model performs on new unseen test data
x_test <- model.matrix(BMI ~.-1, data_test)
y_pred <- fit_final %>% predict(x_test) %>% as.vector()

plot(data_test$BMI, y_pred, xlab = "BMI", ylab = "predicted BMI")
```



```
cor(data_test$BMI, y_pred)
```

```
[1] 0.9095373
```


References

- Blanco-Mallo, Eva, Laura Morán-Fernández, Beatriz Remeseiro, and Verónica Bolón-Canedo. 2023. “Do All Roads Lead to Rome? Studying Distance Measures in the Context of Machine Learning.” *Pattern Recognition* 141. <https://doi.org/10.1016/j.patcog.2023.109646>.
- Fine, Jason P., and Robert J. Gray. 1999. “A Proportional Hazards Model for the Subdistribution of a Competing Risk.” *Journal of the American Statistical Association* 94. <https://doi.org/10.1080/01621459.1999.10474144>.
- Forgy, E. 1965. “Cluster Analysis of Multivariate Data: Efficiency Vs. Interpretability of Classifications.” *Biometrics* 21.
- Gao, Caroline X., Dominic Dwyer, Ye Zhu, Catherine L. Smith, Lan Du, Kate M. Filia, Johanna Bayer, et al. 2023. “An Overview of Clustering Methods with Guidelines for Application in Mental Health Research.” *Psychiatry Research*. <https://doi.org/10.1016/j.psychres.2023.115265>.
- Ghaderyan, Peyvand, and Seyede Marziyeh Ghoreschi Beyrami. 2020. “Neurodegenerative Diseases Detection Using Distance Metrics and Sparse Coding: A New Perspective on Gait Symmetric Features.” *Computers in Biology and Medicine* 120. <https://doi.org/10.1016/j.compbiomed.2020.103736>.
- Gray, Robert J. 1998. “A Class of k-Sample Tests for Comparing the Cumulative Incidence of a Competing Risk.” *The Annals of Statistics* 16. <https://doi.org/10.1214/aos/1176350951>.
- Hartigan, J. A., and M. A. Wong. 1979. “Algorithm AS 136: A k-Means Clustering Algorithm.” *Applied Statistics* 28. <https://doi.org/10.2307/2346830>.
- Lloyd, Stuart P. 1982. “Least Squares Quantization in PCM.” *IEEE Transactions on Information Theory* 28. <https://doi.org/10.1109/TIT.1982.1056489>.
- Preud’homme, Gregoire, Kevin Duarte, Kevin Dalleau, Claire Lacomblez, Emmanuel Bresso, Malika Smail-Tabbone, Miguel Couceiro, et al. 2021. “Head-to-Head Comparison of Clustering Methods for Heterogeneous Data: A Simulation-Driven Benchmark.” *Scientific Reports* 11. <https://doi.org/10.1038/s41598-021-83340-8>.
- Scrucca, L, A Santucci, and F Aversa. 2010. “Regression Modeling of Competing Risk Using r: An in Depth Guide for Clinicians.” *Bone Marrow Transplantation* 45: 1388–95. <https://doi.org/10.1038/bmt.2009.359>.
- Suzuki, Ryota, and Hidetoshi Shimodaira. 2006. “Pvclust: An r Package for Assessing the Uncertainty in Hierarchical Clustering.” *Bioinformatics* 22. <https://doi.org/10.1093/bioinformatics/btl117>.

- Traag, V. A., L. Waltman, and N. J. van Eck. 2019. "From Louvain to Leiden: Guaranteeing Well-Connected Communities." *Scientific Reports* 9. <https://doi.org/10.1038/s41598-019-41695-z>.